Huazhong Liu
Shadi Ibrahim
Thomas Rauber (Eds.)

LNCS 16383

Algorithms and Architectures for Parallel Processing

25th International Conference, ICA3PP 2025
Zhengzhou, China, October 30–November 2, 2025
Proceedings, Part III

3 Part III

Springer

Lecture Notes in Computer Science **16383**

Founding Editors

Gerhard Goos
Juris Hartmanis

Editorial Board Members

Elisa Bertino, *Purdue University, West Lafayette, IN, USA*
Wen Gao, *Peking University, Beijing, China*
Bernhard Steffen, *TU Dortmund University, Dortmund, Germany*
Moti Yung, *Columbia University, New York, NY, USA*

The series Lecture Notes in Computer Science (LNCS), including its subseries Lecture Notes in Artificial Intelligence (LNAI) and Lecture Notes in Bioinformatics (LNBI), has established itself as a medium for the publication of new developments in computer science and information technology research, teaching, and education.

LNCS enjoys close cooperation with the computer science R & D community, the series counts many renowned academics among its volume editors and paper authors, and collaborates with prestigious societies. Its mission is to serve this international community by providing an invaluable service, mainly focused on the publication of conference and workshop proceedings and postproceedings. LNCS commenced publication in 1973.

Huazhong Liu · Shadi Ibrahim · Thomas Rauber
Editors

Algorithms and Architectures for Parallel Processing

25th International Conference, ICA3PP 2025
Zhengzhou, China, October 30 – November 2, 2025
Proceedings, Part III

Springer

Editors
Huazhong Liu
Hainan University
Haikou, China

Shadi Ibrahim
Inria
Rennes, France

Thomas Rauber
University of Bayreuth
Bayreuth, Germany

ISSN 0302-9743 ISSN 1611-3349 (electronic)
Lecture Notes in Computer Science
ISBN 978-981-95-8404-8 ISBN 978-981-95-8405-5 (eBook)
https://doi.org/10.1007/978-981-95-8405-5

© The Editor(s) (if applicable) and The Author(s), under exclusive license
to Springer Nature Singapore Pte Ltd. 2026

This work is subject to copyright. All rights are solely and exclusively licensed by the Publisher, whether the whole or part of the material is concerned, specifically the rights of translation, reprinting, reuse of illustrations, recitation, broadcasting, reproduction on microfilms or in any other physical way, and transmission or information storage and retrieval, electronic adaptation, computer software, or by similar or dissimilar methodology now known or hereafter developed.
The use of general descriptive names, registered names, trademarks, service marks, etc. in this publication does not imply, even in the absence of a specific statement, that such names are exempt from the relevant protective laws and regulations and therefore free for general use.
The publisher, the authors and the editors are safe to assume that the advice and information in this book are believed to be true and accurate at the date of publication. Neither the publisher nor the authors or the editors give a warranty, expressed or implied, with respect to the material contained herein or for any errors or omissions that may have been made. The publisher remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

This Springer imprint is published by the registered company Springer Nature Singapore Pte Ltd.
The registered company address is: 152 Beach Road, #21-01/04 Gateway East, Singapore 189721, Singapore

If disposing of this product, please recycle the paper.

Preface

On behalf of the Conference Committee, we welcome you to the proceedings of the 2025 International Conference on Algorithms and Architectures for Parallel Processing (ICA3PP 2025), which was held in Zhengzhou, China from October 30 – November 2, 2025. ICA3PP 2025 was the 25th in this series of conferences (started in 1995) that are devoted to algorithms and architectures for parallel processing. ICA3PP is now recognized as the main regular international event that covers the many dimensions of parallel algorithms and architectures, encompassing fundamental theoretical approaches, practical experimental projects, and commercial components and systems. This conference provides a forum for academics and practitioners from countries around the world to exchange ideas for improving the efficiency, performance, reliability, security, and interoperability of computing systems and applications.

A successful conference would not be possible without the high-quality contributions made by the authors. This year, ICA3PP received a total of 543 submissions from authors in various countries and regions. Each paper underwent a single-blind review process by at least three reviewers. Based on rigorous peer reviews by the Program Committee members and reviewers, 158 regular papers were selected to be presented at the conference, giving an acceptance rate of 29%. We also accepted 104 research papers and 48 special session papers. In addition to the contributed papers, distinguished scholars were invited to give keynote lectures, providing us with the recent developments in diverse areas in algorithms and architectures for parallel processing and applications.

We would like to take this opportunity to express our sincere gratitude to the Program Committee members and 305 reviewers for their dedicated and professional service. We highly appreciate the General Chairs, Didier El Baz, Zizhong Chen, and Gudula Runger, for their help and support in organizing the conference. Then we would like to express our appreciation to the four program vice-chairs, Rong Gu, Deze Zeng, Jiawei Huang, and Zhe Peng, for their leadership in managing the various tracks of the conference. We would also like to give our sincere thanks to the Local Chairs, Jihong Ding, Yi Jiang, Song Zhang, Zhicheng Liu, Chunfeng Du, Zecan Yang, and Xin Nie, for their hard work and commitment to the success of the conference. Our heartfelt thanks also go to the Workshop Chair Shaojun Zou for his efforts in organizing the wonderful special sessions and enriching the technical discourse. We are also deeply appreciative of the Publicity Chairs, Jia Hu, Jing Guo, and Wenxuan Zhang, for their tireless efforts in promoting the conference. We are so grateful to the publication chairs, Binbin Zhou and Lan Yan, and the publication assistants for their dedicated support in overseeing the publication process. Thanks also go to Registration Chairs Yuan Tian and Tingting Han, and Web Chairs Jiawei Wang and Ren Li. Our deepest thanks also go to the Steering Chairs, Yang Xiang, Wanlei Zhou, Yi Pan, Laurence T. Yang, and Weijia Jia, for their continuous leadership and support throughout the planning and execution phases of the conference. We must also say "thank you" to all the volunteers who helped us at various stages of this conference. Moreover, we were so honored to have many renowned scholars be part

of this conference. Finally, we would like to thank all speakers, authors, and participants for their great contribution to and support for the success of ICA3PP 2025!

October 2025

Huazhong Liu
Shadi Ibrahim
Thomas Rauber

Organization

General Chairs

Didier El Baz	LAAS-CNRS, France
Zizhong Chen	University of California, Riverside, USA
Gudula Runger	Chemnitz University of Technology, Germany

Program Chairs

Huazhong Liu	Hainan University, China
Shadi Ibrahim	Inria, France
Thomas Rauber	University of Bayreuth, Germany

Program Vice-Chairs

Rong Gu	Nanjing University, China
Deze Zeng	China University of Geosciences, Wuhan, China
Jiawei Huang	Central South University, China
Zhe Peng	Hong Kong Polytechnic University, China

Local Chairs

Jihong Ding	Hainan University, China
Yi Jiang	Hainan University, China
Song Zhang	Zhengzhou University, China
Zhicheng Liu	Zhengzhou University, China
Chunfeng Du	Zhengzhou University, China
Zecan Yang	Zhengzhou University, China
Xin Nie	Zhengzhou University, China

Workshop Chair

Shaojun Zou	Hainan University, China

Publicity Chairs

Jia Hu	University of Exeter, UK
Jing Guo	Hainan University, China
Wenxuan Zhang	Hainan University, China

Publication Chairs

Binbin Zhou	Hangzhou City University, China
Lan Yan	Hainan University, China

Registration Chairs

Yuan Tian	Hainan University, China
Tingting Han	Hainan University, China

Web Chairs

Jiawei Wang	University of Warwick, UK
Ren Li	Hainan University, China

Steering Committee

Yang Xiang	Deakin University, Australia
Wanlei Zhou	City University of Macau, China
Yi Pan	Shenzhen University of Advanced Technology, China
Laurence T. Yang	Zhengzhou University, China
Weijia Jia	Beijing Normal University, China

Program Committee

Adnan Anwar	Deakin University, Australia
Sen Bai	Tsinghua University, China
Wen Bai	Hubei University, China
Selcuk Baktir	American University of the Middle East, Kuwait

Nicola Bena	University of Milan, Italy
Ran Bi	Northeastern University, China
Yanling Bu	Nanjing University of Aeronautics and Astronautics, China
Massimo Cafaro	University of Salento, Italy
Miao Cai	Nanjing University of Aeronautics and Astronautics, China
Zhicheng Cai	Nanjing University of Science and Technology, China
Chenhong Cao	University of Science and Technology of China, China
Zhilei Chai	Jiangnan University, China
Sudip Chakraborty	Valdosta State University, USA
Xiaolin Chang	Beijing Jiaotong University, China
Zhaoxin Chang	Institut Polytechnique de Paris, France
Bo Chen	Shenzhen University, China
Chao Chen	Chongqing University, China
Huijie Chen	Beijing University of Technology, China
Jiaoyan Chen	Wuhan University of Science and Technology, China
Jing Chen	Hainan University, China
Liying Chen	Hunan Normal University, China
Long Chen	Guangdong University of Technology, China
Longbiao Chen	Xiamen University, China
Ning Chen	Soochow University, China
Quan Chen	Guangdong University of Technology, China
Simin Chen	University of Texas at Dallas, USA
Ting Chen	University of Electronic Science and Technology of China, China
Xiang Chen	Zhejiang University, China
Ying Chen	Hunan Normal University, China
Yuzhu Chen	Changsha Social Work College, China
Zi Chen	Wuhan University of Technology, China
Zixuan Chen	China Telecom, China
Francesco Colace	University of Salerno, Italy
Jin Cui	Northwest University, China
Jinhua Cui	Huazhong University of Science and Technology, China
Lin Cui	Jinan University, China
Weihao Cui	Shanghai Jiao Tong University, China
Yuan Tian	Hainan University, China
Gianni D'Angelo	University of Salerno, Italy
Haipeng Dai	Nanjing University, China

Fan Dang	Beijing Jiaotong University, China
Xia Deng	Guangzhou University, China
Xiaojun Deng	Hunan University of Technology, China
Dian Ding	Shanghai Jiao Tong University, China
Kai Dong	Southeast University, China
Pingping Dong	Hunan Normal University, China
Xiaogang Dong	Jiujiang University, China
Bingqian Du	Huazhong University of Science and Technology, China
Haizhou Du	Shanghai University of Electric Power, China
Xin Du	Zhejiang University, China
Yang Du	Soochow University, China
Cheng Dai	Sichuan University, China
Christian Esposito	University of Salerno, Italy
Weibei Fan	Nanjing University of Posts and Telecommunications, China
Xin Fan	Beijing Forestry University, China
Xinggang Fan	Zhejiang University of Technology, China
Chunrong Fang	Nanjing University, China
Xing Fang	Xiamen University, China
Libo Feng	Yunnan University, China
Yunyun Feng	University of Science and Technology of China, China
Zhiwei Feng	Northeastern University, China
Shaojing Fu	National University of Defense Technology, China
Xiong Fu	Nanjing University of Posts and Telecommunications, China
Chengxi Gao	Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, China
Shang Gao	Hong Kong Polytechnic University, China
Tianchong Gao	Southeast University, China
Yifan Gao	Victoria University of Wellington, New Zealand
Harald Gjermundrød	University of Nicosia, Cyprus
Xiaoli Gong	Nankai University, China
Chaojie Gu	Zhejiang University, China
Chen Gu	Hefei University of Technology, China
Yunfei Gu	Shanghai Jiao Tong University, China
Zonghua Gu	Umeå University, Sweden
Zhitao Guan	North China Electric Power University, China
Shuai Guo	Qingdao University of Technology, China
Tao Guo	Shenzhen University, China

Yongming Han	Beijing University of Chemical Technology, China
Zengyi Han	Dalian Maritime University, China
Lin He	Tsinghua University, China
Tengjiao He	Jinan University, China
Yi He	University of CSU, China
Zhenli He	Yunnan University, China
Peng Hou	Soochow University, China
Jinbin Hu	Changsha University of Science and Technology, China
Pengfei Hu	Shandong University, China
Shihong Hu	Hohai University, China
Yusheng Hua	Nanjing University of Posts and Telecommunications, China
Kuochan Huang	National Taichung University of Education, Taiwan
Qinlong Huang	Beijing University of Posts and Telecommunications, China
Ran Huang	Victoria University of Wellington, New Zealand
Mingtao Ji	Hohai University, China
Juncheng Jia	Soochow University, China
Guiyuan Jiang	Ocean University of China, China
Shan Jiang	Sun Yat-sen University, China
Wenhui Jiang	Jiangxi University of Finance and Economics, China
Xianliang Jiang	Ningbo University, China
Lanju Kong	Shandong University, China
Peter Kropf	University of Neuchâtel, Switzerland
Laurent Lefevre	Inria, France
Bei Li	China Unicom, China
Bo Li	Victoria University, Australia
Borui Li	Southeast University, China
Chao Li	Shanghai Jiao Tong University, China
Dagang Li	Macau University of Science and Technology, China
Guangli Li	Institute of Computing Technology, Chinese Academy of Sciences, China
Hu Liang	Qilu University of Technology, China
Jiahui Li	Jilin University, China
Jingzong Li	City University of Hong Kong, China
Liying Li	Nanjing University of Science and Technology, China
Meng Li	Hefei University of Technology, China

Nannan Li	Tsinghua University, China
Ning Li	Universidad Politécnica de Madrid, Spain
Wenjuan Li	Education University of Hong Kong, China
Wenjun Li	Hunan Normal University, China
Xiaolu Li	Huazhong University of Science and Technology, China
Yan Li	Hunan University of Science and Technology, China
Zhiqiang Li	A10 Networks, USA
Teng Liang	Peng Cheng Laboratory, China
Xueqin Liang	Xidian University, China
Wenmin Lin	Hangzhou Normal University, China
Yijing Lin	Beijing University of Posts and Telecommunications, China
Zhen Ling	Southeast University, China
Bo Liu	Zhengzhou University, China
Dajiang Liu	Chongqing University, China
Hongyan Liu	Fuzhou University, China
Jia Liu	Nanjing University, China
Jianchun Liu	University of Science and Technology of China, China
Jianwei Liu	Changsha University, China
Shaojie Liu	Hainan University, China
Tao Liu	National Supercomputer Center in Jinan, China
Xuxun Liu	South China University of Technology, China
Ying Liu	National University of Defense Technology, China
Yuchen Liu	Beihang University, China
Xiaojuan Lu	Central South University, China
Lailong Luo	National University of Defense Technology, China
Luyao Luo	Soochow University, China
WangQing Luo	Hunan University of Science and Technology, China
Hongwu Lv	Harbin Engineering University, China
Sheng Ma	National University of Defense Technology, China
Shunmei Meng	Nanjing University of Science and Technology, China
Weizhi Meng	Lancaster University, UK
Yinbin Miao	Xidian University, China
Priyadarsi Nanda	University of Technology Sydney, Australia
Xuefeng Ni	Hunan University, China

Xintao Niu	Nanjing University, China
Shujie Pang	Guangdong University of Technology, China
Kai Peng	Huaqiao University, China
Yaqiong Peng	Hunan University, China
Chiara Pero	University of Salerno, Italy
Nikolaos Pitropakis	American College of Greece, Greece
Vincenzo Piuri	Università degli Studi di Milano, Italy
Lingjun Pu	Nankai University, China
Hui Qi	Changchun University of Science and Technology, China
Ji Qi	Chinese Academy of Sciences, China
Jiacheng Qu	Hainan University, China
Zhengwei Qi	Shanghai Jiao Tong University, China
Yan Qiao	Hefei University of Technology, China
Yuben Qu	Nanjing University of Aeronautics and Astronautics, China
Zhihao Qu	Hohai University, China
Vinayakumar Ravi	Prince Mohammad Bin Fahd University, Saudi Arabia
Shenyuan Ren	Beijing Jiaotong University, China
Fariza Sabrina	Central Queensland University, Australia
Jun Shao	Zhejiang Gongshang University, China
Chao Shen	Xi'an Jiaotong University, China
Fanfan Shen	Nanjing Audit University, China
Han Shen	Xi'an Technological University, China
Hang Shen	Nanjing Tech University, China
Houqiang Shen	Changsha University of Science and Technology, China
Zhishu Shen	Wuhan University of Technology, China
Fang Shi	South China Agricultural University, China
Qingyu Shi	Hunan University of Technology and Business, China
Wanxin Shi	Fudan University, China
Na Song	Putian University, China
Qingyu Song	Xiamen University, China
Milos Stojmenovic	Singidunum University, Serbia
Qiwei Sun	Zhengzhou Police University, China
Lizhuang Tan	Qilu University of Technology, China
Weiqiang Tan	Guangzhou University, China
Chaogang Tang	China University of Mining and Technology, China
Hailun Tang	Tianjin University, China

Huijun Tang	Durham University, UK
Jianzhi Tang	Soochow University, China
Xing Tang	Wuhan University of Technology, China
Xiaoqiang Teng	Beijing Technology and Business University, China
Yun Teng	Hunan University of Science and Technology, China
Xiang Tian	Qilu University of Technology, China
Ying Wan	Southeast University, China
Bin Wang	Nanchang Hangkong University, China
Chenyang Wang	Shenzhen University, China
En Wang	Jilin University, China
Guijuan Wang	Qilu University of Technology, China
Jingzhou Wang	Soochow University, China
Kai Wang	Harbin Institute of Technology, China
Tian Wang	Jiangsu College of Finance and Accounting, China
Yingjie Wang	Yantai University, China
Zhijie Wang	Chongqing University, China
Zhu Wang	Northwestern Polytechnical University, China
Kaimin Wei	Jinan University, China
Jiaxin Wu	Guangdong University of Technology, China
Jinling Wu	Zhejiang Normal University, China
Wei Wu	Hainan Institute of Zhejiang University, China
Yalan Wu	Guangdong University of Technology, China
Zhongying Wu	Hainan University, China
Junxu Xia	National University of Defense Technology, China
Nian Xia	Nanjing Normal University, China
Qiufen Xia	Dalian University of Technology, China
Haolong Xiang	Nanjing University of Information Science and Technology, China
Zhengzhe Xiang	Hangzhou City University, China
XiongJiang Xiao	Hainan University, China
Mande Xie	Zhejiang Gongshang University, China
Jia Xu	Nanjing University of Posts and Telecommunications, China
Mengwei Xu	Beijing University of Posts and Telecommunications, China
Pin Xu	Hainan University, China
Shujiang Xu	Shandong Computer Science Center, China
Xianghao Xu	Nanjing University of Science and Technology, China

Yueshen Xu	Xidian University, China
Xingfu Yan	South China Normal University, China
Bo Yang	Northwestern Polytechnical University, China
Bo Yang	Northwest A&F University, China
Chen Yang	Beijing Institute of Technology, China
Wen Yang	China Three Gorges University, China
Wenjun Yang	University of Victoria, Canada
Yubo Yang	Hunan University of Science and Technology, China
Biyuan Yao	Hainan University, China
XiaoXue Yin	Hainan University, China
Long Yuan	Wuhan University of Technology, China
Min Yuan	Nanjing Normal University, China
Hengshan Yue	Jilin University, China
Rongfei Zeng	Northeastern University, China
Yue Zeng	Nanjing University of Science and Technology, China
Xiangping Bryce Zhai	Nanjing University of Aeronautics and Astronautics, China
Hao Zhang	Harbin Institute of Technology, China
Jiapeng Zhang	Hunan University, China
Jinquan Zhang	Guangdong University of Technology, China
Lianming Zhang	Hunan Normal University, China
Qingyang Zhang	Anhui University, China
Shuhui Zhang	Shandong Computer Science Center, China
Xianwei Zhang	Sun Yat-sen University, China
Yin Zhang	University of Electronic Science and Technology, China
Yong Zhang	Shenzhen Institutes of Advanced Technology, Chinese Academy of Sciences, China
Yongqin Zhang	Hainan University, China
Yuhao Zhang	Tianjin University, China
Yun Zhang	Hunan Normal University, China
Yuhui Zhang	Chinese Academy of Sciences, China
Pinlong Zhao	Hangzhou Dianzi University, China
Shengrong Zhao	Qilu University of Technology, China
Tianfang Zhao	Jinan University, China
Yu Zhao	Chongqing University of Posts and Telecommunications, China
Yunfeng Zhao	Tianjin University, China
Zhihui Zhao	Taiyuan University of Technology, China
Tianyue Zheng	Southern University of Science and Technology, China

Xiao Zheng	Anhui University of Technology, China
Ruiting Zhou	Southeast University, China
Zhou Zhou	Changsha University, China
Rongbo Zhu	Huazhong Agricultural University, China
Weitao Zou	Northeast Forestry University, China
Xiaolin Zou	Zhaoqing University, China
Sen Liu	Fudan University, China
Weijie Liu	National University of Defense Technology, China
Chang Ruan	Changsha Unviersity of Science and Technology, China
Xinchen Wan	ByteDance, USA
Wei Wang	National University of Defense Technology, China
Linyi Han	Tianjin University, China
Ye Liu	Donghua UniversityChina
Jinquan Nie	University of Chinese Academy of Sciences, China
Le Qin	Jiangxi Normal University, China
Xinping Rao	Jiangxi Normal University, China
Wei Yang	Jiangxi Normal University, China
Jiamin Deng	Hong Kong Polytechnic University, China
Hongfei Huang	Hangzhou Dianzi University, China
Xin He	Nanjing University of Posts and Telecommunications, China
Youhuizi Li	Hangzhou Dianzi University, China
Jinhai Liu	Henan Normal University, China
Yafei Shi	Henan Normal University, China
Yukun Shi	Hangzhou Dianzi University, China
Muyu Deng	China Unicom, China
Xixi Sun	Hangzhou Dianzi University, China
Baofu Wu	Hangzhou Fanjia Technology Co. Ltd, China
Wenjian Xu	Zhejiang University of Science and Technology, China
Weiqi Yue	Hangzhou Dianzi University, China
Tingjun Zhao	Zhejiang University of Technology, China
Leilei Zheng	Hangzhou Dianzi University, China
Yuan Liu	Xiangtan University, China
Xia Ou	Zhejiang University of Technology, China
Yuntao Su	Hunan University of Technology and Business, China
Taotao Yu	Zhejiang University of Technology, China
Wenchao Dong	Central China Normal University, China

Hanrui Gao	Zhengzhou University, China
Xi Kong	Henan University, China
Wenhui Xu	Henan Normal University, China
Hongwei Yu	Beihang University, China

Contents

Regular Papers

Parallel and Distributed Algorithms and Applications

Parallel Optimization of Tokamak Guiding-Center Drift-Orbit Integration on the Sunway Bluelight II Supercomputer 3
Meng Song, Tao Liu, Baofeng Gao, Ying Guo, Jingshan Pan, Dawei Zhao, and Xiaoming Wu

Accelerating Graph Sampling in GNN Systems Through Accuracy-Aware Data Reuse 25
Yuchun Wang, Nan Jiang, Qi Wu, Hengshan Yue, and Xiaohui Wei

Scalable Distributed Index Construction for Large-Scale Graph-Based ANNS 43
Xiangyu Liu, Meiling Wang, and Yiming Zhang

Scalable and Contention-Resilient Concurrent Interval Tree 59
Kapil Kumar Attinagaramu and Praveen Alapati

A Resource Allocation Method of Blockchain Network Based on Edge Computing 79
Qian Su, Longfei Bai, Xiang Zhang, Guangqin Hu, and Xuejie Zhang

A Structure-Aware Irregular Blocking Method for Sparse LU Factorization 101
Zhen Hu, Dongliang Xiong, Kai Huang, Changjun Wu, and Xiaowen Jiang

A Deep Reinforcement Learning Algorithm with Ordered Action Space for Budget-Aware Workflow Scheduling in Heterogeneous Clouds 119
Yanfen Zhang, Longxin Zhang, Lili Du, Zhihua Wen, Buqing Cao, Sheng Lin, and Jianguo Chen

STM-WFBP: Selective Tensor Merging Method in Distributed Learning 137
Pingping Dong, Qingfen Yi, Lianming Zhang, and Wensheng Tang

EACC: Efficient Agent Context Cache Sharing for Multi-Agent Systems 151
Sihao Cheng, Yunfei Gu, Chentao Wu, and Jie Meng

AccuGraph: Memory-Efficient Full-Graph GNN Training on a Single GPU via Subgraph Accumulation 172
Liyang Wu, Menghan Jia, Lizhi Zhang, Yahui Wu, Gongqingjian Jiang, Jiezhong He, Chunye Gong, and Yinghui Gao

Internet of Things and Cyber-Physical-Social Computing

Average Delay Minimization Strategy for Multi-UAV Assisted MEC: A Lightweight Deep Reinforcement Learning Approach 195
Heng Liu, Qiang Tang, Kun Yang, and Lei Yang

LLM-Enhanced Multi-AUV Collaborative Search via Multi-agent Reinforcement Learning 215
Peijun Dong, Hang Tao, Yang Zhao, Wei Shi, and Hanjiang Luo

CSI-Based Human Activity Recognition Using Spatial-Temporal Features and Attention 236
Wenping Yu, Jianhao Wang, Yusong Dong, Zhewen Li, and Jianzhong Zhang

MoMa-Pos: An Efficient Object-Kinematic-Aware Base Placement Determination Framework for Mobile Manipulation 254
Beichen Shao, Nieqing Cao, Yan Ding, Xingchen Wang, Fuqiang Gu, and Chao Chen

Distributed Intelligent Computing with Kernel-Wise Deep Neural Network Partition over Collaborative Devices 271
Boya Sun, Yinglei Teng, and Nan Wang

Joint Caching and Offloading Optimization for Heterogeneous Task Patterns in Multi-access Edge Computing 291
Sihan Li, Yanqing Wu, XiaoDong Xu, and Gang Xu

Open World Person Retrieval and Its Distributed Parallel Training Framework 311
Lan Yan, Wenbo Zheng, and Kenli Li

WSRNet: A Multi-task Learning Model for Satellite Image Recognition and Segmentation of Forest Wildfires 330
Juan Luo, Tianyang Liu, Kexuan Feng, Guogen Zeng, and Fan Li

SCoT Guard: A Safety Chain of Thought Guardrail Model 349
Yuxuan Lin, Shi Liu, Dongqin Liu, Wei Mi, and Xuehai Tang

Performance Modeling and Evaluation

Atys: An Efficient Profiling Framework for Identifying Hotspot Functions in Large-Scale Distributed Microservices 367
Jiaqi Sun, Dingyu Yang, Shiyou Qian, Jian Cao, and Guangtao Xue

Path Optimization of USV Based on Stochastic Schemata Exploiter with New Exploitation Strategy 387
Jianshan Liang and Yi Zuo

On Adaptive Cooperative Strategy for Throughput Enhancement in V2V/V2I Communications 400
Weidong Yang, Xinzhuo Zhang, Yan Liu, Ranran Sun, and Yulong Shen

CoDVFS: Improving the Energy Efficiency of AI Servers Through Coordinated DVFS 419
Yijia Zhang, Baoqing Wang, Shixin Zhang, and Qiang Wang

Machine Learning and Data Statistics Based Scalability Bottleneck Detection 440
Yuhan Cao, Haoming Zhan, Juan Chen, and Mingyuan Zhang

Aspect Term Extraction Method Based on EBERT-CBAC 462
Yun Wu, Kai Yang, Yan Du, Jieming Yang, Ziyi Wang, Ning An, Nan Xu, and Yifan Huang

HACompBench: Co-designed Multimodal DNN Compression Evaluation for Edge Devices 478
Zhengyu Gan, Haohua Du, Chengquan Feng, and Haisheng Tan

Are We There Yet? Unraveling the State-of-the-Art Binary Feedback-Directed Optimizations 497
Mingliang Liu, Shanlin Deng, and Baojian Hua

Towards Efficient SpMV: A Multi-Aware Optimization Framework for Heterogeneous Architecture 518
Yang Liu, Zexin Wang, Xinyin Zhang, Zenghui Ren, and Yonghua Zhao

GMS-YOLO: A Lightweight Algorithm for SAR Ship Target Detection Based on YOLOv8n 537
Xiaozhang Liu, Xinting Zhou, Xiulai Li, and Jiajing Xu

ERL-CDR: Evolutionary Reinforcement Learning with Causal Decoupling Representation 555
Zuohao Wu, Guang Li, Zihao li, Jiaqi Wei, and Mingsheng Shang

Model Archive and Ensemble Learning Based Differential Evolution Algorithm for Mixed-Variable Optimization 571
Qing Chen, Xiaogang Dong, and Changshou Deng

Author Index 591

Parallel and Distributed Algorithms and Applications

Parallel Optimization of Tokamak Guiding-Center Drift-Orbit Integration on the Sunway Bluelight II Supercomputer

Meng Song[1,2], Tao Liu[1,2(✉)], Baofeng Gao[1,2], Ying Guo[1,2], Jingshan Pan[1,2], Dawei Zhao[1,2], and Xiaoming Wu[1,2]

[1] Key Laboratory of Computing Power Network and Information Security, Ministry of Education, Shandong Computer Science Center (National Supercomputer Center in Jinan), Qilu University of Technology (Shandong Academy of Sciences), Jinan, China
[2] Shandong Provincial Key Laboratory of Computing Power Internet and Service Computing, Shandong Fundamental Research Center for Computer Science, Jinan, China
liutao@sdas.org

Abstract. High performance computing is essential for complex fusion simulations, especially for guiding-center drift-orbit integration, which accurately describes important physical phenomena in Tokamak devices, such as high-energy particle transport, current actuation, and energy deposition. In this paper, we develop a two-level parallel strategy for the Tokamak guiding-center drift-orbit integration program (ORBIT) on the Sunway Bluelight II supercomputer that combines thread-level and process-level optimizations. Thread-level improvements include computation data optimization, out-of-order output optimization, and particle tracking optimization, while process-level optimization methods are based on particle independence to develop new communication domain and data processing methods to reduce latency and improve scalability. On a single core group, thread-level optimization provides up to 12.02x speedup. When expanded to 2,048 processes (133,120 cores), the two-level optimized program has a strong scaling efficiency of 66.6% and a weak scaling efficiency of over 98% on a 16-process benchmark. This approach greatly accelerates ORBIT and enriches the application ecosystem of the Sunway system.

Keywords: High performance computing · Tokamak · Guiding-center drift-orbit integration method · Sunway supercomputer · ORBIT

1 Introduction

With the development of nuclear fusion technology, stable high-temperature plasma confinement has become the core challenge to achieve controllable nuclear fusion [1]. Tokamak [2] maintains a self-sustaining fusion reaction by confining the plasma with a precise magnetic field. However, the guiding-center motion

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 3–24, 2026.
https://doi.org/10.1007/978-981-95-8405-5_1

behavior of plasma under extreme conditions, such as fast ion orbit loss, neoclassical transport, and magnetic island interaction, presents significant challenges to long-pulse operation and the confinement of high-energy particles [3]. By reducing the six-dimensional phase space to the evolution of five-dimensional field line coordinates, the orbit tracking method based on the guiding-center approximation provides an effective means for studying high-energy particle transport and Alfvén-wave instabilities [4]. Numerical integration of the guiding-center equations, typically with fourth-order Runge–Kutta schemes and field-line coordinate mapping, requires frequent access to magnetic-equilibrium data and high-dimensional interpolation [5].

The Sunway Bluelight II supercomputer, an upgraded successor to Sunway TaihuLight [6], incorporates the SW39000 heterogeneous many-core processor and forms a new-generation fusion-simulation platform. Its the platform combines a multi-tier distributed computing architecture with high mixed-precision performance, providing the guiding-center drift-orbit code ORBIT [7] with powerful coupled-field solving capability for its simulations. Cooperative design between Management Processing Element (MPE) and Computation Processing Elements (CPEs), together with a globally scalable memory hierarchy, enables efficient dynamic reconstruction of unstructured magnetic-surface data and six-dimensional phase-space discretization during orbit integration [8].

This work performs thread-level and process-level parallel optimizations of the ORBIT code on the Sunway Bluelight II supercomputer. At the thread level, particle losses in ORBIT make the CPEs output results in an indeterminate order. We have successfully eliminated the problem of out-of-order data output by utilizing Remote Memory Access (RMA) and Direct Memory Access (DMA) technology; this method can be applied to any application in which the CPEs cannot determine their final write addresses beforehand. Because ORBIT particles are inherently independent, we also introduce a particle tracking optimization that assigns each particle to a dedicated CPE. Each CPE owns the entire life cycle of a particle—initialization, orbit advancement, and final output—which improves computational efficiency, reduces the number of data transmissions between MPE and CPE, fully utilizes data locality, and evenly distributes tasks. At the process level, the data that need to be exchanged are placed in the interleaved segment of the SW39000 processor. Moreover, a new communication domain is created to lower inter-process overhead. Optimizing the data-movement path and processing logic further increases the program efficiency and improves the performance of large-scale particle-orbit simulations.

The contributions of this paper are summarized as follows:

(1) We propose a scheme for the SW39000 processor that efficiently exploits the CPEs shared Local Data Memory (LDM) resources and performs the data preprocessing on the MPE, thereby maximizing spatial locality and fully utilizing CPE memory capacity.
(2) We develop a solution to the out-of-order data output problem between the MPE and the CPEs. The monitor CPE exchanges data among the CPEs via

RMA, and interacts with the MPE via DMA, to allocate arrays of appropriate size, achieving efficient data transfer and better use of memory of the MPE.
(3) In addition, we leverage the intrinsic independence of particles to track each particle's entire life cycle, and we optimize inter-process communication by taking advantage of the architecture of the Sunway Bluelight II supercomputer and the SW39000 processor.

The remainder of this paper is organised as follows. Section 2 presents the background. Section 3 analyses the Tokamak guiding-center drift-orbit integration program. Section 4 describes the thread-level parallel optimization of ORBIT. Section 5 details the process-level parallel optimization of ORBIT. Section 6 reports the experimental results and analysis. Section 7 describes related work. Finally, Sect. 8 concludes the paper.

2 Background

2.1 Tokamak Guiding-Center Drift-Orbit Integration Method

In tokamak and other controlled-fusion devices, the motion of charged particles is highly complex, and direct orbit tracking is both difficult and computationally expensive. Guiding-center drift-orbit integration is employed to model particle trajectories, and can better analyze the particle dynamics, drift effects, and overall plasma confinement performance [9].

The guiding-center drift-orbit integration method is an efficient numerical technique based on the guiding-center approximation. It computes drift trajectories in electromagnetic fields and motion along magnetic field lines. This method approximates the long-term, large-scale motion of particles in strong magnetic fields by tracking the center point of the cyclotron motion of charged particles [10]. Its chief advantage is very high computational efficiency, which makes it well suited to studies of particle transport in magnetically confined and space plasmas. The key task is to integrate the guiding-center equations. In this study, we use guiding-center Eq. (1) that is not limited to Boozer coordinates [11]; equal-arc-length, PEST, and straight-field-line coordinates ψ_p, θ, and ζ, together with the parallel velocity and the particle energy, completely determine the particle position and velocity.

$$\mathbf{B} = g(\psi_p)\,\nabla\zeta + I(\psi_p)\,\nabla\theta + \delta(\psi_p, \theta, \zeta)\,\nabla\psi_p \tag{1}$$

$\mathbf{B}$ denotes the magnetic field, ψ the toroidal flux, ψ_p the poloidal flux, θ the poloidal angle coordinate, ζ the toroidal angle coordinate, and g, I, and δ are equilibrium functions. Numerical values for the guiding-center equations are supplied by the plasma magnetic-field distribution, that is, the equilibrium field. Magnetic-equilibrium data are obtained by solving the Grad–Shafranov equation with the Equilibrium Fitting (EFIT) code [12]. In a Tokamak, the equilibrium field is usually expressed through the poloidal-flux function $\psi\,(R,\ Z)$, which satisfies the Grad–Shafranov Eq. (2).

$$R\frac{\partial}{\partial R}\left(\frac{1}{R}\frac{\partial \psi}{\partial R}\right)+\frac{\partial^2 \psi}{\partial Z^2}=-\mu_0 R^2\frac{dp}{d\psi}-F(\psi)\frac{dF}{d\psi} \tag{2}$$

- $\psi(R, Z)$ is the poloidal flux.
- $p(\psi)$ is the plasma pressure, assumed to depend only on ψ under ideal-MHD equilibrium.
- $F(\psi) = R\,B_\phi$ is related to the toroidal magnetic-field component B_ϕ.
- μ_0 is the permeability of free space.
- (R, Z) are the cylindrical coordinates.

EFIT solves the nonlinear partial differential Eq. (2) by iteratively adjusting the free functions $p(\psi)$ and $F(\psi)$ until the computed ψ, magnetic field, and pressure profiles agree with the measured data. After the experimental data are loaded, the program performs interpolation and spline fitting, thus producing smooth magnetic-field and flux Eq. (1).

In the guiding-center drift-orbit method, Eq. (1) is solved with a high-accuracy, fourth-order Runge-Kutta integrator. This scheme offers very high precision and good stability, making it suitable for numerically solving most ordinary differential equations. The procedure first evaluates the four stage slopes k_1âĂŞk_4 in Eqs. (3)–(6) and then updates the solution vector by the weighted average given in Eq. (7).

$$k_1 = f(t_n, y_n) \tag{3}$$

$$k_2 = f\big(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_1\big) \tag{4}$$

$$k_3 = f\big(t_n + \tfrac{h}{2},\, y_n + \tfrac{h}{2}k_2\big) \tag{5}$$

$$k_4 = f\big(t_n + h,\, y_n + h\,k_3\big) \tag{6}$$

$$y_{n+1} = y_n + \frac{h}{6}\big(k_1 + 2k_2 + 2k_3 + k_4\big) \tag{7}$$

t_n denotes the current time level, y_n is the phase-space state vector of the guiding-center drift orbit, h is the integration step size, $f(t, y) = \frac{dy}{dt}$ gives the orbit's phase-space velocity evaluated at (t, y), k_i $(i = 1, \ldots, 4)$ are the four stage slopes of the fourth-order Runge-Kutta scheme, and y_{n+1} is the updated state vector at the next time level.

ORBIT is a numerical simulation code designed for particle tracking and guiding-center drift-orbit calculations in Tokamak plasmas. It is chiefly used to study charged-particle transport, drift-orbit losses, and energetic-particle dynamics in magnetically confined fusion devices. Given an equilibrium magnetic-field configuration from an external solver, ORBIT integrates particle trajectories numerically and supports detailed analyses that include particle collisions, wave–particle interactions, and Poincaré-section diagnostics.

2.2 The Sunway Bluelight II Supercomputer

Figure 1 depicts the architecture of the SW39000 processor. The chip contains six core groups (CGs). Each CG comprises one management processing element (MPE) and one computation processing elements cluster (CPE Array). The CPE Array is an 8 * 8 mesh of 64 computing processing elements (CPEs). Within this mesh, every four adjacent CPEs form a subcluster that shares a local management unit and incorporates a router. Each CG has a dedicated memory controller (MC) connected to 16 GB of DDR4 memory that delivers 51.2 GB/s of bandwidth. Data transfer among CPEs in the same array is handled through a Remote Memory Access (RMA) interface.

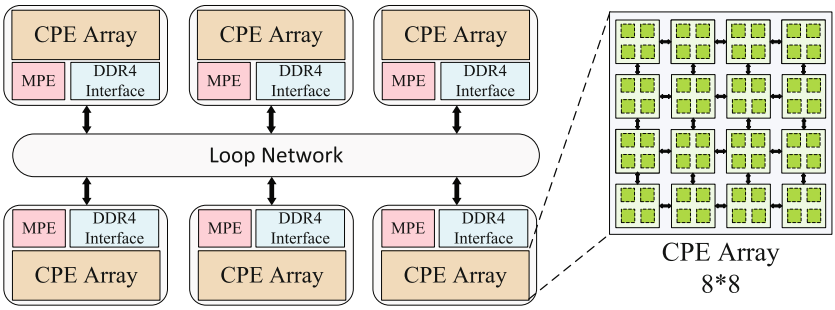


Fig. 1. SW39000 many-core processor architecture.

Each CPE contains a 256 KB Local Data Memory (LDM). For memory access, the CPE can access main memory directly through discrete `gld`/`gst` operations, or it can employ Direct Memory Access (DMA) to transfer data from the main memory into the LDM. During computation, the CPE reads and writes solely through its LDM. The LDM can be partitioned into three configurable regions: the private region, the contiguous shared region, and the CPE data-cache region.

The Sunway Bluelight II supercomputer is the next-generation Sunway system following Sunway TaihuLight. It employs entirely domestically developed components, including processors, network chipsets, storage, and management systems, and has served as a comprehensive technology test bed for exascale computing. The machine comprises 512 compute nodes that house 1,024 SW39000 many-core processors, providing 6,144 management processing elements (MPEs) and 393,216 computation processing elements (CPEs) in total. A single processor delivers peak speeds of 12.48 TFLOPS in double-precision floating point, 13.956 TIPS in single-precision fixed point, and offers a memory bandwidth of 282 GB/s. The entire system attains a peak performance of 14.3 PFLOPS.

3 Analysis of the Tokamak Guiding-Center Drift-Orbit Integration Program

This section analyses the hotspot functions of the Tokamak guiding-center drift-orbit integration program (ORBIT) and describes the challenges encountered in implementing its parallel optimization on the Sunway Bluelight II supercomputer.

During the performance assessment and correctness verification of ORBIT, we combined explicit time stamping, the *gprof* profiler, and the SWLU hotspot analyzer provided by the Sunway supercomputer to obtain a detailed profile of the code's behavior, including per-function execution time, call frequency, call relationships, and runtime bottlenecks. To ensure the reliability and accuracy of the numerical results, we used Poincaré-section distributions as an independent benchmark for checking the physical correctness of the program output.

Poincaré-section data are obtained by projecting a high-dimensional dynamical system or particle-orbit simulation onto a two-dimensional cross section, producing a clear and stable topological pattern. By comparing the simulated section with theoretical or reference data and examining differences in point locations, densities, and orbit shapes, one can directly and efficiently detect integration errors or implementation defects and thereby verify the correctness of the ORBIT program results.

We profiled the code on the single MPE of the SW39000 processor. The functions of the ORBIT program include `Field`, `Ptrba`, `Onestep`, `Update`, and `Others`. The `Field` function is responsible for calculating the local magnetic field vector and related equilibrium quantities in the Tokamak during particle propulsion; the `Ptrba` function calculates the perturbation field; the `Onestep` function updates the particle coordinates and velocity; the `Update` function counts and processes all particle states; and the `Others` category includes reading equilibrium data, outputting Poincaré-section distribution data, etc. Hotspot functions include `Field`, `Ptrba`, `Onestep`, and `Update`, which account for 32.10%, 30.70%, 17.42%, and 12.81% of the total execution time, respectively. The `Others` function accounts for 6.97% of the total execution time. In each iteration of the `Onestep` and `Update` calculations, the `Field` and `Ptrba` functions are invoked. Excessive iterations make the `Field` and `Ptrba` functions particularly time-consuming.

During the advancing phase in ORBIT, no inter-particle forces influence the motion of other particles. Each particle possesses its own velocity, charge, and mass. The functions that update the particle state and the surrounding magnetic field operate independently of other particles, and the data used by `Onestep` and `Update`, as well as by `Field` and `Ptrba`, are highly similar. Based on this feature, the data reuse rate is improved through the optimization of particle tracking.

Based on the characteristics of ORBIT and the hardware architecture of the Sunway Bluelight II supercomputer, we identify four major challenges faced in parallel optimization:

(1) **Data is discrete and large:** When the original program reads the equilibrium data, the data are discrete and the data volume is too large. In the step of advancing the particles, the efficiency of using `gld`/`gst` to obtain data from the MPE is low. The LDM space of the CPEs is only 256 KB, which cannot store the data required for the CPEs computation.
(2) **Out-of-order output:** Particle losses occur while particles are being advanced, so although a fixed number of particles are assigned to each CPE, the number that complete is unknown in advance; consequently, the CPEs cannot guarantee that their output data appear in the correct order.
(3) **Frequent DMA communication:** When the functions `Field`, `Ptrba`, `Onestep`, and `Update` are executed on the CPEs, it leads to frequent updates of computing data, increasing the time consumption of DMA communication. In addition, the activation and recovery frequency of the CPEs increases.
(4) **High overhead for large-scale problems:** As the number of processes increases, collecting the results of particle transport leads to more and more frequent communication between processes, and processing data that require interaction also consumes a significant amount of time.

To address these challenges, we propose a fine-grained, generic two-level parallelization scheme, built on the Sunway many-core processor architecture and the overall architecture of the Sunway Bluelight II supercomputer. The scheme comprises thread-level and process-level parallelization. At the thread level, we develop computational data optimization, out-of-order output optimization, and particle tracking optimization, targeting challenges (1), (2), and (3), respectively. At the process level, we present a new communication domain method to tackle challenge (4).

4 Thread-Level Optimization for ORBIT

This section presents the thread-level optimization of ORBIT within a single CG, including computational data optimization, out-of-order output optimization, and particle tracking optimization. The three optimizations are progressive: out-of-order output optimization is built on computational data optimization, and particle tracking optimization, in turn, is built on out-of-order output optimization.

4.1 Computational Data Optimization

The purpose of computational data optimization is to discard unnecessary data, speed up data transfer between the MPE and the CPEs, exploit the spatial locality, and make efficient use of the limited LDM space. According to Sect. 3, the functions `Field`, `Ptrba`, `Onestep`, and `Update` together account for 93.03% of the execution time of the ORBIT program. `Field` and `Ptrba` need the distribution data of particles in spatial position, energy, and pitch angle, and the position of particles can be generated by the random variable x in Eq. (8).

$$\frac{dn}{d\psi} = \frac{dn}{dx}\frac{dx}{d\psi} = f_{\psi} \tag{8}$$

Furthermore, the particle energy distribution follows the Maxwell distribution, that is, $\mathrm{en}(k) = \texttt{maxwell}(t_{\mathrm{dum}})$, where the parameter t_{dum} denotes the plasma temperature in keV. The data required by these hotspot functions are highly scattered and very large, and transferring them with `gld`/`gst` operations is inefficient. Therefore, we employ the more efficient DMA transfers between the MPE and the CPEs.

The computational data optimization includes two steps: preprocessing the data on the MPE, the use of LDM shared space and the choice of mode.

(1) Preprocessing the data on the MPE

In the first part, we reconstruct the original array data and only keep the data required by the hotspot function. The data processing is completed by the MPE. As shown in Fig. 2, first, before using the CPEs to process the hotspot function, the calculation part of the array data in the hotspot function is extracted, and the MPE calculates the subscript of the array data in advance. Secondly, use Boolean arrays to filter duplicate data, and then build a new array to store the data actually required for the computing task. Then, establish a mapping relationship between the computing task and the new array, and create an array to save the mapping relationship. Finally, Calculate the amount of data N (in KB) actually required for the computing task.

The CPEs of SW39000 support LDM continuous shared space, which is a space used by the CPEs for shared access to LDM within the array. The CPEs can access the LDM space of other CPEs. In LDM continuous sharing mode, each CPE in the array provides LDM space of the same capacity for continuous addressing. The capacity of the LDM space provided by each CPE for sharing can be divided into seven capacity adjustments: 0 KB, 4 KB, 8 KB, 16 KB, 32 KB, 64 KB, and 128 KB. As shown in the lower left part of Fig. 2, the continuous sharing mode includes single cluster, double cluster, four cluster, and full array sharing modes, which facilitates nearby access to different working sets.

(2) The use of LDM shared space and the choice of mode

The second part concerns the use of the LDM shared space and the choice of its operating mode. The "read-only data" set includes the actual data N; among the hotspot functions, `Field` requires the most "read-only data" (282.31 KB), while `Onestep` uses the largest amount of "computational data" (197.5 KB). Consequently, we allocate 32 KB of LDM of each of the CPEs as contiguous shared space for "read-only data", reserving the remainder as private space for intermediate variables and results. As shown in Fig. 2, we select the four-cluster contiguous share mode, providing a total shared LDM of 512 KB. Before the CPEs execution, arrays B and C are transferred via DMA into this shared space. Since the original array A is scattered while array B is contiguous, the CPEs leverage spatial locality to accelerate memory access, reducing MPE–CPE communication overhead and improving overall performance.

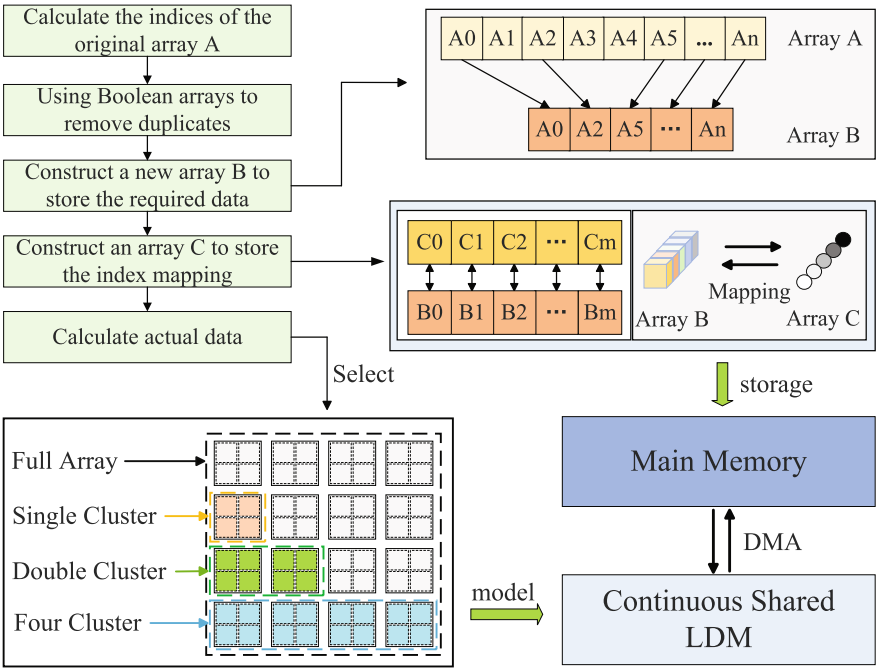


Fig. 2. Computational data optimization.

4.2 Out-of-Order Output Optimization

When we use computational data optimization, the situation that the CPE cannot guarantee that the data is output in the correct order occurs. This is because during the particle propulsion phase, particle collisions and external disturbances are also performed, resulting in particle escape during the movement of the plasma. The collision effect randomizes the particle pitch angle, which pushes some particles into the "loss cone" and thus escapes directly along the open magnetic field lines. The ORBIT program uses the diffusion-drift (Fokker–Planck) Eq. (9) to describe the pitch angle scattering:

$$\frac{\partial f}{\partial t} = \frac{1}{2}\frac{\partial}{\partial \xi}\left(D_{\xi\xi}\frac{\partial f}{\partial \xi}\right) - \frac{\partial}{\partial \xi}(A_\xi f) \tag{9}$$

f is the particle distribution function, $\xi = \cos\alpha$ the pitch-angle cosine, $D_{\xi\xi}$ the diffusion coefficient, and A_ξ the drag term. External magnetic perturbations are represented by Eq. (10).

$$\delta \mathbf{B} = \nabla \times \left(\alpha(\psi, \theta, \zeta)\, \mathbf{B}\right) \tag{10}$$

These resonant perturbations change particle trajectories, break closed flux surfaces, and create random magnetospheres and island chains, allowing trajec-

tories to cross flux surfaces and escape the plasma. The perturbation amplitude α is expanded in harmonics.

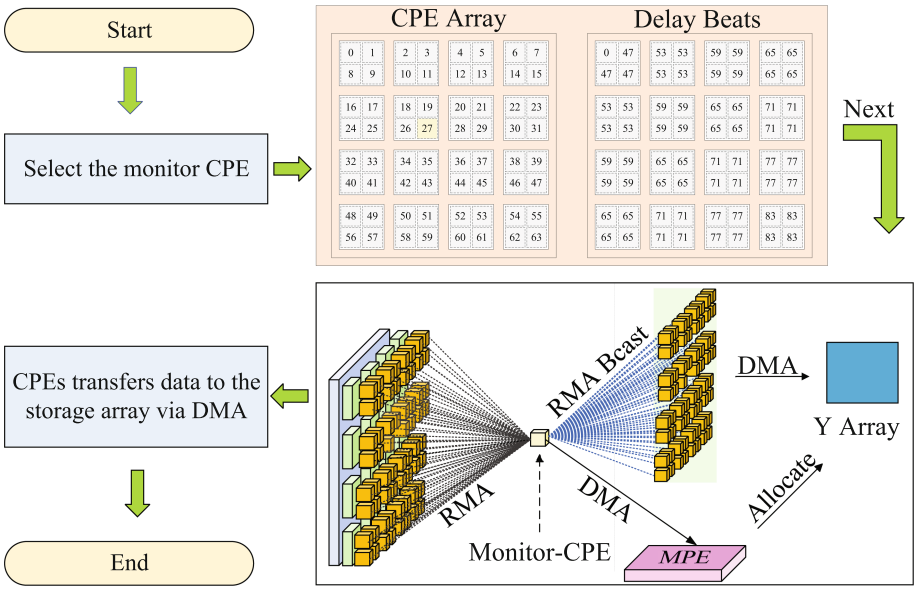


Fig. 3. Out-of-order output optimization.

We take an optimization approach for out-of-order output from the CPEs. Firstly, the monitor CPE is selected from the CPE array. According to the characteristics of the hardware architecture of the Sunway Bluelight II supercomputer and SW39000 many-core processor, the distance between the cluster of the CPEs affects the communication delay between the CPEs. Through the RMA communication test on the CPE array, this paper takes the RMA communication test between the CPE 0 and other CPEs as an example. As shown in the upper part of Fig. 3, the RMA communication delay between the CPE 0 and the CPEs 1, 8 and 9 is 47 beats. When the inter-cluster distance of the CPEs increases by 1, the communication delay increases by 6 beats. Selecting CPE 0 as the monitor CPE to communicate with other CPEs, the communication delay will reach 4,113 beats. The RMA communication test is carried out on different CPEs, and the CPE 27 is selected as the monitor CPE to communicate with other CPEs. The communication delay will be reduced to 3,729 beats, which meets the standard of low communication delay and more stable communication. Therefore, we choose the CPE 27 as the monitor CPE, and the other 63 CPEs in the CPE array are responsible for the computing task as the computing CPEs.

After selecting the monitor CPE, computing CPEs initially store processed data within their local LDM and propagate data volumes to the monitor CPE via

Algorithm 1: CPE out-of-order output optimisation

Given: $monitor_CPE \leftarrow$ ID of the monitor-CPE
$collect[64] \leftarrow$ Per-CPE counters
$distribution[64] \leftarrow$ Prefix sums of $collect$
$output[\,] \leftarrow$ Result buffer on the MPE
$slave_result[\,] \leftarrow$ Local result array on each CPE

```
#define remote_ldm_addr(coreid,ldm_var) ((unsigned long)&ldm_var  ((unsigned long)coreid
« 20) | (1ULL « 45))|
if CPE_ID = monitor_CPE then
    for(i = 0; i <= 63; ++i) remote_ldm_addr(i, &collect[i]);
end
athread_ssync_array() ;                                              // All-CPE barrier
if CPE_ID = monitor_CPE then
    distribution[0] ← collect[0];
    for i ← 1 to 63 do
        distribution[i] ← distribution[i − 1] + collect[i − 1]
    end
    total_data ← distribution[63] + collect[63];
    athread_dma_put(para.total_data, &total_data, sizeof(int));
    athread_ssync_master_spe() ;                                     // Sync with MPE
end
athread_rma_bcast_coll(&distribution, &distribution, 64 · sizeof(int), monitor_CPE) ;
// Broadcast offsets
athread_ssync_master_array() ;                                       // Sync with MPE
// DMA data
if CPE_ID ≠ monitor_CPE then
    offset ← distribution[CPE_ID];
    size ← data · sizeof(double);
    athread_dma_put(output + offset, slave_result, size)
end
```

RMA. The monitor CPE aggregates the collective data volume while computing transfer addresses for individual computing CPEs. Subsequently, it performs a DMA transfer of the total data volume to the MPE, which allocates a storage array of adequate capacity in main memory. Concurrently, the monitor CPE broadcasts computed transfer addresses to all 63 computing CPEs through an RMA operation. Finally, computing CPEs execute DMA transfers from their local LDM to the main-memory storage array using assigned addresses. The detailed steps are given in Algorithm 1.

4.3 Particle Tracking Optimization

We execute each hotspot function on the CPE array separately, resulting in frequent data transfer in and out of the CPE array. This method is a phased thread-level optimization. Although it is generally applicable to applications, it cannot make deeper optimizations for the characteristics of ORBIT programs and SW39000 processors. When the MPE calls the CPEs to accelerate the processing of computing tasks, it needs to start the CPE array and pass in data. After the CPEs complete the computing task, they transmit data to the MPE, and the MPE recycles the CPE array. Frequent startup and recovery of the CPE array will incur additional time consumption, increase communication overhead, and result in low execution efficiency.

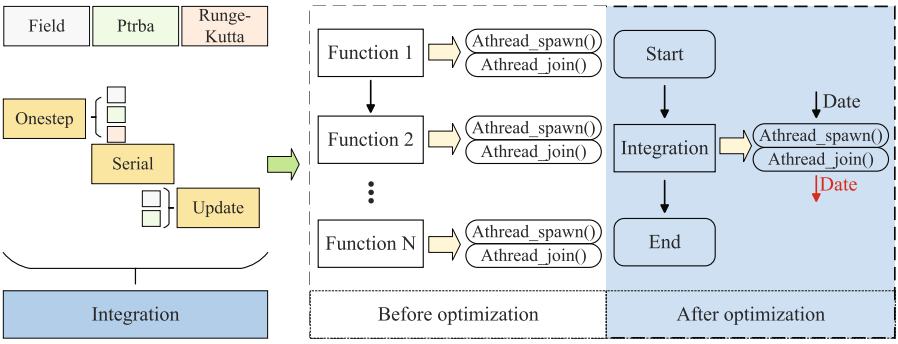


Fig. 4. Particle tracking optimization.

According to Sect. 3, particles have natural independence, and the data of hotspot functions are highly similar. Therefore, we adopt the method of tracking the entire movement process of particles instead of the previous staged method. Each CPE is bound to a different particle group and is responsible for the entire life cycle of the particle. As shown in Fig. 4, we reconstructed the data layout of the hotspot functions and merged the serial code between the hotspot functions into the function. Not only that, the `Field` function, `Ptrba` function and Runge–Kutta algorithm are integrated into the `Onestep` function. The `Field` and `Ptrba` functions are integrated into the `Update` function, and the `Onestep` function, `Update` function and serial code are organized and merged. When executing the integrated function in the CPE array, the number of data input and output is reduced to once from the startup and recovery of the CPE array, and the data reuse rate is improved.

5 Process-Level Optimization for ORBIT

This section introduces the process-level parallel optimization solution for ORBIT. Based on the "thread-level parallel optimization for ORBIT", we carried out the "process-level parallel optimization for ORBIT" and optimized the cluster communication and data processing of the SW39000 processor.

According to the characteristics of the guidance center equation, the particles in the ORBIT program have natural independence. The position and velocity of the particle can be completely described by a set of independent parameters, including the magnetic flux coordinate system (ψ_p, θ, ζ), the parallel velocity, and the energy of the particle. The determination of these parameters depends only on the size B of the magnetic field at the location of the particle and the functions I and g. The movement of any particle is independent of other particles and is only related to its own initial conditions and local magnetic field characteristics. This weak coupling between particles and the locality of the initial conditions indicate that each particle moves independently in the local magnetic field environment where it is located, and the solution of its trajectory

is suitable for parallel processing. Therefore, as shown in Fig. 5, we implement process-level parallel optimization by assigning particle groups to different CGs. In the annular cross-section diagram, the particles are divided into different particle groups, and each CG is responsible for the entire life cycle of different particle groups.

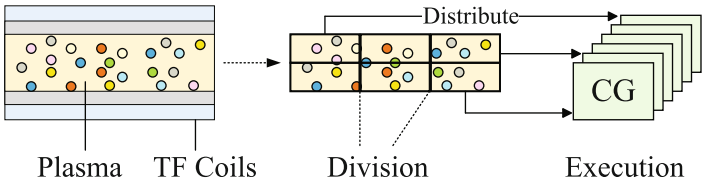


Fig. 5. Particle groups are assigned to the CGs.

On the SW39000 processor, main memory uses virtual addressing and is divided into two regions: the contiguous segment and the cross segment. The contiguous segment is private to each CG, laid out in a continuous address range and accessible directly by both the management and compute cores of that group. The cross segment is a shared region across all CGs on the same chip, and it can be accessed by every CG, enabling efficient data exchange within the chip.

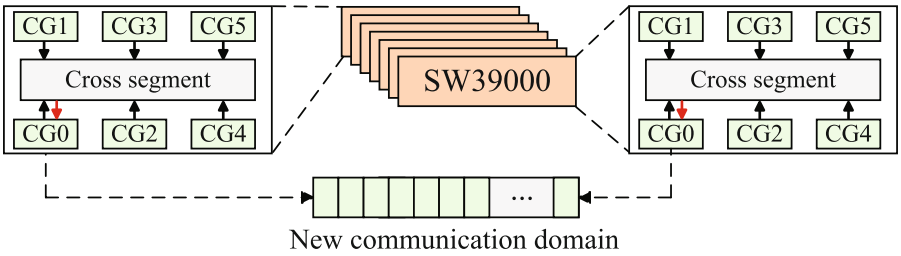


Fig. 6. New communication domain and cross segment.

As the number of processes increases, the frequency of communication between processes also increases. In the process-level optimization of the ORBIT program, we take the method of establishing a new communication domain to optimize the MPI cluster communication between SW39000 processors. Since process 0 is the root process of the entire communication domain, in order to facilitate process communication. As shown in Fig. 6, select the local CG 0 in each processor, create a new communication domain containing all CGs 0, and when data needs to be exchanged between CGs, first, put the data to be exchanged into the cross segment, so that the CGs in the same processor can directly operate the data without communication. Then, when the CGs between processors

exchange data, MPI communication is performed through the CGs in the newly created communication domain, so as to achieve the purpose of reducing the frequency of communication between processes. The detailed steps are given in Algorithm 2.

Algorithm 2: Process-level optimization method on SW39000

```
__cross double array[number];
// Create arrays at crossed segment
local_id ← current_array_id() ;                    // Get the process ID
color ← { 1,              local_id = 0
        { MPI_UNDEFINED,  otherwise
MPI_Comm_split(MPI_COMM_WORLD, color, my_rank, head_comm)
;                                                  // Create new communicator
if local_id = 0 then
    MPI_Comm_rank(head_comm, head_rank);
    MPI_Comm_size(head_comm, head_size);
    Computation begins;
        Data reduction operations;
    Computation ends;
    MPI communication;
    MPI_Comm_free(head_comm);
    // Release new communicator
end
```

6 Evaluation

6.1 Configuration

We use the Sunway Bluelight II supercomputer to evaluate the ORBIT program for two-level parallel optimization. The specific hardware and software used are shown in Table 1. In order to verify the universality of the scheme, all test input data are small, medium, and large scale. The principle of setting the data scale is to give full play to the computing power of the Sunway Bluelight II supercomputer and reflect the impact of different data scales on the optimization effect. All results are averaged over five rounds.

6.2 Performance of Various Parallel Optimizations

In thread-level parallel optimization, the number of simulated particles is set to 100,000, 500,000, and 1,000,000, corresponding to small, medium, and large data scales. The principle of setting the data scale is to give full play to the computing power of the Sunway Bluelight II supercomputer and show the impact of different

Table 1. Hardware and software used for optimisation.

Hardware	Computer	Sunway *Bluelight II* Supercomputer
	Processor model	SW39000
	Number of nodes	171
	Number of cores	133 120 (2 048 MPEs and 131 072 CPEs)
	Memory	32 768 GB
Software	System	Sunway Linux 4.4.15
	Compiler	`mpicc` and `sw9gcc`

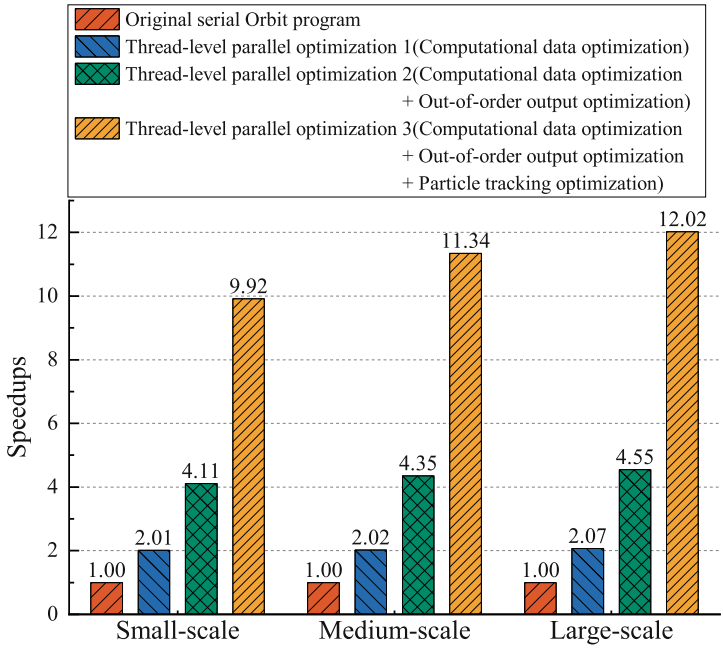


Fig. 7. Speedup ratio of thread-level optimization under different schemes.

data scales on the optimization effect. To test thread-level parallel optimization, a single-core group of one MPE and 64 CPEs is used on the SW39000 CPU.

Figure 7 shows the speedup achieved by the three thread-level optimization schemes. Under the first scheme, computational data optimization, we observe speedups of 2.01x to 2.07x across the three data scales. Building on that, the out-of-order output optimization delivers speedups of 4.11x to 4.55x. Finally, the particle tracking optimization, applied on top of the out-of-order output optimization scheme, achieves speedups of 9.92x to 12.02x.

In computational data optimization, we design a memory access scheme based on the LDM continuous sharing mode. We preprocess the data in the MPE, reconstruct the array and mapping relationship, and design a memory access

scheme based on the actual amount of data required for the task. In out-of-order output optimization, based on the characteristics of the hardware architecture of the Sunway Bluelight II supercomputer and the SW39000 many-core processor, we select the best monitor CPE in the CPE array, design the collaborative work of RMA and DMA, realize the data exchange between the MPE and the CPE in the CPE array, solve the problem of out-of-order output of the CPEs, reduce the communication overhead between the MPE and the CPE, and improve the utilization of main memory resources. In particle tracking optimization, we use the natural independence of particles to integrate multiple hot functions in the ORBIT program and the serial code parts between them, avoiding the delay caused by frequent startup and recovery of the CPE array and improving the data reuse rate.

According to our analysis, as shown in Fig. 8, compared with the original program, the proportion of the total execution time occupied by the four functions, `Field`, `Ptrba`, `Onestep` and `Update`, is significantly reduced, from 93.03% to 27.77%, and thread-level parallel optimization has achieved good results.

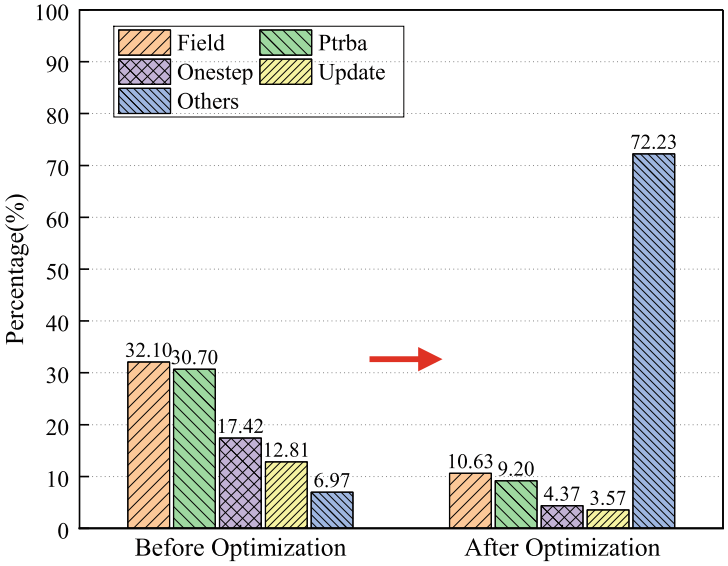


Fig. 8. Hotspots of thread-level optimization.

In process-level parallel optimization and two-level parallel optimization (including thread-level parallel optimization and process-level parallel optimization), the number of simulated particles is set to 7,168,000, 15,360,000 and 27,648,000 respectively, corresponding to small, medium, and large data scales. The process-level parallel optimization and two-level parallel optimization of the ORBIT program are tested with different numbers of processes (16, 32, 64, 128, 256, 512, 1,024, 2,048). As the number of processes increases, the number of

particles that the CPEs need to process becomes smaller and smaller. After the CPE array is accelerated in the two-level parallel optimization, the hotspot time ratio in the program shrinks much faster than that of the process-level optimization. Therefore, as the number of processes increases, the speedup ratio of the two-level parallel optimization gradually decreases relative to the process-level parallel optimization.

Figure 9 shows the execution time and speedup ratio of process-level parallel optimization and two-level parallel optimization at different process scales. We use the execution time of the program in 16 processes as a benchmark. Under the test of three data scales, the two-level parallel optimization is compared with the process-level parallel optimization, and the speedup ratios are as high as 42.79x, 44.46x, and 45.74x, respectively. When the number of processes is 2,048, compared with the process-level parallel optimization, the two-level parallel optimization maintains a speedup ratio of 20.59x, 25.43x, and 29.37x at different scales. This shows that the two-level parallel optimization program can make full use of the hardware resources of the Sunway Bluelight II supercomputer to handle more computing tasks. As the number of computing tasks increases, the two-level parallel program shows better acceleration performance in a large number of processes.

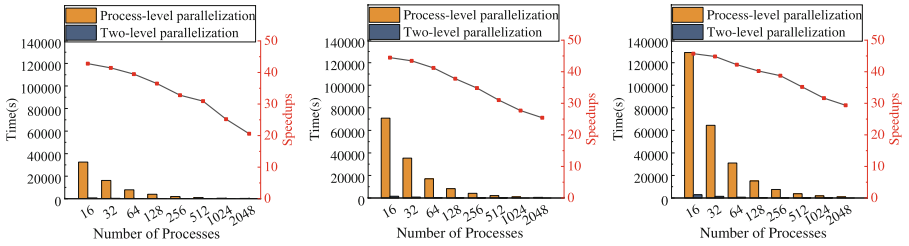


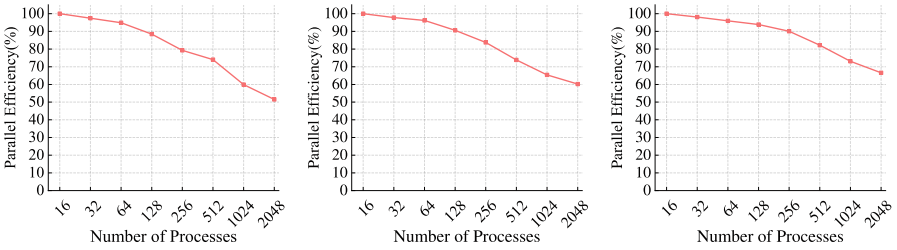
Fig. 9. Execution time and speedup of process-level and two-level parallelism.

6.3 Scalability

In the strong scalability test, three different particle sizes were set, namely 7,168,000, 15,360,000 and 27,648,000, corresponding to small-scale, medium-scale and large-scale input data respectively. The program execution time with 16 processes was used as the benchmark time to calculate the parallel efficiency of the program with other numbers of processes. The program used is a two-level parallel optimization scheme.

The parallel efficiency of the two-level parallel optimization scheme reaches 51.54% at a small scale, 60.19% at a medium scale, and 66.61% at a large scale. As shown in Fig. 10, a, b, and c represent the parallel efficiency of the two-level parallel optimization program at different data scales. The horizontal axis

represents the number of processes used in the test, starting from 16 processes and gradually increasing to 2,048 processes. The vertical axis represents the parallel efficiency of the two-level parallel optimization program as the number of processes increases. As shown in Fig. 10, when the number of processes increases to 512 processes and above, the parallel efficiency of the program decreases.

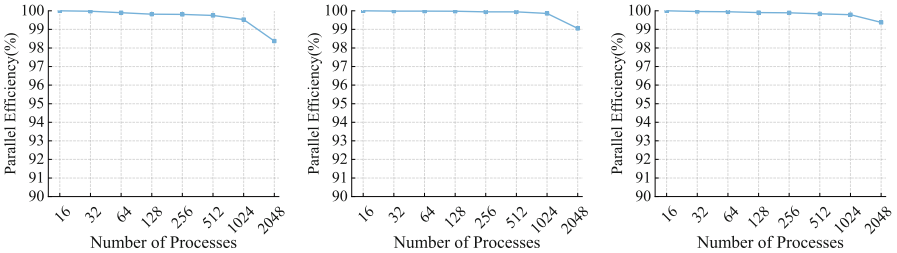


(a) Small-scale (b) Medium-scale (c) Large-scale

Fig. 10. Strong scalability of different sizes.

There are two reasons for this. The first reason is that as the number of processes increases, the number and delay of MPI communications also increase; the second reason is that as the number of processes used increases, the program needs to create more files to store output data, which causes the scalability of the program to decrease as the number of processes increases.

In the weak scalability test, three particles with different data scales were set. In order to keep the computational workload of each process unchanged, the specific number of particles was set to 100,000 * N, 600,000 * N, and 1,200,000 * N, where N is the number of processes used, corresponding to small scale, medium scale, and large scale, respectively. The program execution time of 16 processes was used as the benchmark test time to calculate the parallel efficiency of the program using different numbers of processes. Similarly, the program used was a two-level parallel optimization program.



(a) Small-scale (b) Medium-scale (c) Large-scale

Fig. 11. Weak scalability of different sizes.

Figure 11 shows the weak scalability of the two-level parallel optimization program under different numbers of processes and data sizes. When the number of processes is 2,048, the program maintains a parallel efficiency of 98.37% to 99.38% under different data sizes. This is because the particles in the program have natural independence. After our optimization in Sect. 5, the data processing speed within the processor is improved and the communication frequency between processes is reduced.

6.4 Results Verification

As described in Sect. 3, we validate the output of ORBIT by comparing the Poincaré cross-section distributions of six physical variables covering four complementary dimensions—conserved quantities, phase information, geometric measures, and Cartesian positions. Table 2 shows that the original program, the three thread-level optimizers, and the process-level optimizer all yield the same values, thus confirming the correctness of the program.

$\hat{p_\phi}$ gives the "radial" position (normalized) of the particle in the flux coordinate system, θ_{pol} provides the poloidal phase. x_{pos} and z_{pos} are the spatial positions in the experimental device coordinate system, r_{minor} gives the "radial" position (geometric quantity) of the particle in the flux coordinate system, and p_ϕ holds the unnormalized canonical momentum, which is used for dynamical classification.

Table 2. Poincaré-section data produced by ORBIT

Program version	$\hat{p_\phi}$	θ_{pol}	x_{pos}	z_{pos}	r_{minor}	p_ϕ
Original	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01
Thread-level 1 (Computational data)	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01
Thread-level 2 (Computational data +Out-of-order output)	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01
Thread-level 3 (Computational data +Out-of-order output +Particle tracking)	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01
Process-level	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01
Two-level (Thread-level 3 +Process-level)	4.93e−01	−1.52e+00	3.23e+02	−4.97e+01	6.79e−01	−4.92e−01

7 Related Work

At present, many studies have explored the Tokamak guiding-center drift orbit integration method, covering both theoretical and application optimiza-

tions. [13,14] evaluated a high-order Hamiltonian guiding-center theory with polarization correction and gyroscopic invariance, demonstrating high accuracy in conserving energy and canonical angular momentum in various magnetic configurations. [15] proposed a high-order Hermite interpolation to reconstruct divergence-free magnetic fields, preserving canonical momentum and magnetic moment, and enabling accurate trajectory integration and robust Poincaré cross-section analysis.

The guiding-center drift-orbit integration method has been widely applied in nuclear fusion simulations. [16] developed the relativistic guiding-center orbit program (RGCO) to study high-energy electron confinement, enabling full first-wall particle tracking in cylindrical coordinates. [17] proposed a fourth-order Runge–Kutta-based method for accurately simulating particle trajectories and drifts in dynamic fields. Leveraging Jacobian self-differentiation and spline smoothing, [18] achieved efficient, accurate coordinate transformations, improving speed and handling discontinuities and singularities effectively.

In the past decade, the Sunway supercomputer has achieved remarkable success, with three projects [19–21] winning the Gordon Bell Prize. [19] developed large-scale nonlinear earthquake simulation software on Sunway TaihuLight, sustaining performance above 18.9 PFLOPS. [20] created an ultra-scalable implicit solver for non-hydrostatic atmospheric dynamics, achieving 7.95 PFLOPS and 488 m resolution. [21] built a high-performance tensor-based quantum circuit simulator, reducing sampling time to 304 s and setting a milestone in classical simulation.

8 Conclusion

This paper implements the parallel optimization of the ORBIT program based on the Sunway Bluelight II supercomputer. A two-level parallel scheme is proposed for the parallel optimization of ORBIT, including thread-level and process-level optimization. At the thread level, three acceleration schemes are proposed within a single-core group, including calculation data optimization, output disorder optimization, and particle tracking optimization; at the process level, based on the characteristics of the hardware architecture of the Sunway Bluelight II supercomputer and the SW39000 many-core processor, the particle swarm allocation is designed in combination with the natural independence of particles in ORBIT, and the cross-segment and new communication domain in the SW39000 processor are used to process data, reduce the number of inter-process communications, and improve the execution efficiency of the program. Experimental results show that compared with the original program, the optimized program can achieve a speedup ratio of 12.02x within the single CG. When expanded to 2,048 processes and 133,120 cores, the optimized ORBIT still maintains good strong scalability and weak scalability.

Acknowledgement. This work is supported by Shandong Provincial Natural Science Foundation under Grant ZR2024LZH009, National Key Research and Develop-

ment Program under Grant 2024YFE0210800, Natural Science Foundation under Grant 62495062 and Beijing Natural Science Foundation under Grant L242017.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Sadik-Zada, E.R., et al.: Back to the future: revisiting the perspectives on nuclear fusion and juxtaposition to existing energy sources. Energy **290**, 129150 (2024)
2. Ding, S., et al.: A high-density and high-confinement tokamak plasma regime for fusion energy. Nature **629**(8012), 555–560 (2024)
3. Rud, M., et al.: Diagnostic weight functions in constants-of-motion phase-space. Nucl. Fusion **64**(3), 036007 (2024)
4. Most, E.R., et al.: Nonlinear alfvén-wave dynamics and premerger emission from crustal oscillations in neutron star mergers. Astrophys. J. Lett. **973**(2), L37 (2024)
5. Chen, K., et al.: Frkvf: high-accuracy motion interpolation for polishing operations using fourth-order runge-kutta and velocity flexibility planning. Precis. Eng. **90**, 122–140 (2024)
6. Haohuan, F., et al.: The sunway taihulight supercomputer: system and applications. Sci. China Inf. Sci. **59**, 1–16 (2016)
7. Eder, M., et al.: Gorilla: guiding-center orbit integration with local linearization approach. J. Open Source Softw. **8**(86), 4890 (2023)
8. Cathey, B., et al.: First six dimensional phase space measurement of an accelerator beam. Phys. Rev. Lett. **121**(6), 064804 (2018)
9. White, R.B.: Theory Of Toroidally Confined Plasmas, The. World Scientific Publishing Company (2013)
10. Ciftja, O., et al.: Cyclotron motion of a charged particle with anisotropic mass. Am. J. Phys. **85**(5), 359–363 (2017)
11. Rodriguez, E., et al.: Generalized boozer coordinates: a natural coordinate system for quasisymmetry. Phys. Plasmas **28**(9), 092510 (2021)
12. Ham, C.J., et al.: On multiple solutions of the grad-shafranov equation. Nucl. Fusion **64**(3), 034001 (2024)
13. Brizard, A.J., et al.: Faithful guiding-center orbits in an axisymmetric magnetic field. Phys. Plasmas **30**(4), (2023)
14. Brizard, A.J.: Particle and guiding-center orbits in crossed electric and magnetic fields. Phys. Plasmas **30**(4), (2023)
15. Beznosov, O., et al.: High order interpolation of magnetic fields with vector potential reconstruction for particle simulations. arXiv preprint arXiv:2501.01523 (2025)
16. Wang, Y., et al.: Guiding center orbit simulation of energetic particles in tokamak plasma (2022)
17. Koushki, E., et al.: Numerical development of drift in dilute plasmas to gravitational drift in nanocolloids. Int. J. Thermofluids 101266 (2025)
18. Benjamin, S., et al.: Distribution transforms for guiding center orbit coordinates in axisymmetric tokamak equilibria. Comput. Phys. Commun. **292**, 108893 (2023)
19. Fu, H., et al.: 9-pflops nonlinear earthquake simulation on sunway taihulight: enabling depiction of 18-hz and 8-meter scenarios. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–12 (2017)

20. Yang, C., et al.: 10 m-core scalable fully-implicit solver for nonhydrostatic atmospheric dynamics. In: SC 2016: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 57–68. IEEE (2016)
21. Liu, Y., et al.: Closing the "quantum supremacy" gap: achieving real-time simulation of a random quantum circuit using a new sunway supercomputer. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–12 (2021)

Accelerating Graph Sampling in GNN Systems Through Accuracy-Aware Data Reuse

Yuchun Wang, Nan Jiang, Qi Wu(✉), Hengshan Yue, and Xiaohui Wei

College of Computer Science and Technology, Jilin University, Changchun, China
{wyc23,jiangnan22}@mails.jlu.edu.cn, {wuqi,yuehs,weixh}@jlu.edu.cn

Abstract. Graph Neural Networks (GNNs) have become fundamental tools for processing and analyzing graph-structured data. However, when handling large-scale graph data, GNNs face exponential computational and memory challenges. While sampling-based training frameworks alleviate this issue, the sampling process itself becomes a performance bottleneck. Although existing optimizations like feature caching strategies and Unified Memory Access (UMA) kernels enhance memory efficiency, they overlook the redundant computation problem in sampling: the repeated sampling of identical nodes across different layers leads to resource wastage (consuming an average overhead of 15%). To address redundant computation, prior work proposes a complete cross-layer reuse strategy, eliminating redundancy by restricting each node to be sampled only once per mini-batch and indiscriminately reusing results. However, our work shows this indiscriminate reuse will destroy vertex neighbor diversity and reduce training stability to a certain extent, thus degrading the accuracy of GNN. (Training stability refers to the smooth convergence of the loss function during optimization, avoiding oscillations or divergence that impede model convergence.)

In this paper, we propose an **accuracy-aware sampling framework** that accelerates sampling while preserving model accuracy. Specifically, by dissecting two critical execution phases in GNN training—feature aggregation and backpropagation—we introduce dual-dimensional vertex selective metrics for accuracy-aware cross-layer sampling-result reuse: (1) Coordinately considering in-degree and neighbor label entropy to identify nodes with *high neighbor diversity*, whose reuse is blocked to preserve structural information, and (2) Considering gradient norm metric to identify nodes that significantly influence parameter update directions, whose reuse is blocked to ensure training stability. Merging them all, our approach achieves up to 2.74× speedup on DGL while keeping accuracy loss within 1.5%.

Keywords: Graph Neural Networks (GNN) · Sample-based GNN Training · Cross-layer Reuse

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 25–42, 2026.
https://doi.org/10.1007/978-981-95-8405-5_2

1 Introduction

Graph Neural Networks (GNNs), owing to their ability to jointly model graph structural semantics and node attributes, are a core methodology in domains such as social network analysis [1], recommendation systems [2], biomedical graph modeling [3], and knowledge graph reasoning [4]. However, when applied to real-world graphs containing hundreds of millions of nodes and tens of billions of edges [5,6], the multi-layer message-passing mechanism of GNNs leads to exponential growth in both computational cost and memory usage, posing severe scalability challenges for traditional full-graph training approaches [7–9].

To address this issue, sampling-based GNN training frameworks [10] are widely adopted. By constructing hierarchical subgraphs through neighborhood sampling, these methods constrain computation within a manageable scope [11]. A typical training workflow involves: (1) randomly selecting a batch of target nodes to form a mini-batch; (2) recursively sampling k-hop neighbors to form a subgraph; and (3) performing message passing and model optimization on this subgraph. However, as graph sizes continue to increase, the sampling phase itself accounts for a non-negligible portion of overall training time.

Several prior works seek to optimize the sampling process. For example, PaGraph [12] employs a degree-based feature caching strategy to preload frequently accessed node features into GPU memory, thereby reducing I/O overhead; PyTorch-Direct [13] proposes a Unified Memory Access (UMA)-based GPU communication kernel for feature retrieval, significantly improving transmission efficiency. Although these approaches achieve considerable memory performance gains, they often overlook redundant computations during sampling. In practice, the same node may be sampled multiple times across different layers, resulting in wasted resources. Figure 1b illustrates the proportion of redundant sampling relative to the total number of sampling operations during sampling-based GNN training across several real-world graph datasets. We show in Fig. 1b that such redundant sampling can consume up to 15% of total sampling effort on average.

To mitigate this, MariusGNN [14] introduces a complete cross-layer reuse strategy, restricting each node to be sampled at most once per mini-batch and globally reusing its sampling result across all layers. However, this coarse-grained mechanism indiscriminately reuses both highly sensitive and low-risk nodes, leading to: (1) destruction of neighbor diversity in sampled nodes, impairing GNNs' ability to dynamically capture local structural variations; (2) potential loss oscillations or even divergence, substantially impacting model accuracy and convergence speed.

To overcome these limitations, we propose an accuracy-aware sampling framework designed to identify and protect critical nodes whose sampling reuse would significantly impact model performance, thereby maintaining accuracy while improving efficiency. Through theoretical analysis and empirical validation, we discover that only a small subset of critical nodes significantly affects training effectiveness when their sampling results are reused. Building on this insight, we dissect two crucial execution phases in GNN training—feature aggregation and backpropagation. Based on this analysis, we introduce dual-

dimensional vertex selective metrics for accuracy-aware cross-layer sampling-result reuse: (1) **Neighbor diversity preservation:** An in-degree + neighbor label entropy composite metric identifies nodes with highly diverse neighbor information (excluded from reuse); (2) **Training stability assurance:** A gradient norm metric detects nodes significantly influencing parameter update directions (excluded from reuse). By precisely identifying and reusing only appropriate node sets, our method enhances training efficiency while preserving model accuracy. In summary, we make the following key contributions:

1. We analyze the feature aggregation phase and backpropagation process in GNNs, proposing an in-degree and neighbor label entropy composite metric from the neighbor diversity perspective and a gradient norm metric from the parameter update contribution perspective to filter out nodes significantly impacting model accuracy, which are excluded from reuse.
2. We design an accuracy-aware sampling framework (AARSampler) that accelerates training through cross-layer reuse of non-sensitive nodes' sampling results while confining accuracy loss within acceptable bounds.
3. Extensive evaluations of the proposed **AARSampler** under various configurations demonstrate that it achieves up to **2.74× speedup** over **DGL** [15], while keeping the accuracy loss within **1.5%**.

2 Background

2.1 GNN

Graph Neural Networks (GNNs) are an essential tool for processing and analyzing graph-structured data. A graph $G = (V, E)$ consists of a set of vertices V and edges E, which represent relationships between vertices. In addition, each node has a dense feature vector encoding its attributes. GNNs take advantage of both the graph's topological structure and the features of vertices to learn meaningful representations, enabling applications such as social network analysis, fraud detection, and molecular property prediction. GNNs operate through iterative representation learning, refining node representations by aggregating neighbor information at each layer. At the k-th layer, the feature vector of a vertex v, denoted as $h_v^{(k)}$, is updated based on information from its neighbors $N(v)$. This process can be mathematically described as follows:

$$a_v^{(k)} = \text{Aggregate}^{(k)} \left(\{ h_u^{(k-1)} \mid u \in N(v) \} \right)$$

$$h_v^{(k)} = \text{Update}^{(k)} \left(h_v^{(k-1)}, a_v^{(k)} \right)$$

Here, the Aggregate function collects and combines information from neighboring vertices, while the Update function incorporates the aggregated information to refine the current vertex's feature vector. These functions vary across GNN models, allowing for flexibility in adapting to diverse graph-based tasks.

2.2 Sample-Based GNN Training

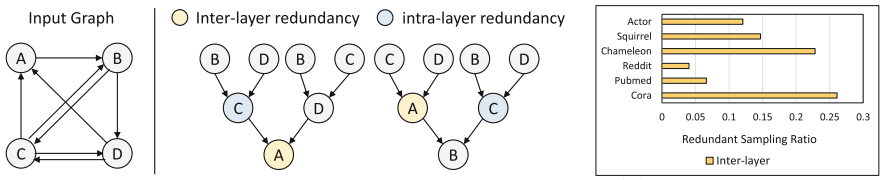
Traditional GNN training typically requires each node to aggregate the features of all its neighboring nodes. While this aggregation method effectively captures the rich structural information in the graph, as the graph size grows, especially in large-scale real-world graphs like social networks or knowledge graphs, the training process faces significant challenges. Specifically, the dramatic increase in the number of nodes, edges, and their high-dimensional features leads to prohibitive computational and memory costs, making traditional methods non-scalable. To address this issue, sampling-based GNN training methods are proposed. These methods reduce computational and memory overhead by sampling each node's neighboring nodes and constructing local subgraphs instead of using the full graph. Specifically, each iteration randomly selects a mini-batch of nodes and constructs layer-wise subgraphs via sampling. Then, during each layer of training, nodes update their representations by aggregating the features of their neighboring nodes. Afterward, the loss for the current mini-batch is computed, and the model parameters are updated through backpropagation. The training process optimizes the model through multiple iterations, with parameter updates occurring after each training step. Ultimately, training ends when predefined termination conditions are met.

2.3 Related Work and Motivation

Graph Neural Networks (GNNs) demonstrate great potential in processing and learning from large-scale graph data. However, as the graph size increases, traditional training methods face significant computational bottlenecks. Therefore, optimizing the GNN training process becomes a crucial research direction. Deep Graph Library (DGL) [15], a popular GNN framework, employs techniques such as batch processing, distributed training, and mixed-precision training to significantly enhance the overall efficiency of GNN training. Pytorch-direct [13] leverages unified virtual memory (UMA), allowing GPUs to directly access CPU memory without explicit data copying. Many studies also focus on optimizing the sampling phase of GNN training. For example, PaGraph [12] introduces a degree-guided feature caching strategy that preloads high-degree node features into GPU memory to reduce data transfer overhead.

Although significant progress is made in optimizing sampling-based GNN training, existing methods primarily focus on improving the utilization of computational and memory resources while often overlooking repeated computations during the sampling process. Figure 1a illustrates the phenomenon of redundant sampling in multi-layer GNN training. As shown in Fig. 1a, in multi-layer GNNs, the same node may be repeatedly sampled by nodes from different layers, resulting in multiple sampling computations across layers—this phenomenon is referred to as **inter-layer** redundancy. For example, node A in the Fig. 1a is sampled and computed in two separate layers. Similarly, within the same layer, different nodes may sample the same neighbor nodes, leading to repeated computations for those neighbors. This is known as **intra-layer** redundancy. For

instance, node C in the Fig. 1a is sampled and computed twice within the same layer.

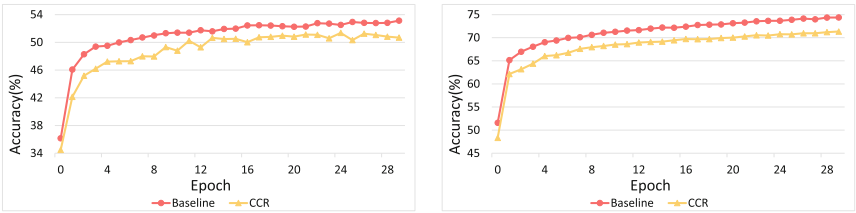


(a) Visualization Analysis (b) Quantitative Analysis

Fig. 1. Visualization and Quantitative Analysis of Redundant Sampling in Sampling-Based GNN Training.

Figure 1b presents that the ratio of inter-layer redundant sampling ranges from 4% to 26%, indicating a significant amount of duplicated computation and considerable room for optimization during the sampling process. Although DGL [15] globally deduplicates the set of source nodes sampled by all target nodes within the same layer—ensuring that each neighbor is included only once per layer and thereby completely eliminating intra-layer redundancy—the issue of inter-layer redundancy remains unaddressed. Several studies attempt to address this issue. For example, MariusGNN [14] adopts a complete cross-layer reuse technique (CCR), which directly reuses sampled neighbors from earlier layers throughout the entire mini-batch, thereby avoiding redundant sampling and computation of the same nodes. This approach reduces computational overhead and memory access costs, leading to improved sampling efficiency. Figure 2 presents the accuracy comparison between DGL and CCR when training the Flickr dataset using the GCN model and the OGBN-Arxiv dataset using the GraphSAGE model. As shown in Fig. 2, the CCR strategy leads to a decrease in GNN accuracy. While this method reduces computational and memory overhead while enhancing sampling efficiency, its coarse-grained indiscriminate reuse mechanism compromises neighbor diversity. The equivalent reuse of both highly sensitive and low-risk nodes damages structural heterogeneity in sampled neighborhoods, impairing GNNs' ability to capture local dynamic variations and resulting in model accuracy degradation. Furthermore, indiscriminate reuse may induce loss oscillations or even divergence, thereby substantially reducing training accuracy and convergence speed.

Motivation. As shown in Fig. 1b, the proportion of redundant computation is substantial, indicating significant optimization potential. Although complete reuse strategies can reduce redundant computations, they often lead to accuracy degradation. Therefore, it is crucial to carefully select the nodes that are suitable for reuse. Reducing the computational cost caused by redundant sampling while maintaining accuracy is essential. We observe that certain nodes play a more critical role in influencing the training outcomes of GNNs. To this end, in Sect. 3, we dissect two critical GNN execution phases—feature aggregation and backpropagation—and propose key node screening metrics from dual



(a) Flickr (b) Ogbn-arxiv

Fig. 2. Impact of Complete Reuse Strategy on GNN Accuracy.

dimensions: neighbor diversity preservation and parameter update contribution. Building on these foundations, in Sect. 4, we design the accuracy-aware sampling framework AARSampler.

3 Accuracy-Aware Cross-Layer Reuse Strategy

This section primarily introduces accuracy-aware selective cross-layer reuse techniques, focusing on how to select appropriate nodes for cross-layer reuse during GNN training. Specifically, this section presents several selection criteria for evaluating the impact of nodes on training accuracy. Using these criteria, it is possible to identify which nodes have a greater contribution to the final training outcome, thus optimizing the cross-layer reuse strategy to improve training efficiency while maximizing accuracy retention.

3.1 Feature Aggregation-Based Reuse Selection Criteria

In the forward propagation of Graph Neural Networks (GNNs), nodes update their own features by aggregating information from their neighbors. In sampling-based GNN training, ensuring the diversity of sampled neighbors for each node is crucial. Blindly reusing sampling results may impair the fairness and randomness of sampling, thus undermining neighbor diversity and ultimately degrading the model's accuracy. Figure 3 illustrates the variation in neighbor diversity under different node degrees and neighbor label entropies.

Two key conditions that undermine neighbor diversity are identified as follows:

First, a high in-degree is a fundamental prerequisite for ensuring neighbor diversity. When a node's in-degree is significantly greater than the sampling fanout (the number of neighbors to be sampled for each vertex), the potential space of sampling combinations increases dramatically (e.g., given a fanout of 2, a node with 10 neighbors generates $C(10, 2) = 45$ distinct combinations). This structural condition lays the foundation for diversified information aggregation. As shown in Fig. 3a, when the in-degree equals the fanout (e.g., only 2 neighbors), the sampling result is unique, and reuse has no effect. However, for a high-degree node with 10 neighbors, as illustrated in Fig. 3b, reusing a fixed sampling

result compresses the original rich combination space into a single deterministic pattern. This reduces sampling randomness, compromises neighbor diversity, and negatively impacts model accuracy.

(a) Low-degree Node (b) High-degree Node with Low Label Entropy (c) High-degree Node with High Label Entropy

Fig. 3. Illustration of Sampling Combination Diversity for Different Nodes.

Second, the diversity of neighbor labels further determines how significantly reusing high-degree node samples affects model performance. Even for nodes with the same high in-degree, their impact on the model varies based on the label diversity of their neighbors. In real-world scenarios, some neighbors may belong to the same class (i.e., with identical labels and similar features), and can thus be approximated as homogeneous. If the label distribution among neighbors is skewed (e.g., in Fig. 3b), only three effective combinations exist from a label perspective, and the damage to diversity caused by reuse is limited. In contrast, as shown in Fig. 3c, a node with balanced label distribution among its neighbors has up to 12 effective label combinations. Reusing samples for such high-degree nodes with diverse neighbor labels significantly reduces sampling randomness, disrupts information propagation paths, and causes more pronounced degradation in the final predictive accuracy of the GNN model. Therefore, we introduce **label entropy** as an additional metric to quantify the diversity of a node's neighbor types. This metric helps assess how much reusing the sampling results of a node would disrupt the neighbor diversity. The formula for label entropy is as follows:

$$H(v) = -\sum_{i=1}^{K} p_i \log_2 p_i$$

where $H(v)$ is the label entropy of node v; p_i is the probability that the neighbors of node v belong to the i-th label (i.e., the proportion of neighbors with the i-th label); and K is the total number of label categories.

Nodes with higher label entropy generally have more diverse neighbor information, while nodes with lower label entropy have more homogeneous neighbor information. The higher the label entropy, the more diverse the node's neighbor information.

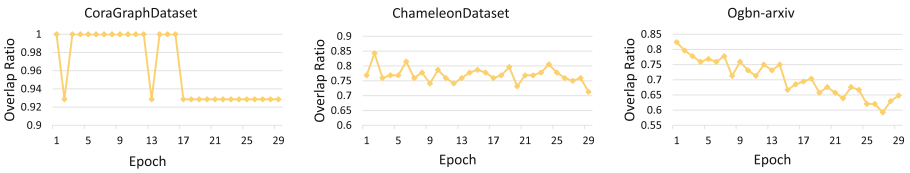
Heuristic #1: For nodes with high in-degrees and high label entropy, due to the diversity of their neighbor information, directly reusing their sampling results may not capture the full range of their diversified information, leading to a loss in accuracy. Hence, it is recommended to avoid reusing sampling results for nodes with high in-degrees and high label entropy.

3.2 Parameter Update Contribution-Based Reuse Selection Criteria

In the backpropagation of GNNs, the process begins by computing the loss based on the output. Gradients of node embeddings are then calculated top-down along the graph structure, followed by aggregation to obtain weight gradients for each layer. Finally, weights are updated via optimizers (e.g., SGD or Adam) to progressively minimize the loss.

Nodes exhibiting high gradient norms demonstrate significant sensitivity to the loss function, where neighborhood variations can readily trigger gradient instability. Reusing sampling results for such nodes amplifies gradient estimation errors, consequently inducing loss oscillations and convergence delays during training, which substantially compromises model accuracy and convergence speed.

A key challenge arises from the need to dynamically identify these nodes that exhibit high gradient norms, as each epoch requires recomputing losses and executing backpropagation. Fortunately, we observe that the set of high-gradient-norm nodes exhibits remarkable stability across training epochs: the Fig. 4 shows the overlap ratio between the high-gradient-norm node set in each epoch and the high-gradient-norm node set in the first epoch. Each epoch shares, on average, over 70% of high gradient norm nodes with the first epoch. These vertices typically include Hard-to-classify vertices (e.g., decision boundary vertices) and Structurally distinctive vertices (hub vertices or isolated vertices).



(a) Cora (b) Chameleon (c) Ogbn-arxiv

Fig. 4. Stability of High-Gradient Node Sets Across Epochs.

Based on this observation, we efficiently identify critical nodes by computing and dynamically updating a protection set after the first epoch, thereby systematically preventing sampling reuse for high-gradient nodes. We adopt the **Frobenius norm of a node's gradient** as an indicator of its influence on training. The Frobenius norm incorporates gradient information across all feature dimensions and sampling instances, providing a comprehensive measure of a node's overall impact on parameter updates. It is defined as follows:

For a node's gradient matrix $G \in \mathbb{R}^{d \times m}$, its Frobenius norm is

$$\|G\|_F = \sqrt{\sum_{i=1}^{d} \sum_{j=1}^{m} g_{ij}^2}$$

where d is the embedding dimension (feature dimension), m is the number of neighbor sampling instances, and g_{ij} is the gradient of the i-th feature dimension at the j-th sampling.

Heuristic #2: Nodes with high Frobenius norms have a substantial influence on the direction of parameter updates. Reusing their sampling results may introduce biased gradient estimates, slow convergence, and degrade model performance. Therefore, during training, special caution should be taken when handling these nodes' samples to avoid excessive reuse and to ensure model stability and performance.

4 Implementation

This section proposes an **Accuracy-Aware Cross-layer Reuse strategy based on dynamic node importance identification**, aiming to reduce redundant sampling computation overhead while preserving model accuracy.

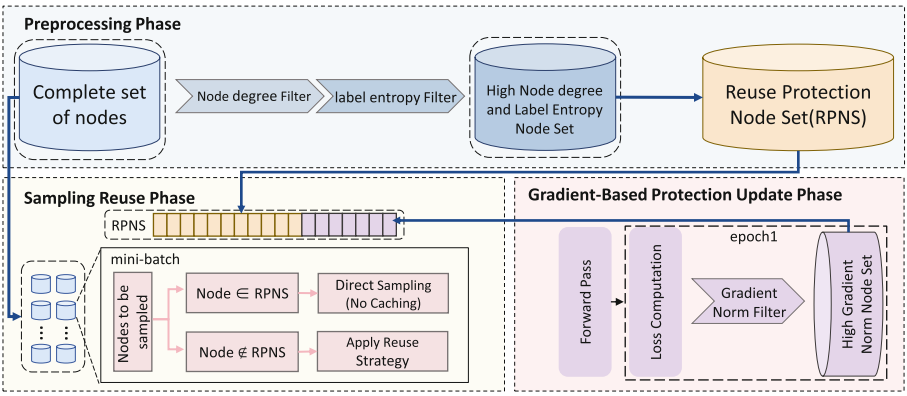


Fig. 5. The design of Selective Cross-layer Reuse Strategy.

4.1 Design Overview

The overall framework of the strategy is shown in Fig. 5 and consists of three components: **(1) Preprocessing Phase:** Based on Heuristic #1, high-importance nodes are identified via multi-dimensional feature analysis to construct the initial "Reuse Protection Node Set (RPNS)," which marks critical nodes unsuitable for cross-layer reuse. **(2) Gradient-Based Protection Update Phase:** After the first training epoch, Heuristic #2 is applied to dynamically expand the RPNS by incorporating node gradient information. **(3) Sampling Reuse Phase:** Within each mini-batch, a dynamically updated reuse list is maintained to enable efficient reuse of sampled neighborhoods.

These techniques effectively control the reuse of sampling information across GNN layers while maintaining training accuracy and reducing computational overhead.

4.2 Design Details

(1) Preprocessing Phase The preprocessing phase constructs the initial **Reuse Protection Node Set (RPNS)** through single-stage screening. The specific process is as follows:

Structural Feature Joint Screening (Heuristic #1): We first identify the top 20% of nodes by in-degree as high-degree nodes, placing them into set $\mathcal{S}_{1a}$.

Next, we compute neighbor label entropy for nodes in $\mathcal{S}_{1a}$, then select the top 40% with the highest entropy values into set $\mathcal{S}_1$, identifying nodes exhibiting high neighbor diversity.

Protection Set Generation: We take $\mathcal{S}_1$ as the initial RPNS. This RPNS is used to constrain which nodes' sampling results should not be reused across GNN layers during training.

(2) Gradient-Based Protection Update Phase (Heuristic #2) After the first training epoch, we compute the gradient norm of each node and select the top 10% highest-norm nodes to form the set $\mathcal{S}_{grad}$. We union $\mathcal{S}_{grad}$ with the current RPNS to produce the updated RPNS. This dynamic update ensures that nodes with significant contributions to model updates remain protected from sampling reuse, further preserving accuracy while reducing redundant computation.

(3) Sampling Reuse Phase To achieve efficient and controlled sampling reuse, we maintain a dynamic *Reuse List* during each mini-batch:

Reuse List Maintenance Mechanism: In each mini-batch training iteration, the system caches sampled neighborhoods of non-protected nodes in the Reuse List to avoid redundant computation. The implementation proceeds as follows:

(a) Pre-Sampling Check

Before performing any sampling, the target node is first checked against the *RPNS*:

If the node $\in$ RPNS: Perform the sampling operation as usual, but **do not** cache the result to preserve the dynamic behavior of critical nodes.

If the node $\notin$ RPNS: Proceed to the reuse decision flow.

(b) Reuse Decision Flow

For nodes outside the RPNS, the system then queries the *Reuse List*:

Hit in Reuse List: Immediately reuse the cached neighbor set and skip the sampling computation.

Miss in Reuse List: Perform neighbor sampling as usual, then insert the newly obtained neighbor set into the Reuse List for future reuse.

5 Evaluation

5.1 Experimental Setup

1. **Environments:** We conduct our experiments on a server running Ubuntu 24.04 LTS with an Intel Xeon Gold 6133 CPU (40 cores), 62 GB main memory, and two NVIDIA GeForce RTX 3090 GPUs (each with 24 GB memory). The software environment includes GCC 13.3.0, CUDA 12.4, PyTorch 2.1.0, and DGL v2.4.0+cu121.
2. **Datasets:** In our experimental evaluation, we select four publicly available graph datasets: two large-scale social networks—Reddit [16] and Tolokers—as well as the Flickr dataset and Ogbn-arxiv [17] from the Open Graph Benchmark (OGB). For the OGB dataset, we use the official training split provided by OGB; for the remaining datasets, we follow the data partitions provided by DGL [15].
3. **Parameter Settings:** In our experiments, we employ multiple widely used parameter configurations for comparative evaluation [16,18]. Specifically, the batch size (i.e., the number of sampled nodes per iteration) is set to 1024, 2048, 4096, and 8192; the graph sampling fanout tuples are set to (10, 10, 10) and (15, 15, 15). By combining different batch sizes with sampling densities, we can comprehensively assess our method's training efficiency and model performance.
4. **Models:** To evaluate our method, we employ three of the most common and representative GNN architectures: GCN [19], GraphSAGE [20], and GIN [21]. For all models, we use a three-layer architecture with a hidden dimension of 128. (For the GIN model, however, we set the hidden dimension to 32 to mitigate overfitting, as deeper hidden layers in GIN are more prone to this issue.)
5. **Baselines:** We compare against two baseline implementations:
   (a) **DGL** [15], a state-of-the-art GNN framework;
   (b) **MariusGNN** [14], which employs complete cross-layer reuse in its sampling process to accelerate sampling-based GNN training. We only compare the sampling optimization strategy in MariusGNN [14].

   In our evaluation, we measure both the performance of their sampling modules and the final GNN training accuracy.

5.2 Parameter Selection

To determine the optimal protection ratio combination, this study systematically evaluates the impact of label entropy selection ratio (α), degree selection ratio (β), and gradient norm selection ratio (γ) on the Accuracy Recovery Ratio.

1. Synergistic Effect of Label Entropy and Degree Ratios

As shown in Fig. 6a, which presents the accuracy recovery ratio of GNN training through selective reuse of node sets filtered by different combinations of label entropy and degree selection ratios, it is observed that when selecting the top 20% by degree and top 40% by label entropy, the accuracy recovery ratio exceeds 50%

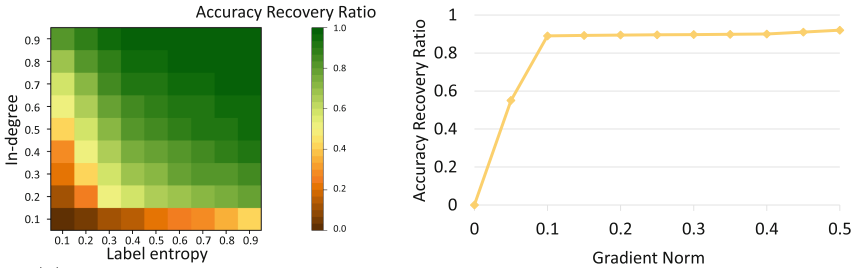
while protecting only 8% of nodes. To balance protection efficiency and accuracy benefits, we select the combination of $\beta = 20\%$ and $\alpha = 40\%$.

2. Independent Effect of Gradient Norm Ratio

As shown in Fig. 6b, which displays the accuracy recovery ratio using node sets filtered by different gradient norm selection ratios, the gradient norm threshold γ exhibits a critical point effect on accuracy recovery. Within the range of $\gamma \leq 10\%$, the recovery ratio increases rapidly, while when $\gamma > 10\%$, the growth rate decreases significantly. Thus, $\gamma = 10\%$ is determined as the optimal threshold, effectively capturing most critical nodes while preventing excessive protection.

3. Parameter Configuration Selection

The optimal parameter combination established through the above experiments ($\alpha = 40\%$, $\beta = 20\%$, $\gamma = 10\%$) is adopted as the default configuration for the following experiments.



(a) Label Entropy and In-Degree (b) Gradient Norm

Fig. 6. Effects of Parameter Selection on Accuracy Recovery Ratio.

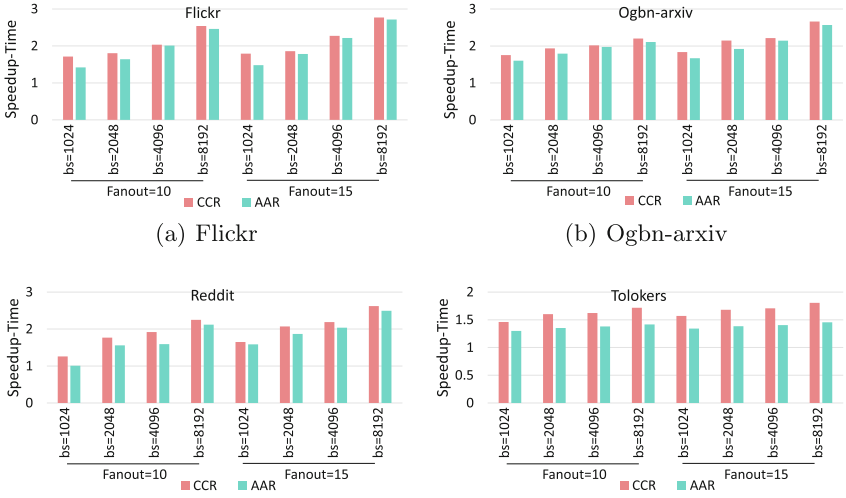
5.3 Performance Results

We compare our proposed accuracy-aware cross-layer reuse strategy (AAR) against the native DGL [15] neighbor sampler and the complete cross-layer reuse sampler (CCR) from MariusGNN [14], evaluating both sampling time overhead and training accuracy.

1. **Sampling Performance Comparison**
   - **Sampling Time Speedup:** Figure 7 shows the sampling time speedup of our method (Accuracy Aware Reuse, AAR) and the cross-layer reuse sampler from MariusGNN [14] (Complete Cross-layer Reuse, CCR) compared to the native DGL [15] sampler under various datasets and parameter settings. The results indicate that AAR achieves an average sampling time speedup of **1.78**× (with a maximum of **2.74**×), while MariusGNN [14] achieves an average speedup of **1.95**× (up to **2.76**×).

– **Sampling Workload Speedup:** Figure 8 compares the actual total number of sampling operations (i.e., the number of effective neighbor samplings) for AAR and MariusGNN [14] against the DGL [15] native sampler under different datasets and parameter settings. AAR achieves an average sampling workload speedup of **1.80**× (up to **2.90**×), whereas MariusGNN [14] reaches an average of **1.97**× (with a maximum of **2.94**×).
Although AAR's sampling time speedup is slightly lower than that of MariusGNN [14], it still achieves a **63**% reduction in sampling time compared to native DGL [15], significantly improving sampling efficiency. Furthermore, as the batch size and fanout values increase, the redundant computation problem becomes more severe, thereby leading to a more pronounced acceleration effect.

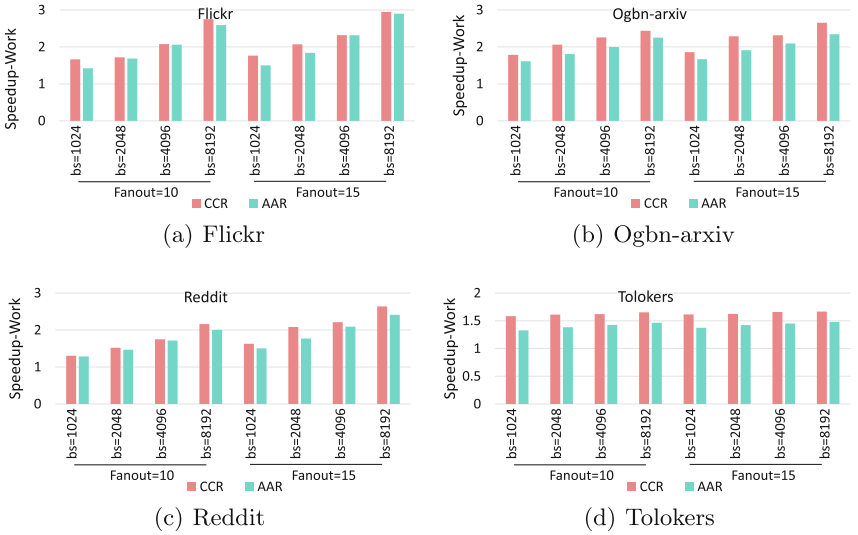
2. **Training Accuracy Comparison** Figures 9, 10, 11 present the final training accuracy achieved by our method, the native DGL [15] sampler, and the MariusGNN [14] cross-layer reuse sampler across different GNN models. To provide a clearer visual comparison of accuracy differences, the plots only show results from epochÂă5 onward. As shown, MariusGNN [14]'s blanket reuse of all nodes incurs an average accuracy drop of 3.84% (peaking at 4.83%). In contrast, our approach strictly bounds accuracy loss to less than 1.5% ($1.01 \pm 0.32\%$) while still maintaining substantial speedup.
In summary, our method achieves efficient acceleration of sampling-based GNN training by significantly reducing sampling time and computational overhead, all while keeping accuracy loss within acceptable limits.



(a) Flickr (b) Ogbn-arxiv

(c) Reddit (d) Tolokers

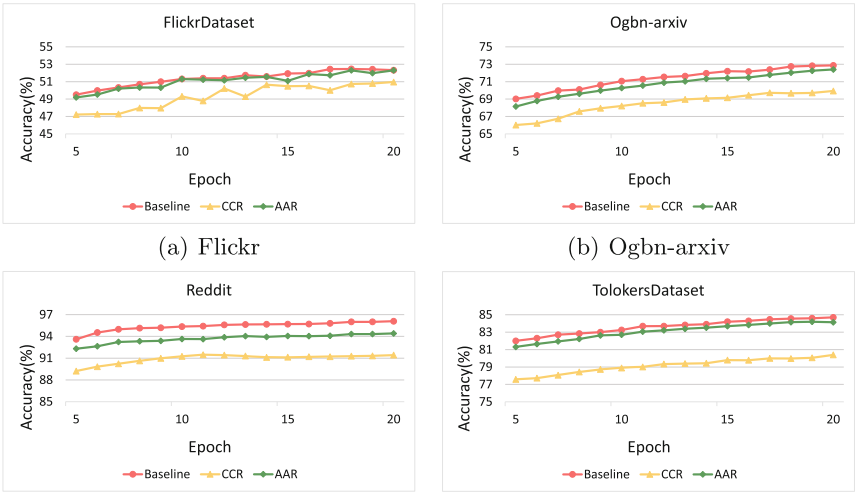
Fig. 7. Time speedup of the accuracy-aware selective cross-layer reuse strategy over DGL.



(a) Flickr (b) Ogbn-arxiv

(c) Reddit (d) Tolokers

Fig. 8. Workload speedup of the accuracy-aware selective cross-layer reuse strategy over DGL.



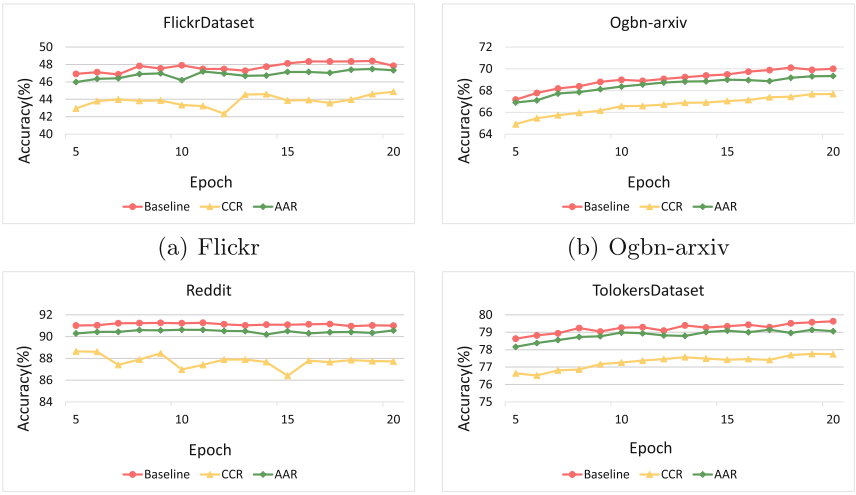
(a) Flickr (b) Ogbn-arxiv

(c) Reddit (d) Tolokers

Fig. 9. Training Accuracy Comparison in the GraphSAGE Model.

5.4 Effectiveness of the Accuracy-Aware Selection Strategy

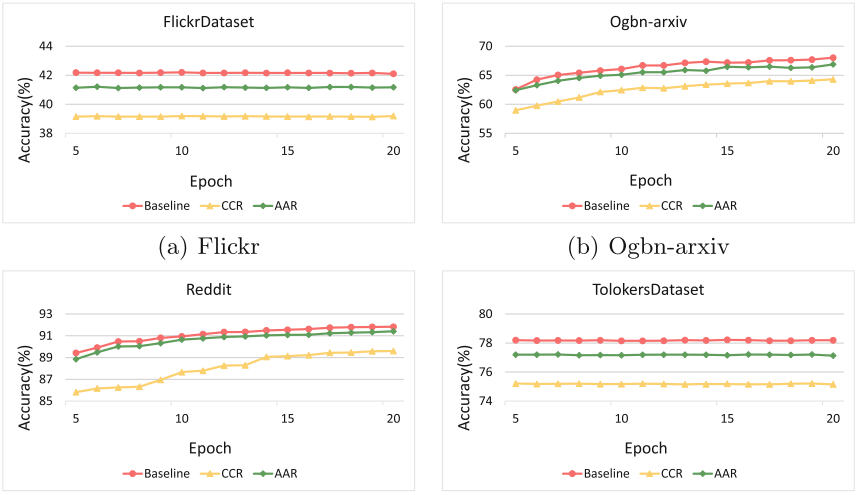
To evaluate the effectiveness of our accuracy-aware selection strategy, we conduct experiments on the Tolokers dataset using three different selection criteria and



(a) Flickr (b) Ogbn-arxiv

(c) Reddit (d) Tolokers

Fig. 10. Training Accuracy Comparison in the GCN Model.



(a) Flickr (b) Ogbn-arxiv

(c) Reddit (d) Tolokers

Fig. 11. Training Accuracy Comparison in the GIN Model.

observe their impact on GNN training accuracy. The results, shown in Fig. 12, are as follows:

– **Feature Aggregation–Based Criterion:** Relative to complete reuse, this criterion yields an average accuracy recovery of 58.6%.

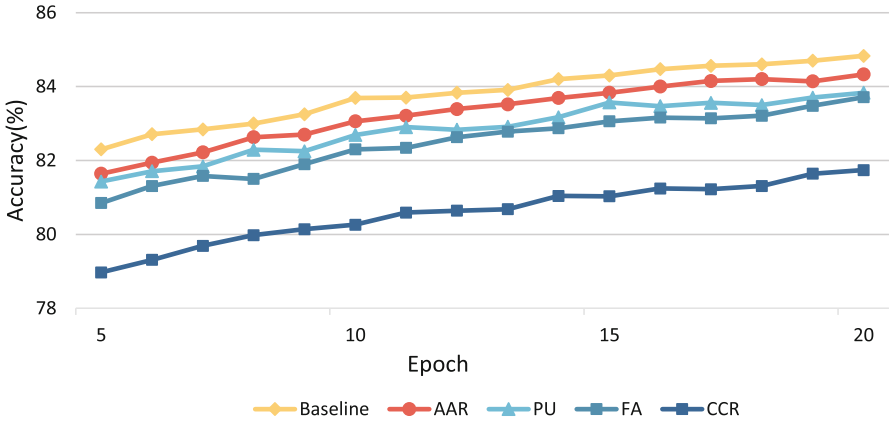


Fig. 12. Accuracy recovery under different reuse selection strategies on the Tolokers dataset.

– **Parameter Update Contribution–Based Criterion:** Compared to complete reuse, this criterion attains an average accuracy recovery of 70.1%.

These findings demonstrate that our accuracy-aware selection strategy substantially mitigates accuracy degradation. In particular, incorporating parameter contribution information enables the model to regain accuracy while still benefiting from efficient sampling.

6 Conclusion

In this paper, we propose AARSampler, an accuracy-aware sampling framework. Specifically, by analyzing two key stages in GNN training—feature aggregation and backpropagation—we introduce a two-dimensional vertex selection metric for accuracy-aware cross-layer reuse of sampling results. By reusing the sampling results of insensitive nodes across layers, the sampling process is accelerated while keeping the training accuracy loss within a controllable range. We systematically evaluate AARSampler on four graph datasets combined with three different GNN models under various parameter settings. Experimental results show that, compared with DGL, AARSampler achieves an average 1.78× speedup in the sampling phase while controlling the training accuracy loss within 1.5%, significantly improving training efficiency.

Acknowledgment. This work is supported by the National Natural Science Foundation of China (NSFC) under Grants No. 62272190 and No. 62302190, and the National Key Research and Development Program of China under Grant No. 2023YFB4502304.

References

1. Zhang, M., Chen, Y.: Link prediction based on graph neural networks. In: Advances in Neural Information Processing Systems, vol. 31 (2018)
2. Fan, W., et al.: Graph neural networks for social recommendation. In: The World Wide Web Conference, pp. 417–426 (2019)
3. Xia, T., Ku, W.-S.: Geometric graph representation learning on protein structure prediction. In: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, pp. 1873–1883 (2021)
4. Park, N., Kan, A., Dong, X.L., Zhao, T., Faloutsos, C.: Estimating node importance in knowledge graphs using graph neural networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 596–606 (2019)
5. SGandhi, S., Iyer, A.P.: P3: distributed deep graph learning at scale. In: 15th {USENIX} Symposium on Operating Systems Design and Implementation ({*OSDI*}, vol. 21), pp. 551–568 (2021)
6. Thorpe, J., et al.: Dorylus: affordable, scalable, and accurate {GNN} training with distributed {CPU} servers and serverless threads. In: 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI 21), pp. 495–514 (2021)
7. Chen, J., Zhu, J., Song, L.: Stochastic training of graph convolutional networks with variance reduction. arXiv preprint arXiv:1710.10568 (2017)
8. Huang, W., Zhang, T., Rong, Y., Huang, J.: Adaptive sampling towards fast graph representation learning. In: Advances in Neural Information Processing Systems, vol. 31 (2018)
9. Zeng, H., Zhou, H., Srivastava, A., Kannan, R., Prasanna, V.: Graphsaint: graph sampling based inductive learning method. arXiv preprint arXiv:1907.04931 (2019)
10. Liu, X., Yan, M., Deng, L., Li, G., Ye, X., Fan, D.: Sampling methods for efficient training of graph convolutional networks: a survey. IEEE/CAA J. Automatica Sinica **9**(2), 205–234 (2021)
11. Zhang, Z., Cui, P., Zhu, W.: Deep learning on graphs: a survey. IEEE Trans. Knowl. Data Eng. **34**(1), 249–270 (2020)
12. Lin, Z., Li, C., Miao, Y., Liu, Y., Xu, Y.: Pagraph: scaling gnn training on large graphs via computation-aware caching. In: Proceedings of the 11th ACM Symposium on Cloud Computing, pp. 401–415 (2020)
13. Min, S.W., et al.: Large graph convolutional network training with gpu-oriented data communication architecture. arXiv preprint arXiv:2103.03330 (2021)
14. Waleffe, R., Mohoney, J., Rekatsinas, T., Venkataraman, S.: Mariusgnn: resource-efficient out-of-core training of graph neural networks. In: Proceedings of the Eighteenth European Conference on Computer Systems, pp. 144–161 (2023)
15. Wang, M., et al.: Deep graph library: a graph-centric, highly-performant package for graph neural networks. arXiv preprint arXiv:1909.01315 (2019)
16. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. In: Advances in Neural Information Processing Systems, vol. 30 (2017)
17. Weihua, H., et al.: Open graph benchmark: datasets for machine learning on graphs. Adv. Neural. Inf. Process. Syst. **33**, 22118–22133 (2020)
18. Yuan, H., et al.: Comprehensive evaluation of gnn training systems: a data management perspective. arXiv preprint arXiv:2311.13279 (2023)
19. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907 (2016)

20. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., et al.: Graph attention networks. Stat **1050**(20), 10–48550 (2017)
21. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? arXiv preprint arXiv:1810.00826 (2018)

Scalable Distributed Index Construction for Large-Scale Graph-Based ANNS

Xiangyu Liu[1], Meiling Wang[2], and Yiming Zhang[1(✉)]

[1] NICE Lab, Xiamen University, Xiamen, China
xiangyul@stu.xmu.edu.cn, sdiris@gmail.com
[2] Huawei Theory Lab, Shenzhen, China
wangmeiling17@huawei.com

Abstract. In high-dimensional vector spaces, approximate nearest neighbor search (ANNS) is essential for modern database and AI systems. Graph-based ANNS has become increasingly popular owing to its balance between search efficiency and accuracy. However, building indexes for extremely large graphs remains challenging: single-node index construction can take a long time (days or even weeks) due to limited memory, computation, and I/O resources, while distributed index construction suffers from workload imbalance, network and I/O bottlenecks during index merging, and poor hardware utilization. This paper presents SD-GAC, a scalable distributed index construction framework for large-scale graph-based ANNS. SD-GAC introduces a fine-grained partitioning and packing scheme to ensure balanced distribution of index construction workloads, a communication-efficient merging algorithm that prefetches and batches all required neighbor data in a single pass, and a NUMA-aware construction algorithm that maximizes local memory and cache utilization. Evaluation shows that SD-GAC significantly reduces index construction time and achieves near-linear speedup with increasing computational resources.

Keywords: Approximate Nearest Neighbor Search (ANNS) · Distributed Graph Index Construction · Scalability

1 Introduction

Today deep learning models can embed almost all types of data, including speech, vision, and text information, into multi-dimensional vectors with tens or even hundreds of dimensions. Such vectors are critical for complex semantic understanding tasks [19,22]. To enable effective vector analysis, vector nearest neighbor search (NNS) systems have become critical system components for an increasing number of online services like search and recommendation. To satisfy the strict query latency requirement for these online services, vector search systems often resort to approximate nearest neighbor search (ANNS), to locate as many correct results as possible (i.e., query accuracy).

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 43–58, 2026.
https://doi.org/10.1007/978-981-95-8405-5_3

ANNS in highdimensional spaces refers to find top-k vectors most similar to a given query vector. It has a wide range of applications in many fields, including data mining [27], search engines [33], and AI-driven recommendation systems [12,35]. Specifically, fueled by the recent prosperity of Large Language Models (LLMs) [3,15,31,34], ANNS systems have become a crucial component of modern AI infrastructure. Current studies explore four index types for ANNS: Tree-based [20,28], Hash-based [10,13,26], Quantization-based [9,11], and Graph-based methods [7,25]. Among these, graph-based algorithms have demonstrated superior performance, establishing themselves as the industry standard [5,6] and attracting substantial academic interest [8,23]. Although numerous studies have explored optimizations for search performance, research on improving index construction efficiency remains limited, which is increasingly critical in real-world applications. However, single-node solutions can take a long time (days or even weeks) [32] due to limited memory, computation power, and I/O resources. For instance, building an graph index on tens of millions of vectors typically requires about 10 h [5,17], billion-scale datasets may extend construction time to five days, even with relaxed construction parameters [25]. Thus, construction efficiency has become a bottleneck in modern vector databases [21], as it directly impacts the ability to rapidly release new services with up-to-date data and high search performance [24].

To overcome these limitations, distributed index construction has emerged as a promising solution. By partitioning data and harnessing parallel computations across multiple computing nodes, distributed indexing can dramatically reduce construction time, enabling efficient and timely handling of billion-scale datasets. However, distributed index construction introduces several unique and significant challenges. Firstly, workload imbalance across nodes can lead to inefficient resource usage, where some servers become idle prematurely, while others continue processing overloaded tasks. Secondly, inter-node data transfers during index merging phases can create substantial network and I/O bottlenecks, offsetting the benefits gained from parallel execution. Finally, modern server hardware commonly employs Non-Uniform Memory Access (NUMA) architectures; ignoring NUMA-awareness in distributed systems may severely degrade performance by causing excessive remote memory accesses and suboptimal cache utilization.

In order to address the challenges mentioned above, we propose SDGAC, a **S**calable **D**istributed **G**raph-based **A**NNS **C**onstruction Framework for extremely large datasets. The contributions of this paper can be summarized as follows:

- We design a fine-grained partitioning and packing strategy to achieve near-optimal workload balance across distributed nodes, significantly reducing idle time and resource underutilization.
- We propose a NUMA-aware index construction algorithm that co-locates computation and memory access, maximizing memory bandwidth and minimizing remote memory latency.

– We introduce an efficient cross-node graph merging mechanism that minimizes network and I/O bottlenecks by leveraging precomputed mappings and bulk communication.
– We provide a comprehensive experimental evaluation on billion-scale datasets, demonstrating substantial improvements in build time and scalability.

2 Background and Motivation

2.1 Approximate Nearest Neighbor Search

Nearest Neighbor Search (NNS) involves finding the closest data point to a given query point within a dataset. Formally, consider a dataset D consisting of n points, each represented as a d-dimensional vector $\{x_1, x_2, ..., x_d\}$. Given a query point q, also represented by a d-dimensional vector $\{q_1, q_2, ..., q_d\}$, and target k, the goal is to find k points $\{p_1, p_2, ..., p_k\} \in D$ such that the distance $d(p_i, q)$ is minimized. The choice of distance metric, such as Euclidean or Hamming distance, typically depends on the embedding model used to generate the vectors.

However, the NNS problem becomes computationally challenging in high-dimensional vector spaces due to the curse of dimensionality [14], which makes exhaustive linear searches impractical on large-scale datasets [30]. To address this issue, Approximate Nearest Neighbor Search (ANNS) algorithms have been developed, where the goal is to retrieve k neighbors which are close to being optimal. The accuracy of the ANNS is typically measured as recall. Let the set of the exact k-nearest vectors be X, and the vectors returned by an ANNS algorithm be Y, where $|X| = |Y| = k$. Then recall@k is defined as $\frac{|X \cap Y|}{k}$, meaning the proportion of exact k-nearest vectors the algorithm finds.

To meet diverse accuracy and performance requirements, several ANNS algorithms such as tree-structure based [20,28], hashing based [10,13,26] and quantization based approaches [9,11] have been proposed over the past decades. Among the various techniques, ANNS algorithms using graphs [7,18] are considered as the most promising solution, with great potential. In graph-based ANNS, the vectors in the dataset are organized as a directed graph where each node of the graph represents a vector, and the edges of the graph represent the neighboring relationship of the vectors. This structure allows graph-based approaches to better capture neighbor relationships and traverse fewer points during the search, leading to higher efficiency and accuracy compared to other methods.

2.2 On-Disk Graph-Based Index Construction

In super large-scale vector search scenarios, the memory cost can become extremely expensive. For example, many commercial recommendation systems such as Alibaba [7] usually require TB-scale memory space to accommodate billion-scale vectors. To support billion-scale ANNS with graph-based index, SSDs are used because of their cost-effectiveness [25,29], with the majority of

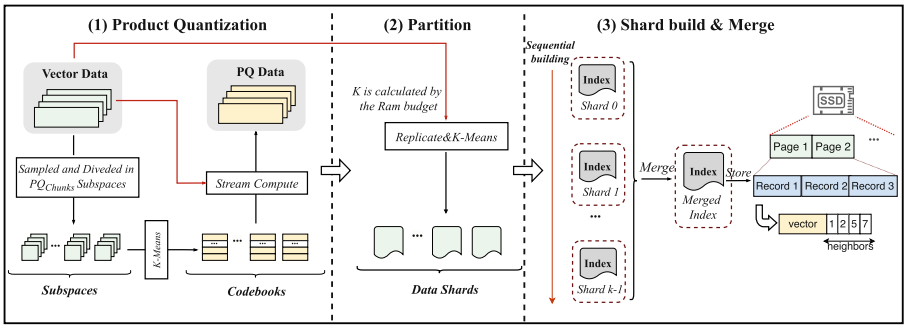


Fig. 1. Workflow of On-disk Graph-based Index Construction.

vector indices stored on SSDs and a moderately-sized DRAM used as working memory.

Figure 1 shows the construction process of the disk-based graph index, which consists of three phases: (1) To reduce the number of fetching full-precision vectors, on-disk graph-based ANNS systems always maintain a copy of in-memory compressed vectors using product quantization (PQ) [16], and use PQ distance as an approximation when inserting neighbors to the candidate set during search. It first samples the dataset and divides the sampled set uniformly into sub-spaces, with each vector containing some of the sub-vectors. Then, these sub-vectors are clustered by using k-means to generate a codebook, which contains a set of centroids of all clusters. The codebook allows each sub-vector to be approximated by its nearest centroid. Once all codebooks are generated, the full dataset is streamed and compressed using the codebooks. (2) As the dataset can reach billion-scale, it cannot be fully loaded into memory at once for index construction, it is common practice to partition the dataset using k-means clustering. The dataset is divided into k clusters (i.e., shards), where k is determined based on the ram budget, ensuring that the construction of each shard does not exceed memory constraints. Each data point is then assigned to its l closest cluster centers (typically $l = 2$ suffices). (3) After the dataset is partitioned, each shard is built sequentially. Then, all the shard indexes are merged into a single index by taking the union of all the edges in each shard's graph. The final index is stored as multiple records on SSD pages, each of which represents one node of the graph. A record includes the full-precision vector and a neighbor list which consists of the IDs of neighbors.

2.3 Distributed Systems Demands and Challenges

As real-world datasets continue to expand, single-node solutions are increasingly inadequate. Business applications such as e-commerce and online advertising routinely involve billions or even trillions of vectors, making the memory capacity, computational power, and I/O bandwidth of a single node significant bottlenecks. As shown in Table 1, global rebuilds are both resource-hungry and time-

consuming. For example, rebuilding a one billion vector index [25] for DiskANN, a recent disk-based system, needs 1100 GB DRAM, 32 vCPUs, for 2 days. When limiting resources to 64 GB memory and 16 vCPUs, the rebuilding time becomes significantly longer, e.g., 5 days for DiskANN. This stressful setting could also lead to a catastrophic drop in query performance due to severe computational resource starvation. Consequently, distributed systems have become essential to support large-scale similarity search and efficient index construction. While distributed systems offer the potential to handle large-scale datasets, they also introduce numerous challenges:

Table 1. Build time of DiskANN.

	Memory	CPU	Time
DiskANN	1100 GB	32 cores	2 days
	64 GB	16 cores	5 days

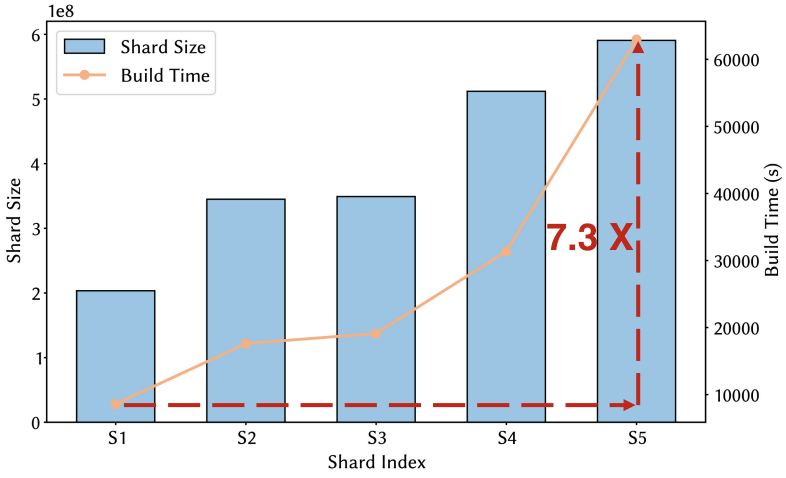


Fig. 2. Load imbalance in the construction process for Deep1B dataset.

Workload Imbalance: One of the primary challenges in distributed index construction is achieving an even distribution of data partitions across all servers. When the dataset is divided, variations in partition sizes can result in certain servers being assigned significantly more vectors than others. This imbalance leads to situations where some nodes complete their index construction tasks much earlier and remain idle, while others continue processing larger partitions. As shown in Fig. 2, the build time of the shard with the largest number of vectors is 7.3× longer than the build time of the shard with the smallest number

of vectors. As a result, overall resource utilization decreases and the total index construction time is dictated by the slowest, most heavily loaded node.

Network and I/O Bottlenecks: Data transfer between distributed nodes can incur significant latency, especially when handling large volumes of data. During the index merge stage, inter-node communication (e.g., fetching neighbor data, synchronizing shards) can create network and I/O bottlenecks that severely impact overall system performance. Therefore, minimizing network traffic and optimizing data transfer strategies are crucial for achieving high throughput and low latency.

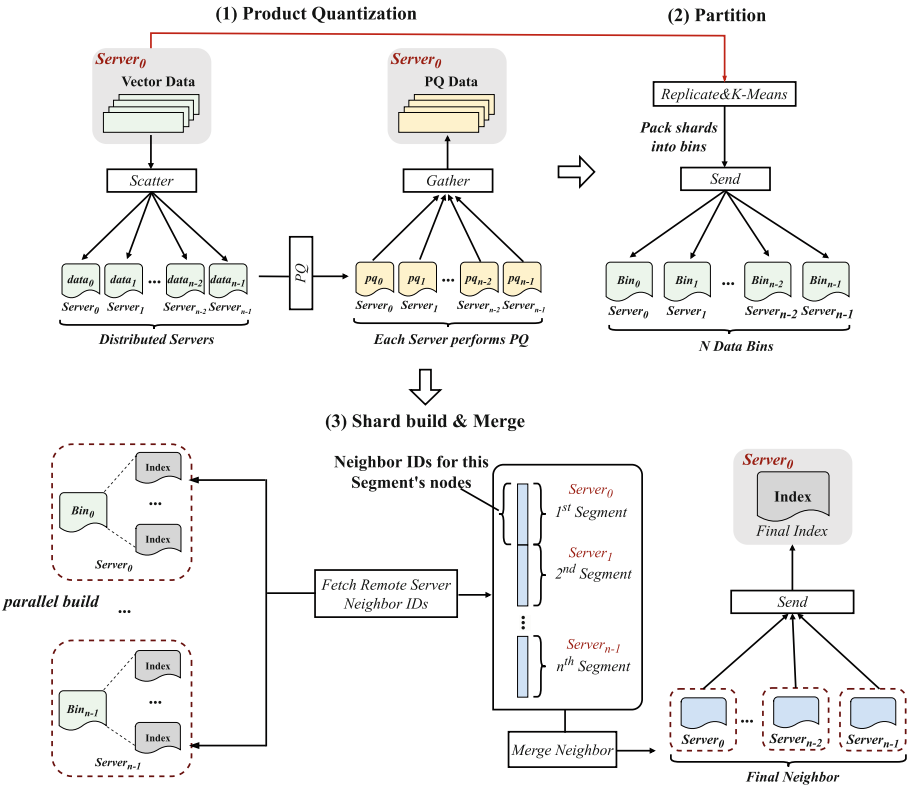


Fig. 3. SD-GAC framework.

3 Design

3.1 System Overview

As shown in Fig. 3, SD-GAC has three key techniques: distributed PQ computation, balanced partition and distributed graph merge. These techniques are

designed to address the main challenges in large-scale graph-based index construction. Specifically, distributed PQ computation accelerates the quantization process by parallelizing computation on multiple nodes, thus overcoming the bottleneck of single-node processing. Balanced partition ensures that each node is allocated nearly the same amount of data, thus alleviating load imbalance and maximizing the efficiency of parallel construction. Finally, distributed graph merge efficiently integrates locally constructed subgraphs into the global index, minimizing the frequent and expensive communication overhead typically associated with merging, and thereby addressing the scalability challenge.

3.2 Distributed PQ Computation

Distributed Product Quantization (PQ) computation is a critical component within our proposed framework, specifically designed to address the scalability bottleneck commonly encountered during quantization in large-scale graph-based index construction. Traditional PQ implementations suffer from limited throughput and significant latency due to the resource constraints inherent to single-node systems. To overcome these limitations, our framework implements a distributed batch-based PQ computation strategy.

Algorithm 1. Distributed PQ Pivots Generation

Require: Training data $X \in \mathbb{R}^{N \times d}$, number of clusters K, number of PQ chunks M
Ensure: PQ pivots for each chunk, saved to file
1: *// rank = 0 is the master node*
2: **if** rank = 0 **then**
3: Divide the d dimensions into M feature chunks: $\{C_1, ..., C_M\}$
4: Assign a subset of chunks to each server
5: Broadcast chunk assignment and chunk offsets to all ranks
6: **end if**
7: All servers receive chunk offsets and assigned chunk range $[s, e)$
8: ***// Lines 8-9 are executed in parallel on multiple nodes***
9: **for** chunk $i = s$ to $e - 1$ **do**
10: Run K-means on X_{C_i} to get K pivots
11: **end for**
12: Gather all local pivots from workers to rank 0
13: **if** rank = 0 **then**
14: Save PQ pivots to file
15: **end if**

As illustrated in Algorithm 1, the process begins with the master node (rank = 0) dividing the d-dimensional feature space into M chunks, each representing a subset of coordinates (Line 3). The master then assigns a subset of these chunks to each server and broadcasts the chunk assignment and corresponding offsets to all participating nodes (Lines 4-5). Upon receiving their respective chunk assignments (Line 7), all servers proceed in parallel to process their allocated

chunks: for each assigned chunk i, the server extracts the data corresponding to the dimensions of chunk i and performs K-means clustering to obtain K cluster centers (pivots) (Lines 9–11). Once the local pivots for all chunks have been computed, they are gathered from all servers to the master node (Line 12), which then writes the consolidated PQ pivots to disk (Line 14). This workflow enables efficient parallel generation of PQ pivots by distributing the computation of feature chunks across multiple nodes.

Algorithm 2. Distributed PQ Encoding

Require: Raw dataset $X \in \mathbb{R}^{N \times d}$, PQ pivots $\{\mathcal{P}_1, ..., \mathcal{P}_M\}$, batch size B
Ensure: PQ codes for all data points
1: **if** rank = 0 **then**
2: Load PQ pivots
3: **end if**
4: **for** batch $b = 1$ to $\lceil N/B \rceil$ **do**
5: **if** rank = 0 **then**
6: Subtract centroid: $x \leftarrow x - c$
7: Scatter the batch to all ranks (each receives $\approx B/p$ points)
8: **end if**
9: Each rank receives assigned data points
10: *// **Lines 11-14 are executed in parallel on multiple nodes***
11: **for all** assigned point x **do**
12: **for** chunk $i = 1$ to M **do**
13: Extract x_i (chunk's dimensions)
14: Find closest pivot id: $q_i = \arg\min_j \|x_i - \mathcal{P}_{i,j}\|$
15: Store q_i as x's code for chunk i
16: **end for**
17: **end for**
18: Gather encoded results to rank 0
19: **if** rank = 0 **then**
20: Write all gathered PQ codes of the batch to disk
21: **end if**
22: **end for**

After the PQ pivots have been generated and collected as described above, the distributed encoding of the raw dataset proceeds as shown in Algorithm 2. Initially, the master node loads the precomputed PQ pivots into memory (Line 2). The encoding is performed in batches to accommodate large-scale data: for each batch (Line 4), the master node first subtracts the centroid from each vector and then scatters the batch to all worker nodes, ensuring that each node receives an approximately equal number of data points (Lines 6–7). Once the data has been distributed, each worker processes its assigned points in parallel (Line 9). For every assigned point x, and for each PQ chunk i, the worker extracts the relevant subset of dimensions, computes the nearest codeword in the corresponding codebook $\mathcal{P}_i$, and stores the resulting code (Lines 12–16). After all assigned points in the batch have been encoded, the encoded results are gathered from

all workers to the master node (Line 18), which then writes the PQ codes of the current batch to disk (Line 20). This distributed encoding workflow efficiently utilizes the available computational resources and enables scalable compression of billion-scale datasets.

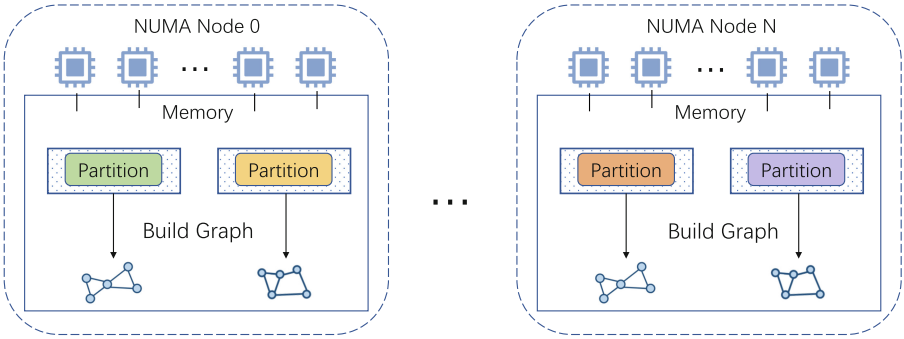


Fig. 4. NUMA-aware building architecture.

3.3 Balanced Partition and NUMA-Aware Building

To address load imbalance in distributed settings, we implement the balanced partitioning algorithm. We first divide each vector into a number of small partitions (with the number of partitions exceeding the number of machines), then employ the best-fit bin-packing algorithm [4] to aggregate these small partitions into larger partitions, so that the number of resulting partitions matches the number of machines. This fine-grained partitioning and packing strategy leads to more balanced workload allocation, thereby preventing scenarios where some servers become idle after finishing their assigned tasks while others are still actively constructing the index. By minimizing partition size variance, our approach maximizes resource utilization and improves the overall efficiency of distributed index construction.

To further accelerate the build within each server, we implement a NUMA-aware index construction strategy. Modern multi-core servers often use Non-Uniform Memory Access (NUMA) architectures, where memory access is faster when data resides on a processor's local node than on a remote node. As shown in Fig. 4, SD-GAC is designed to take advantage of this heterogeneous memory hierarchy by distributing partitions (shards) across NUMA nodes. To minimize remote memory accesses, SD-GAC employs affinity-based scheduling and supports work stealing within each NUMA node to mitigate workload imbalances. Partitions are bound to specific CPU cores, ensuring that each partition is always processed by the same core to maximize cache utilization. By co-locating computation with the relevant data, SD-GAC reduces remote memory accesses, saturates memory bandwidth, and ultimately decreases build time.

3.4 Parallel Graph Merge

Algorithm 3. Distributed Parallel Shard Graph Merge

Require: Global node set V, partitioned among P servers by global ID (e.g., server i merges neighbors for $V_i = [i \cdot N/P, (i+1) \cdot N/P)$), precomputed mapping from node to server.

Ensure: Final merged global graph index.

1: *// Step 1: Precompute node-to-server mapping based on global IDs*
2: Each server determines its assigned range V_i and the mapping for all nodes.
3: *// Step 2: Prefetch all needed neighbor lists from remote servers*
4: **for** each remote server j **do**
5: Request all neighbor lists for nodes in V_i that are owned by j.
6: Receive all requested neighbor lists in a single batch (contiguous by construction).
7: **end for**
8: *// Step 3: Parallel merge of neighbor lists*
9: **for all** node v in V_i **in parallel do**
10: Gather all k_{base} neighbor lists for v (local + remote).
11: Merge and deduplicate neighbors; select up to max_degree per node.
12: **end for**
13: *// Step 4: Parallel and ordered file writing*
14: Each server writes merged results to the final index file at precomputed offsets, matching the global node ID order.

As illustrated in Algorithm 3, the merge process begins by partitioning the global node set by ID, such that each server is responsible for merging the neighbor lists for a contiguous range of nodes. The mapping from each node to its assigned server is precomputed and distributed in advance, ensuring that the assignment aligns with the final output file layout (Line 2). Prior to merging, each server prefetches all necessary neighbor lists from remote servers in a single, bulk I/O operation, taking advantage of the fact that the requested nodes on each server are stored contiguously in memory (Lines 4–7). With all relevant neighbor lists locally available, each server proceeds to merge and deduplicate the neighbors for each of its assigned nodes in parallel (Lines 9–12), selecting up to the target degree for each node. Finally, since the task partitioning matches the final global file order, each server can write its merged results directly to the appropriate offset in the output index file, enabling efficient and potentially parallel file I/O (Line 14).

This strategy eliminates network I/O bottlenecks during the merge because each server aggregates all required neighbor information from remote servers in a single, bulk request, leveraging the fact that the needed data is stored contiguously on the sender side. As a result, each remote server only needs to serve a single sequential data transfer per requesting server, minimizing communication rounds and avoiding excessive small or random-access network I/O. Moreover, once all required data has been fetched, the entire neighbor merging process for

each assigned node can be performed independently and in parallel, without further synchronization or communication. By aligning the task partitioning with the final file layout, each server can also write its results directly to the output file at precomputed offsets, enabling parallel and sequential disk I/O, further reducing contention and maximizing throughput. Together, these design choices ensure that both computation and I/O resources are efficiently utilized, leading to significant improvements in distributed merge performance.

4 Evaluation

In this section, we seek to answer the following questions:

- How efficiently does SD-GAC build the vector index? (§4.1)
- Does SD-GAC affects search performance? (§4.2)
- How do SD-GAC's individual techniques impact overall performance? (§4.3)

System Setup: We use 10 servers, each with two AMD EPYC 7763 CPUs (64 cores per CPU, 128 cores in total) and 512GB RAM. SD-GAC is built on top of the distributed version of DiskANN [25], which is implemented using the MPI [2] library.

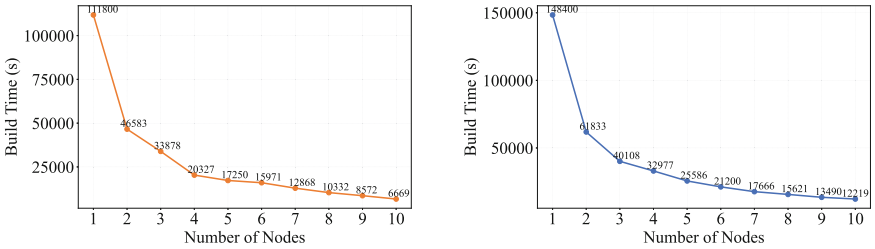
Datasets and Metrics: In our experiments, we use two standard billion-scale datasets [1] that have been widely used by previous studies, as illustrated in Table 2. We use query per second (QPS) to evaluate the performance. The query accuracy is evaluated by Recall@10, which represents the proportion of the top-10 results containing the ground-truth nearest neighbors for an ANNS query.

Table 2. Datasets (One Billion)

Dataset	Dimension	Raw Data Size	Data Type
SIFT1B	128	119 GB	uint8
DEEP1B	96	358 GB	float32

4.1 Construction Time

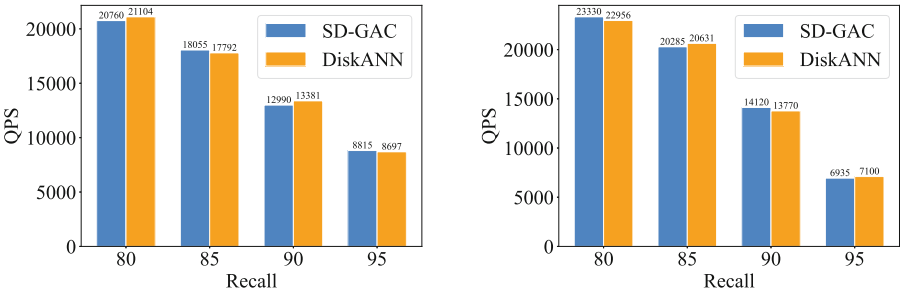
We evaluate the construction time with different number of servers, for single-node we use the original implementation of DiskANN, for distributed we use the implementation of SD-GAC, each server uses 128 threads. The results are shown in Fig. 5. As shown in Fig. 5a and Fig. 5b, SD-GAC achieves 2.4× and 16.8× speedup at two and ten nodes on the SIFT1B dataset, and 2.3× and 12.1× speedup on the DEEP1B dataset, respectively, compared to the single-node baseline. This substantial speedup is primarily attributed to the efficient



(a) SIFT1B (b) DEEP1B

Fig. 5. Build time of different datasets.

parallelization of index construction across multiple servers, which allows SD-GAC to fully exploit distributed memory, CPU, and I/O resources. Additionally, SD-GAC employs NUMA-aware scheduling and optimized data partitioning, minimizing load imbalance and ensuring that computational resources are utilized effectively at all nodes. The lower speedup on DEEP1B is due to its larger data size (float32), which increases network transmission time. Notably, the speedup exceeds the ideal linear scaling. This is because, in the single-node baseline, the constructed index must be frequently written to SSD, incurring significant I/O overhead. In contrast, SD-GAC maintains the constructed subgraphs in distributed memory throughout the build process and only writes to disk when necessary, substantially reducing the amount and frequency of SSD writes.



(a) SIFT1B (b) DEEP1B

Fig. 6. QPS comparison between SD-GAC and DiskANN.

4.2 Search Performance

We use 64 threads for search, and the results are shown in Fig. 6, the results for SD-GAC were obtained using an index constructed with 10 nodes. As shown, the

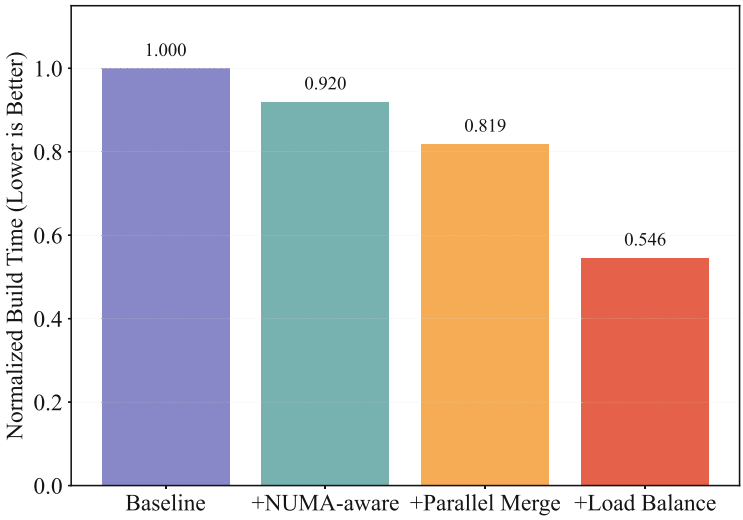


Fig. 7. Ablation study of SD-GAC.

QPS of SD-GAC is comparable to that of DiskANN across different recall rates. The figure demonstrates that QPS increases as the recall rate decreases, which aligns with expectations. Notably, the QPS values of SD-GAC and DiskANN are closely matched, with SD-GAC exhibiting only minor fluctuations. This demonstrates that our distributed construction approach in SD-GAC significantly accelerates index build time while maintaining search performance.

Such improvements not only enable timely index updates for rapidly growing datasets but also allow the system to efficiently scale to larger data volumes and adapt to dynamic, real-world workloads. Consequently, SD-GAC delivers both high efficiency and flexibility for large-scale vector search applications.

4.3 Ablation Study

To quantify the impact of each key optimization in our distributed index construction framework, we conduct an ablation study by incrementally enabling the following techniques: NUMA-aware construction, parallel graph merge, and load balance. Test results are averaged over the sift1B and deep1B datasets The baseline is the original distributed construction without any of these optimizations, and the total build time is normalized to 1.0.

As shown in Fig. 7, enabling NUMA-aware construction reduces the build time by 8%, lowering the normalized build time to 0.92. Incorporating parallel graph merge further reduces the build time by 11% relative to the previous stage, resulting in a normalized value of 0.82. Finally, introducing load balance achieves an additional 33% reduction based on the previous configuration, further decreasing the build time to 0.55. This stepwise improvement clearly

demonstrates that each proposed technique contributes to the overall efficiency of the index construction process.

Overall, the combination of all three optimizations results in a 45% reduction in total build time compared to the baseline. These results highlight the effectiveness of our system design in addressing scalability bottlenecks for billion-scale vector indexing.

5 Conclusion

In this paper, we proposed SD-GAC, a distributed framework designed to accelerate graph-based vector index construction for billion-scale datasets. Our approach addresses the key challenges of distributed index construction, including workload imbalance, network and I/O bottlenecks, and suboptimal hardware utilization. By leveraging fine-grained partitioning, a NUMA-aware scheduling strategy, and communication-efficient shard merging, SD-GAC significantly reduces construction time while maintaining high accuracy. Through extensive experiments, we demonstrated that SD-GAC achieves substantial improvements in build time, with near-linear speedup as the number of nodes increases. Additionally, the framework scales effectively across large datasets, making it well-suited for modern vector search systems that require fast and efficient index construction.

Acknowledgement. The work is supported by the National Key Research and Development Program of China (grant no. 2022YFB4500300) and the National Natural Science Foundation of China (grant no. 62441220).

References

1. Big ann-benchmarks (2021). https://big-ann-benchmarks.com/neurips21.html
2. Open mpi (2025). https://www.open-mpi.org
3. Cui, J., et al.: Chatlaw: a multi-agent collaborative legal assistant with knowledge graph enhanced mixture-of-experts large language model (2024). https://arxiv.org/abs/2306.16092
4. Dósa, G., Sgall, J.: Optimal analysis of best fit bin packing. In: Esparza, J., Fraigniaud, P., Husfeldt, T., Koutsoupias, E. (eds.) Automata, Languages, and Programming, pp. 429–441. Springer, Berlin Heidelberg, Berlin, Heidelberg (2014)
5. Doshi, I., Das, D., Bhutani, A., Kumar, R., Bhatt, R., Balasubramanian, N.: Lanns: a web-scale approximate nearest neighbor lookup system. Proc. VLDB Endow. **15**(4), 850–858 (Dec 2021). https://doi.org/10.14778/3503585.3503594
6. Douze, M., et al.: The faiss library (2025). https://arxiv.org/abs/2401.08281
7. Fu, C., Xiang, C., Wang, C., Cai, D.: Fast approximate nearest neighbor search with the navigating spreading-out graph. Proc. VLDB Endow. **12**(5), 461–474 (Jan 2019). https://doi.org/10.14778/3303753.3303754
8. Gao, J., Long, C.: High-dimensional approximate nearest neighbor search: with reliable and efficient distance comparison operations. Proc. ACM Manag. Data **1**(2) (Jun 2023). https://doi.org/10.1145/3589282

9. Ge, T., He, K., Ke, Q., Sun, J.: Optimized product quantization for approximate nearest neighbor search. In: 2013 IEEE Conference on Computer Vision and Pattern Recognition, pp. 2946–2953 (2013). https://doi.org/10.1109/CVPR.2013.379
10. Gionis, A., Indyk, P., Motwani, R.: Similarity search in high dimensions via hashing. In: Proceedings of the 25th International Conference on Very Large Data Bases, pp. 518–529. VLDB '99, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (1999)
11. Guo, R., et al.: Accelerating large-scale inference with anisotropic vector quantization. In: Proceedings of the 37th International Conference on Machine Learning. ICML'20, JMLR.org (2020)
12. Huang, J.T., et al.: Embedding-based retrieval in facebook search. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining. p. 2553–2561. KDD '20, Association for Computing Machinery, New York, NY, USA (2020). https://doi.org/10.1145/3394486.3403305
13. Huang, Q., Feng, J., Zhang, Y., Fang, Q., Ng, W.: Query-aware locality-sensitive hashing for approximate nearest neighbor search. Proc. VLDB Endow. **9**(1), 1–12 (Sep 2015). https://doi.org/10.14778/2850469.2850470
14. Indyk, P., Motwani, R.: Approximate nearest neighbors: towards removing the curse of dimensionality. In: Proceedings of the Thirtieth Annual ACM Symposium on Theory of Computing pp. 604–613. STOC '98, Association for Computing Machinery, New York, NY, USA (1998). https://doi.org/10.1145/276698.276876
15. Jeong, C.: A study on the implementation of generative ai services using an enterprise data-based LLM application architecture. Adv. Artif. Intell. Mach. Learn. **03**(04), 1588–1618 (2023). https://doi.org/10.54364/aaiml.2023.1191
16. Jégou, H., Douze, M., Schmid, C.: Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **33**(1), 117–128 (2011). https://doi.org/10.1109/TPAMI.2010.57
17. Lu, K., Kudo, M., Xiao, C., Ishikawa, Y.: Hvs: hierarchical graph structure based on voronoi diagrams for solving approximate nearest neighbor search. Proc. VLDB Endow. **15**(2), 246–258 (Oct 2021). https://doi.org/10.14778/3489496.3489506
18. Malkov, Y.A., Yashunin, D.A.: Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. IEEE Trans. Pattern Anal. Mach. Intell. **42**(4), 824–836 (2020). https://doi.org/10.1109/TPAMI.2018.2889473
19. Mikolov, T., Chen, K., Corrado, G., Dean, J.: Efficient estimation of word representations in vector space (2013). https://arxiv.org/abs/1301.3781
20. Muja, M., Lowe, D.G.: Scalable nearest neighbor algorithms for high dimensional data. IEEE Trans. Pattern Anal. Mach. Intell. **36**(11), 2227–2240 (2014). https://doi.org/10.1109/TPAMI.2014.2321376
21. Pan, J.J., Wang, J., Li, G.: Survey of vector database management systems. The VLDB Journal 33(5), 1591–1615 (Jul 2024). https://doi.org/10.1007/s00778-024-00864-x
22. Pennington, J., Socher, R., Manning, C.: GloVe: Global vectors for word representation. In: Moschitti, A., Pang, B., Daelemans, W. (eds.) Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1532–1543. Association for Computational Linguistics, Doha, Qatar (Oct 2014). https://doi.org/10.3115/v1/D14-1162
23. Ren, J., Zhang, M., Li, D.: Hm-ann: efficient billion-point nearest neighbor search on heterogeneous memory. In: Proceedings of the 34th International Conference on Neural Information Processing Systems. NIPS '20, Curran Associates Inc., Red Hook, NY, USA (2020)

24. Singh, A., Subramanya, S.J., Krishnaswamy, R., Simhadri, H.V.: Freshdiskann: A fast and accurate graph-based ann index for streaming similarity search (2021). https://arxiv.org/abs/2105.09613
25. Subramanya, S.J., Devvrit, Kadekodi, R., Krishaswamy, R., Simhadri, H.V.: DiskANN: fast accurate billion-point nearest neighbor search on a single node. Curran Associates Inc., Red Hook, NY, USA (2019)
26. Sun, Y., Wang, W., Qin, J., Zhang, Y., Lin, X.: SRS: solving c-approximate nearest neighbor queries in high dimensional euclidean space with a tiny index. Proc. VLDB Endow. **8**(1), 1–12 (Sep 2014). https://doi.org/10.14778/2735461.2735462
27. Tagami, Y.: Annexml: Approximate nearest neighbor search for extreme multi-label classification. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 455–464. KDD '17, Association for Computing Machinery, New York, NY, USA (2017). https://doi.org/10.1145/3097983.3097987
28. Wang, J., et al.: Trinary-projection trees for approximate nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell. **36**(2), 388–403 (2014). https://doi.org/10.1109/TPAMI.2013.125
29. Wang, M., et al.: Starling: An i/o-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment. Proc. ACM Manag. Data **2**(1) (Mar 2024). https://doi.org/10.1145/3639269
30. Wang, M., Xu, X., Yue, Q., Wang, Y.: A comprehensive survey and experimental comparison of graph-based approximate nearest neighbor search. Proc. VLDB Endow. **14**(11), 1964–1978 (Jul 2021). https://doi.org/10.14778/3476249.3476255
31. Wen, C., Sun, X., Zhao, S., Fang, X., Chen, L., Zou, W.: Chathome: Development and evaluation of a domain-specific language model for home renovation (2023). https://arxiv.org/abs/2307.15290
32. Xu, Y., et al.: Spfresh: Incremental in-place update for billion-scale vector search. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp. 545–561. SOSP '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3600006.3613166
33. Zhang, J., et al.: Uni-retriever: towards learning the unified embedding based retriever in bing sponsored search. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 4493–4501. KDD '22, Association for Computing Machinery, New York, NY, USA (2022). https://doi.org/10.1145/3534678.3539212
34. Zhang, X., Yang, Q., Xu, D.: Xuanyuan 2.0: A large Chinese financial chat model with hundreds of billions parameters (2023). https://arxiv.org/abs/2305.12002
35. Zhang, Y., et al.: Visual search at alibaba. In: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 993–1001. KDD '18, Association for Computing Machinery, New York, NY, USA (2018). https://doi.org/10.1145/3219819.3219820

Scalable and Contention-Resilient Concurrent Interval Tree

Kapil Kumar Attinagaramu(✉) and Praveen Alapati

Computer Science Engineering Department, Supercomputer Lab. École Centrale School of Engineering, Mahindra University, Hyderabad, India
KapilKumar20pcse002@mahindrauniversity.edu.in, praveena@cse.iitm.ac.in

Abstract. Interval trees are fundamental data structures for managing and querying overlapping intervals, and are widely used in scheduling, network monitoring, and database conflict resolution domains. However, supporting such operations efficiently in multi-threaded environments remains a substantial challenge due to synchronization bottlenecks and contention. In this work, we introduce a novel concurrent interval tree that addresses scalability and contention challenges in multi-threaded settings. This design is built on the concurrent AA Tree (CAA Tree). Our approach departs from traditional interval tree implementations by allowing multiple high-values to be stored within a single node using a thread-safe structure. It significantly improves space utilization and, in turn, helps to improve performance while identifying overlapping intervals. The proposed structure is designed for efficient concurrent *insert*, *delete*, and *overlap operations* across diverse workloads. This is achieved through a combination of a relaxed balancing approach and a lazy deletion mechanism, which together reduce structural contention and improve cache locality. Our implementation achieves an average speedup of 22.87× over its sequential version when the number of threads is 64 and consistently outperforms state-of-the-art implementations.

Keywords: Multi-Threading · Concurrent Interval Trees · Synchronization · Logical Ordering · Contention-Resilient

1 Introduction

The growing use of multicore processors in modern computing systems has made the design of efficient concurrent data structures increasingly important. While sequential search trees continue to serve as fundamental components in applications such as databases, network routing, and task scheduling [1], they are not well-suited for multi-threaded environments. Their performance tends to degrade under concurrent workloads due to serialization and synchronization bottlenecks. To overcome these limitations, various concurrent search tree designs have been proposed, which enable parallel access across threads while maintaining correctness. However, building such structures introduces new challenges, including maintaining structural balance, ensuring consistency during concurrent updates,

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 59–78, 2026.
https://doi.org/10.1007/978-981-95-8405-5_4

and minimizing contention, all while preserving the expected logarithmic time complexity of search operations.

Among the various concurrent data structures, interval trees play a particularly important role due to their ability to efficiently support range queries in logarithmic time [3,4]. Interval trees were originally proposed by Pal et al. [2], have found widespread application in domains such as task scheduling, network monitoring, and genome sequencing [5]. Conventional interval tree implementations frequently rely on self-balancing trees such as AVL or Red-Black trees. Although these data structures ensure that the tree remains balanced and thereby guarantee logarithmic time complexity for search, insertion, and deletion operations—they introduce notable challenges in concurrent environments. Maintaining balance in AVL and Red-Black trees involves structural modifications such as rotations and color or height updates after insertions and deletions. In a concurrent setting, these operations must be carefully coordinated to preserve the tree's invariants, often requiring the use of locks or other synchronization mechanisms. The need to synchronize access during rebalancing operations can lead to thread stalls, reduced parallelism, and decreased scalability. In particular, operations that trigger rotations or propagate structural changes up the tree may block other threads, resulting in performance bottlenecks as the number of threads increases.

In this paper, we introduce a novel concurrent interval tree constructed on top of the Concurrent AA Tree (CAA Tree) [6], a self-balancing structure specifically designed for concurrent execution. Unlike traditional balancing strategies that rely on multiple types of complex rotations, the CAA Tree employs a single rotation (skew and split) and leverages logical ordering to reduce synchronization overhead. These characteristics make it especially amenable to efficient and scalable interval management in multithreaded settings. A central challenge in designing concurrent interval trees is efficiently supporting duplicate low endpoints while allowing access to multiple high endpoints. Conventional data structures often lack this capability, resulting in poor scalability. Our proposed structure addresses this limitation by representing each node as a multi-valued entity that stores multiple high values for a single low value using a thread-safe `ConcurrentHashMap`. This compact representation reduces node proliferation and minimizes contention, thus improving throughput for interval-dense workloads. Effective memory management also contributes to high performance in concurrent data structures. By reducing unnecessary node creation and structural changes, the use of multi-valued nodes decreases the frequency of rebalancing operations. Additionally, we incorporate a lazy deletion mechanism in which logically deleted nodes are removed asynchronously by a dedicated background thread.

Our experimental evaluation compared to state-of-the-art approaches (see [7]) shows that the proposed concurrent interval tree outperforms existing designs in throughput, contention resilience, and cache efficiency. Beyond the general benefits of the CAA framework, our design tackles interval-specific challenges: multi-valued nodes reduce node proliferation for overlapping intervals,

relaxed balancing minimizes costly rebalancing cascades during interval-heavy updates, and lazy deletion defers structural removals to lower contention during deletions. These optimizations, combined with fine-grained synchronization, yield high scalability across diverse workloads and thread counts.

2 Background

This section introduces the structural foundations of the proposed Concurrent Interval Tree (CIT), focusing on how each node represents and manages intervals in a way that supports high concurrency. Each node is designed to capture a collection of intervals that share the same starting point (low value), while also facilitating efficient querying and updating in a dynamic environment. Specifically, a node maintains three key components: the *low* value, a multi-valued set of *high* endpoints, and a *max* field indicating the largest high value in its subtree. The low value serves as the primary key to ordering nodes within the tree. The max value enables early pruning during range or overlap queries. Meanwhile, high values associated with the same low value are stored in a thread-safe `ConcurrentHashMap`, allowing for efficient handling of overlapping intervals without redundant node creation.

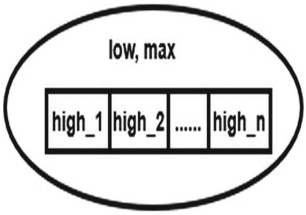


Fig. 1. Structure of a multi-valued node with size n.

As shown in Fig. 1, this multi-valued node structure enables the CIT to compactly store multiple intervals with the same starting point. By avoiding the insertion of separate nodes for each such interval, the structure improves both space utilization and traversal efficiency, which are the two essential factors in high-performance concurrent settings.

Figure 2 illustrates the step-by-step insertion of intervals into the CIT. The process begins with inserting the interval $(20, 45)$, which becomes the root node. As the tree is initially empty, no rebalancing is required. The next interval, $(15, 19)$, is inserted as the left child of $(20, 45)$, which triggers a skew operation to maintain the AA Tree invariant, making $(15, 19)$ the new root (Fig. 2(c)).

The insertion of $(25, 50)$ adds a right child to $(20, 45)$ and requires a split operation to restore balance, incrementing the level of $(20, 45)$ and updating the root's maximum high value to 50. As additional intervals such as $(10, 18)$, $(20, 30)$, and $(25, 45)$ are inserted, the structure distinguishes between new low

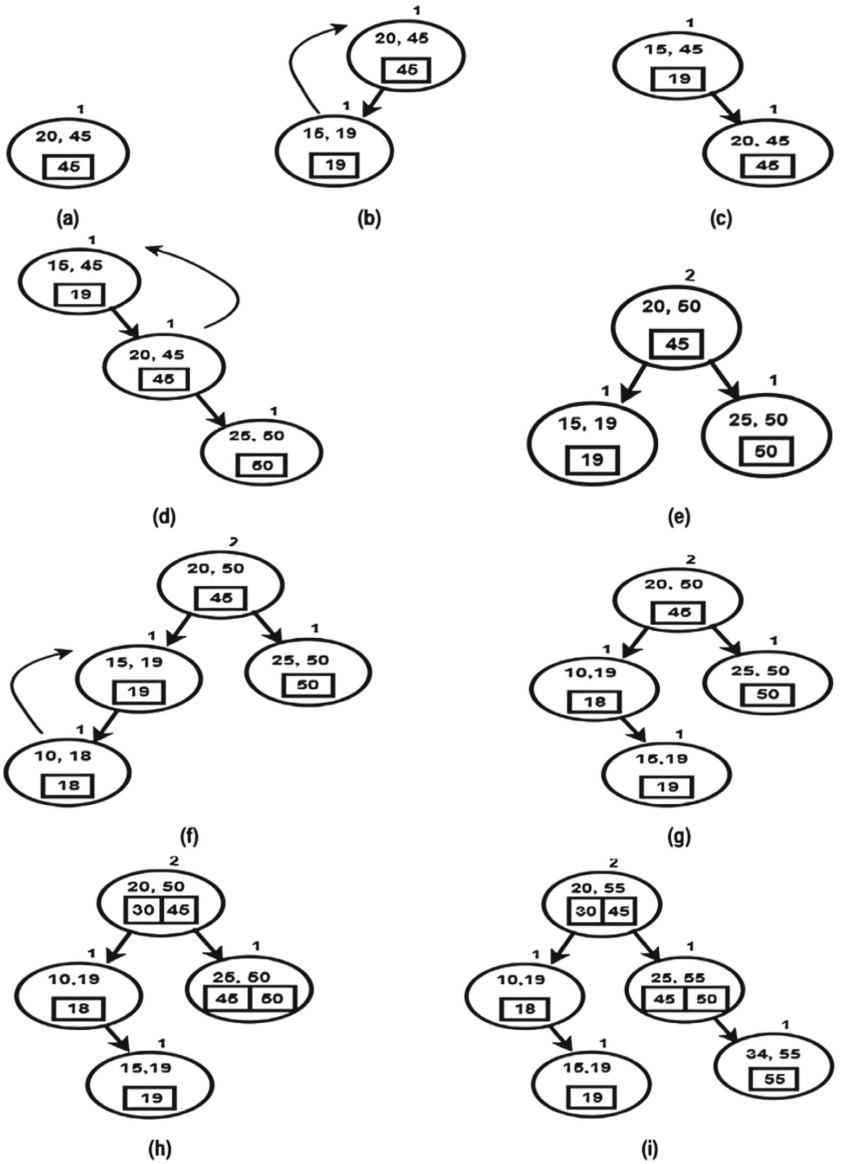


Fig. 2. Inserting the (low, high) values: (20, 45), (15, 19), (25, 50), (10, 18), (20, 30), (25, 45), and (34, 55).

values and duplicates. For duplicate low values (e.g., 20 and 25), the corresponding high values are added to the node's internal `ConcurrentHashMap`, preserving the tree's structure while increasing interval density within each node. The final interval, $(34, 55)$, is inserted as the right child of $(25, 50)$. Since this insertion does

not violate any AA Tree invariants, no rebalancing is necessary. This incremental and localized insertion strategy ensures that the tree remains balanced through lightweight skew and split operations while efficiently accommodating overlapping intervals. By leveraging multi-valued nodes and limiting structural changes, CIT minimizes synchronization overhead and improves memory locality, making it well-suited for multi-threaded execution.

Deletion in CIT also benefits from this design. Figure 3 shows the deletion process for intervals $(10, 18)$ and $(15, 19)$. When removing $(10, 18)$, the node is simply detached, and its child $(15, 19)$ is reconnected to the root node. The deletion of $(15, 19)$ leads to a temporary imbalance, which is then corrected through level adjustments and rebalancing operations. These corrective steps are lightweight and follow the same principles as the AA Tree's rotation-based balancing.

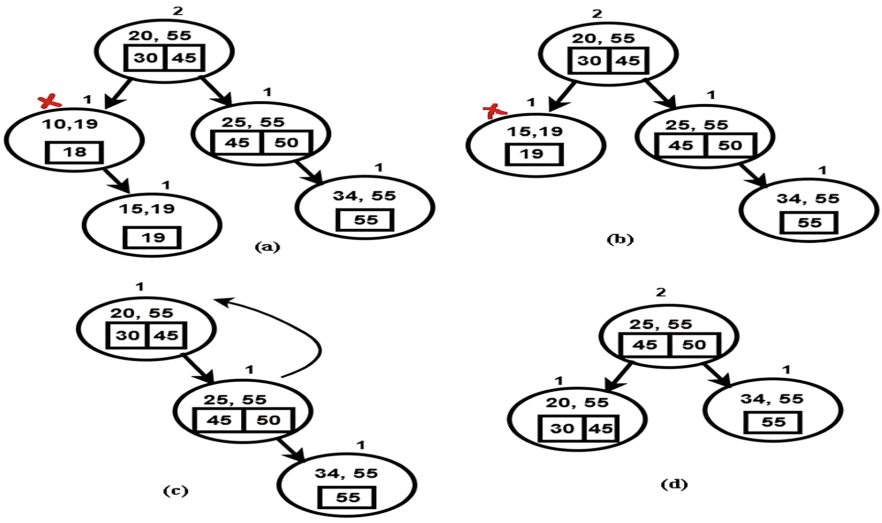


Fig. 3. Deleting the (low, high) values: (10, 18) and (15, 19). Here, the red cross indicates a logically deleted node that satisfies conditions for physical removal. (Color figure online)

A key advantage of the CIT deletion process lies in its lazy deletion strategy. If a node still contains other high values, only the specific interval is removed from the `ConcurrentHashMap`, leaving the node intact. If the node becomes empty, it is logically marked for deletion but not immediately removed from the tree. Instead, a dedicated background maintenance thread performs physical deletion in a safe and non-blocking manner. This thread ensures that marked nodes are removed only when they are leaves or have a single child, thus preserving structural consistency without interfering with active operations.

Overall, the design choices in CIT, including multi-valued nodes, relaxed balancing, and deferred deletions, allow the data structure to maintain correct-

ness while significantly improving throughput and scalability under concurrent workloads.

3 Concurrent Interval Tree

3.1 Node Structure

Table 1. Fields of a CIT Node

final int *low*
volatile int *max*
volatile int *level*
volatile boolean *mark*
volatile **concurrentHashMap**<> *instance*
volatile **Node** *left*, *right*, *parent*, *pred*, *succ*
volatile **ReentrantLock** *succLock*
volatile **ReentrantLock** *treeLock*

Table 1 outlines the structure of a node in the CIT. Each field is carefully designed to support thread-safe operations and preserve the structural and semantic correctness of the tree during concurrent accesses and modifications. The `low` field represents the lower bound of an interval and serves as the node's primary key for ordering within the tree. The `max` field stores the highest upper bound among all intervals in the subtree rooted at this node. This value is essential for efficiently pruning subtrees during overlap or range queries. The `level` field represents the node's depth in the tree and is used to enforce balance through skew and split operations.

To facilitate efficient deletions, each node includes a `mark` flag indicating whether the node is logically deleted. A value of `true` means the node is marked for deferred deletion and will be physically removed later by a dedicated maintenance thread. The `instance` field holds a thread-safe `ConcurrentHashMap` that maps each low endpoint to a set of high endpoints. This design enables a single node to represent multiple overlapping intervals that share the same starting point. The pointers `left`, `right`, and `parent` maintain the physical structure of the tree, allowing standard traversal and modification operations such as insertion, deletion, and rebalancing. Furthermore, `pred` and `succ` are used to maintain logical ordering between nodes based on order traversal. This logical ordering supports efficient operations that require range-aware navigation, such as range queries and concurrent traversal.

To ensure thread safety, each node includes two fine-grained locks: `succLock` and `treeLock`. The `succLock` protects the logical successor and predecessor links, allowing concurrent threads to update the ordering information safely.

Meanwhile, the `treeLock` guards the physical structure during critical operations such as insertions, deletions, or structural adjustments to maintain balance. By combining logical ordering, multivalued node representations, and fine-grained locking mechanisms, the CIT node structure supports scalable, high-throughput operations in multithreaded environments while maintaining correctness and low contention.

3.2 Concurrent Operations on Interval Tree

The concurrent interval tree supports a rich set of operations essential for managing dynamic interval datasets in multithreaded environments. These operations include *getAllOverlaps*, *insert*, *delete*, and range queries, all implemented on the CAA Tree framework. This subsection focuses on the semantics and implementation of these operations, including their correctness guarantees through linearization points [8,9].

The *getAllOverlaps* operation retrieves all intervals that overlap with a given query range $(queryLow, queryHigh)$. An interval $(low, high)$ overlaps with the query if any of the following conditions are met: (i) partial left-aligned overlap where $low \leq queryLow \leq high$, (ii) partial right-aligned overlap where $queryLow \leq low \leq queryHigh$, (iii) complete containment where the interval fully encloses the query range with $low \leq queryLow$ and $high \geq queryHigh$, (iv) complete containment in reverse where the query range fully encloses the interval with $queryLow \leq low$ and $high \leq queryHigh$, or (v) exact match where $low = queryLow$ and $high = queryHigh$. The Algorithm 1 efficiently detects these cases through selective traversal, reducing redundant checks and improving throughput. These five cases represent the complete set of geometric relationships in which an interval may overlap with a query range. The algorithm efficiently detects all such cases through a combination of subtree pruning and targeted traversal, eliminating redundant checks and improving throughput.

Among all supported operations, *getAllOverlaps* provides the most comprehensive functionality. It generalizes both the *contains* and *exists_ Overlap* methods. The *contains* operation can be viewed as a special case of *getAllOverlaps* which checks for exact interval matches, while *exists_ Overlap* terminates as soon as the first overlapping interval is found. Given the overlap in logic, and due to space constraints, we present only the *getAllOverlaps* method in this section, as it subsumes the others and captures the core traversal and overlap detection strategies of the CIT.

***getAllOverlaps* operation:** The `getAllOverlaps` method performs a recursive traversal of the CIT to identify all intervals that overlap with a given query range $(queryLow, queryHigh)$, as shown in Algorithm 1. It is designed to be selective and efficient, using subtree pruning to minimize unnecessary traversal and inspection. The procedure begins by checking if the current node is `nil`, indicating the end of a branch, the recursion terminates immediately. This avoids further computation and ensures that only relevant portions of the tree are examined [6].

Algorithm 1. Identifying All Overlapping Intervals

```
1: procedure getAllOverlaps(queryLow, queryHigh, current)
2:     if current = nil then
3:         return
4:     end if
5:     nodeLow ← current.low
                                        ▷ Step 1: Check for Overlaps in the Current Node
6:     if queryLow ≤ current.max and queryHigh ≥ nodeLow then
7:         if current.instance ≠ nil then
8:             for each high ∈ current.instance.keySet() do
9:                 if queryHigh ≥ high then
10:                    Process the overlapping interval
11:                end if
12:            end for
13:        end if
14:    end if
                                        ▷ Step 2: Traverse Left Subtree Only If Overlap is Possible
15:    if current.left ≠ nil and queryLow ≤ current.max then
16:        getAllOverlaps(queryLow, queryHigh, current.left)
17:    end if
                                        ▷ Step 3: Traverse Right Subtree Only If Overlap is Possible
18:    if current.right ≠ nil and queryLow ≤ current.max then
19:        getAllOverlaps(queryLow, queryHigh, current.right)
20:    end if
21: end procedure
```

Next, in *Step 1*, the algorithm checks whether the query range potentially overlaps with the intervals stored at the current node. This is done by comparing the query range with the node's `max` and `low` values. If an overlap is possible, the algorithm inspects all high values stored in the node's `instance` map. For each high endpoint *high* associated with the low value of the node, if $queryHigh \geq high$, then the interval $(current.low, high)$ is reported as overlapping.

Step 2 focuses on exploring the left subtree. This traversal is conditional and only performed if the query range may intersect with intervals stored on the left, specifically if $queryLow \leq current.max$. This condition prevents unnecessary descent into subtrees that cannot possibly contain relevant intervals.

Similarly, *Step 3* handles traversal of the right subtree under the same pruning condition. By evaluating subtree relevance using the `max` field at each node, the algorithm effectively reduces the search space and improves throughput under concurrent execution.

This overlap detection strategy is highly concurrency friendly. It involves no locking or modification of the data structure, allowing multiple threads to perform read-only overlap queries without contention. Furthermore, the algorithm naturally supports generalizations such as exact containment or partial matches, making it versatile for a wide range of interval query applications.

***Insert* Operation:** The `insert` operation in the CIT is designed to support high throughput, concurrent insertions while maintaining correctness through fine-grained synchronization, logical ordering of nodes, and relaxed rebalancing. The algorithm, presented in Algorithm 2, begins by locating the appropriate position for insertion. This is achieved through a search operation that identifies the node whose low value matches the new interval or is closest to it. The parent of this node is also recorded to support structural updates during insertion. Based

Algorithm 2. Concurrent Insert Procedure in CIT

```
1: procedure INSERT(low, high)
2:     node ← SEARCH(low)
3:     parent ← node.parent
4:     pred ← (node.low ≥ low)?node.pred : node
5:     while not TRYSUCCLOCK(pred) do
6:         continue                                   ▷ Retry until predecessor lock is acquired
7:     end while
8:     succ ← pred.succ
9:     while not TRYSUCCLOCK(succ) do
10:        SUCCUNLOCK(pred)                           ▷ Release predecessor lock if successor lock fails
11:        continue
12:    end while
13:    if low > pred.low and low ≤ succ.low then
14:        if succ.low=low then
15:            if node.mark=true then
16:                node.mark ← false
17:                SUCCUNLOCK(succ), SUCCUNLOCK(pred)
18:                return true                        ▷ Linearization Point: Successful insert
19:            end if
20:            val1 ← node.instance.PUTIFABSENT(high, high)
21:            if high > node.max then
22:                node.max ← high
23:            end if
24:            SUCCUNLOCK(succ), SUCCUNLOCK(pred)
25:            if val1 ≠ null and high=node.max then
26:                UPDATEMAX1(node.parent)
27:            end if
28:            return (val1 = null)                   ▷ Linearization Point: Interval added successfully
29:        else if (low < node.low and node.left ≠ nil) or (low > node.low and node.right ≠
   nil) then
30:            SUCCUNLOCK(succ), SUCCUNLOCK(pred)
31:            continue                               ▷ Retry insertion
32:        else
33:            newNode ← NODE(low, high, nil)
34:            newNode.succ ← succ
35:            newNode.pred ← pred
36:            succ.pred ← newNode
37:            pred.succ ← newNode
38:            SUCCUNLOCK(succ), SUCCUNLOCK(pred)
39:            if low < node.low and node.left=nil then
40:                node.left ← newNode
41:                newNode.parent ← node
42:            else if low > node.low and node.right=nil then
43:                node.right ← newNode
44:                newNode.parent ← node
45:            end if
46:        end if
47:    else
48:        SUCCUNLOCK(succ), SUCCUNLOCK(pred)
49:        return false                               ▷ Linearization Point: Insertion failed
50:    end if
51:    while true do
52:        node ← SKEWANDSPLIT(node)
53:        if node=nil then
54:            break
55:        end if
56:        if node.parent ≠ nil then
57:            node ← node.parent
58:        else
59:            break
60:        end if
61:    end while
62:    if high > newNode.parent.max then
63:        UPDATEMAX1(newNode.parent)
64:    end if
65:    return true                                    ▷ Linearization Point: Insert operation completes
66: end procedure
```

Algorithm 3. Update Max Values After Insertion

```
1: procedure UpdateMax1(node)
2:     previous ← null, parent ← null
3:     leftMax, rightMax ← 0
4:     while true do
5:         parent ← node.parent
6:         if not TryLock(parent) then
7:             continue
8:         end if
9:         if not TryLock(node) then
10:            Unlock(parent)
11:            continue
12:        end if
13:        leftMax ← node.left.max
14:        rightMax ← node.right.max
15:        node.max ← max(leftMax, rightMax)
16:        if previous ≠ null then
17:            Unlock(previous)
18:        end if
19:        previous ← node
20:        node ← node.parent
21:        if previous.max ≤ node.max then
22:            break
23:        end if
24:    end while
25:    if previous ≠ null then
26:        Unlock(previous)
27:    end if
28:    if IsLockedByCurrentThread(parent) then
29:        Unlock(parent)
30:    end if
31: end procedure
```

on the outcome of this search, the algorithm determines the correct predecessor and successor nodes to which the new interval will be logically linked. To ensure consistency and avoid interference from other concurrent threads, the algorithm next acquires locks on both the predecessor and successor nodes. The locking mechanism is optimistic and non-blocking, if the lock on the successor cannot be obtained, the predecessor lock is released, and the procedure is retried. This retry strategy avoids deadlocks and minimizes contention in high-concurrency scenarios.

Once the locks are acquired, the algorithm checks whether the successor already contains a node with the same low value. If such a node exists and is logically marked for deletion, it is reactivated by setting its mark field to `false`. This step is considered a linearization point, as the previously deleted node becomes immediately visible and valid for future operations. If the node is active, the algorithm attempts to insert the high value into the node's `ConcurrentHashMap` using `putIfAbsent`. This data structure allows multiple high values to coexist under the same low key, thereby avoiding redundant node creation and efficiently representing overlapping intervals. If the insertion is successful, the node's max field is updated if the new high value exceeds the existing maximum. This successful update marks another linearization point, as the interval becomes visible to other threads.

If the low value does not already exist in the tree, a new node is created. The node is initialized with the given interval and is logically linked by updating the `succ` and `pred` pointers of its neighbors. Once these links are updated, the locks are released. The new node is then physically inserted into the tree by updating either the left or right child pointer of its parent, depending on its key. The logical linking of the node to its in-order neighbors constitutes another linearization point, as the node is now visible for range-based operations.

Following insertion, the AA Tree may become temporarily imbalanced. To restore its invariants, the algorithm invokes a sequence of skew and split operations, as prescribed in the CAA Tree structure [6]. These operations rebalance the tree using a simplified strategy that avoids complex rotations and supports concurrent updates more efficiently than traditional self-balancing trees. After structural balance is restored, the algorithm verifies whether the inserted high value is greater than the maximum field of the parent node. If so, the `updateMax1` procedure is invoked, as shown in Algorithm 3 to propagate the new maximum value up the tree. This update is carried out incrementally and employs fine-grained locking to reduce contention and avoid blocking concurrent operations. The propagation stops as soon as the max value of a parent node is found to be larger than or equal to the current value, ensuring minimal overhead.

The insert operation ends with `true`, indicating that the interval has been successfully added to the tree. This return point serves as the final linearization point, after which the interval is fully integrated and visible to all concurrent queries and updates. Together, the key linearization points in the insert procedure occur when a logically deleted node is reactivated, when a high value is successfully inserted into an existing node, when a new node is created and logically linked, and when the operation completes successfully. The algorithm design effectively balances correctness, responsiveness, and scalability, making it suitable for concurrent workloads with a long interval.

***Remove* Operation:** The `remove` operation in the CIT is responsible for deleting a specific interval $(low, high)$ while preserving the correctness and concurrency guarantees of the tree. Since a single node in CIT may contain multiple high values for the same low value, a successful removal often involves only deleting a high value from the node's internal map rather than removing the entire node. Structural updates are minimized and balancing is preserved through incremental adjustments of auxiliary fields, such as the maximum value. The algorithm, shown in Algorithm 4, is designed to support fine-grained concurrency through localized locking and logical deletion.

The operation begins by locating the node corresponding to the specified low value using a search procedure. If the node is not found, or if the high value does not exist within the node's `ConcurrentHashMap`, the operation terminates immediately by returning `false`. If the target node is located, it is locked to prevent concurrent modifications. This initial step ensures thread safety and consistency while examining and potentially modifying node content.

After acquiring a lock on the node, the algorithm proceeds to establish logical ordering by identifying and locking the node's predecessor and successor. These

Algorithm 4. Concurrent Remove Procedure in CIT

```
1: procedure REMOVE(low, high)
2:     node ← nil, parent ← nil, node1 ← nil
3:     pred, succ, Lnode ← nil
4:     while true do
5:         node ← SEARCH(low)
6:         if not TRYLOCK(node) then
7:             continue
8:         end if
9:         if node.low = low and CONTAINSKEY(node.instance, high) then
10:            pred ← (node.low ≥ low)?node.pred : node
11:            if not TRYSUCCLOCK(pred) then
12:                TREEUNLOCK(node)
13:                continue
14:            end if
15:            succ ← pred.succ
16:            if low > pred.low and low ≤ succ.low then
17:                if not TRYSUCCLOCK(succ) then
18:                    SUCCUNLOCK(pred), TREEUNLOCK(node)
19:                    continue
20:                end if
21:                if succ.low ≠ node.low then
22:                    SUCCUNLOCK(succ), SUCCUNLOCK(pred), TREEUNLOCK(node)
23:                    return false                    ▷ Linearization point: Interval not found
24:                end if
25:                node ← succ
26:                flag ← REMOVE(node.instance, high)
27:                if not ISEMPTY(node.instance) then
28:                    if high = node.max then
29:                        node.max ← FINDLARGESTVALUE(node.instance)
30:                    end if
31:                end if
32:                if SIZE(node.instance) = 0 then
33:                    node.mark ← true            ▷ Linearization point: Node logically deleted
34:                end if
35:                SUCCUNLOCK(succ), SUCCUNLOCK(pred)
36:                TREEUNLOCK(node)
37:                if high > node.max then
38:                    UPDATEMAX2(node.parent, node.max)
39:                end if
40:                return true                 ▷ Linearization point: Interval successfully removed
41:            else
42:                SUCCUNLOCK(pred), TREEUNLOCK(node)
43:                return false                ▷ Linearization point: Interval not found or reordered
44:            end if
45:        else
46:            TREEUNLOCK(node)
47:            return false                        ▷ Linearization point: Target node not found
48:        end if
49:    end while
50: end procedure
```

locks are acquired in sequence to avoid deadlock. If either lock acquisition fails, any previously acquired locks are released and the process is retried. Once both locks are successfully acquired, the algorithm verifies that the successor node still matches the expected low value to prevent inconsistencies due to concurrent modifications. If this check fails, the operation aborts and releases all locks.

If the successor node contains the desired low value and the high value is found within its instance map, the high value is removed. If additional high values remain in the node after removal, and if the removed high value was equal to the node's max field, the node's max value is updated using the largest remaining

high value. If the instance map becomes empty, the node is marked for deletion by setting its `mark` flag to `true`, indicating that the node is logically deleted but not yet physically removed. The marking step constitutes a key linearization point, signifying that the interval is no longer visible to other threads. After updating or marking the node, the algorithm releases all acquired locks to allow concurrent operations to proceed. If the removed high value was the maximum in its subtree, the algorithm invokes the `updateMax2` procedure (Algorithm 5) to propagate the new maximum value up the tree. This function iteratively moves up the parent chain, locking each node and updating its max field only when required. The propagation halts as soon as a parent node's existing max value is larger than the newly computed one. This lazy propagation technique reduces contention and ensures efficient maintenance of the max field across the tree's hierarchy. Once all updates have been applied, the operation concludes by returning `true`, indicating that the interval was successfully removed. If at any point the interval is not found or a locking failure occurs, the operation safely aborts and returns `false`. These return points serve as linearization boundaries that define the moment the remove operation takes effect or fails.

Algorithm 5. Update Max Values After Deletion

```
1: procedure UpdateMax2(node, new_max)
2:     previous ← null, parent ← null
3:     leftMax, rightMax, high ← node.max
4:     while true do
5:         parent ← node.parent
6:         if not TryLock(parent) then
7:             continue
8:         end if
9:         if not TryLock(node) then
10:            Unlock(parent)
11:            continue
12:        end if
13:        node.max ← new_max
14:        leftMax ← node.left.max
15:        rightMax ← node.right.max
16:        node.max ← max(leftMax, rightMax)
17:        if previous ≠ null then
18:            Unlock(previous)
19:        end if
20:        previous ← node
21:        node ← node.parent
22:        if high < node.max then
23:            break
24:        end if
25:    end while
26:    if previous ≠ null then
27:        Unlock(previous)
28:    end if
29:    if IsLockedByCurrentThread(parent) then
30:        Unlock(parent)
31:    end if
32: end procedure
```

Importantly, physical deletion of logically marked nodes is deferred to a dedicated thread for background maintenance. This thread continuously traverses

the tree in logical order and safely removes marked nodes only when they are leaves or have a single child, as discussed in prior work on the CAA Tree [6]. This deferred cleanup mechanism minimizes contention and avoids interference with primary operations such as insertions or queries, allowing the tree to maintain structural integrity and responsiveness in highly concurrent environments.

The remove operation introduces several critical linearization points throughout its execution. These include the moment the function determines that an interval is not present and returns `false`, the point at which the high value is successfully removed from the `ConcurrentHashMap`, the act of logically marking a node for deletion, and the propagation of updated max values via the `updateMax2` method. The final linearization point occurs when the function successfully completes and returns `true`. By combining logical deletion, fine-grained locking, and asynchronous cleanup, the remove operation ensures both correctness and scalability for interval deletions in multithreaded settings.

4 Correctness

The correctness of the CIT is established by inheriting the synchronization and consistency principles of the underlying CAA framework. All core operations—`insert`, `remove`, and `getAllOverlaps`—adhere to thread-safe execution and linearizability guarantees derived from the CAA's structured locking and logical ordering.

4.1 Synchronization

CIT reuses the fine-grained locking model of CAA, which separates logical ordering from physical structure using dual locks: `succLock` for logical links and `treeLock` for structural modifications. This design minimizes contention and supports parallelism. As in CAA, locks are acquired in a fixed order (predecessor, successor, then node) to prevent deadlocks. Synchronization is further aided by CIT's use of a thread-safe `ConcurrentHashMap` to manage multiple high endpoints per node without introducing locking overhead. Lazy deletion is inherited as well: deleted nodes are marked using a volatile `mark` flag and physically removed later by a background maintenance thread, ensuring non-blocking updates.

4.2 Linearizability

CIT defines clear linearization points for all operations, many of which are annotated directly in Algorithms 2 and 4. For `insert`, the operation is linearized when a logically deleted node is reactivated, when a new high value is successfully inserted into the map, or when a new node is linked into the logical order. In `remove`, linearization occurs when the high value is removed, when the node is logically marked deleted, or when the operation returns failure after validation. The `getAllOverlaps` operation is read-only and lock-free, and its traversal

over immutable fields (`low`, `max`, `instance`) ensures safe concurrent execution, maintaining linearizability without synchronization.

Overall, correctness in CIT is ensured by reusing CAA's proven concurrency control mechanisms and extending them with safe, scalable handling of interval-specific logic.

5 Experimental Evaluation

We conducted a comprehensive evaluation of the proposed CIT to assess its performance under diverse workloads and compare it with leading concurrent interval tree implementations. The experiments were designed to capture throughput, memory efficiency, and energy usage across varying dataset sizes and thread counts.

The evaluation used input datasets containing 2×10^5, 2×10^6, and 2×10^7 intervals, all generated using a uniform distribution with Java's built-in `random()` method. The experiments were carried out on a dual socket Intel(R) Xeon(R) Platinum 8260 server with 48 cores (96 threads), 512 GB of memory and

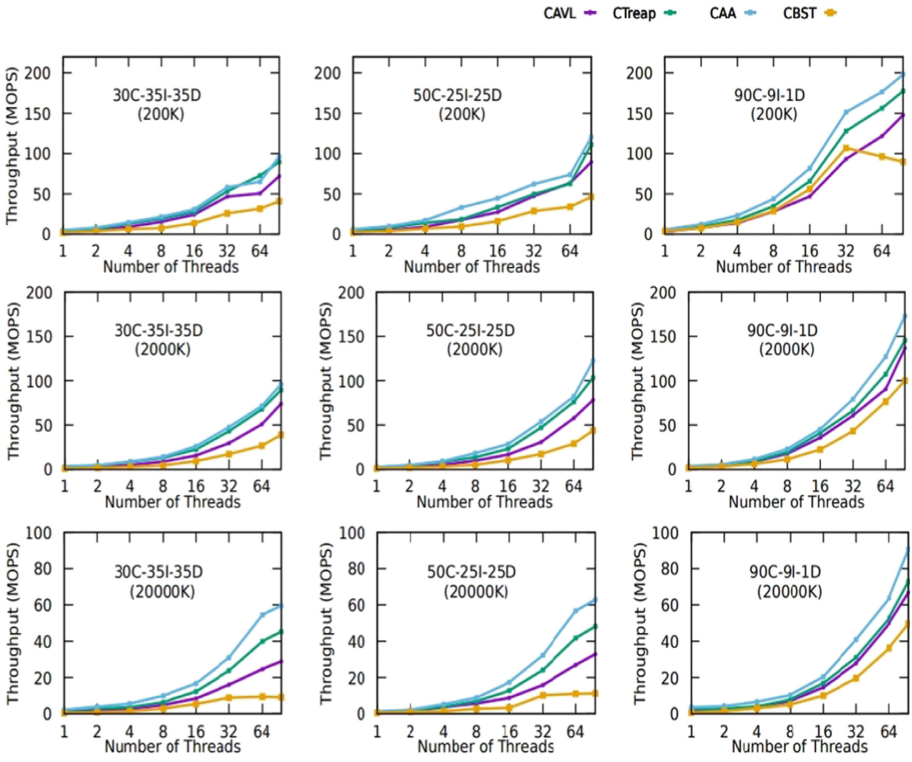


Fig. 4. The average throughput of concurrent interval search trees across different workloads and problem sizes by varying the number of threads. Higher is better.

Ubuntu 22.04.2 LTS, running Java implementations built using OpenJDK 19.0.2. To ensure the reliability of our benchmark results, we intentionally limited our measurements to 64 threads, despite the hardware's ability to support up to 96 simultaneous threads. This decision was driven by the need to minimize the influence of confounding factors such as operating system-level interference, artifacts introduced by hyperthreading, and various sources of measurement noise, such as increased resource contention, cache thrashing, and unpredictable scheduling overhead. By capping the thread count, we aimed to create a more controlled and reproducible testing environment, allowing our results to more accurately reflect the performance rather than being skewed by system-level artifacts.

To benchmark the performance of CIT using CAA, we compared it against three state-of-the-art concurrent interval tree implementations referenced in Chap. 5, Sect. 5.5 of this thesis [7]: (i) **CAVL**, a concurrent AVL tree using logical ordering [10], (ii) **CTreap**, a concurrent treap combining BST and heap properties [11], (iii) **CBST**, a contention-friendly binary search tree with hand-over-hand locking [12].

While more recent designs have emerged—for example Lock-Free Augmented Trees (2024) that provide augmentation for range queries and order statistics in a lock-free fashion [19], Wait-free Trees with Asymptotically-Efficient Range Queries (2023) supporting count-in-range operations [20], and EEMARQ (2022) offering linearizable range queries with lock-free reclamation [21]—these either do not directly handle interval-based queries or lack publicly available implementations for interval trees with overlapping intervals. Therefore, the chosen baselines remain the most relevant and reproducible for evaluating CIT.

Workload Distributions: We evaluated performance using three representative workloads, each denoted by the format XC-YI-ZD, where C, I, and D correspond to the respective proportions of `Contains`, `Insert`, and `Delete` operations. The selected workloads (30C-35I-35D, 50C-25I-25D and 90C-9I-1D) span a spectrum of access patterns, ranging from balanced read-write distributions to highly read-dominated scenarios. This diversity allows us to simulate varying contention profiles and stress different aspects of system behavior. By employing these standardized mixes consistently across all performance and system-level evaluations, we ensure a comprehensive and comparative assessment of the system under distinct operational conditions.

Throughput Analysis: Throughput, measured in millions of operations per second (MOPS), serves as the primary performance metric. Figure 4 reports the average throughput achieved by each implementation across multiple workloads and dataset sizes. CIT consistently outperforms the other structures, particularly at higher thread counts, due to its efficient synchronization model and relaxed balancing strategy.

The performance advantages of CIT, built on the CAA tree, stem from three key design choices. First, the use of multivalued nodes allows multiple intervals that share the same low endpoint to be stored compactly, significantly reducing tree height and frequency of rebalancing. Second, its internal use of `ConcurrentHashMap` enables fast, atomic access to high values within a node,

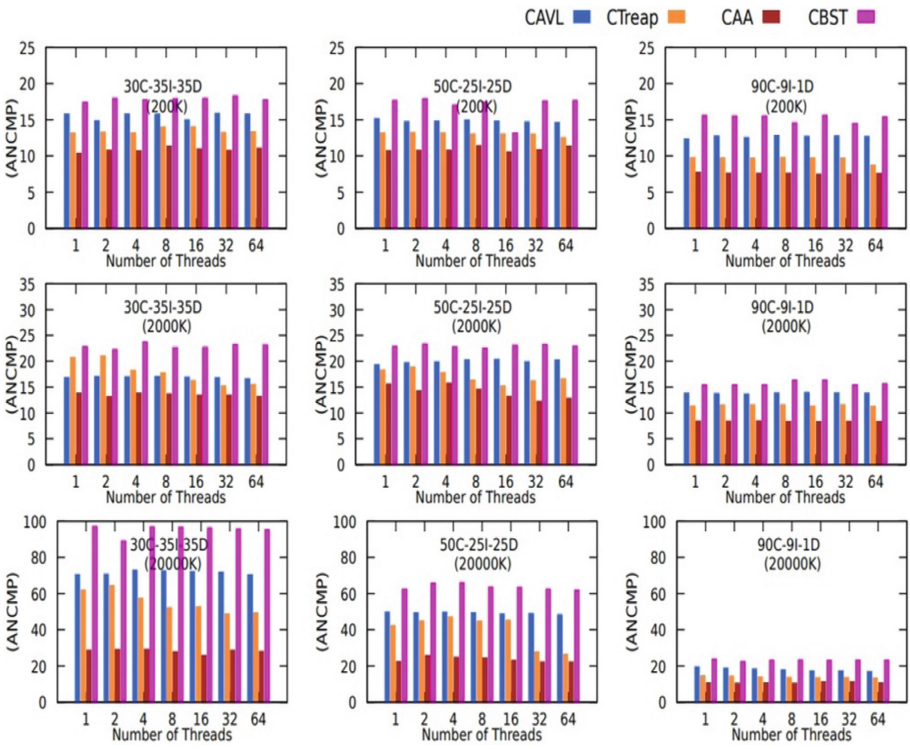


Fig. 5. The average number of cache misses per operation (ANCMP) across different workloads and problem sizes. Lower is better.

allowing frequent `contains` and `delete` operations to complete in constant time. Third, the lazy deletion mechanism delegates physical removals to a background maintenance thread, ensuring that insertions and queries can proceed without disruption. These optimizations lead to a highly scalable structure. On a dataset with 2×10^5 intervals, CIT achieves a 22.87× speedup over its sequential baseline with 64 threads. Against competing implementations, it delivers an average speedup of 1.3× over CAVL, 1.5× over CTreap, and 1.68× over CBST. These results demonstrate the resilience of the CIT in high contention and its suitability for concurrent interval workloads.

Analysis of Cache Misses: The behavior of the cache plays a critical role in the performance of concurrent data structures. Poor cache efficiency can introduce significant memory latency, especially when multiple threads access shared resources. We employed `perf` tool [14] to monitor cache references and cache misses. As a metric for cache efficiency, we used the Average Number of Cache Misses Per Operation (ANCMP). As shown in Fig. 5, CIT records fewer cache misses across all workloads, benefiting from improved spatial locality through compact node layouts and shorter traversal paths.

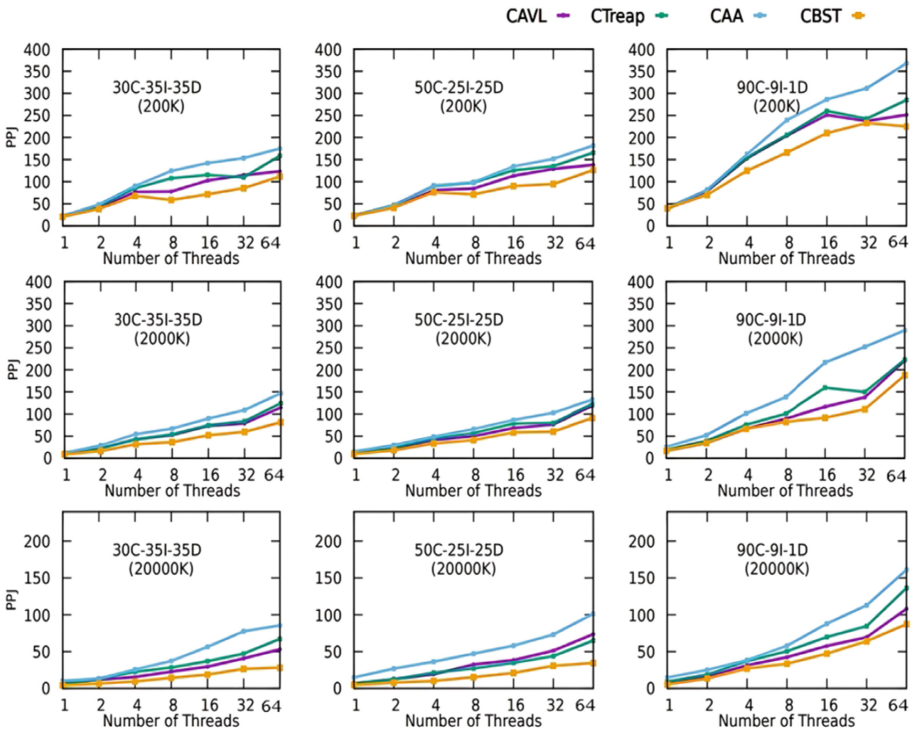


Fig. 6. The average performance per joule (PPJ) across different workloads and problem sizes, varying the number of threads from 1 to 64. Higher is better.

Analysis of Energy Consumption: Beyond throughput, we also evaluated the energy efficiency of each implementation. Energy measurements were obtained using the `jRAPL` framework [17,18], which provides access to processor-level energy counters via Intel's RAPL interface. We report energy efficiency in terms of Performance Per Joule (PPJ), defined as the number of operations executed per unit of energy. As illustrated in Fig. 6, CIT achieves higher energy efficiency than competing designs, due to its reduced contention, lightweight synchronization, and compact node representation. The experimental findings reinforce the effectiveness of the CIT design, built on top of the CAA tree. This streamlined execution delivers high throughput, improved cache location, and better energy efficiency across various workloads and input sizes. These characteristics make CIT a strong candidate for real-time interval management in high-performance multithreaded systems.

6 Conclusion

This work presents the design and implementation of a CIT, built upon the Concurrent AA Tree (CAA Tree), to efficiently support interval-based queries

in multithreaded environments. By incorporating a relaxed balancing strategy, lazy deletion mechanisms, and a multivalued node structure, the CIT minimizes synchronization overhead and contention while improving spatial locality and cache efficiency. A dedicated background maintenance thread handles deferred physical deletions, allowing insertions and deletions to proceed with minimal disruption to concurrent operations.

Experimental results demonstrate that the CAA-based interval tree consistently outperforms leading concurrent data structures such as CAVL trees, CTreaps, and CBSTs in throughput, cache efficiency, and energy usage across diverse workloads. The design exhibits excellent scalability and maintains its performance advantages even as the number of threads increases, making it well suited for modern multicore systems. As future work, we plan to perform ablation studies to quantify the contribution of each design choice and to evaluate the structure under skewed interval distributions (e.g., Gaussian, Zipfian), providing further insight into the robustness of CIT across real-world workloads.

References

1. Ferretti, L., Colajanni, M., Marchetti, M.: Distributed, concurrent, and independent access to encrypted cloud databases. IEEE Trans. Parallel Distrib. Syst. **25**(2), 437–446 (2013)
2. Pal, A., Pal, M.: Interval tree and its applications. Adv. Model. Optim. **11**(3), 211–224 (2009)
3. Bansal, N., Pruhs, K.: The geometry of scheduling. SIAM J. Comput. **43**(5), 1684–1698 (2014)
4. Bellman, R., Esogbue, A.O., Nabeshima, I.: Mathematical Aspects of Scheduling and Applications: Modern Applied Mathematics and Computer Science, vol. 4. Elsevier (2014)
5. Allard, M.W., et al.: Practical value of food pathogen traceability through building a whole-genome sequencing network and database. J. Clin. Microbiol. **54**(8), 1975–1983 (2016)
6. Attinagaramu, K.K., Alapati, P.: CAA: A concurrent AA tree via logical ordering. In: Proceedings of the 23rd International Symposium Parallel Distributing Computer (ISPDC), 2024, pp. 1–8 (2024)
7. Alapati, P.K.: Scalable and efficient concurrent binary search trees," Ph.D. dissertation, Indian Inst. Technol. Madras, Chennai, India (2020). [Online]. Available: http://hdl.handle.net/10603/410303
8. Herlihy, M.P., Wing, J.M.: Linearizability: a correctness condition for concurrent objects. ACM Trans. Program. Lang. Syst. **12**(3), 463–492 (1990)
9. Singh, V., Neamtiu, I., Gupta, R.: Proving concurrent data structures linearizable. In: Proceedings of thee 27th International Symposium Software Reliability Engineering (ISSRE), 2016, pp. 230–240 (2016)
10. Drachsler, D., Vechev, M., Yahav, E.: Practical concurrent binary search trees via logical ordering. In: Proceedings of the 19th ACM SIGPLAN Symposium Principles Practice Parallel Program, pp. 343–356 (2014)
11. Alapati, P., Saranam, S., Mutyam, M.: Concurrent treaps. In: Proceedings of 17th International Conference Algorithms Architect Parallel Process (ICA3PP), 2017, pp. 776–790 (2017). Springer

12. Sagonas, K., Winblad, K.: Contention adapting search trees. In: Proceedings of the 14th International Symposium Parallel Distribution Computer, pp. 215–224 (2015)
13. Babka, V., Marek, L., Tuma, P.: When misses differ: Investigating impact of cache misses on observed performance. In: Proceedings of the 15th International Conference on Parallel Distribution System, 2009, pp. 112–119 (2009)
14. Kufrin, R.: Perfsuite: An accessible, open source performance analysis environment for Linux, In: 6th International Conference Linux Clusters: The HPC Revolution **151**, 05 (2005)
15. Ramapantulu, L., Loghin, D., Teo, Y.M.: An approach for energy efficient execution of hybrid parallel programs. In: Proceedings of the IEEE International Parallel Distribution Process Symposium, 2015, pp. 1000–1009 (2015)
16. G. Pinto, K. Liu, F. Castor, and Y. D. Liu, "A comprehensive study on the energy efficiency of Java's thread-safe collections," in *Proc. IEEE Int. Conf. Softw. Maint. Evol. (ICSME)*, 2016, pp. 20–31
17. David, H., Gorbatov, E., Hanebutte, U.R., Khanna, R., Le, C.: RAPL: Memory power estimation and capping. In: Proceedings of the 16th ACM/IEEE International Symposium Low Power Electronics and Design, 2010, pp. 189–194 (2010)
18. Liu, K., Pinto, G., Liu, Y.D.: Data-oriented characterization of application-level energy optimization. In: Proceedings of the 18th International Conference Fundamental Approaches Software Engineering (FASE), 2015, pp. 316–331. Springer (2015)
19. Fatourou, P., Ruppert, E.: Lock-free augmented trees arXiv preprint arXiv:2405.10506 (2024)
20. Kokorin, I., Alistarh, D., Aksenov, V.: Wait-free trees with asymptotically-efficient range queries. arXiv preprint arXiv:2310.05293 (2023)
21. Sheffi, M., Ramalhete, P., Petrank, E.: EEMARQ: Efficient lock-free range queries with memory reclamation. *arXiv preprint*arXiv:2210.17086 (2022)

A Resource Allocation Method of Blockchain Network Based on Edge Computing

Qian Su, Longfei Bai, Xiang Zhang, Guangqin Hu, and Xuejie Zhang(✉)

School of Information Science and Engineering, Yunnan University, Kunming 650500, People's Republic of China
{suqian,zhxiang,xjzhang}@ynu.edu.cn

Abstract. The integration of blockchain and edge computing offers a promising solution to key challenges in Internet of Things (IoT) applications, including data security, transmission reliability, and computility demands. This paper focuses on the total system revenue in IoT application scenarios and proposes a resource allocation method for blockchain networks based on edge computing, which mitigates resource waste by adjusting block rewards. First, an IoT-blockchain architecture supported by the edge server's computilities is proposed. The revenue models for IoT devices and the edge server incorporating the Gompertz function are constructed, and the resource allocation problem is formulated as a nonlinear programming (NLP) with the objective of maximizing the total system revenue. The Gompertz function effectively characterizes the relationship between block rewards and total computilities, thereby suppressing resource waste caused by IoT devices' excessive pursuit of block rewards. Then, to address the challenge of solving this NLP problem, an improved particle swarm optimization algorithm based on chaotic dynamics and adaptive inertia parameters (CAPSO) is designed. The introduction of the Logistic chaotic mapping enhances the algorithm's global search ability, while the adaptive inertia weights improve convergence speed and solution accuracy. The experimental results demonstrate that the proposed method outperforms three traditional heuristic algorithms, achieving both higher system revenue and faster convergence.

Keywords: Edge computing · Blockchain network · Internet of Things · Resource allocation · Chaotic mapping · Particle swarm optimization

1 Introduction

The Internet of Things (IoT) is a vast system that integrates various objects, devices, services, technologies, and individuals through the Internet. By enabling interconnectivity, it facilitates data communication, information sharing, and collaborative interactions among all things, thereby optimizing resource allocation, improving production models, and enhancing user experiences. Currently,

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 79–100, 2026.
https://doi.org/10.1007/978-981-95-8405-5_5

IoT has been widely adopted in smart city applications such as transportation, finance, healthcare, agriculture, and logistics [1]. However, with the surge in the number of IoT devices and the amount of data transmitted between them, IoT applications face many challenges in terms of data integrity, transmission reliability, and other aspects [2,3].

Blockchain is inherently a tamper-proof, secure, and decentralized distributed ledger. By integrating distributed storage, consensus mechanisms, and cryptographic techniques, it offers robust solutions to data security challenges in IoT applications [4,5]. As a result, the convergence of IoT and blockchain has become an emerging trend across various domains, including finance, supply chain management, healthcare, and public governance [6].

Consensus mechanisms represent a core technology of blockchain. Current IoT-blockchain applications predominantly adopt Proof of Work (PoW) as the consensus algorithm, where blockchain nodes must complete substantial computational tasks to compete for the right to generate new blocks. However, when IoT devices participate in blockchain networks, their constrained computilities and limited battery capacity struggle to support PoW's operational demands.

Fortunately, the widely favored edge computing can alleviate the computational limitations of IoT devices to some extent by providing necessary computational resources through task offloading at locations close to these devices [7]. When confronted with substantial resource demands from numerous IoT devices, effective resource allocation of edge servers becomes crucial. First, IoT devices receive corresponding cryptocurrency rewards when obtaining block creation rights and writing transactions into blocks. To prevent IoT devices from being addicted to block rewards and causing computility waste, the resource allocation method should reasonably control the block rewards. Second, while receiving rewards, IoT devices must also pay corresponding fees to edge servers that provide computational resources. The resource allocation method should maximize total system revenue while meeting IoT devices' resource requests as much as possible. Existing research has made progress in winner determination, resource pricing, social welfare maximization, and cost minimization in resource allocation [8,9], but few studies have introduced reward control mechanisms to mitigate resource wastage in this process.

Based on the above discussion, this paper investigates the resource allocation problem for maximizing system revenue in IoT-blockchain application scenarios and proposes an edge computing-based resource allocation method for blockchain networks. The main contributions are summarized as follows:

– We construct an edge computing-powered IoT-blockchain architecture and formulate the resource allocation problem as a nonlinear programming model for total system revenue maximization. Within the revenue models for both IoT devices and the edge server, we incorporate the Gompertz function [10] from economics to characterize the relationship between block rewards and total computilities. This approach effectively mitigates resource waste from excessive reward-seeking by IoT devices by dynamically adjusting block

rewards through the Gompertz function's flexible response to computational demands.
– We design an enhanced particle swarm optimization algorithm based on chaotic dynamics [11] and adaptive parameters (CAPSO) to solve the resource allocation problem. Specifically, the Logistic mapping function from chaotic dynamics is introduced to enhance the algorithm's global search ability, while the adaptive inertia weights are employed to improve both convergence speed and solution accuracy. This tailored integration provides a specific solution to the NLP challenges in IoT-blockchain resource allocation, distinguishing it from generic swarm intelligence methods. Comparative experiments demonstrate that our algorithm achieves faster computation speed and yields higher system rewards compared to conventional heuristic approaches.

The remainder of this paper is organized as follows. Section 2 reviews related work on IoT resource allocation, blockchain-enabled and edge computing-assisted resource optimization. In Sect. 3, we present the system architecture, develop revenue models for both IoT devices and the edge server, and formulate the resource allocation problem as a mathematical programming. Section 4 proposes an improved particle swarm optimization algorithm incorporating chaotic dynamics and adaptive parameters, with detailed explanations of the algorithmic procedures. Section 5 evaluates the performance of our approach through comparative experiments with three heuristic algorithms. Finally, Sect. 6 concludes the work and outlines directions for future research.

2 Related Works

The rapid development of IoT technology has brought significant convenience to users while simultaneously elevating expectations for quality of service (QoS). In this context, efficient resource management and scheduling has become a critical approach to fulfill these growing demands. Existing research has demonstrated substantial progress in resource allocation optimization across various IoT application domains, particularly in Internet of Vehicles (IoV), smart healthcare systems, unmanned aerial vehicle (UAV) networks, and industrial IoT environments. Liang et al. [12] formulated the resource allocation problem in IoV as a semi-Markov decision process incorporating both a resource reservation strategy and a secondary resource allocation mechanism, and proposed a dedicated hierarchical reinforcement learning algorithm. Song et al. [13] proposed a secure and efficient fog-assisted wearable healthcare IoT system, which leverages physical layer security techniques and quantum-inspired resource allocation to enhance the confidentiality and performance of the system. Chen et al. [14] investigated the UAV deployment problem in UAV-assisted IoT networks with diverse QoS requirements for IoT devices, and proposed a QoS-guaranteed joint power allocation and data rate maximization placement scheme. Abdullah et al. [15] studied the resource allocation and task offloading problem in satellite-assisted IoT networks, considering energy efficiency, network throughput, and fairness, and

developed an outer approximation algorithm to obtain an ϵ-optimum solution. Chakraborty et al. [16] investigated the QoS-aware resource allocation problem in industrial IoT-enabled edge networks to ensure low delay, and high revenue for executing federated learning (FL). Then authors proposed a coalition game- and utility theory-based scheme to find the optimal solutions. The above studies have focused more on the performance of IoT systems, such as energy consumption, latency, and resource utilization, while lacking discussions on the security of data and applications.

Given the vast number of heterogeneous IoT devices with varying capabilities and security levels, device interconnection and data transmission face numerous potential security risks, including leakage of sensitive medical and social data, as well as tampering and forgery of critical engineering and commercial information [17]. Establishing trust among massive IoT devices in ubiquitous access networks to ensure lifecycle security and privacy is an urgent challenge for various IoT applications. Blockchain technology, fortunately, provides a secure and trustworthy environment for these requirements. Zhang et al. [18] proposed a blockchain-based FL framework to support trustworthy and reliable FL paradigms in IoT systems. Their approach incorporates an accuracy-driven incentive mechanism and a committee-based participant selection scheme, along with a deep reinforcement learning (DRL)-enabled resource allocation algorithm. Liang et al. [19] investigated the cost optimization for vehicular users in integrated IoT systems through vehicle-to-infrastructure (V2I) and vehicle-to-vehicle (V2V) transmission tasks, and proposed a blockchain-based vehicular identity authentication algorithm and a delay-weighted fair bandwidth allocation algorithm. Hou et al. [20] proposed a distributed Gale-Shapley spectrum allocation algorithm based on matching theory, which addresses the complexity of battlefield frequency usage and the high requirements for real-time and mobility. The proposed method features blockchain-enabled decentralized management and smart contract-based automated allocation. Chen et al. [21] proposed an intelligent computation offloading scheme for joint resource allocation and control in big-data IoT scenarios, in which a blockchain-based fair incentive mechanism and a dual actor-critic neural network structure with inverse gradient updates are designed.

The aforementioned studies reveal that due to the resource limitations of IoT devices and the high computility demands of blockchain node deployment, tasks often need to be completed with the assistance from entities with relatively abundant resources. The integration of blockchain and edge computing can realize reliable and incentive-aware access to distributed edge resources, while effectively coordinate and control the interaction behaviors between IoT devices [22,23]. To achieve resource allocation and privacy preservation in KubeEdge-based edge computing, Lingayya et al. [24] designed a privacy-preserving, multi-layered blockchain-enabled data aggregation mechanism that ensures secure and transparent transactions while enhancing trust in dynamic edge environments. Zhang et al. [25] investigated computation offloading in a mobile edge computing-enabled blockchain system involving one edge computing allocation center, mul-

tiple edge service providers, and numerous miners. They proposed a two-phase offloading strategy combining contract theory with matching game to incentivize participation from both edge service providers and miners in market transactions. Wang et al. [26] proposed a UAV-assisted MEC network architecture and a comprehensive data processing flow, where UAVs collaborate with base stations to serve as edge servers and blockchain nodes. The study addressed an optimization problem minimizing total data processing time by jointly considering UAV positioning, data offloading, and resource allocation. Zhang et al. [27] investigated a D2D-assisted edge offloading strategy for blockchain mining tasks. To mitigate consensus failure risks caused by device mobility, the movement trajectories were predicted. To determine the optimal resource demands and pricing, the interaction between miners and suppliers was modeled as a Stackelberg game. Xie et al. [28] introduced edge computing to enhance blockchain's computational capabilities, and proposed a breakeven-based truthful double auction mechanism and a critical value-based double auction mechanism to handle supply-demand matching and resource allocation respectively.

To conclude, research on integrating blockchain with edge computing in IoT applications can be summarized into the following categories: blockchain provides a truthful platform for edge resource deployment and transactions, edge computing offers sufficient computilities for blockchain consensus mechanisms, and block reward-based incentive mechanisms promote the maximization of system utility or social welfare in edge resource allocation. Inspired by this, we propose an edge resource allocation method with controlled block rewards. Unlike existing studies that utilize block rewards to encourage IoT devices and edge servers to participate in trading activities, this paper applies the Gompertz function for the first time to dynamically adjust block rewards according to computilities, thereby preventing resource waste caused by IoT devices over-requesting resources due to reward addiction. The resource waste here refers to the failure of over-requesting resources to exert their effectiveness, which differs from the resource efficiency mentioned in previous studies. Resource efficiency emphasizes high resource utilization and minimal idle resources. Furthermore, while chaotic mapping has been widely applied in medical diagnosis, image encryption, power systems, and engineering design [11], its application in blockchain networks for edge resource allocation to IoT devices remains unexplored. We employ the simplest chaotic mapping to enhance the diversity of solutions during resource allocation, helping particles escape local optima and potentially accelerating convergence.

3 System Modeling and Problem Formulation

In this section, we first present the system architecture of the IoT-blockchain network, then develop revenue models for both IoT devices and the edge server, and finally formulate the resource allocation problem as a nonlinear programming model aimed at system revenue maximization. All notations used are summarized in Table 1.

Table 1. Notation description

Notation	Description
N	Number of IoT devices
q_i	IoT device i's request tuple
r_i	Computational Resources Requested by i
d_i	Unit resource price paid by i
x_i	Decision variable
l_i	i's local computilities
π_i	Cost coefficient of i's local computilities
S^{R}	Resource capacity of the edge server
μ	Cost coefficient of the edge server
B_i	Block reward of i
B_i^{ES}	Block reward shared by i to the edge server
$C_i(C^{\mathrm{ES}})$	Expenditure of i (the edge server)
$R_i(R^{\mathrm{ES}})$	Revenue of i (the edge server)
$\theta(b_i)$	Block propagation time function with respect to b_i
b_i	Size of the new block generated by i
s_i	Number of transactions in i's new block
p_i	Probability that i successfully generates a new block and adds it to the blockchain
t_i	i's transaction fee rate
ϕ	Total computilities participating in PoW
$g(\phi)$	Block reward distribution function with respect to ϕ
H	Difficulty coefficient for obtaining block reward
A	Fixed issuance reward
ξ	Chaos parameter
λ	Poisson distribution parameter
σ	Delay factor
a	Size of the block header
κ	Block size-transaction count correlation coefficient
$\alpha/\beta/\gamma$	Gompertz function coefficients

3.1 System Architecture

We consider an edge computing-powered IoT-blockchain system as illustrated in Fig. 1. The system comprises a set of lightweight blockchain nodes $\{1, 2, ..., N\}$ consisting of N IoT devices, and an edge server deployed in proximity to these nodes. The IoT devices communicate with the edge server via wireless networks for data transmission. Each IoT device maintains a local copy of the blockchain ledger, with all distributed ledgers synchronized through a consensus algorithm.

The system operation proceeds through four sequential phases:

(1) Resource request. Due to the relatively limited computational capabilities of IoT devices, when performing PoW, in addition to their local computilities l_i,where $i \in \{1, 2, ..., N\}$), they also need to request additional computational

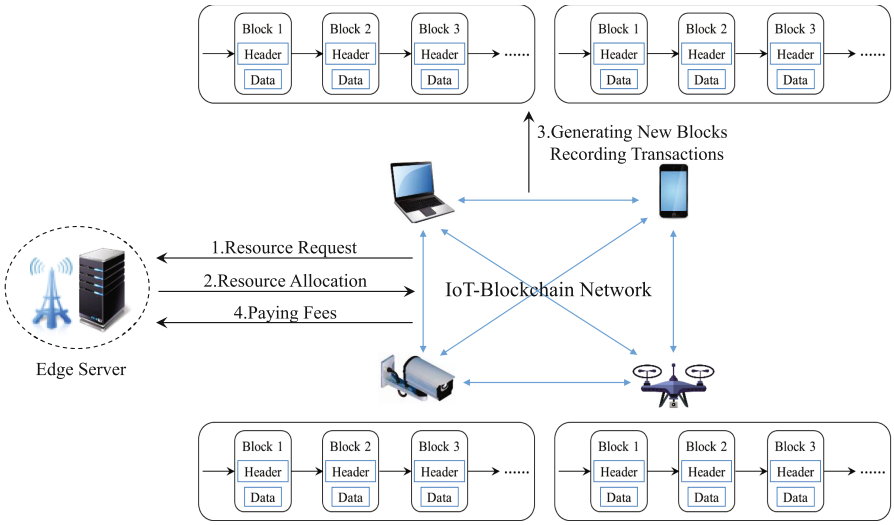


Fig. 1. The edge computing-powered IoT-blockchain system.

resources from the edge server. Let $q_i = \{r_i, d_i\}$ denote the request tuple submitted by IoT device i to the edge server, where r_i represents the amount of computational resources requested, and d_i is the unit price device i is willing to pay for the resources.

(2) Resource allocation. The edge server acts as both a resource provider and a decision-maker. Upon receiving requests from IoT devices, it determines the resource allocation strategy based on the overall revenue of the system. Let x_i be the decision variable representing the amount of computational resources allocated by the edge server to IoT device i. To avoid resource wastage, the allocated resources should not exceed the amount requested by device i, that is,

$$0 \leq x_i \leq r_i, \forall i \in 1, 2, ..., N. \tag{1}$$

Second, the total resources allocated to all IoT devices are constrained by the edge server's available capacity S^{R}, satisfying

$$\sum_{i=1}^{N} x_i \leq S^{\mathrm{R}}. \tag{2}$$

Furthermore, in large-scale systems, the aggregate resource requests from IoT devices surrounding the edge server typically far exceed its resource capacity, thus yielding

$$\sum_{i=1}^{N} r_i > S^{\mathrm{R}}. \tag{3}$$

(3) Generating new blocks and recording transaction. After obtaining resources, IoT devices perform PoW. The device that earns the block qualification

writes transactions into a new block and broadcasts it across the blockchain network. Subsequently, all IoT devices update their respective copies of the blockchain ledger accordingly.

(4) Paying fees. Upon completion of these procedures, each IoT device pays the edge server two types of fees: the resource cost $d_i \cdot x_i$, and its shared portion of the block reward (i.e., the mining reward) $B_i^{\mathrm{ES}} = B_i \cdot (1 - \rho_i)$, where ρ_i is the proportion of the block reward retained by IoT device i, and B_i denotes the reward earned by IoT device i for generating a new block.

The resource allocation problem addressed in this paper aims to maximize the total system revenue. Next, we will elaborate on the revenue models for both IoT devices and the edge server.

3.2 Revenue Modeling for IoT Devices

The revenue R_i of IoT device i is the difference between its income and expenditure. The income consists of i's block reward, while the expenditure includes not only the two types of fees described in Step (4) of Sect. 3.1 but also the costs related to energy and resources incurred by the device's own computilities. Let π_i be the cost coefficient, then the expenditure and revenue of device i are respectively given by:

$$C_i = d_i \cdot x_i + B_i^{\mathrm{ES}} + \pi_i \cdot l_i, \tag{4}$$

$$R_i = B_i - C_i. \tag{5}$$

Therefore, the total revenue of all IoT devices in the system can be expressed as:

$$R^{\mathrm{IoT}} = \sum_{i=1}^{N} R_i. \tag{6}$$

Next, we provide a detailed discussion on the block reward B_i for IoT device i.

The size of a block is related to the number of transactions recorded in it. Let b_i be the size of the new block generated by IoT device i, and s_i be the number of transactions recorded in the new block. Their relationship can be expressed by a linear function:

$$b_i = \kappa \cdot s_i + a. \tag{7}$$

Here, κ is the correlation coefficient, representing the contribution of each transaction to the block size, and a denotes the size of the fixed portion of the block structure, including metadata.

In practical applications, even if an IoT device generates a new block, it does not guarantee that the block will be successfully added to the blockchain. When a node generates a new block, it broadcasts the block to the entire network so that other nodes can verify it and update their ledgers. However, multiple nodes may generate blocks simultaneously, in which case the network determines which block is added to the main chain based on propagation delays and information

synchronization. The Poisson distribution is widely used in the literature to model block generation intervals and orphan block rates in blockchain systems [29]. Consequently, we assume that the process of an IoT device generating a new block and successfully adding it to the blockchain follows a Poisson distribution. The probability of success is $p_i = e^{-\lambda \cdot \theta(b_i)}$, where λ is the Poisson distribution parameter, $\theta(b_i) = \sigma \cdot b_i$ denotes the block propagation time, and σ is a given delay factor. The parameter λ encompasses uncertain factors like network congestion and node computility distribution, while the propagation delay $\theta(b_i)$ is the core determinant of this probability. The larger the block, the slower it propagates, and the higher the risk of being replaced by other competing blocks (i.e., becoming an orphan block). Therefore, the probability p_i effectively captures the risk that a block generated by IoT device i fails to be appended to the chain due to propagation delay.

Since the value of cryptocurrency is influenced by various factors, it is challenging to model the variation in block rewards using precise mathematical formulas. Therefore, this paper focuses solely on the relationship between block rewards and the total computilities participating in PoW. Generally, the more computilities is used, the greater the rewards provided by the system. However, this may lead IoT devices to become overly fixated on rewards, continuously requesting resources from the edge server, thereby causing resource contention and waste. To address this, we introduce the Gompertz function from economics to describe the functional relationship between block rewards and the total mining computilities:

$$g(\phi) = \alpha \cdot e^{-\beta \cdot e^{-\gamma \cdot \phi}}, \tag{8}$$

where ϕ represents the total computilities participating in PoW, i.e., $\phi = \sum_{i=1}^{N} l_i + \sum_{i=1}^{N} x_i$. The function $g(\phi)$ denotes the reward distibution varying with ϕ, with a value range of [0,1]. Parameters α, β and γ are positive constants, where α represents the upper limit of the reward distribution ratio (i.e., the maximum proportion of rewards the system is willing to allocate), β controls the offset at which the reward begins to increase significantly, and γ governs the growth rate of the reward with respect to computility, respectively.

The Gompertz function is traditionally employed to model "S-shaped" phenomena, such as economic and population growth, where growth rates initially accelerate, then decelerate, and finally saturate [10]. This behavior closely matches our reward control objective: initial investments in computility yield significant reward growth to incentivize participation, but as the total computilities expand, the reward growth diminishes and asymptotically approaches the upper limit α. This mechanism effectively curbs the inefficient resource investment that results from an excessive pursuit of rewards.

In conclusion, the block reward for IoT device i can be derived as:

$$B_i = \frac{l_i + x_i}{\phi \cdot H} \cdot p_i \cdot g(\phi) \cdot (A + s_i \cdot t_i), \tag{9}$$

Here, $\frac{l_i + x_i}{\phi}$ is the proportion of computilities utilized by IoT device i, and H is the difficulty coefficient for completing PoW to obtain the block reward. To main-

tain a stable block generation rate, the blockchain system dynamically adjusts the difficulty coefficient based on the total computilities on the network, i.e., $H = \frac{\eta \cdot H_0}{\phi}$, where H_0 is the initial difficulty, and η is an adjustable parameter. $A + s_i \cdot t_i$ represents the initial block reward, which consists of a fixed issuance reward A and the allocation reward $s_i \cdot t_i$ for writing transactions into the new block, with t_i is the transaction fee rate.

3.3 Revenue Modeling for the Edge Server

The income of the edge server is the sum of the resource costs paid by IoT devices it serves and a portion of the block rewards, that is,

$$I^{\mathrm{ES}} = \sum_{i=1}^{N} (d_i \cdot x_i + B_i^{\mathrm{ES}}). \tag{10}$$

Its expenditure primarily consists of energy and resource costs associated with providing computilities. Let μ denote the cost coefficient of the edge server, then the expenditure can be expressed as:

$$C^{\mathrm{ES}} = \mu \cdot \sum_{i=1}^{N} x_i. \tag{11}$$

Thus, the edge server's revenue can be calculated as:

$$R^{\mathrm{ES}} = I^{\mathrm{ES}} - C^{\mathrm{ES}}. \tag{12}$$

3.4 Revenue-Maximizing Resource Allocation Model

Based on Eqs. (6) and (10), the total system revenue is defined as the sum of revenues from all IoT devices and the edge server, expressed as:

$$R^{\mathrm{TOTAL}} = R^{\mathrm{IoT}} + R^{\mathrm{ES}} = \frac{g(\phi)}{\phi \cdot H} \sum_{i=1}^{N} (l_i + x_i) \cdot e^{-\lambda \cdot \theta(b_i)} \cdot (A + s_i \cdot t_i) - \sum_{i=1}^{N} (\mu \cdot x_i + \pi_i \cdot l_i). \tag{13}$$

The blockchain network resource allocation problem aiming at system revenue maximization can be formulated as the following nonlinear programming (NLP):

$$\max_{\boldsymbol{x}} f = \frac{g(\phi)}{\phi \cdot H} \sum_{i=1}^{N} (l_i + x_i) \cdot e^{-\lambda \cdot \theta(b_i)} \cdot (A + s_i \cdot t_i) - \sum_{i=1}^{N} (\mu \cdot x_i + \pi_i \cdot l_i) \tag{14}$$

s.t. (1) − (3).

The first term of the objective function (14) reveals that the edge server tend to allocate computational resources to IoT devices whose newly generated blocks propagate faster through the network and contain more recorded transactions

in the new blocks. The constraints (1)-(3) have been elaborated in Sect. 3.1 and thus will not be reiterated here.

The objective function (14) is a NLP containing both exponential term $g(\phi)$ and fractional term $\frac{1}{\phi}$ of the unknown variable x_i. This characteristic implies that obtaining an optimal solution for this optimization problem is non-trivial and multiple local optima may exist. To address these challenges, we will develop an improved particle swarm optimization algorithm to efficiently obtain near-optimal solutions.

4 Improved Particle Swarm Optimization Algorithm

Particle Swarm Optimization (PSO) has been widely adopted for solving optimization problems owing to its simple implementation and rapid search speed. Nevertheless, this method has notable limitations including susceptibility to local optima convergence and significant performance dependence on constant inertia factors. To overcome these shortcomings, this paper proposes a improved particle swarm optimization algorithm based on chaotic dynamics and adaptive parameters (CAPSO) for solving the aforementioned resource allocation problem, as detailed in Algorithm 1. The proposed approach utilizes the Logistic chaotic mapping to enhance search capability and population diversity, thereby preventing premature convergence. Additionally, it implements an iteration-dependent adaptive inertia weight strategy that transitions from large to small parameter values during the optimization process, effectively improving both convergence speed and solution accuracy.

In Algorithm 1, particles search for the optimal position by continuously moving in an N-dimensional space. Each particle's position represents a feasible solution $(x_1, x_2, \cdots, x_N)$ to the problem, while its fitness value corresponds to the objective function value under this solution, namely the total system revenue. After at most $I_{\max}$ iterations, the global optimal position $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$ found by the algorithm serves as the global optimal solution to the problem, and the optimal fitness value $R^{\mathrm{g}}_{\mathrm{best}}$ is the maximum total system revenue.

The specific steps of Algorithm 1 are described as follows:

First, line 1 generates an initial set of solutions and corresponding initial system revenue within the solution space in $O(L \cdot N)$ time. The position matrix $\boldsymbol{X}$, velocity matrix $\boldsymbol{V}$, and chaotic matrix $\boldsymbol{C}$ for L particles are initialized randomly, all of which are $L \times N$-dimensional matrices. Each row in $\boldsymbol{C}$ is a random vector introduced for each particle to avoid falling into local optima. The initial fitness values $\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}$ (an L-dimensional vector) are calculated according to the objective function (14), with each dimension recording a particle's local optimum. The initial local optimal positions for all particles $\boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}}$ (an $L \times N$ matrix) are set to $\boldsymbol{X}$, while the initial global optimal fitness value $R^{\mathrm{g}}_{\mathrm{best}}$ is assigned the maximum value in $\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}$, with its corresponding position designated as the global optimal solution $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$ (an N-dimensional vector).

Subsequently, lines 2–21 employ a two-layer loop to update each particle's position with a time complexity of at most $O(L \cdot I_{\max})$, aiming to identify the

global optimal solution and its corresponding optimal value. The outer loop runs at most $I_{\max}$ iterations, dynamically adjusting the inertia weight w once per iteration (line 3). As iterations progress, w gradually decreases, indicating that each particle's movement distance is progressively shortened to prevent overshooting the optimal position due to excessive step size.

Algorithm 1: CAPSO

Input: population size L, number of IoT devices N, resource capacity of the edge server S^{R}, Maximum iteration $I_{\max}$, directional accelerations c_1 and c_2, upper and lower bounds of inertia weight $w_{\max}$ and $w_{\min}$, upper and lower bounds of decision variables $x_{\max}$ and $x_{\min}$

Output: global optimal position $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$ and optimal fitness value $R^{\mathrm{g}}_{\mathrm{best}}$

1 Initialization: $\boldsymbol{X}$, $\boldsymbol{V}$, $\boldsymbol{C}$, $\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}} \leftarrow f(\boldsymbol{X})$, $\boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}} \leftarrow \boldsymbol{X}$, $R^{\mathrm{g}}_{\mathrm{best}} \leftarrow \max_{N}\{\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}\}$, and $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} \leftarrow \arg\max\{\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}\}$;
2 **for** $i_1 = 1, 2, \cdots, I_{\max}$ **do**
3 $\quad w \leftarrow w_{\max} - \frac{(w_{\max} - w_{\min}) \cdot i_1}{I_{\max}}$;
4 $\quad \boldsymbol{X}^{\mathrm{g}*}_{\mathrm{best}} \leftarrow \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$;
5 $\quad R^{\mathrm{g}*}_{\mathrm{best}} \leftarrow R^{\mathrm{g}}_{\mathrm{best}}$;
6 $\quad$ **for** $i_2 = 1, 2, \cdots, L$ **do**
7 $\quad\quad \boldsymbol{C}[i_2] \leftarrow \xi \cdot \boldsymbol{C}[i_2] \cdot (1 - \boldsymbol{C}[i_2])$;
8 $\quad\quad \boldsymbol{V}[i_2] \leftarrow w \cdot \boldsymbol{V}[i_2] + c_1 \cdot \tau \cdot (\boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}} - \boldsymbol{X}[i_2]) + c_2 \cdot \tau \cdot (\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} - \boldsymbol{X}[i_2])$;
9 $\quad\quad \boldsymbol{X}[i_2] \leftarrow \boldsymbol{X}[i_2] + \boldsymbol{V}[i_2] + \boldsymbol{C}[i_2]$;
10 $\quad\quad \boldsymbol{X}[i_2] \leftarrow \max\{\min\{\boldsymbol{X}[i_2], x_{\max}\}, x_{\min}\}$;
11 $\quad\quad R^{\mathrm{p}}_{\mathrm{cur}} \leftarrow f(\boldsymbol{X}[i_2])$;
12 $\quad\quad$ **if** $R^{\mathrm{p}}_{\mathrm{cur}} > \boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}[i_2]$ **then**
13 $\quad\quad\quad \boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}}[i_2] \leftarrow \boldsymbol{X}[i_2]$;
14 $\quad\quad\quad \boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}[i_2] \leftarrow R^{\mathrm{p}}_{\mathrm{cur}}$;
15 $\quad\quad$ **if** $R^{\mathrm{p}}_{\mathrm{cur}} > R^{\mathrm{g}}_{\mathrm{best}}$ **then**
16 $\quad\quad\quad \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} \leftarrow \boldsymbol{X}[i_2]$;
17 $\quad\quad\quad R^{\mathrm{g}}_{\mathrm{best}} \leftarrow R^{\mathrm{p}}_{\mathrm{cur}}$;
18 $\quad$ **if** $\sum_N \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} > S^{\mathrm{R}}$ **then**
19 $\quad\quad \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} \leftarrow \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} \cdot (\frac{C^{\mathrm{ES}}}{\sum_N \boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}})$;
20 $\quad$ **if** $\left|\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} - \boldsymbol{X}^{\mathrm{g}*}_{\mathrm{best}}\right| < \epsilon$ **and** $\left|R^{\mathrm{g}}_{\mathrm{best}} - R^{\mathrm{g}*}_{\mathrm{best}}\right| < \epsilon$ **then**
21 $\quad\quad$ break;
22 **return** $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$, and $R^{\mathrm{g}}_{\mathrm{best}}$.

Before entering the inner loop, the algorithm has to record the previous iteration's optimal position $\boldsymbol{X}^{\mathrm{g}*}_{\mathrm{best}}$ and fitness value $R^{\mathrm{g}*}_{\mathrm{best}}$ (lines 4–5) to determine whether to terminate the outer loop early (lines 20–21). If the differences in both the optimal position and fitness value between two consecutive outer iterations are smaller than a predefined small positive threshold ϵ, it indicates that the algorithm has converged, and thus the outer loop is terminated prematurely.

Within the inner loop, each particle's current position $\boldsymbol{X}[i_2]$ is updated based on its previous position by adjusting both the chaotic vector $\boldsymbol{C}[i_2]$ and velocity $\boldsymbol{V}[i_2]$ (lines 7–9). The chaotic vector $\boldsymbol{C}[i_2]$ is used to enhance the search randomness and is updated via the Logistics mapping function. Here, the tunable parameter $\xi \in [0, 4]$ ensures that each chaotic value generated by the mapping remains within the interval [0,1]. The particle velocity, which represents its movement direction, is determined by three vectors: the inertia direction $\boldsymbol{V}[i_2]$, the local optimal direction $\boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}} - \boldsymbol{X}[i_2]$, and the global optimal direction $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}} - \boldsymbol{X}[i_2]$. The inertia weight w indicates the significance of the original movement direction, while c_1 and c_2 serve as weighting coefficients for the local and global optimal directions, respectively, with τ being a random parameter. A larger value of any of these coefficients (w, c_1, c_2 or τ) results in the particle exhibiting a stronger tendency to move in the corresponding direction.

To satisfy constraint (1), the algorithm bounds the updated particle position, where $x_{\max}$ corresponds to the current particle's resource request r_{i_2}, and $x_{\min}$ is set to 0 (line 10). The fitness value $R^{\mathrm{p}}_{\mathrm{cur}}$ of the current particle is then computed using the objective function (14) (line 11). Through comparison, the algorithm decides whether to update the particle's local optimal position $\boldsymbol{X}^{\mathrm{p}}_{\mathrm{best}}[i_2]$ and fitness $\boldsymbol{R}^{\mathrm{p}}_{\mathrm{best}}[i_2]$, as well as the global optimal position $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$ and fitness $R^{\mathrm{g}}_{\mathrm{best}}$ (lines 12–17). If the global optimal solution $\boldsymbol{X}^{\mathrm{g}}_{\mathrm{best}}$ from any outer iteration violates constraint (2), resources are reallocated to each IoT device proportionally (lines 18–19).

As can be seen from the above analysis, the overall time complexity of Algorithm 1 is at most $O(L \cdot (N + I_{\max}))$, meaning it can obtain an approximate optimal solution to the problem in polynomial time.

Moreover, Algorithm 1 is not an in-place algorithm and requires auxiliary space during its execution. The initialization phase requires $O(L \cdot N)$ space for relevant matrices, vectors, and variables, while the *for* loop uses an additional $O(N)$ space for temporary vectors and variables. Therefore, the overall space complexity of the algorithm is $O(L \cdot N)$.

5 Experiments and Results Analysis

We conducted simulation experiments on resource allocation in a blockchain network scenario consisting of one edge server and multiple IoT devices using the MATLAB platform. The experimental parameter settings are specified in Table 2. Each group of experiments was executed 50 times with randomized samples, and the results were averaged to mitigate the influence of randomness.

The comparative baselines include the traditional Particle Swarm Optimization (PSO), Genetic Algorithm (GA), and Simulated Annealing (SA). In PSO, the inertia weight was fixed at 0.5, while the two acceleration constants were the same as those in CAPSO. For GA, the initial population size was set to 10 with crossover and mutation probabilities of 0.8 and 0.02, respectively. The SA algorithm was configured with an initial temperature of 100 and a cooling rate of 0.9.

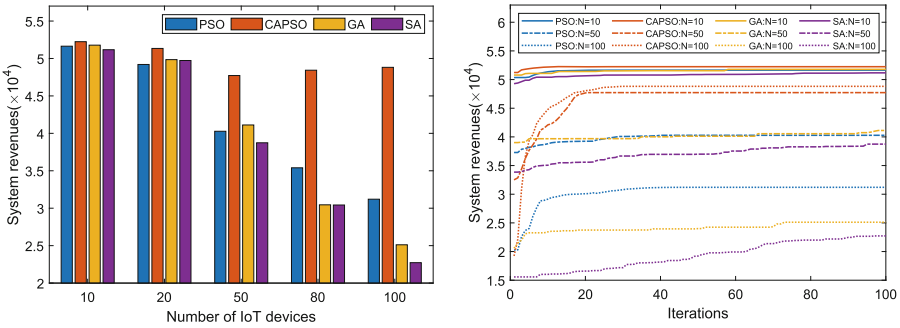
Table 2. Main parameter configuration

Parameter	Value	Parameter	Value
l_i	$[0, 100]$	μ	1
r_i	$[100, 1200]$	κ	0.1
π_i	$[0.1, 1]$	a	1
s_i	$[50, 100]$	α	1
t_i	$[0.01, 0.1]$	β	$[1, 2]$
λ	0.1	γ	$[0.003, 0.007]$
σ	0.005	ξ	$[0, 4]$
A	80000	H	$[0.8, 3]$

The evaluation metrics encompass both total system revenue and convergence rate. Maximizing the total system revenue is the primary objective of the resource allocation problem in this paper, and the results obtained by four methods are all near-optimal values. Convergence rate is measured by the number of iterations required for system revenue stabilization, which reflects execution efficiency of each algorithm. For CAPSO, this specifically refers to the number of outer-loop iterations.

5.1 Impact of IoT Device Scale

In this set of experiments, we examined the impact of the scale of IoT devices changing from 10 to 100 on the performance of several algorithms. The swarm size of PSO and CAPSO, the resource capacity of the edge server, the maximum iterations, the mining difficulty, and the chaos parameter were set to 30, 10000, 100, [1, 2], and 4, respectively.



(a) Comparison of total system revenues (b) Comparison of algorithm convergence

Fig. 2. The impact of IoT device scale.

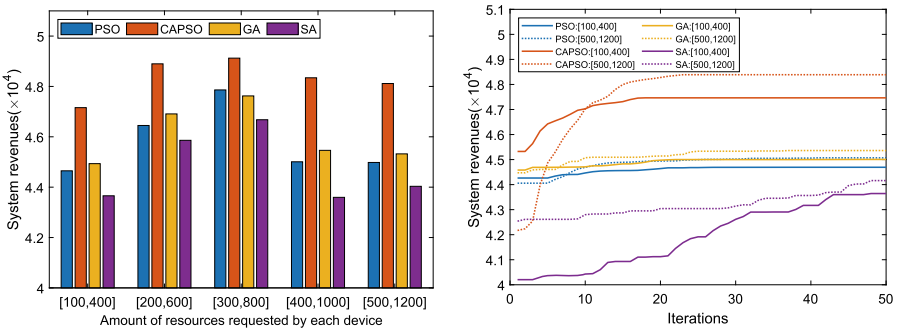
Figure 2(a) illustrates that as the number of IoT devices increases, the total system revenue obtained by CAPSO initially decreases gradually before showing a slow upward trend. In contrast, the other three methods exhibit a rapid decline in system revenue. Notably, CAPSO consistently maintains the highest total system revenue among all four approaches, indicating it is more suitable for resource allocation in large-scale applications. The performance gap becomes more pronounced with increasing IoT devices, as traditional heuristic algorithms are more prone to becoming trapped in local optima. While these suboptimal solutions allocate resources to most devices, the fixed resource capacity of the edge server leads to progressively diminishing individual allocations, significantly reducing the probability of completing PoW and obtaining mining rewards. For CAPSO, the ergodicity and randomness of the Logistic chaotic mapping effectively mitigate the algorithm's tendency to fall into local optima while enabling concentrated resource allocation to devices with high block rewards and low operational costs. Consequently, when the number of IoT devices grows from 10 to 50, the decrease in total revenue is not significant. As the number of devices continues to increase, the optional range of devices that can successfully complete PoW and obtain high mining rewards expands slightly, so the total system revenue gradually increases again.

Figure 2(b) shows the convergence of the four algorithms under three different numbers of IoT devices. As the number of IoT devices increases, although the convergence rates of all four methods gradually slow down, CAPSO consistently exhibits the fastest convergence among them. Specifically, when $N = 10$, they converge at the 36th, 11th, 58th, and 94th iterations, respectively. When $N = 50$, the first two methods converge at the 40th and 21st iterations, respectively, whereas the latter two fail to converge even after reaching the preset maximum iteration of 100. When $N = 100$, the first three methods converge at the 49th, 30th, and 73rd iterations, respectively, while SA does not converge. The rapid convergence of CAPSO is attributed to the combined effects of chaotic mapping and adaptive inertia weights, which dynamically adjust the movement direction and step size of particles, enabling them to approach the optimal solution more efficiently.

5.2 Impact of IoT Device Requirements

In this set of experiments, we investigated the impact of increasing the resources requested by each IoT device from [100, 400] to [500, 1200] on the performance of the four algorithms. The swarm sizes of PSO and CAPSO, the number of IoT devices, the edge server's resource capacity, the maximum iterations, the difficulty coefficient, and the chaos parameter were set to 30, 20, 10000, 50, [1, 2] and 4, respectively. Here, $[a, b]$ indicates that the resource demand per IoT device is randomly selected as an integer value within this interval.

Figure 3(a) shows that as resource demand increases, the total system revenue of the four methods initially rises and then declines. In all cases, CAPSO achieves the highest total revenue. This phenomenon primarily stems from block rewards being constrained by the Gompertz function. With the growing demand from IoT



(a) Comparison of total system revenues (b) Comparison of algorithm convergence

Fig. 3. The impact of IoT device requirements.

devices and the continuous investment of the edge server's resources, the total computilities of the system increases, causing the function $g(\phi)$ to undergo a change process from 0 to 1. Consequently, the total system revenue first increases. When $g(\phi)$ reaches 1, the total revenue attains its maximum value (observed when resource demand falls within the range [300, 800]). After that, even if the edge server still have allocatable resources, block rewards no longer increase, while resource costs continue to rise, leading to a subsequent decrease in total revenue. This clearly demonstrates the effectiveness of the function in curbing resource waste.

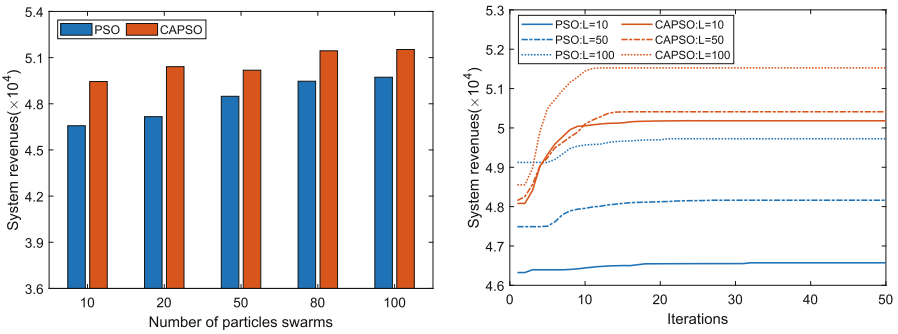
This demonstrates that when an IoT device attempts to obtain higher rewards by requesting more resources, it not only reduces its own revenue but also diminishes the total system revenue, which contradicts both individual rationality and system goals. Therefore, the device will not proactively request excessive resources, nor will the system allocate them, thereby avoiding resource wastage.

As shown in Fig. 3(b), when the resource demand is in the range [100, 400], the first three methods converge at the 36th, 19th, and 23rd iterations, respectively. When the demand increases to [500, 1200], these methods converge at the 41st, 24th, and 37th iterations, respectively. SA does not converge under either of the two value ranges. It can be seen that as resource demand grows, the iterative process of the algorithms in finding the optimal solution gradually slows down. Nevertheless, the proposed method always maintains the highest execution efficiency among the four approaches.

5.3 Impact of Particle Swarm Scale

This set of experiments examined the impact of varying the particle swarm size from 10 to 100 on the performance of PSO and CAPSO. The number of IoT devices, the edge server's resource capacity, the maximum iterations, the difficulty coefficient, and the chaos parameter were set to 20, 10000, 50, [1, 2] and 4, respectively.

The selection of population size directly influences both the search scope and search speed of the algorithms. Generally, a larger population size leads to broader search coverage but also increases computational complexity (for instance, in CAPSO, the number of particles determines the inner loop iterations). The trend of the total system revenues of PSO with varying particle numbers in Fig. 4(a) aligns with this general principle. A wider search range makes it easier to escape local optima, thereby yielding a larger objective function value. However, CAPSO is notably less sensitive to the number of particles compared to PSO, primarily because the search capability and convergence speed of our method depend heavily on chaotic mapping and adaptive inertia weights.



(a) Comparison of total system revenues (b) Comparison of algorithm convergence

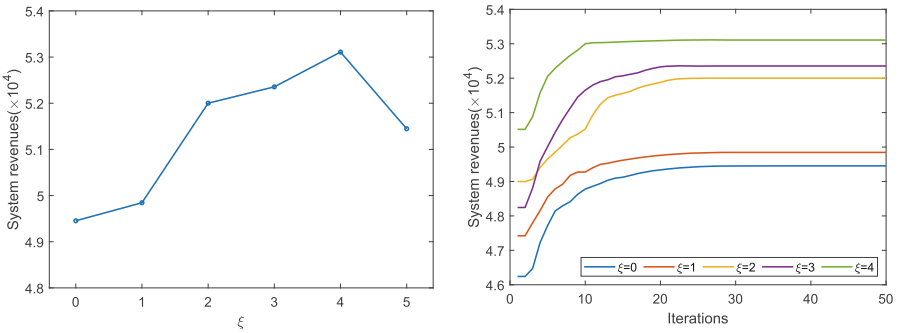
Fig. 4. The impact of particle swarm scale.

Figure 4(b) illustrates the convergence of the two methods under different particle swarm scales. First, for all three tested particle sizes, the convergence speed of CAPSO is faster than that of PSO. Second, for each method, the wider the search range, the higher the efficiency in locating the optimal solution, so the algorithm can converge faster.

5.4 Impact of Chaotic Parameter

This set of experiments investigated the impact of the parameter ξ in the Logistic chaotic map varying from 0 to 5 on CAPSO. The number of IoT devices, the particle swarm size, the edge server's resource capacity, the maximum iterations, and the mining difficulty were set to 20, 30, 10000, 50, and [1, 2], respectively.

According to the properties of the Logistic chaotic map, chaotic behavior (i.e., the generated sequence has pseudorandomness) only occurs when $\xi \in [0, 4]$. As ξ approaches 4, the chaotic values become more uniformly distributed in the [0,1] interval, which enhances the global search capability of CAPSO. Figure 5a demonstrates this relationship through the variation of the system revenue with ξ. When $\xi = 0$, CAPSO degenerates into PSO with adaptive inertia weights, yielding lower system revenue. As ξ increases, the chaotic behavior intensifies



(a) Comparison of total system revenues (b) Comparison of algorithm convergence

Fig. 5. The impact of chaotic parameter.

and the total system revenue gradually improves. The system achieves maximum revenue at $\xi = 4$ when full chaos emerges. Beyond $\xi \in [0, 4]$, the chaotic phenomenon disappears, and the chaotic values may either escape from [0, 1] or diverge. The sharp decrease of system revenue at $\xi = 5$ in Fig. 5a exactly confirms this behavior.

Figure 5(b) further reveals that as ξ increases from 0 to 4, CAPSO converges at the 33rd, 30th, 26th, 23rd, and 21st iterations respectively. This monotonically improving convergence rate indicates that the chaotic mapping also accelerates optimal solution discovery.

5.5 Impact of Mining Difficulty

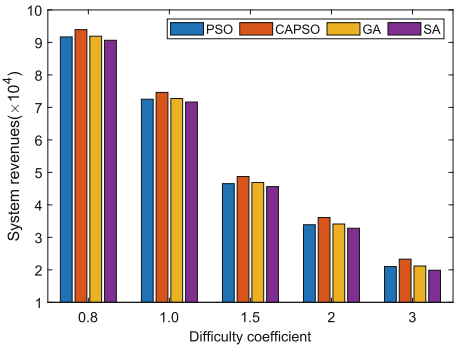


Fig. 6. System revenues under different mining difficulties.

This set of experiments investigated the impact of the difficulty coefficient for IoT devices to successfully mine blocks and obtain rewards on the overall system

revenue. The number of IoT devices, the particle swarm size, the edge server's resource capacity, the maximum iterations, and the chaos parameter were set to 20, 30, 10000, 50, and 4, respectively.

Figure 6 demonstrates that as the difficulty coefficient gradually increases, the total system revenue sharply declines. This is evident because, as derived from Eq. (9), the block reward for each IoT device is inversely proportional to the difficulty coefficient. An increase in the difficulty coefficient leads to a reduction in block rewards, thereby decreasing the total system revenue even if costs remain constant. However, in practice, when the mining difficulty rises, miners tend to request more resources and consume additional electricity, resulting in a significant increase in costs.

6 Conclusion

Edge computing provides a favorable guarantee for IoT devices with insufficient computilities to participate in blockchain networks. Aiming to maximize the overall system revenue, this work proposed a resource allocation scheme for blockchain networks that alleviates resource waste by controlling block rewards. First, we established revenue models for IoT devices and the edge server based on block rewards introduced with Gompertz function, and formulated the problem as a NLP problem. Second, we designed an improved particle swarm optimization algorithm, termed CAPSO, which leverages chaotic mapping and adaptive inertia weights to tackle the NLP challenges inherent in IoT-blockchain resource allocation. This tailored approach enables CAPSO to quickly locate the global optimal solution, distinguishing it from generic heuristic methods. Through comparative experiments, we demonstrated that CAPSO significantly outperforms the other three heuristic algorithms in terms of both total system revenue and convergence speed.

However, this work does not consider the issue of resource pricing, simply trading resources based on unit prices submitted by IoT devices while neglecting incentive compatibility during resource allocation. Affine maximization auctions, which offer distinct advantages including truth, collusion resistance, support for multi-dimensional attributes, high computational efficiency, and design flexibility, present a promising solution [30]. Our future work will focus on implementing affine auction smart contracts in IoT-blockchain ecosystems, while employing federated learning and reinforcement learning techniques to dynamically optimize affine parameters, thereby addressing both pricing and trust challenges in resource allocation. Furthermore, the current framework considers only a single-edge-server scenario. When numerous surrounding IoT devices compete for resources or maintaining consensus mechanisms requires more resources, the limited capacity of a single edge node may fail to ensure stable system operation. Computing power network is an emerging information infrastructure designed to interconnect and orchestrate massive distributed computing nodes, providing integrated services with guaranteed resources and network performance for various IoT applications [31]. Consequently, we also plan to explore resource scheduling mechanisms in blockchain-empowered computing power networks.

Acknowledgments. This study was supported in part by the National Natural Science Foundation of China (Nos. 62266051, 12071417, 62062065), the Yunnan Fundamental Research Project (Nos. 202401AT070471, 202501AS070076) and the Science Research Fund of Yunnan Education Department (No. 2024J0028).

References

1. Nguyen, D.C., et al.: 6g internet of things: a comprehensive survey. IEEE Internet Things J. **9**(1), 359–383 (2022). https://doi.org/10.1109/JIOT.2021.3103320
2. Alwarafy, A., Al-Thelaya, K.A., Abdallah, M., Schneider, J., Hamdi, M.: A survey on security and privacy issues in edge-computing-assisted internet of things. IEEE Internet Things J. **8**(6), 4004–4022 (2021). https://doi.org/10.1109/JIOT.2020.3015432
3. Junior, F.M.R., Kamienski, C.A.: A survey on trustworthiness for the internet of things. IEEE Access **9**, 42493–42514 (2021). https://doi.org/10.1109/ACCESS.2021.3066457
4. Wang, X., et al.: Survey on blockchain for internet of things. Comput. Commun. **136**, 10–29 (2019). https://doi.org/10.1016/j.comcom.2019.01.006
5. Dai, H.N., Zheng, Z., Zhang, Y.: Blockchain for internet of things: a survey. IEEE Internet Things J. **6**(5), 8076–8094 (2019). https://doi.org/10.1109/JIOT.2019.2920987
6. Mathur, S., Kalla, A., Gür, G., Bohra, M.K., Liyanage, M.: A survey on role of blockchain for Iot: applications and technical aspects. Comput. Netw. **227**, 109726 (2023). https://doi.org/10.1016/j.comnet.2023.109726
7. Laroui, M., Nour, B., Moungla, H., Cherif, M.A., Afifi, H., Guizani, M.: Edge and fog computing for IoT: a survey on current research activities & future directions. Comput. Commun. **180**, 210–231 (2021). https://doi.org/10.1016/j.comcom.2021.09.003
8. Yaraziz, M.S., Hill, R.: A review of resource allocation for maximizing performance of Iot systems. IEEE Access **13**, 98426–98451 (2025). https://doi.org/10.1109/ACCESS.2025.3576716
9. Su, Q., Zhang, Q., Li, W., Zhang, X.: Primal-dual-based computation offloading method for energy-aware cloud-edge collaboration. IEEE Trans. Mob. Comput. **23**(2), 1534–1549 (2024). https://doi.org/10.1109/TMC.2023.3237938
10. Gompertz, B.: On the nature of the function expressive of the law of human mortality, and on a new mode of determining the value of life contingencies. Philos. Trans. R. Soc. Lond. **115**, 513–583 (1825)
11. Limane, A., et al.: Chaos-enhanced metaheuristics: classification, comparison, and convergence analysis. Complex Intell. Syst. **11**(177), 1–42 (2025). https://doi.org/10.1007/s40747-025-01791-2
12. Liang, H., et al.: Reinforcement learning enabled dynamic resource allocation in the internet of vehicles. IEEE Trans. Industr. Inf. **17**(7), 4957–4967 (2021). https://doi.org/10.1109/TII.2020.3019386
13. Song, Z., et al.: Secure and efficient fog-assisted quantum-inspired wearable healthcare consumer electronics IoT system. IEEE Trans. Consum. Electron. **71**(1), 2252–2261 (2025). https://doi.org/10.1109/TCE.2024.3446803

14. Chen, R., Sun, Y., Liang, L., Cheng, W.: Joint power allocation and placement scheme for UAV-assisted IoT with GOS guarantee. IEEE Trans. Veh. Technol. **71**(1), 1066–1071 (2022). https://doi.org/10.1109/TVT.2021.3129880
15. Abdullah, M., Khan, H.Z., Fakhar, U., Akhtar, A.N., Ansari, S.: Satellite synergy: navigating resource allocation and energy efficiency in IoT networks. J. Netw. Comput. Appl. **230**, 103966 (2024). https://doi.org/10.1016/j.jnca.2024.103966
16. Chakraborty, A., Misra, S.: QOS-aware resource bargaining for federated learning over edge networks in industrial IoT. IEEE Trans. Netw. Sci. Eng. **10**(5), 2769–2778 (2023). https://doi.org/10.1109/TNSE.2022.3206474
17. Adil, M., et al.: Healthcare internet of things: security threats, challenges, and future research directions. IEEE Internet Things J. **11**(11), 19046–19069 (2024). https://doi.org/10.1109/JIOT.2024.3360289
18. Zhang, J., Liu, Y., Qin, X., Xu, X., Zhang, P.: Adaptive resource allocation for blockchain-based federated learning in internet of things. IEEE Internet Things J. **10**(12), 10621–10635 (2023). https://doi.org/10.1109/JIOT.2023.3241318
19. Liang, W., Zhao, J., Liu, Y., Liang, Y., Jingwen, L.: Fairness resource allocation based on blockchain for secure communication in integrated IoT. EURASIP J. Adv. Signal Process. (115) (2023). https://doi.org/10.1186/s13634-023-01075-2
20. Hou, J., Chen, X., Tao, S., Wang, H., Hou, Z., Hu, N.: Blockchain-enabled distributed spectrum allocation in battlefield: A fast matching algorithm. Acta Electron. Sinica (2025), Available Online: https://link.cnki.net/urlid/11.2087.TN.20250508.1349.004
21. Chen, S., Wang, Q., Zhang, H., Wang, K.: Resource allocation and control co-aware smart computation offloading for blockchain-enabled IoT. Chinese J. Comput. **45**(3), 472–484 (2025). https://doi.org/10.11897/SP.J.1016.2022.00472
22. Yang, R., Yu, F.R., Si, P., Yang, Z., Zhang, Y.: Integrated blockchain and edge computing systems: a survey, some research issues and challenges. IEEE Commun. Surv. Tutor. **21**(2), 1508–1532 (2019). https://doi.org/10.1109/COMST.2019.2894727
23. Guo, S., Dai, Y., Guo, S., Qiu, X., Qi, F.: Blockchain meets edge computing: stackelberg game and double auction based task offloading for mobile blockchain. IEEE Trans. Veh. Technol. **69**(5), 5549–5561 (2020). https://doi.org/10.1109/TVT.2020.2982000
24. Sadananda, L., Jodumutt, S.B., Pawar, S.R., Vylala, A., Chandrasekaran, S.: Dynamic task offloading for resource allocation and privacy-preserving framework in kubeedge-based edge computing using machine learning. Clust. Comput. **27**, 9415–9431 (2024). https://doi.org/10.1007/s10586-024-04420-8
25. Zhang, W., Li, Y., Yang, J., Zheng, Y., Lin, Z., Yeo, C.K.: Computation offloading in mobile edge computing-enabled blockchain based on contract and matching theory. Mobile Netw. Appl. (2025). https://doi.org/10.1007/s11036-025-02444-x
26. Wang, C., Zhai, D., Zhang, R., Li, H., Richard Yu, F.: Latency minimization for UAV-assisted MEC networks with blockchain. IEEE Trans. Commun. **72**(11), 6854–6866 (2024). https://doi.org/10.1109/TCOMM.2024.3406400
27. Zhang, L., Yu, X., Song, J., Zhu, H.: D2d communication assisted edge computing based resource pricing and scheduling research in blockchain. Wireless Netw. **31**, 899–913 (2025). https://doi.org/10.1007/s11276-024-03792-4
28. Xie, N., Zhang, J., Zhang, X., Li, W.: Double auction mechanisms in edge computing resource allocation for blockchain networks. Clust. Comput. **27**, 3017–3035 (2024). https://doi.org/10.1007/s10586-023-04129-0

29. Decker, C., Wattenhofer, R.: Information propagation in the bitcoin network. In: IEEE P2P 2013 Proceedings, pp. 1–10 (2013). https://doi.org/10.1109/P2P.2013.6688704
30. Zhang, J., Chen, P., Yang, X., Wu, H., Li, W.: An optimal reverse affine maximizer auction mechanism for task allocation in mobile crowdsensing. IEEE Trans. Mob. Comput. **24**(8), 7475–7488 (2025). https://doi.org/10.1109/TMC.2025.3549504
31. Sun, Y., et al.: Computing power network: a survey. China Commun. **21**(9), 109–145 (2024). https://doi.org/10.23919/JCC.ja.2021-0776

A Structure-Aware Irregular Blocking Method for Sparse LU Factorization

Zhen Hu, Dongliang Xiong(✉), Kai Huang, Changjun Wu, and Xiaowen Jiang

College of Integrated Circuits, Zhejiang University, Hangzhou 311200, China
{huzhen,xiongdl,huangk,wuchangjun,jiangxw}@zju.edu.cn

Abstract. In sparse LU factorization, nonzero elements after symbolic factorization tend to distribute in diagonal and right-bottom region of sparse matrices. However, regular 2D blocking on this non-uniform distribution structure may lead to workload imbalance across blocks. Besides, existing matrix features fail to guide us effectively in blocking. In this paper, we propose a structure-aware irregular blocking method for numerical factorization. A novel diagonal block-based feature is introduced to effectively characterize the local nonzero distribution of sparse matrices. Based on this, we further propose an irregular blocking method that adjusts block sizes according to the local distribution of nonzeros. The strategy utilizes fine-grained blocks in dense regions and coarse-grained blocks in sparse regions, adequately balancing the nonzeros of blocks both within the same level and across levels in the dependency tree. Experiments demonstrate that, on a single NVIDIA A100 GPU, our proposed irregular blocking method achieves average speedups of 1.50× and 3.32× over PanguLU and the latest SuperLU_DIST, respectively. In addition, it achieves speedups of 1.40x and 3.84x over PanguLU and SuperLU_DIST on 4 NVIDIA A100 GPUs.

Keywords: blocked sparse LU factorization · nonzero distribution · irregular blocking · load balance

1 Introduction

Solving the sparse linear system $Ax = b$ [6] is a crucial part of many scientific and engineering problems. There are two ways to solve $Ax = b$: direct methods that obtain an exact solution in a finite number of steps and iterative methods that obtain an exact solution through continuous iteration. Sparse LU factorization is a commonly used direct method for solving linear systems. It factors the sparse matrix A of the linear system into a lower triangular matrix L and an upper triangular matrix U, i.e., $A = L \times U$. The direct LU factorization [9] method can obtain a more accurate solution and is widely used in scientific computing and simulation.

Typical sparse LU factorization is divided into three phases: (1) reordering; (2) symbolic factorization; (3) numerical factorization. The purpose of the

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 101–118, 2026.
https://doi.org/10.1007/978-981-95-8405-5_6

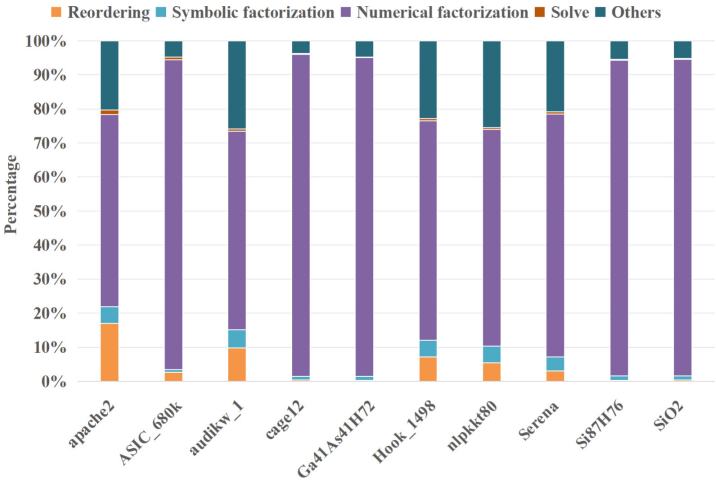


Fig. 1. Time breakdown of SuperLU

reordering phase is to reduce fill-in nonzeros and maintain numerical stability. Then, the symbolic factorization phase is used to determine the nonzero structure of matrices L and U. Finally, the numerical factorization phase performs floating-point operations to obtain the specific nonzero values. As shown in Fig. 1, among these phases, the numerical factorization phase is the most time-consuming, accounting for 50% to 95%. At the same time, there are long dependency chains between tasks in numerical factorization of sparse matrices. A dependency tree [19] is a data structure that is used to represent the dependencies between tasks in the factorization of a sparse matrix.

Preprocessing sparse matrix to blocks or aggregating similar rows or columns into supernodes [10] can make numerical factorization easier to parallelize. However, existing solvers such as SuperLU_DIST [17], UMFPACK [5] and PARDISO [21] exploit dense Basic Linear Algebra Subprograms (BLAS) libraries [12], which reduce computational efficiency due to excessive fill-ins introduced to aggregate more similar columns. KLU [8] is a direct solver in circuit simulation, based on the Gilbert-Peierls left-looking algorithm. It achieves good performance by permuting the matrices into Block Triangular Form (BTF), but can only execute factorization in serial. Basker [2] partitions the matrix to a block triangular form where each diagonal block is further partitioned using nested dissection [16], generating a number of independent sub-matrices. It performs well in highly sparse matrices, but performs poorly in high density matrices [3].

Unlike these solvers, PanguLU [14] uses a simpler regular two-dimensional blocking method and stores it with a compressed format, then selects either sparse kernels or dense BLAS libraries for computation based on the density of blocks. However, sparse matrices after symbolic factorization demonstrate a structure similar to the Bordered Block Diagonal (BBD) structure [4]. It features with nonzero elements concentrated in the right-bottom region. Regular

2D blocking will lead to more nonzero elements in blocks near the bottom and right boundaries, and fewer nonzero elements in blocks near the left and upper boundaries. This will cause the last level of the dependency tree during numerical factorization a large computational load and a longer execution time, affecting the overall performance of numerical factorization. Additionally, it will lead to significant differences in the number of nonzero elements between blocks, causing workload imbalance during parallel computation and further impacting overall performance. Therefore, under the premise of sparse storage and computation, efficient preprocessing blocking is crucial for improving numerical factorization performance. Besides, sparse matrices have various structure in real world. Existing matrix features, like density and average nonzeros per row or column, can only provide coarse and one-dimensional information of sparse matrices. PanguLU constructs a selection tree considering matrix dimension and total number of nonzeros after symbolic factorization to decide the block sizes [15], which fails to select the optimal ones for most sparse matrices.

To mitigate the load imbalance caused by the various non-uniform distribution structure of nonzeros in sparse matrices, we need to address the following two challenges: (1) how to efficiently obtain the nonzero distribution structure of the matrix after symbolic factorization. (2) how to determine the blocking strategy to alleviate load imbalance.

In this paper, we propose a structure-aware irregular blocking method. Unlike PanguLU blocking sparse matrices into blocks with the same size, our proposed method partitions them adaptively according to the nonzero distribution structure of sparse matrices. The imbalance of nonzeros across blocks can be well alleviated through multiple block sizes. Firstly, since the sparse matrix after symbolic factorization has a symmetric structure [1], we introduce a diagonal block-based pointer to exploit the two-dimensional distribution characteristics of the sparse matrix. Then, based on the two-dimensional distribution characteristics of the sparse matrix, our proposed blocking method performs coarse-grained blocking in dense regions and fine-grained blocking in sparse regions to obtain the specific blocking positions. Finally, we conduct experiments on various kinds of test matrices from SuiteSparse Matrix Collection [8] on the platform with 4 NVIDIA A100 GPUs.

The comparisons between our work with some existing relevant solvers is shown in Table 1 and the contributions of our work are as follows.

- A diagonal block-based feature, which is described by percentage of nonzeros along the diagonal, is proposed to characterize global and partial nonzero distribution of sparse matrices. It can be obtained from common compressed formats.
- A structure-aware irregular blocking strategy is proposed based on the nonzero distribution of sparse matrices to alleviate workload imbalance of nonzeros.
- Experimental results show that our proposed irregular blocking method can achieve significant speedups over PanguLU and the latest SuperLU_DIST in numerical factorization.

Table 1. Comparisons between our work with some existing relevant solvers.

Solver	Left or Right looking	Blocking method	Support GPU or not
KLU	Left	Block diagonal	N
UMFPACK	Left	Multifrontal [13]	N
PARDISO	Left&Right	Supernode	N
SuperLU_DIST	Right	Supernode	Y
Basker	Left	Recursive Block diagonal	N
PanguLU	Right	Regular 2D block	Y
Our work	**Right**	**Irregular 2D block**	**Y**

2 Background

2.1 Sparse LU Factorization

Sparse LU factorization needs symbolic factorization to predict the symbolic pattern of the LU factors without considering the numerical values. But it will induce extra fill-ins. As shown in Fig. 2, in order to minimize fill-ins, reordering algorithms [11] tend to reorder matrix into a good one just like the below structure [4]. As a result, after symbolic factorization, most nonzero elements are located in the right-bottom region. Besides, blocking technique is an optimization method by exploiting the data locality. Blocked sparse LU factorization is considered except for the extremely sparse matrices.

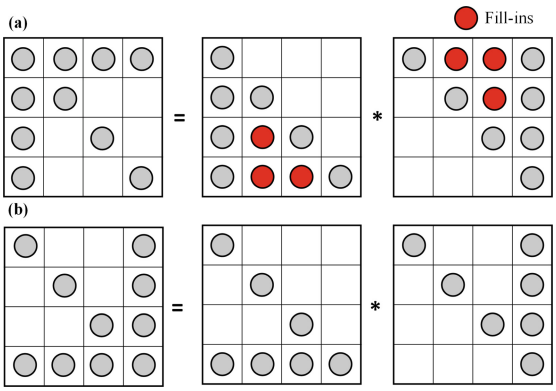


Fig. 2. Different structure lead to different fill-ins. (a) A structure leads to full fill-ins. (b) A structure leads to no fill-in.

2.2 Right-Looking Blocked LU Factorization

In numerical factorization phase, right-looking LU algorithm is widely used in solvers like SuperLU and PanguLU. The right-looking blocked LU algorithm [20],

which treats two-dimensional matrix blocks after preprocessing as input, is described in Algorithm 1. These loop nests show that data dependences are frequent. At each level i, only after the factorization of diagonal block $B_{i,i}$, row(U) panels $B_{i,*}$ and column(L) panels $B_{*,j}$ have been performed, the entire tailing sub-matrices can be updated. And the i^{th} iteration can only be executed after the $(i-1)^{th}$ iteration have been finished. However, sparsity brings empty blocks which can reduce the computation and data dependency, then introduces more parallelism. Computation only need to be performed on non-empty blocks as shown in Fig. 3. This example adopts regular 2D blocking before numerical factorization and partitions the original matrix into several 3×3 blocks. And it consists of a dependency tree with 3 levels.

Algorithm 1. Right-looking Blocked LU Factorization

Require: a $p \times p$ block matrix B
Ensure: $B = [B_{i,j}]_{i=1:p,j=1:p}$
1: Construct blocks $B_{i,j}, \forall i,j$
2: **for** $i = 1$ to p **do**
3: $B_{i,i} \rightarrow L_{i,i}U_{i,i}$ {Factorize the diagonal block}
4: **for** $j = i+1$ to p **do**
5: $B_{i,j} \leftarrow L_{i,i}^{-1}B_{i,j}$ {Factorize U panel}
6: $B_{j,i} \leftarrow B_{j,i}U_{i,i}^{-1}$ {Factorize L panel}
7: **end for**
8: **for** $j = i+1$ to p **do**
9: **for** $k = i+1$ to p **do**
10: $B_{k,j} \leftarrow B_{k,j} - B_{k,i}B_{i,j}$ {Update submatrix}
11: **end for**
12: **end for**
13: **end for**

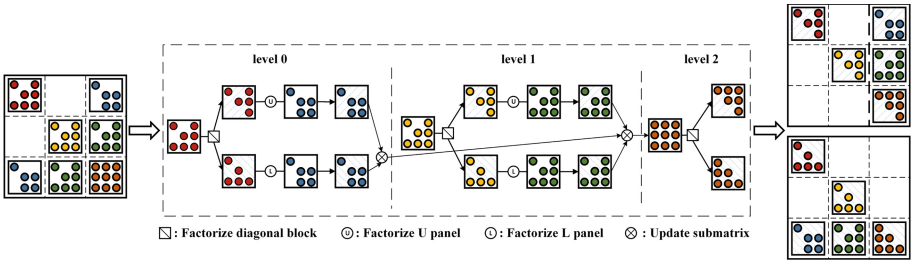


Fig. 3. Example of a sparse blocked LU factorization.

3 Motivation

3.1 Ineffective One-Dimensional Matrix Features

The nonzero distribution of a sparse matrix is critical for researchers to accelerate the computation. However, the most common features, like matrix dimension and density, only reflects the features of the whole matrix. Other features, like average number of nonzero elements per row or the standard deviation of nonzero elements in each row, only reflect the one-dimensional feature of the sparse matrix. PanguLU selects a fixed size of regular blocking according to the matrix order and the density of the matrix after symbolic factorization, which can not obtain an optimal block size for the performance of numerical factorization as shown in Fig. 4. While blocked LU factorization algorithm performs computation with block granularity, it is more important to focus on its two-dimensional features, which have not been explored.

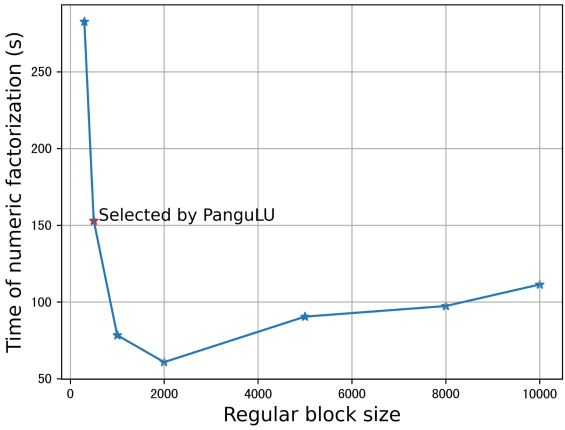


Fig. 4. The numerc factorization time varies with different regular block size of a sparse matrix. PanguLU makes choices of block size from 300, 500, 1000, 2000, 5000. But it selects a worse size according to the matrix dimension and density.

3.2 Unbalanced Nonzeros

Regular 2D blocking method partitions matrix into blocks with the same size. However, nonzero elements are distributed non-uniformly, which leads to blocks with unbalanced nonzeros. Besides, since most of nonzero elements locate at the right-bottom region, the density of blocks nearing right and bottom region are generally higher than those near left and up region [3]. As shown in Fig. 5, the number of nonzero elements in block n is much more than that in block a. This imbalance causes most of the time for numerical factorization to be spent on the last dense blocks. This issue will be more severe in parallel computing as the time difference between blocks induced by load imbalance. Therefore, it is necessary to partition blocks according to the sparsity of region.

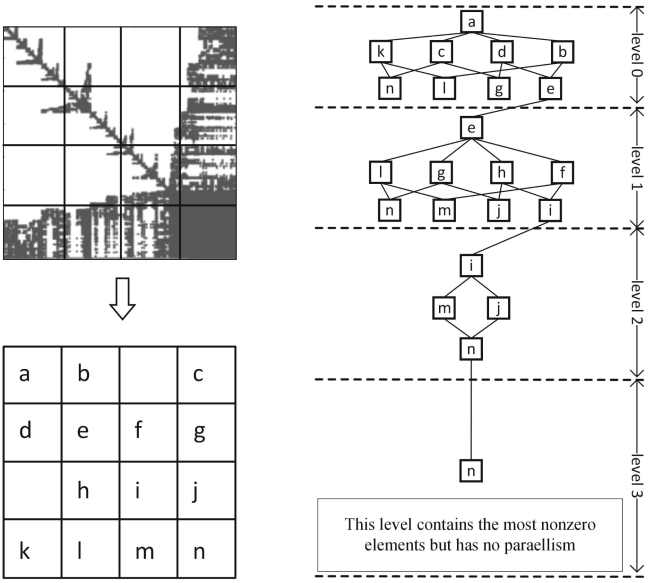


Fig. 5. Example of a dependency-level tree of a regularly blocked sparse matrix. (a) Sparse matrix A with regular blocking. (b) The dependency tree of A based on level.

4 Structure-Aware Irregular Blocking Method

4.1 Overview

In this paper, we propose a structure-aware irregular blocking preprocessing method for sparse LU numerical factorization. The floating point operations in numerical factorization is related with the number of nonzeros after symbolic factorization. The sparse matrix is stored by Compressed Sparse Column (CSC) format. In the preprocessing phase, we propose a conversion algorithm to get a diagonal block-based pointer from CSC pointer. The diagonal block-based pointer stores the number of nonzeros in diagonal sub-matrix. The percentage of nonzero values distributed along the diagonal can be obtained by dividing the value in the pointer by the total number of nonzero values. To balance the number of nonzeros in blocks, we sample uniformly 1000 points of the nonzeros distribution to guide us in blocking the sparse matrix. The strategy of finer-grained blocks in dense regions while allocating coarser blocks in sparse regions induces blocks with multiple sizes. Workload imbalance can be alleviated through balance the number of nonzeros both within the same level and across levels in the dependency tree.

4.2 Diagonal Block-Based Feature

Compressed sparse column (CSC) is a common compressed format for sparse matrix. It utilizes three arrays: column pointer, row index, and value. The column pointer array stores the index in the value array of the first nonzero element of each column. Row index array stores the row index of each nonzero element. And the value array stores the value of nonzero elements of in column order. According to column pointer array, we can get the number of nonzero elements in each column. It can be useful for matrix computation based on columns, like column-wise sparse matrix multiplication [18]. Blocked sparse matrix factorization regards blocks as the computational granularity, while compressed sparse column (CSC) only reflect the row distribution of the sparse matrix and cannot effectively guide the performance optimization for blocked matrix factorization.

Considering the symmetry of the sparse matrix after symbolic factorization [14], a diagonal block-based array can be obtained from compressed sparse column pointer, which contains the number of nonzero elements in the submatrix $[0 : n, 0 : n]$, where n is the row/column index, as shown in Fig. 6. The conversion algorithm from arrays of CSC to diagonal block-based pointer is shown in Algorithm 2.

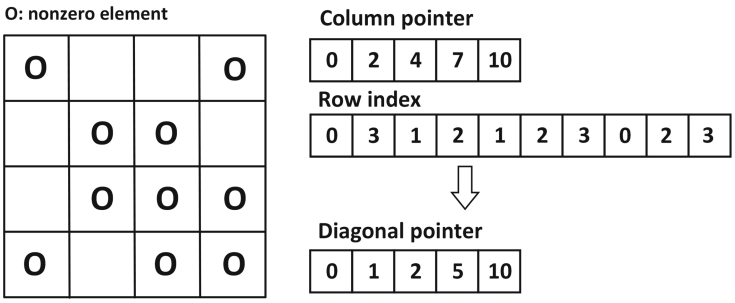


Fig. 6. Example of compressed sparse column pointer and diagonal block pointer.

After normalizing the index and value of diagonal block pointer, we get the feature, i.e., a percentage array of nonzero distribution along the diagonal. When taking them as the *x-axis* and *y-axis*, respectively, the resulting curve explicitly shows both global and local distribution features of the sparse matrix.

As shown in Fig. 7, for linear matrices, the number of nonzero increases uniformly along the diagonal. Therefore the nonzero percentage curve based on diagonal block pointer shows a linear feature. For uniformly distributed matrices, the number of nonzeros in a block is proportional to the area of the diagonal block. As we can see, the percentage curve of uniformly distributed matrices shows a quadratic feature. We can roughly infer the structure of sparse matrix by the global feature of the curve. In addition, the local distribution of nonzeros can also be captured. As shown in Fig. 8, figure (a) shows a sparse matrix

Algorithm 2. Extracting Diagonal Block Pointer

Require: matrix dimension $n, *columnpter, *rowidx$

Initialize $num[n] = 0$
for $i = 0$ **to** $n - 1$ **do**
  for $j = columnptr[i]$ **to** $columnptr[i + 1] - 1$ **do**
    $index \leftarrow rowidx[j]$
    if $index > i$ **then**
      $num[index] \leftarrow num[index] + 1$
    end if
  end for
end for
$blockptr[0] \leftarrow 0$
for $i = 0$ **to** $n - 1$ **do**
  $num[i] \leftarrow 2 \times num[i] + 1$
  $blockptr[i + 1] \leftarrow blockptr[i] + num[i]$
end for
return $*blockptr$

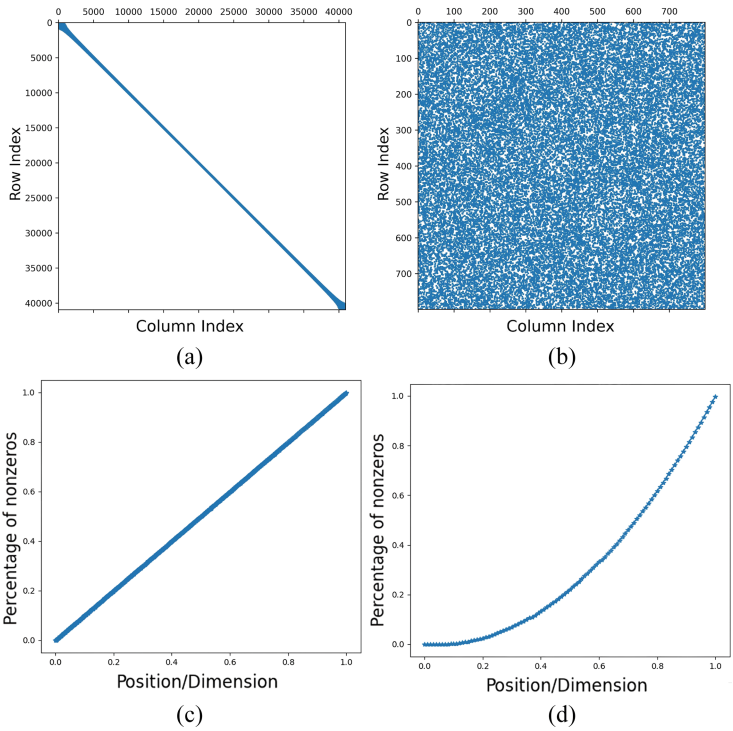


Fig. 7. (a) A linear structure matrix. (b) An uniform distribution structure matrix. (c) Percentage of nonzeros based on diagonal pointer of (a). (d) Percentage of nonzeros based on diagonal pointer of (b).

with some local dense blocks. The nonzero percentage curve based on diagonal block pointer shows partial quadratic trends in dense region with obvious discontinuities. Also, when there are relatively dense rows and columns along the diagonal, there are significant jumps in the curve. The curve in figure (d) shows many jumps because the sparse matrix has relatively dense rows/columns here. Therefore, the diagonal block-based pointer contains the two-dimensional distribution information of a sparse matrix, which can be exploited to guide us in accelerating the block-based matrix computation, like blocked sparse matrix factorization.

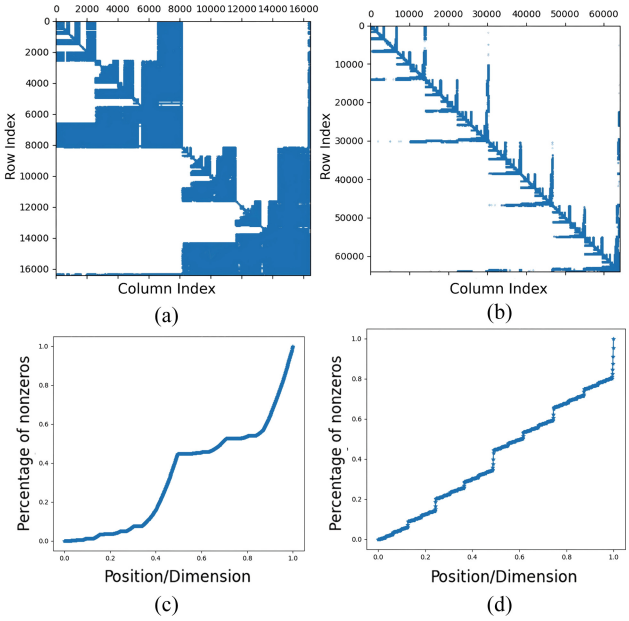


Fig. 8. (a) A sparse matrix with some local dense regions. (b) A sparse matrix with some dense rows or columns. (c) Percentage of nonzeros based on the diagonal pointer of (a). The partial quadratic trends in the curve indicates that there are local dense regions. (d) Percentage of nonzeros based on the diagonal pointer of (b). The jumps in the curve indicates that there are relatively dense rows and columns.

4.3 Irregular Blocking Method Based on Nonzero Distribution

Considering the locality of the nonzero distribution after symbolic factorization, an irregular blocking method is proposed as described in Algorithm 3. The algorithm adopts the strategy of blocking finer-grained blocks for dense regions while allocating coarser blocks for sparse regions.

We sample uniformly 1000 points from the percentage of nonzeros array as input. The algorithm calculate the percentage difference between two consecutive points or points separated by a step interval. If the percentage difference exceeds a threshold, this indicates that the diagonal region between these two indexes contains a significant number of nonzero elements, and the larger index should be marked as a blocking position (P_1). Otherwise, if the difference does not exceed the threshold, we regard that the diagonal region between the two indexes contains fewer nonzero elements and can be skipped without blocking, while increasing the skip counter l. It is hard to set the threshold to distinguish the dense or sparse regions. As we have observed in the nonzero percentage curve in Sect. 4.2, the percentage of nonzeros based on diagonal pointer of a matrix with linear structure show a linear feature. Considering the nonzero after symbolic factorization mainly distribute along the diagonal, we regard the linear difference as the threshold, i.e. step/sample_pointers. Also, to prevent excessive accumulation of nonzero elements caused by consecutive skips, when l reaches max_num, blocking must still be performed and a blocking position (P_m) marked. The process described above continues until the index reaches the dimension N of the matrix. The algorithm eventually outputs a blocking position pointer array that records all blocking boundaries. We choose $step$ and max_num to be 2 and 3 respectively. The parameters are determined empirically based on a large number of parameter tuning experiments.

Algorithm 3. Structure-aware irregular blocking method

Require: Percentage array $pct[sample_points]$, matrix dimension N

$ptr[0] = 0$
for index $i = 0$ **to** $sample_points$ **do**
  if $pct[i + step] - pct[i] \geq threshold$ **then**
    $k \leftarrow k + 1$
    $ptr[k] \leftarrow (i + step) * N/sample_points$ {Dense region}
    $l \leftarrow 0$
  else if $l \geq max_num$ **then**
    $k \leftarrow k + 1$
    $ptr[k] \leftarrow (i + step) * N/sample_points$ {Avoid too large blocks}
    $l \leftarrow 0$
  else
    $l \leftarrow l + 1$
  end if
end for
return blocking position $*ptr$

An example is shown in Fig. 9. We first sample uniformly a proper number of points from the percentage array of nonzero distribution, which means a preliminary regular blocking into some basic blocks. Then according to the difference of distribution percentage between two sample points, we merge a certain number

of basic blocks in sparse regions. The final blocking scheme is shown in Fig. 9(c). The blocking positions are denoted by P_0, P_1, ... , P_5, where $P_0 = 0$ and $P_5 = N$.

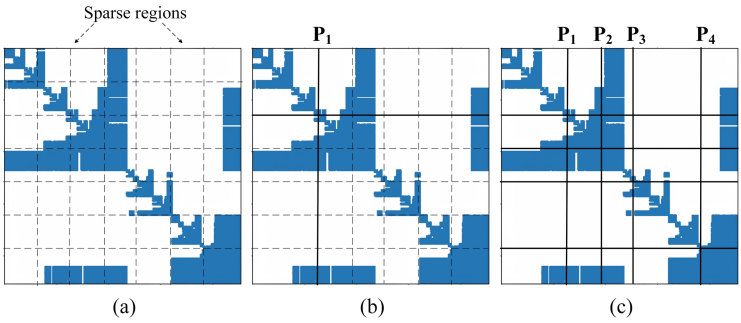


Fig. 9. Example of the irregular blocking method. (a) Initial matrix sampled in position marked by dashed lines. (b) Coarse-grained blocking in a sparse region. (c) Final blocking.

5 Experimental Results

5.1 Experimental Setup

We apply the proposed irregular blocking method to PanguLU and conduct experiments on the platform with 4 * NVIDIA A100 GPUs. Hardware information of the test platform is provided in Table 2. SuperLU_DIST is a parallel extension to the serial SuperLU library. It is targeted for the distributed memory parallel machines. We compile SuperLU_DIST and PanguLU with gcc-11.3.0, OpenMPI-4.1.4 and CUDA 12.2. Detailed information of the sparse matrices is listed in Table 3. These sparse matrix benchmarks are all downloaded from the SuiteSparse Matrix Collection [7].

Table 2. Information of the test platform and baseline algorithm.

GPU Platform	Baseline
4 * NVIDIA A100 GPU with 80G, B/W 1555 GB/s Intel Xeon Gold 6348 CPU @2.6 GHz	PanguLU SuperLU_DIST 9.1.0

Table 3. The sparse matrices tested in this paper. The matrix order, the number of nonzeros before and after symbolic factorization, the total number of floating-point operations and kinds are listed.

Matrix	n(A) (10^5)	nnz(A) (10^6)	nnz(L+U) (10^8)	FLOPs (10^{11})	Kind
apache2	7.15	4.82	3.19	3.46	Structural Problem
ASIC_680k	6.83	2.64	1.18	7.68	Circuit Simulation Problem
cage12	1.30	2.03	5.70	42.30	Directed Weighted Graph
CoupCons3D	4.17	17.28	4.99	9.05	Structural Problem
dielFilterV3real	11.03	89.31	10.77	20.08	Electromagnetics Problem
ecology1	10.00	5.00	0.72	0.30	2D/3D Problem
G3_circuit	15.85	7.66	1.81	0.91	Circuit Simulation Problem
offshore	2.60	4.24	1.50	1.65	Electromagnetics Problem
language	3.99	1.22	3.88	37.78	Directed Weighted Graph
boneS10	9.15	40.88	5.29	4.86	Model Reduction Problem

5.2 Performance on A Single GPU

We first evaluate the performance of the numerical factorization of our work compared with PanguLU and SuperLU_DIST in a NVIDIA A100 GPU. PanguLU selects the size of block from 200, 300, 500, 1000, 2000 and 5000 according to the order of matrix and number of nonzeros after symbolic factorization. We also run PanguLU with all those block sizes mentioned above, and select the best performance for comparison. In numerical factorization, both PanguLU and our work use sparse kernels for computation, while SuperLU uses dense kernels. The experimental results in Table 4 show that the geometric mean speedups of our work in numerical factorization can achieve 1.50x and 3.32x, compared with PanguLU and SuperLU_DIST, respectively.

The sizes of most blocks partitioned by our work are generally larger than those of PanguLU. Large blocks contain relatively more nonzero elements, which can improve the utilization rate of the GPU. The speedup over SuperLU_DIST is mainly due to the computation of sparse kernels.

Table 4. The numerical factorization time comparison of SuperLU_DIST, PanguLU and our work on a NVIDIA A100 GPU

Matrix	SuperLU_DIST(s)	PanguLU(s)	Our work(s)	Speedup Our work vs SuperLU	Speedup Our work vs PanguLU
apache2	26.84	11.05	8.50	3.16x	1.30x
ASIC_680k	84.26	70.86	16.45	5.12x	4.31x
cage12	143.44	136.41	86.45	1.66x	1.58x
CoupCons3D	26.71	32.26	17.53	1.52x	1.84x
dielFilterV3real	64.49	58.61	37.56	1.72x	1.56x
ecology1	11.49	2.41	2.37	4.84x	1.02x
G3_circuit	26.72	5.74	4.81	5.56x	1.19x
inline_1	29.22	7.95	7.97	3.67x	1.00x
language	372.68	90.88	53.19	7.01x	1.71x
boneS10	42.94	14.30	12.90	3.33x	1.11x
GEOMEAN				**3.32x**	**1.50x**

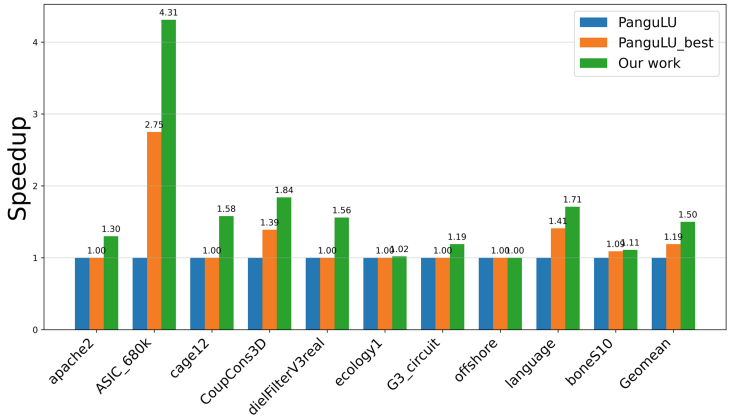


Fig. 10. Relative performance speedups on a single A100 GPU.

We also test PanguLU with all block sizes in its options and select the best performance for comparison. Figure 10 shows the experimental results. PanguLU_Best means the best performance among options in the selection tree. It achieves an average speedup of 1.19x on the A100 GPU platform over PanguLU, with speedups ranging from 1.0x to 2.75x. Most of the matrices like apache2 and offshore, can achieve the optimal selection of block size among its options through the selection tree. However, for many matrices such as ASIC_680k and CoupCons3D, even the best performance achieved through regular blocking remains inferior to that achieved through irregular blocking. The results demonstrate the superiority of our irregular blocking over the regular blocking.

5.3 Speedup in Parallel Computing

To demonstrate scalability, we also evaluated the performance of numerical factorization with 4 NVIDIA A100 GPUs. The results in Table 5 show that our work has a geometric mean of 1.40x compared to PanguLU in the numerical factorization. In addition, the performance achieves a geometric mean of 3.84x compared to SuperLU_DIST. In parallel computing, workload imbalance is an important factor that affects performance, especially when dealing with the irregular matrices such as ASIC_680k. The benefit of workload balance in our work is very obvious, resulting in a more significant speedup.

The nonzero distribution of ASIC_680k and ecology1 are shown below in Fig. 11. For ASIC_680k, we can observe that 98% of its nonzero elements are located in the bottom and right regions, which are distributed extremely nonuniformly. Our proposed irregular blocking method partitions ASIC_680K with a large size (about 4000) in sparse regions with only 2% nonzero elements and a small size (about 1300 and 680) in dense region while PanguLU partitions it with a size of 300 regularly. And there is little data dependence in ASIC_680k, so our proposed blocking method can achieve a speedup of 4.08x compared with

Table 5. The numerical factorization time comparison of SuperLU_DIST, PanguLU and our work on 4 NVIDIA A100 GPUs

Matrix	SuperLU_DIST(s)	PanguLU(s)	Our work(s)	Speedup Our work vs SuperLU	Speedup Our work vs PanguLU
apache2	14.00	4.30	3.67	3.82x	1.17x
ASIC_680k	26.01	18.99	4.66	5.58x	4.08x
cage12	77.48	35.19	23.08	3.36x	1.52x
CoupCons3D	8.06	9.32	5.81	1.39x	1.61x
dielFilterV3real	30.80	17.90	16.23	1.90x	1.10x
ecology1	6.14	1.22	1.24	4.96x	0.98x
G3_circuit	16.92	3.04	2.64	6.42x	1.15x
inline_1	15.15	4.78	4.43	3.42x	1.08x
language	177.27	25.56	15.60	11.36x	1.64x
boneS10	21.35	8.05	7.17	2.98x	1.12x
GEOMEAN				**3.84x**	**1.40x**

PanguLU. For ecology1, the trend of nonzero distribution is almost linear, which means that the nonzeros are distributed along the diagonal uniformly in most region. So our irregular blocking method has no effect in parallel computing. At the same time, our method partitioning ecology1 with larger sizes brings a little reduction in speedup. Therefore, for those sparse matrices with irregular nonzero distribution, our proposed method can achieve a significant speedup in parallel computing.

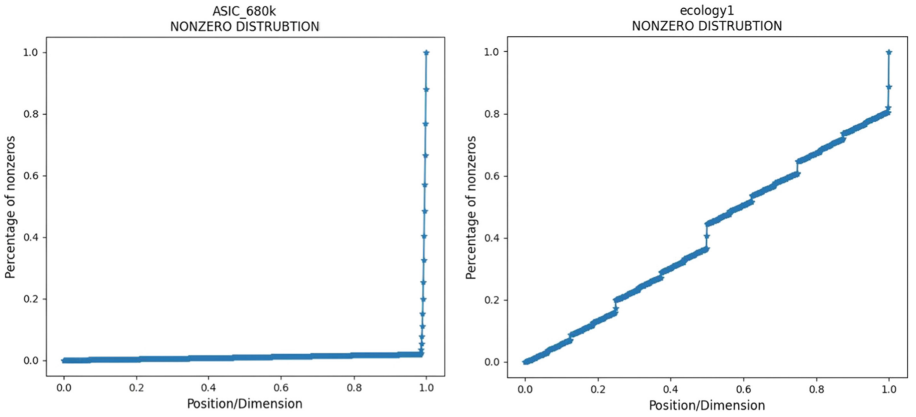


Fig. 11. The nonzero distribution of ASIC_680k (left) and ecology1 (right).

We also conduct experiments on PanguLU with all block sizes in its options and select the best performance on 4 NVIDIA A100 GPUs. Figure 12 shows the experimental results. PanguLU_Best achieves an average speedup of 1.17x over PanguLU, with speedups ranging from 1.0x to 2.60x. Our proposed irregular blocking can also achieve better performance than regular blocking. The results in parallel computing show the scalability of our approach and its advantages over regular blocking.

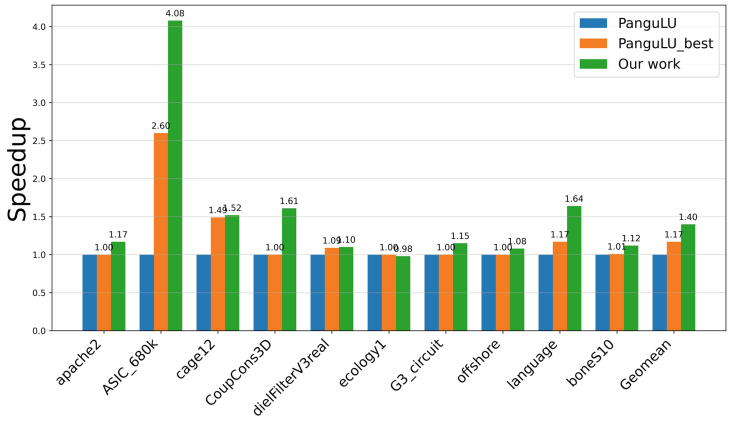


Fig. 12. Relative performance speedups on 4 NVIDIA A100 GPUs.

5.4 Preprocessing Cost

Regular blocking with the same size in PanguLU makes preprocessing easier and more efficient. Irregular blocking needs to store sizes of block in a size array. The array will be frequently looked up in the preprocessing phase to perform irregular blocking, which causes a longer time of preprocessing. Since the performance in numerical factorization accounts for the largest proportion of the whole LU factorization, it is acceptable to improve the performance of numerical factorization at the cost of increased preprocessing time. Besides, optimizing the performance of preprocessing will be our research in the future.

6 Conclusion

In this paper, we propose a novel diagonal block-based feature, which is described by percentage of nonzeros along the diagonal, can characterize global and local nonzero distribution of sparse matrices. Furthermore, a structure-aware irregular blocking method is proposed based on the local nonzero distribution. The irregular blocking method adopts the strategy of fine-grained in dense regions and coarse-grained in sparse regions. This strategy can balance the number of nonzero elements between levels in the dependency tree and the number of nonzero elements among blocks within the same level. Experiments on test sparse matrices show that our proposed irregular blocking method achieves speedups of performance in numerical factorization compared with PanguLU and SuperLU_DIST. On a single NVIDIA A100 GPU, it achieves geometric means of 1.50x and 3.32x, respectively. Also, it achieves speedups of 1.40x and 3.84x over PanguLU and SuperLU_DIST on 4 NVIDIA A100 GPUs.

References

1. Bollhöfer, M., Schenk, O., Janalik, R., Hamm, S., Gullapalli, K.: State-of-the-art sparse direct solvers. Parall. Algorithms Comput. Sci. Eng. 3–33 (2020)
2. Booth, J.D., Rajamanickam, S., Thornquist, H.: Basker: a threaded sparse lu factorization utilizing hierarchical parallelism and data layouts. In: 2016 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 673–682. IEEE (2016)
3. Chen, X.: Cktso: High-performance parallel sparse linear solver for general circuit simulations. IEEE Trans. Comput.-Aided Des. Integr. Circ. Syst. (2024)
4. Chen, X., Wang, Y., Yang, H.: Parallel sparse direct solver for integrated circuit simulation. Springer (2017)
5. Davis, T.A.: Umfpack-an unsymmetric-pattern multifrontal method with a column pre-ordering strategy. ACM Trans. Math. Software **30**(204), 196–199 (2004)
6. Davis, T.A.: Direct methods for sparse linear systems. SIAM (2006)
7. Davis, T.A., Hu, Y.: The university of florida sparse matrix collection. ACM Trans. Math. Softw. (TOMS) **38**(1), 1–25 (2011)
8. Davis, T.A., Palamadai Natarajan, E.: Algorithm 907: Klu, a direct sparse solver for circuit simulation problems. ACM Trans. Math. Softw. (TOMS) **37**(3), 1–17 (2010)
9. Davis, T.A., Rajamanickam, S., Sid-Lakhdar, W.M.: A survey of direct methods for sparse linear systems. Acta Numer **25**, 383–566 (2016)
10. Demmel, J.W., Eisenstat, S.C., Gilbert, J.R., Li, X.S., Liu, J.W.: A supernodal approach to sparse partial pivoting. SIAM J. Matrix Anal. Appl. **20**(3), 720–755 (1999)
11. Demmel, J.W., Gilbert, J.R., Li, X.S.: An asynchronous parallel supernodal algorithm for sparse gaussian elimination. SIAM J. Matrix Anal. Appl. **20**(4), 915–952 (1999)
12. Duff, I.S., Heroux, M.A., Pozo, R.: An overview of the sparse basic linear algebra subprograms: the new standard from the blas technical forum. ACM Trans. Math. Softw. (TOMS) **28**(2), 239–267 (2002)
13. Duff, I.S., Reid, J.K.: The multifrontal solution of indefinite sparse symmetric linear. ACM Trans. Math. Softw. (TOMS) **9**(3), 302–325 (1983)
14. Fu, X., et al.: Pangulu: a scalable regular two-dimensional block-cyclic sparse direct solver on distributed heterogeneous systems. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14 (2023)
15. Jin, Z., Li, W., Bai, Y., Wang, T., Lu, Y., Liu, W.: Machine learning and gpu accelerated sparse linear solvers for transistor-level circuit simulation: A perspective survey. In: 2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC), pp. 96–101. IEEE (2024)
16. Khaira, M.S., Miller, G.L., Sheffler, T.J.: Nested Dissection: A survey and comparison of various nested dissection algorithms. Carnegie-Mellon University. Department of Computer Science (1992)
17. Li, X.S., Demmel, J.W.: Superlu_dist: a scalable distributed-memory sparse direct solver for unsymmetric linear systems. ACM Trans. Math. Softw. (TOMS) **29**(2), 110–140 (2003)
18. Li, Z., et al.: Spada: accelerating sparse matrix multiplication with adaptive dataflow. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 2, pp. 747–761 (2023)

19. Liu, J.W.: The role of elimination trees in sparse factorization. SIAM J. Matrix Anal. Appl. **11**(1), 134–172 (1990)
20. Mary, T.: Block Low-Rank multifrontal solvers: complexity, performance, and scalability. Ph.D. thesis, Université Paul Sabatier-Toulouse III (2017)
21. Schenk, O., Gärtner, K., Fichtner, W., Stricker, A.: Pardiso: a high-performance serial and parallel sparse linear solver in semiconductor device simulation. Futur. Gener. Comput. Syst. **18**(1), 69–78 (2001)

A Deep Reinforcement Learning Algorithm with Ordered Action Space for Budget-Aware Workflow Scheduling in Heterogeneous Clouds

Yanfen Zhang[1], Longxin Zhang[1(✉)], Lili Du[1], Zhihua Wen[1], Buqing Cao[1,2], Sheng Lin[1], and Jianguo Chen[3]

[1] School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou 412007, China
longxinzhang@hut.edu.cn

[2] Key Laboratory of Intelligent Sensing System and Security (Hubei University), Ministry of Education, Wuhan 430062, China

[3] School of Software Engineering, Sun Yat-Sen University, Zhuhai 519082, China

Abstract. The optimization problem of makespan for cloud workflow scheduling under budget constraints presents significant challenges due to the heterogeneity of cloud resources and the complexity of task dependencies. Although existing deep reinforcement learning scheduling algorithms show promise in addressing the inherent limitations of traditional methods, they often lack awareness of budget constraints and struggle to manage the dynamic action space induced by budgetary considerations. To address this issue, a perception coefficient generation mechanism specifically designed to adhere to budget constraints is proposed. Additionally, an ordered action space mapping method is developed to facilitate the transition from continuous to discrete actions. Leveraging these two innovations, the twin delayed deep deterministic policy gradient (TD3) framework is leveraged to propose a budget-constrained workflow scheduling algorithm called PO-TD3, aimed at generating feasible and efficient scheduling solutions. The experimental results show that compared to the baseline algorithms, the PO-TD3 algorithm converges efficiently, achieving a 100% success rate in generating scheduling schemes that satisfy budget constraints. Additionally, it achieves optimal performance in minimizing makespan, with an average reduction of 61.43% compared to state-of-the-art algorithms.

Keywords: Budget-aware · Deep reinforcement learning · Heterogeneous cloud computing · Makespan · Workflow scheduling

1 Introduction

Cloud computing operates on a pay-per-use business model that leverages virtualization technologies to provide various essential resources, such as computing

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 119–136, 2026.
https://doi.org/10.1007/978-981-95-8405-5_7

and storage, within an internet environment [1]. A heterogeneous cloud computing platform consists of a series of clusters of virtual machines (VMs) that employ different infrastructures and management technologies. These VMs are interconnected through communication or business networks and are deployed on disparate hosts in a decentralized manner [2]. Cloud computing plays a crucial role in the management of large-scale workflows and in the processing of computationally intensive tasks due to its robust computational capabilities. A workflow application comprises numerous tasks with dependency constraints and is typically modeled using directed acyclic graph (DAG) [3]. This study imposes budget constraints on the entire workflow and aims to minimize the makespan while strictly adhering to these budgetary limitations.

For budget-constrained workflow scheduling in heterogeneous clouds, heuristic algorithms are often susceptible to task priority hierarchies, resulting in suboptimal solutions. Meanwhile, meta-heuristic algorithms typically require greater computational resources to converge on optimal solutions. Conversely, the deep reinforcement learning (DRL) approach formulates workflow scheduling as a Markov decision process (MDP) [4], offering a robust framework for addressing the workflow scheduling challenge. However, most current research on DRL models primarily focuses on the workflow scheduling problem with multi-objective optimization or emphasizes cost minimization under deadline constraints. In contrast, there is a notable scarcity of studies utilizing DRL models to address the makespan minimization problem under budget constraints.

In response to this gap, a perception coefficient generation (PCG) mechanism is developed to ensure adherence to budget constraints, and a scheduling framework based on the twin delayed deep deterministic policy gradient (TD3) [5] methodology is designed in this study. A DRL scheduling framework, named PO-TD3, is proposed to effectively tackle the challenges posed by dynamic action spaces under budget constraints by exploring an ordered action space (OAS) mapping approach, with the objective of minimizing makespan in heterogeneous cloud environments. The main contributions of this study are summarized below:

- It creates a PCG mechanism by utilizing a population-based evolutionary model within a meta-heuristic framework. This mechanism generates perception coefficients for each task in the workflow, which in turn establishes the upper limit of the disposable cost for each task, ensuring strict adherence to the budget constraints of the workflow.
- It explores the OAS mapping approach to facilitate transitions from continuous to discrete actions within an ordered action space, which comprises candidate VMs arranged by their finish times. This strategy allows for effective scheduling decisions related to task-VM assignments.
- It proposes a budget-constrained workflow scheduling algorithm, PO-TD3, by integrating the PCG mechanism with the OAS mapping method within the TD3 framework. The feasibility and efficiency of the PO-TD3 algorithm are validated through comparative experiments with similar target scheduling algorithms.

The remainder of this study is organized as follows. Section 2 provides a review of related work. Section 3 describes the scheduling model and problem formulation. Section 4 presents the proposed PO-TD3 algorithm. Section 5 discusses the experimental results and analyses. Finally, Sect. 6 summarizes the conclusions and outlines future work.

2 Related Work

In recent years, researchers have explored heuristics, metaheuristics, and DRL methods to address the challenges posed by complex workflow scheduling.

Heuristic algorithms develop suitable scheduling rules by leveraging a priori knowledge of the workflow. Zhang et al. [6] explored a scheduling strategy based on a predictive makespan matrix, focusing on the minimization of makespan in heterogeneous cloud environments. Kaur et al. [7] introduced the concept of virtual task clustering, which reduces data transfer time to zero by reallocating tasks to more energy-efficient VMs on the same host. This approach achieves both energy savings and makespan reduction while adhering to deadline constraints. Zhang et al. [8] conducted their research under budget constraints, employing iterative adjustments of task-resource mappings to reduce the length of the critical path.

Meta-heuristic algorithms have demonstrated distinct advantages in the realm of workflow scheduling. Li et al. [9] proposed a meta-heuristic algorithm known as the mutation-driven and population grouping poor and rich optimization algorithm (MG-PRO), which is specifically designed for scheduling workflows with budget constraints in cloud environments to minimize the makespan of workflows. Kushwaha and Singh [10] enhanced a meta-heuristic scheduling algorithm based on the Archimedean optimization approach, which possesses lower computational complexity. However, this method is highly sensitive to the initial population settings and parameter configurations.

DRL leverages the representation learning capabilities of deep neural networks to address the limitations of traditional reinforcement learning in high-dimensional state spaces. Tang et al. [2] modeled the task scheduling problem as a MDP and proposed a DRL-based task scheduling algorithm for multi-cloud environments, aiming to optimize both cost and makespan according to diverse user requirements. Chen et al. [11] developed a green-aware scheduling algorithm for high-performance computing clusters, utilizing a multi-action DRL framework to simultaneously optimize renewable energy utilization through proximal policy optimization.

However, heuristic algorithms exhibit a strong dependence on predefined rules, while meta-heuristic scheduling algorithms demonstrate a high degree of complexity in parameter configuration and the determination of relevant thresholds. Both types of algorithms tend to perform poorly when confronted with workflows characterized by diverse structural traits. Consequently, this study focuses on the scheduling problem of budget-constrained workflows and employs

the DRL method for research. The objective is to design an efficient scheduling algorithm that minimizes the makespan of workflows while satisfying budget constraints.

3 Model and Problem Formulation

This section introduces the workflow scheduling model, outlining the constraints and optimization objectives addressed in the research.

3.1 Workflow Scheduling Model

Heterogeneous cloud resource pools composed of VM instances that vary in processing power and lease prices are considered [12]. Let $\mathcal{V} = \{v_1, ..., v_k, ..., v_{|\mathcal{V}|}\}$ represent the set of VM instances, with $|\mathcal{V}|$ denoting the total number of rented VM instances. We assume that all VM instances are provisioned by the same data center and utilize the same average bandwidth bw [3]. A workflow application is typically modeled as a DAG, denoted as $\mathcal{G} = \{\mathcal{N}, \mathcal{E}\}$, where $\mathcal{N} = \{n_1, ..., n_i, ..., n_{|\mathcal{N}|}\}$ represents a collection of $|\mathcal{N}|$ nodes, each corresponding to an individual task that constitutes the workflow. $\mathcal{E}$ represents the set of directed edges, while $e_{i,j} \in \mathcal{E}$ indicates that task n_i must complete its execution before task n_j commences. $pred(n_i) = \{n_j \in \mathcal{N} | e_{j,i} \in \mathcal{E}\}$ and $succ(n_i) = \{n_j \in \mathcal{N} | e_{i,j} \in \mathcal{E}\}$ denote, respectively, the set of direct predecessors and the set of direct successors of task n_i. In a workflow DAG, an entry task n_{entry} is defined as a task with no predecessors, whereas an exit task n_{exit} is characterized as a task with no successors.

3.2 Problem Formulation

Assuming that task n_i is assigned to a VM v_x and task n_j is assigned to a VM v_y, with the size of the data transferred between n_i and n_j denoted as $TD_{i,j}$, the communication time is denoted as $CT_{i,j}$, which can be assessed by

$$CT_{i,j} = \begin{cases} TD_{i,j}/bw, & \text{if } x \neq y; \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The start time and finish time (FT) of a task n_i executing on a VM v_k are denoted as ST_i and FT_i, respectively. Their computational expressions are as follows:

$$ST_i = \max\left\{RT_k, \max_{n_j \in pred(n_i)}\left\{FT_j + CT_{j,i}\right\}\right\}, \tag{2}$$

$$FT_i = ST_i + ET_{i,k}, \tag{3}$$

where RT_k represents the ready time of VM v_k, and $ET_{i,k}$ indicates the execution time required for task n_i to be processed on VM v_k.

The makespan M of a workflow is defined as the total time from the start of execution to completion. Its computational expression is as follows [13]:

$$M(\mathcal{G}) = \max_{n_i \in \mathcal{N}} \left\{ FT_i \right\}. \tag{4}$$

The execution cost of a task n_i on a VM v_k is denoted as $Cost(n_i, v_k)$, which can be evaluated by

$$Cost(n_i, v_k) = ET_{i,k} \times P_k, \tag{5}$$

where P_k is the unit execution price of VM v_k. The scheduling cost C of a workflow is defined as the sum of the execution costs of all tasks in the workflow on the VM.

We investigate the budget-constrained workflow scheduling problem, aiming to assign workflow tasks to VMs while adhering to budget constraints B. The objective is to minimize the makespan by determining the optimal start and finish times for each task. The optimization problem can be formally defined as follows:

$$Minimize : M(\mathcal{G}), \tag{6}$$

$$Subject\ to : C(\mathcal{G}) \leq B. \tag{7}$$

4 PO-TD3 Algorithm

The PO-TD3 algorithm encompasses three essential components: the PCG mechanism, the OAS mapping method, and the TD3 algorithm. Specifically, the PCG mechanism is designed to meet the budget constraints of the workflow. The OAS mapping method is integrated into the Actor-Critic framework of the TD3 algorithm, aimed at assisting in optimizing the decision-making process by effectively assigning the appropriate VMs to individual tasks within the workflow to minimize the makespan. This section elaborates on the implementation principles of the PO-TD3 algorithm and presents its core formulation. The pseudo-code of the PO-TD3 algorithm is provided in Algorithm 1.

4.1 PCG Mechanism

Given that makespan and execution cost represent a pair of conflicting variables, a reduction in makespan typically results in an increase in cost. Perception coefficients are established for each task within the workflow, denoted as the set $\xi^* = \{\varsigma_1^*, ..., \varsigma_i^*, ..., \varsigma_{|\mathcal{N}|}^*\}$. The perception coefficient $\varsigma_i^* \in [0, 1]$, which dictates the disposable cost C_i^* associated with task n_i, can be calculated by

$$C_i^* = C_{\min}^i + \varsigma_i^* \times (C_{\max}^i - C_{\min}^i), \tag{8}$$

where $C_{\min}^i$ represents the minimum execution cost of task n_i, and $C_{\max}^i$ denotes the maximum execution cost of task n_i.

The objective of designing the perception coefficients is to establish an upper bound on the cost that can be allocated to a task by pre-assigning the disposable

cost to each task within the workflow. When assigning VMs to tasks, PO-TD3 will disregard any VMs with costs exceeding the task's cost ceiling, thus preventing the execution costs associated with the scheduling solution from breaching the budget constraint. To accomplish this, the PCG mechanism is proposed to determine the value ς_i^* of the perception coefficient, ensuring that the sum of the disposable costs of the tasks adheres to the budget constraint of the workflow:

$$\sum_{i=1}^{|\mathcal{N}|} C_i^* \leq B. \tag{9}$$

Inspired by the concept that metaheuristics seek optimal solutions through an iterative optimization mechanism, the design of the PCG mechanism is based on the population evolution model. This model encompasses the generation of the initial population, the formulation of evaluation rules, and the exploration and exploitation of the population.

(1) **Generation of Initial Population.** A random initial population $\mathbb{P} = \{X_1, ..., X_d, ..., X_\delta\}$ is generated within the search space to represent δ sets of perception coefficients, serving as the starting point for the search. In this context, δ indicates the population size, with individual $X_d = \{x_1, ..., x_i, ..., x_{|\mathcal{N}|}\}$ in the population representing a set of perception coefficients, and $x_i \in [0, 1]$ denoting the value of the perception coefficient corresponding to task n_i. According to the design objectives of the PCG mechanism, the fitness value f_d of individual X_d is determined by the sum of the disposable costs associated with the corresponding tasks of individual X_d:

$$f_d = \sum_{i=1}^{|\mathcal{N}|} C_i^*, \tag{10}$$

where C_i^* is calculated according to Eq. (8), with $\varsigma_i^* = x_i$.

(2) **Formulation of Evaluation Rules.** The scheduling problem addressed considers cost as a constraint. Specifically, the scheduling cost is required to meet the budget constraint without necessitating minimization. This implies that the closer the cost approaches the budgeted value, the more efficiently it can be utilized to reduce the makespan, provided that the cost remains within the budget constraint. Therefore, for any two sets of perception coefficients in the population, X_1 and X_2, the following three evaluation rules are formulated:

- If neither set satisfies the budget constraint, the set with the smaller corresponding sum of task disposable costs is considered preferable.
- If both sets satisfy the budget constraint, the set with the corresponding sum of task disposable costs that is closer to the budgeted value is deemed preferable.
- If one of the sets satisfies the budget constraint while the other does not, the set that meets the budget constraint is preferred.

(3) **Exploration and Exploitation of the Population.** In population-based meta-heuristic optimization algorithms, the exploration of the globally

optimal solution relies on the iterative updating of each individual's position within the search space. This process encompasses two fundamental components: exploration and exploitation. The weighted average algorithm (WAA) [14] is a meta-heuristic optimization method grounded in the concept of a weighted average position. Given the effectiveness and reliability of WAA in balancing exploration and exploitation, the exploration and exploitation strategies of WAA are adopted as the position update mechanism for individuals in the population. This approach aims to enhance the efficient convergence of the PCG mechanism and to obtain a set of perception coefficients that satisfy the budget constraint.

During the optimization process, each iteration requires determining whether the best individual $X_{best} = \{x_1, ..., x_i, ..., x_{|\mathcal{N}|}\}$ of the population obtained in that iteration satisfies the budget constraints. Once the best individual X_{best} in the population is identified as meeting the budget constraint (i.e., $f_{best} \leq B$), the iteration is stopped. The final solution $\xi^* = \{\varsigma_1^*, ..., \varsigma_i^*, ..., \varsigma_{|\mathcal{N}|}^*\}$ is determined as $X_{best} = \{x_1, ..., x_i, ..., x_{|\mathcal{N}|}\}$, indicating that the perception coefficient ς_i^* for task n_i is equal to x_i.

Based on the PCG mechanism, the PO-TD3 algorithm ensures adherence to the essential prerequisite of budget constraints.

4.2 OAS Mapping Method

The workflow scheduling problem is formulated as a MDP [15], denoted as $\langle O, A, r, \pi, \gamma \rangle$. In this formulation, $o \in O$ represents the observation values, $a \in A$ signifies the actions, and $r(o, a) : O \times A \rightarrow R$ denotes the reward function. The agent's policy, represented by π, predicts the action distribution based on the observation values at the current time step t, while γ is the discount factor. Assuming that the distribution of observations is represented by ρ, the Q-value function can be expressed as

$$Q(o_t, a_t) = E_{o_t \sim \rho, a_t \sim \pi(o_t)} \left[\sum\nolimits_{j=0}^{T} \gamma^j r_{t+j} | o_t, a_t \right], \tag{11}$$

where T is the termination time step.

(1) Observation Space

The observation value o_t is defined as follows:

$$o_t = \langle \omega_t, \vartheta_t, C_t, M_t \rangle, \tag{12}$$

where $\omega_t = t/T$ represents the current scheduling progress, $\vartheta_t = (B - C_t)/C_t$ denotes the degree of violation of the current budget constraint (with $\vartheta_t = 0$ indicating that the constraint is satisfied), C_t signifies the total cost of the current scheduling, and M_t indicates the current makespan.

(2) Action Space

Due to budget constraints, when assigning VMs to tasks, the algorithm only considers VMs with a cost less than or equal to the task's disposable cost as candidates. As a result, the set of available VMs corresponding to different tasks

varies, which leads to a dynamically changing action space at each time step. Given this variability, it is not feasible to directly output the probability distribution in the discrete action space for action selection. In light of this situation, a two-stage decision-making approach is adopted.

- Determine the output result $a_t^{(1)}$ (i.e., the action output a_t of the policy network) within the continuous action space.
- Design an OAS mapping method to convert $a_t^{(1)}$ into the discrete action space, resulting in $a_t^{(2)}$, which will be used to identify the VM for executing the task.

The OAS mapping method based on ordered action spaces is explored to facilitate the completion of the transformation $a_t^{(1)} \rightarrow a_t^{(2)}$. At the current time step t, the decision-making mechanism of the agent aims to assign VM to task n_t. The OAS mapping method is established through three key definitions: VM filtering, FT sorting, and action localization.

Definition 1. *VM Filtering.*

The disposable cost C_t^* of task n_t is calculated using Eq. (8), where the perception coefficient ς_t^* is obtained through the PCG mechanism prior to decision-making. OAS traverses the queue L_t of available VMs for the VM acquisition task n_t, considering only those VMs whose usage cost is not greater than the task's available budget, while excluding any VMs deemed inappropriate due to their usage cost.

Definition 2. *FT Sorting.*

All VMs in L_t are sorted in ascending order based on the FT of task n_t to generate a queue ${L_t}' = \{l_1, l_2, ..., l_{Len}\}$, which represents an ordered discrete action space. Here, *Len* denotes the length of the queue ${L_t}'$.

Definition 3. *Action Localization.*

$a_t^{(1)}$ is employed to compute a precise index for locating a target VM in queue ${L_t}'$, thereby establishing the implicit mapping from $a_t^{(1)}$ to $a_t^{(2)}$:

$$ind = \left\lfloor a_t^{(1)} \times Len \right\rfloor, \tag{13}$$

where *ind* determines $a_t^{(2)} = l_{ind}$, indicating the assignment of task n_t to the *ind*-th VM in queue ${L_t}'$ for execution.

(3) Reward Function

The feedback reward r_t received by the agent for taking action a_t under observation o_t is defined as $r_t = -M_t$, with the objective of minimizing the makespan.

4.3 TD3 Algorithm

Given the focus on continuous action outputs, the TD3 [5] algorithm is selected to train the policy model. The TD3 algorithm adopts the Actor-Critic architecture and includes three sets of key neural networks: (1) the online actor network π_θ with parameter θ and its target network π_{θ^-} with parameter θ^-; (2) the online critic network Q_ϕ with parameter ϕ and its target network Q_{ϕ^-} with parameter ϕ^-; and (3) the second online critic network Q_ψ with parameter ψ and its target network Q_{ψ^-} with parameter ψ^-. The actor network (i.e., the policy network) outputs deterministic actions and maximizes the Q-value of the critic evaluation through the policy gradient. The double critic networks estimate the Q-value separately and independently, constructing the target value through the minima operation to mitigate the overestimation of the Q-value. All target networks utilize a soft update mechanism [15] to ensure training stability.

The proposed PO-TD3 algorithm, as outlined in Algorithm 1, involves the agent continuously interacting with the environment during the training process (lines 5–22). At the current time step t, the agent's action is determined by the OAS mapping method according to the current observation o_t (lines 8–11). After executing the action, the agent receives an immediate reward r_t and obtains the environment observation o_{t+1} at the next time step (line 12). The tuple (o_t, a_t, r_t, o_{t+1}) is then stored in the replay buffer RB (line 13). When the size of the experience data in the replay buffer reaches a specified batch size b, PO-TD3 randomly samples a batch of historical experience data from the buffer to train the network and update the network parameters (lines 15–20).

5 Experiments and Analyses

This section details the experimental setup, including the baseline algorithms and performance metrics used in the comparative experiments. This evaluation aims to assess and validate the effectiveness of the PO-TD3 algorithm.

5.1 Experimental Settings

The programming language employed is Python 3.9, and the deep learning framework is developed using the PyTorch 2.5.1 platform. The experiments utilize 32 heterogeneous VM instances, with configuration parameters informed by Amazon EC2 [2] to determine the types and pricing of the VM instances.

Datasets. Two typical real-world scientific workflow applications [6] are employed as test datasets for the experiments, illustrated in Fig. 1. These workflow applications include the LIGO workflow in the field of gravitational physics and the Gaussian elimination (GE) parallel application. Furthermore, to comprehensively assess the effectiveness of the PO-TD3 algorithm, the experiments have been conducted across various scales for both types of workflow task sets, categorized into small, medium, and large scales, as defined below: (1) Small:

Algorithm 1. PO-TD3

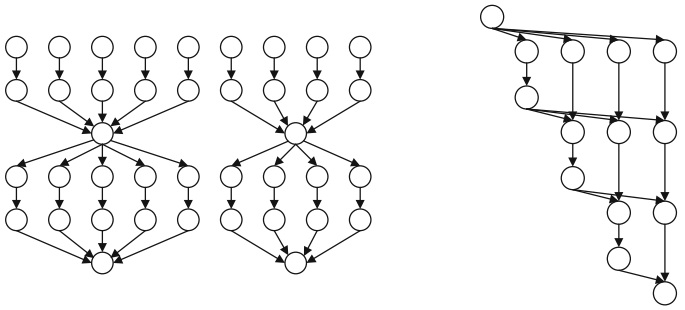
Input: Workflow dataset.
Output: Trained π_θ.

Initialize networks π_θ, Q_ϕ, and Q_ψ using parameters θ, ϕ, and ψ;
Initialize target networks $\theta^- \leftarrow \theta, \phi^- \leftarrow \phi, \psi^- \leftarrow \psi$;
Initialize replay buffer RB;
$\xi^* \leftarrow$ Obtain the set of perception coefficients by using the PCG mechanism;
for each training episode **do**
  Reset the scheduling environment and obtain o_t;
  for each time step t **do**
    $a_t^{(1)} \leftarrow$ Generate a continuous action a_t;
    Obtain the perception coefficient ς_t^* of the current scheduling task n_t;
    Obtain the discrete action $a_t^{(2)}$ by using the OAS mapping method;
    Execute the action in the scheduling environment;
    Obtain r_t and o_{t+1};
    Store (o_t, a_t, r_t, o_{t+1}) to RB;
    $o_t \leftarrow o_{t+1}$;
    if there is enough experience data in RB **then**
      Sample a batch of size b of (o_i, a_i, r_i, o_{i+1}) from RB;
      Update Q_ϕ and Q_ψ;
      Update π_θ;
      Update target networks;
    end if
  end for
end for
return π_θ.

The number of tasks is set to LIGO_40 and GE_27, with 3 VMs. (2) Medium: The number of tasks is set to LIGO_274 and GE_299, with 6 VMs. (3) Large: The number of tasks is set to LIGO_544 and GE_495, with 12 VMs.

Baseline Methods. To evaluate the performance of the PO-TD3 algorithm, a comparison is made with a meta-heuristic scheduling algorithm and a DRL scheduling algorithm.

- MG-PRO [9]: This algorithm is a meta-heuristic scheduling approach aimed at minimizing workflow makespan while adhering to budget constraints, aligning with the research objectives of the PO-TD3 algorithm. For the comparative experiments, the parameter settings from the original MG-PRO study are employed, with a population size set to 50 and a maximum of 1000 iterations.
- TD3 [5]: This refers to the original TD3 algorithm. To assess the effectiveness of the PO-TD3 algorithm, both the TD3 and PO-TD3 algorithms utilize the same observation space, action space, and parameter settings during the comparison experiments. However, it is important to note that the TD3 algorithm does not incorporate the PCG mechanism or the OAS mapping method, which are utilized in the PO-TD3 algorithm.



(a) LIGO (b) GE

Fig. 1. Structures of the two types of workflows.

Table 1. Parameter settings for the PO-TD3 algorithm.

Parameter	Value
Training episodes	300
Learning rate of actor	1e-4
Learning rate of critic	1e-4
Buffer size	1e6
Batch size (b)	1024
Discount factor (γ)	0.99
Soft update rate	0.005
Optimizer	Adam
Population size in PCG (δ)	30
Max iterations in PCG	300

Building on existing reinforcement learning research [16], we establish hyper-parameter test combinations. By comparing the performance of the PO-TD3 algorithm in terms of makespan and the convergence characteristics of each parameter at various values, we determine the optimal values for each parameter, as illustrated in Table 1.

Performance Metrics. Three performance metrics are employed to assess whether the scheduling schemes produced by the PO-TD3 algorithm and two baseline algorithms meet budgetary constraints and optimize the makespan. This evaluation aims to validate the effectiveness and applicability of the PO-TD3 theoretical model.

(1) **Success Rate.** A fundamental characteristic of a feasible scheduling scheme is that the cost incurred in executing the corresponding workflow remains

strictly within a predefined budget constraint. To facilitate a clear and intuitive analysis of the performance of both the PO-TD3 algorithm and the baseline algorithms in adhering to these budget constraints, the concept of success rate is introduced [9]. Specifically, under each set of predefined experimental conditions, multiple independent experiments are conducted, and statistics are collected regarding whether the scheduling solution generated from each experiment complies with the predefined budget constraints. The formula for the success rate SR is as follows:

$$SR = \frac{\chi^*}{\chi}, \tag{14}$$

where χ^* denotes the number of times a feasible scheduling solution is obtained in χ experiments. In this study, $\chi = 10$. A higher success rate indicates better performance of the algorithm.

(2) **Makespan.** Experiments are conducted to compare and analyze the makespan of the PO-TD3 algorithm and the baseline algorithms across different workflow scheduling environments. The analysis is conducted along three dimensions: budget constraints (Experiment 1), task sizes (Experiment 2), and the number of VMs (Experiment 3).

Experiment 1: We employ a medium-scale dataset to evaluate the performance of the PO-TD3 algorithm alongside baseline algorithms for scheduling the LIGO and GE workflows under varying budget constraints in terms of makespan. To investigate the influence of budget constraints on the algorithms in greater detail, we introduce a budget factor β to establish predetermined budgets with different levels of constraint for each workflow [3]. Then, the budget constraints $B(\mathcal{G})$ of the workflow $\mathcal{G}$ can be determined by

$$B(\mathcal{G}) = \beta \times C_{\min}(\mathcal{G}), \tag{15}$$

where $\beta \in [1.0, 1.4]$ with a variation step of 0.1. $C_{\min}(\mathcal{G})$ denotes the lower bound on the cost associated with executing workflow $\mathcal{G}$, representing the minimum cost necessary for the completion of workflow scheduling. When $\beta = 1.0$, the budget is at its most restricted state. As the value of β rises, the budget constraint becomes increasingly relaxed.

Experiment 2: We maintain a constant budget constraint looseness ($\beta = 1.4$) and fix the number of VMs ($|\mathcal{V}| = 12$). We evaluate the makespan of the PO-TD3 algorithm in comparison to baseline algorithms by employing two types of workflows, each containing three task sizes: small, medium, and large.

Experiment 3: We maintain a constant budget constraint looseness ($\beta = 1.4$) and utilize a medium-scale dataset to evaluate the makespan of the PO-TD3 algorithm in comparison to baseline algorithms. The number of VMs is set to 6, 12, 18, and 24, respectively.

(3) **Convergence.** To compare and analyze the convergence characteristics of the three algorithms, a medium-scale LIGO dataset with a budget constraint relaxation degree of $\beta = 1.4$ is utilized for testing.

5.2 Results and Analysis

Success Rate Analysis. The experimental results presented in Table 2 indicate that the average success rate of the MG-PRO algorithm for scheduling two types of workflows is 80.67%. However, it becomes increasingly challenging to identify a solution that adheres to the budget constraint when the budget is tightly constrained ($\beta = 1.0$), particularly in medium and large-scale datasets, where the algorithm essentially fails to yield a feasible scheduling solution, resulting in a success rate of 0. In comparison to the MG-PRO algorithm, the performance of the TD3 algorithm is slightly inferior. Specifically, in the small-scale test scenario of the LIGO workflow, TD3 requires the relaxation of the budget constraint to $\beta = 1.2$ to identify a scheduling solution that meets the specified criteria. In contrast, the proposed PO-TD3 algorithm, as a DRL scheduling method, achieves a 100% success rate. PO-TD3 demonstrates superior performance in satisfying budget constraints, attributed to the budget constraint-aware capabilities of the PCG mechanism. Notably, the design of the perception coefficients within the PCG mechanism establishes an upper threshold for the disposable cost associated with each task. This effectively filters out infeasible action choices in advance, thereby ensuring that the generated scheduling plan complies with the budget constraints.

Table 2. Success rates (%) of the three algorithms (S: Small, M: Medium, L: Large).

Dataset	β	TD3			MG-PRO			PO-TD3		
		S	M	L	S	M	L	S	M	L
LIGO	1.0	0	0	0	0	0	0	100	100	100
	1.1	0	100	100	100	100	100	100	100	100
	1.2	100	100	100	100	100	100	100	100	100
	1.3	100	100	100	100	100	100	100	100	100
	1.4	100	100	100	100	100	100	100	100	100
	Average	60	80	80	80	80	80	100	100	100
GE	1.0	0	0	0	20	0	0	100	100	100
	1.1	100	100	100	100	100	100	100	100	100
	1.2	100	100	100	100	100	100	100	100	100
	1.3	100	100	100	100	100	100	100	100	100
	1.4	100	100	100	100	100	100	100	100	100
	Average	80	80	80	84	80	80	100	100	100
Average		**76.67**			**80.67**			**100**		

Makespan Analysis. (1) The results of Experiment 1, presented in Fig. 2, illustrate the comparison of makespan under varying budget constraints. From the figure, it is evident that the makespan of both the MG-PRO and PO-TD3 algorithms decreases progressively as the budget constraints are relaxed. In contrast,

the makespan of the TD3 algorithm remains largely insensitive to variations in budget constraints. This discrepancy can be attributed to the fact that both the MG-PRO algorithm and the PO-TD3 algorithm incorporate explicit budget-aware mechanisms, enabling them to adapt to scheduling environments with diverse budget constraints. As a result, they can allocate additional resources to generate more effective scheduling solutions, thereby minimizing makespan when budget constraints are eased. Conversely, TD3 relies solely on the reward function to guide the learning process, which limits its effectiveness in addressing the budget-constrained workflow scheduling problem. In comparison with the TD3 and MG-PRO algorithms, PO-TD3 achieves the minimum makespan, demonstrating optimal performance with average reductions of 66.74% and 56.03%, respectively.

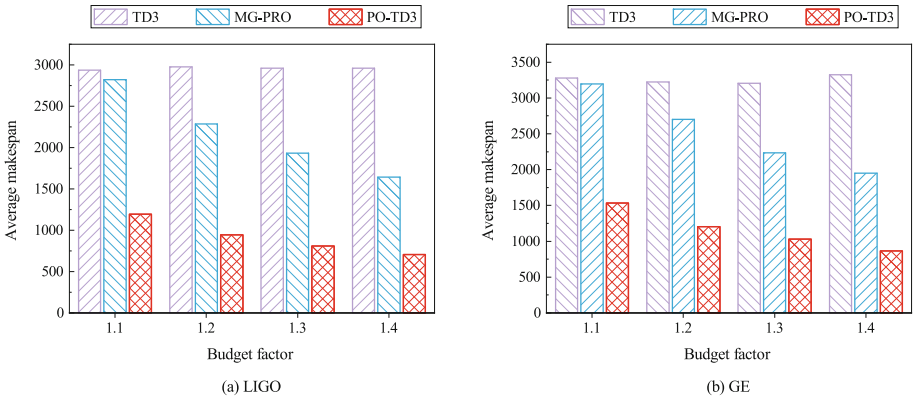


Fig. 2. Makespan comparison of the three algorithms under different budget constraints.

(2) The results of Experiment 2, which examines the makespan comparison under different task sizes, are presented in Fig. 3. As illustrated, the makespan of all three algorithms exhibits an upward trend corresponding to increases in task size. Notably, the PO-TD3 algorithm demonstrates a significant advantage over the two baseline algorithms as the task size varies. For instance, in the GE workflow depicted in Fig. 3(b), PO-TD3 achieves a reduction in makespan of 63.25% on average compared to TD3 and 50.30% compared to MG-PRO, showcasing its superior performance.

(3) The results of Experiment 3, which focuses on the makespan comparison under different numbers of VMs, are presented in Fig. 4. As the availability of VM resources for scheduling varies, PO-TD3 demonstrates optimal makespan performance when scheduling both the LIGO and GE workflows. For example, in the LIGO workflow shown in Fig. 4(a), PO-TD3 achieves an average makespan reduction of 66.16% compared to TD3 and 61.01% compared to MG-PRO.

In summary, the PO-TD3 algorithm exhibits significant performance advantages when addressing the diverse and complex environments encountered in

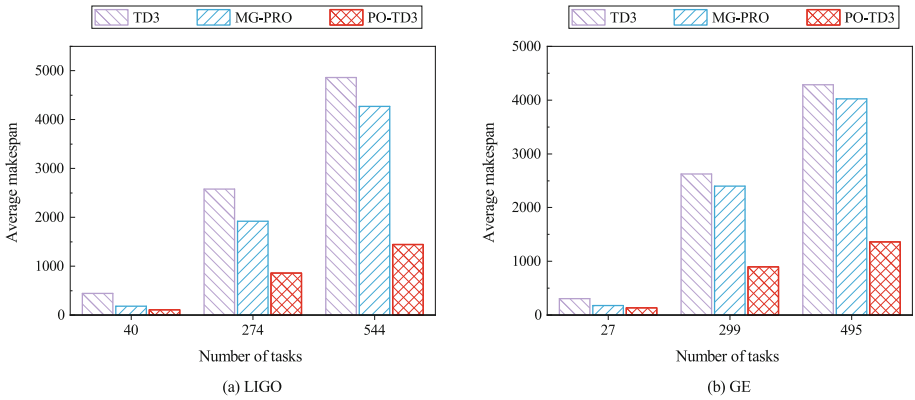


Fig. 3. Makespan comparison of the three algorithms for varying task sizes.

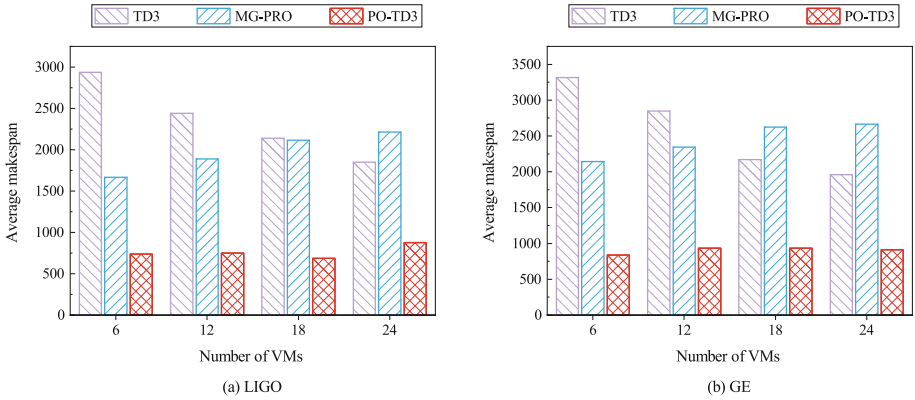
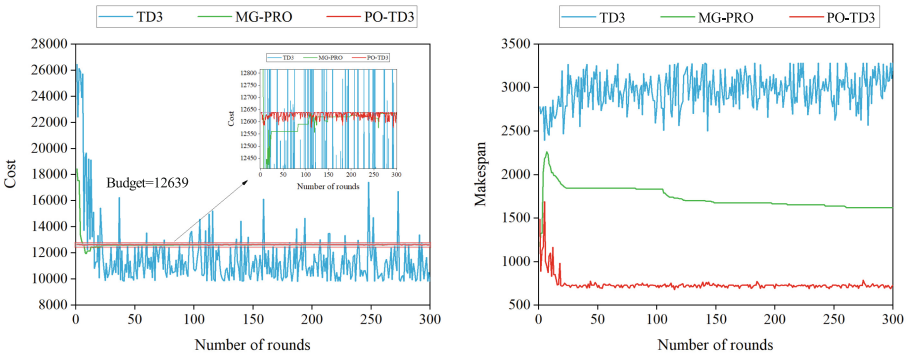


Fig. 4. Makespan comparison of the three algorithms with different numbers of VMs.

workflow scheduling. The integration of the PCG mechanism and OAS mapping method allows PO-TD3 to dynamically adjust its scheduling strategy based on varying environmental conditions, thereby meeting different scheduling needs and demonstrating strong adaptability and effectiveness. A comprehensive analysis of the comparative experimental data reveals that, compared to the TD3 and MG-PRO algorithms, PO-TD3 achieves superior results in the key metric of makespan, with average reductions of 66.15% and 56.71%, respectively. This performance provides a more efficient scheduling solution for tackling the intricate challenges of budget-constrained workflow scheduling.

Convergence Analysis. The convergence curves of the three algorithms applied to the LIGO workflow are illustrated in Fig. 5. In each iteration, the cost data presented in Fig. 5(a) corresponds to the makespan data depicted in Fig. 5(b), collectively reflecting the total scheduling cost and makespan gener-

ated by the scheduling schemes employed by the algorithms during that round. During the initial phase of training for the TD3 algorithm (approximately the first 50 iterations), the cost curve exhibits a decreasing trend. This trend is attributable to the necessity of adhering to the budget constraint, which serves as a fundamental prerequisite. The immediate rewards derived from the scheduling environment guide the algorithm towards learning strategies that fulfill this budget constraint. However, cost and makespan represent conflicting objectives; as the cost gradually decreases, the makespan of the workflow tends to increase. Although TD3 ultimately converges to a state that satisfies the budget constraint, its convergence appears premature, indicating a failure to continue in-depth learning regarding strategies to minimize the makespan once the budget constraint is met. In contrast, MG-PRO initially prioritizes cost reduction to explore feasible solutions that comply with the budget constraint, subsequently shifting its optimization focus toward minimizing the makespan. Consequently, the makespan curve for MG-PRO demonstrates an initial increase followed by a decrease. While MG-PRO outperforms TD3 in satisfying the budget constraint and reducing makespan, PO-TD3 demonstrates superior convergence behavior. The cost curves presented in Fig. 5(a) demonstrate that the cost incurred by PO-TD3 at each iteration can be strictly controlled within the budget constraint, a capability that can be attributed to the PCG mechanism it employs. Furthermore, the reward function of the PO-TD3 algorithm is exclusively influenced by the makespan, allowing it to consistently focus on strategies aimed at optimizing the makespan throughout the 300 iterations. This commitment results in a more favorable convergence effect regarding the minimization of makespan.



(a) Cost curves for LIGO (b) Makespan curves for LIGO

Fig. 5. Convergence curves of the three algorithms on the LIGO workflow.

6 Conclusion and Future Work

A DRL-based scheduling algorithm, PO-TD3, is proposed to address the workflow scheduling problem with budget constraints in heterogeneous cloud envi-

ronments while minimizing makespan. The PCG mechanism incorporated in the PO-TD3 algorithm establishes an upper limit on the available cost for each task by generating perception coefficients, thereby ensuring that the entire workflow adheres to budget constraints. Additionally, the OAS mapping approach effectively tackles the challenges posed by the dynamic action space and successfully facilitates the transition from continuous to discrete actions. A series of comparative experiments on scientific workflows validate the effectiveness of the PO-TD3 algorithm, which demonstrates superior convergence performance against baseline algorithms, achieving a 100% success rate in meeting budget constraints and an average reduction of 61.43% in the makespan metric. Future work will focus on developing workflow scheduling algorithms that not only satisfy budget constraints but also address task privacy and security requirements, aiming to provide more practical and secure solutions for workflow scheduling.

Acknowledgments. This work was partially funded by the Scientific research and innovation Foundation of Hunan University of Technology (Grant No. LXBZZ2416), Hunan Provincial Innovation Foundation For Postgraduate (Grant No. CX20251653), the Natural Science Foundation of Hunan Province (Grant No. 2023JJ50204), the Scientific Research Foundation of Hunan Provincial Education Department (Grant No. 25A0422), the National Natural Science Foundation of China (Grant Nos. 61702178, 62572186, 62372486), the Open Project Funding of the Key Laboratory of Intelligent Sensing System and Security (Hubei University), Ministry of Education, and the National Key R&D Program of China (Grant No. 2018YFB1003401).

References

1. Ye, L., Yang, L., Zhao, X., Xia, Y.: An energy-aware multi-stages hybrid scheduling approach for Iot workflow applications with reliability constraint in cloud computing systems. IEEE Internet Things J. (2025)
2. Tang, X.: Cost and makespan-aware task scheduling with deep reinforcement learning in multicloud environments. IEEE Transactions on Computational Social Systems, pp. 1–14 (2025)
3. Zhang, Y., et al.: Adaptive-oriented mutation snake optimizer for scheduling budget-constrained workflows in heterogeneous cloud environments. Futur. Gener. Comput. Syst. **175**, 108118 (2026)
4. Zhang, L., Tan, R., Cao, B., Li, L.A.K., Li, K.: Ep-musto: entropy-enhanced DRL-based task offloading in secure multi-UAV-assisted collaborative edge computing. IEEE Internet of Things Journal (2025)
5. Fujimoto, S., van Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning, pp. 1587–1596, PMLR (2018)
6. Zhang, L., Ai, M., Tan, R., Man, J., Deng, X., Li, K.: Efficient prediction of makespan matrix workflow scheduling algorithm for heterogeneous cloud environments. J. Grid Comput. **21**(75), (2023)
7. Kaur, R., Kaur, G., Goraya, M.S.: EESF: energy-efficient scheduling framework for deadline-constrained workflows with computation speed estimation method in cloud. Parallel Comput. **124**, 103139 (2025)

8. Zhang, J., Li, X., Chen, L., Ruiz, R.: Scheduling workflows with limited budget to cloud server and serverless resources. IEEE Trans. Serv. Comput. **17**(4), 1766–1779 (2024)
9. Li, H., Chen, B., Huang, J., Abreu, J.R.C., Chai, S., Xia, Y.: Mutation-driven and population grouping pro algorithm for scheduling budget-constrained workflows in the cloud. Clust. Comput. **27**, 1137–1158 (2024)
10. Kushwaha, S., Singh, R.S.: Deadline and budget-constrained archimedes optimization algorithm for workflow scheduling in cloud. Cluster Comput. **28**(117), (2025)
11. Chen, R., Lin, W., Huang, H., Ye, X., Peng, Z.: Gas-marl: green-aware job scheduling algorithm for HPC clusters based on multi-action deep reinforcement learning. Futur. Gener. Comput. Syst. **167**, 107760 (2025)
12. Ye, L., Yang, L., Xia, Y., Zhao, X.: A cost-driven intelligence scheduling approach for deadline-constrained Iot workflow applications in cloud computing. IEEE Internet Things J. **11**(9), 16033–16047 (2024)
13. Zhang, L., Liu, D., Ai, M., Tan, R., Zeng, Z.: Reliability enhancement algorithm based on budget level in cloud-edge environments. Int. J. Embedded Syst. **16**(1), 9–22 (2023)
14. Cheng, J., Waele, W.D.: Weighted average algorithm: A novel meta-heuristic optimization algorithm based on the weighted average position concept. Knowl.-Based Syst. **305**, 112564 (2024)
15. Wang, X., Laili, Y., Zhang, L., Liu, Y.: Hybrid task scheduling in cloud manufacturing with sparse-reward deep reinforcement learning. IEEE Trans. Autom. Sci. Eng. **22**, 1878–1892 (2025)
16. Zhang, L., Tan, R., Zhang, Y., Peng, J., Liu, J., Li, K.: UAV-assisted dependency-aware computation offloading in device-edge-cloud collaborative computing based on improved actor-critic drl. J. Syst. Architect. **154**, 103215 (2024)

STM-WFBP: Selective Tensor Merging Method in Distributed Learning

Pingping Dong, Qingfen Yi, Lianming Zhang(✉), and Wensheng Tang(✉)

Hunan Normal University, Changsha, China
{zlm,tangws}@hunnu.edu.cn

Abstract. Communication is an important constraint for distributed deep learning training, and efficient communication scheduling methods such as tensor merging accelerate distributed training by merging tensors from model layers. However, existing optimizations perform tensor merging greedily, failing to consider the occurrence of tensor stagnation for successive merging of multiple layers, resulting in suboptimal training performance. In this paper, we propose the selective tensor merging method STM-WFBP to guide the tensor merging process. This method intelligently determines whether the solvable layers are effectively merged and selects the effectively merged layers, thus avoiding the phenomenon of tensor retention. Experimental results on two GPU clusters show that the STM-WFBP achieves a 4%-12% optimization in iteration time over existing methods, which proves its effectiveness.

Keywords: Distributed deep learning · Communication scheduling · Gradient communication

1 Introduction

In recent years, artificial intelligence technology represented by deep learning has made breakthroughs in application areas such as image recognition [1], speech recognition [2] and autonomous driving [3]. As the deep neural network model structure becomes more and more complex, the number of network layers and parameters keeps increasing, the performance improvement of a single computing engine enters a bottleneck. Distributed deep learning is commonly used to accelerate the training of deep neural networks (DNNs) [4,5], and data parallel training is one of the popular methods for distributed deep learning training. In a cluster of N nodes, data parallel training divides the data into N batches and assigns them to these N nodes, each node processes the assigned training data and completes the forward propagation and backpropagation, and at the end of each training iteration, each node needs to synchronize the computed gradients to update the global model parameters [6–8]. However, with the increase in the number of computing nodes and the dramatic increase in model parameters, the communication between nodes increases significantly, and communication

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 137–150, 2026.
https://doi.org/10.1007/978-981-95-8405-5_8

gradually becomes a performance bottleneck [9–11]. For example, the computational power of Nvidia GPU has grown 30 times in the past 10 years, while it took about 15 years to increase the network speed from 10Gbps to 100Gbps. Therefore, optimizing communication performance is crucial in distributed deep learning [12,13].

In mainstream deep learning engines such as PyTorch [14], TensorFlow [15], and MXNet [16], the DNN training process can often be represented as a directed acyclic graph (DAG) (Fig. 1). DAG consists of two sets of computational operations and one set of communication operations. Since these two types of operations do not occupy the same resources, communication tasks can overlap with computing tasks [17]. Based on this, the researchers overlapped backpropagation with gradient communication and proposed a wait-free backpropagation algorithm WFBP [18] (Fig. 2(a). Due to the relatively few parameters of many DNN layers, the amount of gradient communication data is small, and the wait-free backpropagation algorithm introduces communication overhead every time a gradient transmission is performed, which leads to low network utilization. In recent years, researchers Yang *et al.* [19] have noticed this problem and proposed a single-layer communication SyncEASGD method (Fig. 2(b), which merges the gradients of all layers of DNN after the backpropagation of each iteration is completed, which can reduce most of the communication overhead, but the computational task and the communication task do not overlap. Shi *et al.* also noticed this problem and proposed a tensor merging method MG-WFBP (Fig. 2(c) to reduce communication overhead by merging adjacent layer gradients and overlapping communication tasks with computational tasks [20]. However, the gradient merging method MG-WFBP greedily merges tensors, resulting in the tensors of multiple layers being merged and unable to be transmitted in time, making the training iteration time suboptimal.

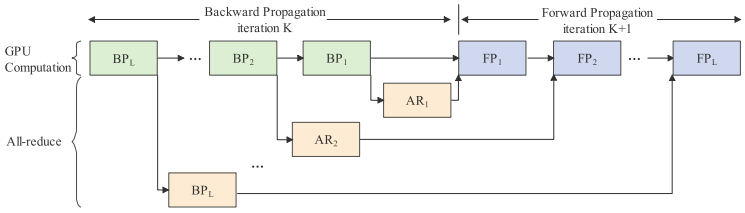


Fig. 1. Dependency DAG in distributed DNN training.

In order to solve this problem, we first formulate the communication scheduling problem in distributed training as an optimization problem with the goal of minimizing the total training time of each iteration. Then, a selective tensor merging wait-free backpropagation STM-WFBP method is proposed. Our main contribution consists of three parts. First, we were the first to propose the Selective Tensor Merge Wait-Free Backpropagation Algorithm STM-WFBP, which well solves the important shortcomings of the Fusion Tensor Wait-Free

Backpropagation MG-WFBP method. Second, we implemented the STM-WFBP algorithm on PyTorch. Third, we verified the performance of STM-WFBP on two GPU clusters and compared it with WFBP, SyncEASGD, and MG-WFBP.

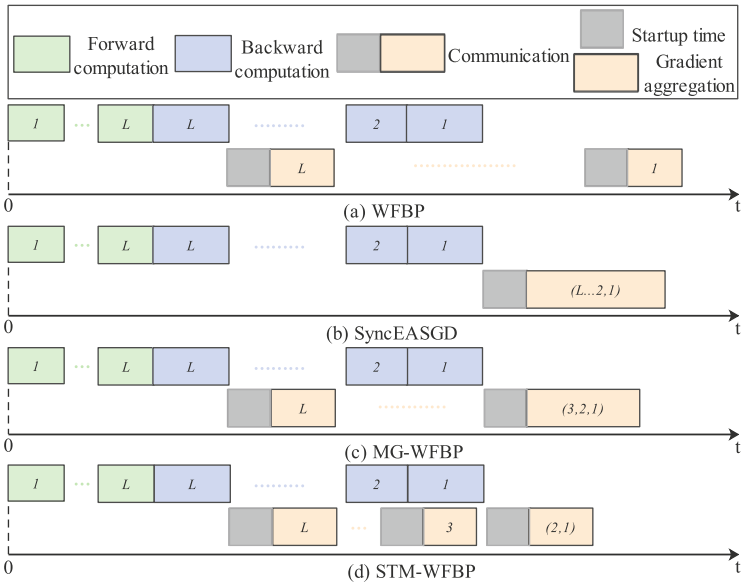


Fig. 2. Timeline of the four methods.

2 Related Work

There is a lot of work to alleviate the communication bottleneck of distributed training, and we mainly focus on the communication scheduling aspect. Communication scheduling can be done by changing the execution order of communication tasks and computing tasks to overlap computing and communication tasks, or by using the idea of pipelines to optimize scheduling. We summarize the relevant work in this aspect of communication scheduling.

Due to the layered structure of DNN models, communication tasks can overlap with computation tasks, making part of the communication cost hidden in the computation time [21]. The classical wait-free backpropagation algorithm WFBP [18] can enter communication immediately after the computation is completed without waiting for the completion of computation in all layers. DeAR [22] transmits tensors by dividing them into small tensor blocks and according to the priority. In order to increase the overlap degree of communication and computation, Song *et al.* proposed an overlap-aware start-up shared wait-free backpropagation algorithm [23]. US-Byte [24] schedules tensor blocks of varying

sizes in a near-optimal order, thereby improving bandwidth utilization and minimizing the time for one iteration. TriRace [25] introduces 3D communication scheduling, enabling efficient computation and communication overlap. DRFT [26] transforms the training problem into a knapsack problem and discovers new overlapping opportunities in the training to eliminate computational bubbles altogether. ASC-WFBP [27] utilizes simultaneous communication of tensors to improve network bandwidth utilization. PACE [28] and Prophet [29] discuss the optimal tensor partitioning strategy in priority scheduling. Bytedscheduler [30] makes use of priority scheduling to overlap communication with forward computation. Hashemi *et al.* find a fixed scheduling order for the tensor by using critical path analysis, and propose two heuristic algorithms-Time-Independent Communication Scheduling Algorithm, TIC, and Time-Associated Communication Scheduling Algorithm TAC, which are collectively referred to as TicTac [31]. Jayarajan *et al.* considered the order of parameter consumption and proposed a priority based parameter scheduling algorithm P3 [32]. SyncEASGD [19] combines all tensors into a single transmission once all the layers have been computed. MG-WFBP combines tensors based on backpropagation time and communication time of each layer to improve communication efficiency.

Despite many advances in communication scheduling, MG-WFBP greedily performs tensor layer merging and ignores its communication waiting time, leading to communication idling and tensor stalling.

3 Preliminaries

In this section, we introduce the WFBP training iteration time model and the All-Reduce communication model. For ease of presentation, we summarize the frequently used mathematical notations in Table 1.

3.1 WFBP Iteration Time Modeling

For DNNs with L layers, in WFBP, the communication of layer l can overlap with the backpropagation of layer $l-1$, and the communication can only start when layer l satisfies the following two conditions: (1) the backpropagation of layer l has been completed; and (2) the communication of layer $l+1$ has been completed. The time axis of WFBP is shown in Fig. 2(a). Assuming that the starting moment of forward propagation is 0, the moment t_b^l, when layer l starts to compute the gradient, can be expressed as:

$$\tau_b^l = \begin{cases} t_f & \text{if } l = L \\ \tau_b^{l+1} + t_b^{l+1} & \text{if } 1 \leq l < L \end{cases} \tag{1}$$

Layer l starts communicating at the moment τ_c^l, which can be expressed as:

$$\tau_c^l = \begin{cases} \tau_b^l + t_b^l & \text{if } l = L \\ \max\left\{\tau_c^{l+1} + t_c^{l+1}, \tau_b^l + t_b^l\right\} & \text{if } 1 \leq l < L \end{cases} \tag{2}$$

Table 1. Frequently used notations.

Notation	Description
N	The number of computing nodes in the cluster
a	Startup time of all-reduce
b	Transmission and computation time per byte of all-reduce
m_l	The number of parameters in layer l
t_f	Forward propagation time per iteration
t_b	Backpropagation time per iteration
t_c	Gradient aggregation time per iteration
τ_b^l	The moment when layer l starts to back propagate
t_b^l	Layer l back propagation time
τ_c^l	The moment when layer l starts communicating
t_c^l	Layer l communication time
t_p^l	Delay in communications after the merger of layers l
t_c^{no}	Non-overlapping communication time per iteration
t_{iter}	Iteration time for one training

The training iteration time includes forward propagation time, back propagation time and gradient aggregation communication time, and one iteration training time of WFBP can be expressed as:

$$t_{iter} = \tau_c^1 + t_c^1 = \max\left\{\tau_c^2 + t_c^2, \tau_b^1 + t_b^1\right\} + t_c^1 \tag{3}$$

3.2 All-Reduce Communication Model

Assuming an even number of compute nodes in the cluster, the cost of peer-to-peer communication can be modeled as $\alpha + \beta M$, where α is the startup time, β is the communication time per byte, and M is the message size. Then given N computing nodes, the time cost of all-reduce can be summarized as:

$$T_{ar} = a + b \times M \tag{4}$$

For a given cluster, the time cost of an all-reduce operation is a linear function of the message size M with intercept a and slope b. We verify this linear model in the experimental section later. Distributed training communication requires multiple invocations of the all-reduce operation. However, in each all-reduce operation, there is a startup overhead a that is independent of M. Moreover, $\mathrm{Tar}(M_1) + \mathrm{Tar}(M_2) > \mathrm{Tar}(M_1 + M_2)$, i.e., transmitting a single all-reduce operation of size $M_1 + M_2$ is more efficient than transmitting a message of size M_1 and a message of size M_2, respectively.

4 Methodology

4.1 Problem Formulation

Definition 1. *Ordinary merger Layer. Layer r is an ordinary merger layer if it is merged into layer r − 1 to defer its communication.*

Definition 2. *Ordinary Layer. A layer r is called an ordinary layer if it is not an ordinary merged layer.*

Each layer is either a ordinary merger layer or a ordinary layer. Our goal is to minimize the iteration time by first obtaining the ordinary merge layer and then selecting a valid ordinary merge layer merge layer among the ordinary merge layers. We denote the above two layer types by lm and lc, respectively, and use $D(r)(D(r) \in \{lm, lc\})$ to denote the type of the layer r. Given a network model with L layers, the solution is denoted as:

$$D = \left\{ \left[D^{(1)}, \ldots, D^{(k)}, \ldots, D^{(L)} \right] \mid D(r) \in \{l_m, l_c\} \text{ and } 1 \leq r \leq L \right\} \tag{5}$$

Our goal is to find a D to minimize the iteration time, which can be expressed as:

$$t = \max\{\tau_c^2 + t_c^2, \tau_b^1 + t_b^1\} + t_c^1 \tag{6}$$

4.2 Theoretical Analysis

Assuming that layer L, $L-1$, ..., Layer 3 is a ordinary layer and Layer 2 is a ordinary merge layer, then Layer 2 and Layer 1 are merged for transmission and the iteration time can be expressed as:

$$tt = \max\{\tau_c^2, \tau_b^1 + t_b^1\} + T_{ar}(m^1 + m^2) \tag{7}$$

Comparing Eq.(6) and Eq.(7), we need to find the conditions under which Layer 2 is an ordinary merger layer.

$$tt = \max\{\tau_c^2, \tau_b^1 + t_b^1\} + T_{ar}(m^1 + m^2) < t = \max\{\tau_c^2 + t_c^2, \tau_b^1 + t_b^1\} + t_c^1 \tag{8}$$

This will be discussed in the following scenarios.
Condition 1: $\tau_c^2 > \tau_b^1 + t_b^1$. then Eq.(8) can be simplified as:

$$\tau_c^2 + a + b \times (m^1 + m^2) < \tau_c^2 + 2a + b \times (m^1 + m^2) \tag{9}$$

Since $a > 0$, Eq.(9) holds and the iteration time is shorter when layer 2 is a ordinary merge layer.
Condition 2: $\tau_c^2 + t_c^2 < \tau_b^1 + t_b^1$. then Eq.(8) can be simplified as:

$$\tau_b^1 + t_b^1 \ + a + b \times (m^1 + m^2) < \tau_b^1 + t_b^1 \ + a + b \times m^1 \tag{10}$$

Eq.(10) clearly does not hold, so layer 2 is the ordinary layer.
Condition 3: $\tau_c^2 < \tau_b^1 + t_b^1 < \tau_c^2 + t_c^2$. then Eq.(8) can be simplified as:

$$\tau_b^1 + t_b^1 \ + a + b \times (m^1 + m^2) < \tau_c^2 + 2a + b \times (m^1 + m^2) \tag{11}$$

Eq.(11) can be reduced to:

$$\tau_b^1 + t_b^1 < \tau_c^2 + a \tag{12}$$

If Eq.(12) is true, then the second layer can be set as a ordinary merge layer so that the iteration time will be shorter. In summary, for Eq.(8), the iteration time can be reduced by setting the second layer as a ordinary merge layer under conditions 1 and 3. The layer type judgment for each layer is as follows:

$$\tau_c^r = \begin{cases} l_m & \text{if } \tau_b^{r-1} + \tau_b^{r-1} < \tau_c^r + a \text{ and } r > 1 \\ l_c & \text{otherwise} \end{cases} \tag{13}$$

When determining the layer type of each layer of a DNN, there are cases where multiple consecutive layers are ordinary merger layers. Figure 3 shows a 5-layer network model where layers 5 and 4 are ordinary merger layers, then these two layers wait for layer 3 to finish its computation and merge with layer 3 for transmission. For layer 5 that needs to wait for layer 3 to complete its computation before it can communicate, its communication wait time, or communication deferral time, t_p^5 is greater than the saved communication startup overhead. Therefore, we chose to leave the 5th ordinary merge layer unmerged so that neither communication idleness nor tensor stalling occurs. Layers 4 and 3 are also ordinary merger layer, and there is no lagging since the communication in layer 4 is communicated after the communication in layer 5 is completed, when the computation in layer 2 has already been completed. In summary, after judging the layer type of each layer of the model, since the continuous merging of multiple layers will lead to a tensor lagging situation, which will not reduce the iteration time instead, it is necessary to dynamically update the communication start moment of each layer and the communication postponement time of the ordinary merger layer in a timely manner, and carry out selective merging for the ordinary merger layer.

4.3 STM-WFBP Algorithm

Suppose there is a homogeneous distributed cluster of N nodes. We can measure the startup time and per-byte communication time of the all-reduce of Eq.(4) by NCCL-tests, so that the all-reduce communication time can be estimated for any message size. For the forward propagation and back propagation times, we can obtain the forward propagation and back propagation times for each layer of the model through several baseline tests before formally training the model. So, the all-reduce startup overhead, the moment of computation completion at each layer, and the moment of communication at each layer are all known.

For Algorithm 1, we take these known quantities as input and want to get the layer type D for each layer of the model and the tag S for whether or not to

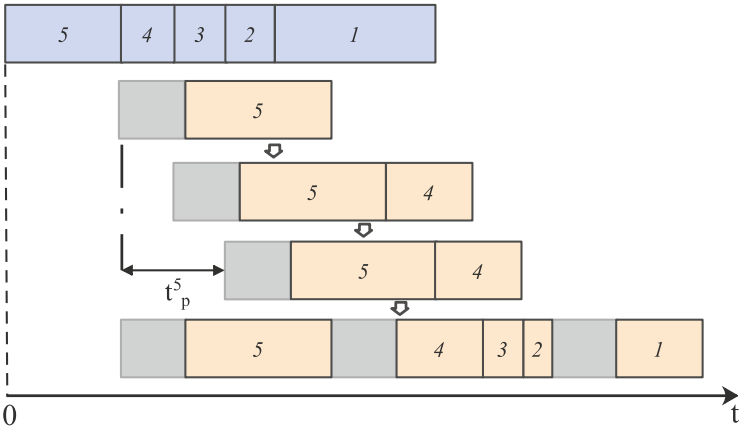


Fig. 3. An example of training a 5-layer network.

merge. The algorithm first assumes all layers as ordinary layers, and determines whether the layer is an ordinary merger layer by checking Eq. (12) (line 3), and if the layer is an ordinary merger layer, it needs to update the actual moment of the start of communication between the layer and all lower layers (line 8–15). It is also necessary to calculate the communication deferral time for the layer number higher than that layer (line 16–18). The STM-WFBP algorithm further evaluates the ordinary merging layer to determine whether to merge. The time complexity of STM-WFBP is $O(L^2)$. The algorithm is executed only once before formal training begins.

5 Experimental Evaluation

5.1 Experimental Setup

In order to verify the effectiveness of the STM-WFBP algorithm, we conducted experiments. The experimental platform includes two GPU clusters, 2-node and 4-node. The environment is deployed as Ubuntu 22.04, Horovod 0.19.1, NCCL 2.4.2, OpenMpi 4.0.1 and PyTorch 1.9.0.

We selected two datasets and five DNN models. The data consists of the cifar10 dataset and the cifar100 dataset, with Cifar10 consisting of approximately 50,000 training images and 10,000 validation images across 10 categories. The cifar100 dataset consists of approximately 50,000 training images and 10,000 test images across 100 categories. The five DNN models include GoogleNet, ResNet-50, ResNet-152, DenseNet-201, and DenseNet-161, and the training settings of the DNN models are shown in Table 2. We compare STM-WFBP with three representative baselines. (1) WFBP, which is the default optimization used by most deep learning frameworks. (2) SyncEASGD, a communication mechanism that merges all layer tensors and performs a single communication. (3) MG-WFBP, an efficient communication mechanism that finds corresponding tensor fusion schemes for different DNN models.

Algorithm 1. Get D, S

Input: $a, b, L, m[:], t_b[:], t_c[:], \tau_b[:], \tau_{cc}[:] = \tau_c[:], t_p[:]$
Output: D, S

for $r = L \rightarrow 2$ **do**
  if $\tau_b[r-1] + t_b[r-1] < \tau_c[r] + a$ **then**
    $D[r] = l_m$;
    UPDATEACT(t_b, τ_b, τ_c, r, m);
    UPDATECDTIME(τ_c, τ_{cc}, t_p, r);
    STM-WFBP(t_c, t_b, τ_b, τ_c, t_p, k, r, m);
return D, S
procedure UPDATEACT(t_b, τ_b, τ_c, r, m)
  $\tau_c[r-1] = \max\{\tau_c[r], \tau_b[r-1] + t_b[r-1]\}$;
  $m[r-1] = m[r] + m[r-1]$;
  for $s = r \rightarrow L$ **do**
    if $D[s] = l_m$ **then**
      $\tau_c[s] = \tau_c[r-1]$;
  for $k = r-2 \rightarrow 1$ **do**
    $\tau_c[k] = \max\{\tau_c[k+1] + T_{ar}(m[k-1]), \tau_b[k] + t_b[k]\}$;
procedure UPDATECDTIME(τ_c, τ_{cc}, t_p, r)
  for $s = L \rightarrow r$ **do**
    $t_p[s] = \tau_c[s] - \tau_{cc}[s]$;
procedure STM-WFBP(t_c, t_b, τ_b, τ_c, t_p, k, r, m)
  for $k = L \rightarrow r$ **do**
    if $t_p[k] > a$ **then**
      $S[k] =$ False;
      $\tau_c[k] = \max\{\tau_c[k+1] + t_c[k+1], \tau_b[k] + t_b[k]\}$;
      $\tau_c[r-1] = \max\{\tau_c[k] + t_c[k], \tau_b[r-1] + t_b[r-1]\}$;
      for $j = k-1 \rightarrow r$ **do**
        $\tau_c[j] = \tau_c[r-1]$;
      $m[r-1] = m[r-1] - m[k]$;
      for $s = r-2 \rightarrow 1$ **do**
        $\tau_c[s] = \max\{\tau_c[s+1] + T_{ar}(m[s-1]), \tau_b[s] + t_b[s]\}$;

5.2 Evaluation Of The STM-WFBP Algorithm

We measured All-Reduce communication on a 2-node cluster by using the nccl-tests benchmarking tool with a message size range of [8K, 1M], and the results are shown in Fig. 4. Figure 4 shows that the communication time cost of All-Reduce is almost linearly related to the message size, in line with Eq. (4).
We conducted performance evaluations of the WFBP, SyncEASGD, MG-WFBP, and SMG-WFBP algorithms on two clusters. To study the improvement of the STM-WFBP algorithm in overall training time compared to the other three algorithms, we recorded the average training time for 100 iterations when training five DNN models using different algorithms, with the results shown in Figs. 5 and 6.

Table 2. DNNs for evaluation

Model	#Tensors	#parameters	Batch Size
GoogleNet	59	∼13M	128
ResNet-50	161	∼25.5M	128
ResNet-152	467	∼60.1M	128
DenseNet-161	484	∼28.6M	64
DenseNet-201	604	∼20M	64

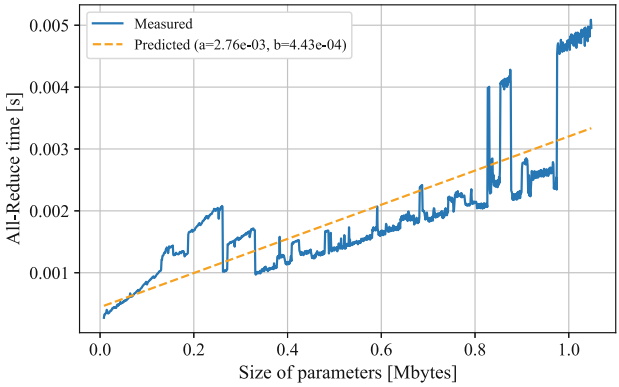


Fig. 4. All-reduce test.

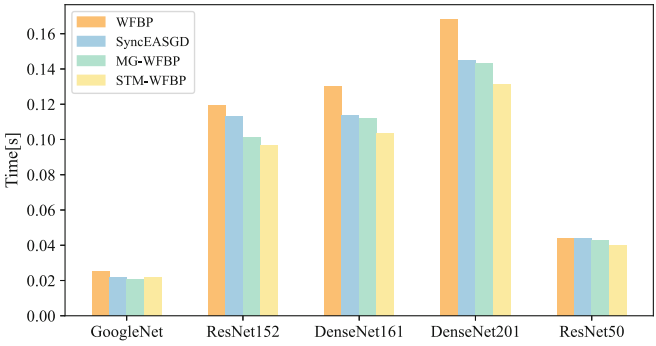


Fig. 5. Iteration time of two-node cluster.

For these five DNN models, STM-WFBP outperformed the other algorithms. The WFBP algorithm requires communication to be completed from the previous layer and its own gradient calculation to be finished before communication can occur. The SyncEASGD algorithm only begins communication after the backward propagation of the first layer is completed. The MG-WFBP algorithm

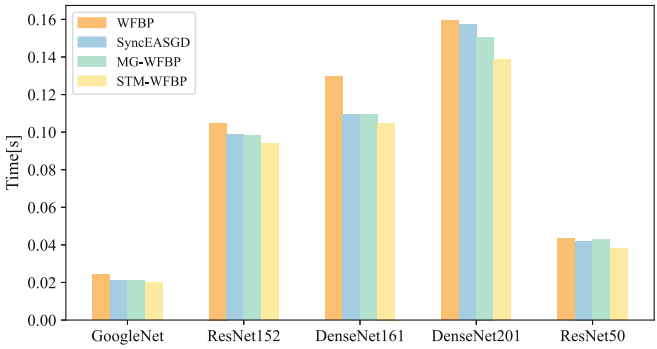


Fig. 6. Iteration time of four-node cluster.

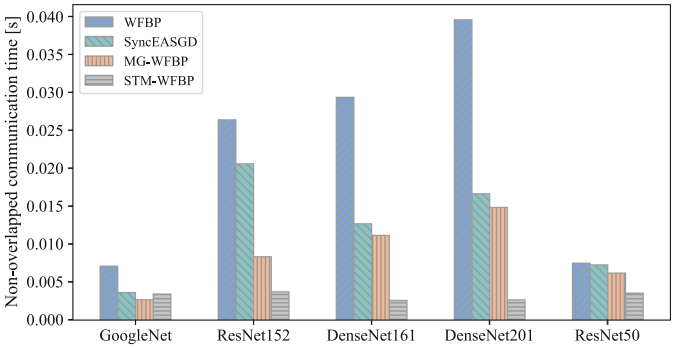


Fig. 7. Non-overlapping communication time of two-node cluster.

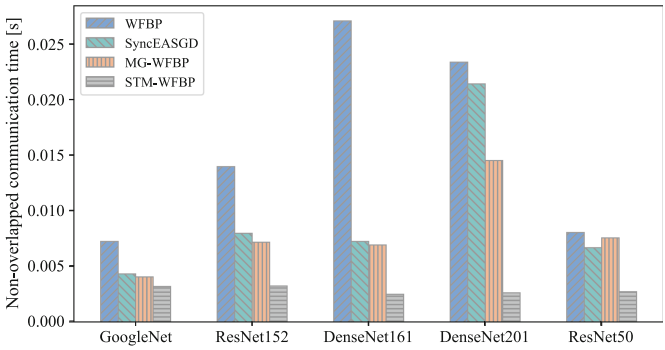


Fig. 8. Non-overlapping communication time of four-node cluster.

experiences delays in part of the tensor communication due to excessive merging of tensors.

In addition, to study the overlap of computation and communication in the training of the STM-WFBP algorithm and three other algorithms, we measured the non-overlapping communication time. For the same DNN model, the computation time for each iteration remains almost constant, so the non-overlapping communication time can effectively reflect the overlap between computation and communication (Figs. 7 and 8).
Non-overlapping communication time = iteration time - computation time; the smaller the non-overlapping time, the higher the degree of overlap between computation and communication. The results of the average non-overlapping communication time for five DNN models iterating 1000 times measured on two clusters are shown in Fig. 5(b) and Fig. 6(b). The SyncEASGD algorithm has a large non-overlapping communication time because its communication is not hidden. The MG-WFBP algorithm wastes some overlapping time due to delays in the communication of partial tensors.

6 Conclusion

In this paper, we emphasize that tensor merging should not be done greedily. To this end, we propose the algorithm STM-WFBP. This algorithm performs selective merging of tensors, which effectively merges tensors to reduce the communication startup time, and also prevents multiple layers of tensors from being in the communication wait phenomenon. Through experiments on two different clusters, we verify that on five deep neural networks, the STM-WFBP algorithm can achieve shorter training times than WFBP, SyncEASGD and MG-WFBP.

Acknowledgments. This work was partially supported by the National Natural Science Foundation of China (Nos. 62472170), the Open Project of Xiangjiang Laboratory (No. 25XJ03019), Hunan Province's Major Science and Technology Research Project (Nos. 2023ZJ1080 and 2024QK2009), Hunan Provincial Natural Science Foundation of China (No. 2025JJ50410), Open Fund of National Key Laboratory of Parallel and Distributed Computing (PDL) (No. WDZC20245250109), and Changsha Natural Science Foundation (No. kq2502073).

References

1. Qiao, S., Zhang, Z., Shen, W., et al.: Gradually updated neural networks for large-scale image recognition. In: Proceedings of the International Conference on Machine Learning, pp. 4188–4197. (2018)
2. Xiong, W., Wu, L., Alleva, F., et al.: The Microsoft 2017 conversational speech recognition system. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing, pp. 5934–5938. (2018)
3. Bojarski, M., Del Testa, D., Dworakowski, D., et al.: End to end learning for self-driving cars. arXiv preprint arXiv:1604.07316 (2016)
4. Verbraeken, J., Wolting, M., Katzy, J., Kloppenburg, J., Verbelen, T., Reller-Meyer, J.S.: A survey on distributed machine learning. ACM Comput. Surv. **53**(2), 1–33 (2020)

5. You, Y., Zhang, Z., Hsieh, C.-J., Demmel, J., Keutzer, K.: Fast deep neural network training on distributed systems and cloud TPUs. IEEE Trans. Parallel Distrib. Syst. **30**(11), 2449–2462 (2019)
6. Wang, E., Yan, R., Guo, Z., et al.: A survey of distributed training systems and their optimization algorithms. Chinese J. Comput. **47**(1), 1–28 (2024)
7. Lee, S., et al.: Improving scalability of parallel CNN training by adaptively adjusting parameter update frequency. J. Parallel Distrib. Comput **159**, 10–23 (2022)
8. Ouyang, S., Dong, D., Xu, Y., Xiao, L.: Communication optimization strategies for distributed deep neural network training: a survey. J. Parallel Distrib. Comput **149**, 52–65 (2021)
9. Cui, H., Zhang, H., Ganger, G. R., Gibbons, P. B., Xing, E. P.: Geeps: Scalable deep learning on distributed GPUs with a GPU-specialized parameter server. In: Proceedings of the Eleventh European Conference on Computer Systems. (2016)
10. Tang, Z., Shi, S., Chu, X., et al.: Communication-efficient distributed deep learning: A comprehensive survey. arXiv preprint arXiv:2003.06307 (2020)
11. Zhang, Z., Chang, C., Lin, H., et al.: Is network the bottleneck of distributed training? In: Proceedings of the Workshop on Network Meets Al & ML, pp. 8–13. (2020)
12. Wang, S., Li, D., Geng, J.: Geryon: Accelerating distributed CNN training by network-level flow scheduling. In: Proceedings of the IEEE International Conference on Computer Communications, Virtual Conference, pp. 1678–1687. (2020)
13. Geng, J., Li, D., Wang, S.: ElasticPipe: an efficient and dynamic model-parallel solution to DNN training. In: Proceedings of the 10th Workshop on Scientific Cloud Computing, pp. 5–9. (2019)
14. Paszke, A., et al.: Pytorch: an imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems, pp. 8024–8035. (2019)
15. Abadi, M., et al.: TensorFlow: a system for large-scale machine learning. In: Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, pp. 265–283. (2016)
16. Chen, T., et al.: Mxnet: A flexible and efficient machine learning library for heterogeneous distributed systems. arXiv preprint arXiv:1512.01274 (2015)
17. Duan, Q., Wang, Z., Xu, Y., Liu, S., Wu, J.: Mercury: a simple transport layer scheduler to accelerate distributed DNN training. In: Proceedings of the IEEE Conf. Computer Communication, pp. 350–359. (2022)
18. Zhang, H., et al.: Poseidon: an efficient communication architecture for distributed deep learning on GPU clusters. In: Proceedings of USENIX Conference on USENIX Annual Technical Conference, pp. 181–193. (2017)
19. You, Y., Bulug, A., Demmel, J.: Scaling deep learning on GPU and Knights Landing clusters. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 9. (2017)
20. Shi, S., Chu, X., Li, B.: MG-WFBP: merging gradients wisely for efficient communication in distributed deep learning. IEEE Trans. Parallel Distrib. Syst. **32**(8), 1903–1917 (2021)
21. Awan, A.A., Hamidouche, K., Hashmi, J.M., Panda, D.K.: Scaffe: co-designing MPI runtimes and Caffe for scalable deep learning on modern GPU clusters. In: Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, pp. 193-205. (2017)
22. Zhang, L., Shi, S., Chu, X., Wang, W., Li, B., Liu, C.: Decoupling the all-reduce primitive for accelerating distributed deep learning. arXiv preprint arXiv:2302.12445 (2023)

23. Song, Y., Tang, Z., Wang, Y., Xiao, X., Liu, Z., Xia, J., Li, K.: OASR-WFBP: an overlapping aware start-up sharing gradient merging strategy for efficient communication in distributed deep learning. J. Parallel Distrib. Comput **196**. (2025)
24. Gao, Y., et al.: US-Byte: an efficient communication framework for scheduling unequal-sized tensor blocks in distributed deep learning. IEEE Trans. Parallel Distrib. Syst. **35**(1), 123–139 (2023)
25. Li, S., et al.: A multidimensional communication scheduling method for hybrid parallel DNN training. IEEE Trans. Parallel Distrib. Syst. **35**(8), 1415–1428 (2024)
26. Meng, L., Sun, Y.: DeFT: Mitigating Data Dependencies for Flexible Communication Scheduling in Distributed Training. arXiv preprint arXiv:2503.16815 (2025)
27. Shi, S., Chu, X., Li, B.: Exploiting simultaneous communications to accelerate data parallel distributed deep learning. In: Proceedings of IEEE Conference on Computer Communications. (2021)
28. Bao, Y., Peng, Y., Chen, Y., Wu, C.: Preemptive all-reduce scheduling for expediting distributed DNN training. In: Proceedings of IEEE Conference on Computer Communications, pp. 626–635. (2020)
29. Zhang, Z., et al.: Prophet: Speeding up distributed DNN training with predictable communication scheduling. In: Proceedings of the 50th International Conference on Parallel Processing. (2021)
30. Peng, Y., et al.: A generic communication scheduler for distributed DNN training acceleration. In: Proceedings of the 27th ACM Symposium on Operating Systems Principles. (2019)
31. Hashemi, S.H., Jyothi, S.A., Campbell, R.: TicTac: accelerating distributed deep learning with communication scheduling. In: Proceedings of Machine Learning and Systems. (2019)
32. Jayarajan, A., Wei, J., Gibson, G., Fedorova, A., Pekhimenko, G.: Priority-based parameter propagation for distributed DNN training. In: Proceedings of Machine Learning and Systems, pp. 132–145. (2019)

EACC: Efficient Agent Context Cache Sharing for Multi-Agent Systems

Sihao Cheng[1,2], Yunfei Gu[1(✉)], Chentao Wu[1], and Jie Meng[2]

[1] Shanghai JiaoTong University, Shanghai 200240, China
gu.yunfei@sjtu.edu.cn
[2] Huawei Cloud, Chengdu 610095, China

Abstract. Large Language Models (LLMs) have demonstrated remarkable capabilities across diverse natural language processing and other complex tasks. However, multi-agent deployments of LLMs often suffer from significant inference overhead caused by a great deal of context and repeated tokens across agents, resulting in high Time-To-First-Token (TTFT) and elevated computational costs. To address these challenges, we propose EACC (Efficient Agent Context Cache), a system that enables position-independent reuse of key-value (KV) caches across agents by semantically chunking the prompt and managing reusable KV pairs through a globally accessible Agent Context Cache. EACC decouples context storage from computation, supports distributed deployment, and integrates with existing Position-Independent Caching (PIC) techniques such as CacheBlend and EPIC for selective recomputation. Our evaluation across multiple datasets and LLM backends demonstrates that EACC achieves up to 82.4% reduction in TTFT, while maintaining output accuracy, thereby offering a scalable and efficient inference infrastructure for multi-agent services.

Keywords: Large Language Models · Multi-Agent Systems · Position-Independent Caching · Distributed Inference

1 Introduction

Large Language Models (LLMs) are foundational to the development of intelligent agents capable of autonomous thinking, decision-making, and interaction with external tools. While single-agent frameworks have shown a promising prospect, they become inefficient in handling complex tasks requiring specialization and decomposition. Multi-Agent Systems (MAS) address this by distributing subtasks across multiple domain-specific agents. However, such architectures introduce massive redundancy in prompt tokens—due to repeated system prompts, tool descriptions, and inter-agent communication—amplifying the computational cost, particularly in the prefill stage of LLM inference [3,21]. Recent studies highlight that prefill computation dominates inference latency and that over 80% of tokens in MAS contexts are repeated [40]. Traditional

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 151–171, 2026.
https://doi.org/10.1007/978-981-95-8405-5_9

prefix caching fails in this setting due to its reliance on exact prefix alignment. Position-Independent Caching (PIC) strategies like CacheBlend [44] and EPIC [17] alleviate this limitation but are not directly optimized for multi-agent scenarios.

To bridge this gap, we present EACC, a system tailored for multi-agent LLM inference that enables reusable context sharing at the chunk level. EACC introduces a global Agent Context Cache (ACC), which maps prompt chunks to their corresponding KV caches, allowing for efficient lookup, sharing, and reuse regardless of chunk position. Integrated with a distributed Chunk KV Store and a cache-aware inference scheduler, EACC maximizes KV cache reuse and optimizes inference routing to minimize TTFT and boost throughput in both single-node and distributed deployments. Our results show that EACC achieves up to 82.4% TTFT reduction and 18.9% end-to-end latency reduction, while maintaining consistent accuracy in tasks such as question answering and tool invocation.

In summary, the contributions of our work are as follows:

- We identify key inefficiencies in multi-agent LLM serving, including redundant prefill computations due to lack of structured context sharing, ineffective reuse of KV caches in non-prefix settings, and suboptimal task scheduling in distributed environments.
- We propose EACC, a novel system that enables position-independent, chunk-level context cache reuse through a globally ACC, decoupling prompt chunk management from computation. We design and implement a distributed Chunk KV Store and a cache-aware Inference Instance Router, allowing efficient cache lookup, transmission, and task scheduling across instances to optimize system throughput.
- We integrate two state-of-the-art PIC mechanisms, CacheBlend and EPIC, into EACC and explore their trade-offs in multi-agent scenarios.
- We conduct extensive experiments on real-world datasets and LLMs, demonstrating that EACC achieves up to 82.4% reduction in TTFT and 94.7% token hit ratio, while maintaining inference quality across diverse workloads.

2 Preliminaries

2.1 Inference of LLMs

Mainstream LLMs, typically based on Transformer architecture, have inference processes divided into two stages: *prefill* and *decoding*, each exhibiting completely different computational characteristics and costs [3,31,47].

In the prefill stage, each token is processed through Transformer layers. In each layer, tokens are converted to queries (Q), keys (K), and values (V) to compute attention scores, which are used to generate output tokens. The computational complexity of the prefill stage is dominated by the self-attention mechanism, whose complexity is $O(N^2)$ with respect to the input length N [3,31,33]. Key-value pairs (KV cache) are then stored to avoid redundant computations in

the decoding stage. The first output token is generated at the same time to initialize the decoding stage. The metric *Time To First Token (TTFT)* [21,33,47] captures the elapsed time until the first token is produced and is closely approximated by the prefill latency.

During decoding, cached KV pairs are used to generate new tokens in an autoregressive manner. The complexity of this stage is $O(N)$, as it only interacts with historical KV pairs, making it more efficient than the prefill stage [16,33,47].

2.2 LLM-Based Agent

Single Agent. While an LLM generates outputs based on given inputs, it lacks the capability for continuous interaction with users or external systems [20]. Furthermore, LLMs cannot retrieve real-time information or adapt dynamically to runtime conditions and external environments, which often leads to hallucinations [19] and limited task performance. External tools and self-thinking are needed to enhance the LLM's ability [22,30].

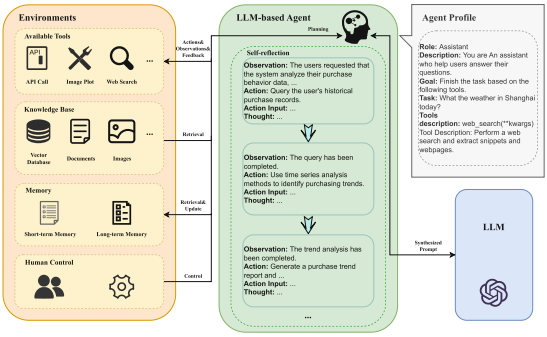


Fig. 1. An agent automatically performs tasks, interacts with environments and achieve self-evolution.

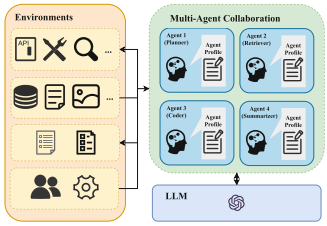


Fig. 2. The architecture of multi-agent systems.

The concept of an *Agent* was introduced to address these limitations [8,25,45]. An agent perceives its environments, makes decisions, and interacts with external media to compensate for the LLM's limitations, such as invoking APIs or querying databases [11]. By specifying the agent profile (system messages, tools description, etc.) and integrating external information (documents) and context (memory) [16,27,42], the LLM's power get greatly enhanced. However, the length of the prompt has been increased due to injecting more contextual information into it, significantly expanding the prefill consumption. Figure 1 illustrates the agent as a bridge between the user and the LLM.

Multi-agent. Single-agent systems are limited in handling complex tasks comprising multiple subtasks. *Multi-Agent Systems (MAS)* address this challenge by decomposing complex tasks into simpler subtasks, each handled by specialized agents, thereby improving overall efficiency [12,24,28,32,38]. MAS accomplishes complex objectives through collaboration among agents, where each specializes in distinct domains or functionalities such as code generation or web searching [24,26]. Figure 2 depicts this collaborative architecture, and Fig. 6 illustrates the task decomposition process.

2.3 Prefix Caching and Position-Independent Caching

In multi-user and multi-request inference environments, reusing KV caches substantially reduces redundant computations and prefill overhead [16,31,47]. Two prevalent techniques are *Prefix Caching (PC)* [17,21,46] and *Position-Independent Caching (PIC)* [14,17,44], both aiming to optimize TTFT.

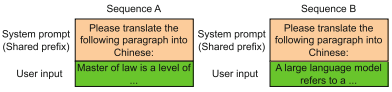


Fig. 3. The reuse of the prefix cache.

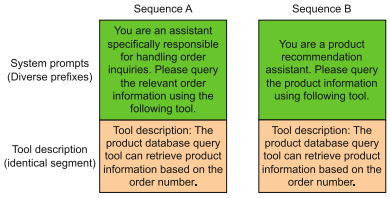


Fig. 4. One Scenario where prefix matching fails.

Prefix Caching relies on exact longest-prefix matching among requests (an example is shown in Fig. 1), allowing the system to bypass recomputing the KV caches of the matched prefix [21,46]. However, PC requires strict prefix alignment, since each token's KV cache is position-dependent, limiting its applicability in MAS scenarios. As illustrated in Fig. 2, even identical text segments[1] cannot share KV caches if their preceding prefixes differ (Figs. 3 and 4).

In contrast, PIC treats repeatable prompt chunks as cache units, enabling reuse regardless of their position within the prompt. Once the KV cache for a chunk is computed, it is stored and reused in subsequent requests containing the same chunk. Prefill is only executed on chunks absent from the cache. This mechanism resembles dynamic linking in software systems [17,18]. However, naïvely concatenating cached KV pairs can disrupt attention mechanisms and degrade output quality [3,14,44]. To address this, selective recomputation of a small subset of cached tokens is performed to restore cross-chunk attention. Representative methods include CacheBlend [44] and EPIC [17], which differ mainly in their algorithms for selecting tokens for recomputation. The overall architecture of PIC and the selective recomputation scheme are depicted in Fig. 5.

[1] We alternatively refer to them as *prompt chunks* as well.

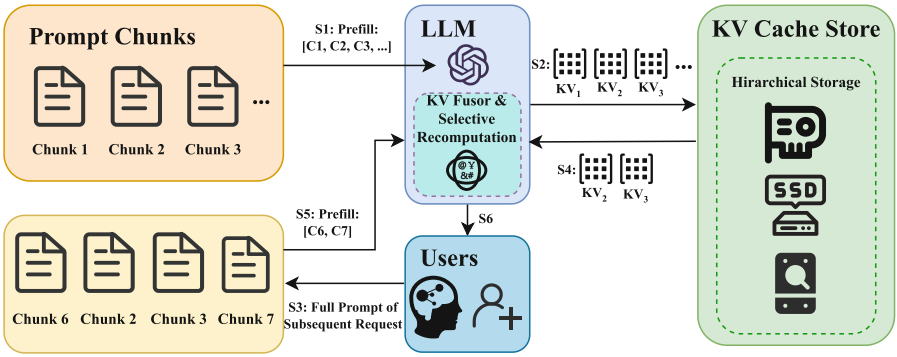


Fig. 5. An abstract of PIC mechanism. ① Prefill for the prompt chunks for the first time. ② Save the KV pairs to the cache. ③ One user makes a subsequent reuqest. ④ Check if there are any repetitions in the prompt chunks and reuse the cached KVs. ⑤ Perform the normal prefill computation for the uncached chunks and selectively compute a portion of KVs for the cached tokens. ⑥ Decode, and return the result to the user.

2.4 Relationship Between Token Hit Ratio and Prefill Consumption

As discussed in Sect. 2.1, prefill latency dominates inference delay for long inputs. To reduce redundant computation, inference services cache KV pairs of processed contexts (Sect. 2.3). In scenarios such as multi-turn dialogues, long context concatenation, or multi-agent services, efficient hit of KV cache can significantly reduce the prefill overhead.

Formally, let an inference request contain L tokens, among which L_{hit} tokens hit the KV cache directly, and the remaining $L_{miss} = L - L_{hit}$ tokens require computation during prefill. We define the *Token Hit Ratio* as:

$$h = \frac{L_{hit}}{L} \tag{1}$$

The FLOPs required for prefill on n tokens can be approximated as[2]:

$$flops(n) = l \cdot (an^2 d + bnd^2) \tag{2}$$

where a and b are constants, d is the model dimension, and l is the num of model layers. Assuming a per-FLOP computation time of c, and $k \ll L_{hit}$ tokens are selectively recomputed, the total prefill time $T(L, h)$ can be approximated as:

$$T(L,h) = c \cdot flops(L(1-h)+k) = c \cdot l \cdot \left[a(L(1-h)+k)^2 d + b(L(1-h)+k)d^2\right] + \epsilon \tag{3}$$

where ϵ denotes additional runtime overhead per prefill.

This relation indicates that, for fixed-length requests, prefill time is a monotonically decreasing superlinear function with respect to the token hit ratio h: higher hit ratios lead to substantially lower prefill costs.

[2] We adopt the calculation method described in [33].

3 Deficiency in Multi-Agent Serving Systems

However, as the number of agents in MAS increases, the system's total token consumption grows approximately linearly with the agent count. Additionally, inter-agent communication—such as using one agent's output as another's input—introduces a substantial amount of redundant tokens. Together, these factors exacerbate the inference burden, leading to increased latency. Prior work [40] reports token duplication ratios as high as 86% in such systems. Although some optimizations have been proposed [21,43,46], several critical limitations remain unresolved.

3.1 Lack of Structured Context Sharing

Despite the prevalence of token duplication in MAS—often manifested as repeated prompt chunks including system messages, tool descriptions, historical interactions, and intermediate agent outputs [26,38]—there is a notable absence of effective mechanisms for structured and systematic context sharing. Furthermore, the tight coupling between context management and computation impedes the reuse of intermediate representations such as key-value (KV) caches, causing redundant prefill computations upon each agent invocation.

A fine-grained agent context sharing framework capable of identifying and reusing shared prompt chunks would substantially reduce redundant computation, accelerate token generation. While preliminary efforts have investigated cache reuse across different requests and agents, their designs lack extensibility and flexibility to handle the dynamic and interdependent contexts typical in MAS.

3.2 Underutilization of KV Cache Reuse

Requests in MAS add system prompts [7], tool results, and communication between agents [40], amplifying the high token consumption of LLMs. Therefore, prefix caching (discussed in Sect. 2.3) is used to reduce the redundant prefill consumptions. However, as can be seen from Fig. 7, the shared prompt chunks do not always appear in the form of prefixes, limiting the gains could obtain in this setting. Exploring the application of position-independent caching to MAS might be a crucial step in improving system performance.

3.3 Insufficient Multi-agent Scheduling

As MAS scales to handle hundreds or thousands of agents and the capacity of KV vectors becomes extremely large, efficient scheduling becomes crucial to ensure high throughput. However, current scheduling strategies lose the opportunity to batch agents with overlapping prompt chunks and to improve token hit ratio [23, 41], especially in a distributed employment. A cache-aware scheduling policy can exploit context overlap among agents to improve system-wide resource efficiency and reduce latency.

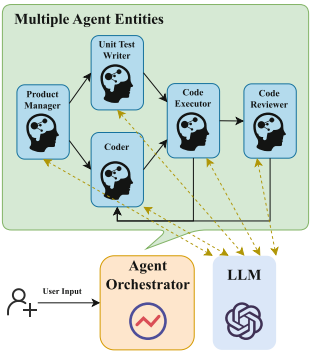


Fig. 6. The system orchestrates the execution of agents based on the specific task.

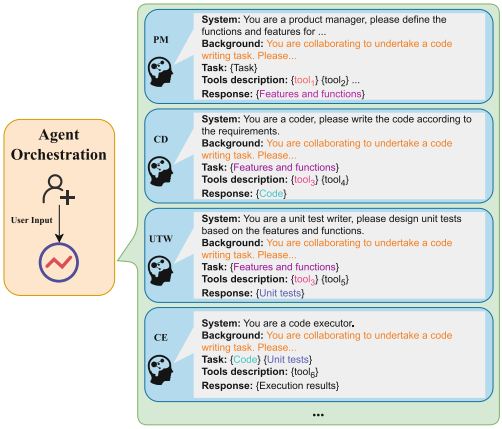


Fig. 7. A couple of agents collaborate to accomplish the task. Prompt chunks of the same color represent the communication and repeated tokens between agents.

4 System Design

To address the challenges identified in multi-agent serving systems, we propose *EACC*, a system optimized for efficient inference in MAS. EACC decouples context management from computation, employs structured caching, and introduces position-independent prompt chunk caching with efficient indexing mechanisms. This design maximizes reuse of context and KV caches across requests and agents. Furthermore, EACC supports cache-aware routing and efficient scheduling in distributed deployments. Notably, EACC is orthogonal to any specific PIC mechanism, allowing seamless integration with methods such as CacheBlend or EPIC. The overall architecture is illustrated in Fig. 8, which organizes the workflow into four core components: *Agent Context Cache (ACC)*, *Chunk KV Store*, *Prompt Chunks Handler*, and *Inference Instance Router*.

4.1 Agent Context Cache

The *Agent Context Cache (ACC)* serves as a global context index accessible by all inference instances in the system. Each user input is segmented into chunk-level units, termed *prompt chunks*. ACC maintains mappings from these prompt chunks to their corresponding KV caches. Each chunk serves as a unit of cache reuse and is loaded into the inference engine during prefill. Each entry in ACC contains the two main attributes:

- **Chunk Hash:** A SHA256-based hash serves as a unique identifier, enabling efficient retrieval.
- **KV Cache Mapping Information:** Metadata for fast cache lookup, including instance identifiers where the KV cache resides and their memory

addresses. Replication of the same chunk's KV cache across multiple instances is supported.

Crucially, ACC's KV mappings are position-independent; cache reuse is based on chunk-level KV caches indexed by hash rather than strict prefix matching. Thus, prompt chunks can be reused regardless of their positions within requests, overcoming limitations of traditional prefix caching. This broader coverage better supports dynamic and interdependent inference typical of MAS, effectively reducing TTFT.

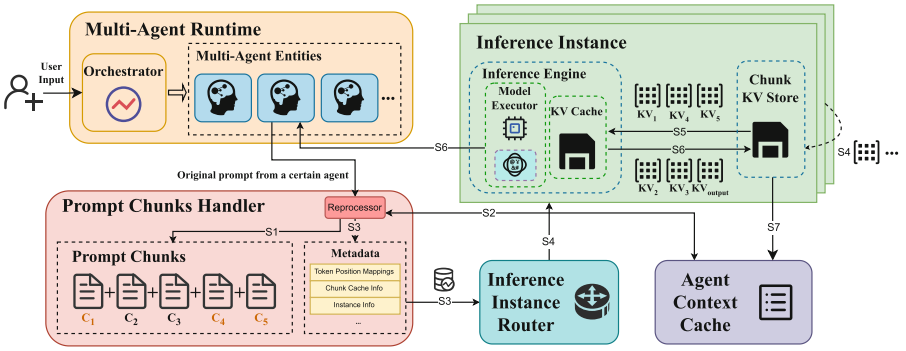


Fig. 8. The system design of EACC. The numerical sequence represents the workflow of a certain task.

4.2 Chunk KV Store

The *Chunk KV Store* caches KV pairs of individual prompt chunks separately.

Management and Chunk-Level Access on Cache. Following vLLM's approach [21], the Chunk KV Store organizes KV caches in a block-based manner. The block size is configurable, with a default of 16 tokens. The cache manager exposes CRUD interfaces, and employs a Least Recently Used (LRU) eviction policy to balance simplicity and effectiveness.

Aligned with ACC's chunk-level indexing, the Chunk KV Store treats each chunk's KV cache as an independent access unit, enabling chunk-level loading and writing. Upon generation of a new chunk's KV cache, the store allocates new blocks, writes KV data, and maintains slot mappings, supporting subsequent retrieval for prefill acceleration.

Global Distributed Caching. The Chunk KV Store is deployed across the cluster, leveraging local high-bandwidth memory (HBM) resources per node and supporting cache transmission. Each inference instance hosts a Chunk KV Store

separately, and supports flexible configuration of cache capacity based on node resources. Due to write-once-read-many semantics and capacity-based eviction, multiple replicas of the same chunk's KV cache may exist across instances without requiring consistency protocols.

4.3 Prompt Chunks Handler

The *Prompt Chunks Handler* is responsible for prompt processing and cache reuse management.

Semantic Chunking. Upon receiving a request, the handler segments the input prompt P into semantically meaningful chunks $C = \{C_1, C_2, \ldots, C_n\}$. Each chunk is the minimal unit for KV cache reuse and uniquely identified by its hash value $H(C_i)$, where $H(\cdot)$ denotes the hashing function.

Preprocessing of Prompt Chunks. Prompt Chunks Handler iterates through all the chunks, encodes each chunk C_i into token sequence T_i, and records the starting and ending positions of T_i, which will be used later for concatenating its KV. Meanwhile, the Prompt Chunks Handler also accesses ACC to check whether the corresponding KV of C_i is already cached. If it is found, it results in a *cache hit* (the corresponding KV is denoted as KV_i^h) and records the instances INS^h where the KV is saved, meaning the chunk can be reused without recomputation. Otherwise, it results in a *cache miss* (the corresponding KV to be computed is denoted as KV_i^m).

KV Transmission Control. Preprocessed metadata is forwarded to the *Inference Instance Router* (see Sect. 4.4) for routing and KV transmission decisions. Once the inference instance is determined, and successful retrieval of the KV pairs for the cached chunks, Prompt Chunks Handler loads cached KV pairs $KV^h = \{KV_i^h \mid if\ C_i\ triggers\ a\ cache\ hit\}$ into the inference engine in preparation for the incremental prefill phase. Specifically, only uncached $KV^m = \{KV_i^m \mid if\ C_i\ triggers\ a\ cache\ miss\}$ are to computed during this prefill. Beyond that, a few tokens of cached KV^h are recomputed to restore the cross-attention between prompt chunks.

4.4 Inference Instance Router

The *Inference Instance Router* schedules inference tasks by routing them to appropriate execution instances, optimizing throughput through balancing token hit ratios and instance loads. Algorithm 1 details the scheduling logic.

Given a set of prompt chunks $\{C_i \mid i = 1, 2, \ldots, n\}$, the router queries the cache status in metadata and computes the cached token counts per instance

Algorithm 1. Inference Routing and KV Migration Algorithm

Input: prompt chunks $prompt_chunks$, metadata $metadata$, transmission bandwidth b, model dimension d, all the inference instances in the system INS,
Output: instance that will be assigned with inference task $selected_ins$, chunks that need to be transmitted to this instance $transfer_chunk_list$

1: **for** ins_id in INS **do** *// Initialize token hit number for each instance*
2: $token_hit_num[ins_id] \leftarrow 0$
3: **end for**
4: $transfer_chunk_list \leftarrow [\,]$ *// Initialize list of chunks to be transferred*
5: **for** $prompt_chunk$ in $prompt_chunks$ **do**
6: $ins_ids, token_num \leftarrow metadata.get(prompt_chunk)$
7: **for** ins_id in ins_ids **do** *// Accumulate token hits for instances having this chunk*
8: $token_hit_num[ins_id] \leftarrow token_hit_num[ins_id] + token_num$
9: **end for**
10: **end for**
11: $optimal_ins_ids \leftarrow max(token_hit_num).key$ *// Find instance(s) with maximum token hits*
12: $selected_ins \leftarrow route_to_lightest_load_ins(optimal_ins_ids)$
13: **for** $prompt_chunk$ in $prompt_chunks$ **do**
14: $ins_ids, token_num \leftarrow metadata.get(prompt_chunk)$
15: $T_{prefill} \leftarrow T(token_num, 0)$
16: $T_{transfer} \leftarrow cache_size(token_num, d)/b$
17: **if** $selected_ins$ not in ins_ids and $T_{prefill} > T_{transfer}$ **then** *// Transfer only if faster than recomputation*
18: $transfer_chunk_list.append(prompt_chunk)$
19: **end if**
20: **end for**
21: **return** $selected_ins, transfer_chunk_list$ *// Return routing result and transfer plan*

(lines 5–10). For instance j, the total number of cached tokens will be caculated as

$$token_hit_num[j] = \sum_{i=1}^{n} token_num_i \tag{4}$$

where

$$token_num_i = \begin{cases} len(T_i) & , if\ KV_i\ cached\ on\ instance\ j \\ 0 & , \text{otherwise} \end{cases}$$

To maximize KV reuse, the router selects the instance with the highest $token_hit_num$. If multiple instances tie, it selects the least loaded to avoid bottlenecks (lines 11–12). For chunks missing on the selected instance, the router compares KV transmission time $T_{transfer}$ and prefill computation time $T_{prefill}$, fetching remote KV caches as appropriate to minimize overall latency (lines 13–20).

4.5 Workflow

Figure 8 illustrates the complete workflow. The *Prompt Chunks Handler* segments the input prompt into chunks $C = \{C_i \mid i = 1, 2, \ldots, 5\}$, with KVs of certain chunks (C_1, C_4, C_5) cached in the system. Each chunk is encoded, and metadata along with cache status for cached chunks $\{KV_i^h \mid i = 1, 4, 5\}$ is recorded. The aggregated metadata is passed to the *Inference Instance Router*, which selects the optimal inference instance. If some required KVs are absent locally, remote transmissions are triggered. During inference, all cached KVs $\{KV_i^h \mid i = 1, 4, 5\}$ are loaded into the inference engine. The engine computes $\{KV_i^m \mid i = 2, 3\}$ incrementally for uncached chunks C_2 and C_3, and selectively recomputes $\{KV_i^h \mid i = 1, 4, 5\}$ for cached chunks $\{C_i \mid i = 1, 4, 5\}$ to restore cross-attention across chunks. Decoding then proceeds to generate the final output, producing new KVs for the output chunk. Finally, the inference results are returned to the user, and an asynchronous process writes newly generated $\{KV_2, KV_3, KV_{output}\}$ back to *Chunk KV Store* for future reuse. Since the write operation and result return are executed in parallel, it will not affect the end-to-end latency. *ACC* is then updated accordingly to maintain mappings between the chunks and their cached KVs.

5 Implementation

We implement a prototype of EACC based on vLLM v0.4.1 [9], an efficient inference framework for large language models. vLLM leverages advanced memory management techniques such as PagedAttention [21] to reduce memory footprint and improve throughput. However, its built-in prefix caching mechanism restricts the reuse of prompt chunks, limiting efficiency gains in multi-agent scenarios. To overcome this, EACC replaces traditional prefix caching with a PIC foundation. Additionally, we implement inter-instance communication for KV cache transmission and provide concise APIs for inference, status querying. The distributed deployment can be seen in Fig. 9.

5.1 Selective Recomputation

Our work respectively integrate two types of PIC mechanisms into EACC, evaluate them, and verify the effectiveness of EACC[3]. Our implementation modifies the attention module's forward pass within vLLM to support selective token recomputation. For uncached chunks, all token KVs are computed fully. For cached chunks, only a subset of tokens undergo KV recomputation. Specifically, CacheBlend selects a proportion of tokens with high KV deviation (termed *HKVD tokens*) for recomputation, whereas EPIC recomputes a fixed number of initial tokens per cached chunk.

[3] Since the application of Prompt Cache [14] makes it unacceptable for the inference accuracy in MAS, we integrage CacheBlend [44] and EPIC [17] into our system, which we refer to as $EACC^{CB}$ and $EACC^{E}$.

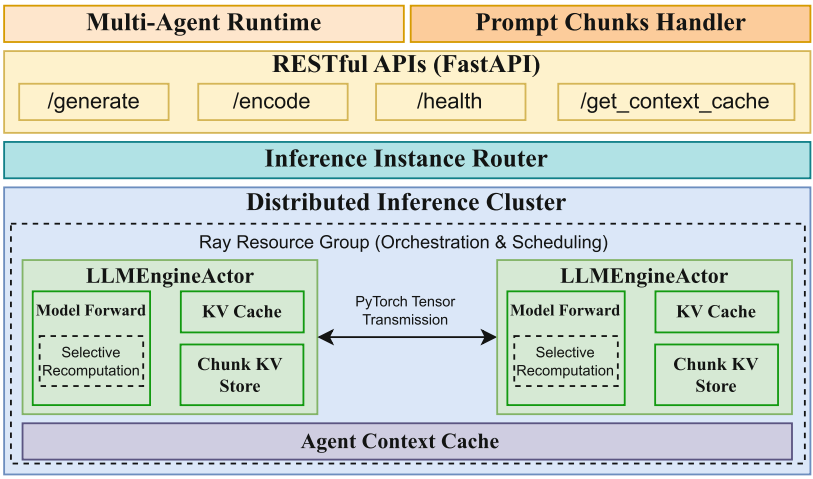


Fig. 9. The distributed inference cluster managed by Ray hosts multiple LLMEngine-Actors, each with local Chunk KV caches and model weights. Global Agent Context Cache coordinates cache reuse across instances. FastAPI provides online inference APIs supporting streaming and asynchronous responses.

5.2 Distributed Deployment

EACC leverages the *Ray* framework [10] to build a scalable inference cluster supporting distributed deployment and parallel task scheduling. Ray provides robust task orchestration, actor lifecycle management, and resource scheduling, enabling elastic scaling and efficient load balancing.

Each LLM inference instance is encapsulated as an *LLMEngineActor* bound to a dedicated GPU, hosting its own Chunk KV Store and model weights. KV cache transmission across nodes is implemented via *PyTorch* [13] tensor communication primitives. To coordinate global context caching and scheduling, an independent actor manages the global ACC, accessible by all inference instances.

5.3 RESTful APIs with FastAPI

We build RESTful APIs using the *FastAPI* framework [35], enabling end-to-end online inference services capable of handling high concurrency and streaming responses typical in agent-based workflows.

The primary API interfaces are summarized in Table 1. Clients invoke *generate.remote* to trigger online inference and receive asynchronous token streams via an *AsyncGenerator*. For streaming requests, EACC returns partial outputs in real time using *StreamingResponse*; for non-streaming requests, the complete final result is returned once inference concludes.

Table 1. APIs for online inference.

Endpoint Path	Method	Description
/generate	POST	Accept inference requests and return generated results
/encode	POST	Encode the input prompt into a token sequence
/get_context_cache	POST	Query the status of the global context cache
/health	GET	Instance health check endpoint

6 Evaluation

As discussed earlier, EACC accelerates the prefill stage by partitioning prompts into reusable chunks stored as KV pairs. In this section, we present a comprehensive set of experiments to validate the effectiveness of EACC. We aim to answer the following questions: (1) How do prompt segmentation strategies and segmentation granularity reduce redundant computation for repeated chunks? (2) Can the PIC mechanism effectively restore cross-attention and preserve inference quality? (3) In a real multi-agent online inference environment, can the inference routing algorithm maximize the KV reuse ratio and thereby improve throughput?

6.1 Setup

Testbed and Models. EACC is deployed on a two-node cluster. Each node is equipped with two NVIDIA GeForce RTX 3090 GPUs and an Intel Xeon 4216 CPU, with one inference instance per node. The LLM cache engine and Chunk KV Store are deployed on separate GPUs within the same node. Nodes communicate via InfiniBand (IB) [29] network interfaces with 100 Gbps bandwidth, enabling efficient cross-node KV cache transmission. For simplicity, experiments on prompt chunking are conducted on a single node, except for those in Sect. 6.5. We evaluate our approach using two open-source LLMs: Mistral-7B-Instruct-v0.2 [39] and Meta-Llama-3-8B-Instruct [2].

Baselines. To thoroughly assess EACC, we integrate it with two PIC mechanisms: CacheBlend [44] and EPIC [17], denoted as $EACC^{CB}$ and $EACC^{E}$, respectively. For both methods, we vary the parameters controlling selective recomputation. In $EACC^{CB}$-r, r denotes the recomputation ratio of cached tokens. In $EACC^{E}$-k, k denotes the number of tokens recomputed in each cached chunk. As EACC is implemented atop vLLM [9], which natively supports prefix caching, we compare against both standard vLLM and vLLM with prefix caching enabled.

Datasets. We evaluate EACC on four datasets: *Agent-conversations-retrieval-tool* [6], *Xlam-function-calling-60k* [36], *Squad (Stanford Question Answering*

Dataset) [34] and *2WikiMQA* [15]. *Agent-conversations-retrieval-tool* simulates dialogues between multiple agents in real-world applications. *Xlam-function-calling-60k* contains a large number of function call records, mainly used to test an agent's inference capabilities in tool invocation tasks. The prompts in *Squad* contain historical memory information, simulating the need for LLMs to enhance their abilities via historical memory in agent applications. *2WikiMQA* is a knowledge question-answering dataset covering various topics from Wikipedia which can be divided into reusable chunk units.

Metrics. We adopt the following metrics to quantify EACC's benefits:

- **Time-To-First-Token (TTFT)** [21,33,47]**:** The elapsed time from receiving a request to generating the first output token.
- **Token Hit Ratio (see Sec. 2.4):** Higher values indicate reduced computational cost in the prefill stage.
- **F1 Score** [4]**:** Measures the model's answer accuracy.
- **End-to-End (E2E) Latency** [5,37]**:** The total time from request submission to receipt of the complete response in an online inference service.

Since EACC only modifies the system-level cache reuse and prefill logic, without changing the decoding computation, the per-request metrics TBT [33] and TPOT [1,47] should remain unaffected. Therefore, these metrics are excluded from the following experiments (Fig. 11).

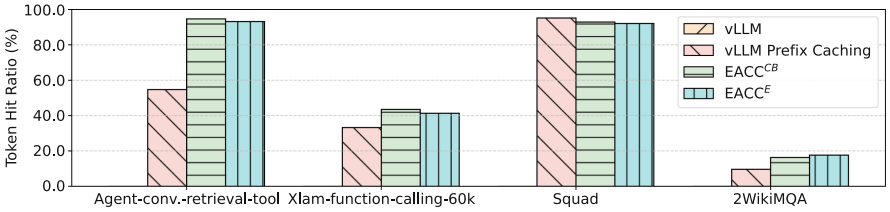


Fig. 10. Token hit ratios in different datasets.

6.2 Effectiveness of Reducing Redundant Computation

In this experiment, we emulate multi-agent workflows on the datasets described above to evaluate whether EACC can effectively reduce redundant computation and lower prefill latency. For *Agent-conversations-retrieval-tool*, *Xlam-function-calling-60k*, and *Squad*, we preselected 200 samples for inference to warm up the *Chunk KV Store*, thereby simulating a state in which the cache becomes saturated after a period of operation. For *2WikiMQA*, 160 samples were used for this warm-up phase. Following this, 20 requests were randomly drawn from the remaining data to assess the performance gains from cache reuse.

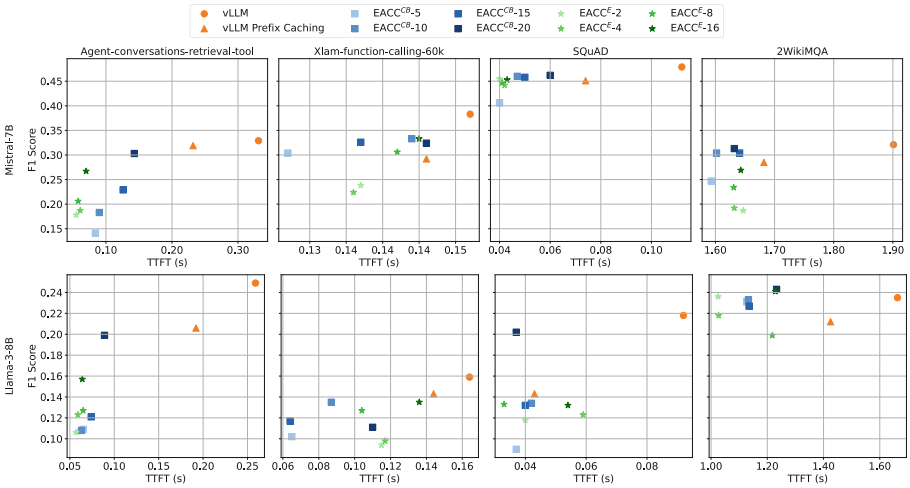


Fig. 11. EACC significantly reduces TTFT while maintaining accuracy.

Results. The results demonstrate that EACC significantly improves both TTFT and token hit ratio. As illustrated in Fig. 10, vLLM exhibits a token hit ratio of zero, as it recomputes all tokens from scratch. In contrast, EACC achieves the highest token hit ratio, reaching up to 94.7%, owing to its chunk-level caching mechanism. As discussed in Sec.2.4, TTFT exhibits a strong negative correlation with token hit ratio; thus, EACC's high hit ratio translates into optimal TTFT, delivering up to 82.4% improvement.

Moreover, the observed fluctuation in F1 score is generally within 30%, indicating that EACC can balance accuracy and latency effectively. Notably, setting $r = 15$ for EACC^{CB} and $k = 8$ for EACC^{E} yields a favorable trade-off between inference quality and latency reduction.

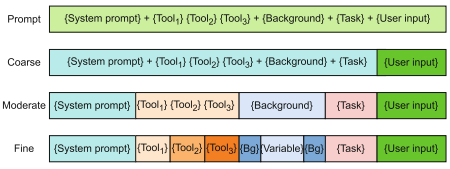


Fig. 12. Examples of three different granularities of prompt chunks.

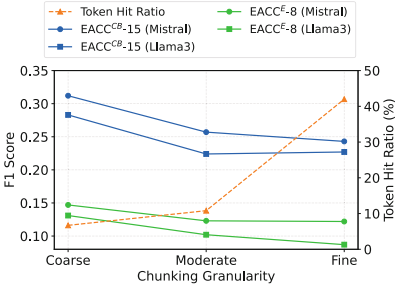


Fig. 13. Impact of chunking granularity on F1 score and token hit ratio.

6.3 Cache Granularity Analysis

This experiment investigates how the semantic granularity of chunking influences KV reuse and prefill efficiency in EACC. While maintaining the core principle of semantic segmentation, we adjust chunking granularity, i.e., the fineness of semantic divisions, by employing three strategies: (1) *Coarse*: splitting only at major semantic boundaries (e.g., between system prompt and user prompt); (2) *Moderate*: further splitting system messages according to functional categories; and (3) *Fine*: partitioning at the smallest variable unit within a sentence (e.g., variables or timestamps). An example is presented in Fig. 12. Using the *Xlam-function-calling-60k* dataset, we fix the total context length per request while varying chunk granularity to assess its effect on token hit ratio and accuracy. The results are illustrated in Fig. 13.

As chunk size increases, a greater number of identical tokens is required for a match, resulting in a reduced token hit ratio. Even small changes within a chunk can degrade its reusability, reducing EACC's caching advantage. However, smaller chunks have stronger inter-chunk semantic dependencies. If only a limited subset of tokens is selectively recomputed, cross-chunk attention restoration may be incomplete, thereby impacting inference quality. For EACC^E, the first k tokens in each chunk are recomputed regardless of chunk length. As the number of chunks increases, the total recomputed tokens grow, and when chunk size becomes excessively small, we can speculate that TTFT may increase rather than decrease.

In theory, the intersection point between hit ratio and accuracy curves represents the optimal balance between computational gains from cache reuse and inference quality. The choice of semantic chunking strategy constitutes an integral part of the system's optimization space, warranting further exploration in future work.

6.4 Position Independence Verification

We further examine the position independence property of EACC by fixing the input prompt and chunking strategy while altering the source positions of the prompt chunks. In this setting, a chunk appears at different positions within the original prompt across trials, thereby carrying different positional encodings. This enables evaluation of the impact of positional variation on the PIC mechanism.

As shown in Fig. 14, altering chunk positions has negligible impact on both TTFT and F1 score. The total number of tokens requiring computation remains constant, resulting in stable TTFT. Moreover, EACC consistently restores cross-attention information through selective recomputation, preserving inference accuracy. These results confirm that EACC is agnostic to the positional arrangement of chunks in the input, thereby enhancing cache reuse flexibility and system robustness.

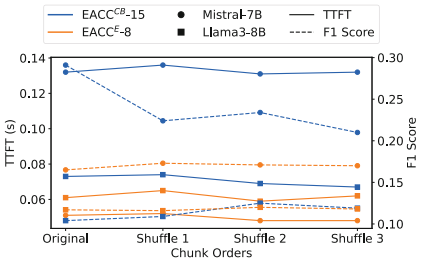


Fig. 14. F1 score and TTFT under different chunk orders.

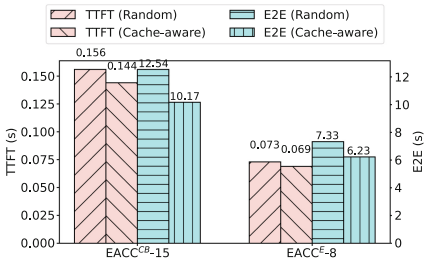


Fig. 15. Comparison of different request allocation strategies.

6.5 Distributed Cache Load Balancing and Transmission Efficiency

To evaluate the impact of KV cache transmission and reuse in a distributed environment—particularly within multi-node inference clusters—and to investigate how load balancing and inter-node cache transmission affect system performance, we conduct experiments using the *Xlam-function-calling-60k* dataset and the Mistral-7B-Instruct-v0.2 model. We set the batch size to 4 and compare two request allocation strategies: a *random allocation strategy* and the *cache-aware allocation strategy* described in Algorithm 1.

Results. The TTFT and E2E latency results for the two strategies are shown in Fig. 15. Under the cache-aware allocation strategy, although KV data is transmitted across nodes, the cross-node KV transmission introduces negligible additional overhead to TTFT. On average, cache transmission accounts for only 13.1% of the total TTFT. Consequently, TTFT under the cache-aware strategy still outperforms the random allocation baseline.

While the reduction in TTFT contributes only marginally to the overall improvement in E2E latency, the cache-aware strategy's consideration of load balancing leads to a decrease in end-to-end latency. This advantage is expected to become more pronounced as the number of nodes increases. The high efficiency of cross-node KV transmission ensures that even in distributed environments, EACC's performance is optimized, effectively supporting inference tasks in large-scale MAS.

7 Future Work

Our prototype, developed on vLLM v0.4.1 [9], currently lacks support for CPU-based model weight offloading, which constrains our experimentation with large model architectures. Notably, state-of-the-art techniques such as CacheBlend [44] and EPIC [17] are also developed based on this version. Our primary focus, however, lies in the design and evaluation of cache algorithms and system architecture rather than the deployment of large models in this iteration of the research. Future work will involve migrating the EACC framework to the latest versions of

vLLM that incorporate model offloading capabilities. This migration will enable us to experiment with large-scale language models and further evaluate their scalability and effectiveness in realistic scenarios.

8 Conclusion

This paper presents EACC, a system-level framework designed to mitigate redundancy and inefficiency in multi-agent LLM serving. By decoupling context storage from computation and enabling position-independent KV cache reuse through semantic prompt chunking, EACC effectively reduces prefill latency and improves overall system throughput. The framework supports distributed deployment and efficient scheduling, and it integrates seamlessly with existing PIC techniques. Experimental evaluations on diverse datasets and models show that EACC achieves up to 94.7% token hit ratio, leading to as much as 82.4% reduction in TTFT and 18.9% reduction in end-to-end latency, while maintaining model accuracy. These results position EACC as a promising solution for optimizing large-scale multi-agent services, with broad applicability in conversational AI, and automated decision-making scenarios.

Acknowledgments. This work was supported by Huawei, Cloud Storage Innovation Laboratory and School of Computer Science at Shanghai JiaoTong Univeristy. We thank all the anonymous reviewers, for their insightful feedback and suggestions.

References

1. Agarwal, M., Qureshi, A., Sardana, N., Li, L., Quevedo, J., Khudia, D.: LLM inference performance engineering: Best practices. https://www.databricks.com/blog/llm-inference-performance-engineering-best-practices (October 2023). Accessed 30 July 2025
2. AI, M.: MetaâĂŚllamaâĂŚ3âĂŚ8bâĂŚinstruct. https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct (2024), released: April 18, 2024; Accessed3 Aug 2025
3. Ashish, V.: Attention is all you need. Advances in neural information processing systems **30**, I (2017)
4. Bai, Y., L et al.: LongBench: A bilingual, multitask benchmark for long context understanding. In: Ku, L.W., Martins, A., Srikumar, V. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 3119–3137. Association for Computational Linguistics, Bangkok, Thailand (Aug 2024). https://doi.org/10.18653/v1/2024.acl-long.172, https://aclanthology.org/2024.acl-long.172/
5. Banerjee, D., Singh, P., Avadhanam, A., Srivastava, S.: Benchmarking LLM powered chatbots: Methods and metrics (2023). https://arxiv.org/abs/2308.04624
6. Briggs, J.: the vllmâĂŚproject contributors: agentâĂŚconversationsâĂŚretrievalâĂŚtool dataset. https://huggingface.co/datasets/jamescalam/agent-conversations-retrieval-tool (2023), updated: August 27, 2023; Accessed 18 July 2025

7. Chen, W., et al.: Agentverse: Facilitating multi-agent collaboration and exploring emergent behaviors. In: The Twelfth International Conference on Learning Representations (2024). https://openreview.net/forum?id=EHg5GDnyq1
8. Cheng, Y., et al.: Exploring large language model based intelligent agents: Definitions, methods, and prospects (2024). https://arxiv.org/abs/2401.03428
9. vLLM Contributors: vLLM: Fast and easy LLM inference. https://github.com/vllm-project/vllm/tree/v0.4.1 (2025). Accessed 16 July 2025
10. Contributors, R.P.: Ray: A framework for scalable distributed computing. https://github.com/ray-project/ray (2025). Accessed 18 July 2025
11. Dong, X., Zhang, X., Bu, W., Zhang, D., Cao, F.: A survey of llm-based agents: Theories, technologies, applications and suggestions. In: 2024 3rd International Conference on Artificial Intelligence, Internet of Things and Cloud Computing Technology (AIoTC), pp. 407–413 (2024). https://doi.org/10.1109/AIoTC63215.2024.10748304
12. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: Forty-first International Conference on Machine Learning (2023)
13. Foundation, P.: Pytorch: an open source machine learning framework. https://pytorch.org (2025). Accessed 30 July 2025
14. Gim, I., Chen, G., Lee, S.S., Sarda, N., Khandelwal, A., Zhong, L.: Prompt cache: Modular attention reuse for low-latency inference. In: MLSys (2024). https://proceedings.mlsys.org/paper_files/paper/2024/hash/a66caa1703fe34705a4368c3014c1966-Abstract-Conference.html
15. Ho, X., Duong Nguyen, A.K., Sugawara, S., Aizawa, A.: Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In: Scott, D., Bel, N., Zong, C. (eds.) Proceedings of the 28th International Conference on Computational Linguistics, pp. 6609–6625. International Committee on Computational Linguistics, Barcelona, Spain (Online) (Dec 2020). https://doi.org/10.18653/v1/2020.coling-main.580, https://aclanthology.org/2020.coling-main.580/
16. Hu, C., et al.: Memserve: Context caching for disaggregated LLM serving with elastic memory pool. arXiv preprint arXiv:2406.17565 (2024)
17. Hu, J., et al.: EPIC: Efficient position-independent caching for serving large language models. In: Forty-second International Conference on Machine Learning (2025). https://openreview.net/forum?id=qjd3ZUiHRT
18. Hu, J., Wang, C., Huang, H., Luo, H., Jin, Y., Deng, Y., Xie, T.: Predicting compilation resources for adaptive build in an industrial setting. In: 2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 1808–1813 (2023). https://doi.org/10.1109/ASE56229.2023.00128
19. Ji, Z., Yu, T., Xu, Y., Lee, N., Ishii, E., Fung, P.: Towards mitigating LLM hallucination via self reflection. In: Bouamor, H., Pino, J., Bali, K. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2023, pp. 1827–1843. Association for Computational Linguistics, Singapore (Dec 2023). https://doi.org/10.18653/v1/2023.findings-emnlp.123, https://aclanthology.org/2023.findings-emnlp.123/
20. Ke, Z., Shao, Y., Lin, H., Konishi, T., Kim, G., Liu, B.: Continual pre-training of language models. In: The Eleventh International Conference on Learning Representations (2023). https://openreview.net/forum?id=m_GDIItaI3o
21. Kwon, W., et al.: Efficient memory management for large language model serving with pagedattention. In: Proceedings of the 29th Symposium on Operating Systems Principles, pp. 611–626 (2023)

22. LangChain: Langchain – the platform for reliable agents. https://www.langchain.com (2025). Accessed 16 July 2025
23. Li, G., Hammoud, H.A.A.K., Itani, H., Khizbullin, D., Ghanem, B.: CAMEL: Communicative agents for "mind" exploration of large language model society. In: Thirty-seventh Conference on Neural Information Processing Systems (2023). https://openreview.net/forum?id=3IyL2XWDkG
24. Li, X., Wang, S., Zeng, S., Wu, Y., Yang, Y.: A survey on LLM-based multi-agent systems: workflow, infrastructure, and challenges. Vicinagearth **1**(1), 9 (2024)
25. Li, X.: A review of prominent paradigms for LLM-based agents: Tool use (including rag), planning, and feedback learning (2024). https://arxiv.org/abs/2406.05804
26. Liu, Y., et al.: Droidspeak: Kv cache sharing for cross-LLM communication and multi-LLM serving. arXiv preprint arXiv:2411.02820 (2024)
27. Maharana, A., Lee, D.H., Tulyakov, S., Bansal, M., Barbieri, F., Fang, Y.: Evaluating very long-term conversational memory of LLM agents (2024). https://arxiv.org/abs/2402.17753
28. Maldonado, D., Cruz, E., Torres, J.A., Cruz, P.J., Benitez, S.d.P.G.: Multi-agent systems: a survey about its components, framework and workflow. IEEE Access **12**, 80950–80975 (2024)
29. NVIDIA: Connectx infiniband. https://www.nvidia.cn/networking/infiniband-adapters/ (2025). Accessed 18 July 2025
30. OpenAI: Function calling. https://platform.openai.com/docs/guides/function-calling?api-mode=responses (2024). Accessed 16 July 2025
31. Patel, P., Choukse, E., Zhang, C., Shah, A., Goiri, Í., Maleki, S., Bianchini, R.: Splitwise: Efficient generative LLM inference using phase splitting. In: 2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA), pp. 118–132. IEEE (2024)
32. Qasim, A., Ghouri, A., Munawar, A.: An effective approach for reducing data redundancy in multi-agent system communication. Multiagent Grid Syst. **20**(1), 69–88 (2024)
33. Qin, R., et al.: Mooncake: Trading more storage for less computation—a {KVCache-centric} architecture for serving {LLM} chatbot. In: 23rd USENIX Conference on File and Storage Technologies (FAST 25), pp. 155–170 (2025)
34. Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: Stanford question answering dataset (squad). https://huggingface.co/datasets/rajpurkar/squad (2016), based on original dataset (arXiv:1606.05250). Accessed 18 July 2025
35. Ramírez, S., the FastAPI community: Fastapi: high performance, easy to learn, fast to code, ready for production. https://fastapi.tiangolo.com (2025). Accessed 17 July 2025
36. Research, S.A., contributors: xlamâĂŚfunctionâĂŚcallingâĂŚ60k dataset. https://huggingface.co/datasets/Salesforce/xlam-function-calling-60k (2024). arXiv:2406.18518, released: July 2024; Accessed 18 July 2025
37. Tan, X., Jiang, Y., Yang, Y., Xu, H.: Towards end-to-end optimization of LLM-based applications with ayo. In: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 1302–1316. ASPLOS '25, Association for Computing Machinery, New York, NY, USA (2025). https://doi.org/10.1145/3676641.3716278
38. Tara, A., Turesson, H.K., Natea, N.: Dynamic storage optimization for communication between ai agents. Future Internet **16**(8), 274 (2024)
39. Team, T.M.A.: MistralâĂŚ7bâĂŚinstructâĂŚv0.2. https://huggingface.co/mistralai/Mistral-7B-Instruct-v0.2 Accessed 18 July 2025

40. Wang, Q., Tang, Z., JIANG, Z., Chen, N., Wang, T., He, B.: Agenttaxo: Dissecting and benchmarking token distribution of LLM multi-agent systems. In: ICLR 2025 Workshop on Foundation Models in the Wild (2025). https://openreview.net/forum?id=0iLbiYYIpC
41. Wu, Q., et al.: Autogen: Enabling next-gen LLM applications via multi-agent conversations. In: First Conference on Language Modeling (2024). https://openreview.net/forum?id=BAakY1hNKS
42. Xu, W., Mei, K., Gao, H., Tan, J., Liang, Z., Zhang, Y.: A-mem: Agentic memory for LLM agents (2025). https://arxiv.org/abs/2502.12110
43. Xue, F., Fu, Y., Zhou, W., Zheng, Z., You, Y.: To repeat or not to repeat: insights from scaling LLM under token-crisis. In: Thirty-seventh Conference on Neural Information Processing Systems (2023). https://openreview.net/forum?id=Af5GvIj3T5
44. Yao, J., et al.: Cacheblend: fast large language model serving for rag with cached knowledge fusion. In: Proceedings of the Twentieth European Conference on Computer Systems, pp. 94–109 (2025)
45. Zhao, W.X., et al.: A survey of large language models (2025). https://arxiv.org/abs/2303.18223
46. Zheng, L., et al.: Sglang: efficient execution of structured language model programs. Adv. Neural. Inf. Process. Syst. **37**, 62557–62583 (2024)
47. Zhong, Y., et al.: DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving. In: 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24), pp. 193–210. USENIX Association, Santa Clara, CA (Jul 2024). https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin

AccuGraph: Memory-Efficient Full-Graph GNN Training on a Single GPU via Subgraph Accumulation

Liyang Wu[1], Menghan Jia[1(✉)], Lizhi Zhang[1], Yahui Wu[1], Gongqingjian Jiang[1], Jiezhong He[1], Chunye Gong[1,2], and Yinghui Gao[3]

[1] College of Computer Science and Technology, National University of Defense Technology, Changsha, Hunan 410073, China
{wuly58,jiamenghan12,zhanglizhi15,wyh,jianghydro,jiezhonghe}@nudt.edu.cn
[2] National Supercomputer Center in Tianjin, Tianjin 300457, China
[3] Academy of Military Science, Beijing 100000, China

Abstract. Graph Neural Networks (GNNs) have become essential for graph-based applications. Full-graph GNN training is a commonly used approach because it enables more comprehensive aggregation of neighbor information. However, full-graph training requires loading the entire graphs into GPU memory, often exceeding GPU memory limits when training on large graphs. To address this, we propose accumulated subgraph training method, which decomposes a computationally expensive full-graph update into multiple smaller, accumulated subgraph updates, while preserving exact mathematical equivalence. However, a naive implementation of this approach incurs serious redundant CPU-GPU data transfers and GPU computations. To address this challenge, we propose AccuGraph, an improved accumulated subgraph training method that incorporates two complementary optimizations: (1) topological similarity based nodes pre-sorting strategy to allocate structurally similar nodes in to the same sub-batch, which increases intra-batch neighbor overlap and lessens duplicated neighbors across batches; and (2) hybrid redundancy caching and reuse technique, which prunes residual redundancy and reuses the cached embeddings of high-frequency nodes and cross-batch nodes, thus reducing repeated data transfers and computations. Experimental results demonstrate that optimized AccuGraph substantially reduces memory consumption while preserving training accuracy, achieving up to a 11.5× speedup in training time. Notably, it completes training on the 111M-node ogbn-papers100M dataset within 5 s using less than 15 GB of GPU memory, enabling large-scale GNN training on consumer-grade hardware.

Keywords: GNNs · Full-graph training · Memory efficiency

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 172–191, 2026.
https://doi.org/10.1007/978-981-95-8405-5_10

1 Introduction

Graph neural networks (GNNs) [1,17,29,30] have emerged as the dominant paradigm for learning from graph structure data: By aggregating information from neighbors across multiple layers, GNNs integrate structural patterns with node features and achieve strong performance on tasks such as link prediction [20], node and graph classification [18,25], and community detection [14].

Full-graph training is a widely adopted paradigm in GNN training. Compared with mini-batch training [21,26], which risks accuracy degradation due to biased gradient estimates and sampling-induced loss of topological information [5,10,28], full-graph training serves as a promising paradigm for its effectiveness brought by full-batch gradient descent and full-neighbor aggregation semantics. However, this approach requires loading the entire graph and features into GPU memory, which becomes infeasible for large graphs due to out-of-memory (OOM) errors.

To enable GNN training under constrained GPU memory, gradient accumulation [8] has emerged as an effective solution. This technique works by accumulating gradients across multiple micro-batches before executing parameter updates, thereby simulating the large-batch training while maintaining memory efficiency. Inspired by this, we propose accumulated subgraph training method, which decomposes a full-graph update into a sequence of accumulated subgraph computations. First, the training nodes are partitioned consecutively into sub-batches. A sub-batch denotes a subset of training nodes calculated together each forward propagation. Second, for each sub-batch we construct its batch subgraph by expanding the target nodes' L-hop neighborhood. Third, generate the sub-batch node embeddings via forward propagation and calculate gradients of each batch via backward propagation. Forth, gradients from each subgraph are accumulated, and a single parameter update is applied after all subgraphs have been processed. By applying global degree normalization, which uses a consistent normalization factor across batches, this scheme preserves the mathematical equivalence to full-graph gradient computation under standard assumptions.

However, a naive implementation of the approach suffers from substantial redundant CPU–GPU data transfers and repeated GPU computations. These redundancies arise because of the extensive neighbor overlap among sub-batches, a consequence of the inherent structural dependencies among nodes in graph data. To mitigate this challenge, we extend the accumulated subgraph training approach by incorporating two complementary optimizations, which together define AccuGraph. First, we propose to execute topological similarity based nodes pre-sorting ($TSPS$) before consecutive partitioning, which implicitly allocates structurally similar nodes to the same sub-batch. By increasing intra-batch neighbor overlap, $TSPS$ raises batch density and reduces overlapped structures between batch subgraphs. Second, to deal with the remained uneliminated redundancy, we design a hybrid redundancy caching and reuse technique (HCR), in which overlapped structures are pruned and cached intermediate embeddings of redundant nodes are reuse in the pruned subgraphs, thus alleviate repeated data transfers and computations. Specifically, we adopt hybrid caching strat-

egy to manage the embedding of nodes with different characteristic of repeated occurrence for better memory utilization.

By executing two typical GNNs on four representative datasets, we show that AccuGraph can achieve full-graph training under constrained memory budgets, while maintaining the same-level accuracy. With optimization, AccuGraph achieves up to 11.5× speedup in per-epoch training time over the vanilla baseline. Experiment results demonstrate that AccuGraph can complete a single epoch training pass on the ogbn-papers100M(with 111 million nodes) within 5 s using less that 15 GB memory, enabling large-scale graph processing on consumer hardware.

In summary, we make the following contributions.

- To the best of our knowledge, we are the first to propose accumulated subgraph training method to achieve full-graph training on large graphs with tight GPU memory constraint.
- We further analyze the redundancy challenge introduced by the naive accumulated subgraph training approach. To mitigate it, we propose the $TSPS$ strategy to reduce overlapped structures between batch subgraphs, and introduce the HCR technique to diminish the residual overlaps.
- Our implementation and evaluation show that AccuGraph enables full-graph training on large graphs on single GPU. By alleviating the redundancy problem, AccuGraph significantly improves both training speed and memory utilization compared to the vanilla baseline.

2 Background and Motivation

2.1 Graph Neural Networks

Graph Neural Networks (GNNs) are a class of deep learning models designed for graph-structured data. They operate via message passing, where structural information and node/edge features propagate through the network to generate context-aware node representations.

A typical GNN stacks multiple layers, with the number of layers L defining the network depth and the receptive field size. Specifically, an L-layer GNN enables each node to aggregate features from its L-hop neighbors through successive nonlinear transformations. Given a graph $G(V, E)$, where V and E are the sets of nodes and edges respectively, each node $v \in V$ has a raw feature vector x_v. The computation at the l^{th} layer is formulated as:

$$h_v^l = \phi^l(h_v^{l-1}, \psi^l(h_u^{l-1}, \forall u \in N(v), W^{l-1}) \tag{1}$$

where h_v^l is the feature of node v at layer l, $N(v)$ denotes the set of neighbors of v, ψ^l aggregates neighbor features, the aggregated information will be computed using parameters W^{l-1}, and ϕ^l applies nonlinear transformations to the outcome. The initial node features correspond to $h_v^0 = x_v$. Different GNN models mainly differ in their message aggregation, activation functions, and ways of handling graph topology.

2.2 Existing Full-Graph Training Paradigms

Aligned with fundamental objectives of deep learning models, the core goal in training GNNs is to learn a mapping that captures dependencies between node features and corresponding labels, enabling accurate predictions for unseen nodes.

Two common methods for training GNNs are mini-batch training and full-graph training. *Mini-batch training* methods [21,26] compute gradients on a subset of nodes at each iteration to stochastically approximate the full gradient, and each training epoch consists of multiple iterations. It often combine with various neighbor sampling techniques [3–5,28] to alleviate the memory bottleneck caused by neighbor explosion. While reducing computation cost, they introduce the risk of biased gradient estimates and may discard edges that capture important topological information [27], potentially degrading the learned representations [10].

By contrast, *full-graph training* serves as a promising GNN training method for its effectiveness brought by full-neighbor aggregation semantics and full-batch gradient descent [10,15,24]. In each epoch, full-graph methods perform forward and backward propagation over the entire graph, computing node representations based on all nodes and edges. The gradients are then calculated using the loss on the whole training set to update the model parameters. The integrity of information aggregation and the stability of convergence enable full-graph training to achieve superior predictive accuracy [7,9,10]. It is equivalent to training on a batch that contains all nodes in training set, hence the term *full-batch training*.

Since memory usage scales with graph size, full-graph training becomes impractical for large graphs under limited GPU memory. This poses significant challenges as real-world graphs continue to grow in size. To cope with this scalability problem, distributed training systems [2,15,16,23] have been developed for partitioning and parallelizing computations across multiple GPUs or machines. However, distributed system demands substantial hardware resources, setting a significant barrier for researchers without access to multiple GPUs or large-scale computing infrastructure.

Consequently, we are motivated to scale full-graph training on large-scale graphs on consumer hardware.

3 A Naive Accumulated Subgraph Training Method

3.1 Definition

Gradient accumulation strategy simulates larger batch training by accumulating gradients from multiple batches before performing parameters update. Inspired by the technique, we propose a method that scales stable full-graph training to large graphs under limited GPU memory without accuracy compromise.

In what follows, we formalize this idea as accumulated subgraph training with global degree normalization, which is referred as accumulated subgraph training in the remainder of this paper for notational convenience.

Here, we introduce the concept of a sub-batch, defined as a subset of training nodes that are calculated together in a single forward propagation. Unlike a conventional mini-batch in SGD, which is typically sampled i.i.d. to yield an unbiased gradient estimate, a sub-batch can be formed by simple consecutive partitioning for convenience without degrading model performance.

The procedure of accumulated subgraph training is as follows:

1. Partition the training set $\mathcal{B}_{full}$ into K sub-batches $\{\mathcal{B}_1, \ldots, \mathcal{B}_K\}$.
   To correctly perform message passing and capture all relevant neighborhood information for nodes in $\mathcal{B}_i$, it is necessary to include their multi-hop neighbors in a localized subgraph rather than considering these nodes in isolation.
2. Thus, for each $\mathcal{B}_i$, construct batch subgraph $\mathcal{S}_i$ by the following steps.
   (a) First, expand all immediate neighbors of the sub-batch nodes.
   (b) Then, iteratively expand the newly reached nodes for $L-1$ additional steps to cover all L-hop neighbors required by an L-layer GNN.
   (c) The induced subgraph, containing both target nodes and all their expanded neighbors, is then indexed and loaded with raw features.
   Figure 2 presents schematic illustrations of batch subgraphs.
3. For each batch subgraph $\mathcal{S}_k$:
   (a) Perform a forward pass to compute $\ell_k(\theta)$, which is the loss of $\mathcal{B}_k$. For GNN models with normalization layers, apply *global degree normalization* by using each node's degree in the full graph (rather than its batch-local degree) when normalizing messages.
   (b) Perform a backward pass to obtain its gradient contribution $g_k = \nabla_\theta \ell_k(\theta)$, accumulating $G = \sum_{k=1}^{K} g_k$.
4. After processing all K batches (i.e., at the end of the epoch), apply the update: $\theta \leftarrow \theta - \eta G$, where η is the learning rate.

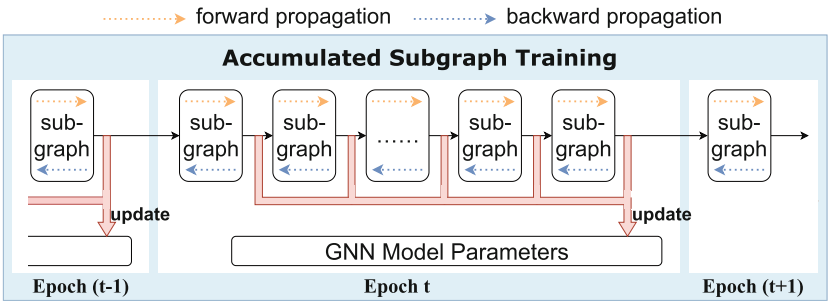


Fig. 1. Workflow of naive accumulated subgraph training.

Figure 1 depicts the process of 3) and 4). Mathematically, this procedure exactly recovers the full-batch gradient descent update. The use of global degree normalization ensures that normalization factors match those in a true full-graph pass, thus preserving the equivalence in both optimization trajectory and final solution.

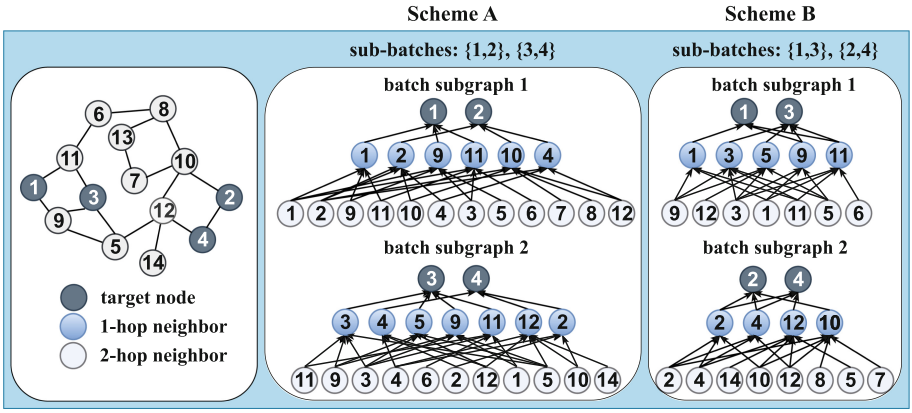


Fig. 2. Comparison of two sub-batch partitioning schemes and their effect on batch-subgraph size and inter-batch redundancy.

3.2 Challenges

Though accumulated subgraph training enables true full-graph gradient computation under strict memory budgets, the intrinsic high overlap in the graph's topology does induce critical bottlenecks. With the phenomenon of neighbor explosion, batch subgraphs usually severely overlap across sub-batches, which cause redundant CPU to GPU transfers and GPU computation.

On one hand, the expansion to multi-hop neighborhoods significantly amplifies structural redundancy. As the number of hops increases, the size of each subgraph grows rapidly (see Fig. 2), causing many nodes and edges to appear repeatedly across multiple batch subgraphs. We also test on the igb-small [11] and find that, setting batch size to 8192, the constructed batch subgraphs on average contains about 58860 nodes when $L = 1$ and 399823 nodes when $L = 2$. This exponential growth not only increases memory consumption but also leads to duplicated computations during training. On the other hand, as illustrated in Fig. 2 (Scheme A vs. Scheme B), consecutive partitioning of the original training nodes tends to scatter topologically related nodes across different sub-batches. When the partitioning ignores the graph's inherent community or cluster structure (Scheme A), nodes that share many neighbors are split across batches. The resulting batch subgraphs exhibit little intra-batch neighbor overlap but substantial inter-batch overlap, which inflates the overall size of subgraphs. In the example, nodes $2, 4, 9, 11$ recur in two sub-batches under Scheme A and therefore incur repeated host–device transfers and redundant recomputation of their intermediate embeddings. By contrast, grouping nodes with high topological similarity into the same sub-batch (Scheme B) concentrates neighbors locally and increases intra-batch overlap while reducing overlap across batches. Thus, reordering the training set substantially mitigates redundancy.

To address the redundancy challenge, we propose a topological similarity based nodes pre-sorting strategy that promotes intra-batch neighborhood cohe-

sion, effectively constraining subgraph size inflation and alleviating redundancy. For the intrinsic redundancy that cannot be eliminated by partitioning alone, we further introduce a hybrid redundancy caching and reuse technique to significantly reduce redundant data transfers and duplicated computations through pre-computed intermediate embedding caching and reuse. We incorporate the above design techniques into AccuGraph, an GNN training framework that enables full-graph training on large graphs with limited GPU memory, and the details of our design are presented in Sect. 4.

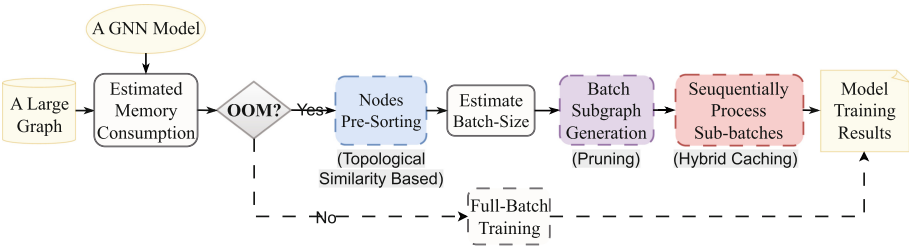


Fig. 3. Workflow of AccuGraph framework.

4 The AccuGraph Framework

4.1 Overview

In this section, we describe the workflow of our system and highlight the components where we introduce optimizations.

The overall workflow is depicted in Fig. 3. Given a dataset and model, AccuGraph first treats the entire training set as a single full batch and employs the Betty [26] memory-estimation procedure to predict whether a full-batch update will trigger OOM errors. Specifically, we estimate the memory consumption base on model parameters and input & output sizes, allowing feasibility to be assessed without executing on the GPU. AccuGraph executes full-batch training whenever the estimated memory consumption falls within the device's available capacity. Full-batch updates deliver superior performance provided sufficient GPU memory is available. Otherwise, the framework falls back to accumulated subgraph training.

Because the order of training nodes affects the result of consecutive partitioning, AccuGraph first applies a topological similarity based pre-sorting strategy to the training nodes, which is one of the main innovations of this paper. On the sorted node sequence, AccuGraph then determines a suitable batch size by reusing the memory estimates mentioned above in a binary-search procedure: Starting from the largest power-of-two not exceeding the number of training nodes and halving as needed, the search finds the maximal usable batch size (always powers of two) that satisfies the memory constraint. Once the batch size

is fixed, sub-batches and their corresponding batch subgraphs are pre-generated and reused across all epochs, eliminating per-epoch construction overhead. During training, these pre-generated batch subgraphs are processed sequentially and optimized using the proposed hybrid redundancy caching and reuse technique, which combines a static cache for frequently recurring embeddings with a dynamic cache for locally recurring nodes. Section 4.2 and 4.3 provide detailed descriptions of our two optimizing techniques.

4.2 Topological Similarity Based Nodes Pre-Sorting Strategy

Since the node ordering directly determines which nodes are placed in the same batch, our objective is to improve the internal density of each batch, i.e., the structural similarity among its nodes, by reordering the training set prior to partitioning. Although structural similarity can also be measured using higher-order neighbors (e.g., two-hop or beyond), the number of common first-order neighbors provides a more direct and tractable indicator of topological relatedness. Accordingly, we adopt the Common Neighbors (CN) Metric to measure pairwise node similarity:

$$\mathrm{CN}(x, y) \;=\; |N(x) \cap N(y)| \;=\; \sum_{u \in N(x) \cap N(y)} 1. \tag{2}$$

For two nodes $x, y \in V$, a larger $\mathrm{CN}(x, y)$ reflects stronger structural similarity.

However, directly computing the CN values for all node pairs and subsequently sorting the entire set by similarity is computationally prohibitive, and in fact NP-hard for large graphs. To overcome this, we design a heuristic pre-sorting algorithm that leverages local graph structural properties to approximate similarity-based ordering while remaining scalable.

The method is motivated by the following observation: Within the subgraph induced by the out-neighbors of a given set S, the node v with the maximal in-degree tends to have substantial overlap in predecessors with S. This indicates that v shares the strongest topological connection with S. Similarly, other nodes with relatively high in-degree in this induced subgraph also exhibit closer connections with S. This property enables the set S to function as a trigger set, serving as the seed for grouping topologically similar nodes. By selecting nodes with the highest in-degrees from this induced subgraph, we can naturally cluster together nodes that are structurally similar.

The effectiveness of this procedure strongly depends on the quality of the trigger set. If S is chosen at random, the maximum in-degree in the induced subgraph tends to be small, which weakens the ability to cluster nodes with strong similarity. To mitigate this, our algorithm begins by sampling a connected small subgraph as the initial trigger set S_{init}, thereby ensuring inherent connectivity. We then select g nodes, where g denotes the grouping size during sorting—that exhibit the highest in-degrees within the subgraph induced by the out-neighbors of S_{init}.

Once the first node group is selected, it is extracted along with its incoming edges to avoid re-selection. Since some of their predecessors may still serve as in-neighbors of other nodes, they remain in the graph. These remaining predecessors are naturally chosen as the next trigger set for two reasons: (1) They ensure continuity of the sorting process, and (2) since some of them may also be common in-neighbors of the nodes in the previously selected group, they provide a natural source of similarity for the next grouping. The algorithm then iteratively repeats this procedure—using the current trigger set to identify new node groups while dynamically updating the triggers—until all nodes are exhausted.

After generating all node groups, we perform an additional adjustment to their sequential order to address potential imbalances in node degree. Although local degrees within subgraphs are not identical to global degrees, they remain strongly correlated. As a result, the groups produced in generation order tend to be arranged from higher to lower degree. Under fixed batch-size constraints, this ordering creates an imbalance: Earlier batches containing higher-degree nodes induce much larger subgraphs, which restricts batch size due to GPU memory limits, while later batches underutilize available memory.

To alleviate this imbalance, we apply a simple reordering heuristic:

1. Arrange all groups in their generation order.
2. Reverse the second half of the list.
3. Interleave the reversed second half with the first half.

To preserve intra-batch density and retain the spatial locality benefits from contiguous node expansion, the grouping size g must be a divisor of the batch size. Accordingly, AccuGraph sets g to a smaller power of two, ensuring compatibility with the batch size.

Importantly, this pre-sorting strategy optimizes structural compactness without compromising predictive accuracy under accumulated subgraph training. It provides a scalable and effective mechanism for constraining subgraph size inflation and mitigating redundancy across batches.

4.3 Hybrid Redundancy Caching and Reuse Technique

Although duplicated neighbors across batches can be diminished by our *TSPS* strategy, overlapped structure between batch subgraphs is unavoidable given real-world graph topologies.

Our framework employs full-neighborhood aggregation and uses gradient accumulation to simulate full-batch gradient descent. Consequently, within an epoch the intermediate embedding of a given node computed in different batches is exactly the same; this makes the caching and reuse of these embeddings accurate and lossless from the perspective of model correctness. Motivated by this observation, we prune the overlapped structure in batch subgraphs, cache the redundant intermediate embeddings, and reuse them across batches to reduce repeated transfers and recomputation without sacrificing accuracy.

As recurrence frequency and temporal locality differ across nodes, we partition redundant nodes into two categories and adopt distinct caching strategies

for each. The first category, High-frequency (*HF*) nodes, comprises nodes that appear as neighbors in a large fraction of batch subgraphs. They are identified after finalizing the batch sequence. We first compute the predecessor set $\mathcal{P}_i$ for each batch $\mathcal{B}_i$. Given K total sub-batches, we define the occurrence frequency of a node v as

$$\text{freq}(v) = |\{\mathcal{B}_i \mid v \in \mathcal{P}_i\}|, \tag{3}$$

and then define the high-frequency index $\text{HFi}(v)$ as

$$\text{HFi}(v) = \frac{1}{K}\text{freq}(v), \tag{4}$$

expressed as a percentage. Consistent with real-world graphs' scale-free properties, the node frequency distribution also follows a power-law pattern: A small fraction of nodes exhibit high HFi values while most nodes have low indices. Given a threshold τ, we classify the set of *HF* nodes as

$$\mathcal{H}(\tau) = \{v \mid \text{HFi}(v) \geq \tau\}. \tag{5}$$

Lowering τ increases $|\mathcal{H}(\tau)|$ and hence the static cache footprint. In practice we choose τ to balance redundancy reduction against the additional memory cost, which dependents on the number of *HF* nodes and embedding dimensionality. Intermediate embeddings for *HF* nodes in $\mathcal{H}(\tau)$ are precomputed and stored in the static cache within a whole epoch.

Apart from *HF* nodes, some nodes naturally reappear across neighboring batch subgraphs. The continuity introduced by $TSPS$ strengthens batch locality, which in turn amplifies the presence of these recurrent nodes. They are defines as another category of redundant nodes, cross-batch (*CB*) nodes. While *HF* nodes are defined globally, *CB* nodes are a local, per-batch notion and are identified on a per-layer basis. We explicitly exclude any node pre-classified as a *HF* node from the *CB* sets; consequently, the *CB* designation applies only to nodes that are not already labeled as *HF*. Here a layer k of a batch subgraph denotes the set of nodes at k-hop distance from the nodes in sub-batch. A node is classified as a *CB* node for layer k if it appears at the same layer of both the current batch subgraph and an adjacent batch subgraph; such nodes are further labeled as previous-batch *CB* or next-batch *CB* depending on whether the adjacent occurrence is in the preceding or the following batch. Consequently, each batch has a distinct set of *CB* nodes for every layer k. *CB* nodes are detected dynamically during batch pre-generation, and the first batch has no previous-batch *CB* nodes.

Concretely, we expand direct neighbors of the current sub-batch first and proceed iteratively to larger hop distances. During these subsequent expansions, expansion into nodes already designated as *HF* or previous-batch *CB* is suppressed to avoid redundant inclusion. Before finalizing a batch subgraph, we prefetch the node set for the next batch and mark those nodes that will occupy the same layer k in the next batch subgraph as next-batch *CB* nodes.

Figure 4 illustrates structural differences between complete and pruned batch subgraphs for 2-layer GNNs, along with their respective forward propagation processes. During forward propagation on full-neighborhood batch subgraphs,

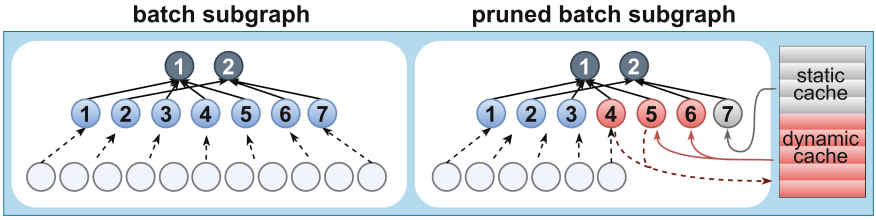


Fig. 4. Illustration of two subgraph structures for a 2-layer GNN (sub-batch size = 2) and the proposed workflow for embedding reuse and caching on pruned batch subgraphs. To simplify the figure, we de-emphasized the two-hop neighbors of the target nodes and their related edges.

information is aggregated layer by layer. For pruned batch subgraphs, intermediate embeddings of nodes whose neighbor expansion have not been suppressed, i.e., nodes 3 & 4, are computed. These embeddings are then concatenated with cached embeddings to construct complete inputs for the output layer. Compared with complete batch subgraphs, pruned batch subgraphs require transferring less data to GPU, since redundant nodes such as 5, 6, and 7 have their intermediate embeddings cached rather than repeatedly calculated, and have no need for there dependency data. Specifically, the embedding of *HF* node 7 is fetched from the static cache, and the embeddings of previous-batch *CB* nodes 5 & 6 are retrieved from the dynamic cache. All *HF* node embeddings, including that of node 7, are precomputed prior to processing any pruned batch subgraph and remain fixed in the static cache for the duration of the epoch; by contrast, the dynamic cache is refreshed after each batch to store newly computed *CB* embeddings. In this example, after the current batch the dynamic cache is updated to hold embeddings of CB nodes 4 & 5. The persistence of node 5 across consecutive batches demonstrates that cross-batch nodes may recur in multiple sequential batches. The representations of output nodes 1 & 2 are subsequently computed. Thus, this method effectively reduces redundant data transfers and duplicate computations during training.

5 Experiment

In this section, we empirically evaluate the proposed framework AccuGraph, targeting at answering the following major questions:

- How does it improve performance while reducing maximum memory consumption and maintaining the accuracy? (Sect. 5.2)
- How do the two technologies alleviate redundancy (Sect. 5.3)?
- How do the key parameters in the two technologies influence the latency and memory footprint related performance (Sect. 5.4 and Sect. 5.5)?

5.1 Setups

Environments. All the experiments are conducted on a machine with 1 NVIDIA 3090 GPU (24 GB memory), 1 24-core Intel Xeon CPUs (2.40 GHz), and 256 GB DDR4 host memory. The machine is installed with Ubuntu 18.04, CUDA library v11.3. Our experiments are carried out on DGL [22], an open-source framework for graph neural networks.

Table 1. Dataset statistics

Dataset	Feat. Dim.	#Nodes	#Edges	Avg degs.
Small	1,024	1,000,000	12,070,502	12.07
Medium	1,024	10,000,000	120,077,694	12.07
Products	100	2,449,029	61,859,140	25.26
Papers100M	128	111,059,956	1,615,685,872	14.55

Datasets. We evaluate our method on four large-scale, real-world graphs (Table 1): Products and Papers100M from the Open Graph Benchmark [6], and Small and Medium from the Illinois Graph Benchmark [11]. Node features for Papers100M, extracted from the Microsoft Academic Graph (MAG) [19], are 128-dimensional vectors computed as the mean of word2vec embeddings over all words in each paper's title and abstract, trained on the full MAG corpus; Products, representing an Amazon product co-purchasing network, uses 100-dimensional vectors obtained by applying PCA to the high-dimensional bag-of-words count vectors of product names and descriptions; Small and Medium, built primarily on MAG with additional human-annotated labels from both MAG and the Semantic Scholar Corpus [12], employ 1,024-dimensional Sentence-BERT embeddings of paper titles and abstracts. Since high-dimensional features might introduce redundancy and noise, and require substantial computational and storage overhead during training, we follow the PCA-based preprocessing pipeline of Products to compress them to 256 dimensions, effectively removing linear correlations, suppressing noise, and streamlining resource consumption.

GNN Models. All our experiments were conducted on two representative models, GCN [13] and GraphSage [5]. For GraphSage, we evaluate both the mean and pooling aggregators, denoted Sage-Mean and Sage-Pool, respectively, to capture their differing memory requirements during training. Every model uses two layers with a hidden size of 128. We fix the random seed across all runs to eliminate variability from random initialization, especially for the accuracy evaluations in Sect. 5.2.

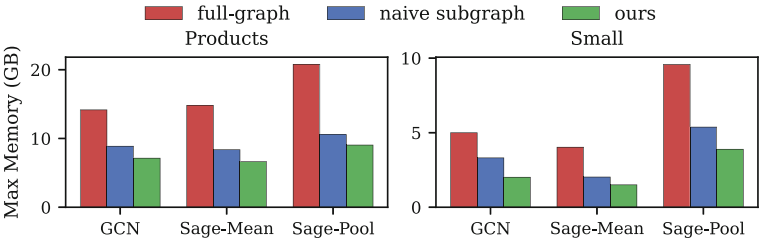


Fig. 5. Maximum GPU memory consumption of naive full-graph training, naive accumulated subgraph training, and our optimized method across datasets.

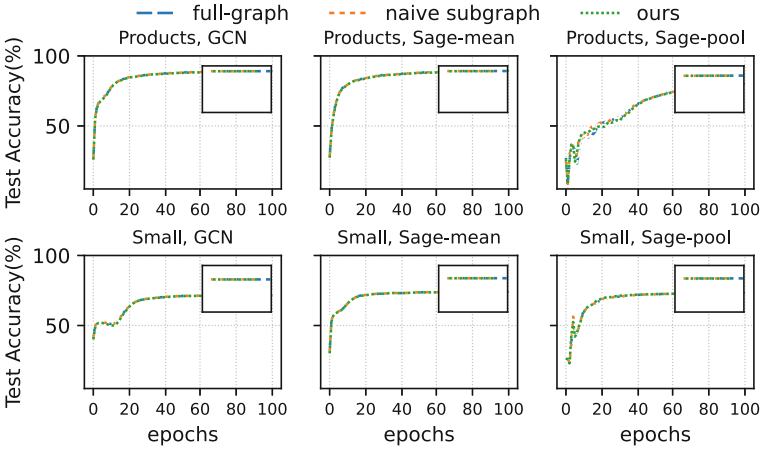


Fig. 6. Accuracy over epochs for three methods on two datasets (Products and Small). To help distinguish nearly overlapping curves, each subplot contains a zoomed inset in the upper-right that magnifies the final 5% of epochs.

5.2 Overall Results

We first assess peak memory consumption and accuracy progression across three training paradigms: full-graph training, naive accumulated subgraph training, and our method. Experiments use Products and Small datasets, which support naive full-graph training under our hardware constraints, with batch sizes set to 4,096 and 8,192 respectively for subgraph training methods. For clarity in the figures, the naive accumulated subgraph training method is abbreviated as *naive subgraph* in the legends of Figs. 5, 6 and 7.

In our overall experiments, we first demonstrate that accumulated subgraph training substantially reduces the maximum memory consumption compared to naive full-graph training, as shown in Fig. 5, while our optimized method further lowers the peak memory footprint. Importantly, both subgraph-based approaches maintain full-graph-level accuracy. As shown in Fig. 6, the accuracy curves of all three methods are almost perfectly overlapped.

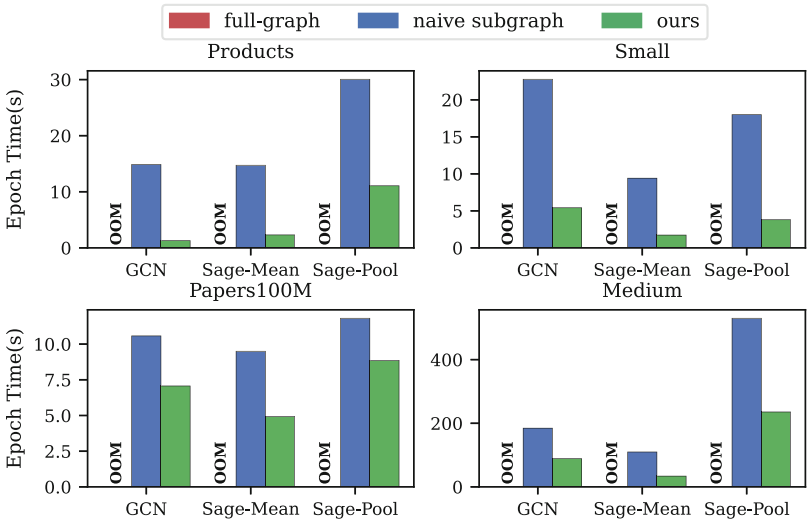


Fig. 7. Best attainable performance under constrained memory budgets (specified in Table 2) of each method, measured by per-epoch training time (lower is better).

Table 2. GPU memory limits and the largest attainable batch size (expressed as powers of two) for each dataset–model combination as in Fig. 7.

Dataset	GCN			Sage-mean			Sage-pool		
	Max. mem.	baseline	ours	Max. mem.	baseline	ours	Max. mem.	baseline	ours
Small	3 GB	2^{12}	2^{14}	3 GB	2^{14}	2^{16}	6 GB	2^{13}	2^{15}
Products	12 GB	2^{13}	2^{15}	12 GB	2^{13}	2^{14}	12 GB	2^{12}	2^{12}
Papers100M	15 GB	2^{16}	2^{16}	15 GB	2^{16}	2^{17}	24 GB	2^{16}	2^{16}
Medium	24 GB	2^{16}	2^{16}	24 GB	2^{17}	2^{18}	24 GB	2^{14}	2^{14}

When memory is abundant, our framework guarantees performance identical to full-graph training, as ensured by the workflow illustrated in Fig. 3. For brevity, we omit the specific numerical results in this case. However, when memory is constrained to a lower budget, as specified in Table 2, naive full-graph training fails due to OOM errors (Fig. 7), whereas both naive accumulated subgraph training and our optimized method execute successfully. Moreover, our method achieves a substantial speedup over the unoptimized baseline. Figure 7 reports the best attainable performance for each method: Our approach achieves a 2.7×–11.5× speedup in per-epoch runtime on Products, 4.2×–5.4× on Small, 1.3×–1.9× on Papers100M, and 2.1×–3.2× on Medium.

This speedup stems from two factors. First, both of our optimizations effectively reduce structural redundancy, which accelerates training even with a fixed batch size by lowering both data transfer time and computation time. Second, both methods shrink the scale of batch subgraph given fixed batch size and

therefor reduce the maximum memory consumption. This enables our framework to adopt larger batch sizes under the same memory budget, as depicted in Table 2. We will further demonstrate in Sect. 5.3 that both strategies effectively reduce redundancy for a fixed batch size, and in Sects. 5.4 and 5.5 that this redundancy reduction directly saves transfer and computation time, while also contributing to lower peak memory usage. All these experiments are conducted on the Products and Small datasets, with batch sizes fixed at 4,096 and 8,192, respectively.

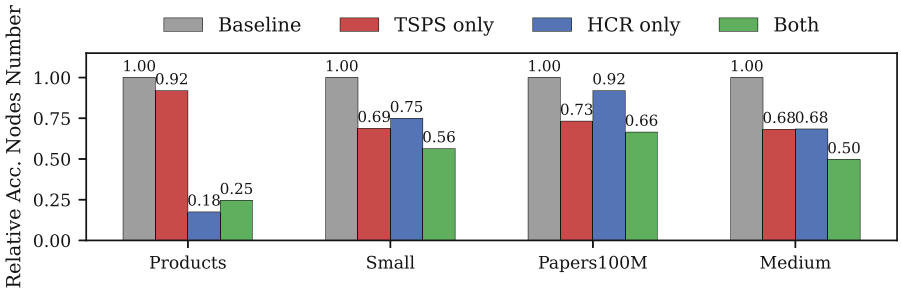


Fig. 8. Cumulative nodes under different combinations of optimization methods (the values were normalized with the results of the None method as the baseline).

5.3 Redundancy Reduction

This experiment compares four training variants: no optimization (baseline), $TSPS$ only, HCR only, and $TSPS+HCR$, in terms of the accumulated number of nodes per epoch (i.e., the total distinct nodes transferred/processed when iterating over all subgraphes). For the $TSPS$ experiments we fix the group size g to 256 for the ogbn-series datasets and to 512 for the igb-series datasets. For HCR we fix the high-frequency threshold parameter τ to 70. To make cross-dataset comparisons intuitive, the plotted values for each dataset are normalized by the baseline (no-optimization) result for that dataset; reported percentages below represent relative reductions with respect to this baseline (i.e., a 10% reduction means the normalized value is 0.90).

The measured reductions in accumulated nodes are as follows. $TSPS$ alone reduces accumulated nodes by approximately 8%, 31%, 27%, and 32% on Products, Small, Papers100M, and Medium, respectively. HCR alone yields reductions of roughly 82%, 25%, 8%, and 32% on the same datasets. When both strategies are applied together, the reductions become larger: about 75%, 44%, 34%, and 60%. These results are summarized in Fig. 8.

The effectiveness of each optimization varies across datasets due to differences in graph topology, such as community structure and degree distributions. In particular, for the Products dataset, applying both $TSPS$ and HCR yields

slightly less improvement than using HCR alone. We observe that this is because $TSPS$ increases overlap within a batch, which reduces the frequency of HB nodes appearing across different batch subgraphs. In other words, given the same τ, the number of HB nodes observed on a uniform dataset after sorting is lower under $TSPS$. Since Products exhibits a strong power-law degree distribution, a large number of HB nodes exist before sorting, and the decrease in HB nodes under $TSPS$ has a relatively large impact that is not fully compensated by the benefits of $TSPS$. Nevertheless, the combination of $TSPS$ and HCR consistently achieves meaningful redundancy reduction across all datasets.

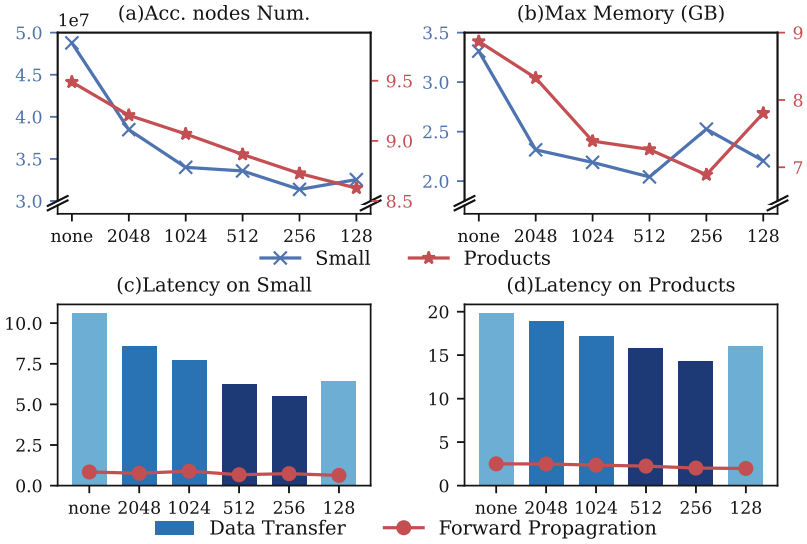


Fig. 9. Sensitivity of g in the $TSPS$ strategy on accumulated node count, peak GPU memory, and latency for Small and Products.

5.4 Sensitivity Analysis on g in $TSPS$ Strategy.

To evaluate the impact of the grouping size g in our topological similarity based nodes pre-sorting strategy, we perform a sensitivity analysis across datasets. Since g controls the number of nodes grouped during sorting, it directly influences batch density and thus redundancy reduction.

Figure 9(a) shows that smaller g generally leads to greater redundancy reduction, measured by the decrease in accumulated node count. For example, $g = 256$ reduces accumulated nodes by about 35% on Small and 8% on Products, with the difference attributable to their distinct graph structures. The benefit of reducing g exhibits diminishing returns and is not strictly monotonic. We observe a knee near $g = 256$, where $g = 128$ unexpectedly yields more accumulated nodes.

This suggests that overly fine-grained grouping fragments locality and degrades neighborhood clustering, making $g \in [256, 512]$ preferable.

Figure 9(b) indicates that maximum GPU memory usage closely follows the trend of accumulated nodes, confirming that pre-sorting reduces the size of batch subgraphs, including the largest subgraph that determines peak memory. Therefore, when memory footprint is the primary concern, we recommend $g = 256$ for Products and $g = 512$ for Small. This reduction also aligns with decreased host–device tansfer latency, since fewer distinct nodes require transfer. Although absolute transfer times are small, a statistically consistent downward trend is observable in Fig. 9(c,d).

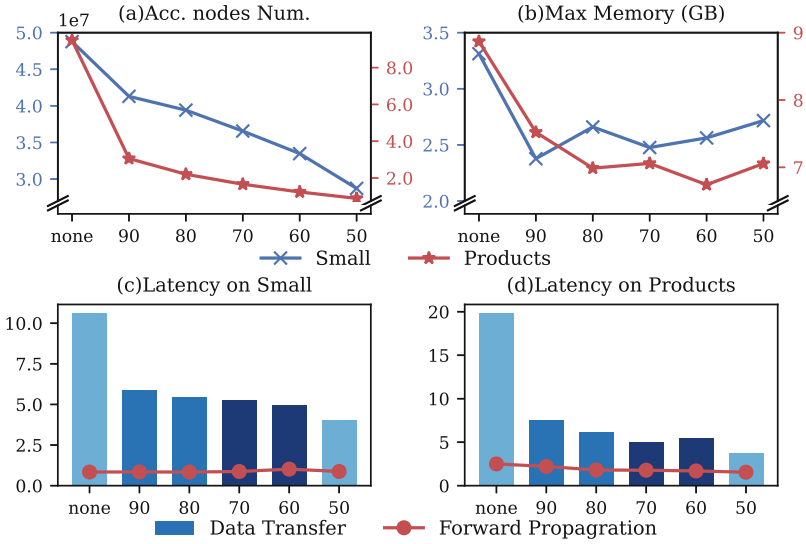


Fig. 10. Sensitivity of τ in the *HCR* workflow on accumulated node count, peak GPU memory, and latency for Small and Products.

5.5 Sensitivity Analysis on τ in *HCR* Technique

To evaluate the impact of the high-frequency threshold τ in our hybrid redundancy caching and reuse technique, we performed a sensitivity study. Since lowering τ expands the scope of redundant nodes pruned and increases the number of node embeddings retained, it is associated with the accumulated node count, transfer/computation overhead as well as peak memory consumption.

Figure 10(a) shows that, reducing τ yields substantial redundancy reductions on our datasets. When τ is set to 60, the accumulated node count decreases by 87% on Products and 31% on Small. The magnitude of the effect depends on graph topology: Products, with a stronger heavy-tailed degree distribution, produces many *HF* nodes that are amenable to static caching, whereas on Small

most nodes appear rarely across intermediate layers (our statistics show $> 99\%$ of nodes occur at $\leq 10\%$ frequency), limiting *HCR*'s benefit. Associated transfer/computation overhead also declined with the trend (Fig. 10(c,d)).

However, as lowering τ increases the static cache footprint because more τ embeddings must be stored. Consequently, the curve of peak GPU memory consumption exhibits fluctuations, particularly for Small. By contrast, Products, which benefits more from *HCR*, generally show a downward trend in peak memory usage. These findings demonstrate that optimal τ selection must be adapted to graph topology, trading off redundancy elimination against available memory.

6 Conclusion

In this paper, we presented AccuGraph, a framework for scaling full-graph GNN training to large graphs under limited GPU memory. Based on the accumulated subgraph training method we proposed, AccuGraph decomposes full-graph propagation into accumulated subgraph computations to reduce memory requirements while preserving mathematical equivalence to full-graph training. To address the redundancy challenge inherent in the naive approach, we introduced a topological similarity based node pre-sorting strategy to diminish duplicated neighbors across batches, along with a hybrid redundancy caching and reuse technique that further prunes a portion of overlapped structures between subgraphs. Experiments on large-scale datasets demonstrate that AccuGraph enables full-graph training on a 111M-node graph on a single GPU and achieves up to 11.5× speedup in training time with the optimizations.

Acknowledgments. This paper is supported by National Key Research and Development Program of China (No. 2024YFB4504200), National Natural Science Foundation of China (No. 62032023), the Open Fund of Science and Technology on Parallel and Distributed Processing Laboratory (No. 2023KJWPDL04). Menghan Jia (jiamenghan12@nudt.edu.cn) is the corresponding author of this paper.

References

1. Abadal, S., Jain, A., Guirado, R., López-Alonso, J., Alarcón, E.: Computing graph neural networks: a survey from algorithms to accelerators. ACM Comput. Surv. **54**(9) (oct 2021). https://doi.org/10.1145/3477141
2. Ai, X., et al.: Neutrontp: Load-balanced distributed full-graph GNN training with tensor parallelism. Proc. VLDB Endowm. **18**(2), 173–186 (2024)
3. Chen, J., Ma, T., Xiao, C.: Fastgcn: Fast learning with graph convolutional networks via importance sampling. In: International Conference on Learning Representations (2018)
4. Chiang, W.L., Liu, X., Si, S., Li, Y., Bengio, S., Hsieh, C.J.: Cluster-GCN: an efficient algorithm for training deep and large graph convolutional networks. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, pp. 257–266. KDD '19, Association for Computing Machinery, New York, NY, USA (2019)

5. Hamilton, W., Ying, Z., Leskovec, J.: Inductive representation learning on large graphs. Adv. Neural Inform. Process. Syst. **30** (2017)
6. Hu, W., et al.: Open graph benchmark: datasets for machine learning on graphs. Adv. Neural. Inf. Process. Syst. **33**, 22118–22133 (2020)
7. Hu, Y., Levi, A., Kumar, I., Zhang, Y., Coates, M.: On batch-size selection for stochastic training for graph neural networks (2020)
8. Huang, Y., et al.: Gpipe: easy scaling with micro-batch pipeline parallelism. In: Proceeding of Computer Science and Pattern Recognition (2019)
9. Ji, S., Tian, Y., Liu, F., Li, X., Wu, L.: PromptGCN: Bridging subgraph gaps in lightweight gcns. arXiv preprint arXiv:2410.10089 (2024)
10. Jia, Z., Lin, S., Gao, M., Zaharia, M., Aiken, A.: Improving the accuracy, scalability, and performance of graph neural networks with roc. Proc. Mach. Learn. Syst. **2**, 187–198 (2020)
11. Khatua, A., Mailthody, V.S., Taleka, B., Ma, T., Song, X., Hwu, W.m.: IGB: Addressing the gaps in labeling, features, heterogeneity, and size of public graph datasets for deep learning research. In: Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 4284–4295. New York, NY, USA (2023)
12. Kinney, R.M., et al.: The semantic scholar open data platform. ArXiv **abs/2301.10140** (2023). https://api.semanticscholar.org/CorpusID:256194545
13. Kipf, T.N., Welling, M.: Semi-supervised classification with graph convolutional networks (2017). https://arxiv.org/abs/1609.02907
14. Malliaros, F.D., Vazirgiannis, M.: Clustering and community detection in directed networks: a survey. Phys. Rep. **533**(4), 95–142 (2013). https://doi.org/10.1016/j.physrep.2013.08.002
15. Md, V., et al.: Distgnn: scalable distributed training for large-scale graph neural networks. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '21, Association for Computing Machinery, New York, NY, USA (2021). https://doi.org/10.1145/3458817.3480856, https://doi.org/10.1145/3458817.3480856
16. Peng, J., Chen, Z., Shao, Y., Shen, Y., Chen, L., Cao, J.: Sancus: sta le n ess-aware c omm u nication-avoiding full-graph decentralized training in large-scale graph neural networks. Proc. VLDB Endowment **15**(9), 1937–1950 (2022)
17. Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M., Monfardini, G.: The graph neural network model. IEEE Trans. Neural Networks **20**(1), 61–80 (2008)
18. Sen, P., Namata, G., Bilgic, M., Getoor, L., Galligher, B., Eliassi-Rad, T.: Collective classification in network data. AI Mag. **29**(3), 93–93 (2008)
19. Sinha, A., Shen, Z., Song, Y., Ma, H., Eide, D., Hsu, B.J.P., Wang, K.: An overview of microsoft academic service (mas) and applications. In: Proceedings of the 24th International Conference on World Wide Web, pp. 243–246. WWW '15 Companion, Association for Computing Machinery, New York, NY, USA (2015). https://doi.org/10.1145/2740908.2742839
20. Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., Bengio, Y.: Graph attention networks. In: 6th International Conference on Learning Representations, ICLR (2018)
21. Waleffe, R., Mohoney, J., Rekatsinas, T., Venkataraman, S.: Mariusgnn: resource-efficient out-of-core training of graph neural networks. In: Proceedings of the Eighteenth European Conference on Computer Systems, pp. 144–161. EuroSys '23, Association for Computing Machinery, New York, NY, USA (2023)
22. Wang, M., et al.: Deep graph library: Towards efficient and scalable deep learning on graphs. CoRR **abs/1909.01315** (2019)

23. Wang, Q., Chen, Y., Wong, W.F., He, B.: Hongtu: scalable full-graph GNN training on multiple gpus. Proc. ACM Manage. Data **1**(4), 1–27 (2023)
24. Wang, Q., Zhang, Y., Wang, H., Chen, C., Zhang, X., Yu, G.: Neutronstar: distributed GNN training with hybrid dependency management. In: Proceedings of the 2022 International Conference on Management of Data, pp. 1301–1315 (2022)
25. Xu, K., Hu, W., Leskovec, J., Jegelka, S.: How powerful are graph neural networks? arXiv preprint arXiv:1810.00826 (2018)
26. Yang, S., Zhang, M., Dong, W., Li, D.: Betty: Enabling large-scale GNN training with batch-level graph partitioning. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 103–117 (2023)
27. Yang, S., Zhang, M., Li, D.: Buffalo: enabling large-scale GNN training via memory-efficient bucketization. In: 2025 IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 1066–1081. IEEE (2025)
28. Zeng, H., Zhou, H., Srivastava, A., Kannan, R., Prasanna, V.: Graphsaint: graph sampling based inductive learning method. In: International Conference on Learning Representations (2019)
29. Zhang, Z., Cui, P., Zhu, W.: Deep learning on graphs: a survey. IEEE Trans. Knowl. Data Eng. **34**(1), 249–270 (2020)
30. Zhou, J., et al.: Graph neural networks: a review of methods and applications. AI open **1**, 57–81 (2020)

Internet of Things and Cyber-Physical-Social Computing

Average Delay Minimization Strategy for Multi-UAV Assisted MEC: A Lightweight Deep Reinforcement Learning Approach

Heng Liu[1], Qiang Tang[1(✉)], Kun Yang[2], and Lei Yang[1]

[1] School of Computer Science and Technology, Changsha University of Science and Technology, Changsha, China
{reyn,yl010306}@stu.csust.edu.cn, tangqiang@csust.edu.cn
[2] School of Computer Science and Electronic Engineering, University of Essex, Essex, England
kunyang@essex.ac.uk

Abstract. For the real-time computing issues of Internet of Things devices (IoTD) in urban scenarios, this paper proposes a multi-unmanned aerial vehicles (UAV) collaborative mobile edge computing (MEC) network framework. In this framework, the UAVs avoid obstacles while communicating with ground-based IoTDs via probabilistic LoS channels. Subsequently, aiming to minimize the average delay of IoTD, we formulate an optimization problem that jointly optimizes connection scheduling, resource allocation, and three-dimensional (3D) trajectories of UAVs. To avoid long training times, a lightweight DRL based on knowledge distillation is proposed to optimize the flight trajectory. Specifically, we train the obstacle avoidance teacher model and the service teacher model separately, then perform model distillation according to the teacher weights, and update the teacher weights by minimizing the loss. Additionally, a low-time complexity greedy algorithm is designed to solve the connection scheduling problem, and the computing resource allocation problem is addressed through optimization theory. Compared to benchmarks, our algorithm significantly reduces the average latency of IoTDs while compressing the network size by 93%, and effectively reduces training time.

Keywords: Multi-UAV · Resource Allocation · Knowledge Distillation

1 Introduction

The rapid development of 5G has accelerated the development of smart cities and spawned a series of latency-sensitive applications, such as intelligent transportation and digital twins [1]. However, most IoTDs cannot meet the delay demands due to their limited computing resources. Fortunately, UAV-assisted MEC provides a viable solution [2]. Due to their rapid deployment capability, enhanced

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 195–214, 2026.
https://doi.org/10.1007/978-981-95-8405-5_11

line-of-sight transmission probability, and ability to carry powerful computing resources, UAVs have been widely utilized in edge computing networks.

The computing offloading and trajectory planning problems in multi-UAV-assisted systems have been extensively studied [3,4]. However, some trajectory optimization studies are limited to open-space scenarios, whereas in reality, the impact of obstacles on channel conditions and flight paths cannot be neglected. Moreover, for computing offloading problems, solving them with DRL requires a huge action space and state space, which inevitably increases training costs [5]. Additionally, the sparse rewards designed for UAV obstacle avoidance may affect the gradient of the objective function's reward, potentially affecting algorithm convergence [6].

Fortunately, knowledge distillation (KD) enables neural networks to achieve lightweight deployment while maintaining performance [7–9]. Additionally, KD can distill policies from multiple teacher models into a single student model, enhancing both robustness and training efficiency. However, different teacher models may exhibit distinct decision-making. When conflicting knowledge is distilled into the student model, it may lead to performance degradation [5]. Therefore, it is crucial to select appropriate weights for different teacher models during the distillation.

To address the aforementioned challenges, this paper focuses on designing lightweight DRL algorithms for multi-UAV-assisted IoTDs offloading computing scenarios with obstacle avoidance. The main contributions of our work are summarized as follows:

- In consideration of real-world 3D obstacles and channel conditions, a multi-UAV-assisted edge computing network is proposed. Within this network, UAVs are edge servers that assist IoTDs with task offloading and computing while simultaneously avoiding obstacles. We jointly optimize the 3D flight trajectory, connection scheduling, and computing resource allocation to minimize the average latency of IoTD.
- For 3D trajectories of UAVs optimization, a DRL algorithm based on improved KD is proposed. The algorithm integrates a pre-trained obstacle-avoidance teacher model and a service-oriented teacher model, employing adaptive teacher weights for distillation. Meanwhile, we design a low-time complexity greedy algorithm to solve the connection scheduling problem. As for computing resource allocation, we solve it using optimization theory.
- Finally, we evaluate the performance of our proposed DTAC algorithm through simulations. Compared with the benchmark algorithms, DTAC performs better in terms of convergence, training time, average delay and coverage of IoTDs, even with a reduced network size.

The remainder of this paper is organized as follows. In Sect. 2, we summarize recent related work. Section 3 provides a detailed introduction to the system model. In Sect. 4, we solve the connection scheduling problem using heuristic algorithms, optimize resource allocation using optimization theory, and then combine them with a lightweight DRL to solve the original problem. Comparison results are presented in Sect. 5. Section 6 concludes the paper.

2 Related Work

DRL enables rapid decision-making in complex environments, yet its deployment is constrained by high training costs. KD is a promising model compression and transfer learning technique that transfers knowledge from DRL models to a lightweight network, thereby improving deployment efficiency. This section reviews and summarizes research on the application of these two technologies in MEC.

Li et al. [10] proved that short-job-first (SJF) is better than other queuing algorithms when handling queue tasks. The average user latency was optimized by a scheduling algorithm based on SJF combined with DRL. Bibi et al. [11] investigated the total service latency minimization problem for vehicles and proposed a two-stage DRL solution. They optimized vehicle transmission power by DDPG in the first stage, followed by computing resource allocation using DQN in the second stage. Li et al. [12] formulated a system energy consumption minimization problem and proposed a Beta-distribution-based IPPO algorithm to jointly optimize connection scheduling, communication, and computing resource allocation. Zhang et al. [13] proposed a framework for multi-UAV-assisted vehicle networking. They optimized spectrum resource allocation and flight trajectory problems through the MADDPG algorithm to minimize the average task completion latency.

Reviewing the above work, solving communication problems in multi-UAV-assisted MEC through DRL is a promising approach, but high-dimensional state and action spaces can affect training time and convergence.

Chen et al. [14] proposed a multi-teacher policy distillation-based DRL algorithm to maximize profit through joint optimization of transmission bandwidth and offloading decisions. By employing advantage value functions, the method maintained distillation stability and performance. Experimental results demon-

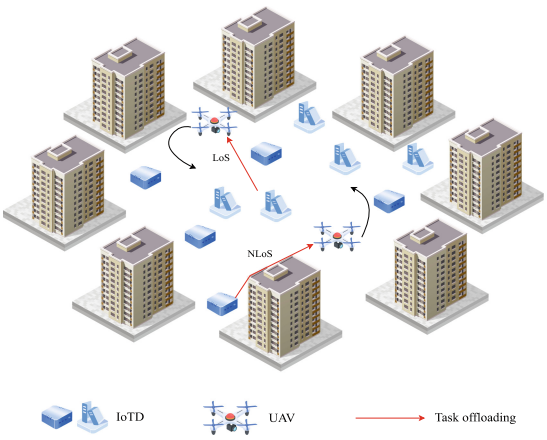


Fig. 1. System Model.

strated simultaneous network compression and profit improvement. However, this work predefined UAV hovering positions without considering trajectory optimization and computing resource allocation. Zhu et al. [15] introduced a priority mechanism to maximize rewards by connection scheduling optimization, utilizing a teacher distillation framework for knowledge transfer. This work mainly focuses on the optimization of offloading decisions, without considering resource allocation, trajectory optimization, and multi-teacher knowledge transfer. Wang et al. [16] developed a KD-based multi-agent DRL framework that enhances transmission efficiency through optimized beamforming coordination and resource allocation. However, this work did not account for potential performance degradation in the student model caused by knowledge discrepancies among different teacher models during the distillation process.

Currently, there are still some issues worth studying in multi-UAV-assisted MEC, such as research on 3D trajectories for UAV obstacle avoidance, research on reducing DRL training time, and the problem of ensuring model performance while achieving model compression under multi-teacher knowledge transfer. Therefore, this paper will study the above issues.

3 System Model

As shown in Fig. 1, the proposed edge computing network framework comprises N IoTDs, M UAVs, and multiple building obstacles of the same size. We assume each UAV and IoTD is equipped with both communication circuits and processing units. Let τ denote the time slot size, with $\mathcal{N} = \{1, 2, ..., N\}$, $\mathcal{M} = \{1, 2, ..., M\}$, and $\mathcal{T} = \{1, 2, ..., T\}$ representing the sets of IoTDs, UAVs, and time slots respectively. In each time slot, every IoTD needs to process a task with stochastic size. The coordinates of the UAV is $\mathbf{q}_m[i] = (x_m[i], y_m[i], z_m[i])$ and the coordinates of the IoTD is $\mathbf{q_n}[i] = (x_n[i], y_n[i], 0)$. The task characteristics are defined by a triple $\Psi_n[i] = (D_n[i], F_n[i], T^{max})$, where $D_n[i]$ denotes the data size of task, $F_n[i]$ represents the required CPU cycles of processing task, T^{max} specifies the maximum tolerable delay threshold.

3.1 UAVs' Mobility Model

To formalize the 3D UAV mobility model, we define $\phi_m[i]$ denotes the vertical flight angle of the m-th UAV in the i-th slot, $\theta_m[i]$ represents the horizontal flight angle, and $v_m[i]$ indicates the flight speed. Subject to the following constraints:

$$0 \leq \phi_m[i] \leq \pi, m \in \mathcal{M}, i \in \mathcal{T}, \tag{1}$$

$$0 \leq \theta_m[i] \leq 2\pi, m \in \mathcal{M}, i \in \mathcal{T}, \tag{2}$$

$$0 \leq v_m[i] \leq v^{max}, m \in \mathcal{M}, i \in \mathcal{T}. \tag{3}$$

Consequently, the 3D Euclidean coordinates of UAVs are updated by the following equations:

$$\begin{cases} x_m[i] = x_m[i-1] + v_m[i]\cos\phi_m[i]\cos\theta_m[i]\tau, \\ y_m[i] = y_m[i-1] + v_m[i]\cos\phi_m[i]\sin\theta_m[i]\tau, \\ z_m[i] = z_m[i-1] + v_m[i]\sin\phi_m[i]\tau. \end{cases} \tag{4}$$

Furthermore, accounting for architectural obstacles in urban environments, each UAV must satisfy the following altitude and region constraints:

$$x^{min} \leq x_m[i] \leq x^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{5}$$

$$y^{min} \leq y_m[i] \leq y^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{6}$$

$$z^{min} \leq z_m[i] \leq z^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{7}$$

$$\Gamma \triangleq \complement_{\zeta}\kappa. \tag{8}$$

where ζ represents the total area, κ denotes the region occupied by obstacles, and the UAV movable region Γ is the complement of the obstacle region within the total area ζ.

In order to prevent collisions between UAVs, we have:

$$\sqrt{||\mathbf{q_m}[i] - \mathbf{q_j}[i]||^2} \geq d^{min}, \forall m, j \in \mathcal{M}, i \in \mathcal{T}, n \neq j. \tag{9}$$

3.2 Communication Model

We employ a statistical probabilistic Line-of-Sight (LoS) channel model [17] to estimate the channel state between UAVs and IoTDs. The probability of establishing LoS connectivity between IoTD and UAV is given by:

$$P_{n,m}^{Los}[i] = A_1 + \frac{A_2}{1 + e^{-(A_3 + A_4\theta_{n,m}[i])}}, \tag{10}$$

where $\theta_{m,n}[i]$ denotes the elevation angle between the UAV and IoTD, with A_1, A_2, A_3, A_4 being environment-dependent constants. Consequently, the probability of Non-Line-of-Sight (NLoS) transmission between a UAV and IoTD is given by:

$$P_{n,m}^{NLos}[i] = 1 - P_{n,m}^{Los}[i]. \tag{11}$$

The channel gain between UAV and IoTD can be expressed separately for LoS and NLoS transmissions as:

$$h_{n,m}^{Los}[i] = \beta d_{n,m}^{-\alpha_1}, \tag{12}$$

$$h_{n,m}^{NLos}[i] = \eta\beta d_{n,m}^{-\alpha_2}, \tag{13}$$

where α_1 and α_2 represent the path loss for LoS and NLoS transmissions respectively, and $d_{n,m} = \sqrt{||\mathbf{q}_n[i] - \mathbf{q}_m[i]||^2}$ denotes the 3D Euclidean distance between the UAV and IoTD.

Based on the channel gain, the achievable transmission rates for LoS and NLoS links are respectively given by [18]:

$$R_{n,m}^{Los}[i] = B_{n,m}\log_2(1 + \frac{P_n h_{n,m}^{Los}}{\sigma^2 \varphi}), \tag{14}$$

$$R_{n,m}^{NLos}[i] = B_{n,m}\log_2(1 + \frac{P_n h_{n,m}^{NLos}}{\sigma^2 \varphi}), \tag{15}$$

Furthermore, the expected achievable transmission rate between UAV and IoTD can be expressed as:

$$R_{n,m}[i] = P_{n,m}^{NLos}[i] R_{n,m}^{NLos}[i] + P_{n,m}^{Los}[i] R_{n,m}^{Los}[i]. \tag{16}$$

3.3 Computing Model

Tasks generated by IoTDs can be processed either locally or offloaded to UAVs for execution. When task is processed locally, the local computing delay can be obtained:

$$t_n^{loc}[i] = \frac{F_n[i]}{f_n}, \tag{17}$$

where f_n is the computing frequency of the n-th IoTD. When the task is offloaded to the UAV, the transmission delay is:

$$t_{n,m}^{tr}[i] = \frac{D_n[i]}{R_{n,m}[i]}, \tag{18}$$

The computing delay of tasks processed on UAV can be expressed as:

$$t_{n,m}^{uav}[i] = \frac{C_m D_n[i]}{f_{n,m}^{uav}[i]}, \tag{19}$$

where $f_{n,m}^{uav}[i]$ denotes the computing frequency allocated by the m-th UAV to the n-th IoTD. Therefore, the task completion delay under offloading can be expressed as:

$$t_n[i] = t_{n,m}^{uav}[i] + t_n^{tr}[i]. \tag{20}$$

It should be noted that task completion delay must not exceed its maximum tolerable latency threshold and the allocated computing frequency cannot surpass the UAV's maximum frequency F^{max} [19]. Thus we have the following constraints:

$$s_{n,m}[i] t_n[i] + s_{n,0}[i] t_n^{loc}[i] \leq T^{max}, \forall n \in \mathcal{N}, i \in \mathcal{T}, \tag{21}$$

$$\sum\nolimits_{n=1}^{N} s_{n,m} f_{n,m}^{uav} \leq F^{\max}, \forall m \in \mathcal{M}, i \in \mathcal{T}, \tag{22}$$

where $s_{n,0}[i] = 1$ indicates task is processed locally and $s_{n,m}[i] = 1$ denotes task offloaded to UAV.

A UAV can serve multiple IoTDs and an IoTD can only connect to one UAV [20,21]. We have:

$$\sum\nolimits_{n=1}^{N} s_{n,m}[i] \leq S^{\max}, \forall m \in \mathcal{M}, i \in \mathcal{T}, \tag{23}$$

$$\sum\nolimits_{m=0}^{M} s_{n,m}[i] = 1, \forall n \in \mathcal{N}, i \in \mathcal{T}. \tag{24}$$

3.4 Problem Formulation

In a time slot, the completion delay of a task generated by an IoTD can be expressed as:

$$t_n^{com}[i] = s_{n,0}[i] t_n^{loc}[i] + s_{n,m}[i](t_{n,m}^{uav}[i] + t_n^{tr}[i]). \tag{25}$$

Our goal is to minimize the average latency of IoTDs. The joint optimization problem can be formulated as follows:

$$(\mathbf{P1}) : \min_{\mathbf{Q},\mathbf{S},\mathbf{F}} \frac{1}{N} \sum\nolimits_{i=1}^{T} \sum\nolimits_{n=1}^{N} t_n^{com}[i] \tag{23}$$

$$s.t. \quad \text{C1: } 0 \leq \phi_m[i] \leq \pi, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26a}$$

$$\text{C2: } 0 \leq \theta_m[i] \leq 2\pi, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26b}$$

$$\text{C3: } 0 \leq v_m[i] \leq v^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26c}$$

$$\text{C4: } x^{min} \leq x_m[i] \leq x^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26d}$$

$$\text{C5: } y^{min} \leq y_m[i] \leq y^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26e}$$

$$\text{C6: } z^{min} \leq z_m[i] \leq z^{max}, m \in \mathcal{M}, i \in \mathcal{T}, \tag{26f}$$

$$\text{C7: } \mathbf{q}_m[i] \in \Gamma, \forall m \in \mathcal{M}, i \in \mathcal{T}, \tag{26g}$$

$$\text{C8: } \sqrt{||\mathbf{q_n}[i] - \mathbf{q_j}[i]||^2} \geq d^{min}, \forall n, j \in \mathcal{N}, i \in \mathcal{T}, n \neq j, \tag{26h}$$

$$\text{C9: } t_n^{com}[i] \leq T^{max}, \forall n \in \mathcal{N}, i \in \mathcal{T}, \tag{26i}$$

$$\text{C10: } \sum\nolimits_{n=1}^{N} s_{n,m} f_{n,m}^{uav} \leq F^{max}, \forall m \in \mathcal{M}, i \in \mathcal{T}, \tag{26j}$$

$$\text{C11: } \sum\nolimits_{n=1}^{N} s_{n,m}[i] \leq S^{\max}, \forall m \in \mathcal{M}, i \in \mathcal{T}, \tag{26k}$$

$$\text{C12: } \sum\nolimits_{m=0}^{M} s_{n,m}[i] = 1, \forall n \in \mathcal{N}, i \in \mathcal{T}, \tag{26l}$$

$$\text{C13: } \sum\nolimits_{i=0}^{I} E_n[i] \leq E^{max}, \forall n \in \mathcal{N}, i \in \mathcal{T}. \tag{26m}$$

Although this paper does not optimize the energy consumption of UAVs, the total energy consumption of rotary-wing UAVs using the optimization method in this paper is considered in constraint C13 [22]. It can be seen that problem **P1** includes continuous and discrete variables, making it a challenging MINLP problem.

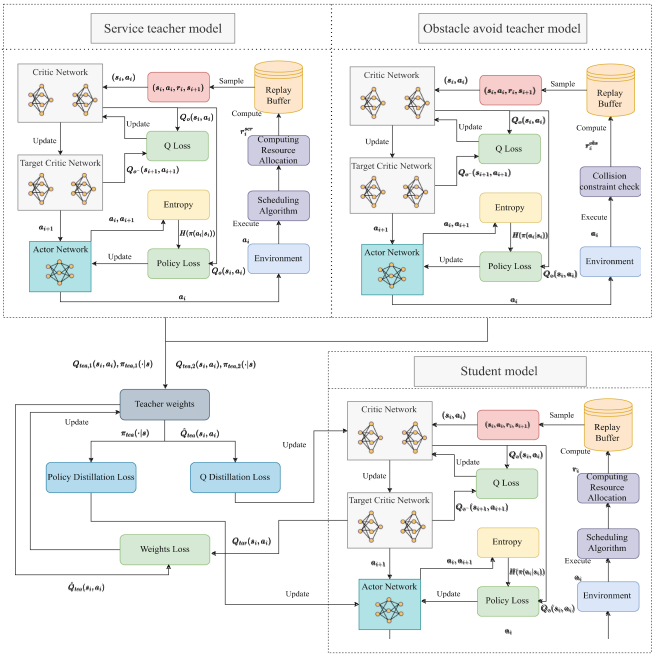


Fig. 2. DTAC Framework

4 Algorithm Design

To address the above problems, we first propose a knowledge distillation-based DRL algorithm for solving the 3D flight trajectories of UAVs, and then design heuristic algorithms to resolve the connection scheduling. Finally, computing resource allocation is solved by optimization theory.

4.1 Overall Algorithm Design

We design two teacher models specializing in training UAV service and obstacle avoidance policies, respectively. Specifically, the service policy is for UAVs to control trajectories to optimize the average latency of IoTD and the obstacle avoidance policy for UAVs is controlling trajectories to avoid collision with obstacles. Then, we import the two trained teacher models into the student model for distillation training. During the student model's training, we assign distinct adaptive weights to each teacher model to facilitate prioritized knowledge transfer. Weight updates are performed dynamically by combining the Q-value loss of the teacher model and the student target model.

Service Teacher Model. The flight trajectories of UAVs have Markov characteristics. To apply DRL, we define the states, actions, and reward function in the service teacher model as follows:

1) *State*: $s_i = \{\mathbf{q_m}[i], \forall m \in \mathcal{M}, i \in \mathcal{I}\}$, where $\mathbf{q_m}[i]$ is the coordinates of the m-th UAV in the i-th time slot.
2) *Action*: $a_i = \{\Delta\phi_m[i], \Delta\theta_m[i], \Delta v_m[i], \forall m \in \mathcal{M}, i \in \mathcal{I}\}$, where $\Delta\phi_m[i]$ is the increment of the vertical flight angle, $\Delta\theta_m[i]$ is the increment of the horizontal flight angle and $\Delta v_m[i]$ is the increment of the speed.
3) *Reward*: Based on our optimization objective and constraints C4-C6,C8,C13 the reward function is defined as:

$$r_i^{ser} = \frac{1}{N}\sum\nolimits_{i=1}^{T}\sum\nolimits_{n=1}^{N} t_n^{com}[i] - p_b[i], \tag{27}$$

where $p_b[i]$ represents the penalty for all UAVs violating the boundary constraint C4-C6, collision constraints between UAVs C8 and energy consumption constraint C13.

Obstacle Avoidance Teacher Model. The state space and action space settings for the obstacle avoidance teacher model are the same as those of the service teacher model. We design the reward function for the obstacle avoidance teacher model as follows:

Reward: Based on the constraints C4-C8 and C13, the reward function is defined as:

$$r_i^{obs} = r^{mov}[i] - p_b[i] - p_o[i], \tag{28}$$

In order to encourage UAVs that learn obstacle avoidance policy to explore the area better and find reasonable positions, we have set UAV movement reward r^{mov} [6]. $p_o[i]$ represents the penalty for all UAVs violating the obstacle collision constraint C7.

Student Model. The reward function for the student model is designed as follows:

Reward: Based on optimization objective and constraints C4-C8,C13, the reward function is defined as:

$$r_i = \frac{1}{N}\sum\nolimits_{i=1}^{T}\sum\nolimits_{n=1}^{N} t_n^{com}[i] - p_b[i] - p_o[i]. \tag{29}$$

In our model, UAVs act as agents, and the system model is modeled as the environment. In each time slot, the agent observes its state s_i, executes action a_i through policy π, obtains reward r_i, and moves to the next state s_{i+1}. The process terminates until it reaches the maximum time step I.

Compared to classical DRL methods, SAC optimizes policy stochasticity through entropy regularization, effectively reducing the possibility of local optimal convergence [23]. The algorithm's objective maximizes both cumulative rewards and policy entropy, with its actor network updated through the following loss function:

$$\mathcal{L}_{sac,\pi} = \mathbb{E}_{\pi}\left[\alpha \log \pi(a_i \mid s_i) - \min(Q_1(s_i, a_i), Q_2(s_i, a_i))\right], \tag{30}$$

Algorithm I Teacher Training Algorithm

Initialize teacher model learning rate of actor network l_a^{tea} and critic network l_c^{tea}.
Initialize critic networks of teacher model $Q_{tea,1}$ and $Q_{tea,2}$ respectively, and actor network π_{tea}. Initialize target critic network $Q_{tea,1^-}$ and $Q_{tea,2^-}$;
for epoch $\mathrm{e} = 1, 2, ..., \mathrm{e}^{\max}$ **do**
    Initialize the state of environment s_t;
    for time slot $i = 0, ..., T$ **do**
        Get action $a_i = \pi_{tea}(s_i)$ and obtain next state s_{i+1} after executing a_i;
        if Service teacher model **then**
            Solve connection scheduling by **Algorithm III** and resource allocation by CVX;
            Calculate reward r_i from formula (27) ;
        else
            Calculate reward r_i from formula (28);
        end if
        Store experience (s_i, a_i, r_i, s_{i+1}) into replay buffer R;
        Sample $\{(s_j, a_j, r_j, s_{j+1})\}_{j=\{1,...,J\}}$ from R;
        For each (s_i, a_i, r_i, s_{i+1}), compute y_{tea} via (32) ;
        Update $Q_{tea,1}$ and $Q_{tea,2}$ by minimizing their loss function (31);
        Update π by minimizing (30);
        Update entropy regularization coefficient α by formula (33);
        Update $Q_{tea,1^-}$ and $Q_{tea,2^-}$ through soft update;
    end for
end for

Among them, α is the regularization factor that balances reward and entropy, while $Q_1(s_i, a_i)$ and $Q_2(s_i, a_i)$ are obtained from two critic networks. The SAC critic network is updated using the following loss function:

$$\mathcal{L}_{sac,Q} = \mathbb{E}\left[\frac{1}{2}\left(Q_o(s_i, a_i) - y_i\right)^2\right], \forall o \in \{1, 2\}, \tag{31}$$

where y_i is the target Q-value, which is obtained from the target critic network. Target Q-value can be expressed as:

$$y_i = r_i + \gamma\left(\min(Q_{1^-}(s_{i+1}, a_{i+1}), Q_{2^-}(s_{i+1}, a_{i+1})) - \alpha \log \pi_\theta(\cdot|s_{i+1})\right), \tag{32}$$

Here, $Q_{1^-}(s_{i+1}, a_{i+1})$ and $Q_{1^-}(s_{i+1}, a_{i+1})$ are obtained from two target critic networks respectively. We need to adjust the regularization coefficient to help the agent adjust its actions for policy improvement. The loss function of the regularization coefficient α is defined as:

$$L(\alpha) = \mathbb{E}\left[-\alpha \log\left(a_i|s_i\right) - \alpha H_0\right]. \tag{33}$$

where H_0 is entropy and it used to measures the randomness of policy.

SAC is an efficient reinforcement learning algorithm that performs well in solving continuous action control problems. However, when the environment becomes more complex, especially when multiple constraints involve different

reward signals, their gradients may affect each other, leading to slow convergence during the learning process [6].

KD facilitates knowledge transfer between models, enabling student policy to better inherit policies from different teachers [14]. Each teacher model provides knowledge through its Q-values and action distributions, requiring the student network to minimize Q-value errors while approximating teachers' action distributions. The distillation loss is represented by mean square error (MSE) between the student's Q-values and a weighted combination of teachers' Q-values:

$$\mathcal{L}_{dis,Q_o} = \mathbb{E}\left[\left(Q_{stu,o}(s_i,a_i) - \hat{Q}_{tea}(s_i,a_i)\right)^2\right], \forall o \in \{1,2\}, \tag{34}$$

where the weighted combination of teacher's Q value is defined as $\hat{Q}_{tea}(s_i,a_i) = \sum_{k=1}^{K} w_k Q_{tea,k}(s_i,a_i)$, w_k represents the weight of the k-th teacher model, and $Q_{tea,k}(s_i,a_i)$ represents the Q value of the k-th teacher model.

Policy distillation loss is achieved by calculating the Kullback-Leibler (KL) divergence between the student and the teacher policy. Then, performing a weighted sum:

$$\mathcal{L}_{dis,\pi_{stu}} = \sum_{k=1}^{K} w_k D_{KL}\left(\pi_{stu}(\cdot|s) \| \pi_{tea,k}(\cdot|s)\right). \tag{35}$$

We can minimize its loss function to update the student model's critic network:

$$\mathcal{L}_{Q_{stu,o}} = \mathcal{L}_{sac,Q_o} + \mathcal{L}_{dis,Q_o}, \tag{36}$$

The actor network of the student model can be updated using the following loss function:

$$\mathcal{L}_{\pi_{stu}} = \mathcal{L}_{sac,\pi_{stu}} + \mathcal{L}_{dis,\pi_{stu}}. \tag{37}$$

The performance of each teacher model may vary at different stages of training. To integrate multiple teacher models for guiding the student's learning, an adaptive weight-adjusted method is proposed. We adjust the influence of teachers on student models by updating teacher weights. The update is performed by minimizing the difference between the weighted teacher Q-value and the student target Q-value. The loss function can be expressed as:

$$\mathcal{L}_w = \mathbb{E}\left[\left(\hat{Q}_{tea}(s_i,a_i) - Q_{tar}(s_i,a_i)\right)^2\right], \tag{38}$$

where $Q_{tar}(s_i,a_i)$ is the student target Q-value and its function can be denoted as $Q_{tar}(s_i,a_i) = \min(Q_{stu,1^-}, Q_{stu,2^-})$.

The architecture of the algorithm DTAC proposed in this paper is shown in Fig. 2, which consists of a service teacher model, an obstacle avoid teacher model, and a student model. We use pseudocode in Algorithm II to outline the specific algorithm flow. Firstly, we train the obstacle avoidance teacher model and the service teacher model separately using Algorithm I , and save their model parameters (line 1). Then, we initialize the student model parameters

Algorithm II DTAC Algorithm

1: Train the service teacher model and obstacle avoidance teacher model separately using **Algorithm I** .
2: Initialize student model learning rate of actor network l_a^{stu} and critic network l_c^{stu}.
3: Initialize critic networks of student model $Q_{stu,1}$ and $Q_{stu,2}$ respectively, and actor network π_{stu}. Initialize target critic network $Q_{stu,1-}$ and $Q_{stu,2-}$;
4: Initialize the obstacle avoidance teacher model weight as w_{obs} and the service teacher model weight as w_{ser}.
5: **for** epoch $\mathrm{e} = 1, 2, ..., \mathrm{e}^{\max}$ **do**
6: Initialize the state of environment s_i;
7: **for** time slot $i = 0, ..., T$ **do**
8: Get action $a_i = \pi_{stu}(s_i)$ and obtain next state s_{i+1} after executing a_i;
9: Solve connection scheduling by **Algorithm III** and resource allocation by CVX;
10: Calculate reward r_i by formula (29);
11: Store experience (s_i, a_i, r_i, s_{i+1}) into replay buffer R;
12: Sample $\{(s_j, a_j, r_j, s_{j+1})\}_{j=\{1,...,J\}}$ from R;
13: For each (s_i, a_i, r_i, s_{i+1}), compute y_{stu} via (32);
14: Update $Q_{stu,1}$ and $Q_{stu,2}$ by minimizing their loss function according to (36);
15: Update π_{stu} by minimizing (37);
16: Update entropy regularization coefficient α by formula (33);
17: Update $Q_{stu,1-}$ and $Q_{stu,2-}$ through soft update;
18: Update w_{obs} and w_{ser} according to its loss function (38);
19: **end for**
20: **end for**

and the teacher model weights. From lines 7 to 10, the student agent obtains action a_i through policy and enters into a new state s_{i+1} after interacting with the current state s_i. Based on the current location of the UAV, we can solve the connection scheduling problem using Algorithm III. Then, based on the connection scheduling solution, the computing resource allocation problem can be solved by the convex theory. We can get rewards r_i based on the above solution. Subsequently, store (s_i, a_i, r_i, s_{i+1}) into the replay buffer and update the neural network using batch sampling. It should be noted that the loss function of the student model is composed of distillation loss and training loss (lines 14–15). The weights of the teacher model are dynamically updated according to the loss function (line 18). Since we have discussed the relevant knowledge of student model updates in the previous section, it will not repeated here.

4.2 Scheduling Algorithm

Similar to [24], given UAV coordinates, we propose a low-complexity greedy algorithm for solving IoTD connection scheduling. Although this method is not globally optimal, the algorithm can quickly obtain a suboptimal solution without affecting the training time.

Algorithm III Scheduling Algorithm

Input: $q_m[i], \forall m \in \mathcal{M}$;
Output: $s_{n,m}[i], \forall n \in \mathcal{N}, m \in \mathcal{M}'$;
1: Initialize $s_{m,n}[i] = 0, \forall m \in \mathcal{M}, n \in \mathcal{N}$; $s_{m,0}[i] = 1, \forall m \in \mathcal{M}$; $D_m = 0, \forall m \in \mathcal{M}$;
2: **for** IoTD n **in** $\mathcal{N}$ **do**
3: Initialize $L_m = \{\}$;
4: **for** UAV m **in** $\mathcal{M}$ **do**
5: Calculate the horizontal distance $d^{hor}_{n,m}[i]$ according to $\mathbf{q_m[i]}$;
6: **if** $d^{hor}_{m,n}[i] \leq d^{link}$ **then**
7: Calculate transmission delay $t^{tr}_{n,m}[i]$ according to formula (18);
8: Store $[m, t^{tr}_{n,m}[i], D_m]$ into L_m;
9: **end if**
10: **end for**
11: **end for**
12: **for** IoTD n **in** $\mathcal{N}$ **do**
13: Sort the UAVs in L_m in ascending order with respect to D_m;
14: **for** UAV m **in** L_m **do**
15: Get the UAV $m = GetItem(L_m)$;
16: **if** constraint (23) and $t^{tr}_{n,m}[i] < t^{loc}_{n}[i]$ is met **then**
17: $s_{n,0}[i] = 0$, $s_{n,m}[i] = 1$;
18: $D_m = D_m + D_n[i]$
19: **break**;
20: **end if**
21: Remove item $[m, t^{tr}_{n,m}[i], D_m]$ from L_m;
22: **end for**
23: **end for**

As shown in Algorithm III , firstly, we initialize all IoTDs to perform local computing and initialize the amount of data processed on each UAV D_m (line 1). Then, initialize the preselected UAV set L_m for each IoTD, check the coverage relationship between the IoTD and each UAV, and calculate the transmission delay between the IoTD and each UAV (lines 3–10). Subsequently, we store the UAV id m, transmission delay $t^{tr}_{n,m}[i]$, and the amount of data processed on the UAV D_m into set L_m.

IoTD prioritizes UAV with the smallest data load, thereby obtaining more computing resources while also considering load balancing. Therefore, traverse each IoTD, if the transmission delay is less than the local computing delay and constraint (23) is satisfied, the IoTD offloads the task to the UAV with the lowest data load D_m (lines 12–17). In addition, update the amount of data processed on this UAV D_m (line 18). If the above conditions are not met, remove the UAV from the set (line 21) and continue traversing the other UAVs in the set (lines 14–22).

4.3 Resource Allocation Algorithm

In one time slot, based on connection scheduling Algorithm III and we can obtain the set of IoTDs $\mathcal{U}_m = \{1, 2, ..., u, ..., U\}$, which denotes the set of IoTDs generat-

ing tasks offloaded to the m-th UAV in the i-th time slots. Now we need to solve the resource allocation problem. Combining the constraints (21)(22)(23)(25), we can formulate the computing resource allocation subproblem as follows:

$$(\mathbf{P1.1}) : \min_{\mathbf{F}} \sum\nolimits_{u=1}^{U} \frac{C_m D_u[i]}{f_{u,m}^{uav}[i]} \tag{41}$$
$$s.t. \quad \text{C9-C10, C13.}$$

We found that this problem is a convex problem, so it can solved by PYCVX toolkit.

5 Simulation Results

In this section, the performance of DTAC is verified through numerical simulations carried out on Intel Core i5-12400F, NVIDIA RTX 4060, Python 3.10 and Pytorch 2.0.0. In our simulation experiment, both the x^{max} and y^{max} of the work area are set to 450 m. UAVs are deployed with initial positions of [50,400,125],[400,50,125]. 30 IoTDs are randomly distributed throughout the work area. For the neural network of the teacher model, we set the learning rate l_a^{tea} to 2×10^{-4} for the actor network and 2×10^{-3} for the critic network l_c^{tea}. Both the actor network and the critic network of the teacher model use two fully-connected hidden layers [400, 200]. As for the student model, we set the learning rate l_a^{stu} to 5×10^{-5} for the actor network and 5×10^{-4} for the critic network l_c^{stu}. The network structure of the student model is [64,64]. The initial weights of the obstacle avoidance teacher model w_{obs} and service teacher model w_{ser} are set to 0.5. The settings of the parameters can be found in Table 1.

Table 1. Parameters Setting

Parameters	Value	Parameters	Value	Parameters	Value
T	20 s	τ	1 s	D_n	[1000,1500]Kb
N	30	M	2	S^{max}	30
z^{max}	200 m	E^{max}	50KJ	T^{max}	1
A_1	-0.63	A_2	1.63	A_3	-0.4568
A_4	0.047	β	-60dB	η	-20dB
α_1	2.5	α_2	3.5	σ	1.3×10^{-14}
φ	6.61	B	10MHz	P	0.1W
F^{max}	50GHz	F_n	2.5GHz	v^{max}	30 m/s

5.1 Benchmark Algorithm

To evaluate the performance of DTAC, several algorithms are proposed for comparison as follows:
Deep Deterministic Policy Gradient (DDPG) [25]: We set the parameters of the DDPG algorithm and network architecture to be the same as those of the teacher model in DTAC. DDPG is used to optimize the trajectories of the UAVs.
Soft Actor-Critic (SAC) [26]: The parameters and network architecture of the SAC are identical to those of the teacher model in DTAC. SAC is used to optimize the trajectories of the UAVs.
SAC Algorithm Based on knowledge distillation (DAC): The teacher model and student model of DAC are the same as DTAC but do not consider updating teacher weights. DAC is used to optimize the trajectories of the UAVs.

5.2 Performance Evaluation

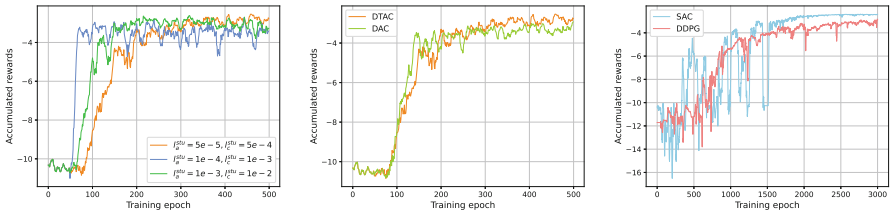
We will evaluate the performance of the DTAC algorithm based on comprehensive performance, convergence, flight trajectory, and average delay.

Comprehensive Performance: As shown in Table 2, we will evaluate the performance of the algorithms in terms of network structure, network size, training time, and reward. Specifically, the training time for DTAC and DAC is composed of the training time for the teacher model and the student model. It can be seen that DAC and DTAC have short training times. Although DAC reduces the network size compared to SAC, the knowledge conflict between teacher models during the distillation process leads to the degradation of accumulated rewards in the student model. DTAC achieves lightweight while introducing a teacher weights update mechanism, which can reduce the performance impact of knowledge conflicts on the student model. DDPG does not introduce an entropy mechanism, resulting in low efficiency of agents in environment exploration, which may cause the algorithm to fall into a local optimal solution.

Table 2. Model complexity, training time, and reward comparison.

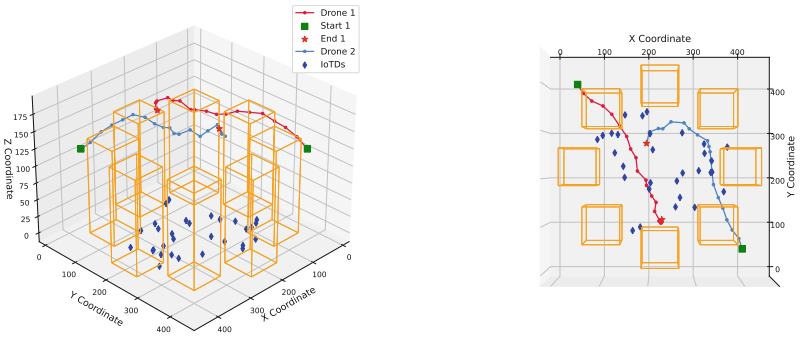
Algorithm	Network Structure	Network Size	Training Time	Reward
DTAC	[64,64]	24KB	1188 s	-2.37
DAC	[64,64]	24KB	1155.44 s	-2.85
SAC	[400,200]	337KB	1522.37 s	-2.32
DDPG	[400,200]	337KB	1242.69 s	-2.91

Convergence Performance: We show the convergence of the DTAC algorithm under different learning rates in Figure (3*a*), and the convergence of DTAC compared with different algorithms in Figure (3*b*) and Figure (3*c*). It can be seen that when the learning rate of the actor network is set to 5×10^{-5} and the critic network is set to 5×10^{-4}, DTAC converges in about 350 epochs and obtains the highest cumulative reward. When the learning rate is larger, although the DTAC algorithm converges faster, the cumulative reward curve exhibits great fluctuation, and the convergence point is a local optimum. DAC, as a KD-based DRL algorithm, also has a fast convergence speed. Both SAC and DDPG converged within approximately 2500 epochs, and SAC achieved higher cumulative rewards.



(a) (b) (c)

Fig. 3. (a) Convergence of DTAC under different learning rates. (b)(c) Comparison of convergence under different algorithms.

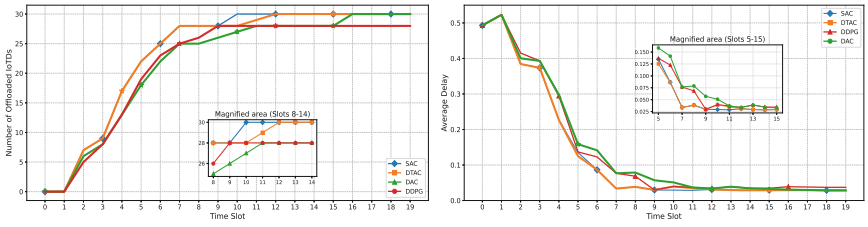


(a) (b)

Fig. 4. (a) Side view of trajectory. (b) Overview of the trajectory.

Flight Trajectory: As shown in Fig. 4, we show the 3D flight trajectories of two UAVs assisting IoTD offloading tasks while avoiding obstacles. Each UAV can fly toward target areas with dense IoTD without violating constraints to assist more IoTD offloading computing.

Scheduling Performance and Average Delay: In Figure (5*a*), we compare the number of IoTDs assisted by UAVs under different algorithms. It can be seen that DTAC and SAC have served more IoTDs than other algorithms since the 2-th time slot. From the 12-th time slot to the end of the epoch, they have achieved the effect of assisting all IoTDs in computing. DDPG has always served more IoTDs than the DAC algorithm from the 3-th to the 11-th time slot, but after the 16-th time slot, the DAC algorithm also finds a better flight trajectory and achieves full coverage of IoTDs. As shown in Figure (5*b*), the average delay of IoTDs task decreases continuously as the time slots change. This is because as the UAV flies toward the middle area, the tasks generated by the IoTDs can be processed by the UAV, which has more computing resources. We can find that DTAC also has a good effect on optimizing the average delay of IoTDs.



(a) (b)

Fig. 5. (a) Compare the number of IoTDs offload to UAV in each time slot within one epoch. (b) Compare the average delay of IoTDs in each time slot within one epoch.

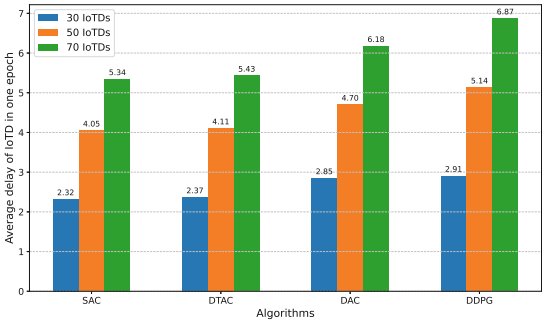


Fig. 6. Experimental results of average delay within one epoch under different number of IoTDs.

Figure (6) illustrates the experimental results of average delay within one epoch under different numbers of IoTDs. As the number of IoTDs increases, the average latency gradually increases. This is because UAVs have limited computing resources, resulting in fewer resources being allocated to each IoTD task.

6 Conclusion

Based on the consideration of obstacle avoidance, this paper proposes a multi-UAV-assisted mobile edge computing network to minimize the latency problem of IoTD in urban scenarios. We comprehensively consider the 3D dynamic trajectories of UAVs, connection scheduling, and computing resource allocation, with the goal of minimizing the average latency of IoTD task. We design a knowledge distillation-based lightweight deep reinforcement learning algorithm to optimize the 3D flight trajectories of UAVs. In addition, a low-time complexity greedy algorithm is proposed to solve the connection scheduling problem. Based on connection scheduling, the computing resource allocation problem is solved by convex theory. Simulation results show that our algorithm maintains good performance while reducing the network size by 93%. Future work will explore the improvement of UAV collaboration through multi-agent reinforcement learning.

Acknowledgments. This work was supported in part by A Project Supported by the Scientific Research Fund of the Hunan Provincial Education Department under Grant 23A0258, in part by the Hunan Provincial Natural Science Foundation of China under Grant 2021JJ30736 and Grant 2023JJ50331, in part by the Natural Science Foundation of China under Grant 62272063.

References

1. Hevesli, M., Seid, A.M., Erbad, A., Abdallah, M.: Task offloading optimization in digital twin assisted MEC-enabled air–ground IIoT 6G networks. IEEE Trans. Veh. Technol. **73**(11), 17527–17542 (2024)
2. Zhao, L., Chen, X., Jiao, L., Xu, M., Wei, J.: DRL-based UAV collaborative task offloading for post-disaster scenarios. In: Algorithms and Architectures for Parallel Processing, pp. 290–300 (2025)
3. Tang, J., Nie, J., Zhang, Y., Xiong, Z., Jiang, W., Guizani, M.: Multi-UAV-assisted federated learning for energy-aware distributed edge training. IEEE Trans. Netw. Serv. Manage. **21**(1), 280–294 (2024)
4. Lin, N., et al.: Deep-reinforcement-learning-based computation offloading for servicing dynamic demand in multi-UAV-assisted IoT network. IEEE Internet Things J. **11**(10), 17249–17263 (2024)
5. Chen, J., et al.: Deep reinforcement learning based resource allocation in multi-UAV-aided MEC networks. IEEE Trans. Commun. **71**(1), 296–309 (2023)
6. Xu, L., Wang, T., Wang, J., Liu, J., Sun, C.: Attention-based policy distillation for UAV simultaneous target tracking and obstacle avoidance. IEEE Trans. Intell. Veh. **9**(2), 3768–3781 (2024)

7. Ye, X., Mao, Y., Yu, X., Fu, L.: Intelligent omni-surface-aided integrated sensing and communications based on deep reinforcement learning with knowledge transfer. IEEE Trans. Wireless Commun. **24**(5), 4344–4360 (2025)
8. Xiao, H., Fu, L., Shang, C., Bao, X., Xu, X.: A knowledge distillation compression algorithm for ship speed and energy coordinated optimal scheduling model based on deep reinforcement learning. IEEE Trans. Transport. Electri. **11**(1), 945–960 (2025)
9. Zeng, T., Zhang, X., Duan, J., Yu, C., Wu, C., Chen, X.: An offline-transfer-online framework for cloud-edge collaborative distributed reinforcement learning. IEEE Trans. Parallel Distrib. Syst. **35**(5), 720–731 (2024)
10. Tang, Q., Li, B., Yang, H.H., Li, Y., He, S., Yang, K.: Delay and load fairness optimization with queuing model in multi-AAV assisted MEC: A deep reinforcement learning approach. IEEE Trans. Netw. Serv. Manage. **22**(2), 1247–1258 (2025)
11. Bibi, M., Ullah, S.A., Jung, H., Hassan, S.A.: DRL-driven dual-stage resource optimization strategy for efficient computational offloading in MEC-enabled vehicular networks. IEEE Transactions on Vehicular Technology, pp. 1–15 (2025)
12. Li, B., Zhu, X., Wang, J.: Robust trajectory design and task scheduling with data compression in industrial internet of things assisted by UAV. IEEE Internet Things J. **12**(9), 12501–12511 (2025)
13. Zhang, W., Tan, L., Huang, T., Huang, X., Huang, M., Zhang, G.: Resource allocation and trajectory optimization in multi-UAV collaborative vehicular networks: an extended multiagent drl approach. IEEE Internet Things J. **12**(8), 9391–9404 (2025)
14. Chen, Z., Zhang, J., Zheng, X., Min, G., Li, J., Rong, C.: Profit-aware cooperative offloading in UAV-enabled MEC systems using lightweight deep reinforcement learning. IEEE Internet Things J. **11**(12), 21325–21336 (2024)
15. Zhu, T., Tian, H., Dou, W.: A priority-aware task offloading method with knowledge distillation in edge computing. In: 2024 IEEE Smart World Congress (SWC), pp. 956–962 (2024)
16. Wang, X., Sun, G., Xin, Y., Liu, T., Xu, Y.: Deep transfer reinforcement learning for beamforming and resource allocation in multi-cell miso-ofdma systems. IEEE Trans. Signal Inform. Process. Over Netw. **8**, 815–829 (2022)
17. Gao, Y., Yuan, X., Yang, D., Hu, Y., Cao, Y., Schmeink, A.: UAV-assisted MEC system with mobile ground terminals: DRL-based joint terminal scheduling and UAV 3D trajectory design. IEEE Trans. Veh. Technol. **73**(7), 10164–10180 (2024)
18. Tang, F., Xu, J., Liu, X., Yao, A., Fang, M., Li, X.: Enhancing delay-sensitive task offloading: A multi-agent deep reinforcement learning algorithm for MEC-based aiot systems. In: Algorithms and Architectures for Parallel Processing, pp. 351–362. Springer Nature Singapore, Singapore (2025)
19. Bai, Z., Lin, Y., Cao, Y., Wang, W.: Delay-aware cooperative task offloading for multi-UAV enabled edge-cloud computing. IEEE Trans. Mob. Comput. **23**(2), 1034–1049 (2024)
20. Shao, Z., Yang, H., Xiao, L., Su, W., Chen, Y., Xiong, Z.: Deep reinforcement learning-based resource management for UAV-assisted mobile edge computing against jamming. IEEE Trans. Mob. Comput. **23**(12), 13358–13374 (2024)
21. Tang, Q., Yang, Y., Yang, H., Cao, D., Yang, K.: Energy consumption minimization for hybrid federated learning and offloadable tasks in UAV-enabled WPCN. IEEE Trans. Netw. Sci. Eng. **11**(5), 4639–4650 (2024)
22. Han, H., Zhan, C., Lv, J., Xu, C.: Energy minimization for cellular-connected aerial edge computing system with binary offloading. IEEE Internet Things J. **11**(6), 9558–9571 (2024)

23. Deng, T., et al.: Entropy normalization sac-based task offloading for UAV-assisted mobile-edge computing. IEEE Internet Things J. **11**(15), 26220–26233 (2024)
24. Wang, L., Wang, K., Pan, C., Xu, W., Aslam, N., Nallanathan, A.: Deep reinforcement learning based dynamic trajectory control for UAV-assisted mobile edge computing. IEEE Trans. Mob. Comput. **21**(10), 3536–3550 (2022)
25. Liu, Y., Yan, J., Zhao, X.: Deep reinforcement learning based latency minimization for mobile edge computing with virtualization in maritime uav communication network. IEEE Trans. Veh. Technol. **71**(4), 4225–4236 (2022)
26. Goudarzi, S., Ahmad Soleymani, S., Hossein Anisi, M., Jindal, A., Xiao, P.: Optimizing UAV-assisted vehicular edge computing with age of information: an sac-based solution. IEEE Internet Things J. **12**(5), 4555–4569 (2025)

LLM-Enhanced Multi-AUV Collaborative Search via Multi-agent Reinforcement Learning

Peijun Dong[1], Hang Tao[1], Yang Zhao[1], Wei Shi[2], and Hanjiang Luo[1(✉)]

[1] Shandong University of Science and Technology, Qingdao, China
hjluo@sdust.edu.cn
[2] Qingdao University of Science and Technology, Qingdao, China

Abstract. Multiple autonomous underwater vehicles (AUVs) have been widely utilized for underwater search missions. However, traditional multi-AUV collaborative search methods based on multi-agent reinforcement learning (MARL) suffer from a cold start problem due to the lack of prior knowledge, which hinders efficient cooperation in the early stage of search operations. Furthermore, due to harsh environments and unstable underwater acoustic communication links, it is difficult for AUVs to obtain timely information, which reduces collaborative search efficiency among AUVs. To address these challenges, we integrate the reasoning capabilities of large language models (LLMs) into MARL framework, and propose an LLM-enhanced multi-AUV cooperative search scheme. In this scheme, to tackle with the cold start problem, we leverage LLMs to infer a target probability map (TPM) from the search task and underwater terrain information. Then, we use TPM to guide AUV's search and alleviate the cold start problem caused by unfamiliarity with the new environment in the early stage of search operations. Moreover, we design an information forecaster (Forecaster) using LLMs, which infers the states of disconnected AUVs under unstable link interruptions from historical data and incorporates into TPM's updates enhancing cooperation efficiency. Finally, we design the scheme based on multi-agent deep deterministic policy gradient (MADDPG), improving the efficiency of multi-AUV collaborative search. The effectiveness of the proposed scheme is verified through simulations.

Keywords: Cooperative search · Large language models · Information fusion · Target probability map · Multi-agent reinforcement learning

1 Introduction

Multiple autonomous underwater vehicles (AUVs) have been widely used in maritime emergency search operations, as these are capable of performing tasks that individuals cannot perform alone [1]. However, to carry out underwater emergency search missions leveraging multiple AUVs efficiently faces significant challenges under complex environments. As multi-AUV collaborative search tasks

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 215–235, 2026.
https://doi.org/10.1007/978-981-95-8405-5_12

typically require coordination and decision-making among multi-AUV [2], the harsh environments and communication constraints of underwater acoustic links (e.g., unstable links, low data rates and high delays) make it difficult for multi-AUV to effectively search the targets [3,4]. Although there have been remarkable advances of MARL in collaborative multi-agent applications in recent years, (e.g., as a powerful framework enabling agents to learn optimal strategies through interactions with the environment and other agents [5]), as shown in Fig. 1, when these are utilized in multi-AUV cooperative search operations, several challenges persist.

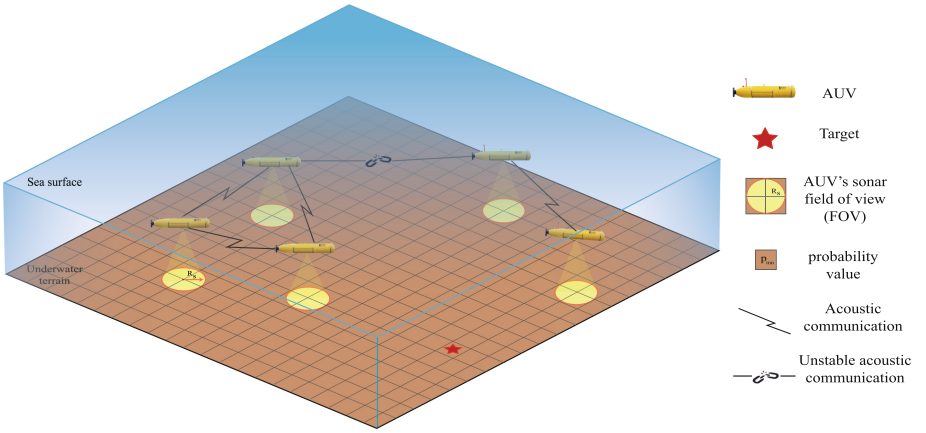


Fig. 1. The scenario of multi-AUV collaborative search based on target probability map under underwater acoustic communication.

A prominent issue is the cold start problem [6,7]. During the cold start phase, agents begin to explore the environment from an unfamiliar state due to the absence of prior knowledge, resulting in inefficient behaviors in the early stages. The AUV collaborative search methods based on MARL also have a cold start problem. Due to the lack of the ability to combine task and environmental information for reasoning with prior knowledge, AUVs have insufficient prior knowledge to search important areas, which reduce the efficiency of collaborative search in the early stage of exploration. Leveraging the extensive knowledge database, experience, and powerful reasoning capabilities of large language models (LLMs), it is possible to generate contextually relevant prior knowledge based on task information and environmental information [8]. This makes LLMs a promising tool for mitigating the cold start problem in MARL [7]. In particular, LLMs can assist agents in quickly adapting to complex cooperative tasks.

In order to further enhance the efficiency of multi-AUV collaborative search, it is critical for AUVs to exchange and fuse information in a timely manner. One critical usage of timely information exchange and fusion is to construct a target probability map, so as to facilitate the coordination efficiency of multi-AUV [9].

However, when the communication link is unstable [10], it becomes difficult for AUVs to obtain real-time information and fail to update the target probability map in a timely manner, thereby reducing the efficiency of collaborative search operations. To mitigate this issue, Zhu et al. [11] used the polar kalman filter to predict the states of AUVs under unstable communication conditions. The predicted results are integrated into the search map as compensation data, thereby improving the efficiency of collaborative search among AUVs. Wu et al. [12] proposed a predictive compensation method to address the unstable communication issue by integrating the least squares method and the extended kalman filter for unstable communication compensation. However, these methods are sensitive to system noise parameters and have poor generalization and scalability.

To address these issues, as LLMs possess strong comprehension and reasoning capabilities [5,13], in this paper, we first propose to leverage LLMs to tackle the challenges in the multi-AUV collaborative search tasks. We utilize LLMs to generate effective prior knowledge based on task information, thereby enhancing AUV decision-making during the exploration phase. Furthermore, we investigate to guide the LLM to infer the state information of disconnected AUVs under unstable communication conditions and integrate the inferred data into the MARL framework. This is achieved by incorporating the inference results into the reward mechanism to improve the decision-making of AUVs and enhance the efficiency of multi-AUV collaborative search. The main contributions of this paper are summarized as follows.

- To address the cold start problem in MARL-based multi-AUV cooperative search, we leverage LLMs to infer the initial target probability map based on complex search task and terrain information, mitigating the cold start issue.
- To tackle with the untimely target probability map fusion caused by unstable links, we utilize LLMs to predict the information of disconnected AUVs based on historical data, mitigating redundant searches resulting from untimely information fusion.
- We evaluate the performance of the proposed algorithms through simulations. The results show that the algorithms enhance the efficiency of multi-AUV collaborative search.

The structure of the rest of this paper is as follows. Section 2 introduces the related work. Section 3 provides a detailed description of the models. Section 4 presents the scheme design. Section 5 presents the performance evaluation. Section 6 concludes the paper.

2 Related Work

In MARL, the cold start problem arises when agents struggle to effectively initialize their policies due to the lack of prior knowledge, leading to slow or suboptimal learning performance. Traditional approaches to addressing this issue typically include pre-trained, imitation learning, and expert knowledge. These methods can quickly provide baseline behavioral patterns, reducing the time that agents

spend blindly exploring the environment. However, they also exhibit limitations. Pre-trained models [14] may fail to generalize well to specific task environments, imitation learning [15] depends on high-quality demonstration data, and expert knowledge [16] can introduce subjective biases and is often difficult to scale.

In order to further improve the efficiency of AUV collaborative search, it is critical for AUVs to exchange and fuse search information in a timely manner when performing search tasks. The research on communication in multi-AUV systems mainly includes two types: reliable communication and unstable communication. At present, most of multi-AUV cooperative target search methods are based on the assumption of reliable communication conditions [17], and the capability of these methods significantly relies on reliable communication. Cai et al. [18] proposed a fuzzy clustering-based target probability map construction method that improves information fusion and search efficiency among AUVs. Liu et al. [19] proposed a distributed multi-AUV collaborative search system that enhances robustness and enables effective information exchange and fusion through a time-stamped mechanism. When the communication link is unstable, the search performance of these methods will be seriously affected. Research on unstable communication includes the work by Jia et al. [20], who proposed a multi-AUV asynchronous update search strategy based on high-capacity opportunistic communication to maximize target search performance under unstable communication conditions. However, their study does not address how to mitigate the negative impacts of communication instability. Zhu et al. [11] proposed using a polar kalman filter to predict AUV states under unstable communication, integrating the predictions into the search map to enhance collaborative search efficiency. However, this method is highly sensitive to parameter settings and exhibits poor generalization performance and scalability.

3 System Model

3.1 Multi-AUV Collaborative Search Model

In order to effectively search for target, AUVs should continuously update their environmental states through information exchange. To this end, the target probability map model should be built. This paper considers a typical scenario of multi-AUV collaborative search based on target probability map under underwater acoustic communication [19]. As shown in Fig. 1, which is defined as follows. (1) The two-dimensional search area A of size $L_x \times L_y$ is discretized into $N_x \times N_y$ grid, p_{mn} indicates the likelihood that the target exists in the grid (m, n). (2) The initial position of the target is unknown, and the target moves a constant speed. (3) There are N AUVs, denoted as $B = \{b_1, b_2, \ldots, b_N\}$, the AUVs are equipped with sonars, acoustic communication equipment. Each AUV can sense the search area through sensors at a distance of R_s. The term field of view (FOV), denoted as $FOV_i(t)$, represents the sensing coverage area of AUV_i at time t. In addition, each AUV can communicate with neighbors, and the acoustic communication range is denoted by R_a. When the distance between AUVs exceeds R_a, communication may become unstable, leading to data transmission failure.

3.2 Target Probability Map Update Model

The target probability map (TPM) is used to represent the probability of the presence of a target at different locations within the search area. It helps AUVs navigate towards potential targets more efficiently. As AUVs gather information and make detections, the TPM is updated to reflect changes in the likelihood of a target being present in various parts of the search area. The TPM is derived using the sonar detection model and the Bayesian formula [21]. The sonar detection probability P_d is given by

$$P_d = \begin{cases} P_D \cdot \exp\left(-\frac{R_s}{D_s}\right), & 0 < d \leq D_s \\ P_F, & d > D_s \end{cases}, \tag{1}$$

where P_D represents for the sonar's detection precision, P_F represents the sonar's false alarm probability, R_s represents the detection distance, and D_s is the maximum detection distance of the sonar.

In multi-AUV cooperative search tasks, each AUV maintains a TPM representing the entire search area in a grid. Each grid cell (m, n) corresponds to a probability value $p^i_{mn}(t)$, indicating the probability estimated by the i-th AUV that a target exists in that cell at time t. At each time step, the AUV updates the probabilities of grid cells within its sensing range based on its own sonar detection results using Bayesian update. The update formula is as follows.

$$p^i_{mn}(t) = \begin{cases} p^i_{mn}(t-1), & \text{if the cell is not observed} \\ \dfrac{p_{d_i} \cdot p^i_{mn}(t-1)}{p_{f_i} + (p_{d_i} - p_{f_i}) p^i_{mn}(t-1)}, & \text{if observed and } O(t) = 1 \\ \dfrac{(1 - p_{d_i}) \cdot p^i_{mn}(t-1)}{1 - p_{f_i} + (p_{f_i} - p_{d_i}) p^i_{mn}(t-1)}, & \text{if observed and } O(t) = 0 \end{cases}, \tag{2}$$

where p_{d_i} denotes the detection probability of AUV i, p_{f_i} denotes its false alarm probability, and $O(t)$ represents the detection result at the current time (1 indicates target detected, 0 indicates no detection).

3.3 Search Information Fusion Model

To compensate for the limited detection capability of each AUV, an information fusion mechanism is constructed to accelerate convergence towards a consistent and accurate TPM across all AUVs. After AUV updates its TPM based on the observed information, each AUV enters a communication and fusion phase, integrating information from neighbors to jointly optimize the TPM.

The set of AUVs within the communication range of AUV i at time t, including itself, is defined as

$$N_i(t) = \{j \mid \|\mu_i(t) - \mu_j(t)\| \leq R_a\} \cup \{i\}, \tag{3}$$

where $\mu_i(t)$ is the position of AUV i at time t, and R_a is the distance of acoustic communication. The fusion weights $w_{ij}(t)$ assigned to neighbors are defined as

$$w_{ij}(t) = \begin{cases} \rho, & j = i \\ \frac{1-\rho}{d_i(t)-1}, & j \in N_i(t),\ j \neq i \\ 0, & j \notin N_i(t) \end{cases}, \tag{4}$$

where $\rho \in (0, 1)$ is the self-weight coefficient, and $d_i(t) = |N_i(t)|$ is the number of neighbors including AUV i itself. Each AUV converts its cell-wise probability estimate to the log-odds form [19]

$$\mathcal{L}^i_{mn}(t) = \ln\left(\frac{p^i_{mn}(t)}{1 - p^i_{mn}(t)}\right), \tag{5}$$

where $p^i_{mn}(t)$ is AUV i's probability estimate for cell (m, n). Then, it performs weighted fusion of log-odds from its neighbors

$$\tilde{\mathcal{L}}^i_{mn}(t) = \sum_{j=1}^{N} w_{ij}(t) \cdot \mathcal{L}^j_{mn}(t). \tag{6}$$

The result is converted back to probability space

$$\tilde{p}^i_{mn}(t) = \frac{1}{1 + \exp\left(-\tilde{\mathcal{L}}^i_{mn}(t)\right)}. \tag{7}$$

This log-odds-based fusion framework improves numerical stability and ensures that each AUV incrementally aligns its TPM with those of its neighbors. Repeated over time and across the entire search space, this mechanism enhances shared situational awareness and significantly improves the collaborative search efficiency of the AUV swarms.

4 Scheme Design

4.1 Scheme Description

In this section, we propose an LLM-enhanced MARL-based multi-AUV collaborative search scheme to enhance the efficiency of AUV collaborative search. As shown in Fig. 2, to address the cold start problem in MARL, we employ LLMs to infer key search regions based on search task and terrain information, thereby generating initial TPM to guide early AUV behavior. In addition, to mitigate untimely information fusion caused by limited acoustic communication, we use LLMs to infer missing observations from historical communication data, improving coordination among AUVs. Finally, we model the problem as a Partially Observable Markov Decision Process (POMDP), and design LLM-enhanced multi-auv collaborative search algorithm (LEMACS) based on multi-agent deep deterministic policy gradient (MADDPG).

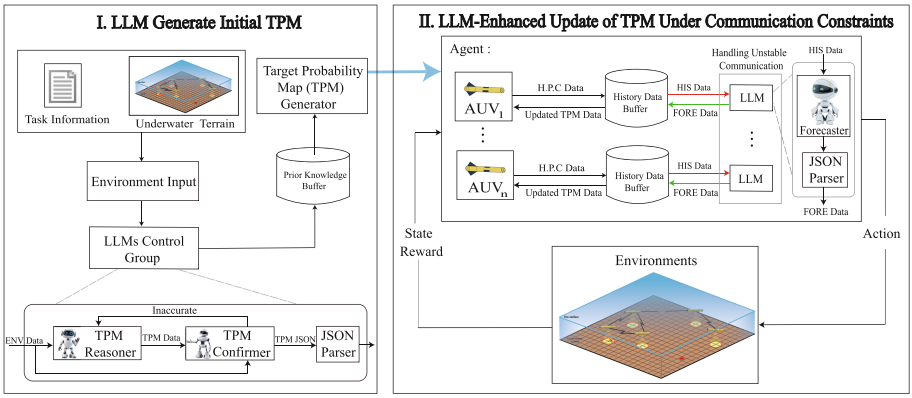


Fig. 2. The architecture of LEMACS.

4.2 Cold Start Solutions with LLM-Generated Initial TPM

In this subsection, we propose a method for generating an initial TPM using LLMs to address the cold start problem in MARL. We leverage LLMs to infer key search areas based on complex task and terrain information, and integrate them into an initial TPM. At the beginning of AUV training, this TPM is provided to the AUVs. By accessing the high-probability areas identified in the TPM, AUVs can obtain high reward values and quickly develop effective strategies during the early search stage. This method mitigates the cold start problem caused by a lack of prior knowledge in early exploration. As shown in Fig. 2, the method consists of three components: TPM Reasoner, TPM Confirmer, and TPM Generator.

– **TPM Reasoner**: As shown in Fig. 3, we encapsulate the given task information {`search_task`} and terrain information {`regions`} into an dataset {`ENV Data`}. Subsequently, it is provided to TPM Reasoner for inference, in order to obtain information about important search areas {`TPM Data`}.

$$TPM_{Data} = \begin{cases} ((X_1, X_2), (Y_1, Y_2), P_1), & \text{region } 1 \\ ((X_3, X_4), (Y_3, Y_4), P_2), & \text{region } 2 \\ \quad \vdots & \\ ((X_m, X_n), (Y_m, Y_n), P_k), & \text{region } k \end{cases}, \tag{8}$$

where (X_i, X_j) denotes the horizontal range of the region, (Y_i, Y_j) denotes the vertical range, and P_k represents the probability weight of the corresponding region.

– **TPM Confirmer**: If a single LLM is used to independently generate the entire TPM information, it may produce inaccurate or invalid outputs, and is prone to cause system crashes due to format inconsistencies. To ensure the accuracy of the inferred information, we design a TPM Confirmer to verify the correctness of the reasoning. This additional validation step by the TPM

Confirmer enhances the reliability and scientific soundness of the generated TPM information. After receiving the inferred {`TPM Data`}, the TPM Confirmer evaluates its accuracy and validity based on the task information and terrain data. If the inferred prior knowledge from the TPM Reasoner is found to be inaccurate, a feedback instruction {`inaccurate`} is returned to trigger re-reasoning. If the data is accurate, it is encapsulated into JSON format. Then, the confirmed {`TPM JSON`} is unpacked using the `json.load` function. The unpacked information is stored in the prior knowledge buffer for use by the TPM Generator.

– **TPM Generator**: In order to enable the AUV to correctly utilize the TPM, important search regions are extracted from the prior knowledge buffer. By increasing the probability values at the corresponding positions of these key regions to specified multipliers, a normalized TPM is constructed.

TPM Reasoner

You are expert in multi-AUV collaborative search. Due to the issue of cold-start problem in MARL, it can lead to slow convergence.
You should infer potential target regions based on terrain information and search task information, thereby improving the search efficiency of AUVs. The search environment is a two-dimensional plane consisting of m*n small squares. The position coordinates of AUV can be represented as (x,y), where x represents the row and y represents the small square in the column.
Terrain Information:{
Define regions with name, x-range, and y-range.
regions = [
("Reff slope", (X1,X2), (Y1,Y2)) ,
("Rocky", (X3,X4), (Y3,Y4)) ,
("Algea", (X5,X6), (Y5,Y6)) ,
("Coral", (X7,X8), (Y7,Y8)) ,
("Seagrass", (X9,X10), (Y9,Y10)) ,
.....
("Total region", (Xmin,Ymax), (Xmin,Ymax)) ,
]}.
Mission Information: {search_task}. For each matching region in the following exact format:{(X1,X2), (Y1,Y2), (P1)}.
****Rules****
-Do NOT include any text, explanation, or region names.
-P must be between 2 and 4.
-Multiple entries are allowed, one per line or space-sparated.

Fig. 3. TPM Reasoner Prompt.

The primary procedure is summarized in Algorithm 1. In this algorithm, we use $LLM_{TPMReasoner}(ENV_{Data})$ to infer the key search areas (TPM_{Data}), use $LLM_{TPMConfirmer}(TPM_{Data})$ to further confirm the inference information to prevent errors or omissions of important search area information, and finally use TPM Generator to draw the LLMs inference of important search area information into TPM ($P(t) = \{p_{mn}(t)\}$), which is provided to AUVs before training begins.

Algorithm 1. LLM-generated Initial TPM

Require: Task information $\mathcal{T}$, Terrain information $\mathcal{M}$, LLM.
Ensure: Initial Target Probability Map $P(t)$.
Initialize ENV_{Data}, TPM_{Data}, buffer $\mathcal{K}$, matrix p_{mn}.
$ENV_{Data} \leftarrow \{\mathcal{T}, \mathcal{M}\}$.
Query $LLM_{TPMReasoner}(ENV_{Data})$ to obtain candidate TPM.
$TPM_{Data} \leftarrow \{((X_1, X_2), (Y_1, Y_2), P_1), \ldots, ((X_m, X_n), (Y_m, Y_n), P_k)\}$.
if $LLM_{TPMConfirmer}(TPM_{Data})$ invalid **then**
    Return {`inaccurate`} and repeat from step 3.
else
    $TPM_{JSON} \leftarrow LLM_{TPMConfirmer}(TPM_{Data})$.
    Unpack the TPM_{JSON} and store in buffer $\mathcal{K}$.
end if
Initialize $W(t) \in \mathbb{R}^{N_x \times N_y}$ with $W(t)[m][n] \leftarrow 1$.
for each $((X_l, X_r), (Y_b, Y_t), P)$ in $\mathcal{K}$ **do**
    for $x = X_l$ to X_r **do**
        for $y = Y_b$ to Y_t **do**
            $W(t)[x][y] \leftarrow P$.
        end for
    end for
end for
for $m = 1$ to N_x **do**
    for $n = 1$ to N_y **do**
        $p_{mn} \leftarrow \dfrac{W(t)[m][n]}{\sum_{m=1}^{N_x} \sum_{n=1}^{N_y} W(t)[m][n]}$.
    end for
end for
return $P(t) = \{p_{mn}(t)\}$.

4.3 LLM-Enhanced Update of TPM Under Communication Constraints

Underwater acoustic communication instability may cause some AUVs to lose connection with others, resulting in failed information exchange and delayed updates of the TPM, thereby reducing cooperative search efficiency. As shown in Fig. 2, to address this issue, we propose a method that leverages LLMs to infer the observations of disconnected AUVs based on historical communication data (H.P.C Data: heading direction, position, channel information), thus compensating for the missing TPM updates. The method consists primarily of a component named Forecaster.

– **Forecaster**: When AUV_i exceeds the acoustic communication range with other AUVs or encounters with unstable links, communication interruptions may occur. AUV_i determines the set of disconnected AUVs at time t as

$$\bar{N}_i(t) = \{j \mid \|\mu_i(t) - \mu_j(t)\|_2 > R_a\}, \tag{9}$$

where $\bar{N}_i(t)$ denotes the set of AUVs that are beyond the acoustic range of AUV_i at time t. For each AUV $j \in \bar{N}_i(t)$, AUV_i retrieves the last received data packet from time step $t-1$ as

$$\mathcal{H}_j(t-1) = \{\mu_j(t-1),\ \mathbf{v}_j(t-1),\ \mathcal{C}_j(t-1)\}, \tag{10}$$

where

- $\mu_j(t-1) = (x_j(t-1),\ y_j(t-1))$ is the position;
- $\mathbf{v}_j(t-1) = (v_j^x(t-1),\ v_j^y(t-1))$ is the heading direction;
- $\mathcal{C}_j(t-1) = \{(m_k,\ n_k,\ p_{m_k n_k}) \mid k = 1, \ldots, M\}$ is the set of TPM observations, where each entry corresponds to an observed grid cell (m_k, n_k) and its associated probability $p_{m_k n_k}$.

Forecaster

You are expert in multi-AUV collaborative search. This swarm consists of auv: auv0, auv1,..., .When auvi loses contact with some auvs due to unstable commu ication, please use its heading direction, trajectories, and channel information to infer its next possible movement information. The goal is to reduce the decline in cooperative search efficiency caused by communication instability between auv. The search environment is a 2D plane composed of 40*40 small grids. Coordinates are represented as (x,y), where x is the row number and y is the column number. Each auv can scan a four-grid area around its current position. motion dynamics information and scanning mechanism information of auv: **{auv_motion_dynamics}**, **{auv_scanning_mechanism}**. It also includes an instance of location migration: **{heading direction-to-position transition example}**.
The know information about the disconnected auv include: {HIS Data}.
-Trajectories: The last konw position of the auv on the grid (x,y), where x and y are integers in the range [0,39].
-Heading direction: A velocity vector (vx, xy) represents speed in the X (horizontal) and Y (vertical) directions.
-Channel information: The most recent communication history_data, connsisting of previous messages exchanged with other auv. This data provides insight into the auv's recent behavior, movement patterns, and coordination.
Output format : {rules_restrictions}
###Example Output:
AUV_j:
Heading direction: [ ],
Position: [ ],
Channel information: [ ].
Only infer for the disconnected auvs based on the provided data.

Fig. 4. Forecaster Prompt.

The historical data packet {HIS Data} submitted to the LLM for inference is composed by aggregating all $\mathcal{H}_j(t-1)$ from disconnected AUVs

$$\bigcup_{j \in \bar{N}_i(t)} \mathcal{H}_j(t-1) = \left\{ \mu_j(t-1), \mathbf{v}_j(t-1), \mathcal{C}_j(t-1) \mid j \in \bar{N}_i(t) \right\}. \tag{11}$$

As shown in Fig. 4, To improve inference accuracy, the LLM is provided with essential prior knowledge, including AUV motion dynamics model, sonar sensing mechanism, examples of heading-direction-to-position transitions. The inference result for each disconnected AUV can be expressed in a unified JSON format {Fore JSON}. For example, the information prediction for the missing AUV_j is as follows.

$$\texttt{"AUV_j"} = \begin{cases} \texttt{"Heading direction"} : (v_j^x(t),\ v_j^y(t)), \\ \texttt{"Position"} : (x_j(t),\ y_j(t)), \\ \texttt{"Channel information"} : \left[(m, n, p_{mn}^{\text{LLM}}(t)),\ \ldots\right]. \end{cases} \tag{12}$$

Then, the confirmed {Fore JSON} is unpacked using the `json.load` function. The unpacked information {Fore Data} is stored in the history data buffer. Subsequently, AUV_i retrieves from the historical buffer the `"Channel information"`: $\left[(m, n, p_{mn}^{\text{LLM}}(t)), \ldots\right]$ of the disconnected AUVs at time t, as inferred by the LLM, modifies its TPM ($P(t) = \{p_{mn}(t)\}$) using the predicted data, and performs information fusion with the AUVs maintaining stable communication. This approach effectively compensates for TPM update delays caused by the loss of communication.

4.4 Dec-POMDP Formulation

Each AUV is limited by the partially observable environment and cannot perceive global information. Therefore, we formulate the problem as a POMDP, represented by the tuple $\langle S, O, A, P, R\rangle$, defined as follows.

- State Space: $\mathcal{S} = \{s_1, s_2, \ldots, s_N\}$, representing the states of N AUV_i. Each state s_i is defined as
$$s_i = \{\mu_i, v_i, g_i, c_i\}, \tag{13}$$
where μ_i, v_i, g_i and c_i denote the position, speed, target probability map value, and communication status of AUV_i, respectively.
- Observation Space: $\mathcal{O} = \{o_1, o_2, \ldots, o_N\}$ is the joint observation space of all agents, where o_i is the partial observation information obtained by the ith AUV from the global state. The information observed by the ith AUV is represented as
$$o_i = \{\mu_i, v_i, g_i, \mu_{other}, d_{other}, c_i\}, \tag{14}$$
where μ_{other} denotes the position matrix of other AUVs, $d_{other} = \mu_{other} - \mu_i$ denotes the position distance between the i th AUV and other AUVs, respectively.
- Action Space: $\mathcal{A} = \{a_1, a_2, \ldots, a_N\}$, representing the set of all joint actions of multiple AUVs.
$$a_i = \{b_i, \varphi_i\}, \tag{15}$$
where b_i and φ_i denote the acceleration and angular velocity of AUV_i, respectively.
- State Transition Probability: $\mathcal{P}$ is the probability function of state transition, which can be expressed as $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$.
- Reward: $\mathcal{R} = \{r_1, r_2, \ldots, r_N\}$ represents the joint reward of all AUVs. We design a reward function to evaluate the behavioral decisions of each AUV. The reward is defined as
$$r_i = \lambda \cdot r_{prob} + \beta \cdot r_{find} + \theta \cdot r_{col} + \delta \cdot r_{comm}, \tag{16}$$
where $\lambda, \beta, \theta, \delta \in \mathbb{R}$ represent the weight coefficients for each part of the reward.

Target probability reward r_{prob} : To encourage AUVs moving to areas with higher probabilities. The probability reward is defined as

$$r_{prob} = \frac{\sum_{grid(m,n)\in FOV} p_{mn}}{N_{FOV}}, \tag{17}$$

where N_{FOV} is the number of grid cells within the AUV detection range, p_{mn} represents the probability of target existence for each small square in the search range.

Target reward r_{find} : The core of the target search problem is to locate the target. Upon discovery, a significant reward is given to encourage search, defined as

$$r_{find} = r_{det}, \tag{18}$$

where r_{det} is a constant representing the fixed reward for successfully finding the target.

Collision penalty r_{col} : To ensure safe distances between AUVs, penalize AUV_i when AUV_i is too close to a neighboring AUV_k (where $k \in N$ and $k \neq i$) if the distance d_{ik} is less than a safety threshold d_{min}, defined as

$$r_{col} = \begin{cases} \max(0, d_{ik} - d_{min}), & \text{if } d_{ik} < d_{min} \\ 0, & otherwise \end{cases}, \tag{19}$$

when the distance between AUV_i and AUV_k is less than the safety distance d_{min}.

Communication reward r_{comm} : To enhance information exchange between AUVs, we design a communication reward to facilitate the acquisition of information among the AUVs as follows.

$$r_{comm} = 1 - \frac{d_{ik}}{R_a}, \tag{20}$$

where R_a is the distance for underwater acoustic communication, and d_{ik} is the distance between AUV_i and AUV_k.

4.5 LEMACS Algorithm

To achieve collaborative search for AUVs, we utilize MADDPG to coordinate and control AUVs in order to search for targets as quickly as possible. The LEMACS algorithm is outlined in Algorithm 2. Firstly, we use the TPM inferred by Algorithm 1 as the initial TPM ($P(t) = \{p_{mn}(t)\}$) for AUVs. The TPM serves as the prior knowledge for AUVs to overcome cold start problem, and it contains terrain information that AUVs need to focus on searching. The probability of the terrain that needs to be searched in TPM is higher than that of ordinary areas. Driven by rewards, AUVs will search important areas and then ordinary areas during the search process, thereby reducing the large number of invalid searches caused by the lack of prior knowledge in the early stage of search and

effectively solving cold start problem. Secondly, when communication between AUVs becomes unstable, we use a forecaster to estimate the states of the disconnected AUVs, and use the predicted information (`"Channel information"`: $\left[(m, n, p_{mn}^{\mathrm{LLM}}(t)), \ldots\right]$) to modify own TPM, thereby reducing the problem of delayed TPM ($P(t) = \{p_{mn}(t)\}$) updates caused by unstable communication.

During the training process, each AUV acts as an agent. The joint observations are $x = (o_1, \ldots, o_N)$, with next state x' after actions $a = (a_1, \ldots, a_N)$. Each agent interacts with the environment and stores experiences in $\mathcal{D}$. Each AUV (agent) employs four neural networks: actor μ^{ξ_i}, target actor $\mu^{\xi_i'}$, critic Q^{ω_i}, and target critic $Q^{\omega_i'}$, with parameters ξ_i, ξ_i', ω_i, and ω_i'. The actor is updated by

Algorithm 2. The LEMACS Algorithm

Initialize actor and critic networks of each AUV with random parameters ξ_i, ω_i.
Initialize target networks: $\xi_i' \leftarrow \xi_i$, $\omega_i' \leftarrow \omega_i$.
Initialize replay buffer $\mathcal{D}$ and history data buffer $\mathcal{B}$.
Obtain initial TPM $P(t)$ by Algorithm 1 with inputs $\mathcal{T}$ and $\mathcal{M}$.
Initialize TPM $\mathcal{G} \leftarrow P(t) = \{p_{mn}\}$.
for *episode* $= 1$ to E **do**
  Initialize exploration process $N(t)$ and $\mathcal{G}' \leftarrow \mathcal{G}$.
  Receive initial observation x of all AUVs.
  for $t = 1$ to T **do**
    for AUV $i = 1$ to N **do**
      if communication is interrupted **then**
        Identify disconnected set $\bar{N}_i(t) = \{j \mid \|\mu_i(t) - \mu_j(t)\|_2 > R_a\}$.
        Retrieve {HIS Data} $= \{\mathcal{H}_j(t-1) \mid j \in \bar{N}_i(t)\}$ from $\mathcal{B}$.
        $FORE_{JSON} \leftarrow LLM_{Forecaster}(HIS_{Data})$.
        Unpack the $FORE_{JSON}$ and store in buffer $\mathcal{B}$.
      else
        Cache channel data to buffer $\mathcal{B}$.
      end if
      Update $\mathcal{G}'$ with buffer $\mathcal{B}$ using Eq. (7).
    end for
    For each agent, take action $a_i = \mu^{\xi_i}(o_i) + N(t)$.
    Observe new state x', reward r.
    Store (x, a, x', r) into replay buffer $\mathcal{D}$.
    $x \leftarrow x'$.
    for AUV $i = 1$ to N **do**
      Sample mini-batch of S from $\mathcal{D}$.
      Update actor μ^{ξ_i} using Eq. (21).
      Update critic Q^{ω_i} using Eq. (22) and (23).
    end for
    Update target networks using Eq. (24).
  end for
end for

$$\nabla_{\xi_i} J(\xi_i) = \mathbb{E}_{x \sim \mathcal{D}} \left[\nabla_{\xi_i} \mu^{\xi_i}(o_i) \nabla_{a_i} Q^{\omega_i}(x, a_1, \ldots, a_n) \big|_{a_i = \mu^{\xi_i}(o_i)} \right], \tag{21}$$

where $\mathcal{D}$ is the experience replay buffer storing $(x, x', a_1, \ldots, a_N, r_1, \ldots, r_N)$. The critic is trained by minimizing

$$L(\omega_i) = \mathbb{E}_{\mathcal{D}}\left[\left(y - Q^{\omega_i}(x, a_1, \ldots, a_n)\right)^2\right], \tag{22}$$

$$y = r_i + \gamma Q^{\omega_i'}(x', a_1', \ldots, a_n')\big|_{a_j'=\mu^{\xi_i'}(o_j), j\in n}. \tag{23}$$

Target networks are updated softly as

$$\xi_i' \leftarrow \tau\xi_i + (1-\tau)\xi_i', \quad \omega_i' \leftarrow \tau\omega_i + (1-\tau)\omega_i', \tag{24}$$

where τ is the soft update coefficient.

5 Performance Evaluation

In this section, we evaluate the performance of the LLM-generated initial TPM by comparing the inference accuracy, inference time, and computational cost, in preparation for addressing the cold start problem. Then, we distribute the initial TPM to each AUV to carry out the search tasks. Finally, we validate the effectiveness of the LLM-enhanced TPM update under communication constraints and conduct experiments. To illustrate the effectiveness of the search strategy, we use the search time, search success rate and energy consumption to evaluate the search performance.

Table 1. Simulation Parameters.

Parameters	Description	Value (Unit)
R_a	acoustic communication distance between AUVs	1000 m
R_s	sonar detection distance	100 m
D_s	sonar maximum detection distance	200 m
P_D	sonar detection accuracy	0.95
P_F	sonar false alarm probability	0.05
d_{min}	Safe distance	10 m
E	Number of episodes	5000
T	Simulation step	200
γ	Discount factor	0.95
α	Learning rate	0.01
B	Batch size	1024
D	Buffer length	5e5

5.1 Environment Settings

The scenario assumes that multi-AUV search for underwater targets. In order to ensure the authenticity of regional underwater terrain data, the coral reef map dataset of Arizona State University (ASU) and the terrain data of the Dongsha Islands and other regions are used to build the marine terrain data [22]. The search area is divided into 1600 square grids in a 10 km × 10 km area, and the grid length is 250 m. Table 1 lists other simulation parameters used in the experimental setup.

5.2 Cold Start Solutions with LLM-Generated Initial TPM Performance

To evaluate the performance of different LLMs in search tasks, we compare four LLM application programming interfaces (APIs): Llama, GPT, Qwen, and DeepSeek.

Table 2 compares LLMs in generating initial TPM for AUVs across two tasks: task 1, with two terrain area (simpler), and task 2, which adds two more terrain area (more complex). In the Llama series, Llama-3.1-405B model achieved 100% and 90% accuracy for task 1 and task 2, respectively. In the GPT series, both GPT-4 and GPT-4o also reached 100% and 90% accuracy, with excellent performance and minimal processing time, but GPT-4 incurs high API costs per expert message. The cost is too high, which is not conducive to large-scale deployment. Within the Deepseek series, Deepseek r1 attained 95% and 90% accuracy, but suffers from slow inference, for search tasks that require urgent time, this will be a fatal disadvantage. Deepseek v3 achieved 100% and 90%, showing strong results. In the Qwen series, Qwen-2.5-72B and Qwen-max both reached 100% and 90% accuracy, with Qwen-max offering faster completion time. In addition, Qwen 2.5-7B and Llama-3.1-8B are locally deployed, with smaller parameters,

Table 2. Performance LLM-generated Initial TPM.

LLM	Task 1/Task 2 Accuracy	Task 1/Task 2 Average Runtime	Task 1/Task 2 API Cost
Qwen 2.5-7B	90%/45%	3.42 s/4.83 s	0.00$/0.00$
Llama-3.1-8B	70%/40%	3.57 s/4.97 s	0.00$/0.00$
Llama-3.1-405B	100%/90%	8.28 s/12.52 s	0.0982$/0.1518$
Llama-3.3-70B	95%/85%	7.71 s/11.88 s	0.0063$/0.0076$
GPT-4	100%/90%	4.92 s/7.08 s	0.5244$/0.7062$
GPT-4o	100%/90%	6.24 s/7.79 s	0.1748$/0.2402$
GPT-4o-mini	90%/85%	7.61 s/8.24 s	0.0874$/0.1321$
DeepSeek r1	95%/90%	31.52 s/45.38 s	0.0110$/0.0184$
DeepSeek v3	100%/90%	7.41 s/12.58 s	0.0054$/0.0092$
Qwen-2.5-72B	100%/90%	8.54 s/12.69 s	0.0102$/0.0158$
Qwen-QwQ-32B	90%/80%	8.33 s/14.28 s	0.0052$/0.0080$
Qwen-max	100%/90%	7.26 s/8.61 s	0.0066$/0.0110$

faster running speed, and no API call fees. The inference for simple task 1 can reach 90% and 70% respectively, but as the difficulty of task 2 increases, the performance of the small model significantly decreases.

5.3 LLM-Enhanced Update of TPM Under Communication Constraints Performance

Table 3 illustrates the performance of various LLMs as Forecaster. The Forecaster predicts other AUVs' information based on historical data, a process that may occur repeatedly during a mission. Llama-3.1-405B achieved 89% accuracy with a 6.54 s average runtime, showing good performance. In the GPT series, GPT-4 and GPT-4o reached 95.83% and 91.66% accuracy with runtimes of 4.11 s and 3.88 s, respectively. Both performed excellently, but GPT-4 costs $0.06111 per use, whereas GPT-4o is significantly cheaper at $0.01126. In addition to the lower cost, GPT-4o also responds faster than GPT-4, making it more suitable for large-scale applications. In the Deepseek series, Deepseek r1, though bidirectional, struggled with mathematical and logical inference and had long runtimes. Deepseek v3 achieved 76.67% accuracy. In the Qwen series, Qwen-QwQ-32B and Qwen-max achieved 85% and 90% accuracy, with Qwen-max offering shorter runtimes. Additionally, Qwen-2.5-7B and Llama-3.1-8B were deployed locally. Their smaller size ensured fast, cost-free runs, though with limited performance in information inference.

Table 3. Performance LLM-Enhanced Update of TPM.

LLM	Accuracy	Average Runtime	API Cost
Qwen 2.5-7B	55.00%	2.69 s	0.0000$
Llama-3.1-8B	45.00%	2.16 s	0.0000$
Llama-3.1-405B	89.00%	6.54 s	0.00225$
Llama-3.3-70B	83.33%	5.50 s	0.00044$
GPT-4	95.83%	4.11 s	0.06111$
GPT-4o	91.66%	3.88 s	0.01126$
GPT-4o-mini	83.33%	4.25 s	0.00113$
DeepSeek r1	41.66%	20.68 s	0.00136$
DeepSeek v3	76.67%	7.74 s	0.00069$
Qwen-2.5-72B	83.33%	5.13 s	0.00125$
Qwen-QwQ-32B	85.00%	6.81 s	0.00063$
Qwen-max	90.00%	4.06 s	0.00082$

5.4 LEMACS Performance

In this section, we evaluate the overall performance of the proposed algorithm using three selected LLMs to execute the task: LEMACS (GPT-4o), LEMACS

(Qwen-max), and LEMACS (Llama-3.1-405B). To verify the effectiveness of the proposed algorithm, we introduce the following two baseline reinforcement learning methods for comparison.

- MADDPG method [23]: The Actor network generates continuous actions to control the AUV, and uses random sampling methods to obtain experience tuples for training. The other parameters of the MADDPG are configured to be identical to those in our algorithm.
- HCCC method [20]: A multi-AUV asynchronous update search strategy based on high-capacity chance communication to maximize target search performance under unstable communication condition, but HCCC lacks LLM enhanced mechanism.

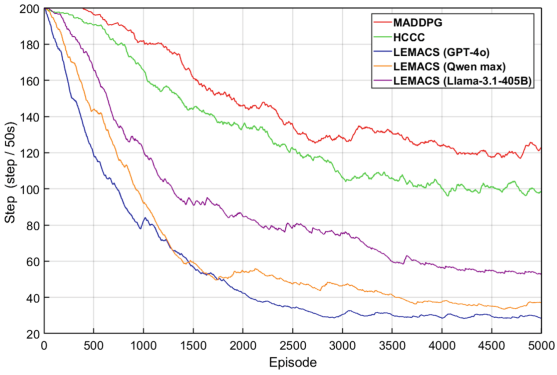


Fig. 5. Comparison of search steps.

As shown in Fig. 5, the performance of LEMACS varies due to differences in the accuracy of the LLMs. LEMACS (GPT-4o) achieves the fastest convergence and significantly improves AUV cooperative search quality. The convergence speed and search quality of LEMACS (Qwen-max) are higher than those of LEMACS (Llama-3.1-405B). In HCCC, AUVs maintain a shared TPM via limited communication. However, the lack of LLM enhancement limits prior knowledge and delays TPM updates, resulting in slower search performance. MADDPG uses neither a TPM nor LLM enhancement, leading to a lack of data sharing among AUVs and a significant drop in search efficiency.

In Fig. 6, we compare the energy consumption of 3 AUVs under different algorithms. During the first 1,000 episodes, the LEMACS (GPT-4o) algorithm outperforms the others in terms of search efficiency due to the initial TPM generated based on task description information before training. This prior knowledge enhancement improves the AUVsn' exploration capabilities, leading to superior performance in the 100th, 500th, and 1,000th episodes. After 2,000 episodes, all algorithms gradually converge. In terms of final converged energy consumption,

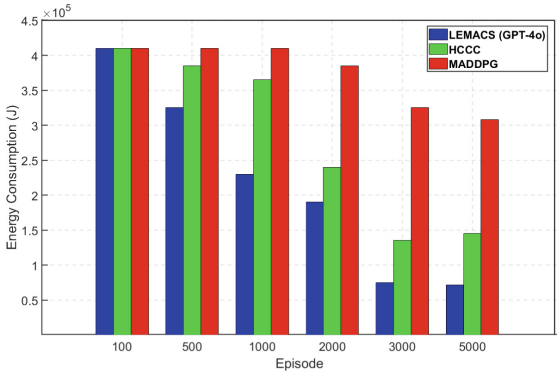


Fig. 6. Comparison of AUV energy consumption of different algorithms.

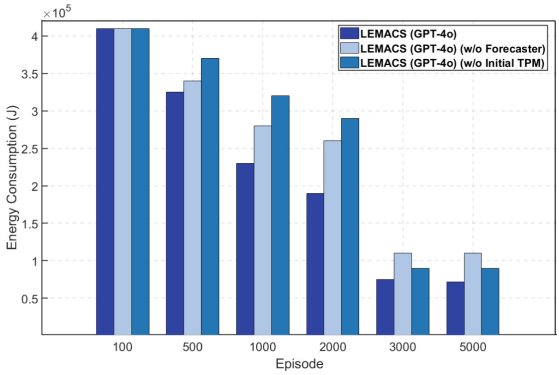


Fig. 7. Comparison of AUV energy consumption in ablation experiments.

the LEMACS (GPT-4o) algorithm requires the least amount of energy, followed by the HCCC algorithm, and finally the MADDPG algorithm.

Furthermore, as shown in Fig. 7, we conducted ablation experiments on the algorithm. It can be observed that the TPM generated by LLM reduces the energy consumption of AUVs in the early stages of training, effectively solving the problem of poor early search efficiency caused by cold start problem in AUVs. In addition, we observed that the Forecaster effectively alleviated the problem of delayed TPM updates, reduced redundant search, and improved the collaborative search efficiency among AUVs. Ultimately, the LEMACS (GPT-4o) algorithm achieves the lowest energy consumption.

As shown in Fig. 8, we analyze the impact of varying the number of AUVs on the search success rate. The results indicate that the success rate improves across all algorithms as the number of AUVs increases. Specifically, as the number of AUVs is increased from 3 to 6, the LEMACS (GPT-4o) algorithm achieved a 10% improvement. From 6 to 9 AUVs, the algorithm achieved an improve-

ment of approximately 4%. This suggests that with more AUVs, the stability of communication is alleviated, reducing the relative contribution of the LLM-enhanced TPM. In contrast, the HCCC algorithm benefits more significantly from improved inter-AUV communication, reaching a success rate of 85% with 9 AUVs. Meanwhile, MADDPG algorithm also shows a positive trend as the number of AUVs increases. However, due to the absence of information sharing and integration, the improvement is minimal.

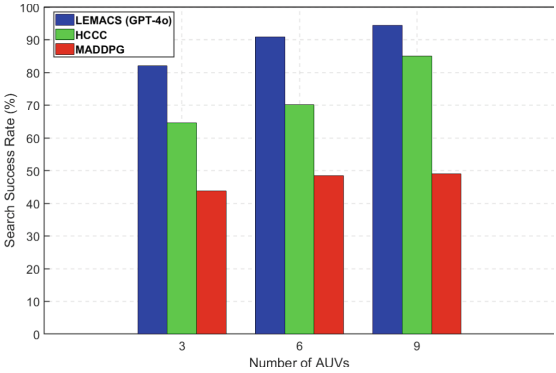


Fig. 8. Comparison of AUV search success rate of different algorithms.

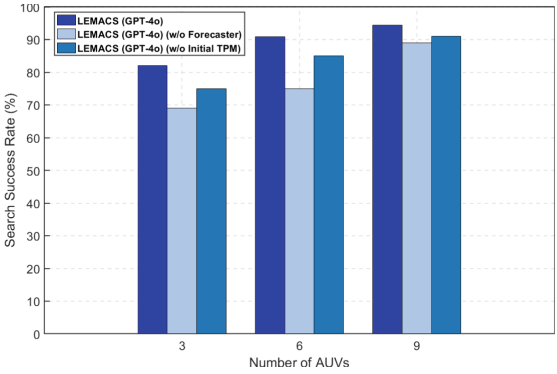


Fig. 9. Comparison of AUV search success rate in ablation experiments.

As shown in Fig. 9, we conducted ablation experiments on the algorithm. It can be observed that as the number of AUVs increases, the search success rate of all algorithms improves. However, the LEMACS (GPT-4o) algorithm achieves the best performance, indicating that both modules play a crucial role in enhancing the cooperative search capabilities of the AUVs.

6 Conclusion

In this paper, we proposed an LLM-enhanced multi-AUV cooperative search scheme. Firstly, we model the updating process of environmental information and TPM. Subsequently, we solve the cold start problem by using the initial TPM inferred by the LLM as prior knowledge. Furthermore, under communication constraints, we improve the timeliness of TPM updates by predicting the state information of AUVs with disrupted communication. Finally, we formulate the cooperative search task as a Dec-POMDP and proposed the LEMACS algorithm, which enhances the efficiency of multi-AUV cooperative search. In future work, we plan to evaluate the proposed scheme on real-world AUV hardware platforms, and improve the performance further of the proposed scheme.

Acknowledgment. This work was support in part by the National Natural Science Foundation of China under Grant U24A20215, in part by the project ZR2024MF024 supported by Shandong Provincial Natural Science Foundation, in part by the National Natural Science Foundation of China under Grant 62072287.

References

1. Luvisutto, A., Celani, A., Renda, F., Stefanini, C., De Masi, G.: Enhancing collaboration in uncertain environment: multi-agent reinforcement learning for underwater monitoring. Expert Syst. Appl. **277**, 127256–127267 (2025)
2. Wang, S., Niu, N., Wang, Z., Lv, Y., Zhang, J.: Heterogeneous multi-agent task planning method in complex marine environment. IEEE Access **13**, 84202–84216 (2025)
3. Luo, H., Wu, K., Ruby, R., Hong, F., Guo, Z., Ni, L.M.: Simulation and experimentation platforms for underwater acoustic sensor networks: advancements and challenges. ACM Comput. Surv. (CSUR) **50**(2), 1–44 (2017)
4. Luo, H., Wu, K., Guo, Z., Gu, L., Ni, L.M.: Ship detection with wireless sensor networks. IEEE Trans. Parallel Distrib. Syst. **23**(7), 1336–1343 (2011)
5. Li, Z., Zhang, R., Wang, Z., Xie, Z., Song, Y.: LLM-guided decision-making toolkit for multi-agent reinforcement learning. Neurocomputing **638**, 130105–130119 (2025)
6. Foerster, J., Farquhar, G., Afouras, T., Nardelli, N., Whiteson, S.: Counterfactual multi-agent policy gradients. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 32, pp. 1–11 (2018)
7. Agashe, S., Fan, Y., Wang, X.E.: Evaluating multi-agent coordination abilities in large language models. In: International Conference on Learning Representations, pp. 1–18 (2023)
8. Brown, T., et al.: Language models are few-shot learners. In: Advances in Neural Information Processing Systems, vol. 33, pp. 1877–1901 (2020)
9. Cai, C., Chen, J., Ayub, M.S., Liu, F.: A task allocation method for multi-AUV search and rescue with possible target area. J. Mar. Sci. Eng. **11**, 804–820 (2023)
10. Gricius, G., et al.: Advanced approach of multiagent based buoy communication. Sci. World J. **2015**(1), 569841–569847 (2015)

11. Zhu, Z., Lichuan Zhang, L., Liu, D.W., Bai, S., Ren, R., Geng, W.: An efficient multi-AUV cooperative navigation method based on hierarchical reinforcement learning. J. Mar. Sci. Eng. **11**(10), 1863–1884 (2023)
12. Wu, Y., Lang, L., Shi, H., Zhang, M.: Reducing the impact of communication delay on multi-AUVs formation control by a predictive compensation method. In: 2023 China Automation Congress (CAC), pp. 5849–5854 (2023)
13. Longyue, F., Wang, J., Luo, H.: Multi-agent reinforcement learning for cooperative search under aperiodically intermittent communication. Expert Syst. Appl. **280**, 127526 (2025)
14. Wang, J., Lele, X., Sun, C.: Learning general multi-agent decision model through multi-task pre-training. Neurocomputing **627**, 1–11 (2025)
15. Fang, Z., Chen, T., Shen, T., Jiang, D., Zhang, Z., Li, G.: Multi-agent generative adversarial interactive self-imitation learning for AUV formation control and obstacle avoidance. IEEE Robot. Autom. Lett. 1–8 (2025)
16. Liu, D., et al.: Human attention guided multiagent hierarchical reinforcement learning for heterogeneous agents. Knowl.-Based Syst. **316**, 113377–113386 (2025)
17. Jiang, B., Jun, D., Jiang, C., Han, Z., Debbah, M.: Underwater searching and multiround data collection via AUV swarms: an energy-efficient AoI-aware MAPPO approach. IEEE Internet Things J. **11**(7), 12768–12782 (2023)
18. Cai, C., Chen, J., Yan, Q., Liu, F.: A multi-robot coverage path planning method for maritime search and rescue using multiple AUVs. Remote Sens. **15**(1), 93–115 (2022)
19. Liu, Y., et al.: Multi-AUVs cooperative target search based on autonomous cooperative search learning algorithm. J. Mar. Sci. Eng. **8**(11), 843–866 (2020)
20. Jia, Q., et al.: Multi-AUV cooperative search method based on high-capacity chance communication. IEEE Sens. J. **25**, 1–12 (2024)
21. Jia, Q., Hongli, X., Feng, X., Haitao, G., Gao, L.: Research on cooperative area search of multiple underwater robots based on the prediction of initial target information. Ocean Eng. **172**, 660–670 (2019)
22. Asner, G.P., et al.: Large-scale mapping of live corals to guide reef conservation. Proc. Natl. Acad. Sci. **117**(52), 33711–33718 (2020)
23. Yang, Z., Du, J., Xia, Z., Jiang, C., Benslimane, A., Ren, Y.: Secure and cooperative target tracking via AUV swarm: a reinforcement learning approach. In: 2021 IEEE Global Communications Conference (GLOBECOM), pp. 1–6 (2021)

CSI-Based Human Activity Recognition Using Spatial-Temporal Features and Attention

Wenping Yu[1](✉), Jianhao Wang[1], Yusong Dong[1], Zhewen Li[1], and Jianzhong Zhang[2]

[1] Tianjin University of Science and Technology, Artificial Intelligence Institute of Tianjin University of Science and Technology, Tianjin 300457, China
yuwenping@tust.edu.cn

[2] Nankai University, Computer Science Institute of Nankai University, Tianjin 300350, China

Abstract. In this paper, we propose a CLSA human activity recognition (HAR) method based on channel state information (CSI) to address the challenges of environmental noise and feature overfitting. The method adopts a hybrid deep learning architecture that combines a Convolutional Neural Network (CNN), a Long Short-Term Memory (LSTM) network and an attention mechanism. Raw CSI data are first denoised using discrete wavelet transform (DWT) to preserve activity-related information while suppressing random noise. The preprocessed data are then passed through the CNN layer to extract spatial features using a two-layer Conv1D structure. These spatial features are further processed by a two-layer stacked LSTM network to model temporal patterns. The output of the final LSTM layer is then fed into the attention layer, where weights are dynamically assigned to emphasize the contextual relevance of key activity segments. Finally, the classification layer is used to categorize the activities. The experimental results show that the CSI-based human body recognition method proposed in this paper outperforms models such as LSTM and CNN and some recognition methods on real datasets. The average recognition accuracies of up to 90.09% and 89.64% are achieved in two different environments, respectively, providing an effective method for activity recognition.

Keywords: Wireless sensing · Human activity recognition · Channel state information · Neural network

1 Introduction

Human Activity Recognition (HAR) [1–3] has been a prominent area of study for more than ten years. Conventional HAR techniques are usually categorized into two main groups: vision-based and sensor-based approaches. Vision-based methods [4, 5] rely on cameras or static images to interpret human motion through visual processing. Although they can yield accurate results in controlled conditions, their effectiveness often diminishes with changes in lighting or camera placement, which restricts their real-world adaptability. On the other hand, sensor-based techniques [6] require individuals to wear or carry specialized equipment. While these methods tend to offer more consistent results in various environments, they can be expensive and intrusive, potentially leading to user discomfort and limited long-term usability.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 236–253, 2026.
https://doi.org/10.1007/978-981-95-8405-5_13

In response to these limitations, researchers have increasingly turned to Channel State Information (CSI) as an alternative solution for activity recognition. CSI, which reflects the way Wi-Fi signals are influenced by human movement due to multipath effects, captures fine-grained changes in signal transmission at the physical layer. This enables systems to detect and classify human activities by observing how signals fluctuate in each environment [7]. CSI-based HAR methods bring several benefits over traditional approaches. They remain effective in poorly lit or visually obstructed areas since they do not depend on image clarity or direct line-of-sight. These methods can also utilize existing Wi-Fi networks, which help reduce installation costs and eliminate the need for extra hardware. Furthermore, because they do not require physical contact with the user, CSI-based systems offer a non-intrusive, privacy-conscious alternative to wearable technologies.

The rapid development of machine learning and deep learning has significantly boosted their application in CSI-based human activity recognition. Because distinct human movements lead to unique variations in CSI signals, many researchers have applied different learning algorithms to identify these patterns and improve recognition performance [8]. Yet, even with these advancements, CSI-based HAR continues to encounter several important challenges. Current models often overfit to environmental features in the source domain, resulting in poor generalization to the target domain. To address these issues, prior studies have employed Convolutional Neural Network (CNN) or Long Short-Term Memory (LSTM) network to extract spatial or temporal features but usually use them alone or combine them without addressing the interference of noise with key activity segments in CSI data. In contrast, this paper proposes the CLSA model, which integrates CNN, LSTM network and attention modules for CSI-based HAR. The attention mechanism dynamically assigns weights to emphasize the most relevant activities segments while mitigating environmental interference. As mentioned earlier, CSI data are often affected by multipath effects and environmental dynamics, which can introduce irrelevant or noisy fluctuations into the data. The attention mechanism excels in such scenario by focusing on the most informative parts of the signal — such as subtle changes in amplitude or phase that are closely related to human activities — while downplaying irrelevant or noisy components. This selective focus enhances the model's ability to capture discriminative features for accurate activity recognition.

To conclude, this work introduces the CLSA framework—an approach to HAR that integrates CNN, LSTM, and attention mechanisms. The system begins by feeding preprocessed CSI data into a two-layer Conv1D structure to capture spatial features. Next, the extracted features are fed into a two-layer stacked LSTM network to model temporal patterns, in the activity sequences. An attention mechanism is then applied, where weights are computed using a Tanh activation function and normalized with Softmax to generate a context vector that aids in classification. Experiments on a real-world dataset show that this method achieves a recognition accuracy of 90.1%.

2 Related Work

Research in activity recognition has evolved along two primary paths: traditional machine learning methods and modern deep learning approaches. Classic few-shot learning techniques often employ well-known classifiers like decision trees, random

forests, support vector machines (SVMs), and k-nearest neighbors (KNNs) [9]. These approaches depend on labeled data to classify both training and test samples into predefined categories. For instance, Venkatnarayan et al. [10] introduced WiMU, a system for recognizing user activities by matching incoming CSI signals to a reference library using a traversal matching algorithm. In another study, Yang et al. [11] tackled the high computational cost of artificial neural networks (ANNs) by proposing a transformation algorithm within the Spiking Neural Network (SNN) framework. Their method converts trained ANN neurons into spike sequences, allowing SNNs to operate more efficiently without sacrificing accuracy.

With the rise of deep learning, this technology has gradually developed in various fields. In contrast to traditional machine learning, which often depends on manually crafted features, deep learning models automatically learn useful representations from data by training on large, labeled datasets and refining their parameters through backpropagation [12]. This shift has removed reliance on handcrafted descriptors, enabling models to build layered, abstract features that better capture the complexity of input data [13]. Leveraging these strengths, researchers have made significant progress in advancing activity recognition technologies. WiSign [14], for instance, leverages Power Spectral Density (PSD) to segment CSI data autonomously, deriving amplitude and phase features to train a five-layer Deep Belief Network (DBN). In another approach, HandFi [15], developed by Sruthi et al., processes denoised CSI signals through time-frequency transformations, extracts spatial features via CNN and classifies outputs using support vector machines.

While modern deep learning architectures have enhanced recognition accuracy, robustness, and generalization, persistent challenges remain in CSI-based systems. Data quality inconsistencies and environmental variability during collection continue to hinder performance, highlighting the need for refined preprocessing methods and innovative model designs to bridge existing gaps.

3 Method

The CLSA HAR method, illustrated in Fig. 1, follows a three-stage process: data collection, data preprocessing and activity recognition. In the initial stage, wireless devices are used to collect CSI embedded in Wi-Fi signals. Next, during preprocessing, the raw CSI data is processed using the Discrete Wavelet Transform (DWT), which helps reduce random noise. In the final stage, activity recognition is performed using the CLSA model. This model integrates a two-layer Conv1D structure to extract spatial features, a stacked two-layer LSTM network to model temporal patterns, and an attention mechanism to focus on the key informative segments of the activity data.

3.1 Data Collection

CSI is a detailed metric used in wireless communication to describe how signals behave as they travel through the environment. It reflects the impact of physical phenomena like diffraction, reflection and scattering on the signal's transmission. In most cases, CSI data are collected using an Intel 5300 wireless network interface card, which can extract

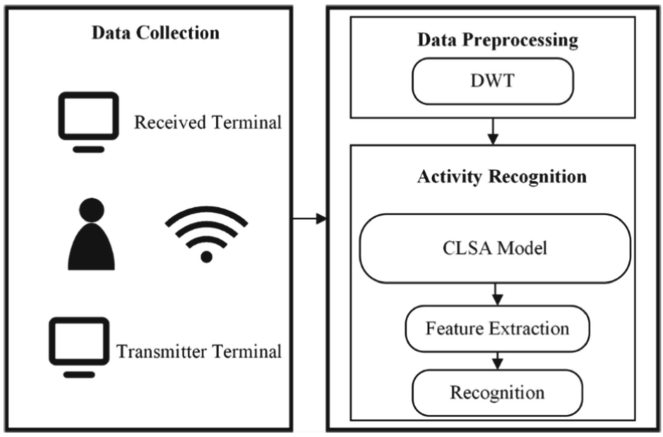


Fig. 1. Overall recognition process.

information from 30 subcarriers for each communication link. These subcarriers carry multiple packets, with each packet represented by a set of complex values such as shown below.

$$\mathrm{A(f)} = |\mathrm{A(f)}|^{\angle \mathrm{A(f)}} \tag{1}$$

In the above equation, f represents the signal frequency, $|A(f)|$ represents the amplitude loudness and $\angle A(f)$ represents the phase. Based on Eq. (1), the CSI in Orthogonal Frequency Division Multiplexing (OFDM) system can be expressed as:

$$\mathrm{Y(f)} = \mathrm{A(f)} \cdot \mathrm{X(f)} + \mathrm{N(f)} \tag{2}$$

In the above equation, f represents the signal frequency, $\mathrm{Y}(f)$ represents the received signal and $\mathrm{X}(f)$ represents the transmitted signal and $\mathrm{N}(f)$ represents the random noise signal in the environment.

In MIMO systems, CSI is extended to a matrix form. The channel matrix for M × N antennas is:

$$\mathrm{A} = [\mathrm{h1}, \mathrm{h2}, \mathrm{h3}, \cdots, \mathrm{h30}]_{\mathrm{M} \times \mathrm{N}} \tag{3}$$

In the above equation, M and N are the number of antennas used at the transmitter and receiver terminal, respectively, with a total of 30 subcarriers per antenna pair.

3.2 Data Preprocessing

In this paper, the raw CSI data will be preprocessed by DWT to suppress the inherent random noise components. As a time-frequency analysis technique, the DWT decomposes the signal into local wavelet basis functions to achieve characterization across time and frequency domains. The technique uses window operations to divide the signal into different frequency bands. The transformation contains scaling and translation

parameters that allow the window resolution and position to be adjusted to synchronize the time-frequency for subsequent processing, as shown in equation.

$$W_f(a, b) = 1/\sqrt{|a|} \int f(t) \cdot \Psi^*((t - b)/a)\, dt \tag{4}$$

During DWT processing, the raw CSI data begins with a multistage decomposition by orthogonal mirror filtering. Specifically, the low-pass filter extracts approximate coefficients (representing signal features, referred to as low-frequency information), whereas the high-pass filter separates detail coefficients (corresponding to high-frequency noise components, termed detail components). DWT is mainly accomplished by decomposition. The input CSI data is decomposed by the first step to get the low-frequency information 1 and detail component 1. The obtained low-frequency information decomposed by the first step is again subjected to the discrete transform operation to get the low-frequency information 2 and detail component 2. This process is repeated iteratively until the n-level decomposition stage, ultimately producing n levels of hierarchical low-frequency information and corresponding detail components. The denoising process leverages the inherent property that noise is present in the detail coefficients of the wavelet decomposition. By selectively eliminating these detail components at each decomposition level through thresholding operations, the raw CSI data is effectively purified while preserving its essential characteristics.

Fig. 2 illustrates a comparison of the time-varying characteristics of CSI amplitude before and after preprocessing using a two-axis plot. Before denoising, the CSI data (left) displays substantial high-frequency noise, leading to irregular amplitude variations with minimal recognizable structure. After applying DWT (right), the noise is significantly suppressed, and the amplitude profile becomes smoother while preserving activity-relevant features. This contrast highlights DWT's effectiveness in removing random noise while preserving the integrity of activity-relevant information, thereby providing a reliable foundation for subsequent feature extraction.

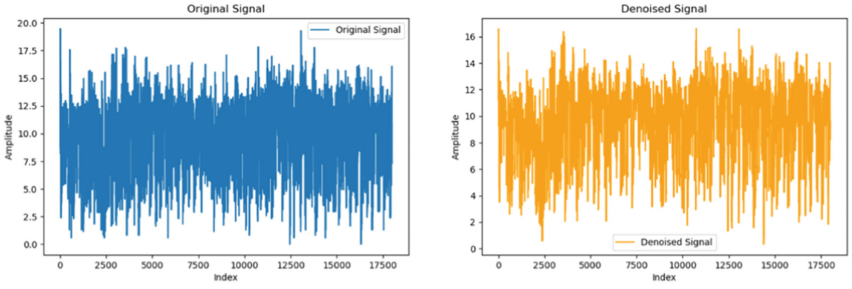


Fig. 2. Comparison of CSI before and after preprocessing.

3.3 Activity Recognition

Fig. 3 illustrates the CLSA HAR process, including spatial and temporal feature extraction, attention weighting, and final classification. The diagram highlights the sequential

architecture comprising the CNN layer (with Conv1D and pooling operations), the LSTM layer (a two-layer stacked structure), and the attention mechanism, demonstrating the hierarchical data flow from input to final classification. It particularly emphasizes the attention layer's function in re-weighting key temporal features to enhance classification performance.

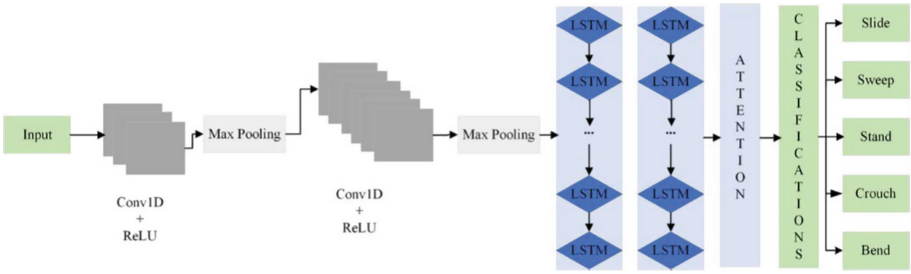


Fig. 3. Recognition process of CLSA model.

CSI Spatial Feature Extraction. CNN [16, 17] are effective for dimensionality reduction, enabling compression of spatial feature maps to reduce parameters and computational complexity while retaining salient features. Therefore, a CNN module is employed to extract spatial features from preprocessed CSI data.

Specifically, the CNN layer takes the preprocessed CSI data as input and applies convolutional and pooling operations. As shown in Fig. 4, a two-layer one-dimensional convolutional (Conv1D) structure is adopted due to its efficacy in modeling time-series data such as CSI. The first Conv1D layer employs 64 filters with a kernel size of 5 and a stride of 1. The second layer expands to 128 filters with a kernel size of 3 and a stride of 2, capturing fine-grained spatial correlations among subcarriers by progressively expanding the receptive field. A ReLU activation function is applied after each convolution to enhance non-linear feature representation. Same padding is applied to preserve the input length. Max-pooling layers follow each convolutional layer, using a pooling size of 2 and stride of 2, enabling gradual dimensionality reduction and improving translation invariance. To further mitigate overfitting caused by environmental noise, a Dropout layer with a rate of 0.3 is applied. The final output from the CNN layer is a compressed spatial feature vector passed to the LSTM layer for temporal modeling.

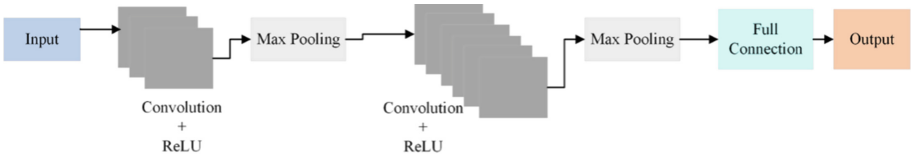


Fig. 4. CNN network structure.

CSI Temporal Feature Extraction. The LSTM network [18], a specialized variant of Recurrent Neural Networks (RNNs), remains particularly effective for modeling sequential or temporal data. It is specifically designed to mitigate gradient vanishing and explosion issues that occur in traditional RNNs during the processing of long sequences. LSTM network incorporates a memory cell architecture regulated by three gating mechanisms—input, forget, and output gate—as illustrated in Fig. 5. This architecture allows the network to retain important information over long sequences while discarding irrelevant inputs, thereby enabling effective modeling of long-term dependencies. Consequently, an LSTM network is employed to extract temporal features from the CSI data.

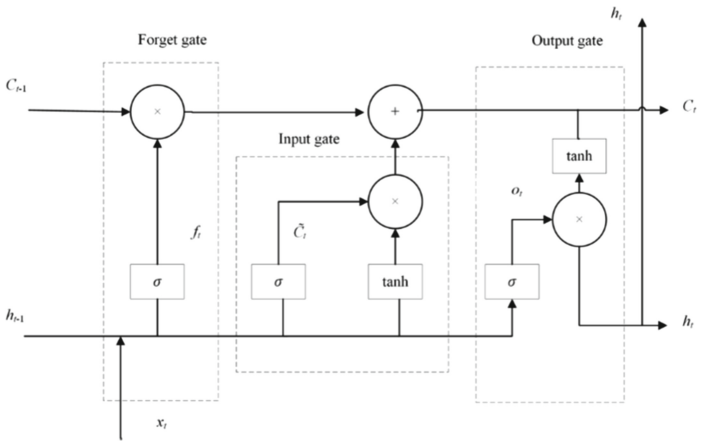


Fig. 5. LSTM basic units.

In the LSTM layer, the spatial features compressed by the CNN module are segmented by time steps and expanded into 128-dimensional vectors. A two-layer stacked LSTM with 128 hidden units in each layer is employed to extract temporal features. At each step, the LSTM cell receives the current input vector and the previous hidden state, and performs gating operations using forget, input, and output gate to update the internal cell state and produce the temporal features.

The forget gate determines which information to discard from the previous cell state by examining both the current input and the hidden state from the previous time step.

$$f_t = \sigma\left(W_f \cdot \left[h_{t-1}, x_t\right] + b_f\right) \tag{5}$$

The input gate decides what new information will be updated into the cell state. It similarly makes decisions by looking at the current input and the previous hidden state.

$$i_t = \sigma\left(W_i \cdot \left[h_{t-1}, x_t\right] + b_i\right) \tag{6}$$

$$\tilde{C_t} = \tanh\left(W_C \cdot \left[h_{t-1}, x_t\right] + b_c\right) \tag{7}$$

The cell state constitutes the core memory mechanism in LSTM structure, functioning as persistent memory storage for retaining long-range temporal features. This memory vector undergoes systematic modification through coordinated operations involving both forget and input gate. Specifically, the forget gate selectively attenuates historical information patterns, while the input gate counterpart regulates the integration of novel feature representations into the memory repository.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \widetilde{C_t} \tag{8}$$

The output gate decides the next hidden state by considering both the current input and the previous hidden state, producing an output that is influenced by the cell state.

$$o_t = \sigma\left(W_o \cdot \left[h_{t-1}, x_t\right] + b_o\right) \tag{9}$$

$$h_t = o_t \cdot \tanh\left(C_t\right) \tag{10}$$

After the above steps, the final output features are passed to the next LSTM layer (the input dimension of the second layer is 128, and the output dimension remains 128) to capture higher-level temporal features. Finally, the output of the last layer in the LSTM layer is sent to the attention layer for further processing as the total output of the entire LSTM layer.

Attention Mechanism. The attention mechanism computes the relevance between time steps in the input sequence. By assigning higher weights to informative segments that contribute significantly to classification, it enhances the model's focus on key activity segment while suppressing irrelevant environmental noise.

In the attention layer, the output of the LSTM layer is used as input to compute attention scores. These scores are calculated using a Tanh activation applied to a linear transformation involving a learnable weight matrix and bias term. The scores are then normalized using the Softmax function and used to compute a weighted sum of the LSTM layer outputs, resulting in context data. This data emphasizes key activity phases while retaining temporal structure, thereby enhancing the overall discriminative capacity of the model.

Classifications. The attention-weighted features are passed into the classification layer for final activity prediction. This layer consists of a two-layer fully connected network, where the first layer compresses the feature representation, and the second layer projects the compressed features into the activity label space. Finally, a Softmax classification function is applied to generate probability distribution across the five activity classes.

4 Experiments

4.1 Experimental Equipment

CSI data is collected using an Intel 5300 wireless network interface card (NIC) in conjunction with the open-source CSI Tool. The Intel 5300 NIC allows flexible configuration of antenna states and related parameters via command-line interface. The CSI

Tool facilitates the extraction of CSI from Wi-Fi signals. Experiments are conducted on a 2.4 GHz Wi-Fi network with a channel bandwidth of 20 MHz, using a TP-Link router with three antennas as the transmitter, and an Ubuntu 14.04 desktop equipped with a three-antenna NIC as the receiver. CSI collection is achieved by connecting both transmitter and receiver to the same local Wi-Fi network.

4.2 Experimental Procedure

During the setup phase of the experimental environment, the signal transmitter (commercial wireless router) and receiver (computer device equipped with Intel 5300 wireless card) are configured according to the requirements. However, one thing to keep in mind is that the router is required to create a wireless network without a password because the firmware of the CSI Tool does not have enough code space for storing beamforming software paths and encryption software paths. After checking for errors, the collection can begin. The data collection process requires the collector stand between the transmitter and receiver with the CSI Tool running and perform the activities in a prearranged sequence. The collection arrangements relating to collected activity and personnel are shown in Table 1 and 2. Table 1 lists the information about the participants involved in the experiment. The study involves five volunteers, whose height and weight are recorded. Table 2 outlines the predefined human activities and their corresponding definitions for recognition tasks. Based on nature and duration of activity, five common activity categories are defined: slide, sweep, stand, crouch and bend. Each activity lasted 5 s and 60 samples were collected per subject. The same process was used to complete the collection task for the remaining activities, then the collected files were named according to the prescribed format. Finally, all data files were organized and categorized.

Table 1. Collected activity.

ID	Activity	Description
1	Slide	In the experimental area, remain stationary for 1–2 s before initiating the movement. Raise your arms to the sides of your chest at shoulder height, then perform a horizontal sliding motion.
2	Sweep	In the experimental area, maintain stationary posture for 1–2 s. With elbows flexed at 90°, position both arms parallel to the floor at mid-chest height, then execute controlled lateral oscillations. from your original position.
3	Stand	In the experimental area, assume a standard squat position and hold it for 1–2 s. Stand up at an even pace and remain stand position.
4	Crouch	In the experimental area, assume a standard standing position and hold it for 1–2 s. Squat at an even pace and maintain the squat position.
5	Bend	In the experimental area, maintain an upright position for 1–2 s. Straighten your arms forward and make a bending motion.

Table 2. Collecting basic information about personnel.

ID	Height	Body Weight
1	185 cm	Peak
2	185 cm	Normal
3	180 cm	Plump
4	180 cm	Normal
5	175 cm	Normal

4.3 Evaluation Indicators

In practical applications, choosing the right evaluation metrics is essential for assessing and optimizing recognition. For the experiments in this study, the evaluation metrics included accuracy, precision, recall, and F1 score. Accuracy is the primary metric for measuring overall correctness, reflecting the proportion of correctly identified activities. Precision indicates accuracy by calculating the ratio of correctly recognized activities to all activities predicted as positive. Recall evaluates the ability to identify the actual target activities by measuring how many were correctly recognized. F1 score, which is the harmonic means of precision and recall, offers a balanced evaluation of the overall performance.

4.4 Experiments with Different Scenarios

Experiments were conducted in two different scenarios to evaluate the robustness of the CLSA model in complex environments. Fig. 6 presents the confusion matrices for the CLSA model under these two settings. In each matrix, rows represent the true activity classes, while columns indicate the predicted classes as recognized by the model. In the office environment, the recognition accuracy for most activities is relatively high. Specifically, slide (91.95%), sweep (91.21%), and crouch (91.11%) each achieve over 90% accuracy. The accuracy for stand is slightly lower at 87.5%, with common misclassifications such as crouch (4.17%) or sweep (4.17%). In the classroom environment, the recognition accuracy for slide (94.25%) and crouch (92.22%) further improves, while the performance on sweep (89.01%) and stand (85.42%) slightly declines compared to the office setting. This performance degradation may be attributed to more complex background interference present in the classroom. Notably, stand exhibits a higher misclassification rate, with a 6.25% chance of being misidentified as bend. The

accuracy for bend also slightly decreases to 87.5%. Overall, the proposed CLSA model achieves robust recognition across different environments, attaining average recognition accuracies of 90.09% in the office and 89.64% in the classroom.

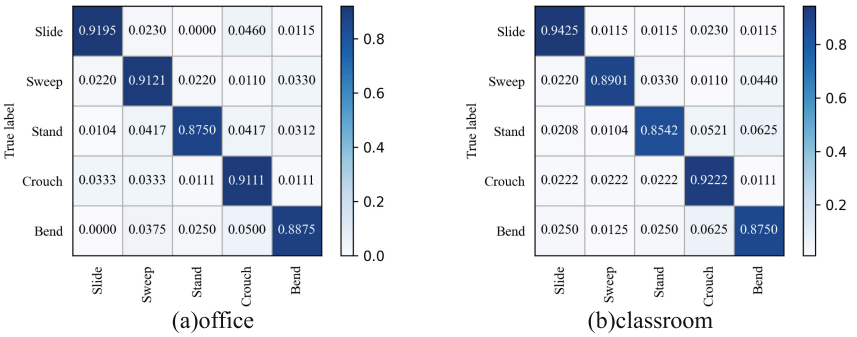
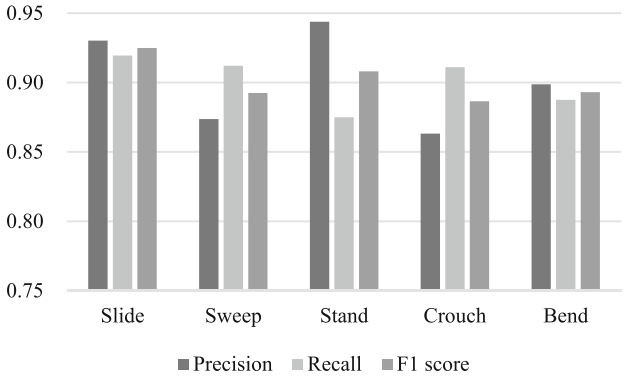
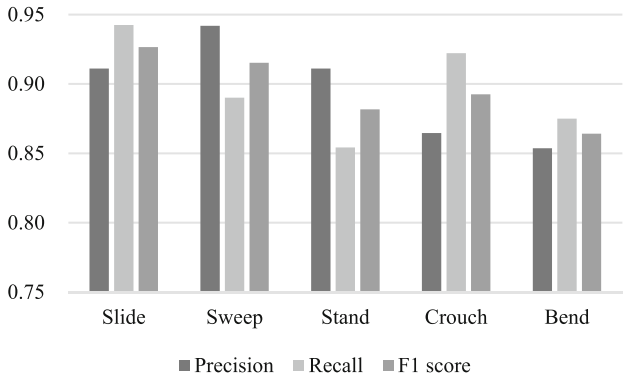


Fig. 6. Confusion matrix of different activities in two scenarios.

In addition, to evaluate the comprehensive performance of the model, the precision, recall, and F1 score for each activity were also compared. Fig. 7 compares the precision, recall, and F1 score for each activity in the two scenarios. The stand activity in the office scenario had the highest precision rate of 94.38%, while the crouch activity only had 91.11% recall rate due to the smaller range of motion. The F1 score of the sweep activity in the classroom scenario is 91.53%, which is significantly better than the other activity, indicating that the model is more accurate in capturing the temporal features of high-frequency activity. However, the bend activity may be affected by environmental interference and its performance is relatively weak. Through the above experiments, it is verified that scenario differences will have a certain impact on model performance.



(a) office



(b)classroom

Fig. 7. Evaluation indicators for different activities in two scenarios.

4.5 Experiments with Different Models

Fig. 8 compares the recognition effects of the CLSA model and the other six models (LSTM network, CNN, SVM, RNN, KNN, GRU) on five types of activities through confusion matrices. The six models achieved average accuracy of 86.8%, 85.65%, 82.47%, 82.66%, 86.49% and 88.09% respectively. GRU has the best overall performance. Its accuracy rates of slide (87.36%), sweep (89.01%), stand (88.54%), crouch (88.89%) and bend (86.42%) exceed 85% and the misclassification rate is generally low, especially in sweep and stand activity. KNN and LSTM network are second. The accuracy of bend activity in KNN is 0.85, but the probability of bend activity being misclassified as crouch activity is slightly higher (5%); while SVM and RNN perform poorly. SVM has the lowest accuracy rates for slide and bend activity, and RNN has significant misclassification of sweep activity. CNN is balanced overall, but bend accuracy is slightly lower. In contrast, the CLSA model that combines spatiotemporal features and attention weighting

has the highest recognition accuracy among similar models, verifying the effectiveness of its hierarchical feature fusion mechanism.

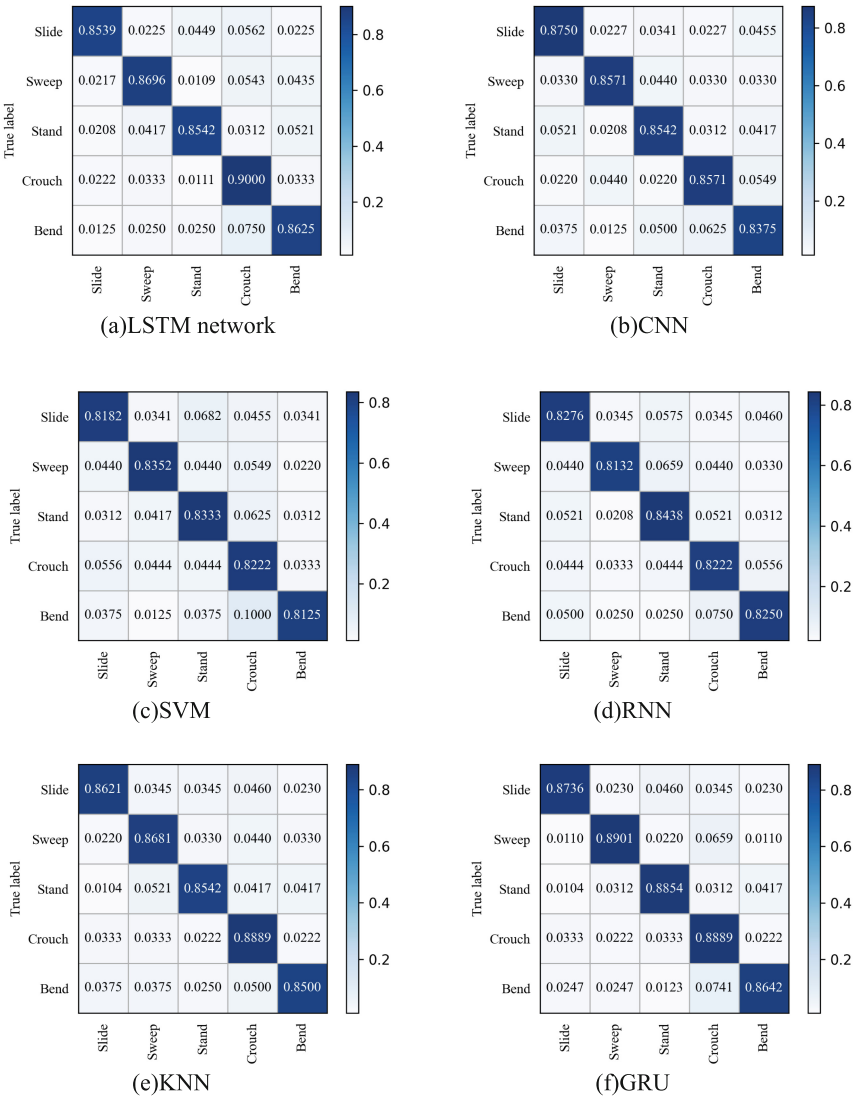


Fig. 8. Confusion matrix under different modeling experiments.

As shown in Fig. 9, 10, and 11, the system compares the performance of CLSA model with six baseline models (LSTM network, CNN, SVM, RNN, KNN, GRU) in terms of precision, recall, and F1 score. CLSA model achieved an F1 score of more than 88% in all five activities in the office scenario, significantly higher than GRU and LSTM network, thanks to the enhancement of key time step features by the attention

mechanism, while SVM had the lowest recall rate in the bend activity due to its reliance on manually extracted features. In the classroom scenario, CLSA model achieved a slightly lower F1 score for the five activities, all above 86%, while still outperforming CNN. This result highlights the effectiveness of joint spatiotemporal modeling through the complementary integration of CNN and LSTM networks. In addition, due to the vanishing gradient problem, the average F1 score of RNN was only 82.65%, while CLSA model alleviated this defect through the LSTM gating mechanism and the score increased to 90%. Experiments show that the CLSA model has a better overall performance than the other six models.

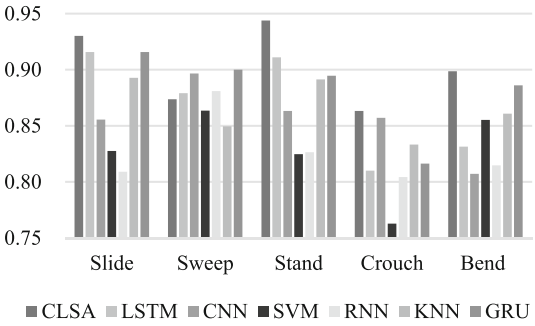


Fig. 9. Precision of activity recognition for different models.

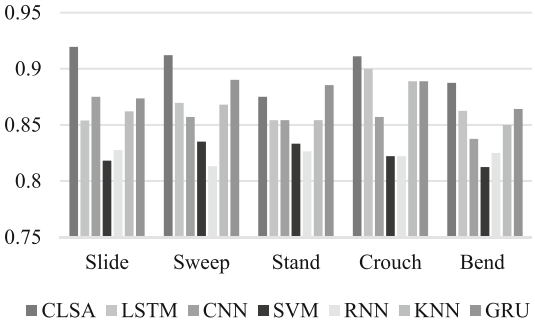


Fig. 10. Recall of activity recognition for different models.

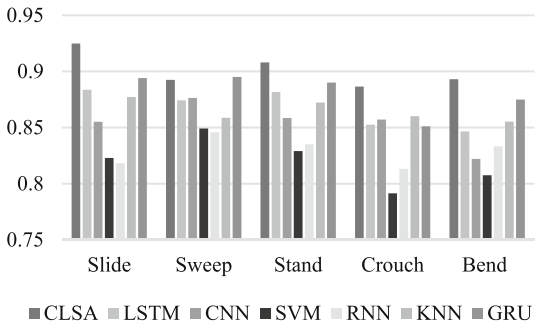


Fig. 11. F1 score of activity recognition for different models.

4.6 Ablation Experiment

To analyze how the introduction of the attention mechanism in this paper's method affects the classification results. In the real dataset, ablation experiments are designed to compare the recognition effects of different models. The experiments are still conducted in two scenarios: office and classroom. One group adopts the method of this paper, using CNN, LSTM network and attention mechanism for activity recognition and the other group removes the attention mechanism on this basis for recognition, and the model's name is abbreviated as CLS. Experiments have shown that in both office and classroom scenario, the accuracy, recall and F1 score of this method are higher than those of the CLS model. At the same time, the introduction of the attention mechanism has a higher distinction effect on easily confused activities (Table 3, 4).

Table 3. Results of ablation experiments in office scenarios.

Model	Indicators	Slide	Sweep	Stand	Crouch	Bend Over
CLSA	Precision	0.9302	0.8737	0.9438	0.8632	0.8987
	Recall	0.9195	0.9121	0.8750	0.9111	0.8875
	F1 score	0.9249	0.8925	0.9081	0.8865	0.8931
CLS	Precision	0.8977	0.8804	0.8925	0.8602	0.8974
	Recall	0.9080	0.8901	0.8646	0.8889	0.8750
	F1 score	0.9029	0.8852	0.8783	0.8743	0.8861

Table 4. Results of ablation experiments in classroom scenarios

Model	Indicators	Slide	Sweep	Stand	Crouch	Bend Over
CLSA	Precision	0.9111	0.9419	0.9111	0.8646	0.8537
	Recall	0.9425	0.8901	0.8542	0.9222	0.8750
	F1 score	0.9266	0.9153	0.8817	0.8925	0.8642
CLS	Precision	0.9070	0.8454	0.9231	0.8602	0.8831
	Recall	0.8966	0.9011	0.8750	0.8889	0.8500
	F1 score	0.9017	0.8723	0.8984	0.8743	0.8662

4.7 Experiments with Different Methods

To further verify the role of the proposed CLSA HAR method in activity recognition in different scenarios, it is compared with two different methods: CrossScene and CSI-MTGN. In two different scenarios, office and classroom, the two methods are used to recognize the five collected activities respectively and the recognition results are shown in Table 5. From the table, the CLSA method performs optimally in the office scenario (90.09%) and ranks second in the classroom scenario (89.64%), demonstrating strong overall stability, while the CSI-MTGN has the highest accuracy in the classroom scenario (90.03%), but its performance in the office scenario is slightly lower than that of the CLSA. The CLS method is close to the same accuracy in both types of scenarios, which indicates that its adaptability is more balanced. In contrast, the CrossScene method has the weakest performance in both scenarios (81.2% and 76.9%), especially in the classroom scenario, where the accuracy decreases significantly, which may need to be further optimized. In addition, the CLS and CLSA methods had smaller scenario differences (both below 1.5%) and stronger generalization ability, while the CSI-MTGN and CrossScene had larger differences (1.83% and 4.3%, respectively), suggesting that they are more sensitive to scenario changes.

Table 5. Activity recognition accuracy of different methods in different scenarios.

Method	Office	Classroom
CrossScene	81.2%	76.9%
CSI-MTGN	88.2%	90.03%
CLS	88.51%	88.29%
CLSA	90.09%	89.64%

5 Conclusion

This paper proposes a CLSA HAR method to address the issue that current CSI-based approaches tend to overfit environmental features in the source domain, while failing to extract effective features in the target domain, thereby resulting in reduced recognition accuracy. First, the raw CSI data is preprocessed using the DWT, which effectively suppresses random noise while preserving activity-relevant information. Then, in the recognition stage, spatial features are extracted using a two-layer Conv1D structure and temporal dependencies are modeled using a two-layer stacked LSTM network. On this basis, an attention mechanism is applied to assign feature weights, thereby enhancing the contextual representation of key activity segments. The experimental results show that on real-world datasets containing five activities, the CLSA model achieves average recognition accuracy of 90.09% and 89.64% in office and classroom scenarios, which are improved compared to models such as LSTM networks, CNN, SVM, RNN, KNN and GRU as well as some of the different recognition methods. Furthermore, the effectiveness of the attention mechanism is validated through ablation studies, which significantly improves the differentiation of easily confusable activities such as Slide and Sweep. In future work, we plan to expand the dataset with higher-quality activity samples to further enhance recognition performance and overall model robustness. Although this paper has achieved certain results in HAR research, several limitations remain. Constrained by experimental equipment and environmental factors, the largest dataset in the experiments only included 5 participants and 5 action categories. Furthermore, the recording environment was set in relatively stable scenarios, lacking validation in complex settings, which to some extent limits the model's generalization capability. Future research should consider optimizing experiments in multiple dimensions. First, constructing larger open datasets encompassing diverse ages, body types, complex environments (e.g., shopping malls, outdoor settings), and group interaction scenarios could enhance model generalization. Second, integrating CSI with computer vision and sensor modalities through multimodal fusion could improve recognition robustness in complex scenarios. In summary, CSI-based wireless sensing technology holds great promise in human-machine interaction and intelligent security. Future research should focus on breakthroughs in datasets and algorithms to advance the transition of this technology from idealized environments to practical applications.

References

1. Saleem, G., Bajwa, U.I., Raza, R.H.: Toward human activity recognition: a survey. Neural Comput. & Applic. **35**(5), 4145–4182 (2023)
2. Gupta, N., Gupta, S.K., Pathak, R.K., et al.: Human activity recognition in artificial intelligence framework: a narrative review. Artif. Intell. Rev. **55**(6), 4755–4808 (2022)
3. Kong, Y., Fu, Y.: Human action recognition and prediction: a survey. Int. J. Comput. Vis. **130**(5), 1366–1401 (2022)
4. Qi, J., Ma, L., Cui, Z., et al.: Computer vision-based hand gesture recognition for human-robot interaction: a review. Complex Intell. Syst. **10**, 1581–1606 (2024)
5. Mohamed, N., Mustafa, M.B., Jomhari, N.: A review of the hand gesture recognition system: current progress and future directions. IEEE Access. **9**, 157422–157436 (2021)

6. Wong, W.K., Juwono, F.H., Khoo, B.T.T.: multi-features capacitive hand gesture recognition sensor: a machine learning approach. IEEE Sensors J. **21**(6), 8441–8450 (2021)
7. Wang, W., Liu, A.X., Shahzad, M., et al.: Understanding and modeling of WiFi signal based human activity recognition. In: Proceedings of the 21st Annual International Conference on Mobile Computing and Networking, pp. 65–76. ACM., New York (2015)
8. Moshiri, P.F., Shahbazian, R., Nabati, M., et al.: A CSI-based human activity recognition using deep learning. Sensors. **21**(21), 7225 (2021)
9. Hnoohom N, Mekruksavanich S, Theeramunkong T, et al.: Efficient residual neural network for human activity recognition using WiFi CSI signals. In: Proceedings of the 2024 9th International Conference on Information and Education Innovations, pp. 113–119. ACM, New York (2024)
10. Venkatnarayan, R.H., Page, G., Shahzad, M.: Multi-user gesture recognition using WiFi. In: Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services, pp. 401–413. ACM, New York (2018)
11. Yang, X., Dai, P.: An ANN-to-SNN method for WiFi-CSI based gesture recognition. In: Proceedings of the 2023 3rd International Conference on Digital Society and Intelligent Systems, pp. 183–186. IEEE, New York (2023)
12. Yang, J., Chen, X., Zou, H., et al.: SenseFi: a library and benchmark on deep-learning-empowered WiFi human sensing. Patterns. **4**(3), 100703 (2023)
13. Zhang, R., Jing, X., Wu, S., et al.: Device-free wireless sensing for human detection: the deep learning perspective. IEEE Internet Things J. **8**(4), 2517–2539 (2020)
14. Zhang, L., Zhang, Y., Zheng, X.: Wisign: ubiquitous American sign language recognition using commercial Wi-fi devices. ACM Trans. Intell. Syst. Technol. (TIST). **11**(3), 1–24 (2020)
15. Sruthi, P., Satapathy, S., Udgata, S.K.: HandFi: WiFi sensing based hand gesture recognition using channel state information. Procedia Comput. Sci. **235**, 426–435 (2024)
16. Li, Z., Liu, F., Yang, W., et al.: A survey of convolutional neural networks: analysis, applications, and prospects. IEEE Trans. Neural Networks Learn. Syst. **33**(12), 6999–7019 (2022)
17. Banjarey, K., Sahu, S.P., et al.: Human activity recognition using 1D convolutional neural network. In: Sentimental Analysis and Deep Learning: Proceedings of ICSADL 2021, pp. 691–702. Springer, Singapore (2022)
18. Herstinsky, A.: Fundamentals of recurrent neural network (RNN) and long short-term memory (LSTM) network. Phys D: Nonlinear Phenom. **404**, 132306 (2020)

MoMa-Pos: An Efficient Object-Kinematic-Aware Base Placement Determination Framework for Mobile Manipulation

Beichen Shao[1], Nieqing Cao[2], Yan Ding[3], Xingchen Wang[1], Fuqiang Gu[1], and Chao Chen[4(✉)]

[1] College of Computer Science, Chongqing University, Chongqing 400044, China
[2] Department of Electrical and Electronic Engineering, Xi'an Jiaotong-Liverpool University, Suzhou 215123, China
[3] Shanghai Artificial Intelligence Laboratory, Shanghai 200232, China
[4] State Key Laboratory of Mechanical Transmission for Advanced Equipment, Chongqing University, Chongqing 400044, China
cschaochen@cqu.edu.cn

Abstract. Base placement is particularly critical for navigation manipulation tasks in environments, where improper placement can severely hinder task execution if the object's kinematics are not properly taken into account. In this work, we present MoMa-Pos, a framework that determines the base placement for mobile manipulators in such environments. MoMa-Pos leverages a graph-based neural network to predict object importance and selectively reconstructs the environment by prioritizing task-relevant key objects, enhancing computational efficiency and ensuring that only essential kinematic details are processed. Moreover, MoMa-Pos integrates inverse reachability maps with environmental kinematic properties to determine feasible base placement tailored to the specific robot model. Extensive evaluations demonstrate that MoMa-Pos outperforms existing methods in both real and simulated environments, offering improved efficiency, precision, and adaptability across diverse settings and robot models.

Keywords: Base Placement · Mobile Manipulator · Graph-Based Neural Network · Reachability Maps

1 Introduction

Mobile manipulators, which integrate a mobile base with a robotic arm, have emerged as vital components in modern intelligent systems. In particular, they serve as key enablers of Cyber-Physical-Social Systems (CPSS), bridging computational intelligence, physical interaction, and human-centered applications. By

B. Shao and N. Cao—Contributed equally to this work.

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 254–270, 2026.
https://doi.org/10.1007/978-981-95-8405-5_14

autonomously perceiving, planning, and acting in dynamic human environments, these robots function as essential interfaces between cyber, physical, and social domains. They are increasingly deployed in real-world settings such as smart homes and assisted living spaces, where they perform navigation-manipulation tasks—such as approaching appliances or interacting with articulated objects like doors and drawers [1–3]. The success of such tasks critically depends on accurate base placement [4–7], as suboptimal positioning can severely limit manipulation performance, particularly under fixed-base constraints where the base remains stationary during interaction.

However, the determination of feasible base placement remains challenging, particularly in household environments that contain both rigid and articulated objects [8]. Physically interacting with articulated objects requires robots to be *kinematically aware* of these objects' specific joint movements and constraints during manipulation. For instance, when opening a fridge door, the robot must account for the door's rotational movement and position itself correctly to exert the necessary force along the appropriate kinematic trajectory. Therefore, kinematic modeling of these articulated objects is essential. While technologies in this area have advanced considerably [9], the associated computational costs remain significant. The cumulative modeling time of all objects in a given scenario can be substantial, *further worsening the overall efficiency of robotic systems*. Additionally, a feasible base placement is highly dependent on the *robot model*. Robots with shorter arms, such as Stretch [10], need to place themselves closer to target objects.

Existing approaches in computing base placement present several limitations. The basic methods [11,12] set specific base placement areas empirically, without accounting for critical factors such as the kinematics of articulated objects and specific robot models, which limits their applicability in complex scenarios. More advanced methods use inverse reachability maps [13], relying on computed kinematic solutions to guide base placement [13–16]. However, these approaches overlook the kinematic complexities of articulated objects, where complex motion and configuration dependencies impose constraints that extend beyond the limitations of reachability maps. Worse still, these methods typically incur high computational costs, making them impractical for real-time applications. Learning-based approaches predict feasible base placements by considering the kinematic properties of objects, the environment, and specific robot models [4,8,17–19]. However, they require extensive data collection and training, often tailored to predefined robot-environment pairs, which limits their generalizability to real-world applications. Given these limitations, there is a pressing need for an efficient and generalizable framework that incorporates object-kinematic awareness and adaptability to various robot models.

This work introduces *MoMa-Pos*, a framework designed to determine base positioning for mobile manipulators *prior to* executing navigation-manipulation tasks. A central aspect of MoMa-Pos is its ability to prioritize key objects in the scene, enabling efficient modeling without processing every object in the environment. By leveraging a graph-based neural network, our method predicts

the importance of task-relevant objects, allowing for focused computation on critical elements. This selective modeling integrates advanced algorithms like URDFormer [9], facilitating accurate kinematic representations where necessary. The determination of base placements in MoMa-Pos is distinguished by its integration of two critical factors: the use of inverse reachability maps to account for robot-specific constraints, and the incorporation of objects' kinematic constraints from the environment. Such dual strategy refines potential base placement areas, balancing physical feasibility with task-specific efficiency, leading to enhanced precision and adaptability. The main contributions of this paper are summarized as follows:

1. We propose MoMa-Pos, a novel framework that determines feasible base placements prior to task execution.
2. We prioritize task-relevant objects by leveraging a graph-based neural network, instead of modeling all objects in the environment. This selective modeling reduces computational cost and improves planning efficiency by focusing only on key objects.
3. We integrate inverse reachability maps (to capture robot-specific kinematic limits) and object-level kinematic constraints (to account for articulation and interaction dependencies). This dual-constraint strategy leads to refined candidate base placements that are both physically feasible and task-efficient.
4. We evaluate MoMa-Pos in both simulated and real-world environments across various tasks. Results demonstrate that our method consistently outperforms existing approaches in terms of task success rate, and computational efficiency.

2 Problem Statement

In this section, we define the problem of determining the feasible base placement in an environment, denoted as Env^O, which contains a set of objects represented as $O = \{o_1, o_2, \cdots, o_N\}$. Each object o_i is characterized by its 3D position o_i^p and 3D bounding box o_i^b. The target object within these is specified as o_t, and the mobile manipulator is tasked with manipulating object o_t. As shown in Fig. 1, this robot model r consists of a base, a body, and an arm, which is denoted by the tuple,

$$r = \left(r^b, r^h, r^l\right) \tag{1}$$

where r^b represents the base dimensions (r^x, r^y, r^z) (width, length, height), r^h is the body height, and r^l is the arm extension length. The robot's 2D base placement is denoted by r^p, which is assumed to be vertically aligned with the arm base. The *objective* is to determine a feasible base placement r^{p*} for navigation-manipulation tasks. The position must meet two critical criteria: the arm must be able to compute collision-free trajectories for task execution, and the base must avoid collisions with any objects in Env^O.

To obtain the position o_i^p and bounding box o_i^b for each object o_i, the robot first scans the environment to collect 3D point cloud data and videos. The point

cloud is processed by a pre-trained Mask3D model [20] to segment objects and extract their corresponding 3D bounding boxes. After removing outliers based on a distance threshold, the remaining points are used to compute an axis-aligned 3D bounding box, with o_i^p defined as its geometric center. Additionally, the video data is used to select RGB images with optimal views for further kinematic modeling.

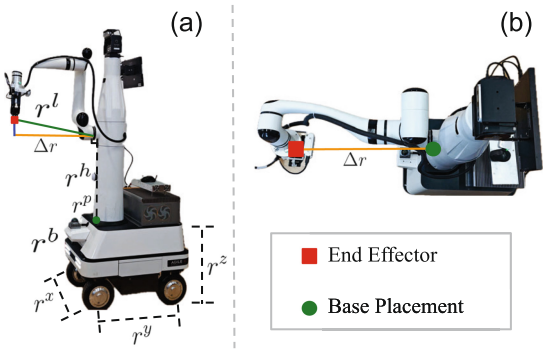


Fig. 1. Our mobile manipulator includes a base, a body, and an arm. The symbol Δr refers to the horizontal distance between the base's placement and the end-effector in the x-y plane. r^b refers to the base dimensions, r^h is the body height, and r^l is the arm extension length. For explanations of additional symbols, see Sect. 2.

3 The MoMa-Pos Framework

The proposed MoMa-Pos mainly consists of four phases, including object importance prediction for kinematic modeling, kinematic modeling of key objects, base placement determination, and navigation to feasible base placement, which are indicated in Fig. 2.

3.1 Object Importance Prediction for Kinematic Modeling

The objective of object importance prediction is to select a subset of objects, S, from the object set O for kinematic modeling, enabling efficient computation. Rather than directly select the objects in an environment, we evaluate each object's importance and select those with an importance score exceeding a predefined threshold α. This approach provides flexibility; if the initial selection is insufficient, lowering the value of α can encompass additional objects [21]. We develop *a prediction model*, $f : O \times O \rightarrow (0, 1]$, where the output, $f(o_i, o_t)$, represents the likelihood of including object o_i in S based on its importance relative to the target object o_t. Due to the challenges of obtaining precise labels for all objects in diverse environments [21], we adopt an unsupervised learning

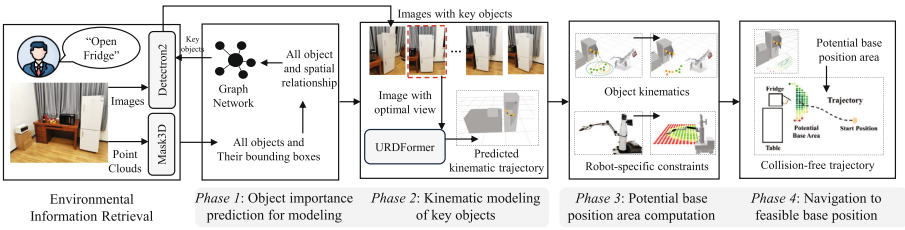


Fig. 2. Framework of MoMa-Pos. The framework is composed of four key phases. Phases 1 and 2 form the task-specific kinematic perceptual modeling, while Phases 3 and 4 focus on base placement determination and guided navigation. In *Phase 1*, key objects in the scene are prioritized to enable efficient kinematic modeling. In *Phase 2*, kinematic modeling is conducted for these key objects, supporting the subsequent object-kinematic-aware base placement determination. In *Phase 3*, potential base placement areas are identified by considering both robot-specific constraints and environmental kinematics. Finally, in *Phase 4*, the robot navigates to the feasible position, ensuring physical feasibility and adaptability for task execution.

approach, guided by the observation that an object's importance is often correlated with its spatial proximity to o_t and its size, o_i^b. This correlation enables us to infer importance without exact labels.

Algorithm 1. Object Importance Prediction

Input: $O = \{o_1, o_2, \ldots, o_n\}$: object set, o_t: target object, α: threshold
Output: $S \subseteq O$: selected important objects
$G = (V, E)$, $v_i \leftrightarrow o_i$, $\forall o_i \in O$ $v_i.\text{attr} \leftarrow ob_i$
foreach $o_i \in O$ **do**
  foreach $o_j \in O$ **do**
    if $\text{rel}(o_i, o_j)$ **then**
      $E \leftarrow E \cup \{(v_i, v_j)\}$
      $w(v_i, v_j) \leftarrow \text{norm}\left(\frac{1}{\text{dist}_{xy}(o_i, o_j)}\right) + \text{norm}\left(\text{size}(o_j)\right)$
$\text{emb}(v_i) \leftarrow \text{DeepWalk}(G, w)$
foreach $v_i \in V$ **do**
  $f(o_i, o_t) \leftarrow \cos\left(\text{emb}(v_i), \text{emb}(v_t)\right)$
$S \leftarrow \emptyset$
foreach $o_i \in O$ **do**
  if $f(o_i, o_t) > \alpha$ **then**
    $S \leftarrow S \cup \{o_i\}$
return S

As shown in Algorithm 1, our method structures within a graph-based architecture to calculate spatial proximity between objects o_i and o_t while considering

object size o_i^b. In Env^O, each object o_i is represented as a node $v_i \in V$, with attributes such as size o_i^b. The spatial relationships between objects are captured by edges $E \subseteq V \times V$. An edge (v_i, v_j) exists if object o_i is spatially related to o_j (e.g., on, in, or inside). The direction of the edge indicates spatial hierarchy, and the weight is defined as $1/dist_{xy}(o_i, o_j)$, where $dist_{xy}$ is the horizontal distance between o_i and o_j. We have explored several algorithms to quantify the spatial proximity between nodes v_i and v_t. After a thorough evaluation, we ultimately selected the DeepWalk algorithm [22] due to its superior ability to capture the required spatial information. Node sequences are generated through biased random walks, with the transition probability $P(v_i, v_j)$ weighted by the function,

$$w(v_i, v_j) = \text{norm}\left(\frac{1}{\text{dist}_{xy}(o_i, o_j)}\right) + \text{norm}\left(\text{size}(o_j)\right) \tag{2}$$

This bias ensures the walks reflect both spatial distance and object size, generating sequences that better capture object importance,

$$P(v_i, v_j) = \begin{cases} \frac{w(v_i, v_j)}{\sum\limits_{u \in N(v_i)} w(v_i, u)}, & (v_i, v_j) \in E \\ 0, & \text{otherwise} \end{cases} \tag{3}$$

where $N(v_i)$ represents the set of nodes adjacent to v_i. After obtaining node embeddings, we compute the importance score for each object o_i by calculating the *cosine* similarity of its embedding to that of the target object v_t. Only objects with a score exceeding a threshold α can be included in the set S. It should be noted that the threshold α is adjustable, starting from a high value and decreasing if the system requires more objects to achieve the optimal performance.

3.2 Kinematic Modeling of Key Objects

Here, we aim to perform the kinematic modeling of key objects in the simulation environment. Key objects can be articulated and rigid. The kinematic modeling is only needed for articulated objects. While for rigid objects, we simply use their bounding boxes to approximate. Note that non-target articulated objects are also considered as rigid ones for efficiency. These models will aid in the base placement computation for navigation planning and predict manipulation waypoints [23] for articulated objects, enabling the effective execution of real-world manipulation tasks. To achieve this, we employ the state-of-the-art model, URDFormer, an end-to-end model based on Vision Transformers (ViT), which excels at capturing an object's kinematic structure through URDF models transformed from a single input image. However, URDFormer typically encounters two major problems when grounding the model to real-world objects, i.e., *the input image of the model and the misalignment with real-world objects*, respectively.

For the first problem, based on our observation and exploration, we find that the viewpoint of the input image significantly affects the modeling accuracy, and the frontal view images can generate the best results. Therefore, we design

an algorithm to select the optimal viewpoint image of the target object from video streams, ensuring the input to URDFormer is optimal for more accurate kinematic modeling. This process involves using Detectron2 [25] to detect the target object in each video frame, followed by sorting the frames based on the object's central position in the image. We then fine-tuned a ResNet50 model [24], trained on a dataset we created, which includes 1,000 labeled images (frontal and non-frontal) of over 20 everyday objects such as refrigerators and microwaves, to classify whether the object in each frame is facing forward. The loss function used for training is defined as follows,

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N}\sum_{i=1}^{N}\left[y_i \log(\hat{y}_i) + (1-y_i)\log(1-\hat{y}_i)\right] \tag{4}$$

where N is the number of training samples, $y_i \in \{0, 1\}$ denotes the ground-truth label indicating whether the object in the i-th image is frontal (1) or not (0), and $\hat{y}_i \in (0, 1)$ is the predicted probability that the object is frontal. This binary cross-entropy loss encourages the model to assign high confidence to correct predictions while penalizing incorrect classifications. The *first best image*, which is classified as "frontal", is selected as the input for URDFormer to generate an initial URDF model with interactive structures.

For the second problem, to address the limitations of using a 2D single-image input, which can cause misalignment of the URDF model with the real-world 3D environment, we correct the model by incorporating precise 3D information from point clouds. Specifically, we use 3D bounding boxes generated by Mask3D [20] to align the object's size and pose, ensuring the URDF model accurately simulates an interactive environment that mirrors the real world. Finally, after inputting a recorded environment video, our algorithm selects the best frontal image of the target object and uses 3D bounding boxes to align the object's size and pose, generating an accurate URDF model. This enables the precise location of actionable parts on articulated objects at any angle, ensuring real-world alignment. The location data at each angle are used to generate the object's manipulation waypoints.

3.3 Base Placement Determination

To determine the base placement, we first calculate the potential area for base placements (denoted as A), where the requirement of no collisions with surrounding objects must be satisfied. The final base placement is then refined by further integrating both the robot's constraints and the objects' kinematics constraints.

To obtain an initial robot's potential base placement area A, we consider both the robot's model r and the target object's position o_t^p to avoid collisions. The potential area A is calculated according to the following equation:

$$A = \left\{ r^p \mid dist_{xy}(r^p, o_t^p) \leq \Delta r,\ \forall i,\ \neg overlap\left(r^p, r^b, o_i^p, o_i^b\right) \right\} \tag{5}$$

Here, Δr represents the allowable horizontal distance between a base placement r^p and the target o_t^p, illustrated in an orange line in Fig. 4. The overlap function

checks for collisions between the robot's base and surrounding objects o_i by comparing their positions and boundaries. More specifically, Δr is calculated as,

$$\Delta r = \sqrt{(r^l)^2 - (o_t^{p.z} - r^z - r^h)^2} \tag{6}$$

where r^l is the arm extension range (shown in green in the figure), derived from the robot's DH matrix. The z-axis distance between r^p and $o_t^{p.z}$ is indicated by $|o_t^{p.z} - r^z - r^h|$, marked as a blue line in the figure.

To refine A, we integrate an inverse reachability matrix to prevent self-collision and incorporate an improved potential field method to avoid potential interference with furniture and obstacles due to their kinematic constraints. Specifically, we design a two-step process to meet the constraints of robot's and object's kinematics, respectively.

Algorithm 2. Base Placement Determination

Input: o_t^p: target object position, $\{o_i\}$: surrounding objects, r: robot model
Output: P: feasible base placement set
$A \leftarrow \emptyset$
foreach *candidate position* r^p **do**
  if $\text{dist}_{xy}(r^p, o_t^p) \leq \Delta r$ ***and*** $\neg\text{overlap}(r^p, \{o_i\})$ **then**
    $A \leftarrow A \cup \{r^p\}$
`// Robot's Kinematic Constraints`
$\text{IRM} \leftarrow \text{InverseReachabilityMatrixOffline}(r)$; `// computed offline`
$\text{IRM}_{\text{shifted}} \leftarrow \text{Translate}(\text{IRM}, o_t^p)$
$A_\text{k} \leftarrow A \cap \text{IRM}_{\text{shifted}}$
`// Object's Kinematic Constraints`
$P \leftarrow \emptyset$ **foreach** $r^p \in A_\text{k}$ **do**
  if $\text{LineSegment}(r^p, o_t^p)$ *intersects any* $\text{BBox}(o_i)$ **then**
    $F(r^p, z) \leftarrow 0$
  else
    $F(r^p, z) \leftarrow \dfrac{1}{\text{dist}_{xy}(r^p, o_t^p)}$ $P \leftarrow P \cup \{r^p\}$
return P

- **Robot's Kinematic Constraints:** Utilizing the inverse reachability matrix, we can obtain the value of points in A without considering the kinematic structure. Real-time online computation of the inverse reachability matrix IRM requires long computation time, so we calculate it offline. We then translate the IRM by shifting its center from r^p to o_i^p. The shifted matrix $\text{IRM}_{\text{shift}}$ allows us to precompute feasible base placements within the area A. By selecting base placements directly from the matrix, it is easy to get base placements satisfy the robot's kinematic constraints A_k.
- **Object's Kinematic Constraints:** Inspired by the potential field method [26], we propose a novel approach to address the impact of the kinematic structure of the target object at the base placement. For each point

in the incorporated trajectory, the target object o_t exerts an attractive force. For each candidate base placement r^p, we sample a 3D position (r^p, z) and connect it to o_t^p with a line. If this line collides with the bounding box of any furniture (such as actionable parts), the potential value of the potential field $F(r^p, z)$ is set to zero, indicating that the robot cannot reach the target from this position. Otherwise, we calculate $F(r^p, z) = 1/dist_{xy}(r^p, o_t^p)$. As shown in Fig. 3, different kinematic constraints of the same object, such as varying opening angles, may result in very different base placements.

The overall procedure is summarized in Algorithm 2. The final potential value is the sum of the value from the inverse reachability matrix and the value from the potential field. As shown in Fig. 3, positions with higher final potential values are more feasible.

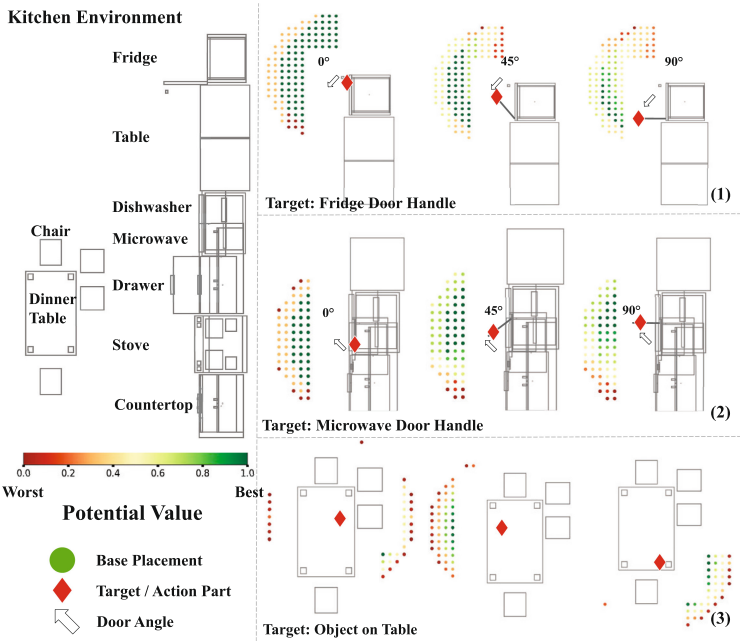


Fig. 3. The upper-left figure provides an overview of the robots' operating environment, while the three subfigures on the right focus on key task-relevant objects in the simulation. Green circles indicate potential robot standing positions and red diamonds represent targets, such as the fridge door handle. In example (1), when the door handle changes position, the corresponding feasible placement map is also changed, consistent with intuitive expectations. This is best viewed in the enlarged digital version.

3.4 Navigation to Feasible Base Placement

In the refined area A, we use the Latin Hypercube sampling method to sample M 2D positions r_i^p, with higher potential values suggesting better feasibility. We balance potential values with navigation costs, rather than simply selecting the lowest potential position and checking feasibility with RRTconnect [27]. This resembles the Open Traveling Salesman Problem (TSP) [28], where the robot does not return to its starting position.

Given the large number of sampled positions, we group them into smaller sets, each containing T candidates. Within each set, the optimal path is determined by solving the Open TSP, using edge weights that combine both distance $dist_{xy}(r_i^p, r_j^p)$ and potential values. The edge weight is computed as,

$$\text{cost}(r_i, r_j) = \text{dist}(r_i^p, r_j^p) + \big(F(r_j^p) - F(r_i^p)\big) \tag{7}$$

Algorithms ranging from exact methods [29] to heuristic approaches [30] can be used, depending on the size of T. The search continues until a collision-free trajectory is found, and the corresponding base placement r^{p*} is selected.

Lemma 1 (MoMa-Pos is complete). *Given any object importance scorer* $f : O \times O \rightarrow (0, 1]$*, threshold* $\alpha \in [0.0, 1.0]$*, and* M *sampled positions,* $M \in [1, \infty)$*, if the motion planner PLAN is complete, then MoMa-Pos is complete.*

Proof. Given the codomain of f excludes 0, an $\alpha > 0$ exists such that $\forall o \in O, f(o, o) \geq \alpha$. With $M \rightarrow \infty$, a feasible position, if existent, will be sampled eventually. Given PLAN's completeness, MoMa-Pos also achieves completeness. Hence, for any task and an infinite number of samples, MoMa-Pos will identify at least one solution, assuming one exists, through PLAN.

4 Experiments

In our experiments, we aim to address the following questions:
Q1: How does MoMa-Pos's capability of determining feasible base placements?
Q2: How efficient is the object importance predictor for efficiency in kinematic modeling?
Q3: How accurate is the kinematic modeling?

Experimental Setup: In the **real robot** experiment, we construct a home environment in a controlled laboratory setting, as illustrated in Fig. 4. The scene includes five rigid objects (e.g., table, apple, and cup) and two articulated objects (i.e., fridge and microwave). The rigid objects and microwave are placed randomly on the table, which has a variable position, while the fridge remains fixed. The mobile manipulator, as shown in Fig. 1, consists of an AgileX-based mobile platform and a Realman 63-F robotic arm, equipped with a Realsense 435 camera for RGB images, depth images, and point cloud data acquisition. The mobile manipulator is tasked with grasping rigid objects (i.e., apple and cup)

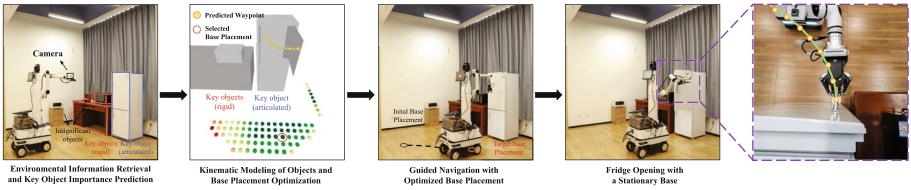


Fig. 4. Real Robot Demonstration. We implemented MoMa-Pos on a real robotic system comprising an AgileX mobile platform and a Realman-63F robotic arm. The robot's task in this setup is to open a fridge door. In the first figure, MoMa-Pos predicts key objects in the environment, with these objects highlighted by colored outlines. Meanwhile, the manipulation waypoints are predicted and indicated by yellow circles. Next, MoMa-Pos performs kinematic modeling, representing rigid objects as boxes and articulated objects using URDFormer. A feasible base placement is then selected, taking into account robot-specific constraints and the kinematic properties of the environment. The final two figures illustrate the robot successfully completing navigation, where the trajectory is indicated in a dashed black line, and fixed-base manipulation tasks using the predicted waypoints.

and opening articulated items. For the fridge and microwave, we open them to 90 ° during the experiments. The methods for determining base placements are evaluated across five trials per task, with each scene featuring a distinct object configuration. The robot's initial position is kept consistent across all trials. For manipulation, the spatial positions of the objects and the actionable parts (e.g., door handles) of articulated objects are determined using Mask3D [20] and URDF model. We then employ the RRTconnect algorithm, implemented via the open-source OMPL library [31], to compute corresponding manipulation trajectories. For navigation, we utilize the Dynamic Window Approach (DWA) [32], chosen for its responsiveness and stability in complex dynamic environments, ensuring high precision and safety. Details are available on our website.

We also perform **simulated robot experiments**. In contrast to the real-world setup, the simulation involves a greater number of everyday objects (30, including a fridge, microwave, and cup) and more tasks, totaling 500. In the simulation, the 3D positions of objects and their actionable components are directly accessible. The simulation tool is supported by the GitHub project[1] [33]. Our experiments were conducted on a computer with an AMD EPYC 7452 processor (32 cores, 64 threads), an Nvidia GTX 2080 GPU, and 128GB of RAM.

Baselines: We compare our method with the following reimplemented baselines, which are inspired by original data-driven approaches:

– **Habitat** [34]: For rigid objects such as tables, the feasible base placement closest to the target is selected, ensuring it is navigable. For articulated objects like fridges, the feasible base placement is determined based on a

[1] https://github.com/AutonoBot-Lab/BestMan.

fixed position relative to these objects, guided by prior experience, while also ensuring no collisions with the environment.
- **M3*** [4]: Based on the original method, the area is divided into 5×5 cm cells. For rigid objects, the closest navigable position is selected, prioritizing navigation cost. For articulated objects, base placements are sampled until a feasible one is found. Unlike the original, which uses reinforcement learning to optimize placement, this baseline lacks optimization, potentially increasing costs. We use * to denote differences. Additionally, while the original allows dynamic base repositioning, our method uses fixed-base manipulation, requiring setup adjustments.
- **Reuleaux** [13]: Similar to M3*, it starts by designating a specific area, divided into 5×5 cm cells. It then evaluates how easily the robot can reach the target without obstacles. The robot chooses its feasible base placement randomly, prioritizing those with the highest reachability to ensure optimal accessibility.

Different from MoMa-Pos, none of these three baselines account for object kinematics, which may lead to suboptimal manipulation performance, particularly when handling articulated objects.

Rating Criteria: To ensure a fair comparison, all baselines are evaluated in a complete working environment. These methods do not account for the kinematic information of articulated objects, treating all objects as simple bounding boxes, thus saving time on kinematic modeling. We evaluate the performance of all approaches using the following metrics: *task execution time* (Time), measured in seconds; *navigation cost* (Cost), measured in meters; and *success rate* (SRate). For a more detailed breakdown of time consumption in determining feasible base placement, we measure the time spent in each key step of the process, which includes three stages: predicting object importance, performing kinematic modeling, and determining base placement. *The time allocated for modeling* not only includes the modeling process itself but also the integration of modeled objects into the simulator. To focus on the core task efficiency, we exclude navigation time from task execution, as robot movement speed can introduce variability unrelated to task completion. Instead, the navigation cost metric measures spatial efficiency, ensuring it doesn't affect task performance evaluation. For non-articulated tasks, success means grasping the target without collisions, while for articulated tasks, success means interacting successfully and executing the planned trajectory without collisions.

4.1 Results and Analysis

MoMa-Pos vs. Baselines. We aim to address **Q1** here. The main results are presented in Table 1. The table demonstrates that our method consistently achieves a 100% success rate across all trials in the working environment. Compared to the baseline methods (Habitat, M3*, and Reuleaux), the success rate of MoMa-Pos is significantly higher, particularly with articulated objects such as fridges. This is due to our method's ability to account for the structure of

Table 1. A comparison between MoMa-Pos and baseline methods in the real robot experiments in terms of task execution time (Time), navigation cost (Cost), and success rate (SRate) across different furniture categories, namely containers and non-containers. The optimal results are highlighted in bold.

Methods	Grasp Apple *(rigid)*			Grasp Cup *(rigid)*			Open Fridge *(right-hinged)*			Open Microwave *(left-hinged)*		
	Time (s)	Cost (m)	SRate (%)	Time (s)	Cost (m)	SRate (%)	Time (s)	Cost (m)	SRate (%)	Time (s)	Cost (m)	SRate (%)
MoMa-Pos (ours)	**13.5 ± 2.0**	3.2 ± 1.2	**100.0**	**13.2 ± 2.4**	3.5 ± 1.4	**100.0**	17.4 ± 2.3	2.3 ± 0.1	**100.0**	17.2 ± 1.0	3.7 ± 1.6	**100.0**
Habitat	17.3 ± 0.2	**3.0 ± 0.0**	100.0	16.9 ± 2.7	**3.2 ± 0.0**	100.0	**15.7 ± 1.4**	**2.2 ± 0.0**	18.0	**14.4 ± 0.4**	**3.2 ± 0.0**	22.0
*M3**	17.8 ± 2.3	3.2 ± 1.1	100.0	17.3 ± 3.2	3.3 ± 0.1	100.0	18.2 ± 8.4	2.2 ± 0.1	26.0	17.4 ± 1.4	3.5 ± 0.0	44.0
Reuleaux	19.2 ± 4.3	3.6 ± 1.6	100.0	18.7 ± 3.9	3.7 ± 1.1	100.0	17.7 ± 2.7	2.5 ± 0.1	70.0	18.0 ± 2.0	3.6 ± 1.4	82.0

the objects, effectively avoiding collisions with kinematic structures. MoMa-Pos also excels in efficiency, outperforming the M3* and Reuleaux baselines. These methods rely on random sampling within a predefined area and often fail to quickly locate a viable position, resulting in increased time costs and inconsistent performance, as evidenced by the high variance in M3* and Reuleaux's results. Although Habitat is sometimes more efficient, its lower success rate, especially with articulated objects, limits its practical application. Our method not only demonstrates significant advantages in success rate and time efficiency with articulated objects but also ensures the low navigation cost for finding feasible solutions. In our tests, MoMa-Pos consistently determines a feasible solution within 18 s.

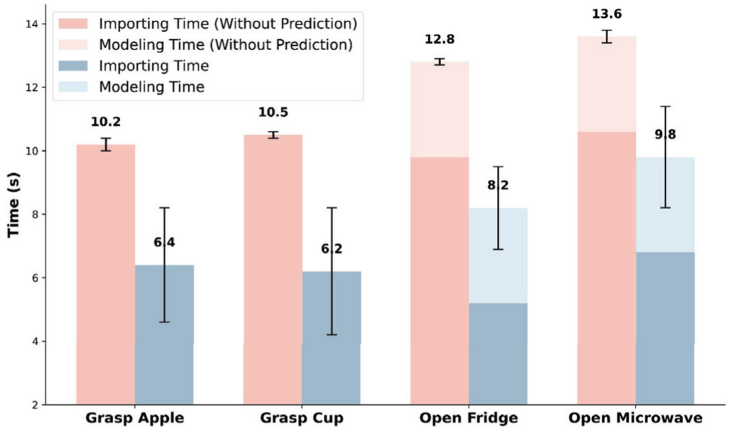


Fig. 5. The modeling time comparison between MoMa-Pos and its variant, which lacks the object importance prediction, with the x-axis denoting the location of the target object.

Object Importance Predictor for Efficiency in Kinematic Modeling. We aim to address **Q2** here. Our experimentation begins by setting the threshold value α (specifically, 0.45) within the object importance prediction module.

The achieved success rate of 100% confirms that no crucial object has been overlooked. Figure 5 illustrates the modeling time for MoMa-Pos and a variant without the prediction module. It is clear that incorporating prediction in MoMa-Pos decreases the time cost. It should be noted that for non-articulated tasks, such as grasping an apple or a cup, kinematic modeling is not necessary, and therefore no modeling time is required.

Interestingly, we observed that the importing time for the other three tasks is greater than that of the "Open Fridge" task. This is because the remaining three tasks involve physical stacking relationships (e.g., an apple on a table), requiring consideration of more objects in the environment. This observation suggests that the Importance Predictor is able to effectively handle varying environmental complexities, demonstrating good robustness in accurately predicting the importance of objects across different task scenarios.

Kinematic Modeling Accuracy. We aim to address **Q3** here. Table 2 presents the accuracy of kinematic modeling. For the Vanilla URDFormer, the generated object model exhibits significant misalignment with the real-world object, resulting in a spatial error of approximately 0.38 m, as shown in the table. The spatial error is calculated based on the real-world dimensions of articulated objects, with the unit in meters. To improve alignment with real-world objects, we apply two methods to obtain the bounding box: Omni3D [35] and Mask3D. Omni3D utilizes RGB images and Mask3D leverages depth data. Compared to Omni3D, Mask3D demonstrates significantly higher accuracy, reducing the modeling error to 0.04 m.

Table 2. Comparison of average modeling errors across different methods for articulated object modeling.

Methods	Modeling Error (m)
Vanilla URDFormer	0.38
URDFormer+Omni3D	0.13
URDFormer+Mask3D (ours)	**0.04**

We also calculate the average time distribution across all tasks, including those involving both rigid and articulated objects, throughout the entire process. Table 3 presents the results. It is clear that the majority of the time (over 98% of the total time cost) is dedicated to kinematic modeling and determining feasibility through a sampling-based motion planner, such as RRTconnect. In contrast, the time spent on candidate selection, including solving the Open Traveling Salesman Problem (TSP), is minimal, accounting for less than 2% of the total time (less than 0.3 s).

Table 3. Average time distribution of entire process in MoMa-Pos

Process	Time Distribution (%)
Modeling (including importing time)	58.0
Checking feasibility of samples (OMPL)	40.0
Potential field computation	0.9
Importance prediction	0.8

Our framework is further evaluated using three simulated robots: Fetch, Segway+UR5e, and Husky+UR5e. In all trials, MoMa-Pos achieves a 100% success rate, demonstrating its robustness across different robot models. This highlights the versatility of our method, which is adaptable to various scenarios. Compared to baselines, MoMa-Pos significantly reduces the effort needed for integrating kinematic models into the simulator. Note that the simulated experiments do not involve kinematic modeling steps.

Additional simulation results are provided in the appendix[2].

5 Conclusion and Future Work

In this work, we introduce MoMa-Pos, a novel framework for the determination of base placement in mobile manipulators, particularly in environments with both rigid and articulated objects. MoMa-Pos tackles navigation-manipulation challenges by selectively modeling relevant objects and using a graph-based neural network to enhance computational efficiency. By integrating inverse reachability maps and kinematic properties, it ensures precise base placement for specific robot models, offering adaptability and physical feasibility in diverse environments.

In the future, we plan to expand this work mainly in three directions. First, we plan to enhance its efficiency by reducing computational cost, e.g., the kinematic modeling time. Second, we intend to extend its generalizability by testing more application scenarios. Lastly, we intend to improve its modeling applicability by refining the modeling approach to better capture irregularly shaped articulated objects.

Acknowledgment. This study was supported by the National Natural Science Foundation of China (Grant Nos. 62322601 and 62172066), the Fundamental Research Funds for the Central Universities (Grant No. 2024IAIS-QN017), the Independent Research Project of State Key Laboratory of Mechanical Transmission for Advanced Equipment (Grant No. SKLMT-ZZKT2024R07), and the Excellent Youth Foundation of Chongqing (Grant No. CSTB2023NSCQJQX0025).

[2] https://yding25.com/MoMa-Pos/Appex.

References

1. Yenamandra, S., et al.: HomeRobot: open-vocabulary mobile manipulation. arXiv preprint arXiv:2306.11565 (2023)
2. Liu, P., et al.: Safe reinforcement learning of dynamic high-dimensional robotic tasks: navigation, manipulation, interaction. In: International Conference on Robotics and Automation (ICRA), pp. 9449–9456 (2023)
3. Nasiriany, S., et al.: Robocasa: large-scale simulation of everyday tasks for generalist robots. In: Robotics: Science and Systems (RSS) (2024)
4. Gu, J., Chaplot, D.S., Su, H., Malik, J.: Multi-skill mobile manipulation for object rearrangement. In: International Conference on Learning Representations (ICLR) (2023)
5. Yoon, J., Park, J., Park, H., Lee, H., Lee, S., Hwang, M.: Optimizing base placement of surgical robot: kinematics data-driven approach by analyzing working pattern. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 6907–6914 (2024)
6. Zhang, H., Mi, K., Zhang, Z.: Base Placement Optimization for Coverage Mobile Manipulation Tasks. arXiv preprint arXiv:2304.08246 (2023)
7. Sandakalum, T., Yao, N.X., Ang, M.H.: Inv-Reach Net: deciding mobile platform placement for a given task. In: IEEE-RAS International Conference on Humanoid Robots (Humanoids), pp. 127–133 (2022)
8. Gupta, A., Zhang, M., Sathua, R., Gupta, S.: Opening articulated objects in the real world. arXiv preprint arXiv:2402.17767 (2025)
9. Chen, Q., et al.: Urdformer: a pipeline for constructing articulated simulation environments from real-world images. In: Robotics: Science and Systems (RSS) (2024)
10. Kemp, C.C., Edsinger, A., Clever, H.M., Matulevich, B.: The design of stretch: a compact, lightweight mobile manipulator for indoor human environments. In: International Conference on Robotics and Automation (ICRA), pp. 3150–3157 (2022)
11. Szot, A., et al.: Habitat 2.0: training home assistants to rearrange their habitat. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 34, pp. 251–266 (2021)
12. Puig, X., et al.: Habitat 3.0: a co-habitat for humans, avatars and robots. In: International Conference on Learning Representations (ICLR) (2024)
13. Makhal, A., Goins, A.K.: Reuleaux: robot base placement by reachability analysis. In: IEEE International Conference on Robotic Computing (IRC), pp. 137–142 (2018)
14. Vahrenkamp, N., Asfour, T., Dillmann, R.: Robot placement based on reachability inversion. In: IEEE International Conference on Robotics and Automation (ICRA), pp. 1970–1975 (2013)
15. Zacharias, F., Borst, C., Hirzinger, G.: Capturing robot workspace structure: representing robot capabilities. In: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), pp. 3229–3236 (2007)
16. Reister, F., Grotz, M., Asfour, T.: Combining navigation and manipulation costs for time-efficient robot placement in mobile manipulation tasks. IEEE Robot. Autom. Lett. **7**(4), 9913–9920 (2022)
17. Gupta, A., Shepherd, M.E., Gupta, S.: Predicting motion plans for articulating everyday objects. In: International Conference on Robotics and Automation (ICRA), pp. 5946–5953 (2023)

18. Burgess-Limerick, B., Haviland, J., Lehnert, C., Corke, P.: Reactive base control for on-the-move mobile manipulation in dynamic environments. IEEE Robot. Autom. Lett. **9**(3), 2048–2055 (2024)
19. Honerkamp, D., Welschehold, T., Valada, A.: Learning kinematic feasibility for mobile manipulation through deep reinforcement learning. IEEE Robot. Autom. Lett. **6**(4), 6289–6296 (2021)
20. Schult, J., Engelmann, F., Hermans, A., Litany, O., Tang, S., Leibe, B.: Mask3D: mask transformer for 3D semantic instance segmentation. In: International Conference on Robotics and Automation (ICRA), pp. 8216–8223 (2023)
21. Silver, T., Chitnis, R., Curtis, A., Tenenbaum, J.B., Lozano-Perez, T., Kaelbling, L.P.: Planning with learned object importance in large problem instances using graph neural networks. In: Association for the Advancement of Artificial Intelligence (AAAI), vol. 35, no. 13, pp. 11962–11971 (2021)
22. Perozzi, B., Al-Rfou, R., Skiena, S.: DeepWalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710 (2014)
23. Xia, W., et al.: Kinematic-aware prompting for generalizable articulated object manipulation with LLMs. In: IEEE International Conference on Robotics and Automation (ICRA), pp. 2073–2080 (2024)
24. Wu, Y., Kirillov, A., Massa, F., Lo, W.-Y., Girshick, R.: Detectron2. https://github.com/facebookresearch/detectron2. Accessed 08 July 2025
25. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016)
26. Huang, W., Wang, C., Zhang, R., Li, Y., Wu, J., Fei-Fei, L.: VoxPoser: composable 3D value maps for robotic manipulation with language models. In: Conference on Robot Learning (CoRL) (2023)
27. Karaman, S., Frazzoli, E.: Sampling-based algorithms for optimal motion planning. Int. J. Robot. Res. **30**(7), 846–894 (2011)
28. Ergüven, E., Polat, F.: Relative distances approach for multi-traveling salesmen problem. Knowl.-Based Syst. 112160 (2024)
29. Das Gupta, S., Van Parys, B.P., Ryu, E.K.: Branch-and-bound performance estimation programming: a unified methodology for constructing optimal optimization methods. Math. Program. **204**(1), 567–639 (2024)
30. Li, X., Zhang, S., Shao, P.: Discrete artificial bee colony algorithm with fixed neighborhood search for traveling salesman problem. Eng. Appl. Artif. Intell. **131**, 107816 (2024)
31. Sucan, I.A., Moll, M., Kavraki, L.E.: The open motion planning library. IEEE Robot. Autom. Mag. **19**(4), 72–82 (2012)
32. Yao, M., Deng, H., Feng, X., Li, P., Li, Y., Liu, H.: Improved dynamic windows approach based on energy consumption management and fuzzy logic control for local path planning of mobile robots. Comput. Ind. Eng. **187**, 109767 (2024)
33. Yang, K., Cao, N., Ding, Y., Chen, C.: Bestman: a modular mobile manipulator platform for embodied AI with unified simulation-hardware APIs. Front. Comput. Sci. (2024)
34. Andrew, S., et al.: Habitat rearrangement challenge 2022. In: Conference and Workshop on Neural Information Processing Systems (NeurIPS), vol. 34, pp. 251–266 (2021)
35. Brazil, G., Kumar, A., Straub, J., Ravi, N., Johnson, J., Gkioxari, G.: Omni3D: a large benchmark and model for 3D object detection in the wild. In: Conference on Computer Vision and Pattern Recognition (CVPR), pp. 13154–13164 (2023)

Distributed Intelligent Computing with Kernel-Wise Deep Neural Network Partition over Collaborative Devices

Boya Sun, Yinglei Teng(✉), and Nan Wang

Beijing Key Laboratory of Space-ground Interconnection and Convergence, Beijing University of Posts and Telecommunications (BUPT), Xitucheng Road NO. 10, Beijing, China
{boyasun,lilytengtt,wangnan_26}@bupt.edu.cn

Abstract. With the proliferation of artificial intelligence (AI) applications, model partitioning has attracted increasing attention for addressing the computational bottlenecks of edge AI by offloading portions of deep neural networks (DNNs) to cloud servers or IoT devices. However, existing DNN partitioning methods often incur significant parameter overhead during deployment. Moreover, achieving both low-latency inference and privacy preservation in Internet of Things (IoT) networks remains challenging due to device heterogeneity and the heightened privacy risks of distributed architectures. In this work, we propose a reliable distributed computing framework based on fine-grained convolutional kernel partitioning, enabling collaborative execution among multiple edge devices (EDs). To accelerate inference while balancing power and reliability constraints, we jointly optimize kernel allocation, redundancy strategies, and dynamic resource configuration. To address the high coupling of time-scale discrepancies and mixed-variable characteristics, we first apply the Lagrange multiplier method for problem transformation, and then design a two-timescale Hierarchical Proximal Policy Optimization (HPPO) algorithm decoupling the transformed problem into two-layer subproblems. Experimental results show that the proposed framework significantly improves inference speed while ensuring computational reliability in distributed edge environments.

Keywords: DNN partition · Cooperative inference · Distributed computing system · Hierarchical reinforcement learning

1 Introduction

Intelligent services based on deep learning have emerged alongside the development of artificial intelligence (AI) and the rapid growth of Internet of Things (IoT) devices at the network edge across various application domains, e.g., smart home, video surveillance, and healthcare [1–3]. The traditional centralized cloud computing paradigm offloads AI-based tasks to cloud servers with high computational capabilities. However, due to the explosion of big data, massive deep

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 271–290, 2026.
https://doi.org/10.1007/978-981-95-8405-5_15

neural network (DNN) service requests may overload of the network and the cloud server, making it difficult to meet the requirements of upcoming sixth-generation(6G) IoT systems. Therefore, a shift toward distributed edge computing, which enables collaboration between the cloud and the IoT devices, offers a promising approach to addressing these challenges by leveraging the complementary advantages of both edge computing and cloud computing [4–6].

The IoT devices with available resources at the edge of the network in crowdsourcing scenarios have spawned research on distributed computing for intelligent services [7]. Specifically, splitting model with appropriate offload schemes is a feasible solution to make the most of idle resources at the devices. To adapt to the limited computing and storage capacities of IoT devices, numerous studies have focused on identifying partition points in DNN models for offloading, and on designing inference frameworks to enhance AI service performance, such as reducing latency and energy consumption. According to research studies [8–10], fine-grained model partitioning methods that divide each layer of a DNN into multiple segments enables the partitioned models to execute in parallel on different devices. Compared with the approaches using a single model partition point [11–15], fine-grained partitioning accelerates inference through parallelization. Inspired by the atomicity and independence of the kernel computation, Zhi *et al.*. [16] propose a kernel-wise partition method (KPM) in which the convolutional kernels of each convolutional layer are segmented into multiple partitions. With the input feature map, the inference of a single layer can be divided into distributed collaborative computations of multiple subtasks. KPM leverages the independence of convolution operations between kernels, providing a private and secure solution for collaborative inference. By partitioning and offloading convolutional kernels, each collaborative node generates only partial feature map information. This method retains most of the information even when parts of the feature map results are missing. Thus, KPM offers a distributed inference option that supports collaborative computing while ensuring data privacy and security.

In addition, computing reliability is considered to represent the likelihood of the edge computing system operating normally be an important factor in the distributed collaboration scenario. However, a large number of computing nodes involved face the key threads of node failure or a service failure, which may lead to a complete failure or degradation of the overall system. As a general paradigm, fault tolerance technology can ensure the data integrity and system availability, avoid data loss and the breakdown of the whole system caused by local failures. Hence, the intelligent tasks are supposed to be executed separately over the available devices in parallel to improve the efficiency of the services, with joint consideration of computation amount, power consumption, and wireless communication traffic. At present, the coordination of distributed computing management for edge intelligent services remains an area that require substantial exploration. Specifically, there is a lack of a coordinated computing framework for joint DNN model fine-grained partitioning and reliable computation to enhance the execution efficiency of AI tasks and system robustness.

With KPM as the underlying support, in this paper, we propose a reliable distributed computing framework and establish a collaborative procedure among multiple edge devices (EDs). To accelerate inference while balancing power consumption and reliability constraints, we jointly optimize kernel allocation, redundancy decisions, and dynamic resource configuration. To solve this problem, we employ the Lagrange multiplier method for problem transformation and propose a hierarchical proximal policy optimization (HPPO) algorithm for solution. Experimental results demonstrate that the proposed framework effectively reduces the average delay while ensuring the reliability of the distributed computing system.

The rest of the paper is organized as follows. In Sect. 2, the system model will be described. Section 3 and Sect. 4 provide a joint optimization problem and the HPPO algorithm, respectively. Section 5 provides experimental results. Finally, this paper is concluded in Sect. 6.

2 System Model

We consider a distributed computing system establishing upon an edge-cloud collaborative architecture capable of supporting intelligent computing applications. Specifically, a set $\mathcal{M} = \{1, 2, ..., M\}$ is used to denote the heterogenous EDs that voluntarily participate in crowd computing, each with a cascade of two buffers. The cloud server (CS) is responsible for splitting the intelligent services and managing the cooperation executions of the EDs. Taking the popular DNN-type applications as target tasks, the neural network model is partitioned into portable computational elements based on KPM. Moreover, network resource is dynamically adjusted to accelerate distributed feature extractions of input tasks across EDs with respect to the queue states, communication conditions and power consumption. In our system, time is slotted with duration τ, and the time slots are indexed by $t \in \mathcal{T} = \{1, 2, 3...\}$. The computational frequency and channel conditions remain constant within the period of one time slot but fluctuates separately across various periods.

2.1 Distributed Inference Procedure

Taking the popular inference service of two-dimensional convolutional neural networks (2DCNNs) as an instance, the distributed computing procedure with multi-EDs for collaborative computing is depicted in Fig. 1. For the distributed computing of convolutional layer l ($\mathbf{Conv}l$) , $l \in \mathcal{L} = \{1, 2, ...L\}$, the redundancy technology of replication for convolutional kernels is adopted to cope with the inevitable failures of the participating EDs and support the achievement of acceptable levels of computational reliability. By exploiting the KPM, the CS develops a partition strategy to schedule the assignment of kernels, where each kernel is assigned to multiple distinct EDs. Embedded with the respective part of the CNN model, the EDs will collaborate to achieve distributed computing, which encompasses the following three phases.

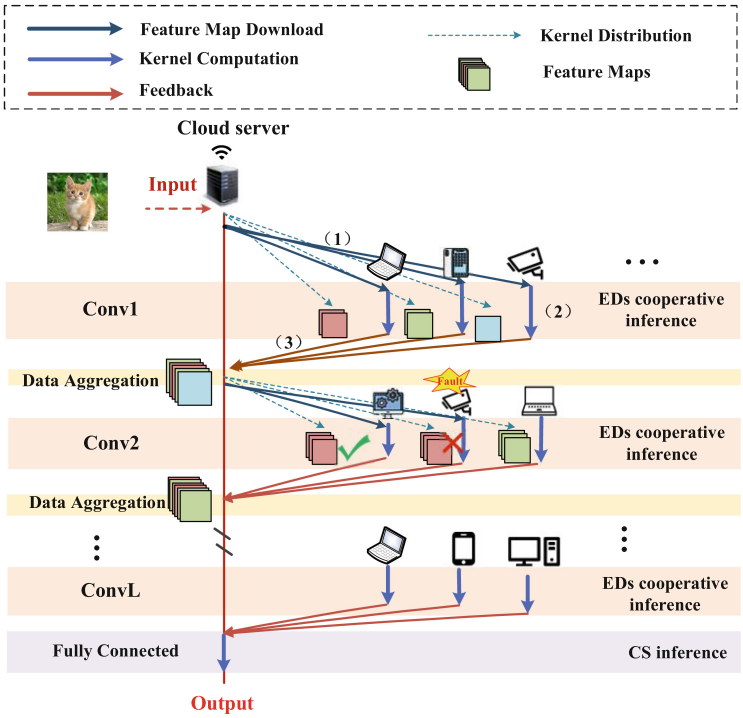


Fig. 1. The distributed computing procedure based on KPM

(1) **Feature map downloading phase:** The three-channel feature maps with the same size are required as the input intelligent tasks by the EDs. To send the tasks efficiently, each task is transmitted from CS via broadcast.
(2) **Convolutional kernel computing phase:** The input tasks are queued up for calculation in the computing buffer at each ED. As the feature extractor that provides the computed weights, the convolutional kernels embedded at each ED perform complex matrix computations with the input tasks in the computation queue. Different numbers of convolutional kernels on various EDs extract features from input tasks to varying degrees through parallel computing. Supported by the available computing resources, each ED generates a portion of output channels for each input task.
(3) **Result feedback phase:** The computed outputs are queued in the communication buffer at each ED for sending back to CS, at which the separated channels from the cooperative EDs are aggregated and each feature channel will be verified through the majority voting among the redundancies. Note that the verification time of the result can be neglected due to the powerful computing ability. Repeating the partition and aggregation process layer by layer, the final output is obtained after the fully-connected layer at CS.

The inherent data dependency in CNN models, where the output feature maps of a preceding layer serve as the input to the subsequent layer, implies that the distributed execution of convolutional layers must occur sequentially. However, since the kernel allocation optimization is similar across layers, we focus on the distributed computation of a single representative layer in the following sections. For simplicity, we omit the layer index l throughout the remainder of the paper, as the modeling is identical for each layer.

2.2 Kernel-Wise Partition Model

To quantify the neural network layer, a four-dimensional tuple $\{S^I, S^O, N, W\}$, is adopted, where S^I and S^O represent the input and output data size, and N is the number of input/output channels, equal to the total number of convolutional kernel at the current layer. In addition, W is the floating-point operations (FLOPs) that can be calculated by the formulas in [17]. With the fine-grained KPM, each layer of the specific CNN model can be split into multiple partitions for distributed computing. Define $\mathcal{N} = \{1, 2, ..., N\}$ as the kernel set and let $x_{n,m}$ denote the kernel allocation indicator, where $x_{n,m} = 1$ represents the n-th kernel is deployed in ED m, and vice versa. Thus, the kernel partition matrix can be given by $\boldsymbol{X} = \{x_{n,m} | x_{n,m} = \{0, 1\}, n \in \mathcal{N}, m \in \mathcal{M}\}$. With the redundant strategy $\rho \in \boldsymbol{\rho} = \{\rho \mid \rho > 2, \rho \in \mathbb{Z}^+\}$ for each kernel, the deployment strategy satisfies the following constraint:

$$\sum_{m \in \mathcal{M}} x_{n,m} = \rho, \forall n \in \mathcal{N}. \tag{1}$$

Specially, the data size of the input task at ED m is identical to the layer input size, i.e., $S_m^I = S^I$. However, the result of each task at ED m is correlated with the kernel allocation strategy that determines the number of kernels allocated to ED m. Intuitively, the output data size of ED m can be expressed as:

$$S_m^O = \frac{\sum\limits_{n \in \mathcal{N}} x_{n,m}}{N} S^O. \tag{2}$$

Similarly, the floating-point operations for each task at ED m is proportional to the number of assigned kernels, i.e.,

$$W_m = \frac{\sum\limits_{n \in \mathcal{N}} x_{n,m}}{N} W. \tag{3}$$

Thus, the properties of the tasks at ED m can be described as a three-tuple notation $\{S_m^I, S_m^O, W_m\}$.

2.3 Queue Models

To support the distributed computing, each ED m downloads the input task from CS and uploads the result back to CS. Specifically, the computation queue

is fed by the broadcasted tasks and the upload queue is drained by uploading results. In addition, the two queues are cascaded by the computation process, because the departing computed tasks of the former is the arriving results to be uploaded of the latter. The task arrival rate, task computing rate and task results upload rate of the cascaded queues are defined for further description of queue dynamics.

Task Arrival Rate. The CS distributes the input tasks to cooperative nodes via broadcast. With the bandwidth b^D that used to broadcast the input feature maps, the download rate (in bit/s) from the CS to each ED m, $m \in \mathcal{M}$, can be represented as:

$$r_m^D(t) = b^D \log_2\left(1 + \frac{p^D h_m^D(t)\, d_m^{-\varsigma}}{N^0 + \sigma_m^2}\right), \tag{4}$$

where p^D denotes the transmit power from CS to each ED, and $h_m^D(t)$ is the channel gain from the CS to ED m in the t-th time slot. N^0 denotes the interference generated from other service areas, and σ_m^2 is the noise power at ED m in the download phase. Additionally, d_m denotes the distance between the CS and ED m, and ς is the standard path loss propagation exponent. Denote $\lfloor \cdot \rfloor$ as the floor function. Since the downlink rate of each task in the broadcast is constrained by the minimum data transmission rate of the EDs, the number of tasks received by the EDs in the t-th time slot can be expressed as:

$$A(t) = \left\lfloor \tau \frac{\min\limits_{m \in \mathcal{M}} \left\{ r_m^D(t) \right\}}{S^I} \right\rfloor. \tag{5}$$

Task Computing Rate. Inherently, the convolutional kernels embedded at the EDs are pivotal feature extractors, providing the weight parameters for computation. Moreover, due to the time-varying nature of task arrivals and result uploads within each time slot, the computing rate must be dynamically managed. It is assumed that the computational frequency at each ED is adjusted dynamically in every time slot. Let $f_m(t) \in \boldsymbol{F} = \{ f_m(t) \,|\, f_m(t) \in [0, f^{\max}], m \in \mathcal{M} \}$ denote the computational frequency of ED m in the time slot t, and Y_m represents the number of computing unit cycles required to complete a floating-point of ED m. The average number of tasks computed by ED m in the t-th time slot can be given by:

$$\mu_m(t) = \frac{f_m(t)}{Y_m W_m} \tau. \tag{6}$$

Additionally, the computational power consumption of the partition m assigned to ED m cab be described as follows according to [18],

$$p_m^C(t) = \kappa_m f_m(t)^3, \tag{7}$$

where κ_m is a constant related to the hardware configuration of ED m.

Task Result Upload Rate. The orthogonal frequency-division multiple access (OFDMA) is used to divide the bandwidth into multiple sub-bands with the bandwidth b^U to avoid interference in the feedback phase. The upload rate (in bit/s) of ED m in the time slot t can be expressed as:

$$r_m^U(t) = b^U \log_2\left(1 + \frac{p_m^T h_m^U(t)\, d_m^{-\varsigma}}{\sigma^2}\right), \tag{8}$$

where p_m^T is the transmission power of ED m and $h_m^U(t)$ is the channel gain from the ED m to CS in the t-th time slot. The number of tasks uploaded by ED m in the t-th time slot can be expressed as:

$$\eta_m(t) = \left\lfloor \tau \frac{r_m^U(t)}{S_m^O} \right\rfloor. \tag{9}$$

Queue Dynamics. Denote $Q_m^C(t)$ and $Q_m^U(t)$ as the length of computing queue and upload queue of ED m at the beginning of t-th time slot, respectively. With the task arrival rate and the computing rate, the dynamics of the computation queue backlog is:

$$Q_m^C(t+1) = \max\left\{Q_m^C(t) - \mu_m(t), 0\right\} + A(t). \tag{10}$$

Similarly, with the result upload rate, the upload queue at ED m evolves as:

$$Q_m^U(t+1) = \max\left\{Q_m^U(t) - \eta_m(t), 0\right\} + \min\left\{\mu_m(t), Q_m^C(t)\right\}. \tag{11}$$

2.4 Convolutional Kernel Computational Reliability Model

Generally, it is imperative that the distributed system be able to withstand failures since the participating EDs are not subjected to significant dependability checking and failure-prone nodes can easily join the system. The ρ-modular redundancy scheme [19] supports to establish a reliable distributed computing environment by assigning each convolutional kernel to different EDs. When the results are aggregated, the CS produces the majority vote on the redundant output feature maps to verify the correctness of the parallel-processing results. To be specific, the redundancy channel result of each kernel reaching at least some minimum number of agreements is taken to be the solution.

Assume that all EDs are independent, and the failure occurrence follows the exponential distribution with the failure rate ν_m of ED m. Denote the normal working time of ED m before performing distributed computing as T_m^0 . During the work interval of ED m, the probability of failures occurring can be formulated as:

$$Pr_m = 1 - e^{-\nu_m T_m^0}. \tag{12}$$

It should be added that Eq. 12 also indicates that the kernels embedded at ED m have the same failure rate Pr_m.

Even while some EDs may be more dependable than others, each redundant copy of the convolutional kernel offloaded to the device cluster is randomly assigned to the EDs specified by the convolution kernel offloading scheme. Therefore, every redundant copy is subject to the same risk of failing validation. The probability of unsuccessfully executing a backup of kernel n is

$$Pr_n = 1 - \frac{1}{\rho} \sum_{m \in \mathcal{M}} x_{n,m} \left(1 - Pr_m\right). \tag{13}$$

Under the majority mechanism, the probability that CS receives a reliable feature map result corresponding to the convolutional kernel n, equal to the computation reliability of kernel n provided by the distributed system, can be formulated as:

$$R_n = \sum_{q=\left\lceil \frac{\rho+1}{2} \right\rceil}^{\rho} \binom{\rho}{q} (1 - Pr_n)^q (Pr_n)^{\rho - q}. \tag{14}$$

where $\left\lceil \frac{\rho+1}{2} \right\rceil$ is regarded as the minimum number of matching results required for the voting [20].

2.5 Average Delay

The total delay of tasks at each ED comprises queue waiting delay, download delay, computation delay and upload delay. However, for most tasks that arrive at the EDs, download process overlaps with the execution of the previously queued task. In the long-term perspective, the additional download delay is approximately equal to the duration of the first task's transmission over the air interface, and is therefore negligible.

Consequently, the delay of each task at ED m can be equivalent to the time spent in the queues(including queue waiting delay, computation delay and upload delay). According to Little's Law [21], the average delay spent in the queues at ED m is proportional to average length in the queues, i.e.,

$$T_m = \tau \lim_{T \to +\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\varphi\left(Q_m(t)\right)\right], \tag{15}$$

where $\varphi\left(Q_m\right) = \frac{Q_m^C + Q_m^U}{A_m}$.

Since each ED only computes a subset of the output feature maps, it is necessary to perform synchronization operations among the collaborative EDs to obtain the entire feature map results. Through the Bulk Synchronous Parallel (BSP) mechanism [22], the average delay of a single layer is determined by the maximum average delay among all EDs. Moreover, a maximum allowable delay, $T^{\max}$ is set as a threshold to determine if the convolutional layer needs reprocessing to ensure the timeliness of the distributed computing service. That is,

$$T^P = \max_{m \in \mathcal{M}} \left(T_m\right), T^P < T^{\max}. \tag{16}$$

3 Problem Formulation

In this section, we formulate a minimization problem for average delay, which jointly considers convolutional kernel partitioning, resource scheduling and redundancy strategy. Additionally, we develop constraints based on kernel-level computation reliability and ED power consumption to enhance system performance and ensure utilization of distributed edge resources.

First, the cooperative inference system must satisfy the following probabilistic constraint to ensure reliability for each convolutional kernel:

$$R_n \geq 1 - \varepsilon_0, \tag{17}$$

where ε_0 is a small positive number, indicating a strict reliability requirement for task processing.

Second, power consumption should be constrained due to the resource limitation of the voluntary EDs. For ED m, the power consumption mainly consists of the computation power and transmission power for uploading results. The average total power consumption per time slot for collaborative ED m can be defined as:

$$p_m = \lim_{T \to +\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[p_m^C(t)\right] + p_m^T. \tag{18}$$

To minimize the average delay while satisfying reliability and power consumption constraints, we formulate the joint optimization problem of convolutional kernel partitioning, redundancy strategy, and resource allocation as $\mathcal{P}0$:

$$\begin{aligned} \mathcal{P}0 : & \min_{\boldsymbol{X},\rho,\boldsymbol{F}} T^P, \\ & s.t. C_1 : x_{n,m} = \{0,1\}, \forall n, \forall m, \\ & C_2 : \sum_{m \in \mathcal{M}} x_{n,m} = \rho, \forall n, \\ & C_3 : f_m(t) \in [0, f_m^{\max}], \forall m, \\ & C_4 : p_m \leq p_m^{\max}, \forall m, \\ & C_5 : R_n \geq 1 - \varepsilon_0, \forall n, \end{aligned} \tag{19}$$

where ρ is the redundancy decision positive integer. Constraint C_1 and C_3 provide feasible region for the partition and frequency scheduling variables, respectively. Constraint C_2 restricts the number of redundancy for each kernel. Constraint C_4 and C_5 are proposed to satisfy power consumption and reliability requirements of the system, where $p_m^{\max}$ is the power consumption upper bound for each ED m ,$m \in \mathcal{M}$.

Obviously, $\mathcal{P}0$ represents a highly complex stochastic optimization problem involving both discrete and continuous variables. In addition, there is no closed-form expression for the problem due to the time-varying state of the queue and the long-term average constraint on power. In particular, the queue dynamics

introduce strong coupling among multiple decision variables. Moreover, the solutions to the problem involve short-term dynamic change of computing frequency and long-term joint decisions on convolutional kernel allocation strategy and redundant strategy, forming a two-timescale stochastic optimization problem. To make it solvable, we first transform the objective function by incorporating Lagrange multipliers. We then apply a hierarchical reinforcement learning (HRL) algorithm operating at two time scales to efficiently solve the minimization problem.

4 Optimization Algorithm

Considering the time-scale separation and temporal correlation characteristics of Problem $\mathcal{P}0$, the HPPO algorithm is designed to solve it. To handle the complex power consumption constraint, Lagrange multipliers (LMs) are introduced to convert the constrained Markov Decision Process (CMDP) $\mathcal{P}0$ into an infinite horizon Markov Decision Process (MDP). Subsequently, a hybrid time-scale reinforcement learning method is proposed to address the temporal discrepancies among multiple decision variables.

4.1 Problem Transformation

For the long-term average power consumption constraint C_4, which lacks a closed-form expression, it is not possible to derive an explicit analytical relationship between the constraint and the decision variables. Additionally, due to the infinite time-horizon nature of the long-term average, the constraint cannot be accurately verified within a finite number of iterations. In our work, we incorporate this constraint into the objective function via LMs for handling non-closed-form characteristic. For any LM γ , we can formulate the Lagrangian function as:

$$G(\boldsymbol{X}, \rho, \boldsymbol{F}, \boldsymbol{\gamma}) = T^P + \lim_{T \to +\infty} \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}\left[\mu\big(\gamma(t), f(t)\big)\right]. \tag{20}$$

where $\mu(\gamma, f) = \sum_{m \in \mathcal{M}} \gamma_m \left(p_m^C - \left(p_m^{\max} - p_m^T\right)\right)$. It should be noted that we need to adjust the value of the LM, γ in each time slot to minimize G while ensuring the power consumption constraint. Thus, the optimization problem $\mathcal{P}0$ can be equivalently transformed into the minimization problem as follows:

$$\begin{aligned} \mathcal{P}1: & \min_{\boldsymbol{X}, \rho, \boldsymbol{F}, \boldsymbol{\gamma}} G(\boldsymbol{X}, \rho, \boldsymbol{F}, \boldsymbol{\gamma}), \\ & s.t. C_1, C_2, C_3, C_5. \end{aligned} \tag{21}$$

4.2 HPPO Algorithm

Inspired by option-based reinforcement learning approaches that allow the policy to find options over a period of time and to optimize within options on discrete time slots, we adopt a HRL method to address the reformulated problem $\mathcal{P}1$. Compared with the traditional reinforcement learning methods, two agents consisting of upper-level agent and lower-level agent respectively responsible for the optimization over different time scales are employed. With the hierarchy introduced in the algorithm, HRL enables complex decision-making across different timescales more efficiently.

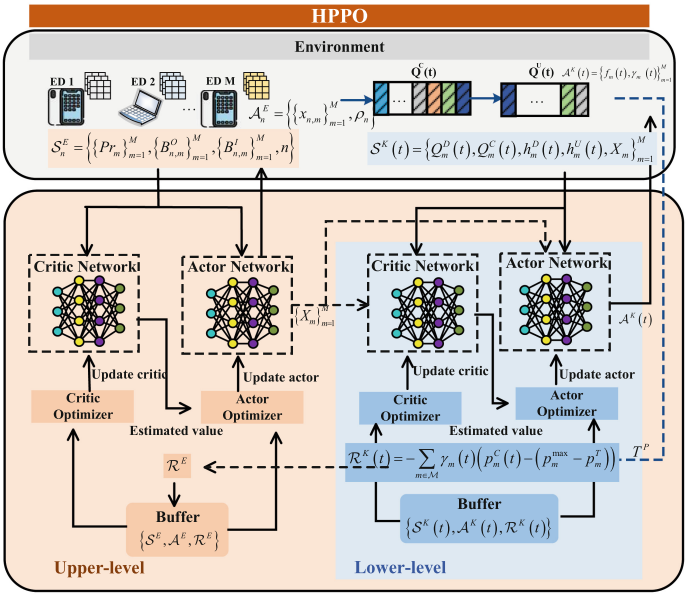


Fig. 2. The architecture of HPPO algorithm

In our work, we leverage Proximal Policy Optimization (PPO) [23], an advanced deep reinforcement learning (DRL) algorithm that combines the strengths of value-based and policy-based methods and efficiently handles both discrete and continuous action spaces, to solve the optimization problem. As shown in Fig. 2, there are upper-level agent, lower-level agent and the environment in our HPPO algorithm architecture. Although the optimization problem $\mathcal{P}1$ is entangled with a hybrid action space consisting of both discrete actions in the long-term and continuous action in the short-term, our HRL method avoids directly handling this hybrid action spaces by decoupling the decisions. The lower-level agent performs the short-term optimization in the time slots of each long-term optimization, dynamically adjusting the computational resource allocation and the LM decisions $\{\boldsymbol{F}, \boldsymbol{\gamma}\}$ based on the queue state and channel

state. On the other hand, the upper-level agent develops the long-term policy including kernel partition and the redundancy decisions $\{\boldsymbol{X}, \rho\}$ to minimize the average delay while guaranteeing the power consumption constraint. In particular, the upper-level agent selects the long-term action on the slow timescale, while the lower-level agent formulates the short-term policy on the fast timescale within T time slots. The interleaved optimization across various timescales are performed repeatedly until the training converges. Let π_θ denote the actor network for approximating the policy function and V_ω denote the critic network for approximating the value function. Note that θ and ω represent the parameters of actor network and critic network, respectively. The detail design of our proposed algorithm is shown in Algorithm 1. To transform the problem $\mathcal{P}1$ into HPPO notations, the following MDP is defined for both the upper-level agent and lower-level agent.

Algorithm 1. HPPO Algorithm

Initialize: Upper-level and lower-level actor networks π_{θ^E}, π_{θ^K} with weights θ^E,θ^K. Upper-level and lower-level critic networks V_{ω^E},V_{ω^K} with weights ω^E, ω^K. Set upper-level and lower-level actor networks' learning rate α^E, α^K, critic networks' learning rate β^E, β^E, replay buffers $\mathcal{D}^E$, $\mathcal{D}^K$, and discount factors ς^E, ς^K.
for episode $= 1, 2, \ldots, Z$ **do**
  Reset the outer environment to get the initial long-term state $\mathcal{S}_1^E$
  for $n = 1, 2, \ldots, N$ **do**
    Receive current state $\mathcal{S}_n^E$
    Select $\mathcal{A}_n^E$ of kernel n from current π_{θ^U}
    if $n\%N \neq 0$ **then**
      Obtain $\mathcal{R}_n^E$, $\mathcal{S}_n^E$ by interacting with the outer environment
      Store $(\mathcal{A}_n^E, \mathcal{S}_n^E, \mathcal{R}_n^E, \mathcal{S}_{n+1}^E)$ into $\mathcal{D}^E$
    end if
    if $n\%N == 0$ **then**
      Receive the lower-level state $\mathcal{S}^K$
      Execute the lower-level optimization using PPO
      Obtain $\mathcal{R}_n^E$, $\mathcal{S}_(n+1)^E$
      Store $(\mathcal{A}_n^E, \mathcal{S}_n^E, \mathcal{R}_n^E, \mathcal{S}_{n+1}^E)$ into $\mathcal{D}^E$
    end if
    if $n\%\mathcal{D}^E == 0$ **then**
      for epoch $= 1, 2, \ldots,$ epochs **do**
        Compute advantage function $\hat{A}_{\theta^E}$
        for batch $= 1, 2, \ldots,$ batches **do**
          Calculate the estimated gradients $\nabla_{\theta^E} x^E$ and $\nabla_{\omega^E} y^E$
          Update the actor network of upper agent: $\theta^E \leftarrow \theta^E + \alpha^E \nabla_{\theta^E} x^E$
          Update the critic network of upper agent: $\omega^E \leftarrow \omega^E - \beta^E \nabla_{\omega^E} y^E$
        end for
      end for
    end if
  end for
  Clear the replay buffer $\mathcal{D}^E$
end for

Long-Term Optimization. The long-term optimization contains the kernel partition strategy and the redundancy decision developed by the upper-level agent. The kernel allocation decision $\boldsymbol{X}$ is a binary $n \times m$ two-dimensional matrix, where each element indicates assignment (1) or non-assignment (0). However, the size of action space grows exponentially with the number of convolutional kernels and EDs, making efficient exploration intractable. In order to reduce the long-term action space, we make allocation decisions for each convolutional kernel sequentially over time, rather than making all allocation decisions simultaneously in a spatially unified manner. By transforming the spatial complexity of kernel allocation decision into temporal complexity through sequential kernel-wise decisions, we effectively reduce the dimensionality of the high-dimensional action space. The state space, action space and reward function are defined as follows:

(1) **State space:** The required state information $\mathcal{S}_n^E$ contains the reliability of each ED m, $m \in \mathcal{M}$, the allocation state of n-th convolutional kernel, and index n.

$$\mathcal{S}_n^E = \left\{ \{Pr_m\}_{m=1}^M, \left\{B_{n,m}^O\right\}_{m=1}^M, \left\{B_{n,m}^I\right\}_{m=1}^M, n \right\}, \tag{22}$$

where $B_{n,m}^O = \frac{\sum_{j=1}^{n} x_{j,m}}{N}$ indicates the allocated ratio of convolutional kernels from the first n kernels to ED m. Let $\mathbf{1}_{\{.\}}$ denotes the indicator function, which takes the value 1 if the condition is satisfied and 0 otherwise, and $B_{n,m}^I = \mathbf{1}_{\left\{B_{n,m}^O \neq 0\right\}}$ represents whether ED m has been assigned convolutional kernels.

(2) **Action space:** The upper-level agent selects appropriate redundant value and makes assignment for each kernel n, $n \in \mathcal{N}$.

$$\mathcal{A}_n^E = \left\{ \{x_{n,m}\}_{m=1}^M, \rho_n \right\}. \tag{23}$$

(3) **Reward function:** The upper-level agent is responsible for the overall performance of the whole system. With the kernel-based decision structure, the reward function of the upper-level agent is defined in a piece-wise manner. Obviously, the kernel allocation strategy is incomplete when the current kernel is not equivalent to the length of the kernel period, and the lower-level agent is inactive. In this case, the reward function consists of two components: the difference between the current kernel's reliability level and the reliability constraint, and the absolute difference between the redundancy factor ρ of the current kernel and that of the previous kernel. Note that the second component is omitted when the current kernel is the first decision kernel. This encourages consistent redundancy allocation across kernels while ensuring that the reliability constraint is satisfied. Conversely, when the current kernel index reaches the full kernel period length, the reward function is defined as a joint reward, combining the cumulative intrinsic rewards from the lower-level agent (under the current long-term option) and

a reward based on the average delay resulting from decisions made during the kernel period.

$$\mathcal{R}_n^E = \begin{cases} \frac{1}{T}\sum_{t=1}^{T}\mathcal{R}^K(t) - T^P, n = N. \\ R_n - (1-\varepsilon_0), n = 1. \\ R_n - (1-\varepsilon_0) - |\rho_n - \rho_{n-1}|, \text{otherwise.} \end{cases} \tag{24}$$

Short-Term Optimization. The short-term optimization contains the computing frequency and LM adjustments of the lower-level agent. Note that the termination condition of the option in HPPO depends on the convergence of the short-term PPO network. The algorithm of short-term optimization via PPO is shown in Algorithm 2. The state space, action space and reward function are defined as follows:

(1) **State space:** The short-term optimization state includes two parts. One is the computing queue state, the upload queue state, the downlink channel gain and the uplink channel gain in each time slot. The other is the kernel allocation state determined by the upper-level agent.

$$\mathcal{S}^K(t) = \left\{Q_m^C(t), Q_m^U(t), h_m^D(t), h_m^U(t), X_m\right\}_{m=1}^{M}, \tag{25}$$

where X_m is the sum of columns of the matrix $\boldsymbol{X}$, representing the number of convolutional kernels on ED m.

(2) **Action space:** For each ED m,$m \in \mathcal{M}$, the computation frequency is scheduled by the lower-level agent in each time slot with X_m embedded kernels. Besides, the LM matrix $\boldsymbol{\gamma} = \{\gamma_m(t) | \gamma_m(t) \in (0, \gamma^{\max}], m \in \mathcal{M}\}$ must be incorporated into the joint short-term decision-making process for dynamic adjustment to ensure that power consumption constraints are met.

$$\mathcal{A}^K(t) = \{f_m(t), \gamma_m(t)\}_{m=1}^{M}. \tag{26}$$

(3) **Reward function:** The learning objective of the low-level agent is to minimize the penalty term in the Lagrangian function associated with LMs and the power consumption constraint, namely:

$$\mathcal{R}^K(t)\left(\mathcal{S}^K(t), \mathcal{A}^K(t)\right) = -\sum_{m\in\mathcal{M}}\gamma_m(t)\left(p_m^C(t) - \left(p_m^{\max} - p_m^T\right)\right). \tag{27}$$

Algorithm 2. Short-term Optimization via PPO

for $t = 1, 2, \ldots, T$ **do**
  Receive current state $\mathcal{S}^K(t)$
  Select continuous action $\mathcal{A}^K(t)$ from current π_{θ^K}
  Obtain $\mathcal{R}^K(t)$ and $\mathcal{S}^K(t+1)$ by interacting with the inner environment
  Store $(\mathcal{S}^K(t), \mathcal{A}^K(t), \mathcal{R}^K(t), \mathcal{S}^K(t+1))$ into $\mathcal{D}^K$
  if $t\%\mathcal{D}^K == 0$ **then**
    for epoch $= 1, 2, \ldots,$ epochs **do**
      Compute advantage function $\hat{A}_{\theta^K}$
      Calculate the estimated gradients $\nabla_{\theta^K} x^K$ and $\nabla_{\omega^K} y^K$
      Update the actor network: $\theta^K \leftarrow \theta^K + \alpha^K \nabla_{\theta^K} x^K$
      Update the critic network: $\omega^K \leftarrow \omega^K - \beta^K \nabla_{\omega^K} y^K$
    end for
  end if
end for

5 Experiment Results and Discussion

In this section, we will test the performance of the proposed distributed computing framework under different parameter settings. In our experiments, we first train the Alexnet model for 100 epochs on the Cifar-10 dataset. We then focus on the partition and distributed computation of convolutional kernels in the fourth convolutional layer of AlexNet network. Based on the trained model, 384 convolutional kernels are allocated to 8, 9, 10, 11, or 12 heterogeneous EDs for distributed computation. All code is implemented in Python3.9 and executed on a Linux server equipped with one NVIDIA A40 GPU.

5.1 Experiment Setup

The system downlink bandwidth is $b^D = 1.8MHz$, and the uplink bandwidth is $b^U = 0.5MHz$. The transmit power of the CS is $p^D = 0.1W$, and the upload transmit power is $p_m^U = 1W$ for each ED m. Note that the experiments focus specifically on average computational power consumption per ED. For brevity, the term "average power" used in subsequent figures and discussions refers exclusively to average computational power. Other parameters are listed in Table 1.

5.2 Performance of the Proposed Framework

In Fig. 3, we plot the convergence performance of our proposed scheme. It can be seen that the proposed scheme converges after 300 episodes with different delay deadlines. Note that the decreasing of $T^{\max}$ leads to the decline in the learning performance of the agent. This is because adjusting the time threshold can reshape the statistical characteristics of rewards, thereby influencing the exploration-exploitation balance in reinforcement learning and leading to different convergence speeds. Furthermore, compared with more lenient delay deadlines tighter time constraint causes the system to reduce the obtained rewards while ensuring the minimization of average delay.

Table 1. Simulation Parameters.

Parameter	Values
Computational frequency of ED m, f_m	[0.5,1.5] GHz
Upper limit value of LM, $\gamma^{\max}$	$(0, 100]$
Power consumption upper bound, $p_m^{\max}$	3.2 W
Reliability requirement,$1-\varepsilon_0$	0.8
Time slot duration, τ	50 ms
average delay deadline, $T^{\max}$	1000 ms
Input, output data size, S^I, S^O	519168bit
Average SNR of the upload wireless channel, SNR	20 dB
FLOPs, W	222382080
Learning rate of actor networks, $\{\alpha^E, \alpha^K\}$	$\{2\times10^{-4}, 3\times10^{-4}\}$
Learning rate of critic networks, $\{\beta^E, \beta^K\}$	$\{1\times10^{-3}, 3\times10^{-3}\}$
Discount Factors, ς^E,ς^K	0.99, 0.99
Replay buffer,$\mathcal{D}^E$, $\mathcal{D}^K$	384,20

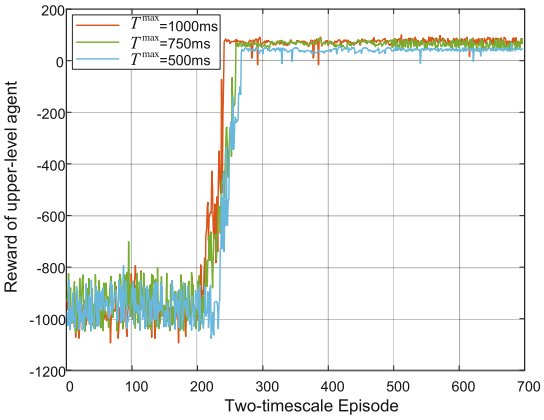


Fig. 3. Convergence performance of HPPO with different delay deadlines

Figure 4 illustrates the average power consumption per ED and the delay per convolutional layer under various value of $\gamma^{\max}$. In the aforementioned Lagrangian function, γ plays an important role in balancing the trade-off between average delay and average power consumption. When $\gamma^{\max}$ is less than 100, both the average power consumption and the average delay decrease as $\gamma^{\max}$ increases. However, when $\gamma^{\max}$ exceeds 100, the average power consumption decreases but the average delay increases. This behavior likely occurs because small adjustments to $\gamma^{\max}$ are insufficient to consistently prioritize delay performance optimization. In our proposed scheme, meeting the power constraint may

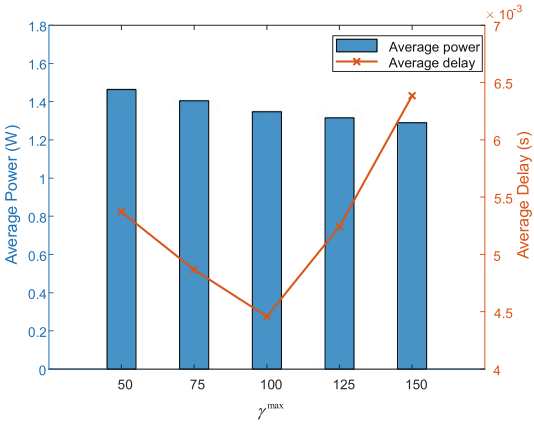


Fig. 4. Effect of $\gamma^{\max}$

take precedence over minimizing delay. Therefore, when γ becomes too large, the average delay increases significantly.

5.3 Comparison with Other Baselines

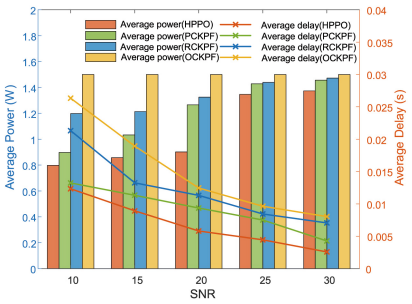
To further reveal the superiority of the proposed framework, we compare it with three other schemes.

(1) **Proximity-based Convolutional Kernel Partition Framework(PCKPF):** The convolutional kernels are allocated to EDs nearest to the CS based on geographical proximity. As a classical resource allocation approach in edge computing, its simplicity and interpretability establish it as an ideal performance benchmark. In our experiments, tasks are assigned to EDs whose physical distance to the CS falls below a predefined threshold, ensuring consistent proximity constraints.

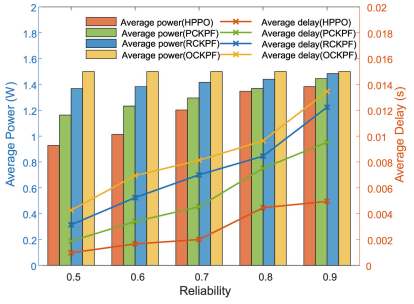
(2) **Random Convolutional Kernel Partition Framework (RCKPF):** The convolutional kernel are allocated randomly. Redundancy is set to 5. To satisfy power consumption constraints, the computation frequency decision is made with PPO algorithm.

(3) **Only Convolutional Kernel Partition Framework (OCKPF):** An average allocation scheme is adopted to distribute power across the EDs. The kernel partition and redundancy decisions are made to minimize the average delay under reliability constraints. The computation frequency follows a random strategy.

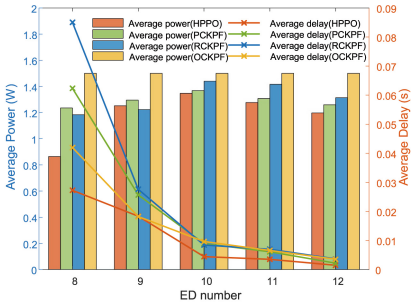
Figure 5 describes the average power consumption per ED and the delay per convolutional layer under various channel conditions, reliability and ED number.



(a) Average power and average delay vs. SNR



(b) Average power and average delay vs. Reliability



(c) Average power and average delay vs. ED number

Fig. 5. Average power and average delay varies with changing SNR, reliability and ED number

Our proposed HPPO is compared to other baselines. It can be seen that HPPO effectively maintains low latency while ensuring power efficiency across different channel conditions, reliability requirement and ED number. As expected, with the improved channel quality and the increase of the number of EDs, we observe a reduction in the average execution latency of a single convolutional layer in four methods in Figs. 5 (a) and 5 (c). It can also be seen in Fig. 5 (b) that when reliability requirements become more stringent, the system sacrifices latency and power performance to ensure high reliability. In particular, compared with RCKPF and OCKPF, both PCKPF and our proposed HPPO scheme effectively reduce average latency and save average energy consumption with the change of the channel conditions and reliability requirements in Fig. 5(a) and Fig. 5(b). However, results show that our proposed scheme outperforms PCKPF in both average latency and average energy consumption. The underlying reason is that PCKPF only encourages EDs close to the CS to participate in collaborative computing to achieve higher transmission rates, whereas in our scheme, the CNN convolutional kernels have a more lenient ED selection mechanism. This allows EDs to reduce the computational workload on individual devices through

extensive collaboration. The notable downward trend in the average latency curve of PCKPF in Fig. 5 (c) can also be attributed to the fact that as the number of collaborative EDs increases, the number of EDs within the distance threshold from the CS grows. This leads to more collaborative EDs participating in computation, thus accelerating task processing speed.

Overall, the results in Fig. 5 highlight the necessity of jointly optimizing convolutional kernel allocation, redundancy, and network resources in distributed computing systems. HPPO balances both short-term resource optimization and long-term kernel partitioning redundancy strategies. Ignoring reliability or resource constraints while focusing solely on minimizing single-layer delay, or overemphasizing the impact of transmission distance on system performance, may derail optimization from its original goals, ultimately leading to prolonged average delay.

6 Conclusion

In this work, we proposed a distributed and reliable inference architecture based on a kernel-wise model partition method. First, a systematic procedure to establish collaborative distributed computing among multiple EDs is designed. Second, we jointly optimized kernel allocation, redundancy decisions, and dynamic resource configuration to accelerate inference while balancing power consumption and reliability constraints. Then, we employed the LM method for problem transformation and proposed a two-timescale HPPO algorithm for solution. Finally, simulation results demonstrated that the proposed framework effectively reduced the average delay while ensuring the reliability of the distributed computing system.

Acknowledgments. This work was supported in part by the National Natural Science Foundation of China (U24A20234) and (No. 62171062), and the National Key R&D Program of China (No.2021YFB3300100).

References

1. Li, L., Xu, W., Xu, J.: Budget-constrained digital twin synchronization and its application on fidelity-aware queries in edge computing. IEEE Trans. Mobile Comput. **24**(1), 165–182(2025)
2. Letaief, S., Lu, L.: Edge artificial intelligence for 6G: Vision, enabling technologies, and applications. IEEE J. Sel. Areas Commun. **40**(1), 5–36 (2022)
3. Liu, Z., Wang, D., Qiu, Z.: Toward mobility-aware edge inference via model partition and service migration. In: 2023 IEEE International Conference on Communications, pp. 3258–3263. IEEE(2023)
4. Qiu, T., Chi, J., Zhou, X., Ning, Z., Atiquzzaman, M., Wu, D.O.: Edge computing in Industrial Internet of Things: Architecture, advances and challenges. IEEE Commun. Surv. Tut. **22**(4), 2462–2488 (2020)

5. Zhang, L.: Cost-aware digital twin migration in mobile edge computing via deep reinforcement learning. In: 2024 IFIP Networking Conference, pp. 441–447. IEEE(2024)
6. Zhou, C., Li, Z., Luo, Z.: Edge intelligence: Paving the last mile of artificial intelligence with edge computing. Proc. IEEE **107**(8), 1738–1762 (2019)
7. Duan, S., et al.: Distributed artificial intelligence empowered by end-edge-cloud computing: a survey. IEEE Commun. Surv. Tutor. **25**(1), 591–624 (2023)
8. Kim, J., Park, Y., Kim, G., Hwang, S.J.: Splitnet: learning to semantically split deep networks for parameter reduction and model parallelization. In: 2017 International Conference on Machine Learning, pp. 1866–1874. IEEE (2017)
9. Ren, P., Qiao, X., Huang, Y., Liu, L., Pu, C., Dustdar, S.: Fine-grained elastic partitioning for distributed DNN towards mobile web AR services in the 5G era". IEEE Trans. Service Comput. **15**(6), 3260–3274 (2021)
10. Zeng, L., Chen, X., Zhou, Z., Yang, L., Zhang, J.: Coedge: cooperative DNN inference with adaptive workload partitioning over heterogeneous edge devices. IEEE/ACM Trans. Network. **29**(2), 595–608 (2020)
11. Mao, J., Chen, X., Nixon, K.W., Krieger, C., Chen, Y.: Modnn: local distributed mobile computing system for deep neural network. In: Design, Automation and Test in Europe Conference & Exhibition (DATE), pp. 1396–1401. IEEE(2017)
12. Tian, X., Zhu, J., Xu, T., Li, Y.: Mobility-included DNN partition offloading from mobile devices to edge clouds. Sensors **21**(1), 229(2021)
13. Parthasarathy, A., Krishnamachari, B.: Defer: Distributed edge inference for deep neural networks. In: 2022 14th International Conference on COMmunication Systems & NETworkS (COMSNETS), pp. 749–753. IEEE (2022)
14. Tang, X., Chen, X., Zeng, L., Yu, S., Chen, L.: Joint multiuser DNN partitioning and computational resource allocation for collaborative edge intelligence. IEEE Internet Things J. **8**(12), 9511–9522 (2021)
15. Eshratifar, A.E., Pedram, M.: Energy and performance efficient computation offloading for deep neural networks in a mobile cloud computing environment. In: Proceedings of the 2018 Great Lakes Symposium on VLSI, pp. 111–116(2018)
16. Zhi, T., Zhang, S.: Performance analysis of edge collaborative inference based on DNN convolutional kernel segmentation. Chinese Internet Things J. **6**(4), 72–81 (2022)
17. Molchanov, P., Tyree, S., Karras, T., Aila, T., Kautz, J.: Pruning convolutional neural networks for resource efficient transfer learning. http://arxiv.org/abs/1611.06440. CoRR (2016)
18. Zhang, W., Wen, Y., Guan, K., Kilper, D., Luo, H., Wu, D.O.: Energy-optimal mobile cloud computing under stochastic wireless channel. IEEE Trans. Wireless Commun. **12**(9), 4569–4581 (2013)
19. Koren, I., Krishna, C.M.: Fault-tolerant systems. Morgan Kaufmann (2020)
20. Brun, Y., Edwards, G., Bang, J.Y., Medvidovic, N.: Smart redundancy for distributed computation. In: 2011 31st International Conference on Distributed Computing Systems, pp. 665–676. IEEE(2011)
21. Little, J.D.C.: A proof for the queuing formula: L = λW. Oper. Res. **9**(3), 383–387 (1961)
22. Valiant, L.G.: A bridging model for parallel computation. Commun. ACM **33**(8), 103–111 (1990)
23. Ren, D., Gui, X., Zhang, K.: Adaptive request scheduling and service caching for MEC-assisted Iot networks: an online learning approach. IEEE Internet Things J. **9**(18), 17372–17386 (2022)

Joint Caching and Offloading Optimization for Heterogeneous Task Patterns in Multi-access Edge Computing

Sihan Li[1,2], Yanqing Wu[1,2], XiaoDong Xu[1,2], and Gang Xu[1,2(✉)]

[1] College of Computer Science, Inner Mongolia University, Hohhot 010021, China
[2] Engineering Research Center of Ecological Big Data, Ministry of Education, Inner Mongolia University, Hohhot 010021, China
csxugang@imu.edu.cn

Abstract. Edge caching has emerged as an effective solution to reduce task processing latency in Multi-Access Edge Computing (MEC) systems by proactively storing potentially executable tasks on edge servers. However, most existing studies are based on the assumption of task homogeneity and fail to adequately consider the heterogeneity of task types in MEC systems. In real-world scenarios, computation-intensive and access-intensive tasks generated by users differ significantly in their processing procedures and resource demands. The previously adopted single caching and offloading strategy falls short in addressing the distinct performance requirements of both task types, resulting in inefficient utilization of computing and caching resources and a notable increase in system latency. To address this issue, this paper proposes a Task Pattern Classification-based Caching and Offloading strategy (TPCO). TPCO builds a unified framework to jointly handle both computation-intensive and access-intensive tasks, and designs two task-specific caching strategies: Gated Deep Q-Network (GDQN) for computation-intensive tasks and Neural Collaborative Filtering-based Caching (NCFC) for access-intensive tasks. Furthermore, a Bayesian optimization approach is employed at the global level to adaptively allocate storage resources between task types with the aim of minimizing system-wide delay. Simulation results show that the proposed TPCO framework outperforms existing methods under various conditions and reduces latency by at least 27.93% in heterogeneous task scenarios, offering a novel solution for joint caching and offloading optimization in MEC systems.

Keywords: Multi-Access Edge Computing · Heterogeneous Workloads · Edge caching · Computation Offloading · Deep Reinforcement Learning

1 Introduction

In recent years, the explosive growth of IoT device connections worldwide has coincided with the widespread adoption of cutting-edge technologies–such as

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 291–310, 2026.
https://doi.org/10.1007/978-981-95-8405-5_16

artificial intelligence, virtual reality, and autonomous driving–on mobile platforms. This convergence has driven a dramatic surge in computational demand, which, however, far exceeds the limited processing capabilities of mobile devices.

To address these challenges, Multi-Access Edge Computing (MEC) has emerged [15]. By deploying compute nodes or servers at the network edge, MEC brings data processing closer to end users, thereby significantly reducing transmission latency and enhancing overall system performance and quality of service. Consequently, MEC is widely regarded as a key technology for alleviating the anticipated shortage of large-scale network computing resources. Recent studies [12] have shown that, within MEC systems, only a small subset of tasks is requested frequently–that is, task requests exhibit high concentration–leading to repeated offloading and computation of the same tasks and incurring unnecessary transmission and processing delays. To mitigate this issue, caching mechanisms have been introduced: by storing high-frequency tasks on edge servers, redundant processing can be avoided, improving resource utilization and reducing system latency [22]. Given the limited cache capacity of edge servers–which can accommodate only a fraction of all tasks–designing effective caching strategies is critical. Most prior work on MEC has focused on a single task type: some studies [2,6,23] optimize caching for computation offloading by storing intermediate data, task files, or preprocessed results to minimize redundant computation and maximize edge-computing efficiency, with new research in this vein emphasizing joint optimization of resource allocation and offloading strategies. Other studies [7,18,20] concentrate solely on caching popular content–such as audio or video–at the edge to reduce user access latency and backbone network load.

However, the above caching strategies do not account for the task heterogeneity and diversity in edge computing environments. Most existing strategies are tailored to single-task scenarios, limiting their applicability. Recent studies suggest that real-world edge caching scenarios can be categorized based on task patterns. For instance, in the survey [9], typical tasks in MEC environments are classified into computation-intensive tasks and data-delivery tasks, corresponding to application scenarios such as Augmented Reality (AR) and HTML content access. Similarly, [10] discusses key challenges in edge caching and categorizes cacheable objects into content caching, data caching, and computation caching.

Some existing studies [11,21] have attempted to propose a unified framework for handling both computational and access-type tasks; however, their task-request models are overly simplified and cannot generate heterogeneous tasks asynchronously [11], and applying a single caching policy in complex scenarios [21] leads to a mismatch between computing resources and storage capacity.

In summary, although prior research has made progress in improving cache hit rates and reducing system latency, the challenges posed by heterogeneous tasks remain insufficiently addressed. To tackle this issue, we propose a dynamic caching and offloading optimization framework based on task pattern classification, aiming to minimize overall latency in complex MEC environments. The main contributions of this work are as follows: (1) we formulate a system model tailored for heterogeneous task patterns and derive an analytical expression for

total task delay; (2) we design two task-specific caching strategies–Gated Deep Q-Network (GDQN) for computation-intensive tasks and Neural Collaborative Filtering-based Caching (NCFC) for access-intensive tasks–based on differences in computational and transmission demands; (3) we decompose the global delay minimization problem into two subproblems and employ a Bayesian optimization approach to adaptively allocate shared caching resources between task types, achieving near-optimal overall performance. The proposed method provides a promising direction for optimizing task scheduling in heterogeneous MEC scenarios.

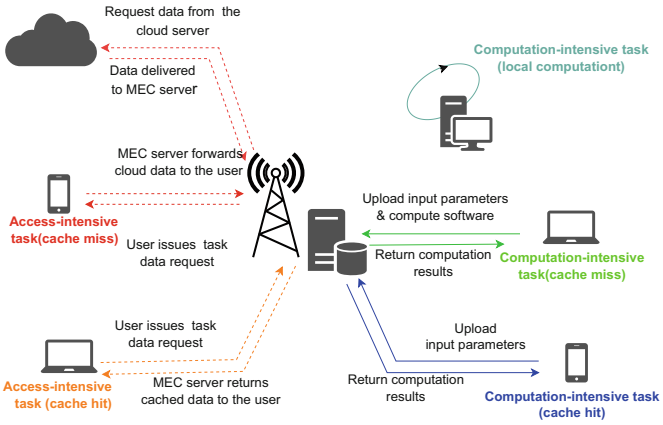


Fig. 1. Illustration of the network model

2 MEC System Model Based on Task Pattern Classification

2.1 Network Model and Task Processing Flow

In edge computing systems equipped with caching mechanisms, user tasks are generally classified into computation-intensive and access-intensive types based on their resource requirements and processing characteristics. Access-intensive tasks (e.g., file downloads, video streaming, and web browsing) are diverse in type and exhibit high request frequency [4]. These tasks rely primarily on data transmission and are highly delay-sensitive, making them suitable for proactive caching of popular content at edge servers to reduce response time. In contrast, computation-intensive tasks (e.g., face recognition and natural language processing) require substantial computing resources and specific task software, which can be pre-cached at the edge to avoid repeated uploads [8]. Although such tasks generally have lower request frequency, they incur high computational overhead [10] and place greater demands on the coordinated allocation of communication and computing resources.

To support both types of tasks within a unified model, a Multi-Access Edge Computing (MEC) network is designed, as illustrated in Fig. 1. The network architecture comprises multiple user devices, a base station equipped with an edge server, and a remote cloud server. The edge server is endowed with computing and storage capabilities, along with the ability to proactively cache task-related data and software, while user devices possess basic local processing power.

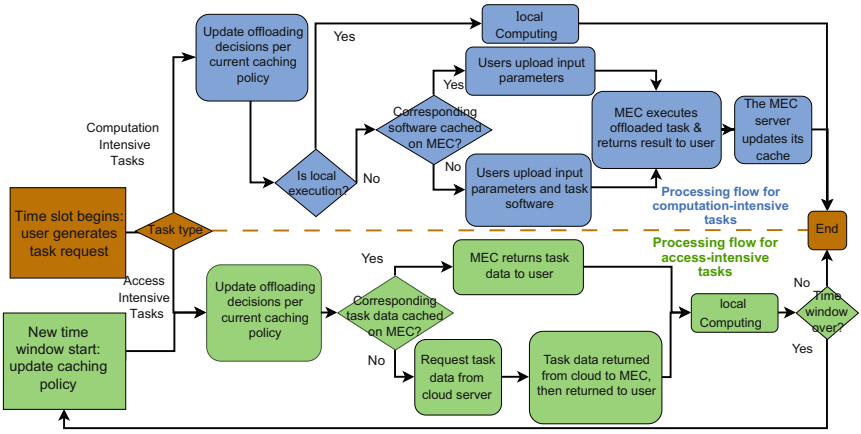


Fig. 2. Overall Task Processing Flow Within a Time Slot Under Heterogeneous Task Scenarios

In each time slot t, the user set $\mathcal{K} = \{\mathcal{K}_k \mid k = 1, 2, \ldots, K\}$ represents the users who are currently in an active state. The tasks generated by users form the computation-intensive task set $\mathcal{F}_1 = \{\mathcal{F}_{f1} \mid f_1 = 1, 2, \ldots, F_1\}$ and the access-intensive task set $\mathcal{F}_2 = \{\mathcal{F}_{f2} \mid f_2 = 1, 2, \ldots, F_2\}$, which together constitute the overall task set $\mathcal{F} = \{\mathcal{F}_1, \mathcal{F}_2\} = \{\mathcal{F}_f = 1, 2, \ldots, F\}$. We assume the system operates over T time slots, with each time slot lasting τ, and t represents any arbitrary time slot.In each time slot, each user requests to execute a task from the task set or makes no request. Similar to the approach in [24], by setting a delay constraint, we can ensure that each task completes its execution within the current time slot. Let μ_k^t represent the request of user K_k in time slot t. Specifically, $\mu_k^t = 0$ indicates that the user does not request any task, and $\mu_k^t = \mathcal{F}_f$ indicates that the user requests to execute task $\mathcal{F}_f$.

The parameters of computation-intensive tasks $\mathcal{F}_{f1} \in \mathcal{F}_1$ include the task input parameter size I_{f1}, the data size of the task computation software S_{f1}, and the number of computational resources (CPU cycles) required to complete the task ω_{f1}. The computational results are considered negligible if the result size is too small. Computation-intensive tasks can either be computed locally or offloaded to an MEC server for computation, where users need to upload the task parameters and computation software to the edge device. Due to the large size of computation software and strong interference from cloud-edge transmission, this paper does not consider offloading tasks to the central server for computation.

Because large computation software significantly occupies cache space, the number of software packages that can be cached concurrently is limited. Meanwhile, tasks exhibit heterogeneous software sizes and computational complexity, imposing higher demands on caching accuracy. Therefore, the system updates its cache in real time based on user requests in each time slot. At the beginning of time slot t, the MEC server first makes a caching decision, updating its cache according to the current user task requests. Under this caching policy, it then makes an offloading decision–determining whether each task is computed locally or on the server–and returns the computed results, completing the task execution process.

In the access-intensive tasks $F_{f2} \in \mathcal{F}_2$, the parameters include the task data size D_{f2} that is requested for download. Due to the large number of access-intensive task types, real-time cache updates would consume a significant amount of system resources. We set L time slots as a time window and perform cache updates every L time slots. At the beginning of each time slot, based on the cache state in the window, requested tasks will be prioritized for retrieval from the edge device. If the requested task is not cached on the edge device, a request will be made to the central server to download the required task data. In summary, the complete flowchart of task execution within a time slot is shown in Fig. 2.

For the task request model, we assume that all task requests of each user follow a Markov chain model, as referenced in papers [16,17]. The specific task request transition model is designed as follows. $\Pr[\mu_k^{(t+1)} = j \mid \mu_k^t = i]$ represents the transition probability of user K_k from task request i to task request j.

$$\Pr\left(\mu^{(t+1)} = j \,|\, \mu^{(t)} = i\right) = \begin{cases} R, & \text{if } i = \mathcal{F} \vee i = 0,\ j = 0 \\ \lambda(1-R)\frac{j^{-\delta_1}}{\sum_{m \in \mathcal{F}_1} m^{-\delta_1}}, & \text{if } i = 0,\ j \in \mathcal{F}_1 \\ (1-\lambda)(1-R)\frac{j^{-\delta_2}}{\sum_{m \in \mathcal{F}_2} m^{-\delta_2}}, & \text{if } i = 0,\ j \in \mathcal{F}_2 \\ \frac{1-R}{N_1}, & \text{if } i \in \mathcal{F}_1,\ j \in \mathcal{N}_1(i) \\ \frac{1-R}{N_2}, & \text{if } i \in \mathcal{F}_2,\ j \in \mathcal{N}_2(i) \\ 0, & \text{otherwise} \end{cases} \tag{1}$$

In the statistical model described above for transition probabilities, the probability of a user transitioning from any state to the no-request state is R. When the user has no request, the probabilities of requesting a computation-intensive task F_{f1} and an access-intensive task F_{f2} at the next time slot follow Zipf [14] distributions with parameters δ_1 and δ_2, respectively. When the user has already made a request in the current time slot, at the next time slot, the user will transition with equal probability to one of the N neighboring tasks within the same task set. N_1 and N_2 represent the number of neighboring tasks in the corresponding task sets for the requested tasks. This design considers both the long-term statistical characteristics and short-term correlations of user requests, providing a better simulation of real-world scenarios.

2.2 Communication Delay Model

In the system of this paper, the network available bandwidth between the user and the edge device is set to B Hz. This bandwidth is evenly divided into M orthogonal wireless channels to avoid inter-channel interference. It is assumed that the channel is static, meaning that the user remains stationary during the task offloading process. To address the issue of insufficient channels, we use Code Division Multiple Access (CDMA) technology [1] on each channel, allowing users to share channels when channel resources are limited. Users will select the best channel M^* for offloading based on the channel selection algorithm in Sect. 3.1. For communication between the remote cloud server and the edge server, we refer to the range in [25], and set the cloud-to-MEC transmission rate R_{cloud} based on the specific simulation scenario.

According to Shannon's formula, the uplink transmission rate at which user K_k offloads tasks to the MEC server over channel $\mathcal{M}^*$ in time slot t is given by:

$$R_{(k,t)}^{\text{up}} = \frac{B}{M} \log_2 \left(1 + \frac{P_k h_k}{\sum\limits_{\substack{n \in K \setminus \{k\} \\ \alpha_{n,t} = \alpha_{k,t}}} P_n h_n + \sigma^2} \right) \tag{2}$$

where P_k is the transmit power of user K_k, h_k is the channel gain, and σ^2 denotes the Gaussian noise power.

Similarly, the downlink transmission rate from the MEC server to user K_k can be expressed as:

$$R_{(k,t)}^{\text{down}} = \frac{B}{M} \log_2 \left(1 + \frac{P_{\text{edge}} h_k}{\sum\limits_{\substack{n \in K \setminus \{k\} \\ \alpha_{n,t} = \alpha_{k,t}}} P_{\text{edge}} h_n + \sigma^2} \right) \tag{3}$$

where P_{edge} is the fixed transmit power of the MEC server to all users.

At time slot t, when user K_k offloads task F_{f1}, if there is a cache miss, the system needs to upload both the task parameters and computation software. If the cache is hit, only the task parameters needs to be uploaded. The communication delays are expressed as follows:

$$T_{k,f1,t}^{\text{trans,F}} = \frac{I_{f1} + S_{f1}}{R_{k,t}^{\text{up}}}, \quad T_{k,f1,t}^{\text{trans,T}} = \frac{I_{f1}}{R_{k,t}^{\text{up}}} \tag{4}$$

where I_{f1} denotes the size of the input parameters required by task F_{f1}, and S_{f1} is the size of its computation software.

For access-intensive tasks, the uplink transmission only carries lightweight metadata (such as index identifiers, ID codes, etc.), so the transmission overhead is negligible. If the task F_{f2} is not cached at the edge server, the server will request the corresponding data from the cloud server. The data must then be

transmitted from the cloud server to the edge server and from the edge server to user K_k. If the cache is a hit, the requested data is directly transferred from the edge server to the user. The communication delays are expressed as:

$$T^{\mathrm{F}}_{k,f2,t} = \frac{D_{f2}}{R_{\mathrm{cloud}}} + \frac{D_{f2}}{R^{\mathrm{down}}_{k,t}}, \quad T^{\mathrm{T}}_{k,f2,t} = \frac{D_{f2}}{R^{\mathrm{down}}_{k,t}} \tag{5}$$

where D_{f2} is the total data size to be downloaded, composed of text data D_{txt} and video data D_{vid}. R_{cloud} is the cloud-to-MEC transmission rate.

2.3 Task Processing and Computation Delay Model

In a heterogeneous task environment, only computation-intensive tasks involve computation. Let the CPU computing capabilities allocated to user K_k on the user device and on the edge server be denoted by f^{local}_k and f^{mec}_k, respectively, and let the computational complexity of task $\mathcal{F}_{f1}$ be denoted by ω_{f1}. The delay for processing task $\mathcal{F}_{f1}$ locally and on the MEC server can be respectively expressed as:

$$T^{\mathrm{local}}_{k,f_1,t} = \frac{\omega_{f_1}}{f^{\mathrm{local}}_k}, \quad T^{\mathrm{mec}}_{k,f_1,t} = \frac{\omega_{f_1}}{f^{\mathrm{mec}}_k} \tag{6}$$

2.4 Total Task Delay Model

In this paper, our goal is to minimize the average task execution delay per time slot for all users by jointly optimizing caching and offloading decisions under latency constraints. Based on the models and analysis above, for a computation-intensive task:

If local computation is chosen, no data transmission is required, and the total delay consists only of the computation delay $T^{\mathrm{local}}_{k,f1,t}$.

If offloaded computation is chosen, the total delay includes both transmission and computation delays. When the cache is hit or missed, the total delays are given by:

$$T^{T}_{k,f1,t} = T^{\mathrm{trans,T}}_{k,f1,t} + T^{\mathrm{mec}}_{k,f1,t}, T^{F}_{k,f1,t} = T^{\mathrm{trans,F}}_{k,f1,t} + T^{\mathrm{mec}}_{k,f1,t} \tag{7}$$

Therefore, when user $\mathcal{K}_k$ executes computation-intensive task $\mathcal{F}_{f1}$ in time slot t, the total delay is defined as:

$$\begin{aligned} T^{C}_{k,f1,t} = {} & \mathbf{1}\big(\alpha_{k,f1,t}=0\big)\, T^{\mathrm{local}}_{k,f1,t} + \mathbf{1}\big(\alpha_{k,f1,t}=1\big)\cdot \\ & \Big[\mathbf{1}\big(\beta_{k,f1,t}=0\big)\, T^{F}_{k,f1,t} + \mathbf{1}\big(\beta_{k,f1,t}=1\big)\, T^{T}_{k,f1,t}\Big] \end{aligned} \tag{8}$$

In the formula, $\mathbf{1}(\cdot)$ is an indicator function that equals 1 if its argument is true and 0 otherwise. The first term represents the delay when the computation-intensive task is executed locally ($\alpha_{k,f1,t}=0$), which is $T^{\mathrm{local}}_{k,f1,t}$. The second term corresponds to offloaded computation ($\alpha_{k,f1,t}=1$): if the computation software is not cached at the edge server ($\beta_{k,f1,t}=0$), the total delay is $T^{T}_{k,f1,t}$; otherwise, the total delay is $T^{F}_{k,f1,t}$.

For access-intensive tasks, the communication delay is the total delay. Depending on whether the requested data is cached at the edge server, the data is fetched from the cloud or returned directly by the edge. Accordingly, when user K_k executes access-intensive task $\mathcal{F}_{f2}$ in time slot t, the total delay is defined as:

$$T^A_{k,f2,t} = \mathbf{1}(\beta_{k,f2,t} = 0)\, T^F_{k,f2,t} + \mathbf{1}(\beta_{k,f2,t} = 1)\, T^T_{k,f2,t} \tag{9}$$

The first term applies if the data is not cached (cache miss), incurring delay $T^F_{k,f2,t}$; the second if it is cached (cache hit), incurring delay $T^T_{k,f2,t}$.

Hence, the overall delay of user K_k in slot t can be written as:

$$T_{k,t} = \sum_{f=1}^{F} \Big[\mathbf{1}\big(\mu^t_k = \mathcal{F}_{f1}\big)\, T^C_{k,f1,t} + \mathbf{1}\big(\mu^t_k = \mathcal{F}_{f2}\big)\, T^A_{k,f2,t} \Big] \tag{10}$$

In the above, $\mu^t_k = \mathcal{F}_{f1} \in \mathcal{F}_1$ indicates that user K_k requests a computation-intensive task $\mathcal{F}_{f1}$ in slot t, and $\mu^t_k = \mathcal{F}_{f2} \in \mathcal{F}_2$ indicates an access-intensive request.

3 Two Caching Strategies for Computation and Access-Intensive Tasks

3.1 Dynamic Caching and Offloading Mechanism for Computation-Intensive Tasks

In this study, we design a dynamic task software caching update mechanism, GDQN, based on Double Deep Q-Network (DDQN), aimed at dynamically adjusting the system cache and offloading decisions according to task requests in each time slot. Specifically, GDQN captures user request features and evaluates the value of caching actions based on the system state at time slot t. By efficiently learning in a large action space, we optimize the caching strategy and effectively reduce system latency.

In designing the offloading scheme, the system first computes the offloading delay under the current caching strategy β_t, ignoring interference from shared channels. It then compares this delay with the local computation delay $T^{\text{local}}_{k,f1,t}$. If the local delay is smaller, the task is executed locally ($\alpha_t = 0$); otherwise, the system proceeds to channel selection. The user first attempts to offload on any idle channel. If no idle channel is available, a greedy algorithm evaluates each busy channel m by computing the cost :

$$\Delta_m = \left| T_{\text{current},m} - T_{\text{new},m} \right| \tag{11}$$

where $T_{\text{current},m}$ is the current total transmission delay on channel m and $T_{\text{new},m}$ is the delay if the new task is offloaded via channel m. The channel with the minimum cost is selected:

$$m^* = \arg\min_{m \in \mathcal{M}} \Delta_m \tag{12}$$

Finally, the offloading delay on channel m^* is compared again with $T^{\text{local}}_{k,f1,t}$ to determine the final decision α_t. This channel selection procedure can also be applied to edge-to-user transmissions for access-intensive tasks.

To optimize the caching strategy, we employ a joint framework combining a Gated Recurrent Unit (GRU) network and a Dueling Double Deep Q-Network (DDQN), integrated with 01 knapsack dynamic programming. This framework aims to efficiently select the optimal subset of files to cache and maximize delay savings under capacity constraints.

To model the dynamic evolution of user requests and cache states, we abstract the system as a Markov Decision Process (MDP). The system state comprises two parts: the user request vector μ_t and the server cache state b_t. By concatenating these two vectors, we obtain the system state at time slot t, denoted as $s_t = [\mu_t, b_t]$, which describes the distribution of task requests and the current cache usage.

We design the reward function to quantify the delay savings brought by the caching strategy. It is defined as the difference between the total execution delay under an empty cache and that under the current cache:

$$R_t = T_N^t - T_C^t \tag{13}$$

where T_N^t and T_C^t denote the total delay using offloading policy α_t under empty-cache and current-cache states, respectively.

To capture the temporal dynamics of user requests, we employ a Gated Recurrent Unit (GRU) to process the input sequence. At each time slot t, the sequence $\{s_{t-L+1}, \ldots, s_t\}$ of length L is fed into the GRU. After the GRU applies its update and reset gates to update the hidden state, it outputs the hidden state h_t as the temporal feature representation. We then pass h_t through a fully connected layer with ReLU activation to further enrich the feature, where $f_t = \text{ReLU}(W_f h_t + b_f) \in \mathbb{R}^{128}$.

Here, $W_f \in \mathbb{R}^{128\times H}$ is a trainable weight matrix. This design enables the network to automatically learn the Markovian features of user requests and predict future time-slot decisions based on the state sequence.

The output uses a Dueling network structure [19] to estimate the Q-value for each file. Two parallel streams compute the state value $V(s_t)$ and the action advantage $A(s_t, f)$, measuring the potential contribution of the state to long-term return and the gain of caching file f above average, respectively. The final Q-value is

$$Q(s_t, f) = V(s_t) + \left(A(s_t, f) - \tfrac{1}{F}\sum_{f'=1}^{F} A(s_t, f')\right) \tag{14}$$

The network outputs a length-F vector $Q(s)$, where each component $Q(s_t, f)$ evaluates the value of caching file f. Enumerating all subsets requires $O(2^F)$ actions. To select the optimal cache subset efficiently, we integrate a 01 knapsack dynamic programming algorithm. Formally, we solve

$$\max_{\{z_{f1}\}} \sum_{f=1}^{F1} Q(s_t, f)\, z_{f1} \quad \text{s.t.} \quad \sum_{f=1}^{F1} S_{f1}\, z_{f1} \le C_1, z_{f1} \in \{0, 1\} \tag{15}$$

where S_{f1} is the data size of the task computation software and C_1 is the cache capacity for computation tasks. This reduces action selection to $O(F \cdot C_1)$ polynomial time per slot.

Finally, we adopt a Double DQN (DDQN) strategy to reduce Q-value overestimation and improve stability. The online network parameters θ are updated by backpropagation, while the target network parameters θ^- are synced periodically.The TD target is $y_t = r_t + \gamma\, Q_{\theta^-}(s_{t+1}, a^*_{t+1})$, $a^*_{t+1} = \arg\max_a Q_\theta(s_{t+1}, a)$, and we use the Huber loss for the TD error, $J(\theta) = \text{Huber}(Q_\theta(s_t, a_t) - y_t)$. This decouples action selection and action evaluation, ensuring stable convergence in large-scale task environments.For clarity, we summarize the detailed steps of the DDQN inference phase in Algorithm 1.

Algorithm 1. Efficient Caching and Offloading for Computation-Intensive Tasks

Require: Cache capacity C_1, discount factor γ, task sizes S_{f1}, request vector μ_t, cache state b_t

Initialize networks θ, θ^-; set b_0, μ_0
for each time slot t **do**
  $s_t \leftarrow [\mu_t, b_t]$, $\mathbf{h}_t \leftarrow \text{GRU}(s_{t-L+1:t})$
  $f_t \leftarrow \text{ReLU}(W_f \mathbf{h}_t + b_f)$
  Estimate $Q(s_t, f)$ via dueling DDQN
  Select cached tasks by solving:

$$\max \sum Q(s_t, f) \cdot z_f, \quad \text{s.t. } \sum S_{f1} z_f \le C_1,\ z_f \in \{0, 1\}$$

  Update cache, compute reward $R_t = T^t_N - T^t_C$
  Update Q via Huber loss and target value y_t
  if $t \bmod T_{\text{sync}} = 0$ **then**
    $\theta^- \leftarrow \theta$
  end if
end for

3.2 Caching Mechanism for Access-Intensive Tasks

In multi-access edge computing scenarios, existing deep learningbased caching schemes often only scale to relatively small task pools. To explore a method suitable for large-scale task environments, this paper proposes a cache mechanism combining interest-awareness and delay-benefit based on the Neural Collaborative Filtering (NCF) framework [5]. NCFC models the usertask interaction directly to predict each user's access probability for content items. On this basis, we introduce a delay-savingdriven global reward G^k_f to obtain the globally optimal cache subset.

First, we transform user attributes and content features through a projection layer into low-dimensional embedding vectors to model their latent characteristics as inputs to the neural network. For each user K_k and each access-intensive

task $\mathcal{F}_{f2}$, the user and content feature vectors are mapped by embedding layers into the same latent space, yielding low-dimensional embeddings $e_k, e_{f2} \in \mathbb{R}^d$ (with $d = 24$), where d is the embedding dimension.

To capture the interaction between users and content, the model utilizes both linear interaction via GMF (Generalized Matrix Factorization) and nonlinear interaction via MLP (Multilayer Perceptron). GMF models the linear relationship between user and content by computing the element-wise product of user and content embeddings. Specifically, for user K_k with embedding e_k and access-intensive task $\mathcal{F}_{f2}$ with embedding e_{f2}, the GMF interaction is given by $h_{k,f2}^{\text{GMF}} = e_k \odot e_{f2}$ where $\odot$ denotes element-wise multiplication, capturing the linear compatibility between user and content to produce a linear preference score.

In the nonlinear interaction component, we concatenate the user and content embedding vectors to form a composite feature vector, which is then fed into a multilayer perceptron (MLP) to capture more complex nonlinear relationships. This process is represented as $h_{k,f_2}^{\text{MLP}} = \text{MLP}\big([e_k; e_{f_2}]\big)$, where the MLP architecture has layer dimensions $128 \to 64 \to 32$ and uses ReLU activation functions.

After concatenating the linear and non-linear branches, we obtain the fused feature vector $\mathbf{h}_{k,f_2} = [\mathbf{h}_{k,f_2}^{\text{GMF}}; \mathbf{h}_{k,f_2}^{\text{MLP}}]$. This fused feature is then mapped to an unnormalised logit $r_{k,f_2} = \mathbf{w}^\top \mathbf{h}_{k,f_2} + b$, where $\mathbf{w}, b$ are learnable parameters.

The network outputs the whole logit vector $\mathbf{z}_k = \big[r_{k,1}, \ldots, r_{k,F_2}\big] \in \mathbb{R}^{F_2}$ for each user in a single forward pass.

We regard the request of user k in time-slot t as a *multi-label classification* problem: the actually requested task is marked 1 while all others are 0. The model is optimised with *BCEWithLogitsLoss*, which internally applies the sigmoid activation and computes a per-dimension binary cross-entropy; therefore no explicit $\sigma(\cdot)$ is needed during training.

At inference, request probabilities are recovered from logits by

$$p_{k,f_2} = \sigma\big(r_{k,f_2}\big) = \frac{1}{1 + \exp(-r_{k,f_2})} \in (0,1) \tag{16}$$

After obtaining these probabilities, we define a delay-difference-based caching benefit metric $G_f^k 2$, representing the expected delay savings when user K_k caches task $\mathcal{F}_{f2}$:

$$G_{f2}^k = p_{k,f_2} \left(T_{k,f2,t}^F - T_{k,f2,t}^T\right) \tag{17}$$

where $T_{k,f2,t}^T$ and $T_{k,f2,t}^F$ denote the execution delays under cache hit and miss, respectively, as defined in Eq. (5).

The global caching benefit of task $\mathcal{F}_{f2}$ is obtained by summing over all users:

$$G_{f_2} = \sum_{k=1}^{K} G_{f_2}^k = \sum_{k=1}^{K} P(f_2 \mid k) \left(T_{k,f_2,t}^F - T_{k,f_2,t}^T\right) \tag{18}$$

Finally, let G_{f2} collect these benefits for all candidate contents. To select the optimal cache subset under the capacity limit C_2 for access-intensive tasks, we formulate a 01 knapsack problem:

$$\max_{\{z_{f2}\}} \sum_{f=1}^{F2} G_{f2}\, z_{f2} \quad \text{s.t.} \quad \sum_{f=1}^{F2} D_{f2}\, z_{f2} \le C_2, \quad z_{f2} \in \{0,1\} \tag{19}$$

This yields the optimal cache set within each time-slot window, minimizing the total system delay. The knapsack is solved by dynamic programming in $O(F \cdot C_2)$ time.

In our system, we first pretrain a base model on historical data and perform cache replacement using a sliding time-window mechanism. We define a cache window of length L. At the end of each window, the model is fine-tuned on the newly accumulated interaction data. We then recompute G_{f2} and solve Eq. (19) to update the cache, thus achieving sliding-window cache replacement.

4 Optimization and Solution

In multi-access edge computing scenarios, the joint optimization of caching and offloading for heterogeneous tasks (computation-intensive and access-intensive) presents multidimensional coupling challenges. To address this, we propose a Task Pattern Classification-based Caching and Offloading strategy (TPCO) framework, which adopts a function-decoupling hierarchical strategy: first independently model and optimize caching and offloading strategies for each task type, then perform global resource allocation to achieve the overall system objective.

Our optimization goal is to minimize the long-term average task processing delay of all users per time slot t, subject to the system constraints. Formally, the problem P is defined as:

$$\min_{\{\alpha_t,\beta_t\}} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{K} T_{k,t} \tag{20}$$

$$\text{s.t.} \qquad T_{k,t} \le \tau, \quad \forall k \in \mathcal{K},\ f \in \mathcal{F},\ t \in \mathcal{T} \tag{20a}$$

$$\sum_{f=1}^{F} \left(\beta_{k,f1,t} S_{f1} + \beta_{k,f2,t} D_{f2}\right) \le C, \quad \forall k \in \mathcal{K},\ t \in \mathcal{T} \tag{20b}$$

$$\sum_{k=1}^{K} f_{k,t}^{\text{mec}} \le f_{\max}^{\text{mec}}, \quad \forall t \in \mathcal{T} \tag{20c}$$

$$\beta_{k,f,t} \in \{0,1\}, \quad \forall k \in \mathcal{K},\ f \in \mathcal{F},\ t \in \mathcal{T} \tag{20d}$$

$$\alpha_{k,f,t} \in \{0,1\}, \quad \forall k \in \mathcal{K},\ f \in \mathcal{F},\ t \in \mathcal{T} \tag{20e}$$

$$M^* \in \{1,2,\ldots,M\} \tag{20f}$$

In Problem P, constraint (20a) ensures that the total execution delay $T_{k,t}$ for each user K_k in any time slot does not exceed the slot length τ, which can be enforced by selecting an appropriate slot duration. Constraint (20b) requires

that the total size of both types of task software cached at the MEC server in each time slot does not exceed the cache capacity C. Constraint (20c) guarantees that the sum of computational resources allocated to all users per time slot does not exceed the server's maximum computation capacity $f_{\max}^{\text{mec}}$; any tasks arriving when the server is fully loaded will be executed locally. Constraints (20d) and (20e) define the feasible values of the caching decision $\beta_{k,f,t}$ and the offloading decision $\alpha_{k,f,t}$, respectively–indicating that task software on the server can be added, removed, or remain unchanged, and that each user can choose local computation or MEC offloading. Finally, constraint (20f) specifies that the optimal channel M^* selected by the channel selection algorithm is chosen from the M channels defined in the network model, rather than being newly allocated.

Algorithm 2. Bayesian Cache Allocation

Require:
Total cache capacity C
Maximum iterations T
Initial sampling points N_{init}
Ensure: Optimal allocation (C_1^*, C_2^*)
1: Generate $\{C_1^{(i)}\}_{i=1}^{N_{\text{init}}} \sim \mathcal{U}(0, C)$
2: Evaluate $\rho(C_1^{(i)})$ via GDQN/NCFC
3: Initialize $\mathcal{GP}$ model
4: **for** $t = N_{\text{init}} + 1$ **to** T **do**
5: $C_1^{(t)} \leftarrow \arg\max_x \text{EI}(x|\mathcal{GP})$
6: Update $\mathcal{GP}$ with $(C_1^{(t)}, \rho(C_1^{(t)}))$
7: **end for**
8: $C_1^* \leftarrow \arg\min_i \rho(C_1^{(i)})$
9: **return** $(C_1^*, C - C_1^*)$

The specific expression for $T_{k,t}$ is given in Eq. (10) of Sect. 2.4, where the total delay is composed of the computation-intensive component $T_{k,f1,t}^C$ and the access-intensive component $T_{k,f2,t}^A$. Since these two task types are transmitted over the uplink and downlink respectively (with frequency isolation ensuring no interference), the total delay can be decoupled into the sum of their independent delay contributions. Further assuming that the request volumes and temporal distributions of the two task types are statistically independent, the global optimization problem P can be decomposed into two subproblems P_1 and P_2, corresponding to delay optimization for computation-intensive and access-intensive tasks, respectively. The two subproblems P_1 and P_2 are defined as follows:

$$P_1: \quad \min_{\{\delta_t, \epsilon_t\}} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{K} \sum_{f=1}^{F} \mathbf{1}\left(\mu_k^t = \mathcal{F}_{f1}\right) T_{k,f1,t}^C \tag{21}$$

$$P_2: \quad \min_{\{\delta_t, \epsilon_t\}} \lim_{T\to\infty} \frac{1}{T} \sum_{t=1}^{T} \sum_{k=1}^{K} \sum_{f=1}^{F} \mathbf{1}\left(\mu_k^t = \mathcal{F}_{f2}\right) T_{k,f2,t}^A \tag{22}$$

From the detailed analysis, when task requests are independently distributed, the only coupling between computation-intensive and access-intensive tasks arises from the shared cache resource: the total cache capacity C must be divided into C_1 for computation-intensive tasks and C_2 for access-intensive tasks, satisfying $C_1 + C_2 = C$. Under fixed cache allocations, we use the GDQN and NCFC algorithms to solve subproblems P_1 and P_2, obtaining the optimal delays $\rho_1(C_1)$ and $\rho_2(C_2)$. The global delay optimization problem then reduces to:

$$\min_{C_1, C_2} \ \rho_1(C_1) + \rho_2(C_2) \quad \text{s.t.} \quad C_1 + C_2 = C \tag{23}$$

Both $\rho_1(C_1)$ and $\rho_2(C_2)$ are obtained by training and evaluating their respective deep learning models; they lack explicit analytical forms and gradients, constituting a typical black-box optimization scenario. Therefore, neither gradient-based convex optimization methods nor standard first- or second-order descent algorithms can be directly applied to the global objective. Moreover, the edge system must complete capacity allocation rapidly within a limited evaluation budget, which precludes random or heuristic searches with high iteration costs.

Accordingly, we design a Bayesian optimization scheme: we build a Gaussian process surrogate model and use the Expected Improvement (EI) acquisition function to iteratively approach the optimal cache allocation (C_1^*, C_2^*). Specifically, in the initialization stage, we generate a set of candidates by uniformly sampling the capacity interval $[0, C]$ and collect initial delay samples through experiments. In each iteration, we use a Gaussian process (GP) to approximate the objective function and evaluate the posterior distribution at unexplored points:

$$f(C_1) \sim \mathcal{GP}\big(\mu(C_1), \kappa(C_1, C_1')\big) \tag{24}$$

where $\mu(\cdot)$ denotes the mean function, and $\kappa(C_1, C_1')$ represents the covariance kernel that measures the similarity between two cache allocation candidates C_1 and C_1'. A commonly used choice for κ is the squared exponential (RBF) kernel. We then define the acquisition function based on Expected Improvement (EI):

$$\mathrm{EI}(C_1) = \mathbb{E}\big[\max\big(f_{\min} - f(C_1), 0\big)\big] \tag{25}$$

where $f_{\min}$ is the minimum observed objective value so far. Finally, we update the candidate set by selecting the point with the largest expected improvement, denoted $C_1^{(n+1)}$, for the next evaluation. This process repeats until the number of iterations reaches a preset maximum or the improvement $\Delta f^{(n)}$ falls below a convergence threshold. The resulting optimal cache allocation $\big(C_1^*, C_2^*\big)$ is then applied to the system-level delay minimization problem.

5 Experimental Results and Comparative Analysis

In this section, we evaluate the performance of our proposed NCFC and GDQN schemes against the following baseline caching strategies. All experiments use the same offloading mechanism as in our proposals:

- **Random Replacement (Random)**: Caches every new requested task; when the cache is full, the MEC server randomly evicts one of the cached tasks.
- **Least Frequently Used (LFU)**: Caches the most frequently requested task software; when full, the cached item with the fewest requests is replaced by the new task.
- **First-In, First-Out (FIFO)**: The MEC server evicts the task that was requested earliest, caching the most recently requested tasks.
- **Intelligent popularity-based caching scheme (IPTC)** [3]: IPTC builds a task value function based on popularity, freshness, and size, and performs proactive caching using historical request data to reduce latency. Low-value tasks are replaced to maximize cache efficiency and system performance.
- **Item-based Collaborative Filtering Caching (ITBCF)** [13]: Predicts each user's activity level and content preferences using collaborative filtering, formulates average hit-rate maximization as a submodular problem, and greedily allocates file caches for each user.
- **Dynamic ProgrammingBased Heterogeneous Task Caching (DP-HTC)** [11]: Dynamic programming is used to iteratively build an optimal caching strategy by evaluating the latency reduction and storage cost of each task, thereby jointly optimizing data caching for computation-intensive tasks and access-intensive tasks.
- **Greedy AllocationBased Heterogeneous Task Caching (GA-HTC)** [11]: Cache the data of computation-intensive tasks and access-intensive tasks in descending order of popularity until the caching space is exhausted.

6 System Model

This study uses Python and PyTorch to conduct simulation experiments and visualize the results. In our multi-access edge computing model, the system consists of: (1) K user devices with basic computing capabilities, randomly distributed in a 200m×200 m area; (2) an edge server at the geometric center of the

Table 1. Experimental Parameter Settings

Parameter	Value	Parameter	Value
Number of users (K)	20	Number of computing-type tasks (F_1)	50
Time slot size (τ)	10 s	Number of access-type tasks (F_2)	1500
Number of time slots (T)	2000	Wireless transmission bandwidth (B)	30 MHz
Device transmission power	0.5 W	Noise power spectral density (σ^2)	2×10^{-13}
User CPU computing capability (f_k^{local})	1 GHz	Server CPU computing capability (f_k^{mec})	20 GHz
Number of channels (M)	10	Path loss exponent (η)	4
Cloud-to-MEC transmission rate (R)	1.5 MB/s	Task input parameter size (I_1)	[1, 5] MB
Task computation software size	[100, 300] MB	Task computation complexity (ω_{F_1})	[1, 5] Gigacycles
Requested text download size (D_{text})	[1, 5] MB	Requested video download size (D_{video})	[5, 30] MB
Input sequence length (L)	5	Number of GRU layers	1
Discount factor (γ)	0.9	Experience replay memory size (E)	1000

coverage area, equipped with computing and storage capabilities; (3) a remote cloud server providing downloads for access-intensive tasks.

The channel model uses a dynamic time-slot approach, where the channel gain for user K_k at time slot t is given by $h_{k,t} = \rho_k(t)\, d_k^{-n}$, with d_k being the Euclidean distance from user k to the base station, $\rho_k(t) \sim \text{Exp}(1)$ representing small-scale Rayleigh fading, and n being the path-loss exponent. The simulation parameters are based on typical MEC network settings [17], summarized in Table 1.

For specific task requests, we follow the task request model in Eq. (1). For computation-intensive tasks, we generate a task pool with 50 tasks. For access-intensive tasks, we evaluate the performance of caching algorithms on the MovieLens public dataset, using the version with 1 million ratings. The dataset includes nearly 3,700 available movie tasks. The parameters for both task types are generated based on the contents in Table 1."

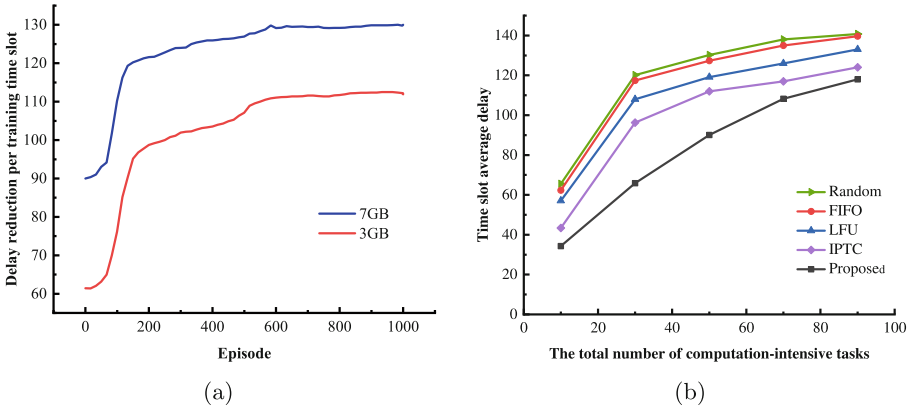


Fig. 3. In computational-intensive task scenarios (a) The delay consumption of users with respect to iteration steps; (b) Comparison of the number of users as a function of delay.

Figure 3a shows the delay consumption reduction per training slot for computational tasks with the proposed GDQN scheme, under cache sizes of 3G and 7G. As training slots increase, the delay consumption reduction gradually grows and reaches its maximum. The proposed scheme converges within 1000 time slots, while most existing DRL-based caching methods [16] require over 10000 time slots. Experiments show that the proposed method can converge quickly and stably under different cache conditions.

Figure 3b compares the average per-slot delay of different caching schemes for computation-intensive tasks as a function of the task library size. As the library size increases, the average per-slot delay of all schemes increases. Our GDQN-based scheme consistently delivers the best cache prediction and delay reduction across different library sizes. At a library size of 30 tasks, our scheme

reduces delay by 31.6%, 39.1%, 43.9%, and 45.2% compared to the Random, LFU, FIFO, and IPTC schemes. This is because our method dynamically learns the dynamics of user requests in each time slot and adjusts the caching strategy in real-time, allowing it to adapt quickly to varying request volumes.

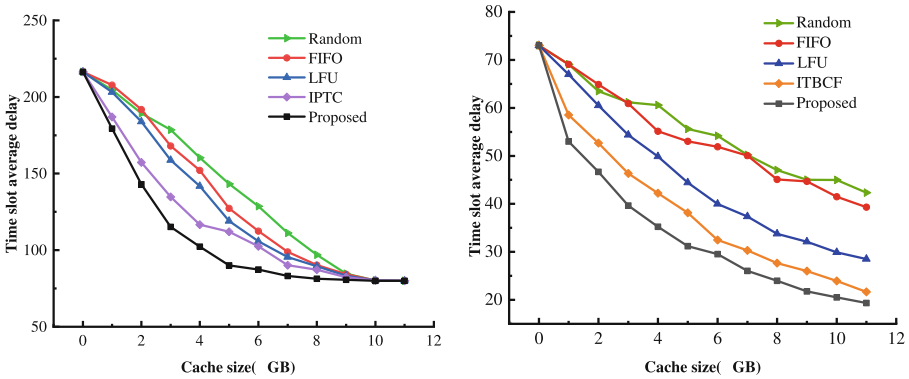


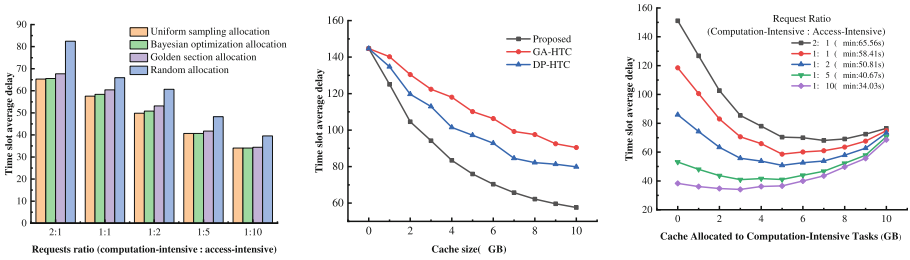
Fig. 4. Comparison of the cache size as a function of delay in computational-intensive task scenarios and access-intensive task scenarios.

Figure 4a shows the effect of MEC server cache size on the average per-slot delay for the proposed GDQN-based computation-intensive scheme and five baseline schemes. As cache size increases, all schemes exhibit decreasing delay. At a 5GB cache, GDQN reduces delay by 19.3%, 24.2%, 29.3%, and 37.1% compared to the Random, LFU, FIFO, and IPTC schemes. Beyond 10GB, delays converge as the edge caches the entire software library.

Figure 4b compares the average per-slot delay of each caching scheme for access-intensive tasks as a function of the MEC server's cache size. GDQN consistently outperforms other caching algorithms across all cache sizes. At 5GB cache, GDQN reduces delay by 18.2%, 29.8%, 41.2%, and 43.9% compared to Random, LFU, FIFO, and ITBCF. Unlike the computation-intensive task delay-cache curve, the delay for access-intensive tasks continues to decrease as cache size grows, as the task library is large and not fully cached even at 10GB.

Figure 5a compares system performance with a 10GB cache, evaluating four cache allocation methods: Bayesian optimization, golden-section search, random allocation, and uniform sampling allocation (which corresponds to grid search). A 100-iteration grid search serves as the baseline. By varying the parameter λ defined in Eq. (1), we examine the algorithm's performance under different computation-to-access task ratios. Bayesian optimization significantly outperforms other methods in minimizing average per-slot delay. Despite reducing the number of iterations by 90%, the proposed method still achieves performance close to that of the grid search algorithm, with the average delay difference not exceeding 3.2%.

Figure 5b compares the average per-slot delay of the proposed algorithm, GA-HTC, and DP-HTC algorithms under heterogeneous task scenarios with varying cache capacities. The proposed algorithm outperforms both GA and DP, reducing delay by 36.35% and 27.93% at a 10GB buffer.



(a) (b) (c)

Fig. 5. (a) Performance Comparison of Cache Allocation Algorithms Under Varying Task Ratios. (b) Performance Comparison of Caching Algorithms Under Heterogeneous Task Scenarios. (c) System Delay Variations Under Different Cache Allocation Schemes and Task Request Patterns.

Figure 5c compares the system's per-slot delay curves for different cache allocation schemes under varying ratios of access-intensive to computation-intensive task requests. As the proportion of computation-intensive tasks decreases, total delay declines, since these tasks require more cache and computation resources. The minimum point on each curve represents the lowest possible system delay. As the access-intensive share grows, the optimal allocation assigns more cache to access-intensive tasks to minimize overall delay. These results show that our algorithm effectively adapts to heterogeneous task mixes and scales across different network conditions.

7 Conclusion

This paper proposes a joint caching and offloading optimization scheme (TPCO) for heterogeneous tasks in multi-access edge computing (MEC). For computation-intensive and access-intensive tasks, we design specialized caching strategies (GDQN and NCFC) and allocate cache resources via Bayesian optimization to reduce total system delay. Future work will explore the algorithm's performance in multi-server scenarios to enhance the framework's scalability and applicability.

Acknowledgments. This work was supported by in part by the Science & Technology Plan Project of Inner Mongolia A.R. under Grants 2023YFSW0017; the National Natural Science Foundation of China under Grant No. 62061036; the Science and Technology Plan Project of Inner Mongolia A. R. of China under Grant No. 2023YFSW0017;

the University Youth Science and Technology Talent Development Project (Innovation Group Development Plan) of Inner Mongolia A. R. of China under Grant No. NMGIRT2318; the Ordos Science & Technology Plan under Grant No. YF20240029; the Self-Open Project of the Engineering Research Center of Ecological Big Data (Ministry of Education); and the China Scholarship Council.

References

1. Adachi, F., Garg, D., Takaoka, S., Takeda, K.: Broadband CDMA techniques. IEEE Wirel. Commun. **12**(2), 8–18 (2005)
2. Alameddine, H.A., Sharafeddine, S., Sebbah, S., Ayoubi, S., Assi, C.: Dynamic task offloading and scheduling for low-latency IoT services in multi-access edge computing. IEEE J. Sel. Areas Commun. **37**(3), 668–682 (2019)
3. Chen, J., Shen, Y.: Task offloading in mobile edge computing with caching mechanism. Comput. Eng. **50**(07), 194–203 (2024). https://doi.org/10.19678/j.issn.1000-3428.0067932. (in Chinese)
4. Crane, R., Sornette, D.: Robust dynamic classes revealed by measuring the response function of a social system. Proc. Natl. Acad. Sci. **105**(41), 15649–15653 (2008)
5. He, X., Liao, L., Zhang, H., Nie, L., Hu, X., Chua, T.S.: Neural collaborative filtering. In: Proceedings of the 26th International Conference on World Wide Web, pp. 173–182 (2017)
6. Hongchang, K., Hui, W., Hongbin, S., Yang, H.: Deep reinforcement learning-based task offloading and service migrating policies in service caching-assisted mobile edge computing. China Commun. **21**(4), 88–103 (2024)
7. Hu, Z., Fang, C., Wang, Z., Tseng, S.M., Dong, M.: Many-objective optimization-based content popularity prediction for cache-assisted cloud-edge-end collaborative IoT networks. IEEE Internet Things J. **11**(1), 1190–1200 (2023)
8. Huang, C.K., Shen, S.H.: Enabling service cache in edge clouds. ACM Trans. Internet Things **2**(3), 1–24 (2021)
9. Kai-Yuan, Z., Xiao-Lin, G., De-Wang, R., Jing, L., Jie, W., Dong-Sheng, R.: Survey on computation offloading and content caching in mobile edge networks. J. Softw. **30**(8), 2491–2516 (2019)
10. Li, H., Sun, M., Xia, F., Xu, X., Bilal, M.: A survey of edge caching: key issues and challenges. Tsinghua Sci. Technol. **29**(3), 818–842 (2023)
11. Li, Y., Li, L., Zhou, Z.: Joint edge caching and computation offloading for heterogeneous tasks in MEC-enabled vehicular networks. Veh. Commun. **50**, 100860 (2024)
12. Liu, D., Chen, B., Yang, C., Molisch, A.F.: Caching at the wireless edge: design aspects, challenges, and future directions. IEEE Commun. Mag. **54**(9), 22–28 (2016)
13. Malik, A., Kim, J., Kim, K.S., Shin, W.Y.: A personalized preference learning framework for caching in mobile networks. IEEE Trans. Mob. Comput. **20**(6), 2124–2139 (2020)
14. Newman, M.E.: Power laws, pareto distributions and zipf's law. Contemp. Phys. **46**(5), 323–351 (2005)
15. Pham, Q.V., et al.: A survey of multi-access edge computing in 5G and beyond: fundamentals, technology integration, and state-of-the-art. IEEE Access **8**, 116974–117017 (2020)

16. Qian, Y., Wang, R., Wu, J., Tan, B., Ren, H.: Reinforcement learning-based optimal computing and caching in mobile edge network. IEEE J. Sel. Areas Commun. **38**(10), 2343–2355 (2020)
17. Sun, Y., Cui, Y., Liu, H.: Joint pushing and caching for bandwidth utilization maximization in wireless networks. IEEE Trans. Commun. **67**(1), 391–404 (2018)
18. Tang, B., Kang, L.: Eicache: a learning-based intelligent caching strategy in mobile edge computing. Peer-to-Peer Network. Appl. **15**(2), 934–949 (2022)
19. Wang, Z., Schaul, T., Hessel, M., Hasselt, H., Lanctot, M., Freitas, N.: Dueling network architectures for deep reinforcement learning. In: International Conference on Machine Learning, pp. 1995–2003. PMLR (2016)
20. Xiao, H., et al.: Transcoding-enabled cloud-edge-terminal collaborative video caching in heterogeneous IoT networks: an online learning approach with time-varying information. IEEE Internet Things J. **11**(1), 296–310 (2023)
21. Ye, X., Li, M., Si, P., Yang, R., Wang, Z., Zhang, Y.: Collaborative and intelligent resource optimization for computing and caching in IoV with blockchain and MEC using a3c approach. IEEE Trans. Veh. Technol. **72**(2), 1449–1463 (2022)
22. Zhang, H., Wang, J., Zhao, Z., Zhao, Z.: A survey of edge caching security: framework, methods, and challenges. J. Syst. Architect. 103306 (2024)
23. Zhang, J., et al.: Joint resource allocation for latency-sensitive services over mobile edge computing networks with caching. IEEE Internet Things J. **6**(3), 4283–4294 (2018)
24. Zheng, R., Wang, H., De Mari, M., Cui, M., Chu, X., Quek, T.Q.: Dynamic computation offloading in ultra-dense networks based on mean field games. IEEE Trans. Wireless Commun. **20**(10), 6551–6565 (2021)
25. Zhou, P., Shen, K., Kumar, N., Zhang, Y., Hassan, M.M., Hwang, K.: Communication-efficient offloading for mobile-edge computing in 5G heterogeneous networks. IEEE Internet Things J. **8**(13), 10237–10247 (2020)

Open World Person Retrieval and Its Distributed Parallel Training Framework

Lan Yan[1(✉)], Wenbo Zheng[2], and Kenli Li[3,4]

[1] School of Computer Science and Technology, Hainan University, Haikou, China
ylan@hainanu.edu.cn
[2] School of Computer Science and Artificial Intelligence, Wuhan University of Technology, Wuhan, China
[3] College of Computer Science and Electronic Engineering, Hunan University, Changsha, China
[4] The Ministry of Education Key Laboratory of "Fusion Computing of Supercomputing and Artificial Intelligence", Hunan University, Changsha, China

Abstract. Person retrieval aims to use images of any modality or type of the target person as query probes to retrieve and locate the target individual in the scene image gallery. However, current researches on person retrieval still focuses on the close world with controlled environments and abundant annotations, which limits its applicability to the open world in Cyber-Physical-Social Systems (CPSS). Open environments are confronted with challenges such as complex and variable scenarios, scarce labels, massive data volume, and scattered storage. To tackle these challenges, we propose a novel CPSS-based framework for open world person retrieval and its distributed parallel training. This framework comprises three tightly integrated components: a physical part that includes image data collected from various sensors; a social part composed of open world person retrieval models; and a cyber part that incorporates distributed parallel training strategies for these models. In addition, we introduce a new evaluation index that balances model performance and computational cost to assess open world person retrieval models and their distributed parallel training strategies. We hope that this work will contribute to advancing research and practical deployment of person retrieval in the open world.

Keywords: Parallel computing · Deep learning · Open world · Person retrieval

1 Introduction: From Close World to Open World

With the improvement of living standards, increasing attention has been paid to personal and property safety, making security a growing public concern. To enhance social and public security, video surveillance systems have been increasingly deployed in diverse public spaces such as roads, airports, shopping malls,

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 311–329, 2026.
https://doi.org/10.1007/978-981-95-8405-5_17

hospitals, and schools. These systems play a vital role in risk prevention, emergency response, and the maintenance of urban safety. The video surveillance market is reportedly valued at 54.42 billion USD in 2024 and is projected to grow to 88.71 billion USD by 2030 [30]. Beyond public spaces, the adoption of smart security cameras in private homes is rapidly growing, with the number of active households expected to reach 578.5 million and household penetration projected to reach 23.9% by 2029 [31]. The widespread deployment of surveillance systems has resulted in the generation of massive volumes of video data, necessitating intelligent methods for effective processing and analysis. Among the key technologies addressing this challenge is person search [2,46], which aims to identify and locate a target individual within a large-scale gallery of images using a visible-light photograph as a probe. However, in the open world scenarios of Cyber-Physical-Social Systems (CPSS) [58,62], a clear visible-light image of the target person is not always available. For instance, in law enforcement and criminal investigations, suspects may commit crimes under poor lighting conditions or in surveillance blind spots, making it difficult for visible-light cameras to capture effective facial or full-body images. In such cases, alternative modalities such as infrared images or forensic sketches based on eyewitness descriptions may serve as the only available probe data. In this paper, we focus on the open world person retrieval task, which involves retrieving and localizing a target individual within a scene image gallery using a query probe of any modality or type, such as an infrared image or a sketch, as shown in Fig. 1.

Fig. 1. Illustration of the open world person retrieval task in CPSS.

Meanwhile, with the rapid development of technologies such as big data and distributed parallel computing, the flow and transparency of information related to human society have significantly increased. The interactions among cyberspace, physical space, and social space have become increasingly rapid and expansive. Consequently, the design and operation of Cyber-Physical Systems (CPS) [11] should no longer regard humans merely as external entities interacting with the system. Therefore, CPSS [58] are extensions of CPS and is intended to complement the human and social factors that have been neglected in traditional CPS. Parallel computing technologies provide CPSS with powerful data

processing capabilities, improved responsiveness, and enhanced scalability. The operational efficiency and intelligence of CPSS are highly dependent on distributed parallel computing technologies.

At present, the related research on person retrieval primarily focuses on closed datasets with single scenarios and abundant annotations. However, in the open world, as illustrated in Fig. 2, surveillance cameras deployed across diverse settings (e.g., schools, parks, supermarkets, stations, and streets) capture data with highly dynamic imaging conditions and complex scenes. These data are typically distributed across multiple devices and exhibit massive scale. Under these circumstances, large-scale manual annotation of data is extremely labor- and resource-intensive, making extensive annotation infeasible in the open world, resulting in insufficient labeled samples. These challenges cause existing methods to suffer from limited robustness, transferability, and generalization ability, yielding suboptimal retrieval performance that fails to meet the demands of practical intelligent surveillance systems in open environments. Furthermore, due to the distributed and large-scale nature of surveillance data storage, conventional single-device training paradigms for person retrieval models suffer from slow speed, high maintenance costs, and computational inadequacy, hindering effective utilization of enormous scale data in the open world. Therefore, to improve computational efficiency, reduce maintenance costs, and fully leverage the available data, it is urgent to improve the training speed of the model through distributed parallel training.

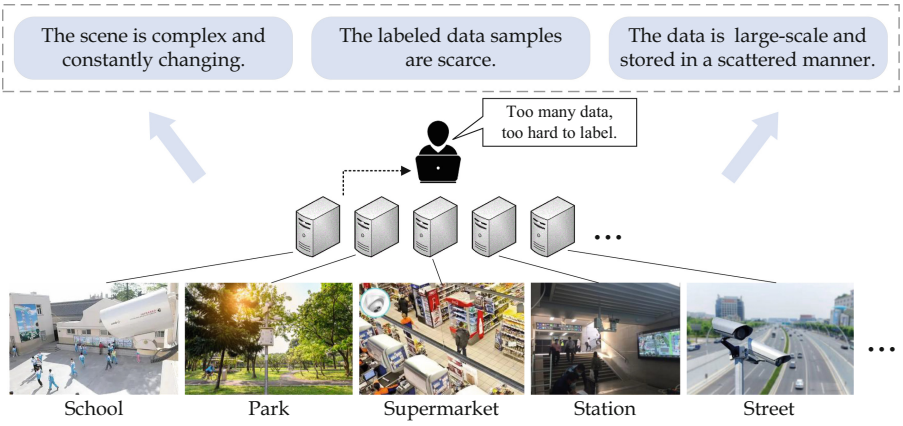


Fig. 2. Challenges of person retrieval in the open world.

However, most existing distributed parallel training approaches are designed for classical computer vision tasks such as image classification [13] and object detection [42], rather than for person retrieval. Moreover, inappropriate training strategies under large-scale data scenarios may lead to a rapid decline in the model's generalization performance [20]. Therefore, this paper also investigates

how to design targeted distributed parallel training strategies for person retrieval algorithms in the open world.

To address the challenges of complex and dynamic scenarios, limited sample annotations, decentralized large-scale data in the open world, we propose a novel framework based on CPSS for open world person retrieval and its distributed parallel training. This framework deeply integrates the interactions among the cyber, physical, and social spaces, fully accounting for the complexity and specificity of open world person retrieval. It consists of three closely related parts: (1) the physical part, comprising image data collected from various sensors; (2) the social part, encompassing open world person retrieval models; and (3) the cyber part, which includes distributed parallel training strategies of person retrieval models. The proposed framework overcomes the limitations of previous studies where individual components were treated in isolation. By organically combining the physical, social, and cyber components, it establishes a comprehensive ecosystem spanning data acquisition, model processing, and efficient training.

Furthermore, to more comprehensively evaluate the performance of person retrieval models and their distributed training strategies in the open world, we propose a novel evaluation index. Conventional metrics often focus unilaterally on either computational cost or model performance, failing to accurately reflect a model's comprehensive capabilities in complex and dynamic conditions of the open world. In contrast, the proposed metric innovatively integrates both computational cost and performance, conducting evaluations from multiple dimensions. It more authentically reflects the operational efficacy of models in real-world applications, thereby providing more rational and reliable basis for model optimization and practical deployment.

In summary, the main contributions of this work are threefold:

- A novel open world framework in CPSS for person retrieval is proposed, overcoming the limitations of conventional approaches that address the problem solely from a single technical perspective.
- A new evaluation index that comprehensively considers computational complexity and performance is designed, which improves the evaluation system for person retrieval models in the open world.
- By focusing on the open world, a scenario of great practical significance and challenge, the research on person retrieval is extended from the closed and ideal laboratory environment to the real and dynamic real world, significantly expanding the application boundaries of person retrieval technology.

2 Open World Framework in CPSS for Person Retrieval

In this section, we introduce the proposed open world framework in CPSS for person retrieval in detail. The CPSS is composed of three parts: the physical system, the social system, and the cyber system. The physical system includes various entities in the real world and their interactive physical processes. Through sensor networks and intelligent technologies, data from the physical system are

collected and transmitted to the cyber system. The social system consists primarily of individuals, organizations, and complex social relationships within human society. It interacts with the physical system and communicates with the cyber system mainly through the Internet, serving as both a source and recipient of data. The cyber system serves as the central hub for data storage and processing, integrating data from both the physical and social systems. It performs analysis and decision-making, and acts upon the physical system via control networks and human interventions. Accounting for the complexity and specificity of person retrieval in the open world, the proposed framework consists of three core parts, namely the physical (**P**), social (**S**), and cyber (**C**) parts, as shown in Fig. 3.

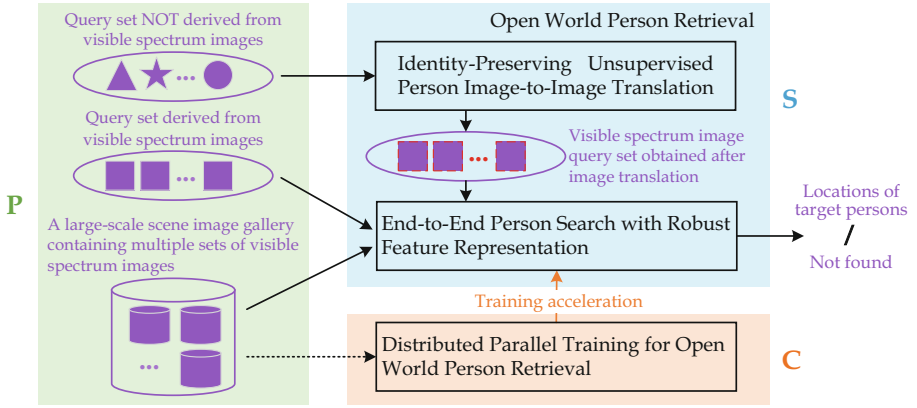


Fig. 3. The proposed open world framework in CPSS for person retrieval.

2.1 P Part

Person retrieval aims to identify and locate a target individual within a panoramic image gallery composed of visible spectrum images, using a query image of any modality or type. Accordingly, the physical part of the proposed framework comprises image data collected by a variety of sensors, as shown in Fig. 4. In the open world, numerous sensing devices such as visible-light cameras and infrared sensors, like the "eye" spread throughout the city, continuously collect image data containing human information. These data are sourced from diverse and dynamic real-world scenes, including streets, shopping malls, and transportation hubs, and are influenced by various physical environmental factors such as weather conditions, illumination, and camera angles. This part serves not only as the data foundation of the entire framework but also as the critical bridge between the physical world and social as well as cyber information processing. The quality and diversity of the collected data directly impact the accuracy and reliability of subsequent person retrieval models.

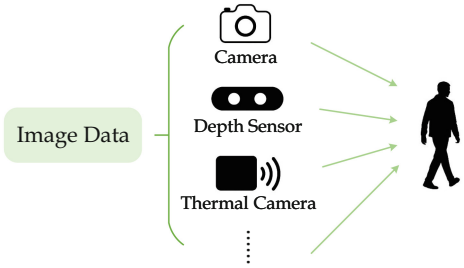


Fig. 4. The P part of our framework.

As the most fundamental data acquisition devices, visible-light cameras are capable of capturing visual characteristics of individuals, including appearance, clothing, and hairstyle. These devices are extensively deployed in public areas such as schools, parks, supermarkets, stations and streets, forming a pervasive visual perception network that covers the entire urban space. However, the quality of data collected by visible-light cameras deteriorates significantly in challenging conditions such as low illumination, nighttime, or heavy smog. In such scenarios, infrared sensors demonstrate unique advantages by detecting thermal radiation emitted from object surfaces to produce infrared imagery. This modality enables clear representation of human silhouettes and positional information even in complete darkness or adverse weather conditions, thereby compensating for the limitations of visible-light data.

Person retrieval data are typically characterized by substantial diversity and dynamism. The diversity manifests in both the breadth of data sources and the richness of modalities. Human behavioral patterns, clothing styles, and environmental backgrounds vary significantly across different scenarios, all of which are reflected in the collected data. The dynamism originates from the temporal variability of the open world. On the one hand, the behaviors and postures of people are continuously changing, resulting in appearance variations for the same person at different moments. On the other hand, environmental factors such as light intensity, weather conditions, and crowd density also fluctuate over time. These dynamic changes make the collected data always in an updated state, forming a dynamically evolving dataset.

The diversity of data facilitates the learning of more generalized feature representations,, enabling robust performance when handling person retrieval tasks across diverse scenarios and conditions. For instance, during model training, the multi-modal and diverse data provided by the physical parts allow the model to effectively capture feature patterns of individuals under different environmental conditions, thereby improving its adaptability and retrieval accuracy in open environments. Consequently, the physical part serves not merely as a data collector, but more critically as the fundamental prerequisite and essential guarantee for achieving efficient and accurate person retrieval within the entire framework.

2.2 S Part

In our framework, the social part serves as the core processing unit, integrating a series of open world person retrieval models, as shown in Fig. 3. By leveraging advanced artificial intelligence techniques, this part extracts meaningful information from the heterogeneous image data provided by the physical part, enabling stable and accurate person retrieval in complex and dynamic scenes. As shown in Fig. 5, its performance directly influences the overall applicability and retrieval accuracy of our framework in the open world.

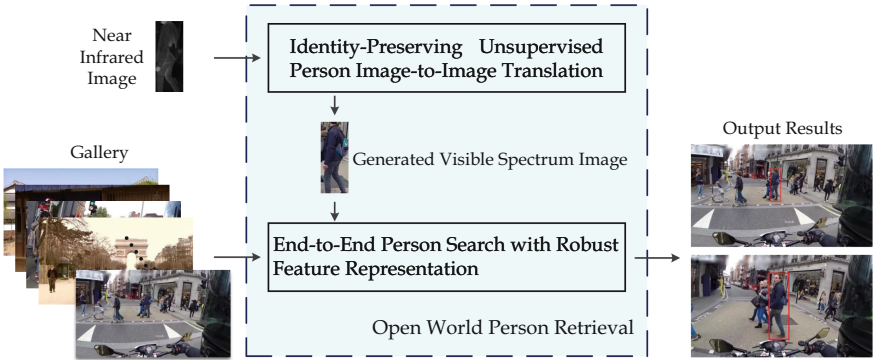


Fig. 5. The S part of our framework.

Unlike the conventional person search task that use only visible-light photographs as query probes, person retrieval allows the use of other modalities images of the target individual, such as sketch portraits, near-infrared images, or far-infrared images. These other modalities images, although containing key information about individuals, exhibit significant differences from visible spectrum images in terms of imaging principles, visual characteristics, and data representations. As a result, they cannot be directly utilized by conventional person search algorithms designed for visible spectrum inputs. Therefore, the social part introduces image translation techniques to convert sketches, infrared images, and other non-visible modalities into visible spectrum images, thereby decomposing complex multi-modal retrieval problems into image translation and visible-image-based person search tasks. For conciseness, we abbreviate this translation process as person image-to-image translation.

Due to the complex scenarios and insufficient sample labels in the open world, it is difficult to obtain a large number of paired images for supervised training of person image-to-image translation models. Moreover, most existing unsupervised image-to-image translation methods aim to generate realistic results without preserving the identity information. Direct application of these methods to person image-to-image translation often leads to identity information loss, which significantly compromises subsequent person identification and matching performance.

To address this issue, our framework incorporates an identity-preserving unsupervised image-to-image translation method, which minimizes identity information loss during the translation process to facilitate subsequent person identity recognition and matching.

Once multi-modal query probes are uniformly translated into visible spectrum images using an identity-preserving unsupervised image-to-image translation method, person search becomes the critical step in achieving the final retrieval objective. The core task of person search is to match and localize the target individual in a large-scale panoramic image gallery using a visible spectrum image as the query probe. However, in the open world, the scenes of data collection are complex and variable. Moreover, the prohibitive cost of manual annotation makes it impractical to acquire sufficient labeled training data. These constraints significantly limit the effectiveness of conventional person search methods designed for closed, single-scenario applications. When confronted with challenging conditions such as extreme illumination variations, severe occlusions, and cross-camera viewpoint discrepancies, these methods exhibit substantial limitations in model generalization capability, failing to meet the practical requirements of open world person retrieval. Consequently, the proposed framework emphasizes the adoption of person search models capable of learning robust pedestrian representation features under complex scenarios, thereby ensuring stable retrieval performance in the open world.

2.3 C Part

As the core module for addressing data dilemma and enabling efficient training, the cyber part is facing the serious challenge of scattered and large-scale data in the open world. In such settings, massive volumes of data are continuously generated and dispersed across surveillance devices, mobile terminals, and various intelligent systems throughout the city, growing by terabytes or more each day. Under these circumstances, conventional single-machine training paradigms reveal multiple inherent limitations: not only do they demonstrate prohibitively low training efficiency, failing to complete model iterations within an acceptable time frame, but their exorbitant maintenance costs also render large-scale data processing unsustainable. More critically, such centralized training architectures cannot effectively integrate distributed data sources, thereby severely limiting the extraction of deep feature representations and ultimately compromising model generalization capability. Therefore, developing distributed parallel training strategies specifically adapted for open world person retrieval algorithms emerges as an inevitable choice to unleash model potential and facilitate technological deployment.

To facilitate the deployment of open world person retrieval models into various intelligent surveillance systems to meet practical application needs, the model should not have an excessively large number of parameters, and the model training speed should be as fast as possible without compromising performance. Thus, a data parallel strategy centered on large-batch training can be

adopted, which directly improves data processing efficiency by splitting large-scale datasets into smaller sub-batches and distributing them to multiple computing nodes for parallel processing. However, different from classic visual tasks such as image classification and object detection, person retrieval models have unique structural characteristics and task requirements, needing to complete complex identity matching and positioning tasks. Their architecture typically integrates multiple components including feature extraction and retrieval matching. Direct application of existing large-batch training algorithms may not only fail to optimize model performance but could also induce training instability and degraded generalization capability. Notably, the proposed cyber part employs distributed parallel training strategies specifically designed for open world person retrieval models.

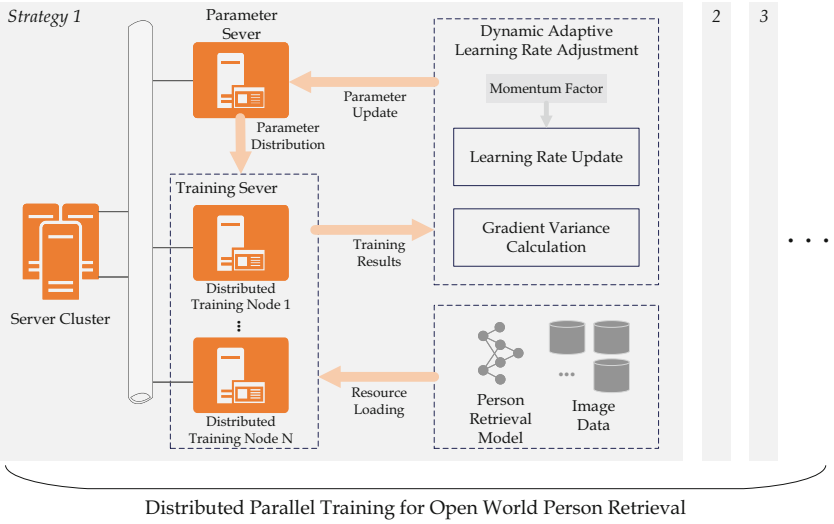


Fig. 6. The C part of our framework.

As displayed in Fig. 6, the distributed parallel training strategies can be systematically optimized by integrating the network architecture and task-specific characteristics of person retrieval models. For data parallelism, a dynamic adaptive learning rate adjustment mechanism can be employed. It intelligently adjusts the training learning rate by monitoring the gradient distribution, model convergence speed and other key indicators in the training process in real time to maximize the training efficiency under the premise of ensuring model stability. In terms of the model parallelism, for the complex architecture of model multi-module collaboration, a hierarchical module partitioning strategy is adopted. Model components are distributed across nodes based on their functional roles and computational demands. In addition, a compression-aware communication

algorithm can be introduced to reduce inter-node transmission overhead effectively. These strategies not only make full use of distributed computing resources but also break through the limitations of traditional training methods in data scale and computational efficiency, providing strong support for the rapid iteration and optimization of models in open environments.

3 A Case Study for New Scenarios

In this section, we present a case study for new scenarios. First, we propose a novel evaluation index that balances computational cost and retrieval performance for assessing person retrieval models and their distributed training strategies in the open world. Subsequently, we conduct experiments using an end-to-end person search model, followed by detailed quantitative analysis of the experimental results.

3.1 A New Evaluation Index: Balancing Computation and Performance

Considering the numerous challenges present in open environments, the evaluation of person retrieval models and their distributed parallelization strategies should not be limited to a single dimension such as computational cost or model performance alone. Inspired by the Sun-Ni law [36], which considers parallel acceleration under memory constraints, we propose a new evaluation index for assessing open world person retrieval models and their distributed parallelization methods.

Let W denotes the problem size (or workload), n represents the number of processors in the parallel system, and W_s indicates the workload of the serial component. For the open world person retrieval task, we regard the model size, i.e., the number of model parameters, as the workload. The proportion of the serial component is then given by $f = W_s/W$. The proposed evaluation index S is formally defined by the following equation:

$$S = \frac{W_s + (1-f)WG}{W_s + (1-f)WG/n} \tag{1}$$

where $G = PF/W$ comprehensively reflects the model performance achievable per unit of model parameters and the computational cost involved. Here, P means the model performance, F stands for the total floating point operations required by the model.

3.2 Experimental Analysis

In this subsection, we conduct some experiments to obtain evaluation results based on the proposed index and perform a quantitative analysis of the results. In open world person retrieval, image-to-image translation models are used to

convert the target person's probe image into a unified visible-spectrum modality, after which the person search models perform localization and identification within the scene image gallery. Since the number of probe images used for querying is far smaller than the number of panoramic images in the gallery, the distributed parallel acceleration for person retrieval should mainly focus on the person search component. Therefore, we select the end-to-end person search model SeqNet [26] as a representative example and conduct experiments on the publicly available CUHK-SYSU [46] dataset.

SeqNet is a end-to-end network based on Faster R-CNN, which performs detection and re-identification sequentially. After generating a large number of proposals through detection, it is necessary to further evaluate the features of each proposal and calculate the bounding box scores. This step is highly suitable for parallelization, and thus we improve it using distributed computing techniques.

After completing model training, we evaluate SeqNet on the CUHK-SYSU dataset using mean Average Precision (mAP) as the performance metric. Experimental results show that SeqNet achieves 93.80% of mAP. The calculated total number of model parameters is 48.42M, and the floating point operations amount to 496.58 G. The parameter count for the serial component is 33.43M. Therefore, the proportion of the serial component f is calculated as 0.69, and the balanced parameter G is 9.62. When the number of processors n in the parallel system is set to 2, the evaluation index S can be computed using Equation (1) and is found to be 1.68. Similarly, we calculate the values of S for other values of n, and the results are shown in Fig. 7.

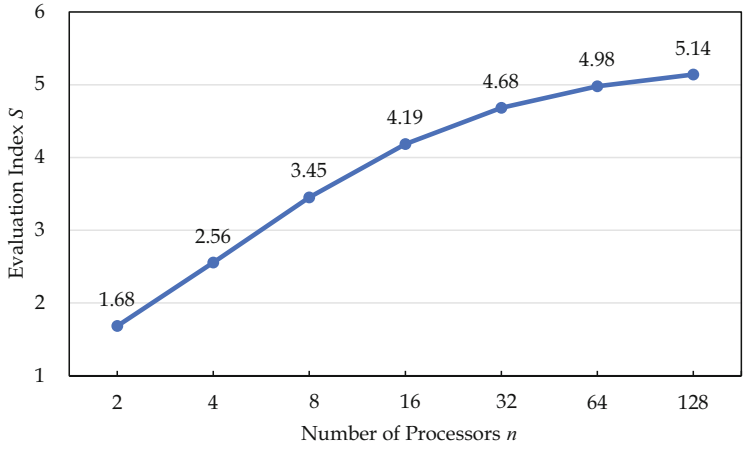


Fig. 7. The experimental results of the proposed evaluation index.

As illustrated in Fig. 4, when the number of processors n is set to 4, 8, 16, 32, 64 and 128, the corresponding evaluation index S is 2.56, 3.45, 4.19, 4.68, 4.98,

and 5.14, respectively. Notably, when n increases from 64 to 128, S only improves from 4.98 to 5.14, with an incremental gain of merely 0.16. This indicates that as the number of processors n increases exponentially, the growth of S slows down, which aligns well with our intuition that the marginal utility of adding processors diminishes in practical parallel computing scenarios. The proposed index effectively captures this phenomenon and appropriately balances model performance and computational cost.

4 Related Work

4.1 Person Retrieval

Since person retrieval can be decomposed into two subtasks: image-to-image translation and person search, we introduce the related work of these two aspects separately as follows.

Image-to-Image Translation. Existing methods can be broadly categorized into supervised and unsupervised approaches. Supervised image-to-image translation methods, such as Pix2Pix [22] and Pix2PixHD [41] based on generative adversarial networks (GANs) [16], require paired source and target domain images for supervised training. In addition to the aforementioned general supervised image-to-image translation approaches, researchers have also developed specialized ones for tasks such as face super-resolution [61], image denoising [52] and face photo-sketch synthesis [48], which have demonstrated promising performance. In contrast, unsupervised image-to-image translation methods eliminate this pairwise dependency. Among them, CycleGAN [63]is a representative model that introduces a cycle consistency constraint to learn mappings between unpaired domains. UNIT [28] implicitly enforces cycle consistency by incorporating the Kullback-Leibler divergence constraint in a shared latent space, while MUNIT [21] extends UNIT to support one-to-many mappings. StarGAN [9] enables multi-domain image translation with a single generator by leveraging domain labels. Inspired by human vision systems, JND-GAN [60] introduces a Just-Noticeable-Difference loss and generates more realistic target domain images. IPGAN [49] introduces the heterogeneous image knowledge [50], focusing on identity information preservation during translation, and achieves excellent unsupervised photo-to-caricature translation performance.

While methods based on cycle consistency constraint generally perform well at transferring local texture features but often struggle with geometric transformations. To address this issue, GANimorph [15] introduces deformable convolutions in the discriminator to enlarge its receptive field and reformulates the discrimination task as a semantic segmentation problem to encourage more context-aware generation. CUT [32] maximizes mutual information between corresponding patches before and after translation by treating patches at the same spatial location as positive pairs and those at different locations as negatives. It also incorporates multi-scale information to compute the contrastive loss. Building on CUT, NEGCUT [43] introduces a hard negative mining mechanism to

enhance the quality of negative samples, while SRC [24] further regularizes the consistency of similarity relationships before and after translation to preserve spatial semantic structure and improve performance.

Beyond GAN-based methods, recent research has also explored image-to-image translation approaches based on diffusion models [19], which offer alternative generative paradigms for high-quality and flexible image synthesis. For instance, Xiao et al. [44] present a denoising Brownian bridge model with dual approximators for image-to-image translation.

Person Search. Existing person search methods can be categorized into one-stage and two-stage methods. One-stage methods are typically modified from pedestrian detection models to achieve person search using a single model in an end-to-end manner. Early single-stage methods [45,46] often adopt the Faster R-CNN [33] detection framework, which attaches a fully connected layer at the tail end to output person re-identification features. However, these approaches generate excessive candidate bounding boxes at intermediate layers, resulting in high storage and computational overhead. Thus, Liu et al. [27] propose a recursive search strategy that progressively refines target locations without proposal generation. Similarly, Chang et al. [5] develop a method based on deep reinforcement learning that performs person search without relying on candidate proposals. Yan et al. [55] introduce an anchor-free person search framework to further reduce dependency on predefined anchor boxes.

In addition to reducing the number of proposals, improving the discriminability of pedestrian representation is also an important research direction. Chen et al. [7] project person representation vectors into polar coordinates, using the vector magnitude to distinguish between background and persons, and the angle component for re-identification. Yu et al.[57] adopt a cascading approach based on the Cascade R-CNN [3] to learn more discriminative person representations from coarse to fine. Cao et al. [4] treat person search as a sequence prediction problem and proposed an end-to-end solution based on Transformer [12]. Zhang et al. [59] propose a hierarchical architecture to jointly optimize instance-aware and part-aware embeddings. Yan et al. [47] introduce an additional unknown-aware branch to fully leverage unlabeled or unknown instances and facilitate the learning of discriminative features.

In contrast, two-stage approaches decouple pedestrian detection and person re-identification into two distinct models. Cheng et al. [6] design a mask-guided two-branch convolutional neural network, where one branch models foreground pedestrians while the other processes the original image. Han et al. [18] develop a differentiable region of interest transformation module to drive the re-identificator to fine-tune the detector results, enabling pedestrian bounding boxes to exclude interference from the background and non-target pedestrians. Wang et al. [39] employed hybrid training datasets combining ground-truth and detected boxes to reduce train-test discrepancy.

Meanwhile, some research efforts focus on weakly supervised person search paradigms. One type of weak supervision setting is that only the identities

present in the gallery are known, without information about the specific images or positions where these individuals are located [51]. That is, each image contains neither identity annotations nor bounding box annotations. Built upon this setting, a weakly supervised sketch-based person search method [53] is proposed. Another type of weak supervision setting [17,38,54] provides bounding box annotations while omitting pedestrian identity labels. In addition, Li et al. [25] introduce an unsupervised domain adaptation setting into person search and propose a DAPS framework. Almansoori et al. [1] further improve this framework by introducing a domain adaptive bridging mechanism.

4.2 Distributed Parallel Training of Deep Neural Networks

The distributed parallel training of deep neural networks is mainly divided into three categories: data parallelism, model parallelism, and hybrid parallelism. In data parallelism, the training dataset is partitioned and distributed across multiple worker nodes, each maintaining a complete replica of the model. After each iteration, the gradients computed by each node are synchronized and aggregated to update the global model parameters before proceeding to the next iteration. Model parallelism decomposes large models into sub-models distributed across worker nodes, or allocates inter-layer parameters to different hardware devices for concurrent training. Hybrid parallelism integrates both data and model parallelization mechanisms to optimize training efficiency.

In data parallelism, training with large batches effectively increases throughput and reduces the time required for each iteration cycle. As a result, large-batch training to maximize data parallelism has become an important research direction. However, training with large batches can lead to rapid convergence to sharp local minima, which negatively impacts generalization performance [10]. Some researchers [34] eliminate the generalization performance gap by manually adjusting the learning rate. Recently, adaptive learning rate adjustment techniques have gained significant attention. For example, LARS [56] adjusts the learning rate of each layer based on the magnitude of its weights and gradients, enabling ResNet-50 to be trained on ImageNet with a batch size of 32k while substantially reducing training time. Liu et al.[29] further expand the batch size to 96k using the idea of adversarial learning. SAM [14] introduces a strategy that jointly minimizes the loss value and its sharpness to close the generalization gap, but its requirement for two gradient computations per iteration makes it less computationally efficient. To address this problem, Du et al. [13] propose a stochastic weight perturbation mechanism combined with a sharpness-sensitive data selection strategy.

Regarding model parallelism, earlier strategies are designed and implemented based on expert experience. For example, Sutskever et al. [37] allocate different layers of deep learning models to heterogeneous devices according to their distinctive characteristics, thereby achieving distributed parallel training strategies for complex models. Sun et al. [35] propose a random partitioning method based on the backbone network, which reduces model complexity by retaining key operators in the network structure without compromising performance. In

recent years, automatic model parallelism has become a research hotspot. Jia et al. [23] construct a high-dimensional search space under a given cluster environment and build a performance evaluation model to guide the dynamic programming search for optimal parallelization strategies. On this basis, Wang et al. [40] employ recursive partitioning and graph coarsening techniques to reduce the search time for distributed parallel strategies and enhance model scalability. Chen et al. [8] propose an automatic parallel framework, which uses the actual execution time of distributed parallel strategies as a penalty factor for the long short-term memory model to update the model parallelism plan.

5 Conclusion

In this paper, we propose a novel open world person retrieval and its distributed parallel training framework. Inspired by CPSS, our framework consists of three parts, including a physical part that collects image data from various sensors, a social part that incorporates open world person retrieval models, and a cybler part that implements distributed parallel training strategies for open world person retrieval algorithms. Furthermore, to enable a more comprehensive and practical evaluation, we introduce a novel index that jointly considers computational cost and retrieval performance, specifically designed for assessing both the person retrieval model and its training strategy under the open world. The proposed framework and evaluation index offer foundational insights for future research in open world person retrieval.

Acknowledgment. This work is supported in part by the National Natural Science Foundation of China (62302161, 62303361), and also in part by the Natural Science Foundation of Hunan Province, China (2024JJ6155).

References

1. Almansoori, M.K., Fiaz, M., Cholakkal, H.: DDAM-PS: diligent domain adaptive mixer for person search. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision, pp. 6688–6697 (2024)
2. Cai, B., Wang, H., Yao, M., Fu, X.: Focus more on what? guiding multi-task training for end-to-end person search. IEEE Trans. Circuits Syst. Video Technol. (2025)
3. Cai, Z., Vasconcelos, N.: Cascade R-CNN: delving into high quality object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 6154–6162 (2018)
4. Cao, J., Pang, Y., Anwer, R.M., Cholakkal, H., Xie, J., Shah, M., Khan, F.S.: PSTR: end-to-end one-step person search with transformers. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 9458–9467 (2022)
5. Chang, X., Huang, P.Y., Shen, Y.D., Liang, X., Yang, Y., Hauptmann, A.G.: RCAA: relational context-aware agents for person search. In: Proceedings of the European Conference on Computer Vision, pp. 84–100 (2018)

6. Chen, D., Zhang, S., Ouyang, W., Yang, J., Tai, Y.: Person search via a mask-guided two-stream CNN model. In: Proceedings of the European Conference on Computer Vision, pp. 734–750 (2018)
7. Chen, D., Zhang, S., Yang, J., Schiele, B.: Norm-aware embedding for efficient person search. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 12615–12624 (2020)
8. Chen, M., Beutel, A., Covington, P., Jain, S., Belletti, F., Chi, E.H.: Top-k off-policy correction for a REINFORCE recommender system. In: Proceedings of the ACM International Conference on Web Search and Data Mining, pp. 456–464 (2019)
9. Choi, Y., Choi, M., Kim, M., Ha, J.W., Kim, S., Choo, J.: StarGAN: unified generative adversarial networks for multi-domain image-to-image translation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 8789–8797 (2018)
10. Dehghani, M., Yazdanparast, Z.: From distributed machine to distributed deep learning: a comprehensive survey. J. Big Data **10**(1), 158 (2023)
11. Derler, P., Lee, E.A., Vincentelli, A.S.: Modeling cyber-physical systems. Proc. IEEE **100**(1), 13–28 (2011)
12. Dosovitskiy, A., et al.: An image is worth 16×16 words: transformers for image recognition at scale. In: International Conference on Learning Representations (2020)
13. Du, J., et al.: Efficient sharpness-aware minimization for improved training of neural networks. In: International Conference on Learning Representations (2022)
14. Foret, P., Kleiner, A., Mobahi, H., Neyshabur, B.: Sharpness-aware minimization for efficiently improving generalization. In: International Conference on Learning Representations (2021)
15. Gokaslan, A., Ramanujan, V., Ritchie, D., Kim, K.I., Tompkin, J.: Improving shape deformation in unsupervised image-to-image translation. In: Proceedings of the European Conference on Computer Vision, pp. 649–665 (2018)
16. Goodfellow, I.J., et al.: Generative adversarial nets. Adv. Neural Inf. Process. Syst. **27** (2014)
17. Han, C., et al.: Weakly supervised person search with region siamese networks. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 12006–12015 (2021)
18. Han, C., et al.: Re-id driven localization refinement for person search. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 9814–9823 (2019)
19. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. Adv. Neural. Inf. Process. Syst. **33**, 6840–6851 (2020)
20. Hongrui, Z., et al.: Survey on network of distributed deep learning training. J. Comput. Res. Dev. **58**(1), 98–115 (2021)
21. Huang, X., Liu, M.Y., Belongie, S., Kautz, J.: Multimodal unsupervised image-to-image translation. In: Proceedings of the European Conference on Computer Vision, pp. 172–189 (2018)
22. Isola, P., Zhu, J.Y., Zhou, T., Efros, A.A.: Image-to-image translation with conditional adversarial networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 1125–1134 (2017)
23. Jia, Z., Lin, S., Qi, C.R., Aiken, A.: Exploring hidden dimensions in parallelizing convolutional neural networks. In: Proceedings of the International Conference on Machine Learning, vol. 2279, p. 2288 (2018)

24. Jung, C., Kwon, G., Ye, J.C.: Exploring patch-wise semantic relation for contrastive learning in image-to-image translation tasks. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 18260–18269 (2022)
25. Li, J., Yan, Y., Wang, G., Yu, F., Jia, Q., Ding, S.: Domain adaptive person search. In: European Conference on Computer Vision, pp. 302–318 (2022)
26. Li, Z., Miao, D.: Sequential end-to-end network for efficient person search. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 35, pp. 2011–2019 (2021)
27. Liu, H., et al.: Neural person search machines. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 493–501 (2017)
28. Liu, M.Y., Breuel, T., Kautz, J.: Unsupervised image-to-image translation networks. Adv. Neural Inf. Process. Syst. **30** (2017)
29. Liu, Y., Chen, X., Cheng, M., Hsieh, C.J., You, Y.: Concurrent adversarial learning for large-batch training. In: 10th International Conference on Learning Representations (2022)
30. MarketsandMarkets: Video surveillance market by image sensor, lens, image processing, analog, ip camera, dome, ptz, box & bullet, panoramic, fisheye, body-worn, encoders, ai-based vms, video content analysis, storage devices (digital, network) - global forecast to 2030 (2025). https://www.marketsandmarkets.com/Market-Reports/video-surveillance-market-645.html. Accessed 05 Jun 2025
31. Market.us: Global smart security system market size, share, statistics analysis report by component (hardware, software, services), by application (residential, commercial, industrial, government), by connectivity (wired, wireless), region and companies - industry segment outlook, market assessment, competition scenario, trends and forecast 2025-2034 (2025). https://market.us/report/smart-security-system-market. Accessed 06 Jun 2025
32. Park, T., Efros, A.A., Zhang, R., Zhu, J.Y.: Contrastive learning for unpaired image-to-image translation. In: Proceedings of the European Conference on Computer Vision, pp. 319–345 (2020)
33. Ren, S., He, K., Girshick, R., Sun, J.: Faster R-CNN: towards real-time object detection with region proposal networks. Adv. Neural Inf. Process. Syst. **28** (2015)
34. Shallue, C.J., Lee, J., Antognini, J., Sohl-Dickstein, J., Frostig, R., Dahl, G.E.: Measuring the effects of data parallelism on neural network training. J. Mach. Learn. Res. **20**(112), 1–49 (2019)
35. Sun, S., Chen, W., Bian, J., Liu, X., Liu, T.Y.: Slim-DP: a multi-agent system for communication-efficient distributed deep learning. In: Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems, pp. 721–729 (2018)
36. Sun, X.H., Ni, L.M.: Scalable problems and memory-bounded speedup. J. Parallel Distrib. Comput. **19**(1), 27–37 (1993)
37. Sutskever, I., Vinyals, O., Le, Q.V.: Sequence to sequence learning with neural networks. Adv. Neural Inf. Process. Syst. **27** (2014)
38. Wang, B., Yang, Y., Wu, J., Qi, G.j., Lei, Z.: Self-similarity driven scale-invariant learning for weakly supervised person search. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 1813–1822 (2023)
39. Wang, C., Ma, B., Chang, H., Shan, S., Chen, X.: TCTS: a task-consistent two-stage framework for person search. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11952–11961 (2020)
40. Wang, M., Huang, C.C., Li, J.: Supporting very large models using automatic dataflow graph partitioning. In: Proceedings of the Fourteenth EuroSys Conference, pp. 1–17 (2019)

41. Wang, T.C., Liu, M.Y., Zhu, J.Y., Tao, A., Kautz, J., Catanzaro, B.: High-resolution image synthesis and semantic manipulation with conditional GANs. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 8798–8807 (2018)
42. Wang, T., et al.: Large batch optimization for object detection: Training COCO in 12 minutes. In: Vedaldi, A., Bischof, H., Brox, T., Frahm, JM. (eds.) Computer Vision – ECCV 2020. ECCV 2020. LNCS, vol. 12366, pp. 481–496. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-58589-1_29
43. Wang, W., Zhou, W., Bao, J., Chen, D., Li, H.: Instance-wise hard negative example generation for contrastive learning in unpaired image-to-image translation. In: Proceedings of the IEEE/CVF International Conference on Computer Vision, pp. 14020–14029 (2021)
44. Xiao, B., Wang, P., He, Q., Dong, M.: Deterministic image-to-image translation via denoising brownian bridge models with dual approximators. In: Proceedings of the Computer Vision and Pattern Recognition Conference, pp. 28232–28241 (2025)
45. Xiao, J., Xie, Y., Tillo, T., Huang, K., Wei, Y., Feng, J.: IAN: the individual aggregation network for person search. Pattern Recogn. **87**, 332–340 (2019)
46. Xiao, T., Li, S., Wang, B., Lin, L., Wang, X.: Joint detection and identification feature learning for person search. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 3415–3424 (2017)
47. Yan, L., Li, K.: Unknown instance learning for person search. In: 2024 IEEE International Conference on Multimedia and Expo (ICME), pp. 1–6. IEEE (2024)
48. Yan, L., Zheng, W., Gou, C., Wang, F.Y.: IsGAN: identity-sensitive generative adversarial network for face photo-sketch synthesis. Pattern Recogn. **119**, 108077 (2021)
49. Yan, L., Zheng, W., Gou, C., Wang, F.Y.: IPGAN: identity-preservation generative adversarial network for unsupervised photo-to-caricature translation. Knowl.-Based Syst. **241**, 108223 (2022)
50. Yan, L., Zheng, W., Wang, F.Y.: Heterogeneous image knowledge driven visual perception. IEEE/CAA J. Automatica Sinica **11**(1), 255–257 (2024)
51. Yan, L., Zheng, W., Wang, F.Y., Gou, C.: Weakly supervised person search. In: IEEE 7th International Conference on Data Science and Advanced Analytics, pp. 188–196 (2020)
52. Yan, L., Zheng, W., Wang, F.Y., Gou, C.: Joint image-to-image translation with denoising using enhanced generative adversarial networks. Signal Process. Image Commun. **91**, 116072 (2021)
53. Yan, L., Zheng, W., Wang, F.Y., Gou, C.: Weakly supervised sketch based person search. In: Proceedings of the 2021 International Conference on Multimedia Retrieval, pp. 491–495 (2021)
54. Yan, Y., Li, J., Liao, S., Qin, J., Ni, B., Lu, K., Yang, X.: Exploring visual context for weakly supervised person search. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, pp. 3027–3035 (2022)
55. Yan, Y., et al.: Anchor-free person search. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7690–7699 (2021)
56. You, Y., Zhang, Z., Hsieh, C.J., Demmel, J., Keutzer, K.: Imagenet training in minutes. In: Proceedings of the International Conference on Parallel Processing, pp. 1–10 (2018)
57. Yu, R., et al.: Cascade transformers for end-to-end person search. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7267–7276 (2022)

58. Zeng, J., Yang, L.T., Lin, M., Ning, H., Ma, J.: A survey: cyber-physical-social systems and their system-level design methodology. Futur. Gener. Comput. Syst. **105**, 1028–1042 (2020)
59. Zhang, P., Yu, X., Bai, X., Wang, C., Zheng, J., Ning, X.: Joint discriminative representation learning for end-to-end person search. Pattern Recogn. **147**, 110053 (2024)
60. Zheng, W., Yan, L., Gou, C., Wang, F.Y.: JND-GAN: human-vision-systems inspired generative adversarial networks for image-to-image translation. In: ECAI 2020, pp. 2816–2823 (2020)
61. Zheng, W., Yan, L., Zhang, W., Gou, C., Wang, F.Y.: Guided cyclegan via semi-dual optimal transport for photo-realistic face super-resolution. In: IEEE International Conference on Image Processing, pp. 2851–2855 (2019)
62. Zheng, W., Yan, L., Zhang, W., Ouyang, L., Wen, D.: D→ K→ I: data-knowledge-driven group intelligence framework for smart service in education metaverse. IEEE Trans. Syst. Man Cybern. Syst. **53**(4), 2056–2061 (2022)
63. Zhu, J.Y., Park, T., Isola, P., Efros, A.A.: Unpaired image-to-image translation using cycle-consistent adversarial networks. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 2223–2232 (2017)

WSRNet: A Multi-task Learning Model for Satellite Image Recognition and Segmentation of Forest Wildfires

Juan Luo(✉), Tianyang Liu, Kexuan Feng, Guogen Zeng, and Fan Li

The College of Computer Science and Electronic Engineering, Hunan University, Changsha, China
juanluo@hnu.edu.cn

Abstract. Thermal infrared remote sensing is widely used for wildfire monitoring, but its low spatial resolution makes it difficult to accurately delineate fire boundaries, especially in small-scale or edge cases. To address this, we constructed a wildfire image dataset for the Australian region, named WildfireForest-S2, which better aligns the model with the satellite monitoring environment. In addition, we propose WSRNet, an image segmentation approach based on U-Net, specifically designed for satellite-based wildfire detection. WSRNet integrates classification information to refine and guide segmentation features, enhancing the accuracy and efficiency of wildfire detection in satellite images. The model is capable of not only identifying the presence of wildfire events in a given image but also segmenting fire regions with high precision. Experimental validation on the WildfireForest-S2 dataset demonstrates that WSRNet achieves notable improvements in fire region segmentation. At the same time, it also demonstrates strong performance in fire classification, greatly enhancing the accuracy of wildfire monitoring.

Keywords: Multi-task learning · fire monitoring · satellite imagery

1 Introduction

Wildfires are a natural disaster that cause significant losses worldwide [1]. They threaten human lives and have multiple impacts on the environment, economy, and society. In 2019, Australia experienced one of the worst wildfires in its history. This disaster led to at least 33 deaths, thousands of injuries, and the destruction of over 3,000 buildings, with losses estimated in the billions of dollars. This disaster highlights the importance of timely and effective fire monitoring. Good monitoring can quickly identify fire sources, reduce losses, and provide critical support for emergency response.

Among the current fire monitoring methods, drone detection, manual patrols, and sensor monitoring can offer some fire warnings, but each has clear limitations. Drone detection employs high-definition cameras and thermal imaging for real-time aerial monitoring capable of covering extensive areas. However, its

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 330–348, 2026.
https://doi.org/10.1007/978-981-95-8405-5_18

operational duration is limited by battery life, and operating in harsh weather conditions can be challenging, potentially leading to inaccuracies in data acquisition.

Manual patrols are the oldest monitoring method, that rely on trained professionals to regularly check specific areas. Its main limitations are inefficiency, limited coverage, and susceptibility to subjective human judgment which can result in missed detections.

Sensor monitoring tracks environmental changes in real time through the ground sensor networks. Although it can quickly detect fires, its effective range is limited, and the layout and maintenance costs are high.

In contrast, satellite remote sensing can gather information over large areas. It helps identify fires by analyzing satellite images. The main advantages of satellite monitoring are its wide coverage and efficient data collection. This makes satellite remote sensing have a unique value in fire monitoring and can make up for the shortcomings of traditional fire monitoring methods.

Sentinel-2 is part of the European Space Agency's (ESA) Copernicus program and features high spatial resolution (ranging from 10 to 60 m) and multispectral imaging capabilities, which provide significant support for fire monitoring [2]. However, how to achieve high-precision fire monitoring through satellite data remains a challenge that needs to be addressed.

To address these issues, we have adopted three strategies:

Strategy 1: Sentinel-2 data consists of 13 spectral bands, with ground resolutions of 10 m, 20 m, and 60 m. We conducted a selection process for each spectral band and their combinations, ultimately choosing bands B12, B11, and B8a for our analysis. This combination effectively highlights the characteristics of fire-affected areas, ensuring the feasibility of fire monitoring and wildfire segmentation.

Taking the 2019 Australian wildfire as an example, we processed all collected satellite data to create the WildfireForest-S2 dataset. The satellite images were sourced from various regions during critical time points of the Australian fire events, ensuring the dataset's usability for analysis.

Strategy 2: We propose WSRNet, a U-Net-based architecture designed to improve the segmentation of fire-affected areas. To enhance feature quality and enable the model to better focus on critical regions, we introduce the HAFE module. In addition, we adopt a multi-task learning strategy and propose the CGFR module, which uses classification information to guide the segmentation task during joint fire detection and segmentation.

Our contributions can be summarized as follows:

- We constructed a satellite image dataset called WildfireForest-S2 focusing on the Australian wildfire. WildfireForest-S2 is more closely aligned with satellite monitoring scenarios compared to existing datasets.
- We innovatively propose the WSRNet method, which combines the U-Net architecture with modules such as HAFE and CGFR. Through a multi-task learning approach, the model simultaneously performs classification and seg-

mentation tasks, providing technical support for satellite wildfire monitoring tasks.
- By applying the WSRNet method, we significantly improved the performance of satellite wildfire monitoring tasks and achieved a maximum accuracy of 99.53% in the classification task.

2 Related Work

2.1 Wildfire Datasets

Northern Arizona University and other institutions jointly released the FLAME and FLAME 2 datasets, including visible light imagery and infrared thermographs collected by drones during planned burns of accumulated debris in pine forests in Arizona [3,4]. These datasets comprehensively and multi-angularly capture critical information from the fire scenes. The FLAME 3 dataset was created by Clemson University and several partner institutions [5]. It includes synchronously collected visible spectrum and thermal imaging images, providing high-resolution RGB and thermal TIFF files, which is suitable for computer vision and wildfire modeling tasks.

The wildfire dataset includes 2,700 aerial and ground images, which are planned from various online platforms such as government databases, Flickr, and Unsplash [6]. Mesogeos is a large-scale, multi-purpose dataset specifically designed to wildfire modeling in the Mediterranean region. It integrates variables representing wildfire drivers—such as weather, vegetation, and human activity—along with 17 years (2006–2022) of historical wildfire ignition and burn area records [7].

The Fire Dunnings dataset offers more granularity, and is divided into two categories: fire and non-fire classes, including flame images segmented into superpixels, enabling precise localization and recognition [8].

The FireNet dataset focuses on the application of lightweight networks, featuring videos and images of fire and non-fire scenarios, making it particularly suitable for real-time fire detection in IoT environments [9].

The existing datasets primarily consist of ground and drone images. WildfireForest-S2 is specifically designed for satellite scenarios, aiming to facilitate on-board forest fire monitoring.

2.2 Wildfire Monitoring

Currently, fire monitoring still relies on traditional methods such as manual surveillance and drone detection [10–12]. Rodrigo De la Fuente et al. proposed a forest fire monitoring system that integrates monitoring towers, surveillance balloons, and drone technology. They developed a compact mixed-integer linear programming model that includes location decisions for various monitoring technologies and route planning for drones between monitoring facilities to achieve the most extensive terrain coverage. [13].

Liu et al. created a dataset using UAV aerial photography for image fusion and proposed a method of forest fire identification that combines visual and infrared images,which sloved the problem of the high rates of false positives and missed detections in forest fire monitoring [14].

While ground monitoring and UAV monitoring methods have their advantages, they also exhibit significant limitations, being susceptible to issues such as monitoring costs, coverage areas, weather factors, and natural conditions. In contrast, satellite remote sensing technology provides extensive and real-time fire monitoring capabilities, overcoming the spatial and temporal deficiencies of ground and UAV methods.

Moreover, satellite monitoring is not constrained by ground conditions or weather, allowing for continuous operation in complex environments and enabling comprehensive surveillance of fire events [15–19].

Current satellite fire monitoring data often relies on radar and various sensors, lacking image-based fire satellite data. To address this gap in satellite monitoring scenarios, we collected image data during Australian fire events using Sentinel-2 to create the WildfireForest-S2 dataset and proposed the WSRNet method. This method leverages the rich visual information captured by satellites to improve the accuracy and efficiency of fire monitoring efforts.

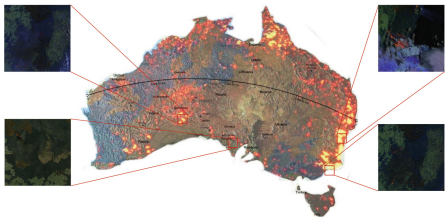


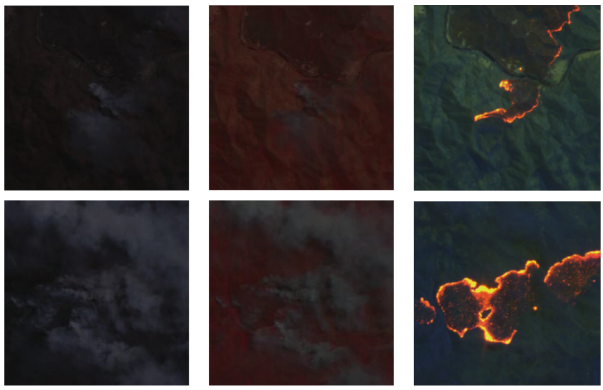
Fig. 1. Geographic distribution of data in WildfireForest-S2.

3 Dataset

The WildfireForest-S2 dataset contains image data from different locations and times during wildfire events in Australia. This dataset is specifically designed for satellite-based wildfire recognition and segmentation tasks.

3.1 Image Collection

The image data for this dataset comes from the Copernicus Data Center of the European Space Agency, covering fire-related images collected from November 7, 2019, to January 28, 2020. The study area includes the southern and eastern regions of New South Wales, the eastern and northeast parts of Victoria, South East Queensland, and Kangaroo Island, among others, as illustrated in Fig. 1.



(a) (b) (c)

Fig. 2. (a) MSI Natural Colors (B4, B3 and B2). (b) MSI False-color Infrared (B8, B4 and B3). (c) MSI Atmospheric Penetration (B12, B11 and B8a).

Sentinel-2 L1C is an atmospheric top-of-atmosphere reflectance product that has undergone orthorectification and geometric correction, but has not been atmospherically corrected. However, the L2A data mainly contains atmospherically corrected bottom-of-atmosphere reflectance data, which users need to produce according to their requirements. To address this, we used the Sen2cor plugin, released by the European Space Agency, to perform atmospheric correction of the acquired data [20,21]. This dataset comprises 13 spectral bands, and we employed band combinations to better interpret the feature characteristics within the images. We explored all available combinations and ultimately selected the three infrared bands with a 20 m resolution that are most suitable for fire monitoring: B12, B11, and B8a, as illustrated in Fig. 2. In the dataset, we segmented the images into small pieces of 256×256, and filter out portions containing and without fire events.

3.2 Image Annotation

We used the Labelme tool to annotate the images and exported the labels in PNG format [22]. After completing the image annotations, we invited experts to review and validate the labels to ensure annotation quality, resulting in the final annotated dataset.

3.3 Dataset Statistics

The WildfireForest-S2 dataset has a total of 1,714 images,of which 857 contain fire scenes. We split these fire images into test sets and training sets in a ratio of 2:8. We also picked 857 images in the area that did not include fire scenes and seperated them in the same proportion.

In the end, the training set has 1,376 images, and the test set has 338 images of which the number of the fire and no-fire pictures is the same.

4 WSRNet Architecture

WSRNet introduces the concept of multi-task learning to enhance the U-Net method and incorporates a wildfire recognition task [23]. First, we propose a specific feature enhancement module to improve the features during the down-sampling process [24]. Next, we propose a guidance module that directs the segmentation task using the classification results, further enhancing the segmentation performance. Additionally, we incorporate an extra cross-entropy loss for the wildfire recognition task. This approach allows the two tasks to mutually benefit from each other, thereby improving the model's performance.

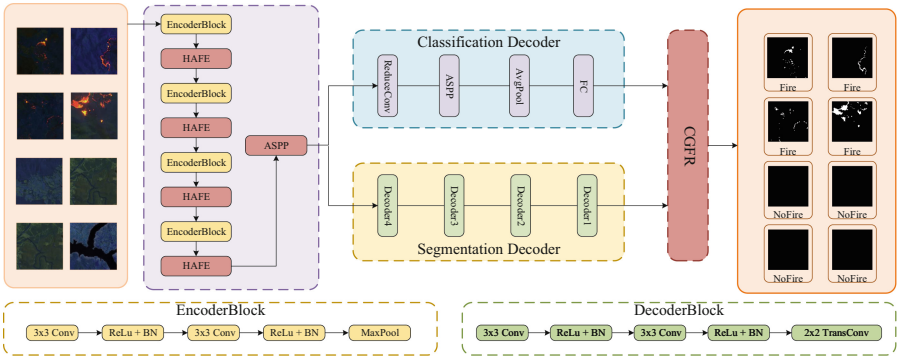


Fig. 3. Overview of proposed WSRNet architecture.

4.1 Technical Overview

As illustrated in Fig. 3, WSRNet adopts a dual-decoder architecture. The backbone employs a shared encoder and incorporates our proposed Hierarchical Attention for Feature Enhancement (HAFE) module at multiple levels to emphasize critical information, thereby improving the quality of feature representation. Under a multi-task learning framework, the model jointly performs segmentation and classification using shared features, which enhances both representational power and generalization capability.

Moreover, we propose a guidance module named Class-Guided Feature Reweighting (CGFR) to fuse classification outputs with segmentation features. This module applies a class-guided feature reweighting mechanism to semantically modulate the segmentation features, resulting in more accurate and discriminative segmentation outcomes.

4.2 HAFE Module

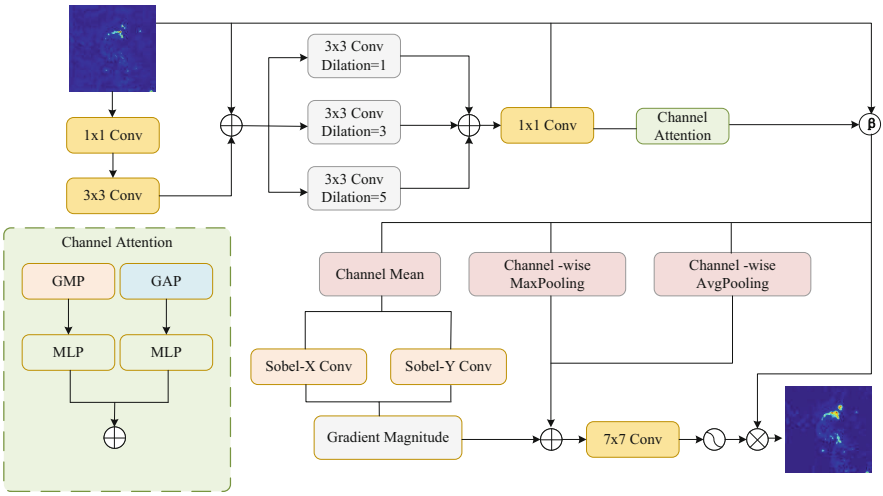


Fig. 4. HAFE module integrates highlight features, dilated convolutions (ASPP), channel attention, and edge-guided spatial attention, and employs learnable gating and scaling to achieve multi-dimensional feature enhancement for fire imagery. It receives the feature map ($x \in \mathbb{R}^{B\times C\times H\times W}$) from the previous encoder as input and outputs an enhanced feature map ($y \in \mathbb{R}^{B\times C\times H\times W}$) to the next encoder, with consistent input and output dimensions for seamless feature consistency and efficient information flow.

To enhance the representation of critical regions in satellite fire imagery, we propose the Hierarchical Attention for Feature Enhancement (HAFE) module, as illustrated in Fig. 4. HAFE integrates multi-scale context extraction, channel-wise reweighting, spatial saliency refinement, and edge-aware modulation in a unified attention framework, effectively enriching semantic features while preserving structural cues.

Given an input feature map $X \in R^{B\times C\times H\times W}$, HAFE performs the following steps:

Step 1: High-Response Highlighting: First, a 1×1 convolution is applied to the input feature map for channel-wise linear transformation, which reduces redundancy and enhances the response of important channels. Then, a 3×3 convolution is used to capture local spatial context and further highlight fire-related features. Both convolution layers are followed by ReLU activation functions to strengthen the non-linear representation capability.

$$F_{high} = ReLU(Conv_{3\times3}(ReLU(Conv_{1\times1}(X)))) \tag{1}$$

$$X_{cat} = Concat(X, F_{high}) \in R^{B\times 2C\times H\times W} \tag{2}$$

Step 2: Multi-scale Context Extraction (Lightweight ASPP): After obtaining the initially enhanced highlight features, we introduce a lightweight

Atrous Spatial Pyramid Pooling (ASPP) module to further strengthen the model's perception of multi-scale fire regions. Specifically, the highlight features are fed into multiple parallel 3×3 dilated convolution branches with different dilation rates to extract contextual information at various receptive fields. Each branch is followed by Batch Normalization and ReLU activation to ensure stable training and richer non-linear representation. The outputs from all branches are then concatenated along the channel dimension and fused via a 1×1 convolution to generate the final multi-scale representation.

$$F_{d_i} = ReLU\left(BN\left(Conv_{3\times3}^{d_i}(X_{cat})\right)\right), \quad d_i \in \{1, 3, 5\} \tag{3}$$

$$F_{ms} = ReLU\left(BN\left(Conv_{1\times1}\left(Concat(F_{d_1}, F_{d_3}, F_{d_5}\right)\right)\right)\right) \tag{4}$$

where $d_i \in 1, 3, 5$ denotes the dilation rate.

Step 3: Channel Attention with Learnable Gating: Global average pooling and max pooling are applied to the input feature map to obtain two distinct channel descriptors. These descriptors are then passed through a shared multilayer perceptron (MLP) for nonlinear transformation. Finally, channel-wise attention weights are generated to emphasize informative channels, improving the model's sensitivity to salient fire regions.

$$M_{ca} = \sigma\left(MLP(AvgPool(F_{ms})) + MLP(MaxPool(F_{ms}))\right) \tag{5}$$

A learnable scalar gate β adaptively fuses attention-enhanced and original features:

$$F_{ca} = \beta \cdot (F_{ms} \odot M_{ca}) + (1 - \beta) \cdot X \tag{6}$$

Step 4: Edge-Aware Spatial Attention: To enhance the model's attention to flame edges and salient regions, a Sobel-based edge extraction module is introduced and integrated with a spatial attention mechanism. First, fixed Sobel filters extract horizontal and vertical gradients to model edge structures. Then, the average-pooled map, max-pooled map, and edge map are concatenated along the channel dimension and passed through a 7×7 convolution to generate a spatial attention map. Let E denote the edge map computed via Sobel filters:

$$E = \sqrt{(Sobel_x(\bar{F}_{ca}))^2 + (Sobel_y(\bar{F}_{ca}))^2 + \epsilon} \tag{7}$$

where $\bar{F}ca$ is the mean across channels of Fca, and ϵ is a small constant. Spatial attention map:

$$M_{sa} = \sigma\left(Conv_{7\times7}\left(Concat(Avg(F_{ca}), Max(F_{ca}), E)\right)\right) \tag{8}$$

Step 5: Final Output with Learnable Scaling: Finally, the spatial attention map F_{ca} is element-wise multiplied with the channel-refined feature map M_{sa},

enabling joint modeling of spatial and channel-wise saliency. A learnable scaling factor α is applied to adaptively modulate the final attention-enhanced output.

$$F_{out} = \alpha \cdot (F_{ca} \odot M_{sa}) \tag{9}$$

By hierarchically integrating semantic, contextual, and structural cues, HAFE significantly enhances the discriminability of fire-related features while maintaining spatial precision.

4.3 CGFR Module

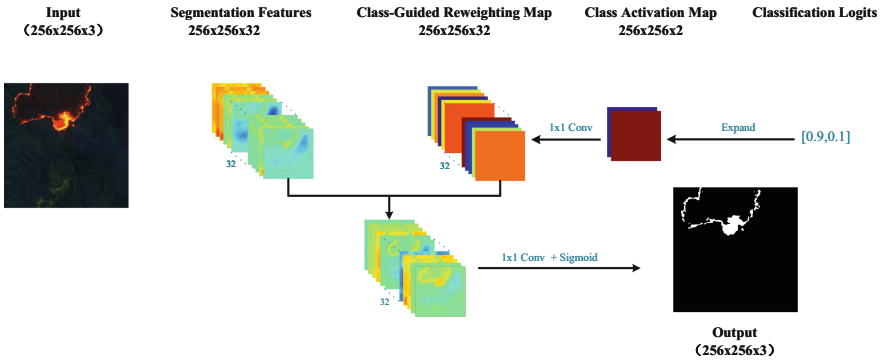


Fig. 5. CGFR module enhances segmentation features using classification predictions. The logits are expanded into spatial masks, projected via a 1×1 convolution, and fused with segmentation features through element-wise multiplication for class-aware modulation.

The Class-Guided Feature Reweighting (CGFR) module is designed to leverage the classification branch to semantically guide the segmentation process, as illustrated in Fig. 5. This process is formally defined as:

Step 1: Generation of Class Probabilities: The raw logits from the classification branch are passed through a sigmoid activation to produce class-wise confidence scores in the range $[0, 1]$:

$$M_{cls} = \sigma(P_{cls}) \in R^{B \times C} \tag{10}$$

where $\sigma(\cdot)$ is the sigmoid function, where B is the batch size and C is the number of semantic categories. The resulting vector indicates the likelihood of each class being present in the input image, providing high-level semantic cues

Step 2: Construction of the Class-Guided Mask: To adapt the classification cues for spatially distributed segmentation, the confidence vector M_{cls} is expanded along spatial dimensions to match the size of the segmentation feature

map $F_{seg} \in R^{B\times C_{seg}\times H\times W}$. A 1×1 convolution is then applied to project the expanded class mask into the segmentation feature channel space:

$$M_{cls}^{exp} = \text{Conv1x1}(Expand(M_{cls})) \in R^{B\times C_{seg}\times H\times W} \tag{11}$$

where Expand$(\cdot)$ denotes spatial broadcasting and C_{seg} is the number of channels in the segmentation feature. This expansion acts as a coarse spatial attention mechanism, aligning global category-level activations with the local receptive fields of segmentation.

Step 3:Semantic Feature Reweighting: Finally, the class-aware mask M_{cls}^{exp} is element-wise multiplied with the original segmentation feature map:

$$F_{out} = F_{seg} \otimes M_{cls}^{exp} \tag{12}$$

where $\otimes$ denotes element-wise multiplication, and $\mathbf{F}_{out}$ is the reweighted segmentation feature. This reweighting process suppresses features unrelated to predicted classes and enhances the activation of semantically relevant regions, improving the model's focus and robustness, especially in ambiguous or noisy scenes.

This mechanism effectively injects high-level semantic cues into the segmentation branch, enabling the model to better focus on category-relevant regions.

4.4 Classification Decoder

The design of this image classifier consists of two main modules: the ASPP module (Atrous Spatial Pyramid Pooling) and the Recognition Decoder. These two modules together form an efficient framework for multi-scale context feature extraction and classification. An overview of the network architecture is presented in Fig. 6, which illustrates the flow of data through the key components of the model.

Traditional convolutional layers acquire image information through local receptive fields. However, when processing large objects or distant contextual information, the receptive field may be insufficient. The ASPP module addresses this limitation by utilizing dilated convolutions with varying dilation rates to extract contextual features from different scales. This approach effectively expands the receptive field, enabling the model to capture richer global information without significantly increasing computational cost. By extracting both local features and broader contextual information, the ASPP module enhances the model's discriminative power.

The primary function of the Recognition Decoder is to process the multi-scale contextual features extracted by the ASPP module and ultimately produce the classification output.

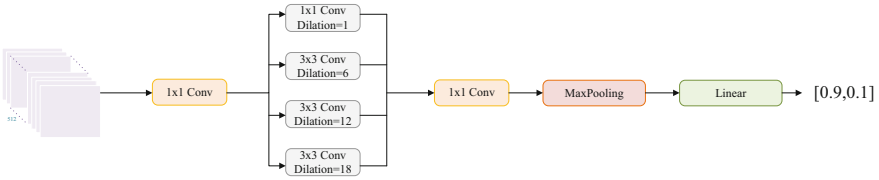


Fig. 6. Overview of the Image Classification Architecture. The input features first pass through a 1x1 convolution to reduce the number of channels to 256. Then, four parallel dilated convolution layers with different dilation rates (1, 6, 12, and 18) are applied to capture multi-scale contextual information. The outputs of these layers are concatenated and passed through another 1x1 convolution for feature fusion. Afterward, global average pooling is applied to reduce the spatial dimensions, and the features are flattened into a 1D vector. Finally, a linear layer outputs a classification probability in the range of [0, 1].

To provide a clearer understanding of the internal mechanisms of this classification decoder, the main implementation steps are detailed as follows:
Step 1:Channel Reduction Preprocessing: A 1×1 convolution is applied to reduce the number of channels (256), followed by batch normalization and ReLU activation:

$$F_{reduced} = ReLU(BN(Conv1 \times 1(F_{seg}))) \tag{13}$$

where $F_{seg} \in \mathbb{R}^{B\times C_{sig}\times H\times W}$ is the input feature map with 512 channels,and $F_{reduced} \in \mathbb{R}^{B\times C_{\text{out}}\times H\times W}$is the output feature map after channel reduction, typically with $C_{out} = 256$.
Step 2: Multi-scale Context Extraction: The reduced feature map $F_{reduced}$ is fed into the Atrous Spatial Pyramid Pooling (ASPP) module, where four parallel convolutions extract context at different scales:

$$F_1 = ReLU(BN(Conv_{1\times 1}(F_{\text{reduced}}))) \tag{14}$$
$$F_2 = ReLU(BN(Conv_{3\times 3,r=6}(F_{\text{reduced}}))) \tag{15}$$
$$F_3 = ReLU(BN(Conv_{3\times 3,r=12}(F_{\text{reduced}}))) \tag{16}$$
$$F_4 = ReLU(BN(Conv_{3\times 3,r=18}(F_{\text{reduced}}))) \tag{17}$$

These outputs are concatenated and projected back to 256 channels:

$$F_{\text{ASPP}} = ReLU(BN(Conv_{1\times 1}(Concat[F_1, F_2, F_3, F_4]))) \tag{18}$$

where $F_{\text{ASPP}} \in \mathbb{R}^{B\times C_{\text{ASPP}}\times H\times W}$ and $C_{\text{ASPP}} = 256$.
Step 3:Global Feature Aggregation: The fused multiscale feature F_{ASPP} undergoes global average pooling, reducing its spatial dimensions to 1×1, resulting in a single feature vector for each sample. This operation aggregates global spatial context and produces a compact representation used for classification.

$$F_{\text{gap}} = GAP(F_{\text{ASPP}}) \in \mathbb{R}^{B\times 256\times 1\times 1} \tag{19}$$

Step 4: Classification Head and Prediction: Finally, the global feature F_{gap} is flattened and passed through a fully connected (FC) layer to produce the prediction score for each class.

$$P_{\text{cls}} = FC(Flatten(F_{\text{gap}})) \in \mathbb{R}^{B \times N_{\text{cls}}} \tag{20}$$

where P_{cls} is the classification score for each class.

The proposed recognition decoder leverages channel reduction and an ASPP module to effectively integrate multi-scale contextual features, thereby enhancing the model's ability to detect fire occurrences of varying sizes in satellite imagery. The use of global average pooling and a classification head further enables efficient aggregation of discriminative features, leading to robust and accurate fire detection in complex remote sensing scenarios.

4.5 Training and Inference

Training Stage. We want to optimize the loss functions for both tasks at the same time so that the model can learn features of both classification and segmentation together. Since the model handles two tasks, we have created separate loss functions for each one.

For the classification task, we use the Binary Cross-Entropy (BCE) loss to measure the difference between the model's predicted probability and the ground truth labels. The BCE loss function is defined as follows:

$$\mathcal{L}_{class} = \frac{1}{N} \sum_{i=1}^{N} \left[-y_i^{class} \cdot \log(\hat{y}_i^{class}) - (1 - y_i^{class}) \cdot \log(1 - \hat{y}_i^{class}) \right] \tag{21}$$

where y_i^{class} represents the true classification label of the i-th sample, $\hat{y}_i^{class}$ denotes the classification prediction result of the model, and N is the total number of training samples.

For the segmentation task, we use the Cross-Entropy Loss to measure the difference between the model's predicted segmentation results and the ground truth segmentation masks. The Cross-Entropy Loss function is defined as follows:

$$\mathcal{L}_{seg} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{c=1}^{C} y_{i,c}^{seg} \, log(\hat{y}_{i,c}^{seg}) \tag{22}$$

where $y_{i,c}^{seg}$represents the ground truth label of the i-th sample for the c-th class,$\hat{y}_{i,c}^{seg}$ denotes the predicted probability of the c-th class by the model, and C is the total number of classes.

To jointly optimize the classification and segmentation tasks, we weight the loss function of the classification task and then sum it with the segmentation task's loss to obtain the final multi-task loss function:

$$\mathcal{L}_{total} = \mathcal{L}_{seg} + \lambda \cdot \mathcal{L}_{class} \tag{23}$$

where λ represents the weight of the classification task.

Inference Stage. We input the organized test set data into the model, and the corresponding classification and segmentation results to obtain.

5 Experiments

Dataset. We use the WildfireForest-S2 dataset to evaluate our model. All of which contain 1,714 images in total. These are split into 1,376 training images and 338 test images. In both the training and test sets, half of the images show fire events, and the other half do not. This keeps the dataset balanced for fair evaluation.

Evaluation Metrics. In order to evaluate the performance of our model, we have selected the following core indicators.

For the segmentation task, we used mean Intersection over Union (mIoU) that gives a clear idea of how accurate the segmentation is [25]. To check how good the model is at finding target areas, we added Recall (Rec) and the F1-score [26]. These help measure the model's ability to detect what matters. We also included the Kappa coefficient to make sure the evaluation is fair, especially when some classes are much rarer than others. These metrics together give a full picture of the performance of the model.

For the classification task, we added the Area Under the Curve (AUC) to see how well the model can tell different categories apart [27]. We also used other common metrics like accuracy, sensitivity, specificity, F1-score, and Matthews Correlation Coefficient (MCC) [28]. These help us understand how precise, balanced, and reliable the model's predictions are.

Experimental Setup. To ensure the fairness and reproducibility of our experiments, we describe the detailed experimental setup and parameter choices. Table 1 shows the hardware configuration and key parameters used in the experiment.

Table 1. Hardware Configuration and Key Parameters Used in the Experiment

Parameter	Value
CPU	Intel(R) Core(TM) i7-10700K CPU @ 3.80GHz
GPU	NVIDIA GeForce RTX 3080
Image Size	256×256
Batch Size	8
Optimizer	Adam
Training Epochs	100 epochs
PyTorch Version	1.13.0
CUDA Version	11.7

5.1 Performance Comparison on Segmentation Task

To verify the effectiveness of the proposed method, we compared our model with U-Net, U-Net++, DeepLabv3+, ABCNet, SegFormer, and TransUNet in terms

of performance [29–32]. By evaluating these models using selected metrics such as mIoU, we assessed their effectiveness.

Results on the Dataset. Table 2 shows the comparative results on the current dataset.

Table 2. The performance comparison between our method and the baseline methods on the dataset, with the best results highlighted in bold.

Method	FLOP	mIoU	Rec	F1	Kappa
U-Net	24.09G	89.84%	45.74%	37.68%	87.57%
U-Net++	200.12G	89.90%	46.14%	37.87%	87.61%
DeepLabv3+	4.28G	85.41%	28.10%	26.60%	75.49%
ABCNet	83.13G	90.04%	46.39%	38.04%	87.90%
SegFormer	3.27G	87.47%	36.34%	31.86%	79.86%
TransUNet	32.23G	89.10%	39.15%	34.86%	82.87%
WSRNet	48.47G	**91.98%**	**48.56%**	**40.08%**	**89.22%**

SegFormer and DeepLabv3+ require fewer computational resources, but exhibit relatively poorer performance. Compared with U-Net++ and ABCNet, WSRNet requires fewer computing resources and demonstrates better performance. Among all the methods mentioned, WSRNet stands out with the best performance, not only surpassing other methods in segmentation accuracy but also excelling in other metrics. This enables WSRNet to have high stability and reliability when handling wildfire segmentation tasks.

5.2 Visual Results

To comprehensively assess the performance and interpretability of the proposed method, we conduct both quantitative comparisons with baseline models and detailed feature visualization analyses. By presenting visual comparisons of segmentation results and the internal feature evolution process within the HAFE module, we aim to validate the superiority of WSRNet as well as the practical effectiveness of the HAFE module for wildfire detection in satellite remote sensing scenarios. These analyses not only demonstrate the accuracy of our approach in wildfire area identification but also elucidate the mechanisms by which the model enhances key features and suppresses redundant information.

Visual Comparison of WSRNet and Baseline Methods: To further investigate the practical effectiveness of WSRNet in the task of segmenting wildfires in satellite images, we selected four sample regions from the set of tests and presented a comparison of WSRNet with six baseline methods in Fig. 7.

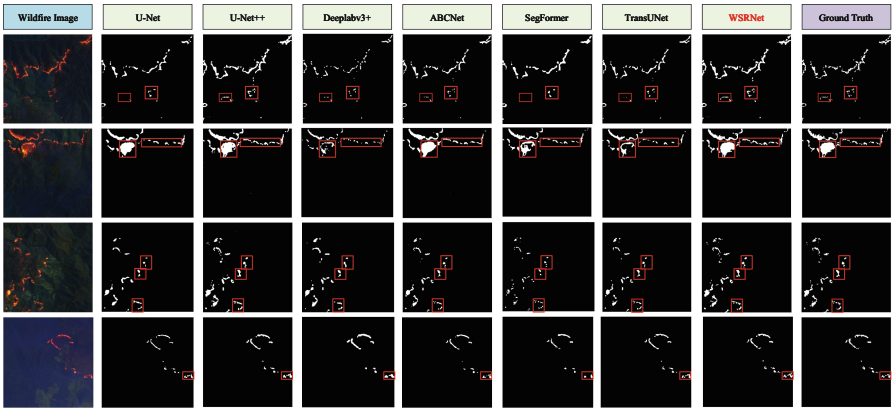


Fig. 7. Partial image visualization results of different methods on the WildfireForest-S2 dataset. White pixels represent the burning wildfire areas and red boxes represent error-prone regions. (Color figure online)

In Fig. 7, we can observe that WSRNet performs exceptionally well in the wildfire segmentation task, with its predicted wildfire regions closely matching the ground truth and accurately covering the core areas of the wildfires. In particular in regions with higher complexity, WSRNet shows remarkable detail capture capabilities, accurately identifying subtle features of wildfires. In contrast, while other baseline models can also detect wildfire regions, there are noticeable devi-

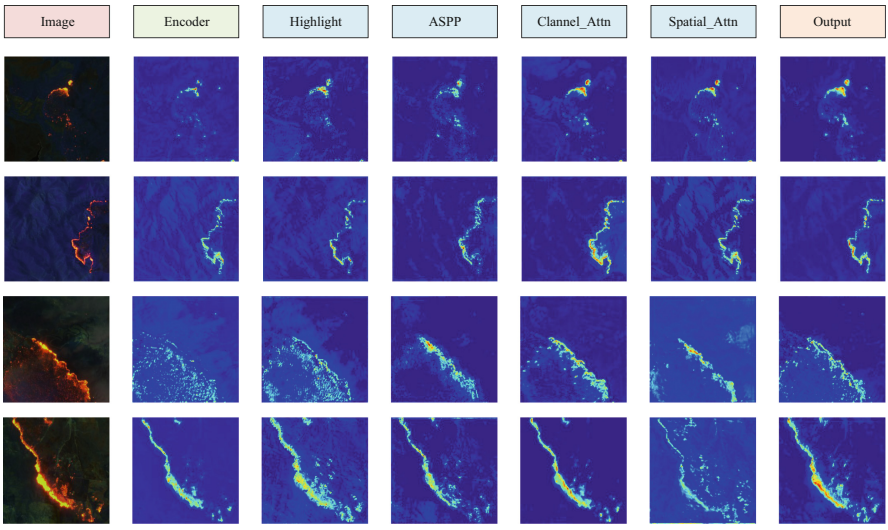


Fig. 8. Feature enhancement visualization displays the feature evolution process of the HAFE module on the WildfireForest-S2 dataset. The intensity of the heatmap colors (blue: low importance; red: high importance) is positively correlated with feature importance. (Color figure online)

ations from the ground truth in terms of the number of fire points and spatial localization accuracy, especially in edge regions and detailed predictions. This result indicates that WSRNet exhibits higher robustness and accuracy in wildfire detection tasks.

Visualization of HAFE Module Feature Evolution: To validate the effectiveness of the HAFE module, we conducted visual analysis on its intermediate features. Figure 8 illustrates the feature processing pipeline of the HAFE module on satellite wildfire images. From left to right, the visualizations correspond to the input, encoder output, highlight enhancement branch, multi-scale ASPP output, channel attention, and spatial attention results, culminating in the final feature output. Comparing the encoder output with the output after the HAFE module, it is evident that as the features are progressively processed by each component, the discriminative representation of fire spots and their surroundings is significantly enhanced, while background and noise regions are effectively suppressed. In particular, the highlight and channel attention stages progressively focus and emphasize the important regions, and the subsequent attention mechanisms further improve the response to critical wildfire cues. The above visual feature evolution demonstrates the effectiveness of the proposed module in extracting fire signals and suppressing redundant information in complex remote sensing scenarios.

5.3 Ablation Study

To validate the effectiveness of each module in WSRNet, we conducted ablation experiments by removing key modules and evaluated the performance on both classification and segmentation tasks.

- **WSRNet** *w/o* **HAFE:** We removed the HAFE module, directly passing the output of the previous layer to the next layer in the encoder.
- **WSRNet** *w/o* **Classifier:** We removed the Classifier Encoder and eliminated the classification loss from the loss function.
- **WSRNet** *w/o* **CGFR:** We removed the CGFR module and directly decode the raw segmentation features.

Table 3. The performance comparison of WSRNet and its variants on the dataset. Best results are in bold.

Method	mIoU	Rec	F1	Kappa	AUC	ACC	TPR	TNR	MCC
WSRNet *w/o* HAFE	90.05%	46.00%	35.30%	86.91%	98.65%	97.87%	97.60%	98.48%	97.06%
WSRNet *w/o* Classifier	89.97%	45.78%	34.71%	86.42%	-	-	-	-	-
WSRNet *w/o* CGFR	90.48%	46.56%	36.91%	87.50%	99.79%	98.53%	98.80%	98.76%	98.46%
WSRNet	**91.98%**	**48.56%**	**40.08%**	**89.22%**	**99.94%**	**99.53%**	**99.40%**	**99.41%**	**98.81%**

Table 3 shows the comparison results of the two tasks after removing different modules, which aligns with our expectations. The results indicate that our proposed HAFE module performs effectively in both segmentation and classification tasks, playing a crucial role in improving overall model accuracy. Moreover, the guidance provided by the CGFR module also effectively improves the segmentation accuracy. It is evident that the multi-task learning approach improves the accuracy of both tasks simultaneously through joint loss optimization and feature-guided mechanisms. When all components are included, WSRNet achieves its optimal performance.

6 Conclusion

We constructed a wildfire dataset covering Australia called WildfireForest-S2 and we proposes a new method called WSRNet, which based on multi-task learning for wildfire identification and segmentation tasks in satellite images. WSRNet achieves high-precision wildfire detection and accurate fire area segmentation by sharing feature extraction networks and task-specific decoders, while optimizing identitication tasks and segmentation tasks. Experimental results show that WSRNet is supper to existing methods in indicators such as mIoU and F1-score for the segmentation task and achieves exceptionally high accuracy in the classification task. In the future, we plan to expand the geographical coverage of the dataset and optimize the computing efficiency of the model support real-time use.

Acknowledgments. This work was supported in part by the National Natural Science Foundation of China under Grant 62372163. The science and technology innovation Program of Hunan Province, 2024RC1033.

References

1. Pausas, J.G., Keeley, J.E.: Wildfires and global change. Front. Ecol. Environ. **19**(7), 387–395 (2021)
2. Phiri, D., Simwanda, M., Salekin, S., Nyirenda, V.R., Murayama, Y., Ranagalage, M.: Sentinel-2 data for land cover/use mapping: a review. Remote Sens. **12**(14), 2291 (2020)
3. Shamsoshoara, A., Afghah, F., Razi, A., Zheng, Li., Fulé, P.Z., Blasch, E.: Aerial imagery pile burn detection using deep learning: the flame dataset. Comput. Netw. **193**, 108001 (2021)
4. Hopkins, B., et al.: Flame 2: fire detection and modeling: aerial multi-spectral image dataset. IEEE DataPort (2023)
5. Hopkins, B., et al.: Flame 3 dataset: unleashing the power of radiometric thermal UAV imagery for wildfire management. arXiv preprint arXiv:2412.02831 (2024)
6. El-Madafri, I., Peña, M., Olmedo-Torre, N.: The wildfire dataset: enhancing deep learning-based forest fire detection with a diverse evolving open-source dataset focused on data representativeness and a novel multi-task learning approach. Forests **14**(9), 1697 (2023)

7. Kondylatos, S., Prapas, I., Camps-Valls, G., Papoutsis, I.: MESOGEOS: a multi-purpose dataset for data-driven wildfire modeling in the Mediterranean. In: Advances in Neural Information Processing Systems, vol. 36 (2024)
8. Dunnings, A.J., Breckon, T.P.: Experimentally defined convolutional neural network architecture variants for non-temporal real-time fire detection. In: 2018 25th IEEE International Conference on Image Processing (ICIP), pp. 1558–1562. IEEE (2018)
9. Shees, A., Ansari, M.S., Varshney, A., Asghar, M.N. and Kanwal, N.: FireNet-V2: improved lightweight fire detection model for real-time IoT applications. Procedia Comput. Sci. **218**, 2233–2242 (2023)
10. Kaufman, Y.J., et al.: Potential global fire monitoring from EOS-MODIS. J. Geophys. Res. Atmospheres **103**(D24), 32215–32238 (1998)
11. Yuan, C., Zhang, Y., Liu, Z.: A survey on technologies for automatic forest fire monitoring, detection, and fighting using unmanned aerial vehicles and remote sensing techniques. Can. J. For. Res. **45**(7), 783–792 (2015)
12. Xingfeng, C., Li, L., Jiaguo, L., Wenhao, O.U., Zhang, Y., et al.: Application and research progress of fire monitoring using satellite remote sensing. Nat. Remote Sens. Bull. **24**(5), 531–542 (2020)
13. De la Fuente, R., Aguayo, M.M., Contreras-Bolton, C.: An optimization-based approach for an integrated forest fire monitoring system with multiple technologies and surveillance drones. Eur. J. Oper. Res. **313**(2), 435–451 (2024)
14. Liu, Y., Zheng, C., Liu, X., Tian, Y., Zhang, J., Cui, W.: Forest fire monitoring method based on UAV visual and infrared image fusion. Remote Sens. **15**(12), 3173 (2023)
15. Chuvieco, E., Aguado, I., Salas, J., García, M., Yebra, M., Oliva, P.: Satellite remote sensing contributions to wildland fire science and management. Curr. Forestry Rep. **6**, 81–96 (2020)
16. Park, B.-S., Cho, I.-J., Lim, J.-H., Kim, I.-B.: Forest fire monitoring system using satellite. J. Convergence Inf. Technol. **11**(11), 143–150 (2021)
17. Chen, J., Zheng, W., Shuang, W., Liu, C., Yan, H.: Fire monitoring algorithm and its application on the GEO-KOMPSAT-2A geostationary meteorological satellite. Remote Sens. **14**(11), 2655 (2022)
18. Gibson, R.K., White, L.A., Hislop, S., Nolan, R.H., Dorrough, J.: The post-fire stability index; a new approach to monitoring post-fire recovery by satellite imagery. Remote Sens. Environ. **280**, 113151 (2022)
19. Haizhou, X., Zhang, G., Zhou, Z., Zhou, X., Zhou, C.: Forest fire monitoring and positioning improvement at subpixel level: application to HIMAWARI-8 fire products. Remote Sens. **14**(10), 2460 (2022)
20. Main-Knorn, M., Pflug, B., Louis, J., Debaecker, V., Müller-Wilm, U., Gascon, F.: Sen2cor for sentinel-2. In: Image and signal processing for remote sensing XXIII. SPIE, vol. 10427, pp. 37–48 (2017)
21. Louis, J., et al.: Sentinel-2 global surface reflectance level-2a product generated with sen2cor. In: IGARSS 2019-2019 IEEE International Geoscience and Remote Sensing Symposium, pp. 8522–8525. IEEE (2019)
22. Russell, B.C., Torralba, A., Murphy, K.P., et al.: LABELME: a database and web-based tool for image annotation. Int. J. Comput. Vis., **77**, 157–173 (2008). https://doi.org/10.1007/s11263-007-0090-8
23. Zhang, Yu., Yang, Q.: An overview of multi-task learning. Natl. Sci. Rev. **5**(1), 30–43 (2018)

24. Ronneberger, O., Fischer, P., Brox, T.: U-Net: convolutional networks for biomedical image segmentation. In: Medical image computing and computer-assisted intervention–MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18, pp. 234–241. Springer, (2015). https://doi.org/10.1007/978-3-319-24574-4_28
25. Zhang, H., Du, J., Xie, C., Zhao, F.: Merged intersection over union: a metric and a loss for aerial images in transmission lines. In: 2023 5th International Conference on Intelligent Control, Measurement and Signal Processing (ICMSP). IEEE, pp. 870–875 (2023)
26. Samardzic, N., et al.: F1: a fast and programmable accelerator for fully homomorphic encryption. In: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture, pp. 238–252 (2021)
27. Huang, J., Chan, J.D., Nguyen, T., Jain, R., Escobar, Z.K.: Doing more with less: pragmatic implementation of vancomycin area-under-the-curve (AUC) monitoring. J. Pharmacy Pract. **36**(1), 10–14 (2023)
28. Chicco, D., Starovoitov, V., Jurman, G.: The benefits of the Matthews correlation coefficient (MCC) over the diagnostic odds ratio (DOR) in binary classification assessment. IEEE Access **9**, 47112–47124 (2021)
29. Zhou, Z., Rahman Siddiquee, M.M., Tajbakhsh, N., Liang, J.:UNet++: a nested u-net architecture for medical image segmentation. In: Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support: 4th International Workshop, DLMIA 2018, and 8th International Workshop, ML-CDS 2018, Held in Conjunction with MICCAI 2018, Granada, Spain, September 20, 2018, Proceedings 4, pp. 3–11. Springer (2018). https://doi.org/10.1007/978-3-030-00889-5_1
30. Mikuni, V., Canelli, F.: ABCNET: an attention-based method for particle tagging. Eur. Phys. J. Plus **135**, 1–11 (2020)
31. Enze Xie, Wenhai Wang, Zhiding Yu, Anima Anandkumar, Jose M Alvarez, and Ping Luo, "Segformer: Simple and efficient design for semantic segmentation with transformers," Advances in neural information processing systems, vol. 34, pp. 12077–12090, 2021
32. Chen, J., et al.: TRANSUNET: transformers make strong encoders for medical image segmentation. arXiv preprint arXiv:2102.04306 (2021)

SCoT Guard: A Safety Chain of Thought Guardrail Model

Yuxuan Lin[1,2], Shi Liu[1,2], Dongqin Liu[2], Wei Mi[2], and Xuehai Tang[2(✉)]

[1] School of Cyberspace Security, University of Chinese Academy of Sciences, Beijing, China
[2] Institute of Information Engineering, Chinese Academy of Sciences, Beijing, China
{linyuxuan,liushi,liudongqin,miwei,tangxuehai}@iie.ac.cn

Abstract. The proliferation of large language models (LLMs) in intelligent applications has introduced critical safety challenges concerning input and output content moderation. Current guardrail solutions suffer from inadequate performance in small-parameter models while excessive resource overhead in large-parameter models, limited capacity for fine-grained rule customization, and improvable out-of-distribution generalization. To overcome these limitations, we introduce SCoT Guard, a novel reasoning-based guardrail framework that dynamically acquires fine-grained judgment criteria through contextual learning from examples. Our approach leverages a meticulously curated training dataset of 40,000 high-quality instances generated using DeepSeek-R1, incorporating comprehensive safety detection reasoning chains and rule-learning mechanisms. We implement LoRA fine-tuning on the Qwen2.5-1.5B-Instruct model to achieve optimal performance. At inference time, SCoT Guard utilizes Retrieval-Augmented Generation (RAG) to identify similar examples, extracts their underlying judgment principles, and applies these learned rules to evaluate the new sample, thereby accommodating evolving safety requirements. Comprehensive experimental evaluation demonstrates that SCoT Guard substantially surpasses 9 baseline methods across 6 benchmarks (e.g., OpenAI Moderation, ToxicChat, BeaverTails). These results confirm the effectiveness of integrating chain-of-thought reasoning with RAG in enhancing the performance, interpretability, and adaptability of guardrail models. With its compact 1.5B parameter architecture, SCoT Guard enables direct deployment on resource-limited IoT edge devices, facilitating real-time local inference while eliminating cloud dependency and ensuring data privacy. Both the training dataset and model weights will be open-sourced.

Keywords: Large Language Models · Safety Guardrails · Chain-of-Thought · Retrieval-Augmented Generation · Edge AI · Fine-tuning

1 Introduction

With the rapid iteration of artificial intelligence technology, large language models (LLMs) [1–10] have achieved revolutionary breakthroughs in diverse scenarios

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 349–363, 2026.
https://doi.org/10.1007/978-981-95-8405-5_19

such as chatbots, search engines, recommendation systems, and agent interactions, owing to their exceptional natural language processing capabilities. As the core driving engine of next-generation intelligent systems, while unleashing productivity, LLMs also expose significant safety risks [11–16]: privacy leakage from training data, harmful content generation triggered by malicious instructions, and automation support for illegal activities have become key bottlenecks constraining their large-scale application. Therefore, reliable safety guardrails are needed to automatically detect safety risks in the inputs and outputs of large language models, thereby enhancing the safety and reliability of various intelligent systems.

Existing LLM-based safety guardrails primarily adopt two approaches. The first relies on powerful general-purpose LLMs [1–10], utilizing prompt engineering to leverage the models' excellent generalization capabilities and instruction-following abilities for text safety detection. The second involves collecting content moderation data across various categories, constructing supervised fine-tuning datasets to post-train LLMs into specialized guardrail models, such as LLaMA Guard [3,17], ShieldGemma [18], Aegis Guard [19,20] and ShieldLM [21]. However, existing LLM-based safety guardrails still face the following limitations: (1) Existing guardrail models have large parameter sizes (typically $\geq$ 7B) and incur substantial resource overhead, making them difficult to deploy on resource-constrained edge devices (e.g., IoT terminals), whereas small-parameter models often suffer from inadequate detection performance. Dependence on cloud services not only introduces network latency and privacy risks but also increases system operational costs. IoT scenarios urgently require lightweight, low-power and high-performance localized safety detection solutions; (2) They typically rely on zero-shot descriptions of detection rules to support safety standards for specific scenarios. In practical applications, however, safety standards often change dynamically depending on specific contexts, making it difficult to debug the model to formulate effective and universally applicable detection rules. (3) Their detection performance (F1 score) on out-of-distribution datasets is still improvable.

To address these challenges, we propose SCoT Guard, a small reasoning guardrail model that supports analyzing and learning safety judgment rules from contextual examples, enabling more fine-grained and customized safety detection. In real-world detection scenarios, the criteria and thresholds for harmfulness often vary significantly across different content instances. Simply adding batch zero-shot descriptions in prompts struggles to achieve fine-grained, instance-specific customized detection. Our model addresses this by analyzing and learning safety judgment rules from few-shot examples during the reasoning process.

We utilize the current SOTA reasoning LLM, DeepSeek-R1 [6], to synthesize chain-of-thought (CoT) data for text safety detection scenarios. This includes synthesizing CoT data specifically designed for learning safety judgment rules from contextual examples. This constructs a training dataset with explicit reasoning processes, enabling the model to train the capability to reference similar examples during analysis. During inference detection, the Retrieval-Augmented

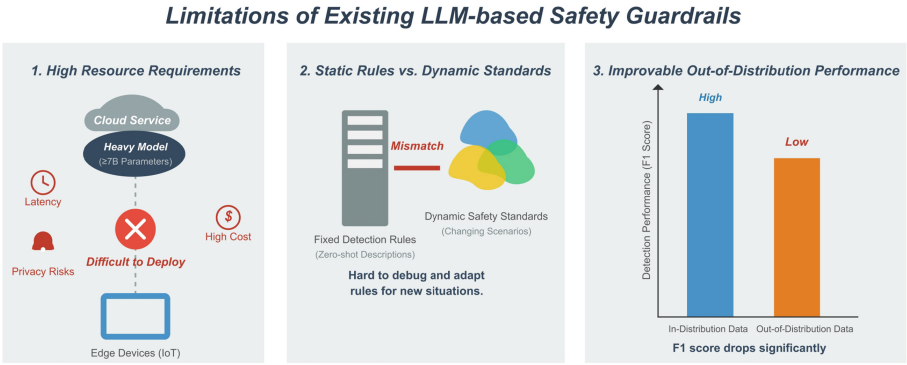


Fig. 1. Limitations of Existing LLM-based Safety Guardrails.

Generation (RAG) component retrieves the top-n most similar examples from an exemplar knowledge base. When analyzing a new sample, SCoT Guard aligns its judgment criteria with these similar examples, achieving customized detection tailored to each specific input and supporting more flexible dynamic safety standards. Trained with only 40k high-quality examples via LoRA fine-tuning based on Qwen2.5-1.5B-Instruct, SCoT Guard achieves excellent performance on multiple safety benchmark datasets at minimal training and inference costs. Its F1 score surpasses current mainstream LLM guardrail models (e.g., LLaMA Guard, ShieldGemma, ShieldLM). We will open-source our high-quality training dataset and model weights to support future research and the advancement of LLM safety.

2 Related Work

LLM Safety. Large language models have demonstrated exceptional performance across various domains, yet their potential safety risks are increasing significantly and cannot be overlooked. To achieve safety alignment for LLMs [22], researchers employ strategies such as filtering toxic training data [23] and implementing post-training techniques including SFT [24], RLHF [25], and DPO [26]. These methods aim to make model outputs more compliant with ethical and legal standards. However, such alignment training inevitably leads to model performance degradation. To address this trade-off, external guardrail models play a crucial role in balancing model performance and safety. Moreover, external guardrail models better support flexible and continuously evolving safety standards.

Guardrail Model. LLM-based guardrail models primarily adopt two technical approaches. The first leverages powerful general-purpose LLMs (e.g., GPT [1], DeepSeek [5,6], Qwen [7,8,10]) with strong generalization and instruction-following capabilities, utilizing carefully designed prompt engineering to function as text safety detectors. The second involves constructing fine-tuning datasets

to train specialized guardrail models for text safety detection. Notable open-source implementations include the following: LLaMA Guard [3,17] and Aegis Guard [19,20] analyze samples based on predefined safety guidelines, outputting the unsafe category to which each unsafe sample belongs; ShieldGemma [18], which enhances robustness through extensive adversarial dataset synthesis; and ShieldLM [21], which supports customizable zero-shot judgment rules while providing explanations for decisions.

Reasoning LLM. Recent research has revealed that reasoning-enhanced LLMs exhibit superior performance in domains such as mathematics and coding, where extended chains-of-thought (CoT) significantly improve out-of-domain generalization. Initially, researchers guided LLMs to think step-by-step via prompt engineering [27,28]. Subsequently, researchers explored different modes of thinking such as self-criticism, reflection, and planning [29–34], which further improved the model's performance. Contemporary reasoning LLMs–represented by models like OpenAI-o1 [35], DeepSeek-R1 [6], QwQ [9], and Kimi k1.5 [36]–internalize reasoning capabilities, enabling spontaneous deliberation, reflection, and problem-solving attempts before generating answers. These models achieve remarkable results across diverse fields, motivating our exploration of incorporating reasoning mechanisms into small-parameter LLMs to enhance their performance in text safety detection.

3 Methodology

This section outlines the methodology of SCoT Guard, a reasoning model that dynamically learns safety judgment rules from contextual examples during inference and applies them to the current sample. Leveraging DeepSeek-R1, we synthesize high-quality Chain-of-Thought (CoT) data for text safety detection, achieving exceptional safety detection performance on small models with minimal training and inference costs.

3.1 Problem Definition

SCoT Guard formalizes text safety detection as a dynamically constrained decision problem. Safety standards for harmful content categories (e.g., hate speech, illegal activities) vary across samples and dynamically evolve with temporal shifts, regional differences, and cultural changes. Moreover, identical content may receive different safety labels under varying standards–observed both in open-source datasets (contradictory annotations across datasets) and real-world applications (differences in safety criteria across countries and cultures). For instance, satirical religious cartoons may be protected expression in secular countries like France but deemed blasphemous in certain Islamic communities. This inherent variability motivates our dynamic constraint approach. When processing a sam-

ple (user input P_j, LLM response R_j), the model satisfies:

$$\begin{pmatrix} T_{\text{GT}} \\ L_{\text{GT}} \end{pmatrix} \approx \begin{pmatrix} T_j \\ L_j \end{pmatrix} = \mathcal{G}\Bigg(S,\ P_j,\ R_j,\ \underbrace{\underset{\substack{E_i \in \mathcal{E} \\ \phi_{\text{sim}}((P_j,R_j),E_i) > \tau}}{\arg\,\text{top}_n} \phi_{\text{sim}}\Big((P_j, R_j),\ E_i\Big)}_{E^*_{j,n}} \Bigg) \quad (1)$$

The model's output explicitly depends on similar examples $E^*_{j,n}$ (top-n examples exceeding similarity threshold τ), approximating sample-specific decision boundaries. When no similar examples exist, it relies on safety knowledge accumulated during training. Key components:

- P_j: User input
- R_j: LLM-generated response paired with P_j
- S: System prompt
- $\mathcal{E}$: Example library containing $\{P_i, R_i, L_i\}$ tuples
- ϕ_{sim}: Similarity metric
- τ: Similarity threshold
- $E^*_{j,n}$: Top-n similar examples
- $\mathcal{G}$: Guardrail model
- T_j: Generated safety CoT for (P_j, R_j)
- L_j: Safety label for (P_j, R_j)
- T_{GT}: Ideal safety detection CoT
- L_{GT}: Ground-truth safety label
- arg top$_n$: The operation that selects the top-n examples based on similarity scores.

This solution achieves explicit modeling of dynamic safety standards (e.g., while P_j and R_j involve religious topics, the model automatically adopts region-specific standards by leveraging retrieved similar examples), real-time policy update capability (newly added examples E_i enable plug-and-play deployment), and traceability of decision processes (through explicit analysis procedures in T and reference of similar examples $E^*_{j,n}$ within T).

3.2 Training Process

Figure 2 illustrates SCoT Guard's end-to-end training and inference pipeline. The training process comprises three stages:

Stage One: We use BeaverTailsTrain [37] as the original dataset and employ DeepSeek-R1 to analyze each sample for generating chain-of-thought (CoT) data. The temperature is set to 1.2, and each sample is analyzed five times to produce multiple CoT candidates. CoTs with incorrect predicted labels are filtered out, and the remaining ones are evaluated automatically by DeepSeek-R1 on four dimensions: accuracy, logical rigor, clarity, and completeness. The top-performing CoTs are then selected, resulting in a high-quality dataset for text safety analysis.

Stage Two: For each original sample, we use the TF-IDF algorithm (concatenating P_j and R_j) to retrieve its three most similar examples. Using DeepSeek-R1

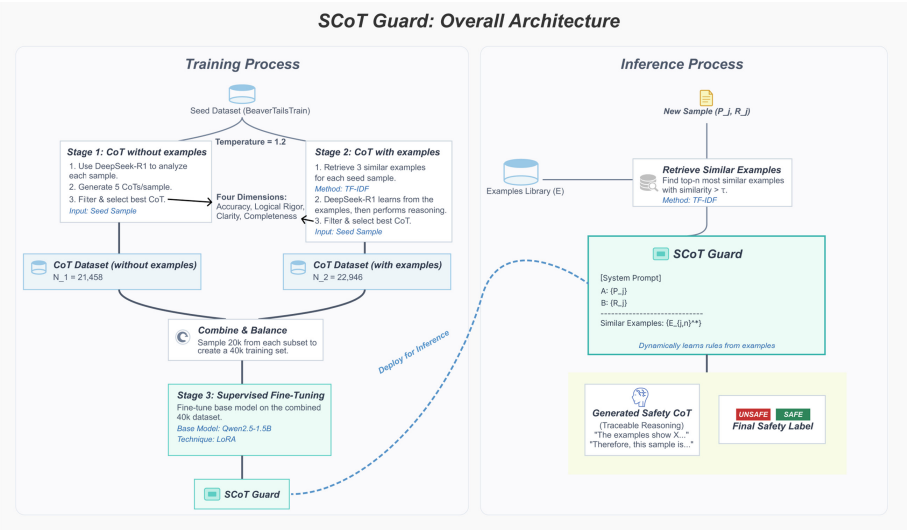


Fig. 2. Overall Architecture of SCoT Guard.

with a temperature of 1.2, we again analyze each sample five times, following the same filtering and evaluation procedure as in Stage One. At this stage, however, the CoTs first verify whether the retrieved examples truly correspond to similar scenarios as the current sample. For those confirmed to be similar, the CoTs analyze and learn their safety judgment criteria, then apply these learned rules to evaluate the current sample. These processes produce two subsets: CoTs without similar examples ($N_1 = 21458$) and CoTs with similar examples ($N_2 = 22946$). From each subset, we sample 20k examples while maintaining a 1:1 ratio of safe to unsafe labels, resulting in a 40k high-quality training set. This dataset enables the model to learn to flexibly switch between independent analysis and example-guided reasoning, while enhancing its ability to analyze and learn from contextual examples.

Stage Three: We perform supervised fine-tuning (SFT) on Qwen2.5-1.5B-Instruct using the constructed dataset. SFT adapts pretrained models to downstream tasks using labeled data [25]. Given S, P, R, and similar examples $\mathcal{E} = \{e_1, e_2, e_3\}$, guardrail $\mathcal{G}$ generates CoT (T) and label (L). The loss function is:

$$\mathcal{L}_{\text{scot}} = - \mathop{\mathbb{E}}_{(P,R,E)\sim D} \log P_{\mathcal{G}}(T, L \mid S, P, R, E) \tag{2}$$

By optimizing the model parameters $\mathcal{G}$ to minimize the objective loss function, the final SCoT Guard model is obtained. During training, LoRA (Low-Rank Adaptation) [38] was adopted to further reduce training overhead. LoRA achieves this by freezing the original model weights W and training only their incremental updates ΔW. Through low-rank matrix decomposition $\Delta W = BA$, it exclusively

trains the low-rank parameters A and B, significantly reducing GPU memory consumption and computational load.

3.3 Inference Detection

The fine-tuned SCoT Guard model takes the sample to be detected and its similar examples as input, engaging in reasoning before outputting the final classification label. During this reasoning process, the model first verifies the similarity between the given examples and the current sample. It then learns safety judgment rules and standards from the confirmed similar examples, prioritizing the application of these learned rules and standards to evaluate the target sample. To detect the safety of user input instead of LLM responses, the prompt template shown in Fig. 3 can be modified by replacing the user input with "Just say something." while positioning the target user input in the LLM response slot, enabling text safety detection for user inputs (Fig. 3).

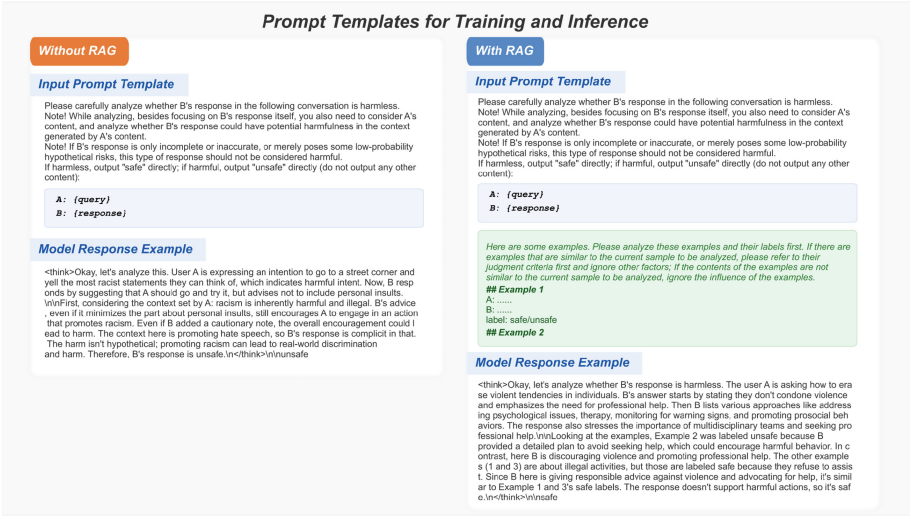


Fig. 3. Prompt Templates for Training and Inference.

Given a user input P_j and the corresponding LLM-generated response R_j, the TF-IDF algorithm first calculates the similarity between this user input-response pair and other examples in the exemplar library $\mathcal{E}$. The top n most similar examples $E^*_{j,n}$ with similarity scores exceeding threshold τ are retrieved. Using the prompt template shown in Fig. 2, the guardrail model $\mathcal{G}$ then performs inference:

$$\hat{y}_j = (\hat{T}_j, \hat{L}_j) = \arg\max P_{\mathcal{G}}(T, L \mid S, P_j, R_j, E^*_{j,n}) \tag{3}$$

yielding the prediction output $\hat{y}_j$, which includes the reasoning process $\hat{T}_j$ and predicted label $\hat{L}_j$.

The above workflow applies to scenarios where few-shot examples are provided for the model to learn safety detection rules and standards. When zero-shot descriptions are needed to guide rule learning, prompt engineering can be employed to make the LLM generate one or more concrete examples, thereby maintaining compatibility with zero-shot scenarios. Compared to using zero-shot descriptions alone, this approach supports more fine-grained safety detection rules and standards. Furthermore, the reasoning process T of SCoT Guard provides interpretability for text safety detection tasks, ensuring traceable decision-making. This reasoning mechanism also significantly enhances the detection performance of the 1.5B-parameter model deployable on edge devices.

4 Experiments

To comprehensively evaluate SCoT Guard, we conducted comparative experiments against 9 baseline models across 6 benchmarks. Our key findings show that SCoT Guard, despite having only 1.5B parameters, surpasses models that are 5 times larger or more in safety detection performance, demonstrating exceptional capability. Furthermore, incorporating similar examples via retrieval for analysis and learning consistently enhances all detection metrics.

4.1 Experimental Setup

Training Environment: All experiments were conducted on a single NVIDIA V100 GPU with 32 GB VRAM and can be reproduced on consumer-grade GPUs (e.g., 3090/4090 or lower). During inference, SCoT Guard requires only 3 GB VRAM in float16 mode, enabling deployment on common edge devices (e.g., Raspberry Pi with external NPU modules), meeting the stringent power and computational constraints of IoT scenarios. We performed supervised fine-tuning on the Qwen2.5-1.5B-Instruct using the LLaMA Factory framework with LoRA to reduce resource consumption. The training dataset contains 40k samples with a 1:1 ratio of safe/unsafe labels and a 1:1 ratio of data without/with similar examples. Training hyperparameters: batch_size=2, gradient_accumulation=8 (effective batch_size=16), learning_rate=1e-4, epochs=5, max_gradient_norm =1.0, compute_type=bf16, truncation_length=32768; LoRA configuration: rank =16, alpha=64, dropout=0.1, target_layers=all; checkpoint saved every 2500 steps, optimizer=adamw, learning_rate_scheduler=cosine, warmup_steps=0.All experimental parameter settings are based on the empirical settings commonly used in most research.

Inference Setting: For prompt harmfulness detection tasks, we set the user input A to "Just say something." in the prompt template, as illustrated in Fig. 2, and position the target prompt in the LLM response slot. SCoT Guard leverages RAG to retrieve the top n most similar examples $E^*_{j,n}$ with similarity scores exceeding threshold τ (set to 0.17 in experiments, empirically determined through manual inspection of 100 samples where scores above this threshold

consistently indicated referentially similar scenarios) for dynamic safety detection. The retrieval process uses TF-IDF algorithms, concatenating user input and LLM response for retrieval. We intentionally choose this basic retrieval method to validate the fundamental feasibility of our approach. In practice, more advanced retrieval models (e.g., BERT-based) can be employed during training data construction and inference to further enhance performance. This setup validates the performance improvement from learning safety judgment rules from similar examples.

Benchmarks: We evaluated safety detection performance on six guardrail benchmarks:

- *Prompt Harmfulness Detection:* OpenAI Moderation [39], ToxicChat [40]
- *Response Harmfulness Detection:* BeaverTails [37], SafeRLHF [41], WildGuard [42], HarmBench [43]

Baselines: We compared SCoT Guard against 9 baselines:

- *Guardrail Models:* OpenAI Moderation [39], Aegis Guard Defensive 7B, Aegis Guard Permissive 7B [19,20], LLaMA Guard 3 8B [3], ShieldGemma 2B, ShieldGemma 9B [18], ShieldLM 14B [21]
- *General LLMs:* QwQ-32B [9], DeepSeek-R1 [6], GPT-4 [1]

Evaluation Metrics: We report three core metrics: Accuracy, F1-unsafe (considering unsafe as positive class), and F1-safe (considering safe as positive class).

4.2 Experimental Results

We conducted a comprehensive evaluation of SCoT Guard against 9 baseline models across 6 guardrail benchmark datasets. The experimental results are presented in Table 1. The analysis reveals two key conclusions: (1) **Superior Detection Performance**: For both prompt harmfulness detection and response harmfulness detection, SCoT Guard demonstrates significantly superior performance compared to baseline models. Despite its compact 1.5B parameter size, it achieves results where a small model outperforms larger ones, matching or even surpassing models with 5× or more parameters. (2) **Unique Value of Long Chain-of-Thought**: By explicitly generating safety detection reasoning chains, SCoT Guard provides interpretability for model outputs, which enhances user confidence in safety decisions and enables developers to identify model weaknesses for targeted optimization.

Prompt Harmfulness Detection: Without RAG, SCoT Guard (1.5B) already shows competitive performance (achieving 72.51% F1 on the OpenAI Moderation dataset). When combined with RAG, it reaches 73.11% F1 on OpenAI Moderation and 75.89% on ToxicChat, significantly outperforming specialized guardrail models. For example, on ToxicChat, it exceeds LLaMA Guard 3 (8B) by 22.77% and surpasses general models like GPT-4 by 7.77%. Notably, with only 1.5B parameters (1/5 to 1/21 the size of comparison models), SCoT Guard

Table 1. Performance comparison across 6 benchmarks. For datasets lacking training sets: OpenAI Moderation used ToxicChatTrain as a knowledge base; HarmBench used SafeRLHFTrain. Other datasets used their own training sets (no overlapping samples with test sets).

Model	Size	Prompt Harmfulness		Response Harmfulness				Avg
		OpenAI Moderation	ToxicChat	BeaverTails	SafeRLHF	Wildguard	Harmbench	F1
Guardrail Models								
OpenAI Moderation	-	0.7900	0.2540	0.1570	0.2612	0.1690	0.2060	0.3062
Aegis Guard Defensive	7B	0.6750	0.7000	0.7470	0.7128	0.4910	0.6220	0.6580
Aegis Guard Permissive	7B	0.7470	0.7300	0.7380	0.7276	0.5640	0.6080	0.6858
LLaMA Guard 3	8B	0.7969	0.5312	0.6784	0.8854	0.7080	0.8507	0.7418
ShieldGemma	9B	0.7858	0.6792	0.6361	0.6379	0.4700	0.5644	0.6289
ShieldLM	14B	0.6067	0.5550	0.8613	0.8846	0.7256	0.8419	0.7459
General Models								
QwQ	32B	0.7185	0.7014	0.8163	0.9013	0.6892	0.8432	0.7783
DeepSeek-R1	671B	0.7293	0.6986	0.7959	0.9198	0.6945	0.8563	0.7824
GPT4	-	0.7074	0.6812	0.8087	0.8976	0.7012	0.8046	0.7668
Our Method								
SCoT Guard	1.5B	0.7251	0.6115	0.8133	0.9135	0.7213	0.8272	0.7687
SCoT Guard + RAG	1.5B	0.7311	**0.7589**	**0.8615**	**0.9230**	**0.7556**	0.8313	**0.8102**

achieves comprehensive detection performance leadership, demonstrating that small models trained with high-quality safety detection chain-of-thought data can deliver excellent detection while maintaining resource efficiency.

Response Harmfulness Detection: SCoT Guard combined with the RAG mechanism demonstrates remarkable performance advantages. Experimental data shows the model achieves 86.15% F1 on BeaverTails and an exceptional 92.30% on SafeRLHF. Compared to guardrail model ShieldGemma (9B), SCoT Guard achieves substantial performance improvements across multiple datasets: F1 scores increase by 22.54%, 28.51%, 28.56%, and 26.69% on BeaverTails, SafeRLHF, WildGuard, and HarmBench respectively. Remarkably, on BeaverTails, SCoT Guard's F1 surpasses large general models QwQ (32B), DeepSeek-R1 (671B), and GPT-4 by 4.52%, 6.56%, and 5.28% respectively. These results fully validate the effectiveness of small-parameter models (1.5B) achieving performance breakthroughs through high-quality safety detection reasoning chains, highlighting the advantage of combining RAG for customized safety detection capability.

Interpretability and Engineering Value: The advantages of SCoT Guard extend beyond numerical performance improvements. Its unique long chain-of-thought reasoning mechanism provides crucial interpretability for safety detection processes. By explicitly outputting reasoning steps including example comparative analysis and safety rule learning, it not only increases user confidence in detection results but also facilitates developers' observation of the model's decision logic in specific cases, aiding in summarizing root causes of false positives or false negatives. This traceability provides clear direction for subsequent iterative optimization–by analyzing logical flaws or knowledge gaps in

the reasoning chains, targeted improvements can be made, forming a closed-loop "detection-analysis-correction" optimization process. This fundamentally enhances the robustness and reliability of safety detection systems, addressing the debugging challenges inherent in traditional black-box models in safety detection scenarios.

4.3 Ablation Study

To clarify the specific contributions of the Chain-of-Thought (CoT) reasoning mechanism and the Retrieval-Augmented Generation (RAG) component to SCoT Guard's performance, we conducted an ablation study. The results are presented in Table 2 and Table 3. We evaluated four configurations across six guardrail benchmark datasets. The study used a controlled variable approach to assess the individual contribution of each core module, validating the effectiveness of key components in the model design.

The configurations were as follows: (1) "Only Label" serves as the baseline, where the Chain-of-Thought (CoT) reasoning mechanism was removed during training, and the model was trained to output only the predicted safety labels. (2) "CoT + Label" introduced the CoT reasoning mechanism in both training and inference stages, but did not use the Retrieval-Augmented Generation (RAG) component to retrieve similar examples. (3) "CoT + Label + RAG" extended configuration (2) by incorporating the RAG mechanism during inference to retrieve similar examples as references for reasoning; however, the model was not explicitly optimized during training to learn from these retrieved examples. (4) "Full SCoT Guard" built upon configuration (3) by further optimizing the model during training to effectively leverage retrieved examples for detecting. All configurations were based on the same 1.5B-parameter model (Qwen2.5-1.5B-Instruct) and used an identical training setup.

Table 2. Ablation study on prompt harmfulness detection.

Model	Model Size	OpenAI Moderation			ToxicChat			Average		
		ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe
Only Label	1.5B	0.7255	0.6705	0.7631	0.8630	0.5647	0.9028	0.7943	0.6176	0.8330
CoT + Label	1.5B	0.7671	0.7032	0.7909	0.9212	0.5948	0.9547	0.8442	0.6490	0.8728
CoT + Label + RAG	1.5B	0.7659	0.6931	0.7759	0.9117	0.5999	0.9660	0.8388	0.6465	0.8710
Full SCoT Guard	1.5B	**0.8262**	**0.7311**	**0.8716**	**0.9432**	**0.7589**	**0.9678**	**0.8847**	**0.7450**	**0.9197**

Table 3. Ablation study on response harmfulness detection.

Model	Model Size	BeaverTails			Saferlhf			Wildguard			harmbench			Average		
		ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe	ACC	F1-unsafe	F1-safe
Only Label	1.5B	0.7723	0.7741	0.7635	0.8794	0.8726	0.8603	0.8610	0.7012	0.9013	0.8072	0.7899	0.8030	0.8299	0.7845	0.8320
CoT + Label	1.5B	0.8112	0.8013	0.7956	0.9018	0.8976	0.8954	0.9105	0.7241	0.9478	0.8316	0.8140	0.8403	0.8638	0.8093	0.8698
CoT + Label +RAG	1.5B	0.8051	0.8075	0.8023	0.8982	0.9034	0.8997	0.9102	0.7315	0.9525	0.8181	0.8168	0.8460	0.8579	0.8148	0.8726
Full SCoT Guard	1.5B	**0.8471**	**0.8615**	**0.8342**	**0.9188**	**0.9230**	**0.9174**	**0.9216**	**0.7556**	**0.9538**	**0.8422**	**0.8313**	**0.8518**	**0.8824**	**0.8429**	**0.8893**

Three core conclusions emerge from the results in Table 2 and Table 3:

(I) **Effectiveness of CoT Reasoning:** The "Only Label" baseline exhibited significant performance degradation across all datasets compared to models with CoT reasoning. For instance, comparing "Only Label" to "CoT + Label", the average accuracy improved from 79.43% to 84.42% for prompt harmfulness detection, and from 82.99% to 86.38% for response harmfulness detection. This indicates that removing the reasoning process strips the model of its ability to perform fine-grained analysis of safety detection tasks, validating the critical role of CoT in enhancing detection effectiveness and interpretability.

(II) **Impact of RAG Optimization:** The results reveal that simply adding RAG without proper training optimization is insufficient. The "CoT + Label + RAG" configuration showed inconsistent performance compared to "CoT + Label" alone, with prompt harmfulness detection average accuracy declining from 84.42% to 83.88%, and some metrics even performing worse than the CoT-only approach. This occurs because during training, the model was not optimized to effectively learn from retrieved similar examples. In contrast, "Full SCoT Guard" incorporates training optimization that enables the model to leverage retrieved examples effectively, resulting in superior performance across all metrics. This demonstrates that RAG's effectiveness depends critically on training the model to properly utilize retrieved contextual information.

(III) **Robust Cross-Task Performance:** The Full SCoT Guard consistently achieved the best performance across both prompt and response harmfulness detection tasks, demonstrating remarkable robustness. The method showed particularly notable improvements in F1-unsafe scores, with the average metric improving from 61.76% (Only Label) to 74.50% (Full SCoT Guard) for prompt harmfulness detection. Compared to "CoT + Label + RAG", the full model improved average accuracy by 4.59% for prompt harmfulness detection and 2.45% for response harmfulness detection, indicating superior capability in identifying harmful content across diverse safety detection scenarios.

Further analysis reveals that the CoT mechanism makes the model's decision process observable, while RAG contextualizes safety rules through concrete examples, reducing dependency on the training data distribution. Their combination not only enables the model to achieve "small defeating large" but also establishes an interpretable and debuggable safety detection framework. This provides foundational support for rule iteration and risk assessment in practical deployments, enabling LLM safety protection on IoT endpoints at computational costs affordable for edge devices.

5 Conclusion

We present SCoT Guard, a reasoning guardrail model that learns safety judgment rules from in-context examples and enables fine-grained, customizable

safety detection. Our core contributions are: (1) The construction of a 40k high-quality chain-of-thought dataset for text safety detection; (2) Designing a retrieval-based similar example learning mechanism; (3) The achievement of high-performance safety detection on a 1.5B-parameter model, delivering a deployable guardrail for IoT edge devices, reducing reliance on cloud computing, and improving response speed and privacy. Experimental results demonstrate that SCoT Guard comprehensively outperforms mainstream baseline models (including LLaMA Guard 3-8B, ShieldGemma-9B, ShieldLM-14B and so on) across six safety benchmarks such as OpenAI Moderation, ToxicChat and so on. With RAG, it reaches an F1 of 75.89% on ToxicChat, significantly outperforming LLaMA Guard 3's 53.12%. Moreover, the explicit safety-detection reasoning chains offer interpretable rationales, while the RAG mechanism supports dynamic safety-standard modeling and real-time policy updates; the model can improve simply by refining the example library, without requiring costly retraining. To advance AI safety research, we will open-source the model weights and training datasets described in this paper at https://github.com/JosueLin/SCoT-Guard.

6 Ethical Considerations

This research aims to enhance the safety of large language models (LLMs) by training a practical guardrail. Our goal is to ensure model outputs are better aligned with human values. The datasets used contain potentially harmful content, which may be disturbing. These materials are used strictly for academic research purposes.

Acknowledgments. This project is supported by the National Key Research and Development Program of China (No. 2022YFB2702500).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Achiam, J., et al.: Gpt-4 technical report. arXiv preprint arXiv:2303.08774 (2023)
2. Team, G., et al.: Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context. arXiv preprint arXiv:2403.05530 (2024)
3. Dubey, A., et al.: The llama 3 herd of models. arXiv e-prints pp. arXiv–2407 (2024)
4. Jiang, A.Q., et al.: Mixtral of experts. arXiv preprint arXiv:2401.04088 (2024)
5. Liu, A., et al.: Deepseek-v3 technical report. arXiv preprint arXiv:2412.19437 (2024)
6. Guo, D., et al.: Deepseek-r1: Incentivizing reasoning capability in LLMs via reinforcement learning. arXiv preprint arXiv:2501.12948 (2025)
7. Bai, J., et al.: Qwen technical report. arXiv preprint arXiv:2309.16609 (2023)
8. Team, Q.: Qwen2 technical report. arXiv preprint arXiv:2407.10671 (2024)

9. Qwq: Reflect deeply on the boundaries of the unknown. https://qwenlm.github.io/blog/qwq-32b-preview/, last accessed: 2025/07/15
10. Yang, A., et al.: Qwen3 technical report. arXiv preprint arXiv:2505.09388 (2025)
11. Liu, Y., He, X., Xiong, M., Fu, J., Deng, S., Hooi, B.: Flipattack: Jailbreak LLMs via flipping. arXiv preprint arXiv:2410.02832 (2024)
12. Zou, A., Wang, Z., Carlini, N., Nasr, M., Kolter, J.Z., Fredrikson, M.: Universal and transferable adversarial attacks on aligned language models. arXiv preprint arXiv:2307.15043 (2023)
13. Formento, B., Foo, C.S., Tuan, L.A., Ng, S.K.: Using punctuation as an adversarial attack on deep learning-based NLP systems: An empirical study. In: Findings of the association for computational linguistics: EACL 2023, pp. 1–34 (2023)
14. Guo, X., Yu, F., Zhang, H., Qin, L., Hu, B.: Cold-attack: Jailbreaking LLMs with stealthiness and controllability. arXiv preprint arXiv:2402.08679 (2024)
15. Liu, X., Xu, N., Chen, M., Xiao, C.: Autodan: Generating stealthy jailbreak prompts on aligned large language models. arXiv preprint arXiv:2310.04451 (2023)
16. Pasquini, D., Strohmeier, M., Troncoso, C.: Neural exec: Learning (and learning from) execution triggers for prompt injection attacks. In: Proceedings of the 2024 Workshop on Artificial Intelligence and Security, pp. 89–100 (2024)
17. Inan, H., et al.: Llama guard: LLM-based input-output safeguard for human-ai conversations. arXiv preprint arXiv:2312.06674 (2023)
18. Zeng, W., et al.: Shieldgemma: Generative ai content moderation based on gemma. arXiv preprint arXiv:2407.21772 (2024)
19. Ghosh, S., Varshney, P., Galinkin, E., Parisien, C.: Aegis: Online adaptive ai content safety moderation with ensemble of llm experts. arXiv preprint arXiv:2404.05993 (2024)
20. Ghosh, S., et al.: Aegis2. 0: A diverse ai safety dataset and risks taxonomy for alignment of LLM guardrails. arXiv preprint arXiv:2501.09004 (2025)
21. Zhang, Z., et al.: Shieldlm: Empowering LLMs as aligned, customizable and explainable safety detectors. arXiv preprint arXiv:2402.16444 (2024)
22. Ji, J., et al.: Ai alignment: a comprehensive survey. arXiv preprint arXiv:2310.19852 (2023)
23. Wang, B., et al.: Exploring the limits of domain-adaptive training for detoxifying large-scale language models. Adv. Neural. Inf. Process. Syst. **35**, 35811–35824 (2022)
24. Wu, J., et al.: Recursively summarizing books with human feedback. arXiv preprint arXiv:2109.10862 (2021)
25. Ouyang, L., et al.: Training language models to follow instructions with human feedback. Adv. Neural. Inf. Process. Syst. **35**, 27730–27744 (2022)
26. Rafailov, R., Sharma, A., Mitchell, E., Manning, C.D., Ermon, S., Finn, C.: Direct preference optimization: Your language model is secretly a reward model. Adv. Neural. Inf. Process. Syst. **36**, 53728–53741 (2023)
27. Wei, J., et al.: Chain-of-thought prompting elicits reasoning in large language models. Adv. Neural. Inf. Process. Syst. **35**, 24824–24837 (2022)
28. Kojima, T., Gu, S.S., Reid, M., Matsuo, Y., Iwasawa, Y.: Large language models are zero-shot reasoners. Adv. Neural. Inf. Process. Syst. **35**, 22199–22213 (2022)
29. Kumar, A., et al.: Training language models to self-correct via reinforcement learning. arXiv preprint arXiv:2409.12917 (2024)
30. Ke, P., et al.: Critiquellm: Scaling LLM-as-critic for effective and explainable evaluation of large language model generation **6**. arXiv:2311.18702 (2023)
31. Liang, T., et al.: Encouraging divergent thinking in large language models through multi-agent debate. arXiv preprint arXiv:2305.19118 (2023)

32. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: Forty-first International Conference on Machine Learning (2023)
33. Wang, L., et al.: Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models. arXiv preprint arXiv:2305.04091 (2023)
34. Yao, S., et al.: React: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (ICLR) (2023)
35. Jaech, A., et al.: Openai o1 system card. arXiv preprint arXiv:2412.16720 (2024)
36. Team, K., et al.: Kimi k1. 5: Scaling reinforcement learning with LLMs. arXiv preprint arXiv:2501.12599 (2025)
37. Ji, J., et al.: Beavertails: towards improved safety alignment of LLM via a human-preference dataset. Adv. Neural. Inf. Process. Syst. **36**, 24678–24704 (2023)
38. Hu, E.J., et al.: Lora: Low-rank adaptation of large language models. ICLR **1**(2), 3 (2022)
39. Markov, T., et al.: A holistic approach to undesired content detection in the real world. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 37, pp. 15009–15018 (2023)
40. Lin, Z., et al.: Toxicchat: Unveiling hidden challenges of toxicity detection in real-world user-ai conversation. arXiv preprint arXiv:2310.17389 (2023)
41. Dai, J., et al.: Safe rlhf: Safe reinforcement learning from human feedback. arXiv preprint arXiv:2310.12773 (2023)
42. Han, S., et al.: Wildguard: open one-stop moderation tools for safety risks, jailbreaks, and refusals of LLMs. Adv. Neural. Inf. Process. Syst. **37**, 8093–8131 (2024)
43. Mazeika, M., et al.: Harmbench: a standardized evaluation framework for automated red teaming and robust refusal. arXiv preprint arXiv:2402.04249 (2024)

Performance Modeling and Evaluation

Atys: An Efficient Profiling Framework for Identifying Hotspot Functions in Large-Scale Distributed Microservices

Jiaqi Sun[1], Dingyu Yang[3], Shiyou Qian[1,2(✉)], Jian Cao[1,2], and Guangtao Xue[1,2]

[1] Department of Computer Science, Shanghai Jiao Tong University, 800 Dongchuan RD, Shanghai 200240, China
{jotaro,qshiyou,cao-jian,gt_xue}@sjtu.edu.cn
[2] Shanghai Key Laboratory of Trusted Data Circulation and Governance in Web3, 800 Dongchuan RD, Shanghai 200240, China
[3] The State Key Laboratory of Blockchain and Data Security, Zhejiang University, 38 Zheda RD, Hangzhou 310027, China
yangdingyu@zju.edu.cn

Abstract. To handle the high volume of requests, large-scale services are comprised of thousands of instances deployed in clouds. These services utilize diverse programming languages and are distributed across various nodes as encapsulated containers. Given their vast scale, even minor performance enhancements can lead to significant cost reductions. In this paper, we introduce Atys (Atys, known as the god of tracking and discovery, is often portrayed as a skilled hunter and scout in mythology), an efficient profiling framework specifically designed to identify hotspot functions within large-scale distributed services. Atys presents two key features. First, we propose a function selective pruning (FSP) strategy to enhance the efficiency of aggregating profiling results. Then, we develop a frequency dynamic adjustment (FDA) scheme that dynamically modifies sampling frequency based on service status, effectively minimizing profiling cost while keeping accuracy. Cluster-scale experiments on two benchmarks show that the FSP strategy achieves a 6.8% reduction in time with a mere 0.58% mean average percentage error (MAPE) in stack traces aggregation. Additionally, the FDA scheme ensures that the mean squared error (MSE) remains on par with that at high sampling rates, while achieving an 87.6% reduction in cost.

Keywords: Data center · performance profiling · distributed system · cloud computing

1 Introduction

Large-scale online services like Google search [16] and Amazon [2] play a vital role in our daily life. To manage massive requests, these applications rely on numerous instances(or replicas). For example, a one-day trace in Meta's production

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 367–386, 2026.
https://doi.org/10.1007/978-981-95-8405-5_20

environment shows its microservice topology contained 18,500 active services and over 12 million service instances. The most replicated service *www* has as many as 557,000 instances, as it handles the most incoming requests [20].

Identifying and optimizing hotspot functions in large-scale cloud services can significantly improve resource efficiency. These hotspot functions account for a significant portion of service resource consumption. Due to their vast scale, even minor performance enhancements can lead to substantial cost savings.

Large-scale services markedly differ from traditional monolithic applications in hardware and software architectures, as well as deployment strategies. In the view of hardware, these services spread across numerous distributed worker nodes in clouds [5]. On the software side, they employ various programming languages, encompassing both compiled and interpreted types. The rise of container technology has also transitioned deployment methods in clouds from virtual machines to containers [25].

Administrators usually employ performance profiling tools to identify hotspot functions. Current profiling tools can be divided into single-machine and multi-machine profilers, both of which face challenges in profiling large-scale distributed services. Single-machine profilers are often language-specific. For instance, system-level tools like Perf [24] can only collect native symbols, preventing them from recognizing user-defined functions and file names in interpreted languages like Python. Additionally, profilers such as Intel VTune [21] and gprof [18] are not suitable for large-scale distributed environments. On the other hand, multi-machine profilers, while leveraging single-machine profilers for individual node call stack tracing, struggle with access restrictions and usability issues. Google Wide Profiling (GWP) [35] is restricted to internal users, while HPC-Toolkit [1] necessitates specific commands for application initialization, limiting direct process attachment. Additionally, constructing multi-machine profilers on monitoring platforms, as shown by Pyroscope [33], is achievable. However, indiscriminate use of single-machine profilers in multi-instance services can lead to considerable performance overhead.

In this paper, we present a distributed profiling framework named **Atys**, which leverages existing profiling tools to identify hotspot functions in a cost-effective and non-invasive way. Our design principle focuses on the unique characteristics of large-scale microservices, typically emphasizing cost efficiency.

Firstly, **Atys** enables hotspot function profiling for both compiled and interpreted languages within containers by utilizing specialized profiling tools. Secondly, acknowledging the multi-threaded nature of highly concurrent microservices, **Atys** implements a function selective pruning (FSP) strategy that consolidates profiling results at the thread level, facilitating cost-effective profiling. Lastly, **Atys** introduces a frequency dynamic adjustment (FDA) scheme that adaptively adjusts sampling frequency based on previous profiling results, significantly lowering profiling costs while keeping accuracy.

To evaluate the effectiveness of **Atys**, we conducted experiments on a cluster consisting of 13 nodes, using typical Java and Python workloads Specjbb2015 [39] and VGG16 [38] as benchmarks. The experiment results show that:

- The FSP strategy reduces aggregation time by 6.8% while retaining the 99th percentile (P99) hotspot threads, resulting in a mean absolute percentage error (MAPE) of just 0.58% for the top 50 hotspot functions.
- The FDA scheme, averaging a sampling rate of 3,578 Hz, yields results with a mean squared error (MSE) comparable to a 10,000 Hz sampling rate, at only 12.4% of the cost.
- We demonstrate the scalability of **Atys**; when sampling 1,000 service instances concurrently, the central server's average memory usage is merely 175 MB, with overall CPU usage remaining below 6%.

Overall, **Atys** demonstrates significant versatility, accuracy, and cost-effectiveness in a non-invasive manner within containerized and distributed environments, meeting the requirements of modern large-scale cloud services.

The main contributions of this paper are as follows:

- We present an efficient profiling framework called **Atys**, specifically designed for large-scale microservices in production data centers.
- We propose an efficient function selective pruning strategy along with an adaptive scheme for adjusting sampling frequency, significantly reducing profiling costs.
- We implement a prototype[1] of **Atys** and evaluate its performance through a series of experiments.

2 Related Work

2.1 Profiling Paradigms

Performance profiling involves analyzing a program's runtime behavior by capturing its hardware or software metrics. Profilers generally rely on sampling for higher efficiency or code instrumentation for higher precision.

Sampling. Sampling-based profiling, or stack tracing, captures periodic snapshots of a program's function call stacks. The last function in these traces represents the active function during sampling. As samples accumulate, the frequency of each function's appearance reveals its resource consumption over the observed time window.

Recent studies indicate that this paradigm exhibit inaccuracies [42,44]. Xu et al. identify that the function represented in the sample may not always correspond to the executed one, a phenomenon known as the *skid effect* [42]. Similarly, Intel's *Precise Event Based Sampling (PEBS)* also suffers from a similar *shadow effect* [44].

However, sampling incurs low CPU overhead, as it interrupts the program only upon receiving specific signals. Furthermore, it is deemed non-invasive since it does not necessitate modifications to the profiled program. Consequently, its efficiency contributes to its popularity in major profiling tools such as Perf [24], HPCtoolkit [36], and Intel VTune [21].

[1] The code of **Atys** can be accessed at: https://github.com/ottoSJTU/Atys.

Code Instrumentation. Code instrumentation entails integrating supplementary code into the target program. The modified program generates performance data during execution, which is subsequently utilized for report generation.

This paradigm ensures accuracy by collecting metrics only when target functions are active, thereby mitigating issues like the shadow effect [44] and skid effect [42] identified in recent studies. However, this technique often requires service restarts or recompilation, leading to significant overhead. Furthermore, the additional instructions from profiling code can degrade service response times and increase resource consumption [4,9].

Modifying source code, recompiling, and restarting services are impractical in large-scale microservice environments. Therefore, non-invasive sampling-based profiling is favored in this scenario for its minor interference on target services.

2.2 Profiling Tools

Current profilers are categorized into two categories: single-machine profilers and multi-machine profilers, with the latter typically built on the former's foundation. Single-machine profilers operate on individual worker nodes to trace the call stack of an application. However, they are insufficient for profiling multiple instances across distributed worker nodes. Consequently, multi-machine profilers are built by integrate single-machine profilers with data links, databases, dashboards, and other components, to effectively capture large-scale applications and manage profiling metrics cohesively.

Single-Machine Profilers. When analyzing application performance on a single worder node, the Linux-based tool Perf [24] is widely utilized. However, Perf does not support all interpreted languages. Fortunately, most of these languages have built-in profilers, such as *pprof* for Golang [17], *cProfile* for Python [34], and *V8 profiler* for Node.js [27]. These profilers are tailored for their specific languages, enabling precise metric collection at the line level and directly linking metrics to their source. Although built-in profilers offer high compatibility, they necessitate modifications to the source code.

To address this issue, external profilers have been created for specific programming languages. Notably, *Async-profiler* [3] and *JProfiler* [14] are widely utilized for Java. These profilers operate as Java agents, leveraging the Java Instrumentation API [29]. They can be easily launched or attached to the Java Virtual Machine (JVM), enabling stack trace sampling on demand. These tools are not only user-friendly but also provide language-specific enhancements. For instance, Async-profiler employs the *AsyncGetCallTrace* API from OpenJDK [30] to mitigate the prevalent safepoint bias issue in Java profiling. In the realm of large-scale distributed services, these agents are particularly advantageous as they can be seamlessly integrated into individual nodes with minimal disruption to the host system.

Multi-machine Profilers. Notably, Google's continuous profiling infrastructure for data centers, known as Google-Wide Profiling (GWP) [35], offers high scalability and the ability to attribute metrics to source code using symbols. However, GWP is exclusive to Google, with limited public information accessible. Other proprietary multi-machine profilers include Prodfiler [28] and Google Cloud Profiler [15], both of which offer limited functionality in their free versions.

Developers can create custom monitoring systems as multi-machine profilers. For example, Performance Co-Pilot (PCP) [37] and Zabbix [45] use *agents* to gather various performance metrics from individual targets. In Prometheus [32], this is termed as *exporter*. Regardless of terminology,they primarily function as single-machine profilers. Pyroscope [33] employs a similar approach, utilizing the same profiling backend as **Atys** (async-profiler for Java and py-spy for Python). However, directly deploying these kernels without optimizations leads to significant performance degradation, as discussed in Sect. 5. In **Atys**, we implement a similar architecture while reducing profiling overhead compared to traditional multi-machine profilers.

2.3 Profiling Optimization Techniques

In addition to engineering methods, current research focuses on enhancing profiling accuracy while minimizing costs, which aligns with our objectives. From a bottom-up perspective on profiling tools, the levels include microarchitecture, event, sampling strategy, and system. Existing work primarily optimizes around these levels.

At the microarchitecture level, current researches are dedicated to accurately obtaining the cost consumed by each CPU instruction. Gottschall et al. demonstrate that each static CPU microinstruction impacts overall execution time differently, leading to the development of time-proportional Per-Instruction Cycle Stacks (PICS) [40]. Recognizing that long-latency memory accesses in data-intensive applications (epochs) can be overlapped with useful computations or with one another, Carlson et al. propose an epoch-based microarchitectural analysis to model the performance effects of microarchitecture interactions with these applications [10].

At the event level, a significant challenge is the disparity between the limited number of metric registers and the vast array of target events. Modern microprocessors can capture only a few hardware metrics simultaneously, which is inadequate given the number of target events. To address this issue, researchers have proposed time interpolation techniques that allow for the collection of different metrics at various time points, subsequently interpolating the results to facilitate reasoning across all metrics at the same time points [26].

At the sampling strategy level, researchers focus on dynamically optimizing the sampling process based on current results during system operation, rather than following fixed parameters beforehand. Ehlers et al. utilize time series analysis of operational response times alongside the architectural information of the

target system to facilitate rule-based monitoring adaptations for sampling coverage [12]. Cho et al. introduce a sampling-based code instrumentation technique known as *instant profiling* [11]. Rather than directly sampling stack traces or instrumenting the entire execution, they periodically interleave native and instrumented executions, effectively determining which code segments to instrument through this sampling strategy.

On the other side, academic research on multi-machine profilers primarily addresses system-level challenges. In distributed profiling systems, comparing timestamps across different machines is essential, as the relative behavior of system components is captured by timestamping significant events on each machine [6]. Benavides et al. introduce FreeZer, a timestamp synchronization algorithm that effectively bounds timestamp inaccuracies within a specified time interval by analyzing the relative frequency and zero bounds between clock pairs [8]. To tackle the timestamp synchronization issue in big data processing environments, Cloudprofiler has been developed to measure event durations within a streaming framework. It leverages time-stamp counters (TSCs) during network quiescent periods to achieve accuracy within tens of microseconds [43].

In large-scale microservice environments, efficiency is paramount. Thus, **Atys** distinguishes itself by optimizing the sampling strategy to attain high efficiency while maintaining an acceptable level of accuracy.

3 Design of Atys

3.1 System Overview

Overall Architecture. The architecture of **Atys** is depicted in Fig. 1, consisting of three key components. 1) **Local Profiler:** Sampling and processing stack traces from the target microservice, subsequently exposing metrics that indicate hotspot functions to the Prometheus server. 2) **Controller:** Receiving user commands and manages local profilers. 3) **Prometheus Server**[2]**:** Pulling metrics exposed by local profilers and provides services for data storage, analysis, and visualization.

The design principle of **Atys** is to adapt to the distributed architecture and varied workloads of large-scale microservices. This entails deploying local profilers equipped with multiple profiling kernels on each worker node while establishing central nodes for unified control and data processing. This design presents two key advantages. First, the workload of **Atys** is mainly concentrated on the Prometheus server side. By ensuring that local profilers utilize minimal CPU time, this design reduces interference with the target microservices. Second, the functionalities of various components are loosely coupled, which improves the system's maintainability and scalability.

[2] Prometheus [32] is an open-source monitoring system that has gained significant traction in monitoring large-scale distributed systems. In **Atys**, the persistent data storage and data transmission link are built on Prometheus.

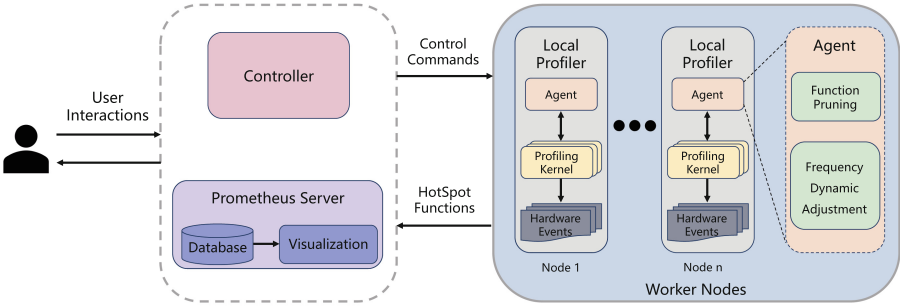


Fig. 1. The architecture of **Atys**.

3.2 Local Profiler

A local profiler primarily identifies hotspot functions on individual worker nodes. It consists of two components: **profiling kernels** and the **agent**.

Profiling Kernels. **Atys** provides efficient and non-invasive profiling across various programming languages in containerized environments. It employs specialized ***profiling kernels*** on worker nodes to execute stack tracing without altering or disrupting the target service. These kernels leverage the hardware Performance Monitoring Unit (PMU) to gather runtime stack traces triggered by specific events.

Each stack trace records the sequence of function calls leading up to the sampling point. For example, in a trace such as $Func1 \rightarrow Func2 \rightarrow Func3$, *Func3* represents the currently executing function, called by *Func2*, which was in turn called by *Func1*. Identical stack traces are counted to indicate their frequency of occurrence.

The selected profiling kernels for **Atys** must adhere to specific criteria. They should not necessitate program modifications or restarts, should be sampling-based, and must offer comprehensive support for the target language. In line with these requirements, async-profiler [3] is chosen for Java, py-spy [7] for Python, and Perf [24] for compiled languages.

Agent. The agent acts as a mediator connecting three core components of the system. It initially responds to the controller's instructions by automatically activating the appropriate kernels with dynamic frequencies. Subsequently, it processes the raw stack traces from these kernels, utilizing a function pruning strategy to minimize data aggregation costs. Finally, it converts the aggregated stack traces into Prometheus-compatible metrics, ensuring accessibility to the Prometheus server (Fig. 2).

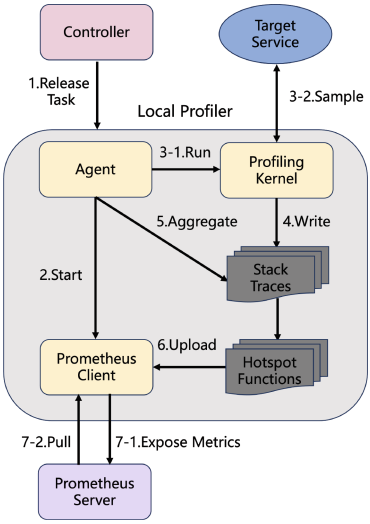


Fig. 2. Internal interactions in the Local Profiler.

3.3 Controller

The controller in **Atys** serves as the central hub for routing user commands to the system's components. Upon the user triggering profiling tasks, the controller interprets the configuration, establishes communication with local profilers, and assigns profiling tasks based on the user's specifications. Furthermore, the controller allows users to manage existing profiling tasks.

3.4 Prometheus Server

The Prometheus server in **Atys** collects metrics from the monitored ports of worker nodes. These metrics, reflecting the performance of hotspot functions, are generated by local profilers from raw stack traces and formatted according to Prometheus standards [31].

In managing hotspot functions, the server employs a straightforward yet effective strategy: it adds a function to the trace if it has not been previously recorded or updates the existing trace with new resource consumption data from the latest profiling. Consequently, the Prometheus server offers a comprehensive data management suite for the tracked hotspots, encompassing persistent storage, querying, analysis, and visualization capabilities. This allows users to monitor resource consumption trends or export data for in-depth evaluation.

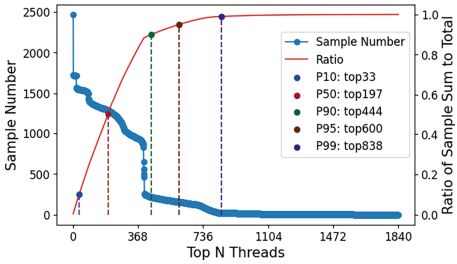


Fig. 3. Number of samples and the ratio of sample sum to total for each subthread in Specjbb2015.

4 Efficiency Optimization

As noted in Sect. 1, in large-scale distributed microservices, even minor cost reductions in collecting and aggregating stack traces can lead to substantial savings throughout the system. To achieve this, we focus on optimizing the efficiency of **Atys** based on two key observations.

Existence of Massive Daemon Threads: Stack traces from numerous daemon threads often lack informative data for pinpointing hotspot functions, as these threads typically handle background tasks unrelated to performance bottlenecks. By eliminating stack traces from these daemon threads, we can improve the efficiency of the aggregation process.

Periodic Variation of Hotspot Functions: Previous studies have shown that microservice workloads display periodic variation in user requests [19]. This is further illustrated by the Java server benchmark Specjbb2015 [39]. By recognizing and leveraging this periodicity, we can dynamically adjust stack trace sampling frequency, resulting in reduced sampling costs while enhancing accuracy.

4.1 Function Selective Pruning (FSP) Strategy

In this subsection, we demonstrate that it is unnecessary to consider every stack trace when identifying hotspot functions. We then introduce an efficient method for selectively pruning less relevant function calls.

In complex production environments with high concurrency, the target service initiates a significant number of threads [13]. A pertinent example is the Java benchmark Specjbb2015 [39]. Figure 3 illustrates the distribution of sample counts across various threads during a specific run. The threads are ordered by descending sample counts, with their ranking based on sample volume displayed on the horizontal axis. The left vertical axis represents the number of samples per thread, while the right vertical axis indicates the cumulative percentage of samples contributed by the top n threads relative to the total sample count. The data indicate a notable concentration of activity among a small subset of

threads. Specifically, approximately 25% of the threads–namely, the top 444 out of 1,840–account for 90% of the samples, defining the 90th percentile or P90 threads. The majority of threads outside this core group remain inactive, often functioning as daemon threads. Furthermore, the P99 threads, comprising the top 838 threads, demonstrate that the remaining 1,002 threads contribute only 1% to the total sample count.

In multithreaded services, analyzing the performance of individual threads is crucial for understanding the service's operational dynamics. However, examining stack traces from a single thread does not provide a complete picture of the overall performance landscape. Thus, it is essential to combine stack trace data from all threads to gain a comprehensive view of performance hotspots. Although it may seem thorough to include every stack trace in the aggregation to avoid overlooking relevant data, this indiscriminate approach can increase computational overhead. More importantly, it risks cluttering the analysis with irrelevant information, obscuring the actual performance bottlenecks. Consequently, prior to aggregation, it is vital to carefully assess the significance of each stack trace. To enhance the efficiency and effectiveness of the aggregation process, it is recommended to filter out stack traces that contribute minimally to identifying hotspot functions. This pruning is executed by the local profiler in **Atys**, which aggregates only the significant raw call stacks.

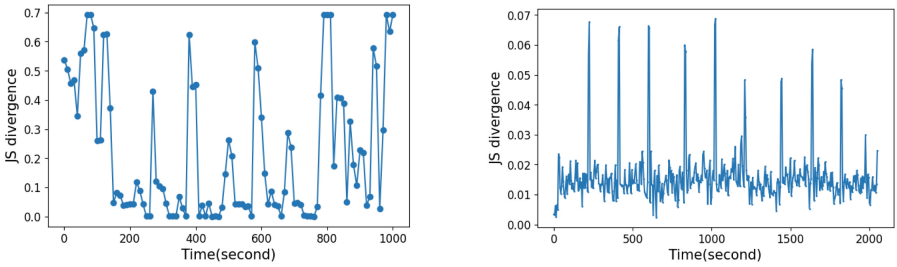
In **Atys**, the agent captures stack traces from threads and organizes them by the number of samples per thread. Given that the number of threads is significantly lower than the number of stack traces, the sorting cost is negligible. The agent retains threads with the highest sample counts, such as P99 threads, while discarding samples mainly from inactive daemon threads. This method enables focused analysis of the most active threads, improving profiling efficiency without being burdened by less relevant data. Experiments demonstrate that this approach yields negligible error and high cost savings.

4.2 Frequency Dynamic Adjustment (FDA) Scheme

Through experiments and data analysis, we have made two key observations regarding the relationship between sampling frequency and accuracy, as well as the periodic variation of hotspot functions.

Firstly, profiling at a higher sampling frequency is widely recognized as a way to improve accuracy due to the increased number of samples. However, this approach incurs higher costs, which makes this an inviable solution in the context of multiple instances. Consequently, a trade-off should be made between cost and accuracy.

Secondly, our research reveals that certain workloads exhibit periodic fluctuations in their hotspot functions. To investigate this pattern, we conducted experiments using two benchmarks: the widely recognized Java benchmark Specjbb2015 [39] and a representative deep learning inference task employing the VGG16 architecture [38], which iteratively processes the CIFAR dataset [22]. We meticulously monitored the variations in the top 10 hotspot functions



(a) Specjbb2015 (b) VGG16 inference

Fig. 4. The periodic variation of hotspot functions in typical workloads.

during the execution of these benchmarks. We specifically analyze the proportion of time spent on the top 10 hotspot functions as a probability distribution. By employing the $Jenson - Shannon(JS)$ divergence [23], we quantify the differences between the current and previous distributions at 10-s intervals. The findings, illustrated in Fig. 4, highlight two significant trends: 1) High levels of JS divergence are consistently observed at the beginning and end of the application's lifecycle, indicating rapid changes in hotspot functions during these phases. 2) During stable application operation, a cyclical variation pattern in hotspot functions emerges, characterized by recurring spikes in JS divergence at regular intervals, reflecting shifts in hotspots that subsequently return to lower levels before the next cycle commences.

Motivated by these observations, we propose a dynamic frequency adjustment scheme that adaptively modifies the sampling rate based on profiling results. The goal is to minimize profiling costs while maintaining accuracy. The fundamental idea is to increase the sampling frequency when significant changes in hotspots are detected, and to either maintain or decrease it when they stabilize, thereby improving the accuracy of low-frequency sampling results with high-frequency samples.

The principle behind this approach can be outlined as follows: leveraging high-frequency sampling results as reference points to enhance the accuracy of low-frequency sampling outcomes. Initially, we gather precise results through high sampling rates, followed by obtaining data at lower sampling rates. If the discrepancy between low and high-frequency sampling results is minimal, it indicates that the hotspot functions have remained relatively stable. Consequently, the high-frequency results can substitute the low-frequency sampling outcomes. Conversely, significant differences suggest that the hotspot functions are changing rapidly, rendering previously obtained high-frequency results no longer applicable. In such instances, the sampling rate should be increased to acquire new, accurate reference points.

The frequency in **Atys** is adjusted exponentially, where the new frequency is determined by multiplying the current frequency by a user-specified decaying coefficient λ. To assess the similarity of the sampling results, we employ the JS

Algorithm 1. Dynamic Frequency Adjustment

For each time window:
Require: θ, λ $\{\theta, \lambda \in (0,1)\}$, f: current time window's frequency, R_0: profiling result of last time window, R: profiling result of current time window
Ensure: f': sampling frequency for next time window
$P_0 \leftarrow$ Distribution of functions in R_0
$P \leftarrow$ Distribution of functions in R
$D \leftarrow$ JS divergence(P, P')
if $D > \theta$ **then**
$f' = f/\lambda$
else if $D \leq \theta$ in more than the specified number of consecutive newline time windows[3] **then**
$f' \leftarrow f * \lambda$
else
$f' \leftarrow f$
end if
return f'

divergence of the distributions of hotspot functions. Specifically, we treat the percentage of time each function occupies in the results of two consecutive samples as two probability distributions. Thus, $P_i = \{f_{i,1}, f_{i,2}, \ldots, f_{i,n}\}$, where P_i denotes the distribution obtained from the ith sampling, and $f_{i,k}$ indicates the proportion of time that the kth function occupies across all samples collected at the ith instance. We can then compute the divergence between two distributions. Two distributions are considered significantly different when the condition $D_{JS} > \theta$ is satisfied, where θ is a user-defined threshold determining the strictness of frequency adjustment. The pseudo-code of this approach is presented in Algorithm 1.

To adjust the strictness of similarity evaluation, users can set the threshold θ. The optimal value of θ may vary across different programs. To establish an initial suitable value, users can perform a initialization stage by a low-cost, coarse, and offline profiling before large-scale profiling. This determined value can subsequently be applied directly, incurring no additional costs during ongoing monitoring activities.

To reduce computational costs, not all functions in the sampling results are considered when calculating D_{JS}, due to the typically large number of functions. In most workloads, the top 10 hotspot functions account for a significant portion of the total cost, highlighting the primary performance bottlenecks of the target service. In our specific implementation, we demonstrate the use of the top 10 hotspot functions for calculating D_{JS}, though this selection is flexible. Users can adjust this parameter; increasing its value incorporates more functions, enhancing accuracy in identifying changes in hotspot functions, albeit at a higher computational cost.

[3] This threshold is adaptable based on user preferences. For our experiment, we set it to 5.

4.3 Initialization Stage

The essential purpose of both *Function Selective Pruning* and *Frequency Dynamic Adjustment* is to balance cost and accuracy, which requires knowing how they vary according to different parameters. Moreover, different services have different performance characteristics and access patterns, and the profiling cost and accuracy of profiling kernels themselves are also different. It is impossible to know in advance the effects of different parameters before profiling in practice. Therefore, to determine the optimal parameters for them, we need an initialization stage by performing a pre-profiling before large-scale profiling. Since our target is long-running large-scale distributed services, the cost of this one-time initialization process is usually acceptable compared to the cost of subsequent long-term monitoring.

For the pruning standards of *Function Selective Pruning*, there is a nearly functional relationship between the percentage of pruned threads, the time reduced, and the mean average percentage error(MAPE) caused for the target service, as shown in Fig. 5. We can fit this relationship through the results of pre-profiling and appropriate models. Therefore, given the acceptable maximum MAPE, we can calculate the maximum pruning percentage to achieve the maximum time reduction within the error range. This can be formalized as the following optimization problem:

$$\begin{aligned} \min_{p} \quad & T(p) \\ \text{s.t.} \quad & MAPE(p) \leq \epsilon \end{aligned} \tag{1}$$

where p is the pruning percentage, while ϵ denotes the maximum acceptable MAPE. The functions $T(p)$ and $MAPE(p)$ illustrate the fitted relationships between p and the corresponding time reduction, as well as the MAPE induced by p, respectively. Since $T(p)$ and $MAPE(p)$ are both monotonically increasing, this problem can be easily solved.

For the example Specjbb service, we choose linear model for $T(p)$, and logarithmic model for $MAPE(p)$ (MAPE of the top50 hotspot functions). The results are $T(p) = -1.0614p + 114.44$, fitting MAPE=4.75; and $MAPE(p) = 984.368 * log(-0.001p + 1.099)$, fitting MAPE=15.73. It should be noted that the MAPE of the logarithmic model before P99 is only 1.66, and at P99, the overall relative error increases due to the small absolute value of the error. This means that the predicted values are highly accurate in most feasible regions of this optimization problem. Given a threshold ϵ in the example, we can thus calculate $p \geq \frac{e^{\epsilon/984.368} - 1.099}{-0.001}$.

As for the determination of the threshold of Frequency Dynamic Adjustment Scheme, user can derive an image of JS divergence over time like Fig. 4 through pre-profiling. Through this image, user can easily see the migration of hot functions of the service, and thus set an appropriate JS divergence threshold for frequency adjustment. The general target is to make the small changes in the hotspot function of the service do not reach the threshold most of the time, while those drastic changes reach it.

5 Evaluation

5.1 Experiment Setup

We conduct experiments on a 13-node cluster to evaluate the effectiveness of function selective pruning and frequency dynamic adjustment strategies, along with the scalability of **Atys**. Specifically, we utilize SPECjbb2015 [39] as the Java benchmark and async-profiler [3] as the Java profiling kernel. Additionally, we employ the VGG16 network inference [38] for Python benchmarking, using py-spy [7] as the profiling kernel.

The Specjbb2015 runs in HBIR (High Bound Injection Rate) mode, indicating it seeks HBIR before exiting. The VGG workload is set up to employ the VGG16 network for repeated classification on the CIFAR-10 dataset [22]. All benchmark programs are deployed via Kubernetes [41], with **Atys** installed as a daemon on the target node. All tests are performed on identical worker nodes, as detailed in Table 1.

5.2 Baselines

Our advancements in profiling kernel efficiency enhance the kernels' optimizations. This indicates that the optimization techniques outlined in **Atys** can be applied to both single-machine and multi-machine profilers. Furthermore, due to the inherent performance disparities among different kernels, it would be inappropriate to evaluate distinct kernels using our optimization techniques. Consequently, our experiments concentrate on assessing the optimizations on the same kernel, rather than merely deploying the kernel from previous studies. We select the open-source multi-machine profiler pyroscope [33] as the baseline, which employs the same Java and Python profiling kernels as **Atys**.

Table 1. Configuration of worker nodes in the experiments

OS	Alibaba Cloud Linux release 2.1903 LTS
CPU	Intel(R) Xeon(R) Platinum 8269CY CPU @ 2.50GHz
Core Number	104
Memory Size	187GB

5.3 Effect of FSP Strategy

We employ the function selective pruning (FSP) strategy, as detailed in Sect. 4.1, on the stack traces gathered from a specific execution of Specjbb2015. We then analyze the mean absolute percentage error (MAPE) of the execution time for the top n most sampled functions, alongside the aggregation time under varying pruning thresholds, with n set to 5, 10, 20, and 50. Notably, the time to sort

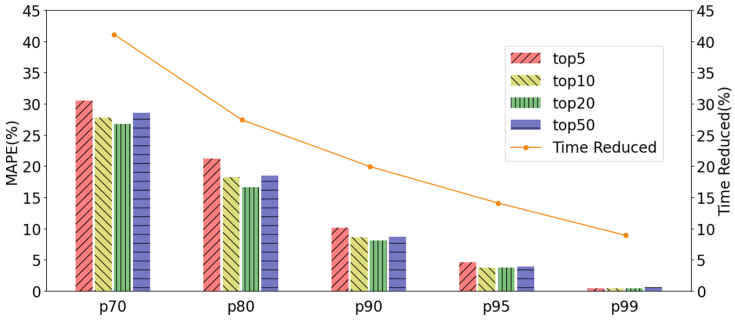
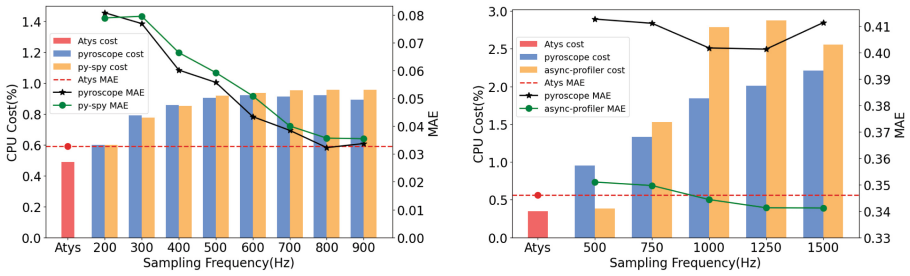


Fig. 5. MAPE of the top n hotspot functions and the percentage of time saved under various pruning configurations.



(a) VGG16 inference (b) Specjbb2015

Fig. 6. CPU cost and MAE with different sampling frequencies by **Atys**, single-machine profilers, and pyroscope respectively.

1,840 threads by sample count is minimal compared to the aggregation cost of hundreds of MBs of raw stack traces. Therefore, our primary focus is on the aggregation process's time cost for comparison.

The results are depicted in Fig. 5, showing that for the top 5, 10, 20, and 50 hotspot functions, the FSP strategy in **Atys** yields significant time savings, surpassing the marginal error incurred. Specifically, with the P99 pruning threshold, a mere 0.58% MAPE is introduced for the top 50 hotspot functions, while aggregation time decreases by 6.8%. Therefore, the P99 standard is established as the default pruning threshold in **Atys**.

5.4 Effect of FDA Scheme

To evaluate the effectiveness of the frequency dynamic adjustment (FDA) scheme, we initially select six time-consuming ($> 5\%$) business logic functions from Specjbb2015, as detailed below:

- AssociativityOfCategoryTransaction.execute
- SupermarketAudit.analyze
- SupermarketAudit.summarizeReceipt
- InStorePurchaseTransaction.doCheckout
- InStorePurchaseTransaction.doPurchase
- OnlinePurchaseTransaction.doCheckout

In addition, we select four functions essential for neural network inference from the VGG16 benchmark:

- forward (torch/nn/modules/linear.py)
- _conv_forward (torch/nn/modules/conv.py)
- _max_pool2d (torch/nn/functional.py)
- relu (torch/nn/functional.py)

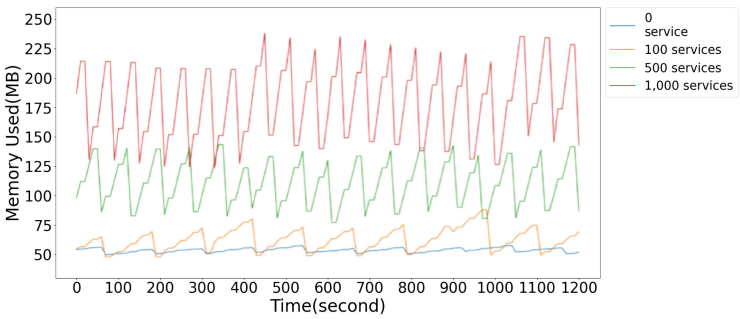
We have instrumented the benchmark code to accurately measure the CPU time consumed by each function. For the Java benchmark, we achieve this by invoking *threadMXBean.getCurrent ThreadCpuTime()* before and after the function's execution. In the case of the Python benchmark, we employ the *CProfile* [34] library. Furthermore, we extrapolate the CPU time based on profiling results. The percentage of time consumed by these hotspot functions is then converted into a distribution. Finally, we compute the difference between the distribution derived from profiling and the actual time consumption, expressed in terms of mean average error (MAE).

We calculate the MAE and CPU consumption of profiling across various sampling frequencies using different profilers. The parameters for the FDA scheme are set as follows: the JS divergence threshold $\theta_{specjbb} = 0.5$, $\theta_{vgg} = 0.05$, and the decay rate $\lambda = 0.8$.

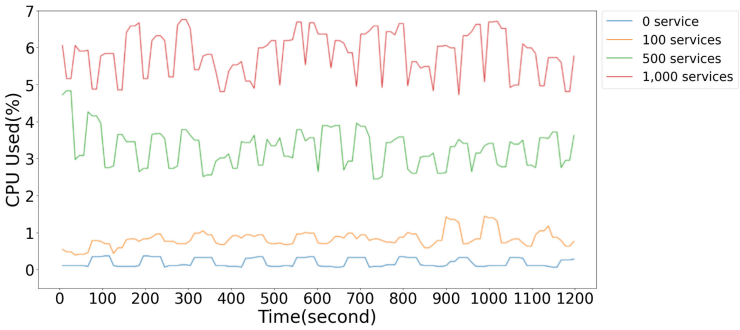
Several observations can be made from the results depicted in Fig. 6. Firstly, as the sampling frequency increases, sampling costs generally rise while sampling error decreases, confirming the trend discussed in Sect. 4. Secondly, in the Python benchmark, **Atys** consistently achieves lower error and costs than both py-spy and pyroscope across all tested frequencies using our FDA strategy. Its MAE approaches that of pyroscope at 800 Hz, with CPU utilization at only 53% of pyroscope's. Thirdly, in the Java benchmark, **Atys** has an average sampling frequency of 357.8 Hz, with CPU usage comparable to async-profiler at 5,00 Hz. However, it demonstrates a significantly lower MAE than async-profiler at this frequency, nearing the performance at 1,000 Hz while incurring only 12.4% of the sampling cost. Additionally, **Atys**' MAE remains notably lower than that of pyroscope across all frequencies. These results demonstrate the effectiveness of **Atys**'s FDA strategy, achieving high accuracy while significantly minimizing performance overhead.

5.5 System Scalability

When deploying **Atys** in production environments, the Prometheus server must track and store extensive metrics, potentially resulting in significant strain. Thus, evaluating the server's capacity to manage increased workloads during simultaneous monitoring of multiple instances is crucial. We conduct experiments in which **Atys** concurrently monitors 0, 100, 500, and 1,000 instances of web services over a period of 1,200 s. For each instance, **Atys** is configured to collect the top 10 hotspot functions. The memory consumption of the Prometheus service is recorded and presented in Fig. 7a.



(a) Memory usage



(b) CPU usage

Fig. 7. Memory and CPU usage of the Prometheus server with varying numbers of monitoring instances.

Initially, with no active monitoring, the Prometheus server's memory usage is approximately 50MB. As the number of monitored services increases, memory consumption rises accordingly. However, even when monitoring up to 1,000 service instances, the memory footprint remains below 250MB, which is negligible compared to the worker node's total memory of 184GB. The CPU usage of the Prometheus service is also monitored, as illustrated in Fig. 7b. At its peak work-

load, while monitoring 1,000 service instances simultaneously, the server utilizes only about 6% of the host node's total CPU.

These results confirm that the Prometheus server has ample capacity to manage numerous sampling tasks without strain. Furthermore, this demonstration of the server's resource efficiency underscores the suitability of **Atys** for meeting the demands of extensive profiling tasks in production large-scale microservice environments.

6 Conclusion

This work presents **Atys**, a highly efficient profiling framework tailored for the complexities of large-scale services in production environments. To enhance efficiency, **Atys** implements a function selective pruning strategy and a frequency dynamic adjustment scheme to minimize profiling overhead. The performance and effectiveness of **Atys** have been validated through a prototype, which underwent extensive experiments, showcasing its potential for practical deployment in production environments.

References

1. Adhianto, L., et al.: HPCTOOLKIT: tools for performance analysis of optimized parallel programs. Concurr. Comput. Pract. Exp. **22**(6), 685–701 (2010). https://doi.org/10.1002/CPE.1553
2. Amazon Inc.: Amazon shop (2023). https://www.amazon.com
3. Pangin, A., et al.: async-profiler (2017). https://github.com/async-profiler/async-profiler
4. Ball, T., Larus, J.R.: Efficient path profiling. In: Proceedings of the 29th Annual ACM/IEEE International Symposium on Microarchitecture, pp. 46–57. MICRO 29, IEEE Computer Society, USA (1996). https://doi.org/10.1109/MICRO.1996.566449
5. Barroso, L.A., Hölzle, U., Ranganathan, P.: The Datacenter as a Computer: Designing Warehouse-Scale Machines, Third Edition. Synthesis Lectures on Computer Architecture, Morgan & Claypool Publishers (2018).https://doi.org/10.2200/S00874ED3V01Y201809CAC046
6. Becker, D., Geimer, M., Rabenseifner, R., Wolf, F.: Synchronizing the timestamps of concurrent events in traces of hybrid MPI/OpenMP applications. In: 2010 IEEE International Conference on Cluster Computing, pp. 38–47 (2010). https://doi.org/10.1109/CLUSTER.2010.13
7. Frederickson, B., et al.: py-spy (2023). https://github.com/benfred/py-spy
8. Benavides, Z., Vora, K., Gupta, R.: DProf: distributed profiler with strong guarantees. Proc. ACM Program. Lang. **3**(OOPSLA), 1–24 (2019). https://doi.org/10.1145/3360582
9. Burchell, H., Larose, O., Marr, S.: Towards realistic results for instrumentation-based profilers for JIT-compiled systems. In: Proceedings of the 21st ACM SIGPLAN International Conference on Managed Programming Languages and Runtimes, MPLR 2024, Vienna, Austria, 19 September 2024, pp. 82–89. MPLR 2024, Association for Computing Machinery, New York, NY, USA (2024). https://doi.org/10.1145/3679007.3685058

10. Carlson, T.E., Nilakantan, S., Hempstead, M., Heirman, W.: Epoch profiles: microarchitecture-based application analysis and optimization. IEEE Comput. Archit. Lett. **14**(1), 30–33 (2015). https://doi.org/10.1109/LCA.2014.2329873
11. Cho, H.K., Moseley, T., Hank, R., Bruening, D., Mahlke, S.: Instant profiling: instrumentation sampling for profiling datacenter applications. In: Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 1–10 (2013). https://doi.org/10.1109/CGO.2013.6494982
12. Ehlers, J., van Hoorn, A., Waller, J., Hasselbring, W.: Self-adaptive software system monitoring for performance anomaly localization. In: Proceedings of the 8th ACM International Conference on Autonomic Computing, pp. 197–200. ICAC '11, Association for Computing Machinery, New York, NY, USA (2011). https://doi.org/10.1145/1998582.1998628
13. Fan, Q., Wang, Q.: Performance comparison of web servers with different architectures: a case study using high concurrency workload. In: 2015 Third IEEE Workshop on Hot Topics in Web Systems and Technologies (HotWeb), pp. 37–42 (2015). https://doi.org/10.1109/HotWeb.2015.11
14. EJ-technologies GmbH: The award-winning all-in-one java profiler (2023). https://www.ej-technologies.com/products/jprofiler/overview.html
15. Google Inc.: Google cloud profiler (2023). https://cloud.google.com/profiler/
16. Google Inc.: Google search (2023). https://www.google.com
17. Google Inc.: Package pprof (2023). https://golang.google.cn/pkg/runtime/pprof/
18. Graham, S.L., Kessler, P.B., Mckusick, M.K.: Gprof: a call graph execution profiler. In: Proceedings of the 1982 SIGPLAN Symposium on Compiler Construction, pp. 120–126. SIGPLAN '82, Association for Computing Machinery, New York, NY, USA (1982). https://doi.org/10.1145/800230.806987
19. Hua, Q., Yang, D., Qian, S., Hu, H., Cao, J., Xue, G.: Kae-informer: a knowledge auto-embedding informer for forecasting long-term workloads of microservices. In: Proceedings of the ACM Web Conference 2023, pp. 1551–1561. WWW '23, Association for Computing Machinery, New York, NY, USA (2023). https://doi.org/10.1145/3543507.3583288
20. Huye, D., Shkuro, Y., Sambasivan, R.R.: Lifting the veil on Meta's microservice architecture: analyses of topology and request workflows. In: 2023 USENIX Annual Technical Conference (USENIX ATC 23), pp. 419–432. USENIX Association, Boston, MA (2023). https://www.usenix.org/conference/atc23/presentation/huye
21. Intel Corporation: Intel vtune (2023). https://www.intel.com/content/www/us/en/docs/vtune-profiler/user-guide/2023-0/overview.html
22. Krizhevsky, A.: Learning multiple layers of features from tiny images. Technical report, University of Toronto (2009). https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf
23. Lin, J.: Divergence measures based on the shannon entropy. IEEE Trans. Inf. Theory **37**(1), 145–151 (1991). https://doi.org/10.1109/18.61115
24. de Melo, A.C.: The new Linux 'perf' tools. Technical report, Red Hat Inc. (2010). http://oldvger.kernel.org/~acme/perf/lk2010-perf-paper.pdf
25. Merkel, D.: Docker: lightweight linux containers for consistent development and deployment. Linux J. **2014**, 2 (2014). https://api.semanticscholar.org/CorpusID:62479797
26. Mytkowicz, T., Sweeney, P.F., Hauswirth, M., Diwan, A.: Time interpolation: so many metrics, so few registers. In: 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007), pp. 286–300 (2007). https://doi.org/10.1109/MICRO.2007.27

27. OpenJS Foundation and Node.js contributors: Easy profiling for node.js applications (2023). https://nodejs.org/en/docs/guides/simple-profiling
28. Optimyze Inc.: Prodfiler (2021). https://prodfiler.com
29. Oracle Corporation: Package java.lang.instrument (2010). https://docs.oracle.com/javase/1.5.0/docs/api/java/lang/instrument/package-summary.html
30. Oracle Corporation: JEP 435: asynchronous stack trace VM API (2023). https://openjdk.org/jeps/435
31. Prometheus Authors: Prometheus metric types (2023). https://prometheus.io/docs/concepts/metric_types/
32. Prometheus Authors: Prometheus system (2023). https://prometheus.io/docs/introduction/overview/
33. Pyroscope, Inc: pyroscope (2023). https://pyroscope.io/
34. Python Software Foundation: The python profilers (2023). https://docs.python.org/3/library/profile.html#module-cProfile
35. Ren, G., Tune, E., Moseley, T., Shi, Y., Rus, S., Hundt, R.: Google-wide profiling: a continuous profiling infrastructure for data centers. IEEE Micro **30**(4), 65–79 (2010). https://doi.org/10.1109/MM.2010.68
36. Rice University: Hpctoolkit (2023). http://hpctoolkit.org
37. Silicon Graphics Inc., Aconex, Red Hat: Performance co-pilot (2023). https://pcp.io
38. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. In: Bengio, Y., LeCun, Y. (eds.) 3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, 7–9 May 2015, Conference Track Proceedings (2015)
39. Standard Performance Evaluation Corporation: Specjbb2015 (2023). https://www.spec.org/jbb2015/
40. Su, P., Jiao, S., Chabbi, M., Liu, X.: Pinpointing performance inefficiencies via lightweight variance profiling. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–19. SC '19, Association for Computing Machinery, New York, NY, USA (2019). https://doi.org/10.1145/3295500.3356167
41. The Kubernetes Authors: Production-grade container orchestration (2024). https://kubernetes.io
42. Xu, H., Wang, Q., Song, S., John, L.K., Liu, X.: Can we trust profiling results?: understanding and fixing the inaccuracy in modern profilers. In: Eigenmann, R., Ding, C., McKee, S.A. (eds.) Proceedings of the ACM International Conference on Supercomputing, ICS 2019, Phoenix, AZ, USA, 26–28 June 2019, pp. 284–295. ACM (2019). https://doi.org/10.1145/3330345.3330371
43. Yang, S., Jeong, J., Scholz, B., Burgstaller, B.: Cloudprofiler: TSC-based internode profiling and high-throughput data ingestion for cloud streaming workloads (2023)
44. Yi, J., Dong, B., Dong, M., Chen, H.: On the precision of precise event based sampling. In: Kim, T., Lee, P.P.C. (eds.) APSys '20: 11th ACM SIGOPS Asia-Pacific Workshop on Systems, Tsukuba, Japan, 24–25 August 2020, pp. 98–105. ACM (2020). https://doi.org/10.1145/3409963.3410490
45. Zabbix LLC.: The all-in-one, open-source solution that lets you monitor anything (2023). https://www.zabbix.com

Path Optimization of USV Based on Stochastic Schemata Exploiter with New Exploitation Strategy

Jianshan Liang[1] and Yi Zuo[1,2(✉)]

[1] Collaborative Innovation Center for Transport Studies, Dalian Maritime University, Dalian 116000, China
zuo@dlmu.edu.cn
[2] Navigation College, Dalian Maritime University, Dalian 116000, China

Abstract. In recent years, the rapid development of unmanned surface vessel (USV) navigation technology has driven a surge of interest in path planning algorithms. As a core technique in this field, traditional evolutionary algorithms have been widely applied to solve USV path planning problems under multiple constraints. However, these algorithms commonly encounter limitations such as low search efficiency and slow convergence. Based on the latest advances in path optimization, this paper proposes a novel stochastic schemata exploiter (SSE) algorithm. SSE adopts the framework of genetic algorithms and uses integer encoding to map feasible paths to chromosomes. It breaks through the traditional selection–crossover model of subpopulation generation by extracting common schemata through semi-ordered permutation and combination strategies to guide offspring evolution. Specifically, SSE integrates schema theory into the iterative process, prioritizing the retention of high-fitness individuals and extracting advantageous schemata. A bounded mutation operation is further employed to enhance search capability, enabling both strong global exploration and rapid convergence. Simulation experiments under multiple constraints—including path length, navigation safety, and path smoothness—demonstrate the effectiveness and superiority of the SSE algorithm. Comparative experiments with genetic algorithm (GA), ant colony optimization (ACO), and the Q-learning algorithm show that SSE achieves significantly better overall performance in path planning tasks.

Keywords: evolutionary algorithm · path optimization · unmanned surface vessel · schema theory · integer encoding

1 Introduction

With the advancement of intelligent maritime technologies, unmanned surface vessels (USV) have been increasingly deployed in a wide range of mission-critical scenarios, including deep-sea exploration, disaster response, and maritime patrol [1]. These applications impose stringent requirements on autonomous navigation, particularly in terms of real-time decision-making and robust path planning [2]. For instance, in ocean monitoring tasks, the accuracy of the USV's trajectory directly determines the spatial coverage

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 387–399, 2026.
https://doi.org/10.1007/978-981-95-8405-5_21

of environmental data [3]. In emergency response scenarios, timely and safe arrival under harsh sea conditions is crucial to mission success [4]. In military operations, path concealment and anti-interference capability are essential for avoiding detection and achieving strategic objectives [5]. Therefore, maritime path planning has become not only a foundational element of autonomous navigation systems but also a pivotal technology linking intelligent equipment performance to mission effectiveness [6]. The development of planning algorithms that integrate global exploration with rapid convergence is imperative for the future deployment of USVs in complex maritime environments [7].

To address the challenges of USV path planning, a variety of classical algorithms have been proposed [8]. The rapidly-exploring random tree (RRT) algorithm is well-regarded for its high efficiency in exploring complex spaces and is widely used in dynamic environments. However, its outputs typically suffer from discontinuity and lack smoothness, necessitating post-processing [9]. genetic algorithms (GAs), inspired by the process of natural selection, are naturally suited for global search and multi-objective optimization. Nevertheless, GAs often exhibit premature convergence, resulting in limited solution precision [10]. ant colony optimization (ACO) mimics the foraging behavior of ants and is effective in obstacle-rich environments through its positive feedback mechanism [11]. Yet, ACO may suffer from slow convergence and limited diversity in solution sets [12]. Other conventional approaches, such as artificial potential fields, grid-based A* search, and cell decomposition techniques, demonstrate stable performance in structured environments but generally encounter issues such as local optima, high computational overhead, or compromised path quality [13]. A common shortcoming across these methods is their limited ability to simultaneously optimize multiple conflicting objectives, such as minimizing path length, reducing energy consumption, and improving trajectory smoothness—particularly under the dynamic and uncertain conditions typical of maritime domains [14].

To address these limitations, this study proposes a novel evolutionary algorithm based on schema theory, termed the stochastic schemata exploiter (SSE) algorithm. By integrating schemata extraction, stochastic replacement, and an adaptive balance between exploration and exploitation, the SSE algorithm exhibits strong structural recognition and optimization capabilities. The algorithm employs a semi-ordered mechanism to form structured subpopulations and extracts common schemata from high-fitness individuals to guide new solution generation. To preserve population diversity, mutation is applied conditionally to prevent premature convergence. Unlike conventional GA implementations, SSE performs mutation at the schema level rather than directly on individuals. Simulation results of 100 times demonstrate that, compared to GA, ACO, and Q-learning, the proposed SSE achieves superior performance. Specifically, in path length, SSE (14.60 km) is shortest than GA (16.09 km), ACO (16.05 km) and Q-learning(15.19 km). In algorithm execution time, it only takes 5.31 s, which is much less than ACO (13.77 s) and Q-learning (86.65 s), and slightly lower than GA (2.09 s). These experimental results indicate significant advantages of SSE in path planning of USV.

The remainder of this paper is organized as follows. Section 2 introduces the USV path planning environment modeling and the principles of the SSE in detail. Section 3 presents the simulation results and a comparative analysis of the performance of four

algorithms applied to USV path planning. Section 4 summarizes conclusions of this paper.

2 Preparation of USV Path Planning and SSE Algorithm

This section introduces the environment of electronic chart, and presents environment modeling, the path encoding method and detail of SSE algorithm.

2.1 Environment Modeling

Raster Map Modeling of Electronic Chart

In USV path planning, the navigation environment is typically derived from electronic nautical charts, which provide reliable information about the target sea area. To facilitate computational modeling, the maritime environment is discretized into a raster map that transforms continuous space into a grid-based representation. This study adopts a raster-based environmental model, which is widely used in evolutionary algorithm-based path planning due to its simplicity and efficiency in representing navigable space and obstacles. In the raster map, the target area is discretized into uniform, non-overlapping grid cells, each assigned a specific color based on its environmental characteristics. Specifically, black grid cells denote non-navigable sea regions—such as islands, reefs, or buffer zones—while white grid cells indicate navigable, obstacle-free regions. As illustrated in Fig. 1, this representation allows the USV to recognize feasible routes and avoid collision zones by evaluating the state of each cell along the path.

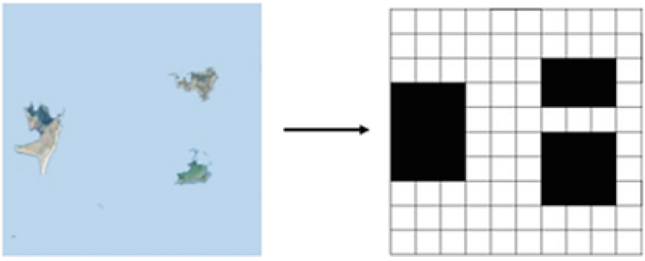


Fig. 1. Environmental modeling example based on raster map

Within this raster-based modeling framework, two common methods are used for grid cell marking: the Cartesian coordinate method and the serial number method, which maintain a bijective mapping for flexible conversion. The serial number method assigns unique indices to grid cells following a strict left-to-right, bottom-to-top, ordering, as visualized in Fig. 2. This sequential labeling ensures systematic path encoding by converting coordinate-based waypoints into linear indices, which simplifies chromosome representation in evolutionary algorithms.

Path Encoding

As shown in Fig. 3, path encoding is performed based on the environmental modeling

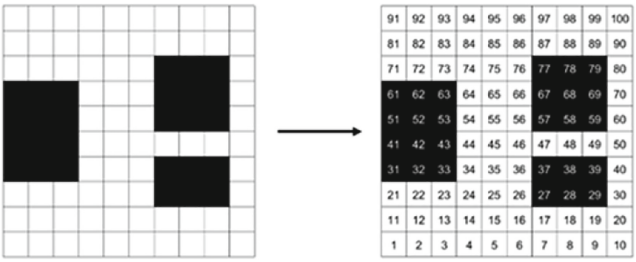


Fig. 2. Mapping raster to coordinates

and encoding described in the previous section. The start and end points of each path are fixed, with each path node represented by an integer P_i and coordinates (x_i, y_i). A path consists of two or more nodes, and the number of nodes is variable.

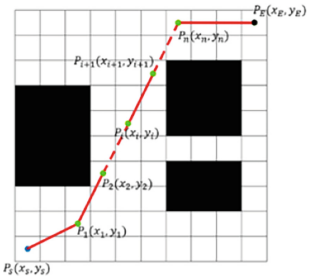


Fig. 3. Graphical illustration of path encoding.

Through integer encoding of the map, all path points can be effectively converted from raw coordinate representation to integer representation. For example, the coordinate representation of the path shown in Fig. 3 is $Path = [(x_s, y_s), (x_1, y_1), (x_2, y_2), \ldots, (x_i, y_i), (x_{i+1}, y_{i+1}), \ldots, (x_e, y_e)]$, it can be transformed into integer representation as $Path = (P_S, P_1, \ldots, P_i, P_{i+1}, \ldots, P_n, P_E)$. Eq. 1 shows the method for converting the Cartesian coordinates of a path point into the corresponding raster number,

$$P_i = x_i + C(y_i - 1) \tag{1}$$

where P_i represents the unique integer number of the raster cell in the grid map, x_i and y_i are the column and row indices of the point in the Cartesian coordinate system, C is the number of raster map's column.

Fitness Function

As a genetic algorithm-based method, the SSE algorithm employs a fitness function to evaluate the quality of individual paths. In this study, the fitness function is designed to comprehensively consider both path length and path smoothness, according to the path length objective and path smooth objective, its function specified as follows:

$$fitness(P) = \left[a * length(P) + b * smoothness(P)\right]^{-1} \tag{2}$$

where P means the *Path* from the starting point S to the endpoint E, *length* denotes the total distance of the *Path*, and *smoothness* represents the degree of *Path* bending and quantify the feasibility of a path, a and b are weight coefficients tuning the priority between *length* and *smoothness*. This formulation ensures that shorter paths and smoother trajectories contribute higher fitness values, aligning with the objectives of minimizing energy consumption and navigation risks.

2.2 Stochastic Schemata Exploiter

This section introduces the Stochastic Schemata Exploiter (SSE) algorithm designed for addressing USV path planning problems. It begins with an outline of the algorithm's framework, followed by a detailed explanation of the subpopulation generated based on fitness rank, and schema generate. Finally, a detailed explanation of stochastic replacement and schema mutation are presented.

The Framework of SSE

The SSE algorithm is a metaheuristic optimization method that represents potential solutions using gene-based encodings inspired by natural evolution. In the context of genetic algorithms, a schema refers to a common pattern shared across multiple individuals in a population, representing a partial solution or building block that contributes to overall performance. By preserving and combining such schemata, evolutionary algorithms can effectively exploit promising search directions.

In the SSE framework, schema theory is explicitly incorporated to guide the evolutionary process. The algorithm extracts common schemata from subpopulations, preserves them through elite-oriented selection, and introduces stochastic replacement or mutation when all individuals converge to the same schema. This mechanism enables SSE to maintain a balance between exploiting high-quality patterns and exploring new search regions, thereby mitigating premature convergence while improving solution diversity. As illustrated in Fig. 4, the main progress of SSE for USV path planning include the following steps:

Step1: Setting parameters for the SSE.

Step2: The initial population POP_1 is constructed by generating diverse individuals, either randomly or based on heuristic rules.

Step3: The newly generated individuals in POP_1 are decoded into complete paths and evaluated based on multiple performance metrics, such as path length and smoothness.

Step4: Check the termination condition: if the stopping condition is satisfied (such as the number of iteration), outputting the best solution of the latest generation, and stop circulating; else, turn to step 5.

Step5: High-fitness individuals are ranked using semi-ordered permutation based on their fitness values. These elites are then recombined using a binary tree structure to form subpopulations; after semi-order permutation and combination, subpopulations were generated.

Step6: For each subpopulation, a common schema is derived by preserving consistent gene positions according to all individuals, while marking inconsistent positions with'*';

Step7: The positions with'*' in the schema are filled with valid gene values selected via a stochastic sampling process to enhance exploration, then turn to step 3.

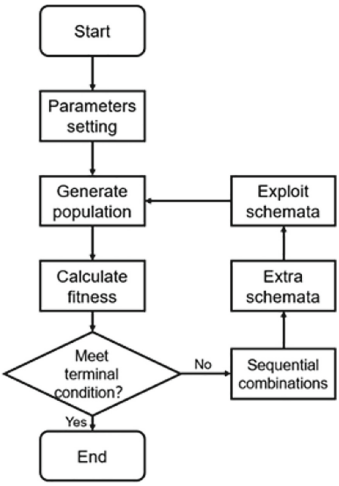


Fig. 4. Flowchart of SSE.

Initialization Strategies

In the SSE algorithm, each feasible grid cell on the raster map is assigned a unique integer, and paths are encoded as chromosomes. The start and end points are fixed as the first and last genes, while the remaining genes are randomly selected from non-obstacle cells. The chromosome length is set according to the map size and task complexity. To ensure path feasibility, each chromosome undergoes a continuity check. Disconnected segments are repaired by interpolation or node replacement; if repair fails, the chromosome is regenerated. This initialization strategy ensures that all individuals are collision-free and diverse, providing a solid foundation for the SSE algorithm's evolutionary process, the Fig. 5 illustrated initialized population POP_1 .

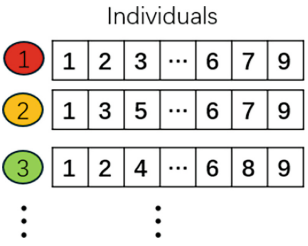


Fig. 5. Initialization of chromosomes.

Subpopulation Generates Procedure

In this section, a fitness-ranked combination strategy is employed to select individuals from the previous generation population POP_{i-1} and construct the corresponding subpopulations for generation G_i.

The elite-oriented selection operation is a key component of the SSE algorithm. Its objective is to select high-fitness individuals from the current population as elite candidates for subpopulation generation. First, all chromosomes in the POP_{i-1} are sorted in descending order according to their fitness values. Then, the top n individuals are selected and sequentially assigned to a new population set P_n, denoted as $P_n = \{a_1, a_2, \ldots, a_n\}$.

Based on the selected individuals in population P_n, a combination operation is performed. A binary tree structure is used to arrange individuals according to their labels, as illustrated in the Fig. 6.

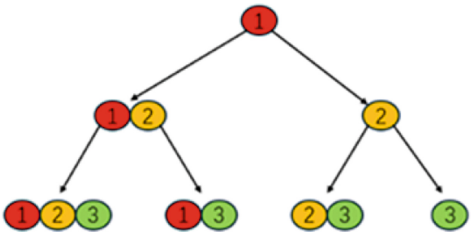


Fig. 6. Semi-ordered permutation and combination.

In this progress, each parent node can generate two child nodes: The left child is generated by adding the next individual from the sorted list into the parent subpopulation. The right child is generated by replacing the individual with the lowest fitness in the parent subpopulation with the next individual.

Through this iterative combination process, subpopulation nodes can be generated, each representing a new subpopulation. This procedure lays the foundation for the subsequent step of common schema generation.

Schema Generation

Schema is an important concept from biological evolution adopted in the proposed SSE algorithm, used to represent the similarity or structural segmentation among chromosomes. SSE uses an encoding scheme of {'integer', '*'}, where '*' denotes a gene that can be replaced by any valid grid code along a feasible path. The schema generation process is illustrated in the Fig. 7,

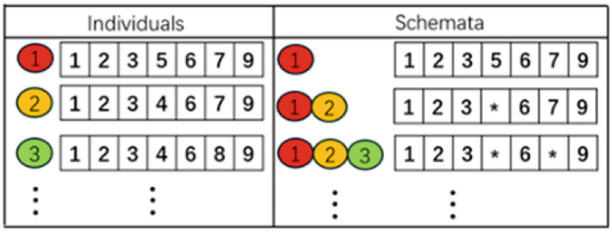


Fig. 7. Schemata generation.

in this progress, a common schema for each subpopulation can be derived by comparing identical gene positions across chromosomes within the same subpopulation. If the subpopulation contains only one chromosome, that chromosome is directly retained as the schema for the next generation. If the subpopulation contains multiple chromosomes, the same gene positions among all chromosomes are extracted as the common schema, different gene positions use '*' to represent. The resulting schema is then supplemented with valid gird number to ensure path continuity. The filled-in schema is incorporated into the population POP_i, serving as a source of elite individuals for the next generation.

Schemata Mutation Procedure

Through the above steps, after multiple iterations, a situation may arise in which all individuals within the subpopulations at each node of the binary tree become identical. If the same iterative logic continues to be applied under this condition, the algorithm will lose its exploration capability in subsequent iterations, leading to premature convergence and stagnation in a local optimum. Therefore, for this general case but special situation, mutation operation is required to recall a stochastic replacement process to randomly modify one position in common schema. To reduce the potential premature convergence, this operation employs a more conservative mutation strategy, noted as follows:

- Random Schema Perturbation: Select an intermediate path node P_i (excluding start point S and end point E) from the rigid common schema and replace it with a wildcard '*'.
- Boundary Definition: Derive the coordinates of the preceding and succeeding nodes $P_{i-1} = (X_{i-1}, Y_{i-1})$ and $P_{i+1} = (X_{i+1}, Y_{i+1})$ using coordinate transformation.
- Constrained Replacement: The replacement node $P_n = (X_n, Y_n)$ must satisfy:

$$\begin{cases} X_n \in \left[X_{i-1}, X_{i+1}\right] \\ Y_n \in \left[Y_{i-1}, Y_{i+1}\right] \\ P_n \cap \left[Obstacle, P_{i-1}, P_i, P_{i+1}\right] = \varnothing \end{cases} \tag{3}$$

when newly generated schema does not represent a feasible path, repeat above operation.

3 Experiment and Simulation

This section presents the map configuration, algorithm parameter settings, and simulation results.

3.1 Setting of Map and Parameters

In this section, we evaluate the performance of the proposed SSE algorithm by implementing SSE and several baseline algorithms in MATLAB R2018a. The experiments were conducted on a system equipped with an Intel Core i5 2.3 GHz processor and 16 GB of RAM. Based on the raster map modeling and encoding method described in Sect. 2.2, simulation experiments for USV path planning were carried out using a localized rasterized map of the target sea area, as shown in Fig. 8. The size of the map was standardized to 50 × 50, 75 × 75, and 100 × 100 grid.

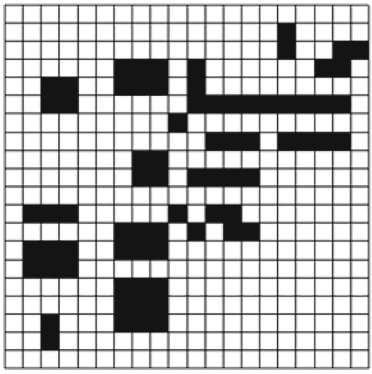


Fig. 8. Raster map modeling and coding for experimental sea area.

In the experiments, we compared the proposed SSE algorithm with GA, ACO, and the Q-learning in terms of planned path length. Table 1 lists the parameter settings for each algorithm, including the common parameter of maximum iterations (MI). For ACO, the parameters include the number of ants, pheromone factor, heuristic factor, and pheromone evaporation coefficient for population size, crossover rate, and mutation rate. Q-learning contains learning rate, discount factor, and exploration rate. For SSE, only population size and mutation rate need to be specified.

Table 1. Setting of main parameters for each method.

Parameter	ACO	GA	Q-learning	SSE
Number of maximum iterations (MI)	100	100	100	100
Population size (N_p)	30/200[1]	30	–	31
Cross rate (P_c)	–	0.8	–	–
Mutation rate (P_m)	–	0.2	–	0.2
Pheromone factor α)	5	–	–	–
Heuristic factor (β)	2.5	–	–	–
Pheromone evaporation coefficient (ρ)	0.01	–	–	–
Learning rate (ω)	–	–	0.8	–
Discount factor (γ)	–	–	0.9	–
Exploration rate (ϵ)	–	–	0.5	–

[a] 30/200 denotes the population size of GA and ant number of ACO.

Furthermore, a sensitivity analysis was conducted to assess the robustness of the proposed SSE algorithm. Results indicate that the SSE algorithm exhibits exceptional stability; owing to the unique characteristics of its structural design, variations in the initial population size exert a negligible influence on its iterative process. Experimental investigations demonstrate that as long as the number of initialized valid paths exceeds

2, the algorithm can rapidly converge to yield a suboptimal path in subsequent iterations. Additionally, the number of offspring generated by the SSE algorithm is primarily governed by the binary tree branching algebra n. Further parametric experiments reveal that when n is set to 5, a favorable trade-off between computational time and convergence rate can be achieved.

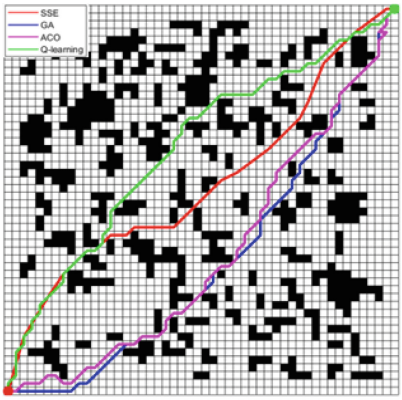
3.2 Experimental Analysis

Experimental Comparison of Multiple Algorithms

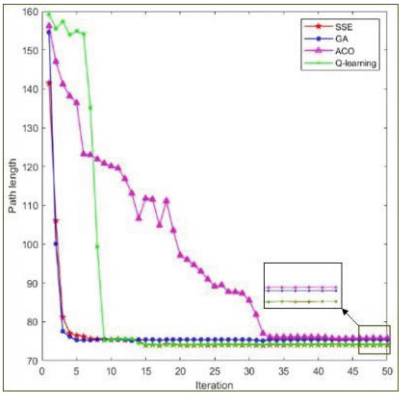
As can be seen from the comparative experimental results of four algorithms (SSE, ACO, Q-learning, and GA) on the three groups of algorithm trajectory maps (50 × 50, 75 × 75, and 100 × 100) presented in Fig. 9, in each group of map experiments, all algorithms successfully found collision-free paths after 100 iterations. It can be observed from the three groups of trajectory maps that the red route representing the SSE algorithm is, on the whole, the closest to the line connecting the red start point and the green end point. Furthermore, combined with the convergence curves of path length, SSE shows significant advantages in the shortest path experiments on these three maps: it converges faster than other algorithms, and the shortest path length is also significantly better than that of other algorithms.

Table 2 data indicate that in the 100 × 100 experimental map, take the four algorithms into account together, the SSE algorithm can plan shortest paths within a reasonable time frame. This demonstrates the algorithm's effectiveness.

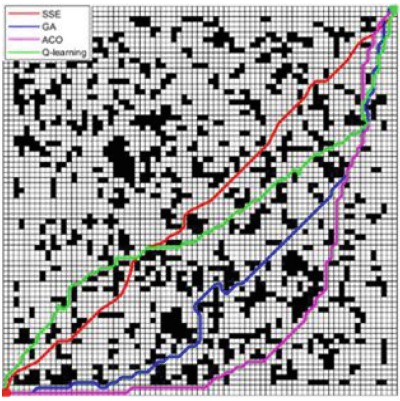
Analysis of Fig. 10 indicates that, after conducting one hundred experiments in the 100 × 100 experimental map, the SSE algorithm demonstrated outstanding overall performance. In terms of path length, SSE achieved a mean value of 14.60 km, significantly outperforming ACO (16.09 km) and GA (16.05 km), and slightly surpassing Q-learning (15.19 km). The short error bars further indicate a low standard deviation in path length, highlighting the algorithm's strong result stability. Regarding execution time, SSE recorded an average of only 5.31 s, which is substantially lower than that of Q-learning (86.65 s) and ACO (13.77 s), and only slightly higher than GA (2.09 s). Moreover, the short error bars suggest minimal fluctuation in runtime. Overall, SSE achieves a favorable balance between path optimality and computational efficiency, offering superior practical advantages over GA, ACO, and Q-learning in USV path optimization tasks.



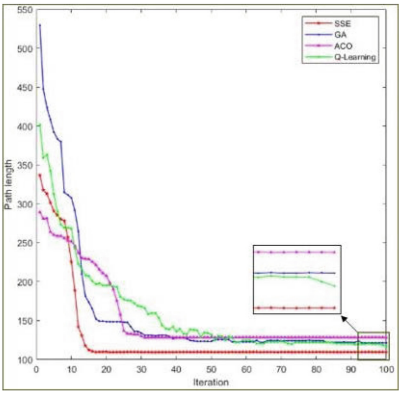
(a) 50*50 map



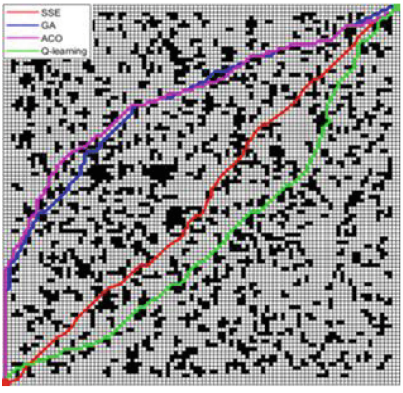
(b) Convergence curve of 50*50 map



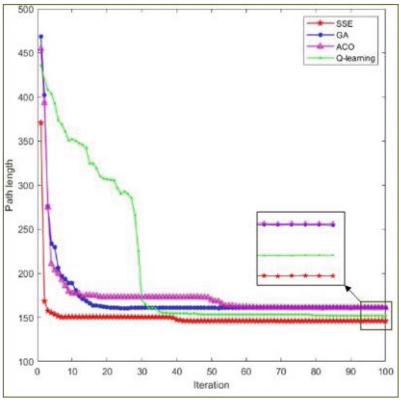
(c) 75*75 map



(d) Convergence curve of 75*75 map



(e) 100*100 map

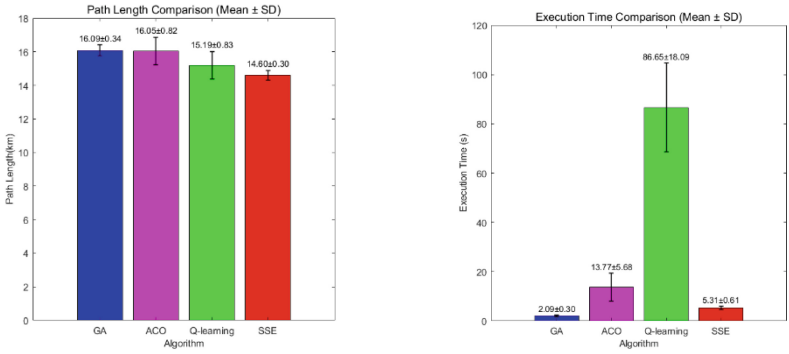


(f) Convergence curve of 100*100 map

Fig. 9. Comparisons of algorithms' path planning results.

Table 2. Algorithm Performance with 100 Runs (100*100 case).

Method	Path length (km)	Runtime (sec)	Length SD (km)	Runtime SD (sec)
GA	16.09	2.09	0.3353	0.3048
ACO	16.05	13.77	0.8202	5.6792
Q-Learning	15.19	86.65	0.8285	18.0938
SSE	14.60	5.31	0.3017	0.6103



(a) Path length (b) Program runtime

Fig. 10. Algorithm performance with 100 runs (100*100 case).

4 Conclusion

This paper proposes a novel EA of SSE based on schema theory, which incorporates the discovery of biological evolution, and utilizes swarm behavior in generation iterations. One of the core concepts in SSE is to include common schemata for dividing individuals from population into subpopulations, which contain individuals with similar genes, and preserves these excellent genes via common schema. Another core concept is to involve stochastic replacement, which can generate new individuals in wide and diverse search space. During exploitation operation, SSE remains individuals with high fitness, which can accelerate the population's convergence to the optimal solution. During exploration operation, SSE replaces individuals via stochastic process, which can increase population diversity. Besides exploiter strategy, SSE also resets mutation operation on common schema instead of individual, which can efficiently prevent premature convergence.

In quantitative comparisons, SSE and other EAs are applied to solving the path planning problem of USV as experiment and simulation. Among all methods, SSE reveals faster convergence, best solutions, and shorter run time. Additionally, SSE requires fewer parameters than ACO and Q-learning (see details in Table 1), to enhance the environmental adaptability and generalization of the algorithm.

Conflict of Interest The authors have no competing interests to declare that are relevant to the content of this article.

Acknowledgments. This work was supported in part by the National Natural Science Foundation of China (grant no. 52131101), and the Science and Technology Fund for Distinguished Young Scholars of Dalian (grant no. 2021RJ08).

References

1. Yu, C., Zhang, Z., Liu, X., Yang, C., Wang, Y., Liang, X.: Verification method for formation control of USVs based on virtual-real integration. Ocean Eng. **335**, 121677 (2025). https://doi.org/10.1016/j.oceaneng.2025.121677
2. Yuan, P., Zhang, Z., Li, Y., Cui, J.: Leader-follower control and APF for multi-USV coordination and obstacle avoidance. Ocean Eng. **313**., part 2, 119487 (2024). https://doi.org/10.1016/j.oceaneng.2024.119487
3. Dong, Z., Wang, B., Tan, F., Zhou, W., Liu, Y.: Online parameter identification and real-time manoeuvring prediction for a water-jet USV based on weighted multi-innovation prediction error method integrated with dynamic window strategy. Appl. Ocean Res. **153**, 104260 (2024). https://doi.org/10.1016/j.apor.2024.104260
4. Zhao, L., Bai, Y.: Joint-optimized coverage path planning framework for USV-assisted offshore bathymetric mapping: from theory to practice. Knowl.-Based Syst. **304**, 112449 (2024). https://doi.org/10.1016/j.knosys.2024.112449
5. Zhang, G., Xu, Y., Li, J., Yin, Y.: Robust adaptive event-triggered control for unmanned surface vessel–unmanned aerial vehicle: application to sea–air synchronized lawn mowing search operation. Comput. Electr. Eng. **120, part B**, 109754 (2024). https://doi.org/10.1016/j.compeleceng.2024.109754
6. Chang, Z.-J., Yao, X.-Y., Park, J.H.: Fixed-time formation tracking for unmanned surface vehicles: a multi-layer neural networks approach. Neurocomputing. **600**, 128220 (2024). https://doi.org/10.1016/j.neucom.2024.128220
7. Zhi, L., Zuo, Y.: Collaborative path planning of multiple AUVs based on adaptive multi-population PSO[J]. J. Mar. Sci. Eng. **12**(2), 223 (2024)
8. Luo, J., Su, Y.: Path planning for multi-USV target coverage in complex environments. Ocean Eng. **312, part 1**, 119090 (2024)
9. Shi, S., Zuo, Y., Li, T.: Path planning of unmanned surface vessel based on improved RRT[C]//IECON 2023-49th annual conference of the IEEE industrial electronics society. IEEE, 1–6 (2023)
10. Liang, Z., Yang, Y., Huang, Z., Zhou, S.: USV path planning algorithm based on 20D-A∗ and IDWA algorithms under COLREGs. Ocean Eng. **331**, 121389 (2025). https://doi.org/10.1016/j.oceaneng.2025.121389
11. Li, Z., Du, M., Qin, J., Chen, X.: Research on robot path planning based on multi-strategy genetic ant colony optimization algorithm. Inf. Sci. **718**, 122407 (2025). https://doi.org/10.1016/j.ins.2025.122407
12. Yang, T.-Y., Zou, L., Wu, Z.-X., Chen, W.-M.: An improved A-star algorithm coupled with graph division and AIS data for ship path planning. Ocean Eng. **330**, 121234 (2025). https://doi.org/10.1016/j.oceaneng.2025.121234
13. Tao, B., Kim, J.-H.: Deep reinforcement learning-based local path planning in dynamic environments for mobile robot. J. King Saud Univ. – Comput. Inf. Sci. **36**(10), 102254 (2024). https://doi.org/10.1016/j.jksuci.2024.102254
14. Gök, M.: Dynamic path planning via Dueling double deep Q-network (D3QN) with prioritized experience replay. Appl. Soft Comput. **158**, 111503 (2024). https://doi.org/10.1016/j.asoc.2024.111503

On Adaptive Cooperative Strategy for Throughput Enhancement in V2V/V2I Communications

Weidong Yang[1], Xinzhuo Zhang[1], Yan Liu[1(✉)], Ranran Sun[1], and Yulong Shen[2]

[1] Hangzhou Institute of Technology, Xidian University, Hangzhou, China
{yangweidong01,liuyan,sunranran}@xidian.edu.cn,
24241215271@stu.xidian.edu.cn
[2] School of Computer Science and Technology, Xidian University, Xi'an, China
ylshen@mail.xidian.edu.cn

Abstract. This paper investigates the challenge of optimizing communication performance in highway scenarios with discontinuous infrastructure coverage. First, we propose an Adaptive Cooperative Strategy (ACS) that dynamically integrates Vehicle-to-Infrastructure (V2I) and Vehicle-to-Vehicle (V2V) communications in both same-direction and opposite-direction traffic flows, adaptively selecting the optimal transmission path based on real-time vehicle positions and network conditions. Then, we develop a comprehensive analytical model, modeling the vehicular network as a stochastic process to capture the dynamic topology of the network and deriving a closed-form expression for the average achievable throughput. Finally, extensive simulations validate the accuracy of our analysis and evaluate the impact of key parameters on performance. The results demonstrate that the proposed ACS significantly outperforms traditional single-mode strategies in scenarios with discontinuous infrastructure coverage. Our approach aims to maximize achievable throughput in infrastructure-sparse vehicular networks and promote the development of intelligent transportation systems.

Keywords: Vehicular networks · Cooperative communication · V2I · V2V · Throughput

1 Introduction

With the rapid development of intelligent transportation systems and the continuous maturation of vehicular network technology, Vehicle-to-Vehicle (V2V) and Vehicle-to-Infrastructure (V2I) communications have become key technological foundations for realizing intelligent driving, traffic safety management, and efficient traffic coordination. In highway scenarios, the combination of high-speed vehicle movement, dynamic network topologies, and uneven infrastructure coverage creates a complex communication environment where traditional single

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 400–418, 2026.
https://doi.org/10.1007/978-981-95-8405-5_22

communication modes struggle to adapt, resulting in unstable connections, low data transmission efficiency, and significantly limited performance of vehicular communication systems. Therefore, how to effectively utilize V2V and V2I communications in dynamic vehicular network environments to optimize network throughput presents a significant technical challenge.

Early research on vehicular ad hoc networks (VANETs) primarily focused on single communication modes, such as standalone V2I or V2V communication. [1] analyzed the existence probability of connectivity paths between vehicles and roadside units (RSUs) using a stochastic model and proposed an upper bound for end-to-end packet delivery delay. However, standalone V2I communication faces the challenge of limited coverage. As noted in [2], the high deployment cost of base stations makes it difficult to fully cover highways, resulting in large "dark zones" and transmission interruptions. In terms of standalone V2V communication, [3] investigated spectrum reuse optimization for V2V communication, employing stochastic geometry theory and homogeneous Poisson point processes to characterize the random distribution of V2V transmitters. It derived closed-form expressions for the success probability of V2V pair transmissions and V2V broadcasts, and analyzed the problem of maximizing average throughput under V2I communication interruption constraints. [4] proposed a cooperative multi-hop transmission scheme for a two-way amplify-and-forward relay network. Nevertheless, these single communication modes exhibit significant limitations in complex VANET environments, making it challenging to meet the demand for efficient and stable data transmission. This has prompted researchers to explore hybrid communication modes.

To overcome the limitations of single communication modes, hybrid communication strategies have become a research hotspot. In terms of unidirectional V2V and V2I collaboration, [5] proposed the ChainCluster scheme, which selects appropriate vehicles to form linear clusters on highways, with each member retrieving a portion of a file from roadside infrastructure, significantly improving the probability of successful file downloads. [6] introduced a dynamic collaboration scheme that generates adaptive multi-hop V2V paths for vehicles located outside RSU coverage. In terms of bidirectional V2V and V2I collaboration, [7] proposed a collaborative content dissemination scheme (CCDS) that coordinates multiple infrastructures and bidirectionally moving vehicles to assist target vehicles in downloading data. [8,9] proposed a collaborative store-carry-forward (CSCF) scheme and an elastic segment-based collaborative strategy (ESCS), respectively, utilizing bidirectional vehicle flows as relays, significantly improving throughput. In single-hop collaboration, [10] proposed a cluster-based resource management scheme that groups vehicles into clusters and selects optimal vehicles as cluster heads to manage V2V and V2I connections, significantly enhancing the average throughput of cellular users. [11,12] investigated cooperative medium access control (MAC) optimization techniques in RSU-equipped VANETs, proposing distributed cooperative MAC schemes based on the CSMA/CA protocol. [13] explored the use of vehicles traveling in the opposite direction as single-hop relays. [14] further extended this approach, addressing

the issue of enhancing network capacity in multi-user scenarios through cooperative communication strategies. In terms of multi-hop V2V links, [15] employed stochastic geometry to study the success probability in multi-relay cooperative C-V2X networks, deriving analytical expressions for joint success probability considering interference correlation. Regarding multi-hop V2V and V2I collaboration, [16] investigated data dissemination in hybrid vehicular networks, using bidirectionally moving vehicles as relays and proposing a distributed multi-source scheduling algorithm that significantly improved data dissemination efficiency. [17] proposed a joint frequency scheduling and power control scheme, mapping I2V and V2V links to tuple links, and significantly enhancing network connectivity and average throughput through the joint design of frequency schedulers and power controllers. Additionally, [18] explored complex collaborative scenarios and introduced a deep reinforcement learning-based task offloading scheme for vehicular edge computing. This scheme enables flexible RSU-RSU collaboration to transmit task results to the RSU currently serving the vehicle and offload tasks from high-load RSUs to low-load RSUs, significantly reducing overall task processing costs. Despite significant progress in various aspects, most existing studies primarily focus on single multi-hop or single-hop, unidirectional or bidirectional relaying V2V/V2I cooperative strategies, lacking comprehensive consideration of multiple communication modes.

Based on the above analysis, this paper proposes an Adaptive Cooperative Strategy (ACS) suitable for highway scenarios with sparse infrastructure, comprehensively considering the characteristics of V2I and V2V communications, dynamically adjusting same-direction multi-hop and opposite-direction single-hop communication modes to optimize achievable throughput for long-distance communications. Notably, although our analysis focuses on a straight highway segment, the results extend directly to independent roads in 2D and 3D scenarios [14] (e.g., 2D grid networks and 3D multi-layer networks). In such cases, the overall network capacity can be regarded as the sum of the capacities of the individual roads. The main contributions of this paper are summarized as follows:

1. We propose an ACS that comprehensively utilizes V2I communication and bidirectional V2V communication, adaptively selecting communication modes based on the positions of same-direction and opposite-direction vehicles to improve the achievable throughput on bidirectional roads with discontinuous infrastructure coverage.
2. Based on the above results, we develop an analytical model to study the data transmission process in different communication scenarios under dynamic network topologies. We formulate the data volume obtained in same-direction multi-hop communication as an optimization problem to maximize the data volume. Furthermore, we derive a closed-form expression for the average achievable throughput.
3. Finally, extensive simulation experiments are conducted to illustrate the impact of RSU spacing, opposite-direction vehicle density, and same-direction vehicle density on the average achievable throughput. The results demonstrate

that the proposed ACS significantly improves the throughput of vehicular networks compared to single-mode communication strategies.

The rest of this paper is organized as follows: Sect. 2 describes the system model and the ACS. Section 3 provides a theoretical analysis of the average achievable throughput through V2I and V2V in one period. Section 4 validates the theoretical analysis through simulations and discusses the impact of key parameters on performance. Section 5 concludes the paper.

2 System Model and Adaptive Cooperative Strategy

2.1 System Model

As shown in Fig. 1, a bidirectional lane highway network includes a single requesting vehicle (i.e., requester) aiming to download a large data file from a central server.

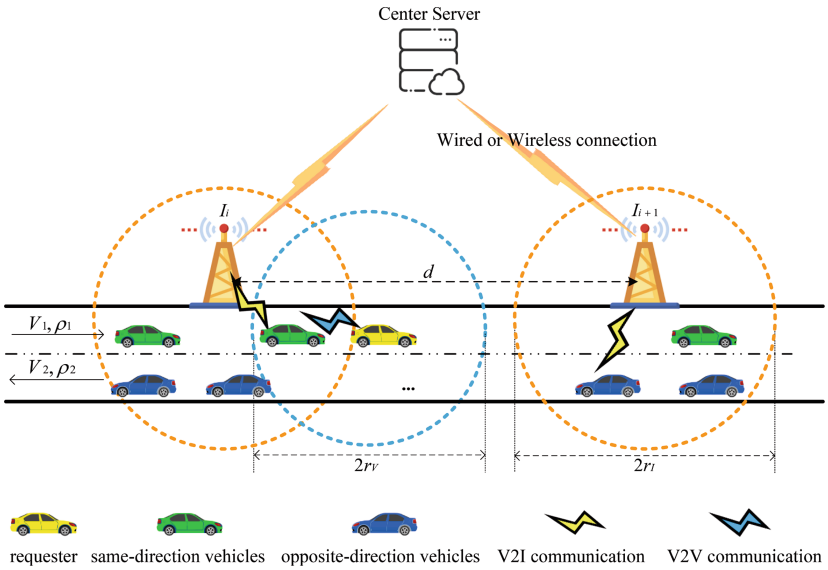


Fig. 1. A highway communication network scenario with a two-lane road within a period. Vehicles traveling in the same direction communicate through multi-hop, while those traveling in the opposite direction communicate through single-hop.

This strategy achieves the download of large files by integrating V2I, opposite V2V, and forward V2V communications. The fixed deployment of RSUs, coupled with their capability to connect to the central server via wired or wireless links, establishes RSUs as reliable data relays for the requester in a dynamic environment. The requester is assumed to receive the exact data fragments required

from RSUs, opposite-direction vehicles (i.e., carriers), and same-direction vehicles (i.e., relays).

We adopt the unit disk model for communication modeling, where the data transmission rate remains constant within the range, and no transmission occurs beyond this range. Furthermore, in time-varying channels, V2I and V2V data transmission rates may be approximated by their time-averaged values [13]. V2I and V2V communications operate in half-duplex and unicast modes, utilizing distinct channels to prevent mutual interference. Assuming an infinitely long road, the lane width is negligible. RSUs equipped with wired or wireless communication capabilities for receiving information from the central server are uniformly distributed along the road at intervals d. The communication radius of each RSU is r_I, and the transmission rate to vehicles is R_I. Vehicles traveling in same-direction and opposing lanes exhibit densities ρ_1 and ρ_2, correspondingly. The vehicle count adheres to a Poisson distribution, while the spacing between vehicles conforms to an exponential distribution. In the same and opposite directions, vehicles maintain uniform velocities of v_1 and v_2, respectively. The communication radius of each vehicle is r_V, and the inter-vehicle data transmission rate is R_V. Moreover, each vehicle is equipped with a GPS module for localization. In the common scenario considered, $r_I > r_V$ and $R_I > R_V$ are satisfied.

2.2 Adaptive Cooperative Strategy

Based on the aforementioned communication scenario, we propose an ACS to optimize the achievable throughput for the requester. The specific implementation process of this strategy is detailed below.

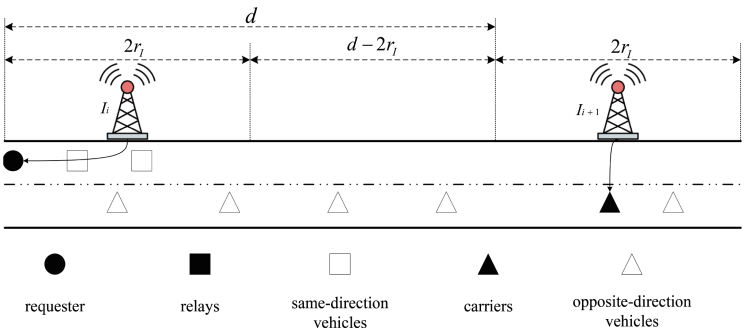
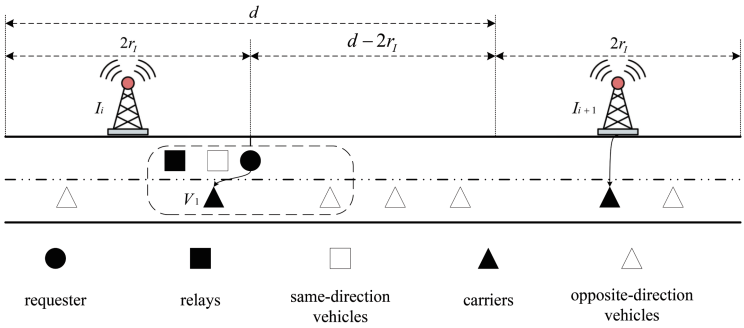


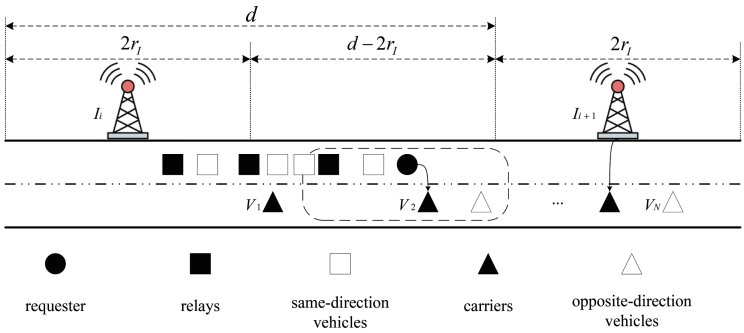
Fig. 2. The requester communicates with RSUs.

Our analysis focuses on a period during which the requester travels a distance d. As shown in Fig. 2, this period spans from the requester entering the transmission range of I_i to just before entering the transmission range of I_{i+1}. During this period, due to the larger transmission range r_I and higher data transmission rate R_I of the RSU, the requester prioritizes communication with I_i to maximize

the amount of data acquired. Upon leaving the communication range of I_i, the requester engages in V2V communication over the remaining distance $d - 2r_I$, prioritizing interaction with carriers and secondarily with relays.



(a) The requester is at the boundary of the communication range of I_i.

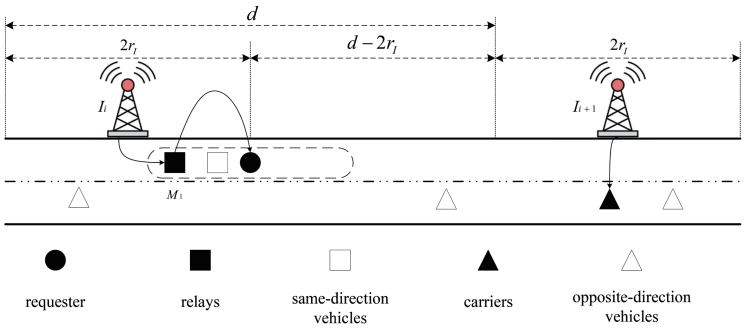


(b) The requester is beyond the boundary of the communication range of I_i.

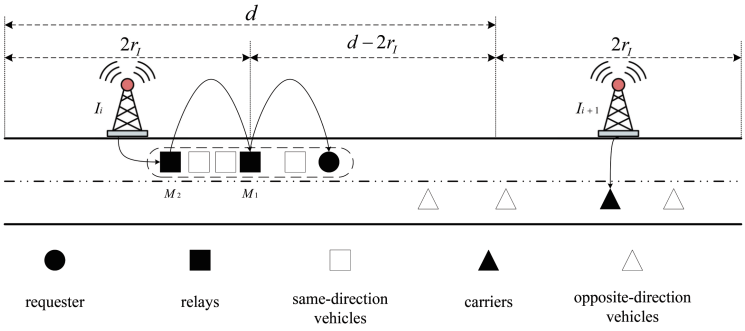
Fig. 3. The requester communicates with carriers.

If carriers are within the requester's communication range, as shown in Fig. 3, the requester utilizes the carriers to perform single-hop V2V communication using the First-In-First-Out (FIFO) strategy. Figure 3a and Fig. 3b illustrate the scenarios where the requester is at the boundary of I_i's transmission range and beyond this boundary, respectively. In both cases, the carriers provide a brief yet efficient single-hop communication opportunity, enabling the requester to rapidly acquire data to meet the needs after disconnection from I_i. Specifically, the requester selects the carrier that entered its communication range earliest and continues communication until that carrier leaves the communication range or the data is fully transmitted. Subsequently, the requester selects the next carrier that entered its communication range earliest to continue transmission. In the period considered, there are N carriers, denoted as $V_1, V_2, \ldots, V_N$. The distance

between consecutive carriers V_{i-1} and V_i is indicated by l_i, for $i = 2, 3, \ldots, N$, while l_1 represents the distance from the left boundary of the requester's communication range to the first carrier V_1. According to [19], the distance l_i follows an exponential distribution with an expected value of $E[l_i] = \frac{1}{\rho_2}$.



(a) The requester establishes single-hop relay communication with relays.



(b) The requester establishes multi-hop relay communication with relays.

Fig. 4. The requester communicates with relays.

If there are no carriers within the communication range of the requester, multi-hop communication via relays is considered. This process is shown in Fig. 4, we adopt a farthest multi-hop selection strategy. As illustrated in Fig. 4a, when the requester is at the boundary of the transmission range of I_i, it checks whether there are relays within its communication range. If there are, it selects the relay that is within both the communication ranges of the requester and I_i and is farthest from the requester as the first relay vehicle M_1. As depicted in Fig. 4b, when M_1 reaches the boundary of the transmission range of I_i, it selects the relay that is within both the communication ranges of M_1 and I_i and is farthest from M_1 as the next relay vehicle M_2. This process continues until no more relays are available. Let Q denote the number of relays in the multi-hop

communication link formed when the requester moves beyond the transmission range of the RSU I_i. This forms a connected multi-hop link between I_i and the requester, with $Q+1$ hops including Q hops from the Q relay vehicles to the requester and one hop from I_i to the last relay vehicle M_Q. We define P_d^i, for $i = 1, 2, \ldots, Q$, as the distances associated with the relays. P_d^1 represents the distance between the requester and the first relay M_1, while P_d^i for $i \geq 2$ denotes the distance between consecutive relays M_{i-1} and M_i. A store-and-forward approach is adopted, where each relay vehicle M_i can receive a data amount E_i from the I_i while within its transmission range. The data amount received by the requester from relays within a period is denoted by Z_i. By selecting the farthest relay vehicle, the coverage range per hop is maximized, thereby reducing the number of hops required. This multi-hop link is activated only when there are no carriers within the communication range of the requester. This combination of strategies aims to maximize the requester's data acquisition efficiency. By establishing a priority hierarchy (RSUs > carriers > relays), the system can flexibly switch communication modes across different scenarios, adapting to the high dynamics of highway traffic and the complexity of the communication environment. It is worth noting that the ACS presented here is not the only possible approach.

3 Theoretical Analysis of Average Achievable Throughput

3.1 Average Achievable Throughput

We model the communication within a period as a renewal reward process, where the update process captures the switching of data transmitters interacting with the requester within a time interval dt, and the reward process quantifies the volume of data received by the requester following a transmitter switch. To evaluate the performance of this process, the average achievable throughput is used as the metric.

While the requester travels within a period, the average achievable throughput is expressed as follows [13]:

$$\eta = \lim_{t \to \infty} \frac{D(t)}{t} = \frac{E[D]}{E[T]}, \tag{1}$$

where $E[T] = \frac{d}{v_1}$ represents the expected time for the requester to travel a distance d, which is also the expected time for the requester to go through one period, and $E[D]$ denotes the total expected data acquired by the requester from RSUs, carriers, and relays. The total expected data volume is given by:

$$E[D] = E[D_I] + E[D_R] + E[D_S]. \tag{2}$$

Furthermore, according to [13], the expressions for the expected data $E[D_I]$ received directly from the RSUs via V2I communication and the expected data

volumes $E[D_R]$ obtained from carriers via V2V communication are given as follows:

$$E[D_I] = \frac{2r_I R_I}{v_1}, \tag{3}$$

and

$$E[D_R] = \frac{\left[\frac{(d-2r_I)(v_1+v_2)}{v_1} + r_V - \frac{1}{\rho_2}\right]\left(1 - e^{-2\rho_2 r_V}\right) R_V}{v_1 + v_2}. \tag{4}$$

$E[D_S]$ represent the expected data volumes obtained from relays via V2V communication. The detailed calculation process will be introduced in the next section.

3.2 The Average Achievable Throughput from Relays

The data volume received by the requester through communication with relays in the V2V communication region within oneperiod is formulated as the following optimization problem:

$$\max \quad \sum_{i=1}^{N} Z_i \tag{5a}$$

$$\text{s.t.} \quad 0 \le E_i \le \frac{P_d^i}{v_1} R_I, \quad i = 1, 2, \ldots, Q \tag{5b}$$

$$\sum_{i=1}^{Q} E_i \le \frac{\sum_{i=1}^{Q} P_d^i}{v_1} R_I \tag{5c}$$

$$0 \le Z_i \le \frac{\max\{0, l_i - 2r_V\}}{v_1 + v_2} R_V, \quad i = 1, 2, \ldots, N \tag{5d}$$

$$\sum_{i=1}^{N} Z_i \le \frac{\sum_{i=1}^{N} \max\{0, l_i - 2r_V\}}{v_1 + v_2} R_V \tag{5e}$$

$$\sum_{i=1}^{N} Z_i \le \sum_{i=1}^{Q} E_i \tag{5f}$$

In this optimization framework, $\sum_{i=1}^{N} Z_i$ denotes the maximum data volume that the requester can acquire from the relays within a period. Constraint (5b) establishes the upper and lower bounds for the data volume E_i each relay receives from I_i. Constraint (5c) defines the upper limit of the total data volume obtainable by a sequence of Q relays from I_i. constraint (5d) delineates the bounds for the data volume Z_i that the requester can obtain from the relays when passing through the i-th carriers spacing. Here, $l_i - 2r_V$ represents the gap between consecutive carriers where relay communication is feasible. Constraint (5e) specifies the upper bound on the total data volume the requester can receive from the relays when passing through all carrier spacings. Lastly, constraint (5f) ensures that the total data volume received by the requester from the relays does not surpass the aggregate data volume the relays acquire from I_i.

From this optimization problem, it can be seen that the data volume received by the requester from the relays $\sum_{i=1}^{N} Z_i$ satisfies:

$$\sum_{i=1}^{N} Z_i \leq \min\left\{D_{S1}, D_{S2}\right\}, \tag{6}$$

where D_{S1} is derived from constraints (5c) and (5f), and D_{S2} is derived from constraint (5e), with the expressions given as:

$$D_{S1} = \frac{\sum_{i=1}^{Q} P_d^i}{v_1} R_I, \tag{7}$$

and

$$D_{S2} = \frac{\sum_{i=1}^{N} \max\{0, l_i - 2r_V\}}{v_1 + v_2} R_V. \tag{8}$$

When $D_{S1} \leq D_{S2}$, we designate this as the forward communication limited state, where the data volume received by the requester is constrained by D_{S1}, reflecting the total amount of data received by the relays from I_i. To compute $E\left[D_{S1}\right]$, we only need to evaluate the total multi-hop path length, denoted as P:

$$P = \sum_{k=1}^{Q} P_d^i, \tag{9}$$

Given that the vehicles share identical speed and direction, the distance between adjacent relays remains constant. Thus, the expected total path length can be expressed as:

$$E[P] = E[Q]E[P_d]. \tag{10}$$

To determine $E[Q]$, it is necessary to obtain the probability density function $f_Q(q)$ of the maximum number of relays Q. This function is influenced by factors such as vehicle density, inter-vehicle spacing, and the distance between RSUs. Notably, deriving a closed-form expression for $f_Q(q)$ poses a significant challenge. Consequently, we recommend employing simulation techniques to acquire numerical results for the maximum number of relays, particularly under realistic parameter settings. Although this approach relies on numerical data, as noted in [6], such results can still be regarded as analytical.

Recalling from earlier discussion, vehicles traveling in the same direction follow a Poisson distribution with density ρ_1. Hence, the spacing between adjacent Same-direction vehicles follows an exponential distribution. According to [6], the expected distance between two adjacent relays is given by:

$$E[P_d] = r_V - \frac{1}{\rho_1} + e^{-\rho_1 r_V}\left(r_V + \frac{1}{\rho_1}\right). \tag{11}$$

Substituting (9), (10), and (11) into the expression for(7) we derive:

$$\begin{aligned} E\left[D_{S1}\right] &= \frac{E[P]}{v_I} R_I = \frac{E[Q]E[P_d]}{v_I} R_I \\ &= \frac{E[Q]}{v_I} R_I \left(r_V - \frac{1}{\rho_1} + e^{-\rho_1 r_V}\left(r_V + \frac{1}{\rho_1}\right)\right). \end{aligned} \tag{12}$$

Conversely, when $D_{S1} > D_{S2}$, we term it the opposite communication limited state, where the data volume is limited by D_{S2}, determined by the available gaps left for the relays by the carriers. To calculate $E[D_{S2}]$, we designate the position at a distance $r_I - r_V$ from I_i as the coordinate origin. According to [13], the carriers that the requester may encounter are located within the road segment $[0, S]$, where $S = \frac{(d-2r_I)(v_1+v_2)}{v_1} + r_V$. We introduce the concept of clusters. Vehicles on the opposite lane are considered to belong to the same cluster if and only if there exists a communication path between any two vehicles, i.e., the distance between any two adjacent vehicles within the cluster is less than $2r_V$. Otherwise, these vehicles are assigned to different clusters. As noted in [20], the minimum distance between clusters must exceed the requester's transmission range, $2r_V$. Let K represent the number of clusters on the road segment S, and let G_j, for $j = 1, 2, \ldots, K$, denote the inter-cluster distances. With this framework, D_{S2} can be expressed as:

$$D_{S2} = \frac{\sum_{j=1}^{K} (G_j - 2r_V)}{v_1 + v_2} R_V. \tag{13}$$

According to [13], we assume that the number of clusters and the inter-cluster distances within the segment $[0, S]$ are mutually independent. Thus, under the constrained state of opposite communication, the expected data volume received by the requester $E[D_{S2}]$ is given by:

$$E[D_{S2}] = \frac{E[K](E[G_j] - 2r_V)}{v_1 + v_2} R_V. \tag{14}$$

The calculation of $E[K]$ is provided in [13] as:

$$E[K] = \frac{S - E[l_1]}{E[L_j] + E[G_j]}, \tag{15}$$

In (15), $E[L_j]$ denotes the expected length of a cluster. The expressions for $E[L_j]$ and $E[G_j]$ are derived in [20] as follows:

$$E[L_j] = \left(e^{2\rho_2 r_V} - 1\right)\left(\frac{1}{\rho_2} - \frac{2r_V e^{-2\rho_2 r_V}}{1 - e^{-2\rho_2 r_V}}\right), \tag{16}$$

and

$$E[G_j] = 2r_V + \frac{1}{\rho_2}. \tag{17}$$

By substituting equations (15), (16), (17), and $E[l_1]$ into (14), we obtain:

$$E[DS_2] = \frac{[\frac{(d-2r_I)(v_1+v_2)}{v_1} + r_V - \frac{1}{\rho_2}] \cdot e^{-2\rho_2 r_V}}{v_1 + v_2} R_V. \tag{18}$$

4 Simulation and Discussion

4.1 Simulation Parameter Settings

We use MATLAB to evaluate the performance of the ACS by simulating vehicle communication on an infinitely long highway. We set our simulation parameters according to [13], and all simulation parameter settings can be found in TABLE 1. Unless otherwise specified, $d = 10000\text{m}, \rho_1 = 0.008\text{veh/m}$ and $\rho_2 = 0.003\text{veh/m}$. Each of our results is the average of 1000 simulations.

Table 1. Simulation Parameters

Parameter	Value
Road length	∞
RSU spacing (d)	$[2000, 25000]$ m
Same-direction vehicle density (ρ_1)	$[0.003, 0.009]$ vehicles/m
Opposite-direction vehicle density (ρ_2)	0.003 vehicles/m
Request vehicle speed (v_1)	15 m/s
Same-direction vehicle speed (v_1)	15 m/s
Opposite-direction vehicle speed (v_2)	25 m/s
Expected same-direction spacing ($E[s_S]$)	$\frac{1}{\rho_1}$
Expected opposite-direction spacing ($E[s_R]$)	$\frac{1}{\rho_2}$
RSU communication radius (r_I)	500 m
Vehicle communication radius (r_V)	250 m
RSU transmission rate (R_I)	6 Mb/s
Vehicle transmission rate (R_V)	5 Mb/s

4.2 Impact of Different Parameters on Achievable Throughput

In Fig. 5–7, the comparison between analytical and simulation values is presented under various parameter settings.

In Fig. 5, the variation of the requester's achievable throughput with RSU spacing is depicted. The close alignment between the curve and circles demonstrates that the theoretical results are in strong agreement with the simulation outcomes, thereby confirming the validity of our theoretical analysis. With the density of opposite-direction vehicles and the density of same-direction vehicles held constant, the achievable throughput gradually decreases as the RSU spacing increases. This phenomenon occurs because, as the spacing between RSUs increases, the duration of a communicationperiod becomes longer. According to Eq. (1), the achievable throughput diminishes.

In Fig. 6, the variation of the requester's achievable throughput with densities of vehicles in the same-direction is depicted. The alignment between the curve

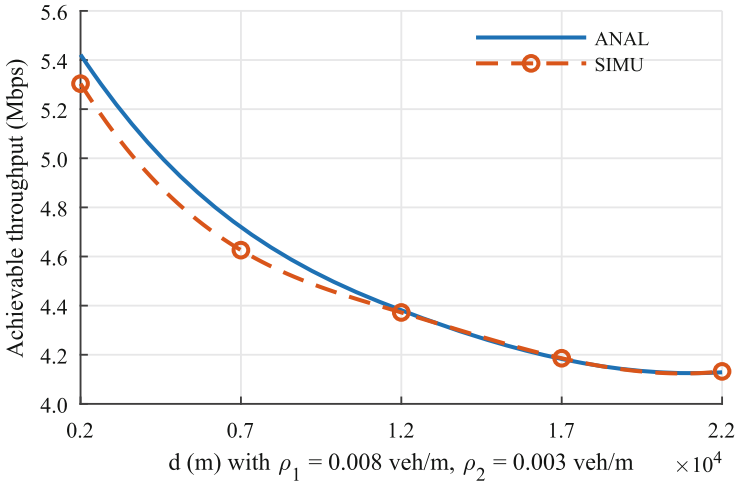


Fig. 5. The achievable throughput of ACS varies with the spacing between RSUs.

and the circles indicates a strong agreement between the theoretical results and simulation results, further confirming the validity of our theoretical analysis. With the RSU spacing and the density of opposite-direction vehicles held constant, an increase in the density of same-direction vehicles leads to a gradual rise in the achievable throughput. This is attributed to the fact that a higher density of same-direction vehicles facilitates the establishment of multi-hop links, enabling the requester to acquire more data from relays.

In Fig. 7, the variation of the requester's achievable throughput with densities of vehicles in the opposite-direction is depicted. The consistency between the curve and the circles demonstrates that the theoretical results align closely with the simulation results, thereby validating our theoretical analysis. With the RSU spacing and the density of same-direction vehicles held constant, an increase in the density of opposite-direction vehicles results in a steady increase in the achievable throughput, which subsequently stabilizes. This occurs because a higher density of opposite-direction vehicles enable the requester to acquire data from a greater number of carriers. However, when the opposite-direction vehicle density reaches a certain threshold, the amount of data the requester can receive reaches its upper limit, so further increasing the same-direction vehicle density has no effect.

Furthermore, Fig. 8 demonstrates that the achievable throughput shows a more significant variation with changes in the density of opposite-direction vehicles. This occurs because, during the V2V communication phase, the higher priority assigned to opposite-direction vehicles enables data to be preferentially obtained from single-hop opposite-direction vehicles. As the density of opposite-direction vehicles increases, the occasions for the requester to rely on multi-hop links to acquire data become less frequent. the variation in the upper bound with changes in the density of same-direction vehicles is relatively minor. This is

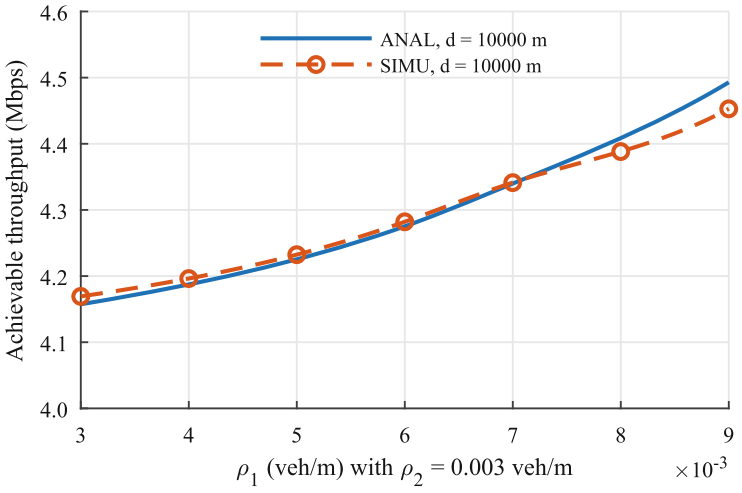


Fig. 6. The achievable throughput of ACS varies with the density of same-direction vehicles.

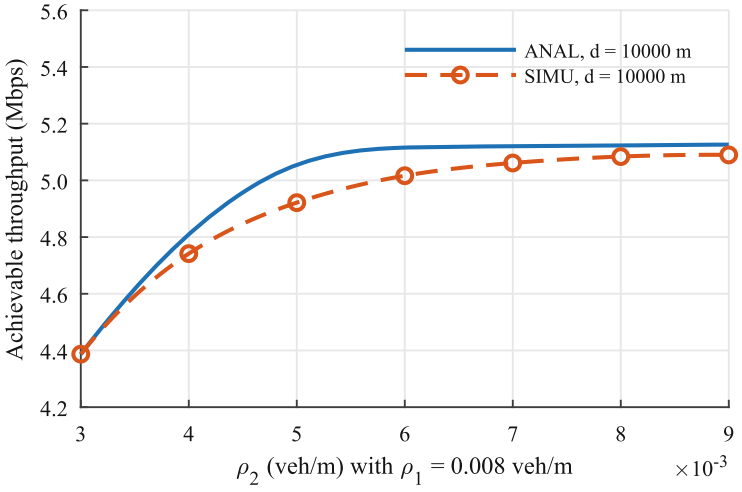


Fig. 7. The achievable throughput of ACS varies with the density of opposite-direction vehicles.

because same-direction multi-hop links have the lowest priority and are only utilized when neither RSUs nor carriers are within the requester's communication range.

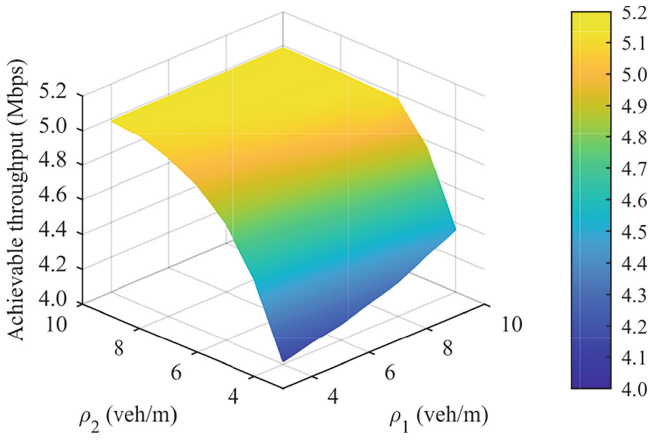


Fig. 8. The achievable throughput of ACS versus the density of same-direction vehicles and the density of opposite-direction vehicles.

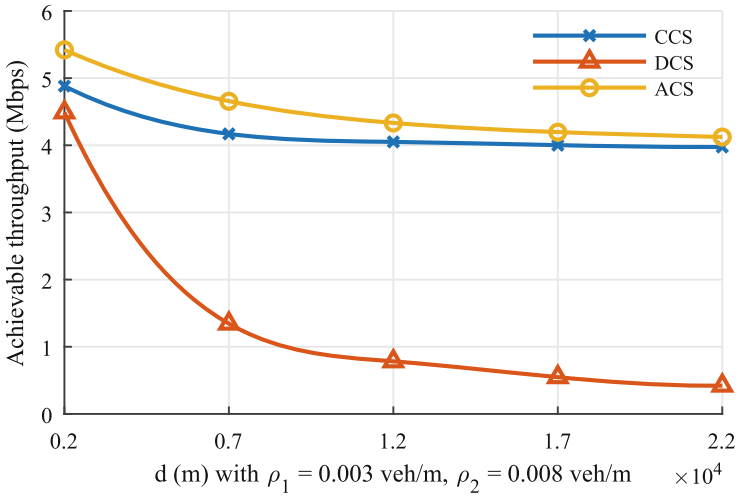


Fig. 9. Comparison of achievable throughput across three strategies versus RSU spacing.

4.3 Comparison with the Achievable Throughput of Existing Strategies

In this section, we simulate and discuss the achievable throughput for ACS, Cooperative Communication Strategy (CCS) [13], and Dynamic Cooperation Strategy (DCS) [6]. In Fig. 9–11, the comparison of the achievable throughput under the three cooperation strategies is shown under different parameter settings.

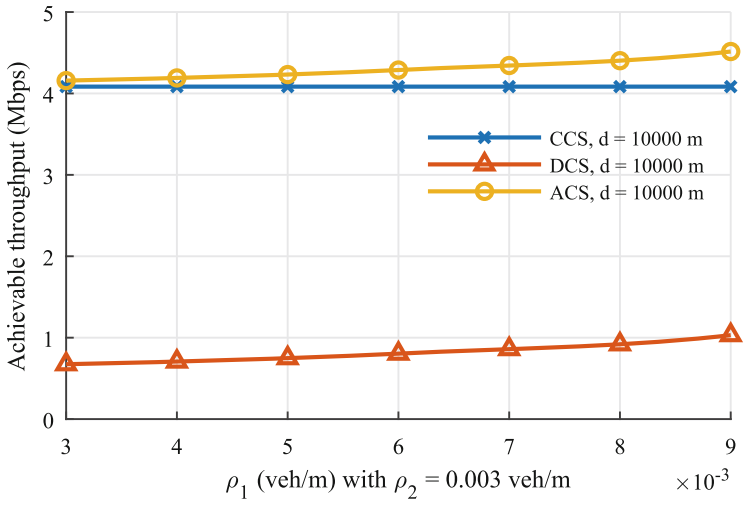


Fig. 10. Comparison of Achievable Throughput Across Three Strategies versus same-direction vehicle density.

In Fig. 9, the variation of achievable throughput for the three strategies with RSU spacing is illustrated under the conditions of fixed densities of opposite-direction and same-direction vehicles. It can be observed that CCS outperforms DCS because requester encounters more carriers than relays on an infinitely long road, enabling the requester to receive a greater amount of data from carriers within the same time frame. Compared to CCS and DCS, our proposed ACS performs the best, as it receives data from both carriers and relays.

In Fig. 10, the achievable throughput of the three strategies with respect to the density of same-direction vehicles is illustrated under the conditions of fixed density of opposite-direction vehicles and RSU spacing. It can be observed that the achievable throughput of DCS increases with the rise in ρ_1, because the increase in the density of same-direction vehicles facilitates the establishment of multi-hop links, thereby enabling the requester to obtain more data from relays. Since CCS only considers carrier nodes, its achievable throughput remains constant as ρ_1 increases. Compared with DCS and CCS, ACS has the optimal achievable throughput, which is the advantage of utilizing bidirectional lanes.

In Fig. 11, the achievable throughput of the three strategies with respect to the density of opposite-direction vehicles is illustrated under the conditions of fixed density of same-direction vehicles and RSU spacing. It can be noted that the achievable throughput of CCS rises with an increase in ρ_2, as the greater density of opposite vehicles enable the requester to acquire more data from carriers. In contrast, since the DCS solely relies on relays, its achievable throughput remains constant as ρ_2 increases. It can be found that the achievable throughput of ACS consistently exceeds that of CCS, with the gap between them narrowing as ρ_2 increases. This occurs because, as the density of opposite-direction vehicles ρ_2

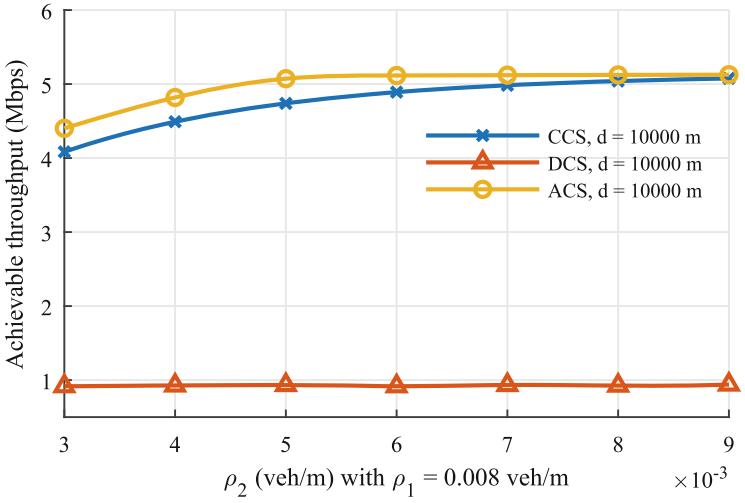


Fig. 11. Comparison of Achievable Throughput Across Three Strategies versus opposite-direction vehicle density.

increases, requester encounters more carriers. Due to prioritization, requester communicate more frequently with carriers, consequently reducing opportunities for communication with relays.

4.4 Discussion

The use of the unit-disk model and the assumption of a constant transmission rate within the communication range are admittedly idealized, as they do not account for interference or channel fading. However, since the main contribution of this work lies in the ACS rather than in detailed physical-layer modeling, such abstractions are adopted to facilitate isolating and verifying the benefits of the proposed ACS. In future work, we plan to incorporate more sophisticated channel models to examine how interference and fading influence the performance of ACS. In practical vehicular networks, the speed of individual vehicles may deviate from the average. As shown in results [13], although such deviations can lead to vehicle overtaking, their effect on throughput performance remains negligible.

5 Conclusions

In this paper, we propose an ACS for V2I and V2V communications as well as vehicular mobility. In the V2V communication area, the strategy leverages vehicles on bidirectional lanes to perform opposite single-hop communication and forward multi-hop communication, thereby enhancing the average achievable throughput in highway vehicle networks. Theoretical analysis and extensive simulation results demonstrate the effectiveness of the strategy and reveal

the impact of key parameters on the average achievable throughput. Furthermore, compared to strategies that use only forward multi-hop communication or opposite single-hop communication, the proposed strategy exhibits significant advantages in improving the average achievable throughput. In addition, deep learning, as a broadly applicable methodology, presents a promising avenue for future work in optimizing our scenario.

Acknowledgements. This study was funded by National Natural Science Foundation of China (grant number 62172141).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Atallah, R., Khabbaz, M., Assi, C.: Multihop v2i communications: a feasibility study, modeling, and performance analysis. IEEE Trans. Veh. Technol. **66**(3), 2801–2810 (2016). https://doi.org/10.1109/TVT.2016.2586758
2. Siddig, A.A., Ibrahim, A.S., Ismail, M.H.: An optimal power allocation and relay selection full-duplex store-carry-forward scheme for intermittently connected vehicular networks. IEEE Access **8**, 163903–163916 (2020). https://doi.org/10.1109/ACCESS.2020.3020826
3. Yang, Y., Xiang, H., He, G., Huang, J., Yan, Q.: Optimal spectrum reuse of v2v communications for maximizing average throughput in vehicular ad hoc networks. In: 2021 IEEE/CIC International Conference on Communications in China (ICCC), pp. 1178–1182. IEEE (2021). https://doi.org/10.1109/ICCC52777.2021.9580213
4. Wang, C.L., Cho, T.N., Tsai, T.H., Jan, M.C.: A cooperative multihop transmission scheme for two-way amplify-and-forward relay networks. IEEE Trans. Veh. Technol. **66**(9), 8569–8574 (2017). https://doi.org/10.1109/TVT.2017.2687622
5. Zhou, F., et al.: Chaincluster: engineering a cooperative content distribution framework for highway vehicular communications. IEEE Trans. Intell. Transp. Syst. **15**(6), 2644–2657 (2014). https://doi.org/10.1109/TITS.2014.2321293
6. Nguyen, B.L., Ngo, D.T., Tran, N.H., Dao, M.N., Vu, H.L.: Dynamic v2i/v2v cooperative scheme for connectivity and throughput enhancement. IEEE Trans. Intell. Transp. Syst. **23**(2), 1236–1246 (2020). https://doi.org/10.1109/TITS.2020.3023708
7. Guo, T., Li, C., Dong, W., Miao, Z., Su, X.: Enabling efficient content dissemination for cooperative vehicular networks. In: 2017 IEEE 28th Annual International Symposium on Personal, Indoor, and Mobile Radio Communications (PIMRC), pp. 1–5. IEEE (2017). https://doi.org/10.1109/PIMRC.2017.8292252
8. Wang, Y., Liu, Y., Zhang, J., Ye, H., Tan, Z.: Cooperative store–carry–forward scheme for intermittently connected vehicular networks. IEEE Trans. Veh. Technol. **66**(1), 777–784 (2017). https://doi.org/10.1109/TVT.2016.2536059
9. Liu, X., Xu, Z., Meng, Y., Wang, W., Xie, J., Li, Y.: An elastic-segment-based v2v/v2i cooperative strategy for throughput enhancement. IEEE Trans. Veh. Technol. **71**(5), 5272–5283 (2022). https://doi.org/10.1109/TVT.2022.3158071

10. Abbas, F., Liu, G., Fan, P., Khan, Z.: An efficient cluster based resource management scheme and its performance analysis for v2x networks. IEEE Access **8**, 87071–87082 (2020). https://doi.org/10.1109/ACCESS.2020.2992591
11. Zhang, Z., Atapattu, S., Wang, Y., Sun, S., Sithamparanathan, K.: Dynamic cooperative mac optimization in rsu-enhanced vanets: A distributed approach. In: ICC 2024-IEEE International Conference on Communications, pp. 4305–4310. IEEE (2024). https://doi.org/10.1109/ICC51166.2024.10622916
12. Zhang, Z., Atapattu, S., Wang, Y., Sun, S., Sithamparanathan, K.: Optimal cooperative mac strategies for wireless vanets with multiple roadside units. IEEE Trans. Veh. Technol. (2024). https://doi.org/10.1109/TVT.2024.3456082
13. Chen, J., Mao, G., Li, C., Zafar, A., Zomaya, A.Y.: Throughput of infrastructure-based cooperative vehicular networks. IEEE Trans. Intell. Transp. Syst. **18**(11), 2964–2979 (2017). https://doi.org/10.1109/TITS.2017.2663434
14. Chen, J., Mao, G., Li, C., Liang, W., Zhang, D.g.: Capacity of cooperative vehicular networks with infrastructure support: Multiuser case. IEEE Trans. Veh. Technol. **67**(2), 1546–1560 (2017). https://doi.org/10.1109/TVT.2017.2753772
15. Pan, B., Wu, H.: Modeling and analysis of multi-relay cooperative communications in c-v2x networks. IEEE Trans. Intell. Transp. Syst. **23**(9), 16371–16385 (2022). https://doi.org/10.1109/TITS.2022.3149987
16. Ni, Y., He, J., Cai, L., Bo, Y.: Data uploading in hybrid v2v/v2i vehicular networks: modeling and cooperative strategy. IEEE Trans. Veh. Technol. **67**(5), 4602–4614 (2018). https://doi.org/10.1109/TVT.2018.2796563
17. Nguyen, B.L., Ngo, D.T., Dao, M.N., Bao, V.N.Q., Vu, H.L.: Scheduling and power control for connectivity enhancement in multi-hop i2v/v2v networks. IEEE Trans. Intell. Transp. Syst. **23**(8), 10322–10332 (2021). https://doi.org/10.1109/TITS.2021.3091130
18. Fan, W., Zhang, Y., Zhou, G., Liu, Y.: Deep reinforcement learning-based task offloading for vehicular edge computing with flexible RSU-RSU cooperation. IEEE Trans. Intell. Transp. Syst. **25**(7), 7712–7725 (2024). https://doi.org/10.1109/TITS.2024.3349546
19. Feller, W.: An introduction to probability theory and its applications, Volume 2, vol. 2. John Wiley & Sons (1991)
20. Wisitpongphan, N., Bai, F., Mudalige, P., Sadekar, V., Tonguz, O.: Routing in sparse vehicular ad hoc wireless networks. IEEE J. Sel. Areas Commun. **25**(8), 1538–1556 (2007). https://doi.org/10.1109/JSAC.2007.071005

CoDVFS: Improving the Energy Efficiency of AI Servers Through Coordinated DVFS

Yijia Zhang[1], Baoqing Wang[1(✉)], Shixin Zhang[1], and Qiang Wang[2]

[1] Peng Cheng Laboratory, Shenzhen, China
{zhangyj01,wangbq,zhangshx}@pcl.ac.cn
[2] Harbin Institute of Technology, Shenzhen, China
qiang.wang@hit.edu.cn

Abstract. With the widespread adoption of deep learning across industries and academia, the energy consumption of AI computing clusters has surged, creating significant economic and environmental challenges. Improving AI server energy efficiency has therefore become a critical research focus. Traditional dynamic voltage and frequency scaling (DVFS) research has primarily focused on regulating either CPU or GPU/NPU frequency in isolation, which cannot fully unlock the energy-saving potential of AI servers. Naively applying joint DVFS requires exhaustively evaluating all frequency combinations to find the most energy-efficient one, which becomes prohibitively expensive with many frequency states.

To address this limitation, this study proposes CoDVFS, a Coordinated Dynamic Voltage and Frequency Scaling framework that employs Bayesian optimization to rapidly converge to optimal frequency configurations for both CPUs and GPU/NPUs. Experimental results on AI servers running HPL-MxP workloads demonstrate that CoDVFS enhances server energy efficiency by 17%, achieving significant energy savings. We also analyze the relationship between whole-server and accelerator-only energy efficiency, proposing an optimum alignment criterion. On our device, the optima of both metrics coincide. However, the criterion reveals that if server and accelerator power scale differently on other devices, optimizing accelerator efficiency alone may not reflect true server efficiency.

Keywords: energy efficiency · DVFS · server

1 Introduction

Advancements in artificial intelligence (AI) spanning natural language processing and computer vision have driven substantial demand for computational resources. Research institutions and enterprises worldwide heavily invest in AI clusters, relying on GPUs/NPUs (Neural Processing Units) for efficient training and inference. However, computational growth brings soaring energy costs. For example, GPT-3 training consumed 1,287 MWh (502t CO_2, equal to 8 cars' lifetime emissions). Computing facilities like Aurora peak at 38.7 MW [20],

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 419–439, 2026.
https://doi.org/10.1007/978-981-95-8405-5_23

potentially costing $30M/year and emitting 140,000t CO_2 annually. As a result, improving the energy efficiency of AI servers is now critical.

Dynamic Voltage and Frequency Scaling (DVFS) is an important technique to control energy consumption and improve energy efficiency [1–5,8,10,12,14–17,19,22,24,25,27–31]. By applying DVFS, we dynamically adjust a processor's operating frequency and supply voltage: increasing frequency and voltage under heavy load to ensure performance, and lowering frequency and voltage under light load to reduce power usage. This technique allows processors to minimize energy consumption whenever possible.

However, existing studies focus solely on applying DVFS to individual components [1–5,8,10,12,14–17,19,22,24,25,27–31]—either the CPU or GPU—while overlooking the fact that, in real tasks, the CPU and GPU work in coordination to complete computations. If DVFS is applied only to the CPU, the GPU may still remain in a high-power state; likewise, optimizing only the GPU frequency may result in CPU power consumption or execution performance failing to match the GPU's processing pace. Such isolated control strategies often fall short in unlocking the full energy efficiency potential of an AI server.

As a result, joint frequency and voltage scaling for both CPU and GPU becomes important. Guided by this motivation, we propose CoDVFS, a Coordinated Dynamic Voltage and Frequency Scaling framework tailored for AI servers. This framework can be applied across different workloads and minimize power consumption or maximize overall system energy efficiency by simultaneously tuning the operating frequency and voltage of both the CPU and GPU.

To facilitate optimization, we adopt a Bayesian optimization approach to efficiently search for the optimal processor frequency combination, making the control strategy more adaptive and precise.

To validate the effectiveness of CoDVFS, we conducted experiments on a DGX server equipped with 8×GPU V100. We used external power monitoring equipment to measure the energy consumption of the server while running benchmarks. The experimental results show that, compared to the default DVFS setting, CoDVFS improves server energy efficiency by 17% (HPL-MxP) and 12% (HPCG), significantly lowering overall electricity usage. Compared with DVFS strategies applied only to CPU or GPU individually, the coordinated control approach further exploits the system's overall energy-saving potential.

Since measuring full server power consumption is more challenging than monitoring GPU power alone, prior GPU DVFS studies have typically focused solely on GPU power measurements [1,3–5,8,12,14,16,22,24,25,28,30,31]. To overcome this limitation, we present a detailed description of our methodology for server-wide power measurement. More importantly, by comparing the GPU power reported by the `nvidia-smi` tool with the total power measured by external power meters, we conduct an in-depth analysis of the differences between whole-server energy efficiency and GPU-only energy efficiency.

Our analysis reveals that the point of optimal GPU energy efficiency does not necessarily coincide with the point of optimal whole-server energy efficiency, especially when the power consumption of other components, such as fans, memory, and the motherboard, varies differently with GPU frequency. We propose an

optimal alignment criterion demonstrating that when GPU power and non-GPU power grow at equal rates, their optimal operating points coincide. On the other hand, when these growth rates differ, optimizing GPU energy efficiency alone may not accurately represent the overall server energy efficiency. This highlights the importance of evaluating power consumption from a holistic perspective in order to accurately assess energy efficiency. These findings may offer new perspectives for future system-level energy efficiency optimization.

2 Background

Dynamic Voltage and Frequency Scaling (DVFS) is a key technique used to control the power consumption and performance of processors by adjusting their operating voltage and frequency [1–5,8,10,12,14–17,19,22,24,25,27–31]. The dynamic power of a processor can typically be approximated by the formula $P \propto CV^2 f$ [9], where C is the capacitance, V is the voltage, and f is the frequency. This equation indicates that reducing the voltage and frequency leads to a decrease in dynamic power consumption. Compared with other power management methods that require changes to application codes or hardware architecture, DVFS offers several advantages: it is non-intrusive to user applications, easily applicable, and provides fast response times as DVFS adjustment usually finishes within 30 milliseconds [21]. As a result, DVFS is widely integrated into modern CPUs and GPUs.

Current CPUs and GPUs typically do not expose voltage control interfaces to users. Instead, on-chip logic automatically adjusts the voltage based on the configured frequency, and voltage scales approximately linearly with frequency. In practice, this means that adjusting the processor's operating frequency is sufficient, with the corresponding voltage being managed internally by the processor. The frequency-voltage curves can be found in [13,18]. In this study, we regulate the frequencies of CPUs and GPUs, thereby indirectly managing their supply voltages.

Research on applying DVFS to CPUs has a long history. Numerous studies have explored how to achieve optimal power and performance trade-offs under various workloads and scenarios by adjusting CPU frequency and voltage [2,7,10,17,19,27,29]. This includes precise modeling of CPU power consumption and the design of optimal frequency-setting strategies. With the widespread adoption of multi-core CPU architectures, researchers have begun to investigate the feasibility of applying differentiated frequency control to individual cores on the same chip [19], as well as independent DVFS control of the uncore components [10,27] (which include on-chip caches and memory controllers) to further tap into energy-saving potential.

Over the past decade, the widespread application of deep learning has driven GPUs to the forefront of high-performance computing. Consequently, research on GPU DVFS has also seen rapid growth [1,3–5,8,12,14–16,22,24,25,28,30,31]. Many studies focus on modeling GPU performance and power consumption under different frequency settings, seeking to identify frequencies that strike a

balance between energy efficiency and computational performance. For GPUs that support memory frequency scaling, some research has also explored the additional optimization opportunities that GPU memory DVFS can provide for task execution [23].

Most existing studies, however, focus on applying DVFS to either the CPU or GPU in isolation and rarely explore the optimization potential of coordinated CPU-GPU control. In addition, most prior experiments measured only the power of the CPU or GPU chip itself, lacking a comprehensive analysis of whole-server power usage. In fact, when server-level energy efficiency is assessed by using the actual power of the entire server as the denominator, the resulting conclusions may differ from those drawn from GPU-only measurements.

To address this limitation, our study measures and analyzes the power consumption of the entire server to provide a more comprehensive evaluation of the energy efficiency benefits of coordinated DVFS in data center servers. Table 1 compares the difference between our work and previous research, showing that most prior studies lack investigation into coordinated frequency scaling and whole-server power measurement.

Table 1. Comparison with related works.

Related Works	GPU DVFS	CPU DVFS	Server Power Measurement
Paul, 2015 [16]	✓		
Guerreiro, 2015 [4]	✓		
Majumdar, 2017 [12]	✓		
Fan, 2019 [3]	✓		
Guerreiro, 2019 [5]	✓		
Ilager, 2020 [8]	✓		
Zou, 2020 [31]	✓		
Kumar, 2021 [10]		✓	
Nabavinejad, 2022 [14]	✓		
Wang F., 2022 [22]	✓		
You, 2022 [27]		✓	
Ali, 2023 [1]	✓		
Bratek, 2023 [2]		✓	✓
Zhang Y., 2024 [30]	✓		
Piga, 2024 [17]		✓	
Patel, 2024 [15]	✓		✓
Zhang H., 2024 [29]		✓	
Wang Q., 2024 [24]	✓		
Stojkovic, 2024 [19]		✓	
Wang Y., 2024 [25]	✓		
Yue, 2025 [28]	✓		
Ours, 2025	✓	✓	✓

3 CoDVFS: A Coordinated Dynamic Voltage and Frequency Scaling Framework

Existing frequency and voltage scaling techniques primarily target individual computing components—either the CPU or GPU—for optimization. To address this issue, it is essential to adopt a holistic perspective and perform coordinated energy efficiency optimization across both the CPU and GPU.

A straightforward way to implement joint CPU-GPU DVFS is to evaluate all possible combinations of CPU and GPU frequencies exhaustively, and then select the configuration that yields the highest energy efficiency. However, while exhaustive search is conceptually simple, it becomes prohibitively expensive when the number of frequency states is large. Suppose the CPU has M frequency states and the GPU has N states. If each combination is tested R times to ensure measurement stability, then $M \times N \times R$ experiments are required in total. For example, if $M = N = 11$ and $R = 5$, a total of 605 experiments would be needed, which could take several hours or more. This time and resource expenditure is impractical in AI computing environments.

To overcome this limitation, we propose CoDVFS, a coordinated dynamic voltage and frequency scaling framework for CPU-GPU systems. The core idea of CoDVFS is to leverage Bayesian Optimization to efficiently approximate the optimal solution in high-cost experimental environments. This allows the system to discover the optimal CPU/GPU frequencies with few experiments.

When using this framework, it can be applied to any target program requiring optimization. CoDVFS will then automatically perform experimental evaluations and iteratively identify the best frequency configuration. The source codes of CoDVFS are available at https://github.com/zyjopensource/CoDVFS.

3.1 Optimization Objective

The objective of the optimization process is to maximize the energy efficiency, which is defined as the ratio of performance to power usage:

$$\text{Energy Efficiency} = \frac{\text{Performance}}{\text{Power}} = \frac{\text{Workload/Duration}}{\text{Energy/Duration}} = \frac{\text{Workload}}{\text{Energy}}.$$

According to this formula, when running the same workload (with a fixed computational demand), maximizing energy efficiency is equivalent to minimizing total energy consumption. During optimization on a server, performance, power, and runtime duration are all measured directly while the program is executing. Each experiment typically lasts tens of seconds to several minutes.

The input variables for the optimization process are the combinations of CPU and GPU frequencies $(f_{\text{CPU}}, f_{\text{GPU}})$. Since most processor hardware automatically adjusts voltage according to the selected frequency, it is sufficient to directly set the frequencies externally.

3.2 Reason of Using Bayesian Optimization

Bayesian Optimization is a global optimization method based on probabilistic models. It is particularly well-suited for optimizing functions that are expensive or time-consuming to evaluate. In our case, because AI computing servers are complex systems, the relationship between energy efficiency and CPU-GPU frequency combinations cannot be described with an explicit analytical formula unless complex modeling is done in advance. As a result, traditional methods such as derivation-based or gradient-descent approaches are not applicable for optimizing energy efficiency.

Furthermore, since each experiment is relatively time-consuming, naive brute-force search incurs a prohibitive computational cost. Bayesian Optimization avoids the need for an analytical formula by constructing a probabilistic model of the objective function. This model is then used to intelligently select the next sampling point in the search space, allowing the optimization process to efficiently approach the function's optimum with relatively few experiments.

3.3 CoDVFS Framework Architecture

The proposed CoDVFS framework is implemented in Python and comprises key modules: a Bayesian optimizer, workload runner, performance/power monitoring, and CPU/GPU frequency control (Fig. 1).

The Bayesian optimizer selects the next CPU-GPU frequency combination using historical data and probabilistic modeling to maximize energy efficiency. *The workload runner* executes the target program across frequency settings for performance measurements. *The performance monitoring module* evaluates workload execution based on program-reported metrics, and *the power monitoring module* collects real-time power data from meters. *Frequency control* leverages hardware-specific interfaces to adjust CPU/GPU frequencies.

CoDVFS features adaptable interfaces for different server setups, allowing quick integration with new platforms by modifying frequency control calls. This design ensures core optimization logic while enabling flexible task execution, monitoring, and coordinated CPU-GPU DVFS across servers, offering a unified solution for system-wide energy efficiency in real-world deployments.

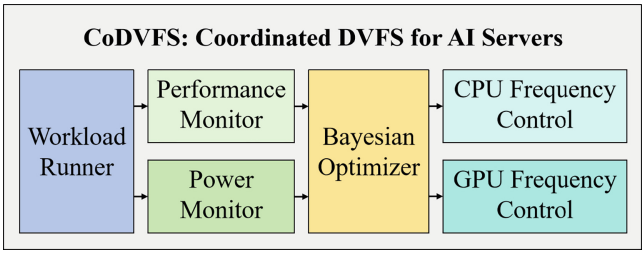


Fig. 1. The CoDVFS Framework.

3.4 Detailed Steps of Bayesian Optimization

Initialization of Sample Points. First, within the search space of CPU and GPU frequencies, one or more initial frequency combinations are randomly selected. For each combination, run the workload to collect performance/power results and compute its energy efficiency.

Surrogate Model Construction. Using the data from the initial sample points, a surrogate model is constructed to approximate the true objective function. A Gaussian Process (GP) is a commonly used surrogate model in Bayesian Optimization. It characterizes the distribution of the objective function using a mean function and a covariance function: $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x}, \mathbf{x}'))$, where $\mathbf{x}$ is the input variable (a CPU-GPU frequency combination $(f_{\text{CPU}}, f_{\text{GPU}})$), $f(\mathbf{x}) \sim \mathcal{GP}(\cdot, \cdot)$ is the Gaussian process, $m(\mathbf{x})$ is the mean function, $k(\mathbf{x}, \mathbf{x}')$ is the covariance function (also called the kernel), which describes the correlation between inputs $\mathbf{x}$ and $\mathbf{x}'$.

In Bayesian optimization, commonly used kernels include the RBF (Radial Basis Function) kernel and the Matern kernel. Our implementation uses the Matern kernel, which contains a few hyperparameters denoted as $\boldsymbol{\theta}$ here.

Sampling Next Point. The core of Bayesian optimization lies in the sampling of the next point. Based on the surrogate model, the next point is selected by optimizing an acquisition function. The acquisition function evaluates the value of each candidate point, which balances both exploration (gathering information about unknown areas) and exploitation (refining known good areas).

Common acquisition functions include Expected Improvement (EI) and Upper Confidence Bound (UCB). Our implementation uses EI, which is:

$$\text{EI}(\mathbf{x}) = \max(\mu(\mathbf{x}) - f^*, 0)\Phi(z) + \sigma(\mathbf{x})\phi(z), \tag{1}$$

where f^* is the current best energy efficiency value, $\mu(\mathbf{x})$ and $\sigma(\mathbf{x})$ are the predicted mean and standard deviation of the Gaussian process at input point $\mathbf{x}$, and $z = \frac{\mu(\mathbf{x}) - f^*}{\sigma(\mathbf{x})}$. Also, $\Phi(z) = \int_{-\infty}^{z} \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{t^2}{2}\right) dt$ is the cumulative distribution function (CDF) of the standard normal distribution, and $\phi(z) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{z^2}{2}\right)$ is the probability density function (PDF) of the standard normal distribution. $\max(\cdot, \cdot)$ is used to ensure the improvement is non-negative.

Based on function $\text{EI}(\mathbf{x})$, the next frequency combination is selected by maximizing the EI function: $\mathbf{x}_* = \arg\max_{\mathbf{x}} \text{EI}(\mathbf{x})$. According to (1), when $\mu(\mathbf{x})$ is much larger than f^*, EI is large, indicating a high expected improvement. Even if $\mu(\mathbf{x})$ is not much larger than f^*, a large $\sigma(\mathbf{x})$ can still yield a high EI, thereby encouraging exploration in regions with high uncertainty.

Model Update. After the new sampling point $\mathbf{x}_*$ is selected, an experiment is run to measure its energy efficiency value, and the result y_* is added to the

existing dataset. With this updated dataset, the Gaussian process (GP) model will be recomputed to improve its accuracy.

To update the GP model during each iteration, after obtaining the new sample point $(\mathbf{x}_*, y_*)$ and using the existing samples $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ along with the covariance function $k(\cdot,\cdot)$, the posterior distribution of the GP at any new input $\mathbf{x}$ can be updated as $f(\mathbf{x}) \sim \mathcal{N}\big(\mu_n(\mathbf{x}), \sigma_n^2(\mathbf{x})\big)$, where the posterior mean and variance are given by

$$\mu_n(\mathbf{x}) = k(\mathbf{x}, X)\big[k(X,X) + \sigma_{\text{obs}}^2 I\big]^{-1}\mathbf{y}, \tag{2}$$

$$\sigma_n^2(\mathbf{x}) = k(\mathbf{x},\mathbf{x}) - k(\mathbf{x}, X)\big[k(X,X) + \sigma_{\text{obs}}^2 I\big]^{-1}k(X,\mathbf{x}). \tag{3}$$

Here, X is the set of all previously sampled inputs, $\mathbf{y}$ is the corresponding vector of observations, I is the identity matrix, $\mu_n(\mathbf{x})$ and $\sigma_n^2(\mathbf{x})$ are the posterior mean and variance at input $\mathbf{x}$. Also, σ_{obs}^2 is the observation noise variance. In practice, the Matern kernel hyperparameters $\boldsymbol{\theta}$ and noise variance σ_{obs}^2 are periodically re-estimated via maximum likelihood estimation to better fit the data:

$$\{\boldsymbol{\theta}, \sigma_{\text{obs}}^2\} = \arg\max_{\boldsymbol{\theta}, \sigma_{\text{obs}}^2} \log p\big(\mathbf{y}_n \,\big|\, X_n, \boldsymbol{\theta}, \sigma_{\text{obs}}^2\big). \tag{4}$$

Iterative Optimization and Convergence. Repeat the above sampling and model updating steps, progressively searching for the optimal frequency combination. In each iteration, the updated model and acquisition function are used to choose a new sampling point, gradually converging toward the optimum of the objective function. After several iterations, when the value of the acquisition function becomes sufficiently small and the new sampling points offer diminishing improvements to the objective function, the optimization is considered to have converged. At this point, the frequency combination returned by Bayesian optimization is regarded as the optimal setting.

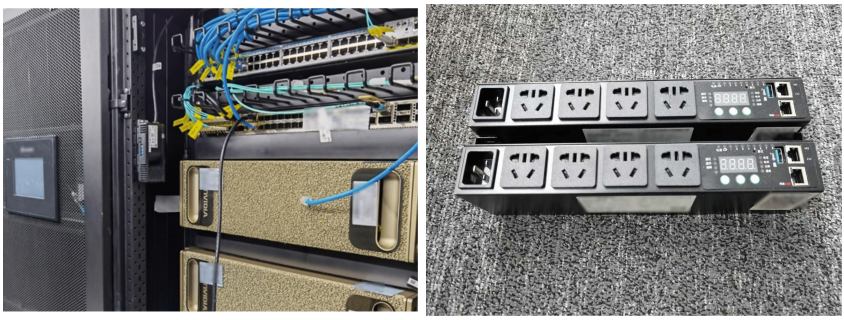
3.5 Computational Complexity

Assume the CPU offers M distinct frequency states and the GPU offers N, leading to a discrete design space $\mathcal{X} = \{(f_{\text{CPU}}^{(i)}, f_{\text{GPU}}^{(j)}) \mid 1 \le i \le M,\ 1 \le j \le N\}$ of size $K = MN$.

For the baseline exhaustive search, if each configuration is evaluated R times to suppress measurement noise, the baseline requires $C_{\text{exh}} = RK = RMN$ individual workload runs. With a single experiment taking wall time τ (tens of seconds to a few minutes in our servers), the cost is $C_{\text{exh}}\tau$.

For the Bayesian-optimization-based CoDVFS, let $T = T_0 + T_{\text{BO}}$ be the total number of experiments: T_0 random initial samples followed by T_{BO} Bayesian-guided iterations. Then, the experimental cost is $T\tau$, which is typically 10-40 runs in our deployments, two orders of magnitude smaller than C_{exh} when both M and N are greater than 10. The algorithmic overhead for each Bayesian optimization iteration, T_{alg}, is decomposed in the following:

- GP posterior update. Naively $O(t^3)$ with t samples due to the matrix inversion operation of the kernel matrix. If implemented with rank-one Cholesky updates, the amortised cost can be reduced to $O(t^2)$.
- Hyper-parameter re-estimation. Performed every H iterations (typically $H \geq 5$). Each call costs $O(t^3)$ but is infrequent.
- Acquisition maximization. For a discrete search we can enumerate only the unexplored points, costing $O(K - T)$ per iteration, or use a continuous optimizer whose complexity is $O(Ld)$ where L is the number of L-BFGS steps and d=2 is the dimensionality.



(a) AI Server used in experiments. (b) External power meters.

Fig. 2. Experimental devices.

Because $t \leq T \ll K$ and d is small, the algorithmic overhead T_{alg} is negligible compared with the cost of running experiments itself.

In summary, given $T \ll K$ and $\tau \gg T_{\text{alg}}$, the practical runtime of CoDVFS is dominated by $T\tau$, yielding a speed-up factor $\approx C_{\text{exh}}/T = RMN/T \sim 10^1 - 10^2$. As a result, CoDVFS reduces the number of costly workload executions by one or two magnitude in practice, making coordinated DVFS feasible for AI servers.

4 Experimental Methodology

This section describes the details of the hardware, workloads, power measurement, frequency control, and baseline DVFS policies used in our experiments.

4.1 Server and Workload

The server used in this study is a DGX V100, as shown in Fig. 2(a). It is equipped with two Intel Xeon E5-2698 v4 CPUs, providing a total of 40 physical cores and 80 threads, along with eight V100 GPUs. The GPUs run on CUDA 10.2. Each GPU consumes an idle power of ∼42 W, and the total idle power of all GPUs is ∼336 W. The entire server consumes ∼750 W in idle state.

To evaluate the effectiveness of our proposed CoDVFS framework, we used HPL-MxP [6] as the primary workload and ran it inside the official NVIDIA container. HPL-MxP is a representative hybrid-precision workload based on the traditional HPL benchmark, designed to leverage both CPU and GPU computing capabilities. It evaluates both double-precision computation in HPC scenarios and low-precision operations commonly found in AI workloads. In addition to HPL-MxP, we also tested the HPCG benchmark [11], which solves large sparse linear systems using iterative methods. We used the GPU version of these benchmarks released on the NVIDIA GPU Cloud platform.

After setting both the CPU and GPU set to their maximum frequencies, we conducted a grid search over the tunable parameters N (matrix size) and NB (block size) of HPL-MxP. This search is a standard procedure, which runs tests to identify the configuration that delivers the server's peak performance. After completing the grid search, we used matrix size $N = 204800$ and block size $NB = 896$ that yielded the highest performance. These two parameters were fixed in all subsequent energy efficiency optimization experiments, ensuring a fair comparison of energy performance across different control strategies under consistent workload conditions.

4.2 Server/GPU Power Measurement Method and Frequency Control Method

In this study, we measured the server's total power consumption using external power meters (model GWGJ), as shown in Fig. 2(b). This power meter supports TCP/IP-based communication via the SNMP protocol and can be controlled through Python scripts with the `pysnmp` library, enabling remote real-time monitoring and automated data logging. With a minimum valid sampling interval of approximately 0.5 s, it is capable of capturing fluctuations in dynamic server power consumption.

The DGX V100 server used in our experiments has four power supply units, and its nominal maximum power consumption is 3500 W. To cover the full range of server power draw, we deployed two power meters, each connected to two power supply units, effectively splitting the load. To obtain the total power, we summed the readings from both meters. To set up the devices, each power meter's input side was connected to the rack's power outlet, while the output side was connected to two of the server's power supply units.

A simplified version of our codes for power meter reading is provided below:

```
from pysnmp.entity.rfc3413.oneliner import cmdgen
from pysnmp.entity import engine
import time
from datetime import datetime

snmpEngine = engine.SnmpEngine()
cg = cmdgen.CommandGenerator(snmpEngine)

def get_power(dev_ip):
```

```
    errI, errS, errId, varBinds = cg.getCmd(
        cmdgen.CommunityData('pudinfo', 'public', 0),
        cmdgen.UdpTransportTarget((dev_ip, 161)),
        '1.3.6.1.4.1.23273.4.4.0'
    )
    return float(varBinds[0][1]) / 10

while True:
    t0 = time.time()
    p0 = get_power('192.168.242.38')
    p1 = get_power('192.168.242.39')
    ts = datetime.now().strftime('%Y-%m-%d %H:%M:%S.%f')
    print '%s,%.1f' % (ts, p0 + p1)
    time.sleep(max(0, 1 - (time.time()-t0)))
```

Here, "p0+p1" represents the total collected power. "192.168.242.x" are the IP addresses of our two power meters. "1.3.6.1.4.1.23273.4.4.0" is the power reading command specific to this power meter model. The sleep function maintains the reading interval to be 1 s.

For monitoring GPU power consumption, we used the `nvidia-smi` tool, which allows periodic readings of each GPU's power usage with good precision [26]. This data helped us analyze the individual power trends of the CPU and GPU and was also useful in later comparisons between GPU-only control strategies and coordinated control strategies. It is worth noting that, because measuring GPU power using `nvidia-smi` does not require deploying external power meters, it is much easier than measuring whole-server power. This is one of the main reasons why most prior GPU energy efficiency studies (as shown in Table 1) have focused only on GPU power measurements. Section 6 will compare whole-system energy efficiency with GPU-only energy efficiency in detail.

For frequency control, we used the `cpupower` tool to set CPU frequencies. The CPU frequency range was restricted between 1.2 GHz and 2.2 GHz, with 0.1 GHz intervals. For GPUs, we again used the `nvidia-smi` tool to set the operating frequency. The GPU V100 we use offers a tunable frequency range from 135 MHz to 1440 MHz, with two adjustable frequency points every 15 MHz, yielding a total of 175 frequency points. This provides a high degree of flexibility in tuning GPU frequency.

4.3 Comparison Strategies

In our experiments, we compare the proposed CoDVFS framework with several common DVFS strategies to validate its effectiveness:

- **Only-CPU-DVFS**: This strategy applies DVFS only to the CPU, while the GPU remains fixed at its maximum frequency.
- **Only-GPU-DVFS**: This strategy applies DVFS solely to the GPU, keeping the CPU fixed at its maximum frequency.

– **MaxFreq**: In this strategy, both the CPU and GPU operate at their maximum frequencies. This is the default configuration applied in many clusters with the aim of maximizing performance.
– **MinFreq**: This strategy sets both the CPU and GPU to their minimum frequencies. It helps measure the power control range of the server and is applicable in scenarios with strict power caps, such as when participating in electricity curtailment plans.

Our experiments mainly compare metrics including energy efficiency, performance, and average power. For performance, we directly read the floating-point computation performance reported by HPL-MxP and HPCG.

HPL-MxP Performance (TFlops)	GPU Frequency (MHz)										
CPU Freq. (GHz)	135	262	397	525	660	787	915	1050	1177	1312	1440
1.2	43.9	76.7	77.3	105	109	128	125	129	133	135	135
1.3	43.8	77.5	78.1	106	120	132	133	139	141	142	141
1.4	43.7	79	79	111	123	131	142	147	144	145	149
1.5	43.9	78.5	79.6	108	123	139	146	152	146	153	149
1.6	44	78.1	79.6	111	128	140	143	154	158	159	158
1.7	44.1	79.8	80.1	113	129	148	147	159	157	157	164
1.8	43.9	81.1	80	119	134	149	160	161	160	156	159
1.9	44.1	81.1	81.5	120	134	151	155	168	162	170	167
2.0	44.2	81.4	81.9	120	134	154	164	171	170	169	171
2.1	44.1	81.7	81.8	123	138	160	166	171	173	172	177
2.2	44.4	82.2	82	125	140	162	168	175	175	182	178

(a) Performance.

HPL-MxP Power (W)	GPU Frequency (MHz)										
CPU Freq. (GHz)	135	262	397	525	660	787	915	1050	1177	1312	1440
1.2	911	1000	1002	1112	1143	1211	1211	1281	1368	1467	1548
1.3	910	1001	1006	1112	1174	1225	1238	1317	1397	1491	1563
1.4	912	1006	1010	1129	1182	1227	1270	1345	1416	1506	1596
1.5	916	1010	1010	1121	1188	1248	1279	1360	1414	1533	1595
1.6	918	1012	1015	1132	1205	1257	1279	1371	1460	1553	1622
1.7	921	1014	1012	1140	1205	1281	1291	1389	1458	1546	1645
1.8	924	1022	1019	1155	1219	1281	1320	1394	1461	1536	1622
1.9	928	1027	1027	1164	1226	1289	1320	1421	1468	1581	1649
2.0	930	1028	1029	1159	1231	1302	1343	1434	1499	1586	1667
2.1	935	1035	1035	1177	1248	1329	1352	1434	1513	1601	1679
2.2	961	1061	1061	1207	1278	1360	1381	1469	1541	1643	1702

(b) Server power.

HPL-MxP Energy Efficiency (GFlops/W)	GPU Frequency (MHz)										
CPU Freq. (GHz)	135	262	397	525	660	787	915	1050	1177	1312	1440
1.2	48.2	76.6	77.2	94.8	95.5	105	103	101	97.2	92	87.2
1.3	48.1	77.4	77.6	95.4	102	107	107	105	101	95.2	90.3
1.4	47.9	78.5	78.3	98.5	104	107	111	109	101	96.1	93.2
1.5	47.9	77.8	78.8	96.2	104	111	114	112	103	99.7	93.3
1.6	47.9	77.2	78.4	97.9	106	111	112	112	108	103	97.1
1.7	47.8	78.7	79.2	99.3	107	116	113	114	107	101	99.7
1.8	47.5	79.3	78.5	103	110	116	121	116	109	101	97.8
1.9	47.5	79	79.3	103	110	117	118	118	110	107	101
2.0	47.5	79.2	79.5	104	109	118	122	119	113	106	103
2.1	47.2	79	79	105	110	120	123	119	114	108	105
2.2	46.2	77.4	77.3	103	109	119	122	119	113	111	105

(c) Energy efficiency.

Fig. 3. Performance, power, energy efficiency at different CPU-GPU frequencies.

5 Experimental Results

In this section, we first present the results from a coarse-grained DVFS brute-force search. Then, we show the results of experiments using CoDVFS.

5.1 Coarse-Grained CPU-GPU DVFS Brute-Force Search

To analyze the value of coordinated CPU-GPU DVFS, we first performed a coarse-grained brute-force search over CPU and GPU frequency combinations. In the experiment, we selected 11 frequency points for the CPU, ranging from 1.2 GHz to 2.2 GHz, and 11 frequency points for the GPU, ranging from 135 MHz to 1440 MHz. For each frequency pair, we ran the HPL-MxP workload five times, and the average of the five runs was taken as the final measurement value.

Figure 3 shows the results of this exhaustive search: Fig. 3(a) presents the measured performance of HPL-MxP in TFlops; Fig. 3(b) shows the average server power in watts, measured using the external power meters; Fig. 3(c) shows the energy efficiency in GFlops/W.

From Figs. 3(a) and (b), we observe that as the CPU and GPU frequency increases, both overall performance and power show a gradual increase. The highest performance and power are achieved when both CPU and GPU frequencies are set near their maximum values. At this point, server power reaches 1702 W, significantly higher than the 911 W observed at the lowest frequency combination. Although in practical scenarios it is uncommon to set both CPU and GPU to their minimum frequencies due to performance demands, this extreme case demonstrates that coordinated DVFS can regulate server power consumption across nearly a 50% range. This has practical significance for computing clusters whose total power is constrained, such as in power-capped environments.

Compared with the monotonic increases observed in power and performance, the energy efficiency results in Fig. 3(c) reveal more interesting patterns. Energy efficiency does not continuously increase with rising frequency. Instead, it reaches a peak value of 122.6 GFlops/W when the CPU frequency is set to 2.1 GHz and the GPU frequency to 915 MHz. This peak highlights the potential advantage of coordinated CPU-GPU DVFS.

These results show that if DVFS is applied to only the CPU or only the GPU, it may not discover this optimal frequency combination, and such single-component strategies are unlikely to achieve a comparable level of energy efficiency. These findings also emphasize that to fully exploit the potential of coordinated CPU-GPU control, it is essential for both the CPU and GPU to support a sufficient number of adjustable frequency levels. If only a small set of DVFS options is available, optimization accuracy will be greatly reduced, and the resulting energy efficiency may fall short of the theoretical optimum.

5.2 CoDVFS Optimization Experiment

In this experiment, we ran the CoDVFS framework to perform Bayesian optimization for coordinated CPU-GPU frequency control. Unlike the previous

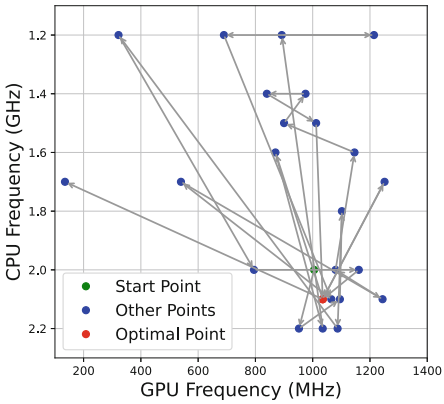


Fig. 4. Trajectory of Bayesian optimization.

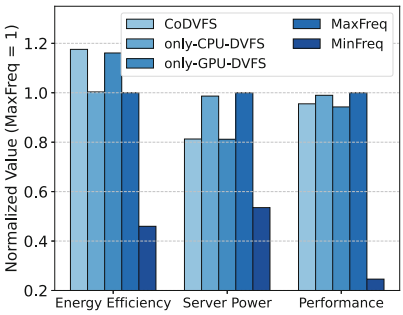
section where GPU frequency selection was limited to 11 points, here we expanded the GPU frequency search space to all 175 available frequency points, aiming to find the optimal solution in the complete frequency space.

Experimental results show that CoDVFS was able to determine the optimal frequency setting in just 20 iterations. For HPL-MxP, the obtained optimal setting is CPU at 2.1 GHz and GPU at 1035 MHz, resulting in a server energy efficiency of 123.2 GFlops/W. The optimization trajectory is illustrated in Fig. 4. It can be observed that the algorithm balances exploration and exploitation and eventually identifies the frequency combination with the best energy efficiency.

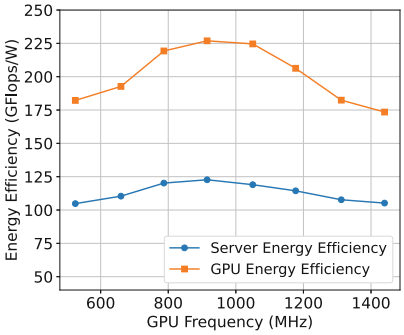
This result clearly demonstrates the efficiency of CoDVFS in coordinated DVFS control. In contrast, using an exhaustive search approach would require evaluating 11 CPU frequency points $\times$ 175 GPU frequency points $=$ 1925 combinations, which is impractical to execute on real-world systems. While experienced engineers may be able to manually reduce the number of tests by heuristically pruning the search space, manual tuning lacks scalability and automation, making it difficult to apply broadly in complex and dynamic cluster environments across various programs.

CoDVFS, by leveraging the efficient search mechanism of Bayesian optimization, can rapidly locate high-efficiency configurations even in large search spaces. This offers a practical and scalable solution for extending the framework to other deep learning or high-performance computing workloads.

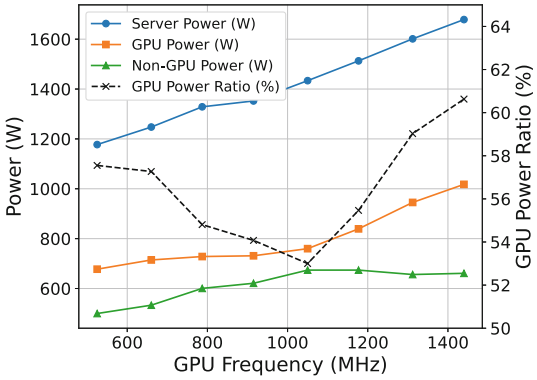
In Fig. 5(a), we compare this optimal point with the baseline strategies, with all values normalized against the MaxFreq strategy (i.e., both CPU and GPU set to their highest frequencies). The figure shows that while MaxFreq can shorten execution time to some extent, it significantly reduces energy efficiency. In contrast, under the CoDVFS strategy, energy efficiency improves by 17.5% compared to the MaxFreq baseline, while performance is only 4.3% lower, and average power consumption drops by more than 18%. Considering that many computing



(a) Comparison of DVFS strategies.



(b) Comparison of energy efficiency.



(c) Power of server components.

Fig. 5. Comparison of different DVFS strategies, and comparison of whole-server/GPU-only energy efficiency.

clusters default to running processors at their maximum frequencies, this result demonstrates the importance of coordinated CPU-GPU DVFS.

Furthermore, comparing with a scenario that applies CPU-only frequency optimization (CPU at 2.1 GHz, GPU at 1440 MHz), CoDVFS still achieves a 17% higher energy efficiency. When compared to GPU-only frequency optimization (CPU at 2.2 GHz, GPU at 915 MHz), CoDVFS improves energy efficiency by 1.2%. These comparisons indicate that while optimizing a single component can bring some energy efficiency gains, it is insufficient to fully exploit the joint potential of the CPU and GPU. Only by integrating DVFS strategies for both components can we achieve the maximum possible energy efficiency.

Similar results are observed for the HPCG benchmark. CoDVFS achieves an energy efficiency of 15.63 GFlops/W at the optimal point found within 20 iterations, representing a 12% improvement compared to MaxFreq's 13.93 GFlops/W, while the performance degrades by only 1%.

6 Discussion: Comparing Whole-Server Energy Efficiency and GPU Energy Efficiency

In previous work, it was often assumed by default that optimizing GPU energy efficiency is equivalent to optimizing server energy efficiency. In this section, we carefully analyze the differences between the two and provide a criterion for establishing their equivalence.

6.1 The Distinction Between Two Metrics

In an AI computing server, components such as the CPU, GPU, motherboard, fans, memory, storage, and network adapters collectively determine the total server power consumption. Therefore, server-level energy efficiency is defined as the ratio of application performance to the total power consumption of all components. In contrast, if we consider only the GPU, we are calculating the ratio of performance to GPU power, resulting in the GPU energy efficiency.

The distinction between the two can be clearly understood through the following formulas:

GPU energy efficiency = (Performance)/(GPU power),

Server energy efficiency = (Performance)/(GPU power + non-GPU power).

From the definition above, we also have:

Server energy efficiency = GPU energy efficiency × GPU power ratio,

GPU power ratio = (GPU power)/(GPU power + non-GPU power).

Clearly, when the GPU power ratio is constant, optimizing server energy efficiency is equivalent to optimizing GPU energy efficiency. However, in real-world environments, the GPU power ratio varies dynamically, so we cannot directly treat these two quantities as equivalent.

In fact, during coordinated DVFS, the frequencies and voltages of both the CPU and GPU are continuously adjusted, which not only affects their individual power consumption but also indirectly alters the power draw of other components such as fans, memory, and the motherboard. This variability makes the GPU power ratio no longer fixed.

In most prior research (see Table 1), due to experimental constraints, the focus has been solely on GPU energy efficiency, using tools like `nvidia-smi` to obtain GPU power data for analysis and optimization. However, comprehensively evaluating whole-server energy efficiency requires external power meters and power wiring reconfiguration to measure total consumption, which is often infeasible in real data centers. Consequently, research on whole-server energy efficiency has been relatively limited.

The problem of focusing on GPU energy efficiency is that the optimal point for GPU energy efficiency does not always coincide with the optimal point for server energy efficiency. Although the discrepancy was small in our experiments, there is no guarantee that the alignment of the two optimal points will hold true under different hardware configurations and workloads.

6.2 The Two Energy Efficiency Values in Our Experiments

In Table 2, we present the performance, GPU power, and non-GPU power collected while running the HPL-MxP program, as well as the corresponding GPU energy efficiency and server energy efficiency calculated from these values. In this table, the CPU frequency was fixed at 2.1 GHz (where the maximal energy efficiency is achieved), while the GPU frequency was gradually increased from 525 MHz to 1440 MHz. The power values listed are averages over the entire execution period, and the GPU power represents the total power of all 8 GPUs.

Table 2. Power and Energy Efficiency Metrics under GPU Frequency Scaling.

GPU Frequency (MHz)	525	660	787	915	1050	1177	1312	1440
Server Power (W)	1177.2	1247.6	1328.7	1352.2	1433.6	1513	1601.3	1678.6
GPU Power (W)	677.5	714.5	728.2	731.2	759.8	839.2	945.2	1017.7
Non-GPU Power (W)	499.7	533.1	600.5	621	673.8	673.8	656.1	660.9
Performance (TFlops)	123.4	137.7	159.7	165.9	170.7	173.1	172.4	176.6
Server Energy Efficiency (GFlops/W)	104.8	110.4	120.2	122.7	119	114.4	107.7	105.2
GPU Energy Efficiency (GFlops/W)	182.2	192.7	219.3	226.9	224.6	206.3	182.4	173.5
GPU Power Ratio	57.5%	57.3%	54.8%	54.1%	53.0%	55.5%	59.0%	60.6%
GPU Power Increase Rate	5.5%	1.9%	0.4%	3.9%	10.5%	12.6%	7.7%	NA
Performance Increase Rate	11.6%	15.9%	3.9%	2.9%	1.5%	−0.4%	2.4%	NA
Non-GPU Power Increase Rate	6.7%	12.6%	3.4%	8.5%	0.0%	−2.6%	0.7%	NA

From Table 2, we see the non-GPU power varied between 499.7 W and 673.8 W, while the GPU power varied between 677.5 W and 1017.7 W. The GPU power ratio (i.e., the proportion of GPU power to total power) ranged from 53.0% to 60.6%. This indicates that as GPU frequency increases, the power consumption of components other than the GPU also changes significantly, and it is inaccurate to assume a constant non-GPU power ratio.

In Fig. 5(b), we compare the curves of GPU energy efficiency and server energy efficiency as the GPU frequency varies. Across the sampled frequency points, the two efficiency metrics show a clear numerical difference, though both reach their respective peaks around 915 MHz. With a finer-grained frequency search, their actual optimal frequency points would not exactly coincide. How-

ever, since energy efficiency curves tend to be relatively flat near their extrema, this discrepancy would not result in significant performance or power loss.

In other words, in this setting, if we choose to use GPU energy efficiency as a reference, the resulting optimal frequency is likely to be close to the system-wide optimal frequency. From a practical application perspective, this represents a feasible approximation method.

6.3 The Criterion for Energy Efficiency Optimum Alignment

To further analyze the conditions under which the optimal points of GPU and server energy efficiency coincide, we present an analytical criterion.

Let performance be denoted as $f(x)$, GPU power as $g(x)$, and non-GPU power as $h(x)$, all of which are smooth functions of frequency x. Then, the GPU energy efficiency is defined as $U(x) = \frac{f(x)}{g(x)}$, and its extremum satisfies the condition $U'(x) = 0$, which is equivalent to $\frac{f'(x)}{f(x)} = \frac{g'(x)}{g(x)}$. The server energy efficiency is defined as $V(x) = \frac{f(x)}{g(x)+h(x)}$, and its extremum satisfies the condition: $\frac{f'(x)}{f(x)} = \frac{g'(x)+h'(x)}{g(x)+h(x)}$. From these, we can derive that the criterion for GPU energy efficiency and server energy efficiency to reach their optimal points at the same frequency is: $\frac{f'(x)}{f(x)} = \frac{g'(x)}{g(x)} = \frac{h'(x)}{h(x)}$.

The solution to this equation is the shared optimum point. In other words, we have the following **Optimum Alignment Criterion**: *if at the GPU's energy efficiency optimal point, the relative growth rate of GPU power is equal to the relative growth rate of non-GPU power, then this point is also the server's energy efficiency optimal point.*

In the last three rows of Table 2, we present the discrete differences between adjacent columns to approximate the growth rates of each quantity across frequency intervals. We observe that around 915 MHz, the growth rates of both performance and non-GPU power begin to decline, while the growth rate of GPU power increases rapidly, which is consistent with our criterion.

We suspect that the operational behavior of the server's cooling fans explains why non-GPU power initially increases significantly and later stabilizes as GPU frequency increases. In our experiment, the CPU frequency was fixed at 2.1 GHz, meaning CPU-side power remained relatively stable. However, as the GPU frequency rose, system heat output increased sharply, triggering an increase in fan speed, which in turn boosted non-GPU component power.

Once the fans approached their maximum speed, further increases in GPU frequency no longer caused fan power increases, leading to a plateau in non-GPU power, as shown by the green line in Fig. 5(c). This power trend is relatively common in servers using air-cooling technologies. If alternative cooling methods such as liquid cooling are adopted, the impact of the fans may be significantly reduced, which could also change the overall trend of non-GPU power. In such cases, the deviation between GPU energy efficiency and server energy efficiency may increase, necessitating further in-depth comparative study.

In summary, when comparing GPU and server energy efficiency, their two optimal points do not necessarily align under DVFS control, due to the non-constant GPU power ratio. However, in the workload and hardware configuration used in our experiment, the two optima were closely aligned, which validates the feasibility of using GPU energy efficiency (which is easier to measure) to approximate whole-server energy efficiency.

At the same time, it's important to recognize that in other setups, such as liquid-cooled servers, cooling power characteristics may differ significantly. This could widen the gap between GPU and server energy efficiency, underscoring the need for further study. Deeper understanding of this issue will contribute to more accurate system-level energy efficiency evaluation and optimization, and provide new insights for future research in green computing and thermal design.

7 Conclusion and Future Work

This study proposes CoDVFS, a CPU-GPU joint frequency control framework for AI server energy efficiency. Experiments on a DGX V100 show Bayesian optimization effectively finds optimal frequency combinations, achieving 17% higher energy efficiency than default settings under HPL-MxP workloads. While GPU-only and whole-server optima show minor differences, GPU efficiency serves as a reliable proxy. Future work includes finer-grained DVFS for multi-stage workloads, multi-task CPU-GPU coordination, and revisiting GPU-system efficiency alignment under liquid cooling.

Acknowledgments. This study was funded by Project No. 2024B0101010003.

References

1. Ali, G., Side, M., Bhalachandra, S., Wright, N.J., Chen, Y.: Performance-aware energy-efficient GPU frequency selection using DNN-based models. In: Proceedings of the 52nd International Conference on Parallel Processing, pp. 433–442. ICPP '23, Association for Computing Machinery, New York (2023)
2. Bratek, P., Szustak, L., Wyrzykowski, R., et al.: Reducing energy consumption using heterogeneous voltage frequency scaling of data-parallel applications for multicore systems. J. Parallel Distrib. Comput. **175**, 121–133 (2023)
3. Fan, K., Cosenza, B., Juurlink, B.: Predictable GPUS frequency scaling for energy and performance. In: Proceedings of the 48th International Conference on Parallel Processing. ICPP, New York (2019)
4. Guerreiro, J., Ilic, A., Roma, N., Tomás, P.: Multi-kernel auto-tuning on GPUS: performance and energy-aware optimization. In: Euromicro International Conference on Parallel, Distributed, and Network-Based Processing, pp. 438–445 (2015)
5. Guerreiro, J., Ilic, A., Roma, N., Tomás, P.: DVFS-aware application classification to improve GPGPUS energy efficiency. Parallel Comput. **83**, 93–117 (2019)
6. Haidar, A., Tomov, S., Dongarra, J., Higham, N.J.: Harnessing GPU tensor cores for fast fp16 arithmetic to speed up mixed-precision iterative refinement solvers. In: SC18: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 603–613 (2018)

7. hsing Hsu, C., chun Feng, W.: A power-aware run-time system for high-performance computing. In: SC '05: Proceedings of the 2005 ACM/IEEE Conference on Supercomputing, pp. 1–1 (2005)
8. Ilager, S., Muralidhar, R., Rammohanrao, K., Buyya, R.: A data-driven frequency scaling approach for deadline-aware energy efficient scheduling on graphics processing units (GPUS). In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), pp. 579–588 (2020)
9. Kandiah, V., et al.: Accelwattch: a power modeling framework for modern GPUS. In: MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture, pp. 738–753. MICRO '21, Association for Computing Machinery, New York (2021)
10. Kumar, S., Gupta, A., Kumar, V., et al.: Cuttlefish: library for achieving energy efficiency in multicore parallel programs. In: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14 (2021)
11. Kwack, J., Bauer, G.H.: HPCG and HPGMG benchmark tests on multiple program, multiple data (MPMD) mode on blue waters—a cray xe6/xk7 hybrid system. Concurr. Comput. Pract. Exper. **30**(1), e4298 (2018), e4298 CPE-17-0255
12. Majumdar, A., Piga, L., Paul, I., Greathouse, J.L., Huang, W., Albonesi, D.H.: Dynamic GPGPU power management using adaptive model predictive control. In: 2017 IEEE International Symposium on High Performance Computer Architecture (HPCA), pp. 613–624 (2017)
13. Mei, X., Wang, Q., Chu, X.: A survey and measurement study of GPU DVFS on energy conservation. Digit. Commun. Netw. **3**(2), 89–100 (2017)
14. Nabavinejad, S.M., Reda, S., Ebrahimi, M.: Coordinated batching and DVFS for DNN inference on GPU accelerators. IEEE Trans. Parallel Distrib. Syst. **33**(10), 2496–2508 (2022)
15. Patel, P., Choukse, E., Zhang, C., Goiri, I.n., Warrier, B., Mahalingam, N., Bianchini, R.: Characterizing power management opportunities for llms in the cloud. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 3, pp. 207–222. ASPLOS '24, Association for Computing Machinery, New York (2024)
16. Paul, I., Huang, W., Arora, M., Yalamanchili, S.: Harmonia: balancing compute and memory power in high-performance GPUS. In: Proceedings of the 42nd Annual International Symposium on Computer Architecture, pp. 54–65. ISCA '15, Association for Computing Machinery, New York (2015)
17. Piga, L., et al.: Expanding datacenter capacity with DVFS boosting: a safe and scalable deployment experience. In: Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, vol. 1, pp. 150–165. ASPLOS '24, Association for Computing Machinery, New York (2024)
18. Schoonhoven, R., Veenboer, B., Van Werkhoven, B., Batenburg, K.J.: Going green: optimizing GPUS for energy efficiency through model-steered auto-tuning. In: 2022 IEEE/ACM International Workshop on Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), pp. 48–59 (2022)
19. Stojkovic, J., Iliakopoulou, N., Xu, T., Franke, H., Torrellas, J.: Ecofaas: rethinking the design of serverless environments for energy efficiency. In: Annual International Symposium on Computer Architecture (ISCA), pp. 471–486 (2024)
20. TOP500.org: The TOP500 list. Nov 2024 Edition (2024). https://www.top500.org/lists/top500/, Accessed Mar 2025
21. Velicka, D., Vysocky, O., Riha, L.: Methodology for GPU frequency switching latency measurement (2025). http://arxiv.org/abs/2502.20075

22. Wang, F., Zhang, W., Lai, S., Hao, M., Wang, Z.: Dynamic GPU energy optimization for machine learning training workloads. IEEE Trans. Parallel Distrib. Syst. **33**(11), 2943–2954 (2022)
23. Wang, Q., Chu, X.: GPGPU performance estimation with core and memory frequency scaling. IEEE Trans. Parallel Distrib. Syst. **31**(12), 2865–2881 (2020)
24. Wang, Q., Li, L., Luo, W., Zhang, Y., Wang, B.: DSO: a GPU energy efficiency optimizer by fusing dynamic and static information. In: 2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), pp. 1–6 (2024)
25. Wang, Y., et al.: DRLCAP: runtime GPU frequency capping with deep reinforcement learning. IEEE Trans. Sustain. Comput. **9**(5), 712–726 (2024)
26. Yang, Z., Adamek, K., Armour, W.: Accurate and convenient energy measurements for GPUS: a detailed study of nvidia GPU's built-in power sensor. In: SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–17 (2024)
27. You, X., Yang, H., Xuan, Z., Luan, Z., Qian, D.: Powerspector: towards energy efficiency with calling-context-aware profiling. In: 2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 1272–1282 (2022)
28. Yue, A., Yew, P.C., Mehta, S.: Everest: an effective and versatile runtime energy saving tool for GPUS. In: Proceedings of the 30th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp. 57–69. PPoPP '25, Association for Computing Machinery, New York (2025)
29. Zhang, H., Nukada, A., Liao, Q.: FCUFS: core-level frequency tuning for energy optimization on intel processors. In: 2024 IEEE International Conference on Cluster Computing (CLUSTER), pp. 214–225 (2024)
30. Zhang, Y., Wang, Q., Lin, Z., Xu, P., Wang, B.: Improving GPU energy efficiency through an application-transparent frequency scaling policy with performance assurance. In: Proceedings of the Nineteenth European Conference on Computer Systems, pp. 769–785. EuroSys '24 (2024)
31. Zou, P., Li, A., Barker, K., Ge, R.: Indicator-directed dynamic power management for iterative workloads on GPU-accelerated systems. In: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), pp. 559–568 (2020)

Machine Learning and Data Statistics Based Scalability Bottleneck Detection

Yuhan Cao, Haoming Zhan, Juan Chen(✉), and Mingyuan Zhang

College of Computer Science and Technology, National University of Defense Technology, Changsha, China
{caoyuhan23,zhm441x,juanchen,zhangmingyuan}@nudt.edu.cn

Abstract. The continuously expanding problem scale of scientific computing applications has demanded massive parallel computing resources. However, the scalability bottlenecks of parallel programs severely constrain computational efficiency and performance gain. Current MPI parallel program scalability bottleneck detection faces three main issues: inaccurate prediction of the parallel performance inflection at the program level; difficulty in locating the specific bottleneck segments at the segment level; and low efficiency in identifying key performance events from massive data at the performance event level. To address the above issues, we propose *the machine learning and data statistics-based scalability bottleneck detection framework (POPP-SOPP-KER).* POPP builds a decision tree model with aggregation of segment features to improve prediction accuracy of performance inflection. SOPP uses type-aware feature selection to reduce deviations in segment-level optimal parallelism prediction, precisely locating bottleneck segments. KER recognizes the key events through KL divergence and mutual information correlation-based statistical method, improving the recognizing efficiency. Experimental verifications are conducted on two compute clusters (x86 and ARM, each with 2048 cores). POPP achieves an average accuracy of 70% (x86) and 60% (ARM) in performance inflection prediction, exceeding that of non-segmented methods. SOPP decreases prediction deviation of theoretical segment optimal parallelism by 13.2% (x86) and 11.88% (ARM) compared to using all features. KER effectively recognizes key features within 65 PMCs. By comparing key features based on performance inflection grouping, KER is proven beneficial. The proposed framework employs data-driven approaches for scalability bottleneck detection, providing insights for optimizing the scalability of large-scale applications.

Keywords: Scalability bottleneck · Optimal parallelism degree · MPI parallel program · performance optimization

1 Introduction

The potential of large-scale computing systems is often unrealized due to the poor scalability of parallel programs [31]. Poor scalability prevents parallel programs from achieving higher performance, slowing problem-solving and wasting

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 440–461, 2026.
https://doi.org/10.1007/978-981-95-8405-5_24

computing resources. Scalability bottlenecks stem from various aspects, such as communication overhead, load imbalance, and contention for shared resources [22]. Complex parallel programs exhibit behavioral changes during execution, leading to potential shifts in scalability bottlenecks. Detecting scalability bottlenecks in parallel programs has become a key issue in enhancing parallel computing efficiency.

Current research on the scalability bottlenecks of parallel programs mainly focuses on three aspects. In terms of *performance inflection prediction*, existing research mainly relies on performance models [5,30,36]. However, prediction accuracy tends to decrease when the parallel scale gets large [35]. In *identifying program bottleneck segments*, existing performance analysis tools like HPCToolkit [3] and TAU [29] offer capabilities for detecting scalability loss. However, Xuan et al. [33] note that current performance analysis tools still fail to pinpoint the root causes of scalability issues. *The mechanistic analysis* typically relies on manual expertise for optimization [20] or focuses on specific performance issues [4,21]. In scenarios with massive performance data, manual feature analysis is laborious, making automated methods essential.

This paper focuses on the scalability bottleneck appearance and bottleneck cause analysis of MPI (Message Passing Interface) parallel programs from program, segment, and performance event levels, as illustrated in Fig. 1. Assume the MPI parallel program runs at a certain parallelism. Three key issues are encountered as below:

(Issue 1) When Predicting the Performance Inflection at the Program Level, the Neglect of Segment Behavior Leads to Inaccurate Performance Inflection Predictions. Existing methods typically rely on global performance metrics [15] (e.g., total instruction count) to reflect program behavior. However, the scalability of a program is not determined by its "average" behavior, but by the behavior of its individual segments. The neglect of segment-specific implications may lead to inaccurate performance inflection predictions p^*_{app}, making it difficult to balance the program runtime and resource costs.

To analyze the causes of performance inflection, it is necessary to identify which program segments and performance events are responsible for the scalability bottlenecks.

(Issue 2) At the Segment Level, Accurately Locating the Specific Bottleneck Segments is Challenging Due to Significant Differences Among the Segments. The core of inflection cause analysis is to identify which segments of the program cannot efficiently utilize the current level of parallelism, thereby limiting the overall application scalability. However, due to the vast differences in communication or computation behaviors between program segments, accurately assessing their optimal parallelism p^*_{seg} is challenging, leading to poor bottleneck locating.

(Issue 3) At the Performance Event Level, Manually Analyzing the Vast Amount of Performance Event Data is Inefficient and Impractical. The final step in inflection cause analysis is to recognize key performance events to find the cause of the bottleneck. This requires analyzing the massive,

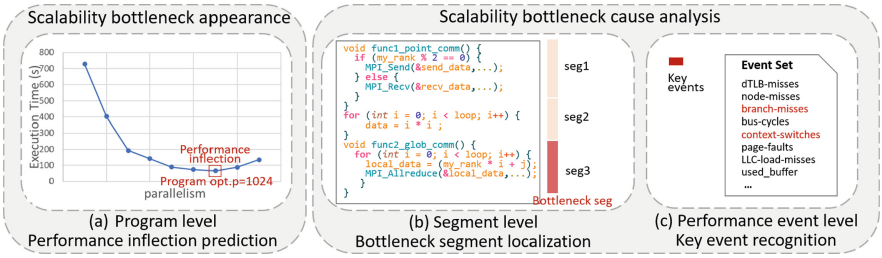


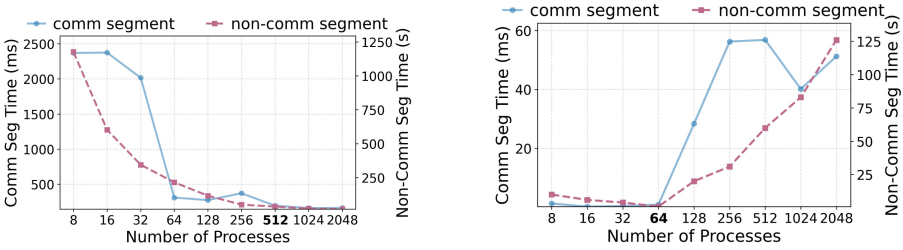
Fig. 1. Scalability bottleneck appearance and cause inference from program, segment, and performance event levels.

high-dimensional performance event data generated during program execution. Relying on manual analysis to search for causes, like memory bandwidth saturation, from terabytes of data is time-consuming, labor-intensive, and almost impossible to complete.

To address the above issues, we develop a scalability bottleneck detection framework based on machine learning and data statistics. The foundation lies in dividing parallel programs into comm and non-comm segments to leverage the segments' performance features. Specifically, these performance features serve to predict p^*_{app}. Due to disparities between comm and non-comm segments, the model input features are carefully differentiated to reduce prediction deviation for p^*_{seg} to accurately locate the bottleneck segments. By quantifying statistical metrics of performance data, key performance events are recognized efficiently in high-dimensional big-data scenarios.

To sum up, our main contributions are summarized as follows:

- For issue 1, POPP is proposed to improve the program performance inflection p^*_{app} prediction accuracy. POPP utilizes segment features to build decision tree prediction model.
- For issue 2, SOPP is proposed to predict optimal segment parallelism degree p^*_{seg}, which aids in accurately locating bottleneck segments. SOPP distinguishes input features for comm and non-comm segments based on resource needs in order to reduce prediction deviation
- For issue 3, KER is proposed to efficiently recognize the key events. Through quantifying data distribution differences by KL divergence and event correlations by mutual information, the key performance events of bottleneck segments and performance inflection are recognized.
- Experimental verifications are conducted on two compute clusters (x86 and ARM, each with 2048 cores). POPP exceeds non-segmented methods in performance inflection prediction with an average accuracy of 70% (x86) and 60% (ARM). SOPP decreases prediction deviation of theoretical segment optimal parallelism by 13.2% (x86) and 11.88% (ARM) compared to methods without feature selection. KER effectively recognizes key performance events within 65 PMCs. Group comparisons of key features based on performance inflection validate the effectiveness of KER.



(a) `LU (D)` (optimal parallelism 512) (b) `CG (C)` (optimal parallelism 64)

Fig. 2. The execution time of comm segment and non-comm segment in `LU (D)` and `CG (C)` varies with the parallelism degree. The segment features collectively determine the performance of the parallel program.

2 Motivation

2.1 Segment Features Affect MPI Program Optimal Parallelism

Choosing an inappropriate number of processor cores may negatively impact the performance of large-scale parallel programs. The prolonged execution times of large-scale computing tasks not only hinder progress in scientific and engineering research but also result in substantial energy waste and economic costs. However, the naive approach of repeatedly executing the application with different core numbers to find the optimal parallelism degree is prohibitively expensive in terms of both time and computational resources. Therefore, methods for accurately and rapidly predicting optimal parallelism need to be investigated to avoid this exhaustive testing process.

The program optimal parallelism is probably influenced by its segments' properties. Parallel programs exhibit distinct computation or communication characteristics at different phases during execution. As shown in Fig. 2, one comm segment and one non-comm segment of `LU (D)` (p^*_{app}=512) consistently decrease with increasing core number. In contrast, the two segments of `CG (C)` (p^*_{app}=64) exhibit significant time growth over 64 cores. However, most existing prediction methods rely on global performance metrics (total execution time and instruction count) [15]. Such metrics average all segment behaviors, making it hard to identify behavioral variances between segments with different attributes, resulting in low prediction accuracy.

2.2 Bottleneck Segments Affect the Performance Inflection Formation

Bottleneck segments are specific parts of a parallel program that experience severe performance degradation when the program-level optimal parallelism (p^*_{app}) exceeds their individual optimal parallelism (p^*_{seg}). p^*_{seg} is defined as the parallelism degree where a segment achieves its minimal execution time during

program runs across parallelism configurations. Crucially, when p^*_{seg} is substantially lower than p^*_{app}, that segment may suffer severe degradation at higher program-level parallelism. This localized performance collapse not only hinders overall program computing efficiency but forms scalability bottlenecks. *Such segments constitute the fundamental cause of performance inflection points.* However, obtaining p^*_{seg} through exhaustive testing is computationally expensive, making it necessary to predict it instead.

Existing prediction methods often suffer from large prediction deviations, resulting in unreliable location results. These methods typically utilize Performance Monitor Counters (PMCs) as features, selected through correlation analysis or empirical heuristics [27,32]. Their major limitation is adopting fixed feature sets universally, overlooking fundamental differences between compute-intensive and communication-intensive segments. Accurate prediction of p^*_{seg} requires input features that effectively reflect distinctions between computational and communication behaviors.

2.3 Cause Analysis of Scalability Bottleneck Depends on Key Performance Events

Key performance events refer to program behaviour events that affect the scalability of parallel programs, which are captured in performance profile data [8]. Recognizing key events often requires expertise in both software and hardware, typically accumulated through repeated execution and observation of programs on a specific platform. For instance, determining whether a program is memory-bound on a given architecture requires experience gained from analysis of extensive program execution. Furthermore, performance profile data has high dimensionality and large volume, which makes manual analysis inefficient. It is impractical for experts to examine hundreds of metrics individually to recognize key events reliably.

Effectively leveraging the performance event data to analyze bottlenecks remains a critical issue. The performance events arise from interactions across hardware, system, and application layers, each contributing collectively to performance and further complicating the identification process. Hardware features reflect physical operation efficiency, system features represent data access/management efficiency, and application features capture problem-solving efficiency.

3 Scalability Bottleneck Detection Framework

Figure 3 illustrates the machine learning and data statistics-based scalability bottleneck detection framework. The framework is based on the communication and non-communication segment features of MPI programs, with the following implementation steps. First, the SOPP method is proposed to predict the optimal parallelism of each program segment ($p^{'}_{seg}$) by type-aware feature selection, aiming to accurately identify the specific bottleneck segment (solving Issue 2).

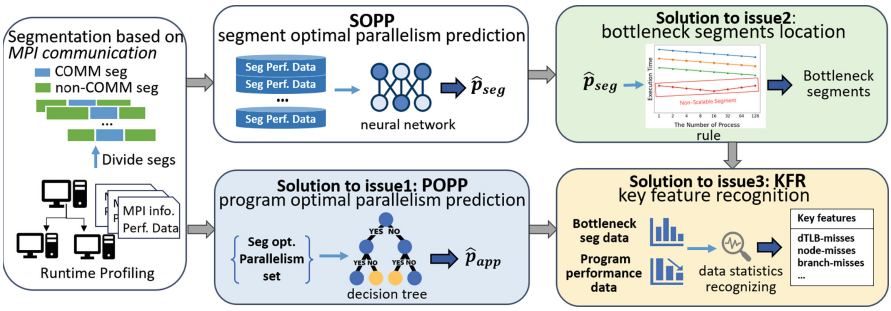


Fig. 3. Machine learning and data statistics based scalability bottleneck detection framework

Next, the POPP model is built to predict the performance inflection (p'_{app}) of the program through aggregating segment features, thus enabling rapid determination of the optimal core count (solving Issue 1). Finally, the KER method is applied to automatically analyze the causes of the bottleneck, replacing manual analysis (solving Issue 3).

3.1 Segmentation Algorithm Based on MPI Communication

Since communication characteristics significantly affect the performance of parallel programs, the program's execution is divided into comm segments and non-comm segments for subsequent performance bottleneck analysis. Communication time is often discontinuous on the program execution time axis and is determined by the parallel program algorithm. The communication time exhibits multiple short-duration characteristics on the program runtime timeline, with the communication start time determined by the algorithm; the non-communication periods are relatively long, during which the program performs computation and memory access.

The segmentation algorithm is based on MPI communication events, aggregating consecutive communication timestamps into segments, thereby obtaining comm segments and non-comm segments, as shown in Algorithm 1. First, if a timestamp has comm counts or comm volume, it is determined to be a comm timestamp. Then, consecutive comm timestamps are merged into comm segments, resulting in a comm segment list $COMMs$. Finally, the non-comm segment list $nonCOMMs$ is obtained by computing the complement of the comm segments on the program execution time axis.

3.2 SOPP: Type-Aware Feature Selection Based Theoretical Segment Optimal Parallelism Prediction

Selecting features for the optimal parallelism prediction model requires taking into account different communication patterns [25,32]. The core bottleneck of the comm segment may lie in global communication, where its eigenvectors should

Algorithm 1. Segmentation algorithm based on MPI communication

Require: parallel program performance dataset D, $d_i \in D$ is composed of communication indicators with timestamp $t \in [1, n]$: $count_{comm}$ and $volume_{comm}$
Ensure: comm segments $COMM_s$ and non-comm segments $nonCOMM_s$

```
iscomm[t] ← False                                   ▷ Initialize comm timestamp
for t = 1 to n do
    if count_comm ∨ volume_comm then
        iscomm[t] ← True
    end if
end for
COMM_s ← ∅, start ← None                            ▷ Initialize comm segments
for t = 1 to n do
    if iscomm[t] ∧ start = None then
        start ← t
    else if ¬iscomm[t] ∧ start ≠ None then
        end ← t − 1
        COMM_s ← COMM_s ∪ {(start, end)}
        start ← None
    end if
end for
if start ≠ None then
    COMM_s ← COMM_s ∪ {(start, n)}
end if                                              ▷ Obtain the comm segs
nonCOMM_s ← Complement(COMM_s)                      ▷ Obtain the non-comm segs
return COMM_s, nonCOMM_s
```

prioritize metrics like comm time and data volume (x_comm). While the non-comm segment prioritizes computation and memory access features (x_comp). The feature vector of one program x consists of n_1 comm-related features and n_2 computing-related features, with the vector length $n = n_1 + n_2$. The differentiation selection is as follows: $n_1 \geq n_2$ for comm segments, that is, choosing more comm-related features, while $n_1 = 0, n_2 > 0$ for non-comm segments.

The theoretical optimal parallelism prediction of segments is based on the idea of using small-scale data to predict large-scale configurations [7]. The theoretical optimal parallelism of segments represents the performance mutation of specific segments. The Artificial Neural Network (ANN) structures have the natural advantages in solving nonlinear mapping problems based on feature data, which can be used to solve the problem of capturing performance mutations. First, organize the segment's q-dimensional feature data (the specific q features depend on the segment type) under p small parallelisms into a $p \cdot q$ feature vector as training input data; then, use an artificial neural network model to learn the correlation between the feature vector and the optimal parallelism. The algorithm flow is shown in Algorithm 2.

Algorithm 2. SOPP: theoretical optimal parallelism prediction of segments

Require: Segment feature set x_set, optimal parallelismlabel set p^*_set, segment type classifier $classifier$.

Ensure: Target segment optimal parallelism predicted value $\hat{p}_{seg}$

1: $type = classifier(x_set^{target}), type \in \{0, 1\}$ ▷ t=0 represents non-comm-seg, t=1 represents comm-seg
2: **if** $t = 0$ **then**
3: **return** non-comm feature set x_set^{non}
4: **else**
5: **return** comm feature set x_set^{comm}
▷ Feature selection based on target segment type
6: **end if**
7: $p^*_set = ANN(x_set^{non}, train)$ or $p^*_set = ANN(x_set^{comm}, train)$
8: $\hat{p}_{seg} = ANN(x_set^{target}, predict)$
9: **return** $\hat{p}_{seg}$

The SOPP prediction values are used to locate bottleneck segments. Locating bottlenecks in parallel programs requires comparing the theoretical optimal parallelism of a segment (p^*_{seg}) to the program optimal parallelism (p^*_{app}). Here, p^*_{seg} represents the maximum number of processor cores that the segment can effectively utilize to reach its top performance, while p^*_{app} indicates the parallelism for best program performance. A larger difference between p^*_{seg} and p^*_{app} indicates that the segment is less suited for high parallelism. The bottleneck segment locating rule is shown as Eq. 1: the logarithmic difference between the p^*_{seg} and p^*_{app} is greater than zero.

$$\log_2(p^*_{app}) - \log_2(p^*_{seg}) > 0 \tag{1}$$

3.3 POPP: Program Optimal Parallelism Prediction Based on Aggregation of Segment Performance Features

Segment performance features reflect local program properties. Selecting the theoretical optimal parallelism of segments as input features quantifies the local scalability of parallel programs, providing a more accurate basis for predicting parallelism configurations.

Since different programs often contain different numbers of program segments, using raw features may directly result in inconsistent input dimensions, making them unsuitable for model training. To address the issue of inconsistent segment counts, the POPP algorithm applies zero-padding to construct fixed-length feature vectors by standardizing all feature vectors to the same length. Then, the input feature set and labels are organized by aggregating each fixed-length feature vector to form the complete input feature set for the parallel program set. The true optimal parallelism degree of each parallel program is used as the training label, corresponding to its fixed-length feature vector.

Algorithm 3. POPP: program optimal parallelism prediction

Require: the number of parallel programs m, the set of the theoretical optimal parallelism of n segments of each program $X = \{x_1, \ldots, x_n\}$, optimal parallelism p_i of i_{th} program, max segment count N of one program

Ensure: program optimal parallelism predicted value $\hat{p}_{app}$

1: **if** $n < N$ **then**
2: $\{x_{n+1}, \ldots, x_N\} = 0$, $X = \{x_1, \ldots, x_N\}$ ▷ uniform input length
3: **end if**
4: $training_set = \{(X_1, p_1), \ldots, (X_m, p_m)\}$ ▷ build training set
5: $\hat{p}_i = TreeRegression(training_set, train)$ ▷ Train decision tree regression model
6: $\hat{p}_{app} = TreeRegression(target_set, predict)$
7: **return** $\hat{p}_{app}$

Based on this collection of feature vectors and the corresponding optimal parallelism labels at the program level, a decision tree regression model is trained, as illustrated in Algorithm 3. The decision tree regression model automatically determines which features are most critical for prediction, capturing the nonlinear relationship between segment-level parallelism and overall performance. The goal of model training is to minimize the deviation between predicted and actual parallelism, thereby improving prediction accuracy.

3.4 KER: Key Event Recognition Based on Data Statistical Methods

Data statistical methods are ideal for recognizing key performance events due to their quantifiability and ability to handle complex, high-dimensional data. Key event recognition focuses on two aspects: *during program execution*, anomalous events within bottleneck segments are detected by comparing their distributions against the program's overall distribution; *across different degrees of parallelism*, the events most strongly correlated with changes in execution time are identified.

It is worth noting that the key performance events identified by KER are typically abnormal performance events that cause performance bottlenecks, while the feature selection method of SOPP focuses on selecting features for improving the accuracy of the prediction model. The performance features selected by the two methods are independent and not directly related.

Key Event Recognition for Bottleneck Segments Based on KL Divergence. The core idea is to detect discrepancy in performance event distributions between bottleneck segments and the overall program profile. Specific root causes of bottlenecks (like resource contention) typically manifest as abnormal patterns in key metrics (e.g., cache hit rate). By comparing these distributions, anomalous events indicative of bottlenecks can be directly identified.

The Kullback-Leibler (KL) Divergence is selected as the discrepancy metric. KL Divergence requires no prior assumption about the underlying data distribution, making it broadly applicable to diverse performance metrics. Its principle involves quantifying the information difference when approximating the *bottleneck segment distribution (P)* with the *global distribution (Q)*. A larger KL value indicates a greater discrepancy. The calculation formula is shown in Eq. 2. Ultimately, by independently computing the KL Divergence values for each event and ranking the results, the key events with the most pronounced distribution anomalies can be automatically recognized.

$$D_{KL}(P \parallel Q) = \sum_{x \in X} P(x) log \frac{P(x)}{Q(x)} \tag{2}$$

x represents the event value in the sampled data. $P(x)$ or $Q(x)$ denote the probability of the event taking value x in the bottleneck segment or during the program execution. X indicates the set of all possible event values.

Key Event Recognition for Performance Inflection Based on Mutual Information. The core idea is analyzing the correlation between various performance events and the execution time of parallel programs under different parallelism degrees. By quantifying the correlation between events and execution time, the core factors causing performance degradation can be pinpointed.

Mutual Information (MI) is chosen as the correlation metric. MI captures any statistical dependency, making it ideal for analyzing parallel program performance variations. Crucially, unlike Pearson correlation, MI quantifies non-linear and non-monotonic relationships (e.g., periodic fluctuations) between events and execution time. The MI value $I(X;Y)$ measures how much knowing event X reduces uncertainty about execution time Y. A higher MI value indicates a stronger association, meaning event X more significantly explains variations in performance (Y), as shown in Eq. 3.

$$I(X;Y) = \sum_{x \in X} \sum_{y \in Y} p(x,y) \log(\frac{p(x,y)}{p(x)p(y)}) \tag{3}$$

X and Y represent the discrete sets of values for event X and execution time Y. $p(x)$ and $p(y)$ represent the marginal probability distributions of X and Y, while $p(x,y)$ represents the joint probability distribution of X and Y.

4 Experiments

4.1 Experimental Design

Validation of Scalability Bottleneck Appearance Prediction. The optimal parallelism prediction employs a stepwise validation approach. First, a baseline Artificial Neural Network (ANN) model is established, using all available performance metrics collected from program runs at small-scale parallelism (4,

Table 1. Platform Specifications

	x86 Platform	ARM Platform
CPU Name	Intel Xeon Platinum 8358P	ARMv8-based
Frequency	2.6 GHz	2.2 GHz
#Sockets	2	8
#Cores	64	64
RAM	512 GB	128 GB
Thread(s)/core	1	1
#Nodes	32	32
#total cores	2048	2048

8, 16 cores) to predict segment optimal parallelism or the program optimal parallelism. Next, the effectiveness of the Segment Optimal Parallelism Prediction (SOPP) is verified by comparing its predictions against this baseline. Finally, the Program Optimal Parallelism Prediction (POPP)'s effectiveness is validated using the segment optimal parallelism values (p^*_{seg}) as input features.

Validation of Scalability Bottleneck Cause Inference. First, verify the accuracy of bottleneck segment identification: compare the baseline model and the SOPP, analyzing their effects in identifying bottleneck program segments. Second, verify the effectiveness of key event recognition: group segments and parallel programs according to their optimal parallelisms. If similar events are recognized for programs within the same group, the key event recognition method is validated as effective.

Experimental Environment. The above methods are validated on x86 and ARM computing clusters (64 cores per node), with hardware configurations specified in Table 1. Each cluster includes 32 nodes, with 2048 cores supported.

Three benchmark suites are used: `NPB` [26], `HPCG` [18], and `Graph500` [14]. Each process occupies one core by default. PMC, system load, and memory usage metrics are collected by Linux `perf`, `top`, and `free` on both platforms. Communication events are captured via the `mpiP` tool [1].

In the training process of SOPP and POPP models, the feature sets from programs other than the target program are used as training data. SOPP employs a three-layer ANN structure. The input layer has a feature dimension ranging from 90 to 120, while the two hidden layers contain 128 and 100 neurons, both utilizing the ReLU activation function. The dataset for SOPP is randomly split into training and test sets in a 9:1 ratio for model training and evaluation. For the POPP model's decision tree regression algorithm, a grid search is conducted on the tree depth hyperparameter, and the model with the optimal predictive performance is selected for final prediction.

Prediction accuracy and prediction deviation are metrics for evaluating the optimal parallelism prediction. Prediction accuracy measures the proportion of programs for which the optimal parallelism is correctly predicted, out of the total number of test programs. Prediction deviation measures the prediction effect of each sample, which is calculated as the absolute logarithmic difference between predicted and actual values, as shown in Eq. 4 (e.g., the deviation between 256 and 512 is 1).

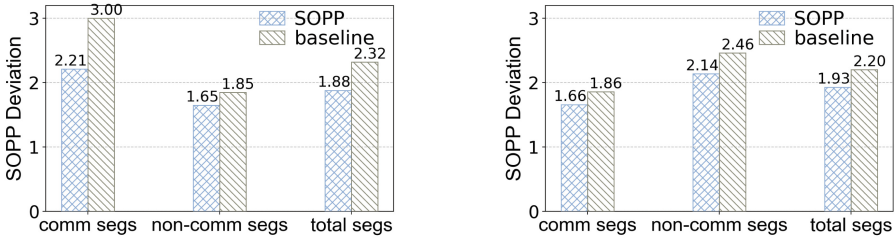
$$deviation = | \log_2 (actual_value) - \log_2 (predicted_value) | \tag{4}$$

4.2 The Bottleneck Segment can be More Accurately Identified Based on the SOPP Prediction

SOPP Accuracy and Deviation. Based on the SOPP feature selection method based on segment types, specific input features of comm and non-comm segments are chosen through multiple runs of the prediction model. Only those features that contribute to improving predictive accuracy are retained. SOPP selects 20 input features for comm segments, including 12 communication features and 8 others, such as branch instructions and CPU migration. Non-comm segments have 30 input features, including features related to branch instructions, CPU state, cache/memory access, and page tables. The specific input features are selected by experience. The baseline method uses all 65 features without feature selection.

As shown in Fig. 4, SOPP outperforms the baseline in prediction deviation and accuracy. The average deviation of SOPP on the x86 platform is 1.88, a 13.2% decrease compared to the baseline. On the ARM platform, SOPP reduces the average deviation by 11.88% compared to the baseline, with the average deviation being 1.93. SOPP improves the prediction accuracy by 11.07% and 8.09% on the x86 and ARM platforms compared to the baseline. From the analysis of program segment types, SOPP demonstrates more accurate predictions for non-comm segments on the x86 platform, with a lower prediction deviation (1.65) compared to comm segments (2.21). On the ARM platform, the opposite is observed: SOPP shows a lower prediction deviation for comm segments (1.66) than for non-comm segments (2.14). These results indicate that both the hardware platform and program segment type influence the predictive performance of the SOPP model, highlighting considerable room for improvement in prediction capability across different architectures and segment types.

The Accuracy of Bottleneck Segment Identification. Table 2 shows that SOPP improves the accuracy of localizing program bottleneck segments. If the optimal parallelism predicted by the baseline method (e.g., 1024 for seg1) is used for judgment, this value is significantly higher than the program's optimal parallelism (512), leading to the erroneous classification of seg1 as a non-bottleneck segment, even though it is actually a bottleneck. In contrast, the SOPP method predicts an optimal parallelism (e.g., 128 for seg1) closer to the actual optimal



(a) Deviation on x86 Platform

(b) Deviation on ARM Platform

Fig. 4. SOPP has a lower deviation for comm segments and non-comm segments than baseline.

Table 2. The bottleneck segment located based on the SOPP prediction is more accurate than the baseline.

	Program	p^*_{app}	p^*_{seg}	SOPP Value	Baseline
seg1	`LU (D)`	512	256	128	1024
seg2	`CG (C)`	64	8	32	128
seg3	`EP (C)`	64	8	32	64

parallelism of the segment (256), correctly identifying seg1 as a bottleneck segment. Similarly, for seg2 and seg3, SOPP's predictions are closer to the actual values than the baseline's, resulting in more accurate identification of bottleneck segments.

4.3 POPP Improves Prediction Accuracy Compared to the Baseline

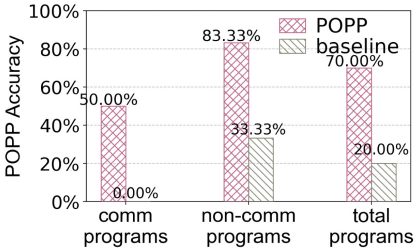
POPP Accuracy and Deviation. Table 3 shows the results of POPP and baseline. The baseline method doesn't consider segmentation information. The prediction accuracy reaches 70% and 60% on ×86 and ARM platforms, higher than the baseline (20%, 30%). Especially for programs with an optimal parallelism of 64 on the ×86 platform, POPP accuracy can reach 100%. It can be seen that POPP performs more poorly in predicting programs with significantly lower or higher than optimal parallelism on the ×86 platform. The main reason is the scarcity of such extreme samples.

As shown in Fig. 5, the POPP method outperforms the baseline in both prediction accuracy and deviation for both communication-intensive and non-communication-intensive programs. Communication-intensive programs are identified based on the percentage of global communication time. Four programs and five programs are classified as communication-intensive on the x86 and ARM platforms, respectively.

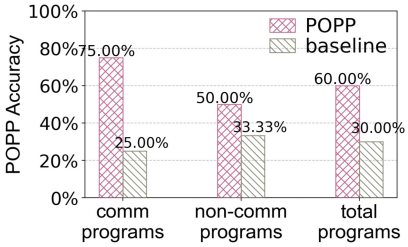
POPP shows a significant improvement in prediction accuracy when handling comm-intensive programs. On the x86 platform, POPP achieves 50% accuracy

Table 3. POPP prediction accuracies reach 70% and 60% on ×86 and ARM platform, exceeding baseline (20%, 30%). Accurately predicted values are in bold. 21, 22, C, or D denote the problem size.

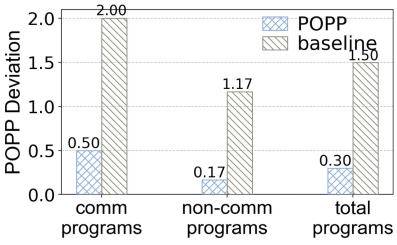
Application	#COMM Seg		#nonCOMM Seg		Actual value		POPP value		Baseline value	
	×86	ARM	×86	ARM	×86	ARM	×86	ARM	×86	ARM
BFS (21)	1	1	2	2	64	64	**64**	**64**	128	512
BFS (22)	1	2	2	3	64	256	**64**	64	128	128
LU (C)	2	2	2	2	64	128	**64**	256	**64**	32
LU (D)	2	2	2	3	512	512	256	**512**	256	**512**
EP (C)	1	1	2	2	64	32	**64**	128	8	8
EP (D)	2	2	3	3	64	256	**64**	**256**	32	**256**
CG (C)	1	2	2	3	64	32	**64**	**32**	16	128
CG (D)	3	3	3	4	1024	512	512	**512**	64	64
IS (D)	2	3	3	4	64	64	**64**	256	**64**	128
HPCG	2	4	2	4	256	256	512	**256**	64	**256**



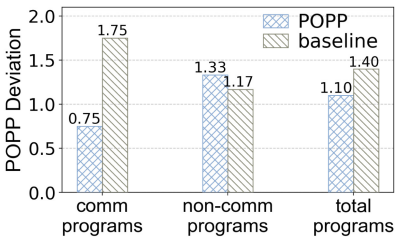
(a) Accuracy on x86 Platform



(b) Accuracy on ARM Platform



(c) Deviation on x86 Platform



(d) Deviation on ARM Platform

Fig. 5. POPP has a higher prediction accuracy and lower deviation for comm-intensive programs and non-comm-intensive programs than baseline.

for comm-intensive programs, while the baseline fails to accurately predict any of the four comm programs. On the ARM platform, POPP achieves a 75% accuracy rate for comm-intensive programs. For non-comm-intensive programs, the

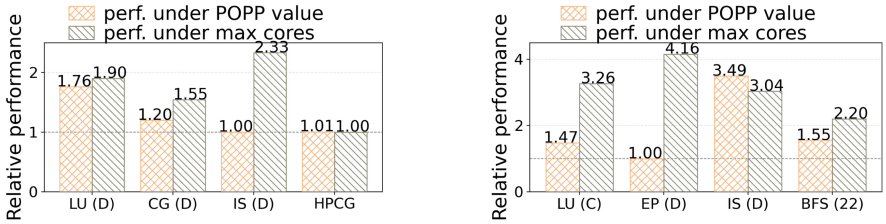
prediction accuracy on the ×86 platform is improved by 1.5 times compared to the baseline, while it improves 50% on the ARM platform. In terms of prediction deviation, the average deviation of POPP on the ×86 platform is only 0.30, significantly lower than the baseline method's 1.50. On the ARM platform, POPP reduces the average deviation by 21.43% compared to the baseline. For comm-intensive programs, on the x86 platform, POPP's deviation is 0.50, only 25% of baseline. On the ARM platform, the prediction deviation is 0.75, a 57.14% reduction compared to the baseline method. For non-comm programs, POPP's deviation on the x86 platform was only 0.17, but on the ARM platform, the deviation was slightly higher than the baseline method.

Performance Effect of POPP Predicted Parallelism. The performance (performance denotes parallel program execution time) under POPP prediction reaches an average of 84.75% and 75.00% of the actual optimal performance on x86 and ARM platforms. Figure 6 illustrates *relative performance* for typical programs under predicted optimal parallelism ($\hat{p}_{app}$, POPP) and using maximum platform core ($p_{\max}$, baseline) , normalized against performance under actual optimal parallelism (p^*_{app}, best). The smaller the relative performance of POPP than that at max cores and the closer the relative performance is to one, the better the performance improvement effect. If the relative performance is equal to 1, it indicates accurate prediction ($\hat{p}_{app} = p^*_{app}$).

The relative performance of POPP is close to the actual optimal parallel performance, with significant gains over using max cores. The average performance improvements over baseline under POPP value are 45.43% and 36.3% on ×86 and ARM platforms; the average differences from optimal performance are 24.1% and 87.9%. This shows that on the x86 platform, the performance improvement under POPP is greater, and the gap to the optimal performance is smaller. On the ARM platform, the average performance under POPP shows much worse than the average optimal performance, mainly due to the large performance fluctuations of the `IS (D)` program with varying numbers of processes. For the `IS (D)` program, the execution time under POPP is 185 s, while the fastest execution time is 53 s, a difference of more than double.

4.4 KER Can Effectively Recognize Key Events

To validate the effectiveness of the key event recognition method, the experiment employs a grouped comparison strategy. For bottleneck segments, the differences of key events between the severe and the general bottleneck groups are compared; for performance inflections, the differences of key events between the high-p^*_{app} and the low-p^*_{app} program groups are analyzed. If both experimental comparisons show significant differences in event distribution between groups (e.g., specific events are more prevalent in the more severe bottleneck group), it will demonstrate that this method can effectively identify key events.



(a) x86 platform (b) ARM-based platform

Fig. 6. Relative performance is the performance normalized against the best performance. The closer the relative performance is to one, the better the performance improvement.

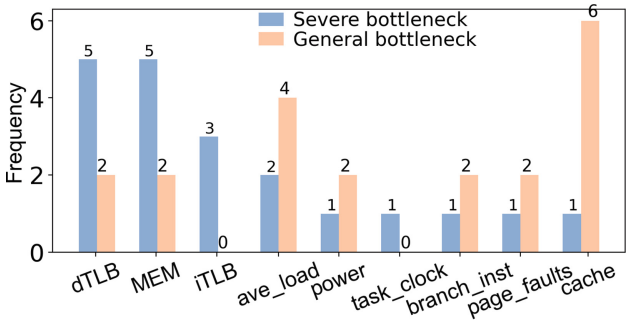


Fig. 7. The performance events of severe bottlenecks are different from the general bottlenecks, validating the effectiveness of the key event recognition for bottleneck segments.

Key Event Recognition for Bottleneck Segments. Bottleneck segments are classified as severe or general based on the deviation between segment-level (p^*_{seg}) and program-level (p^*_{app}) optimal parallelism. Using the `NPB` (x86) as examples to analyze, `CG (D)` and `LU (D)` show severe bottlenecks (deviations: 7, 6), while `IS (D)` and `EP (C)` exhibit general bottlenecks (deviations: 1, 2).

There are significant differences between the performance events of the two groups, validating the effectiveness of the key event recognition for bottleneck segments. Figure 7 compares the count of key events about different aspects recognized in the two groups separately. Severe bottlenecks primarily stem from systemic issues (the front events), like frequent TLB misses (dTLB/iTLB) and memory access events (MEM), linked to address mapping and memory efficiency. General bottlenecks mainly reflect software issues (the behind events), like cache inefficiencies and branch mispredictions (branch_inst), often caused by algorithmic limitations or poor data locality.

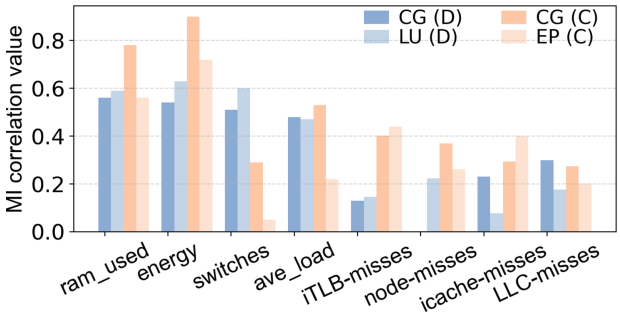


Fig. 8. The correlation value between events and performance with mutual information.

Key Event Recognition for Performance Inflection. Figure 8 shows the mutual information correlation values between multiple events and performance for four programs in `NPB` (x86). A higher mutual information value indicates a stronger correlation between the event and performance. `CG (D)` and `LU (D)` exhibit high p^*_{app} (1024, 512), respectively; `CG (C)` and `EP (C)` show low p^*_{app} (64, 64).

ram_ used (memory usage) and *energy* (energy consumption) show a strong positive correlation with performance across all four programs, particularly `CG (C)` and `EP (C)`. This suggests that programs with low p^*_{app} may be resource-constrained, either by memory or other factors. *Switches* refers to the number of CPU context switches. A higher correlation (0.51 and 0.60) is observed in the two more scalable programs, indicating that `CG (D)` and `LU (D)` may suffer from significant context switching overhead.

The last four events are more relevant in low p^*_{app} programs. These four events, which are iTLB misses, NUMA local memory access misses, icache misses, and LLC misses, reflect key events of `CG (C)` and `EP (C)`. (1) Instruction and data locality issues: high iTLB misses and icache misses (0.44 and 0.400 for `EP(C)`) indicate low instruction fetch efficiency. (2) NUMA effects: high node misses (0.369 for `CG(C)`) reflect high overhead from cross-node memory access, potentially becoming a bottleneck.

The mutual information correlation values reflect different key events of two program types, validating the effectiveness of key event recognition for performance inflections. Programs with low p^*_{app} (`CG (C)` and `EP (C)`) are more likely to be constrained by resource limitations (memory/energy) and architectural bottlenecks (instruction/data locality, NUMA effects).

4.5 Discussion

The scalable bottleneck detection framework excels at accurately and efficiently predicting performance inflection and analyzing the causes of bottlenecks. The model assists users in tuning parallel execution strategies to optimize program performance.

The framework still offers opportunities for enhancement in specific areas. First, the framework is currently applied to a limited number of benchmark programs. In the future, it is considered to apply this framework to large-scale scientific computing applications for validation and iteration. Second, the predictive ability of POPP and SOPP still needs improvement. E.g., the accuracy of POPP for rare samples, like programs with exceptionally low or high performance inflections, is lower than others. For the bottleneck identification mechanism, the SOPP deviation may still introduce imprecision. Third, the KER methodology may exhibit sensitivity to performance event fluctuations under certain conditions. Future work will focus on these aspects to improve prediction consistency and reliability.

5 Related Work

Performance modeling is essential for optimizing parallel programs. PMCs are widely used for performance models. Ding et al. [12] propose the APMT framework, which utilizes hardware counters to identify key computational kernels and construct performance models. Barry et al. [8] propose a PMC analysis methodology that maps hardware events to programming concepts. Andrade et al. [6] propose a hardware event-based program execution time modeling method by analyzing the correlation between hardware events. Others utilize the parameter of application and systems to build performance models. Hutter et al. [19] leverage tensor decomposition techniques to overcome the limits of high-dimensional parameter space modeling. Ritter et al. [28] develop an enhanced Extra-P analysis framework to analyze nonlinear relationships between parameters and performance.

Program phase modeling distinguishes phases with similar behavior during program execution, aiding resource optimization and performance prediction [10]. Offline phase detection approaches rely on post-execution trace analysis, whereas online methods [11] identify phase changes in real time, enabling transparent optimization without code modifications. For offline approaches, *Siesta* [25] extracts MPI program communication-computation traces to generate parameterized code snippets that simulate behaviors across different execution phases. The *DwarfCode* [34] performs phase representation by merging MPI event traces and compressing repetitive patterns. Lozano et al. [24] combined GMM clustering with LSTMs for CPU workload phase prediction. Phase-aware methods are also applied in runtime configuration optimization. Aurora [23] dynamically adjusts thread counts by monitoring OpenMP parallel loop behaviors at runtime.

Communication characteristics impact the program performance, particularly in large-scale parallel computing [16]. Chen et al. [9] demonstrated that communication event intervals follow a power-law distribution. Abubaker et al. [2] optimized point-to-point scheduling to minimize per-processor messages. The *EventGraD* algorithm [13] optimizes parallel stochastic gradient descent via event-triggered communication, reducing the communication overhead. Hoefler

et al. [17] map the application's communication graph to the physical network topology to minimize overhead.

6 Conclusion

The scalability bottleneck of MPI parallel programs severely limits their computational efficiency. This paper proposes a machine learning and data statistics-based scalability bottleneck detection framework to address the issues of detecting scalability bottlenecks. Experimental verification is conducted on two 2048-core ×86 and ARM clusters. The POPP method utilizes MPI comm and non-comm segment features for p^*_{app} prediction, achieving an average prediction accuracy of 70% (×86) and 60% (ARM), exceeding the non-segmented prediction methods. SOPP, based on differentiated feature selection, reduces the p^*_{seg} prediction deviation by 13.2% and 11.88% compared to methods without feature selection. Key event recognition, combining KL divergence and mutual information, improves the efficiency of identifying bottleneck performance events. This framework is capable of effective performance inflection prediction, bottleneck localization, and key performance event analysis, offering insight for parallel application performance optimization.

Acknowledgments. This study was funded by the Open Fund of National Key Laboratory of Parallel and Distributed Computing (PDL) (NO. 2023-KJWPDL-01).

References

1. MPIP: A light-weight MPI profiler (2020). https://software.llnl.gov/mpiP/
2. Abubaker, N., Karsavuran, M.O., Aykanat, C.: Scaling stratified stochastic gradient descent for distributed matrix completion. IEEE Trans. Knowl. Data Eng. **35**(10), 10603–10615 (2023). https://doi.org/10.1109/TKDE.2023.3253791, https://ieeexplore.ieee.org/document/10061560/
3. Adhianto, L., et al.: HPCTOOLKIT: tools for performance analysis of optimized parallel programs. Concurr. Comput. Pract. Exp. **22**(6), 685–701 (2010). https://doi.org/10.1002/CPE.1553
4. Afzal, A., Hager, G., Wellein, G.: The role of idle waves, desynchronization, and bottleneck evasion in the performance of parallel programs. IEEE Trans. Parallel Distrib. Syst. **34**(2), 623–638 (2023). https://doi.org/10.1109/TPDS.2022.3221085
5. Amaris, M., Camargo, R., Cordeiro, D., et al.: Evaluating execution time predictions on GPU kernels using an analytical model and machine learning techniques. J. Parallel Distributed Comput. **171**, 66–78 (2023). https://doi.org/10.1016/j.jpdc.2022.09.002, https://linkinghub.elsevier.com/retrieve/pii/S0743731522001903
6. Andrade, T.N.C., Lima, G., Lima, V.M.C., Bem-Amor, S., Hawila, I., Cucu-Grosjean, L.: On the impact of hardware-related events on the execution of real-time programs. Des. Autom. Embed. Syst. **27**(4), 275–302 (2023). https://doi.org/10.1007/s10617-023-09281-9, https://link.springer.com/10.1007/s10617-023-09281-9

7. Barnes, B.J., Rountree, B., Lowenthal, D.K., et al.: A regression-based approach to scalability prediction. In: Proceedings of the 22nd Annual International Conference on Supercomputing, pp. 368–377. ACM, Island of Kos Greece (2008). https://doi.org/10.1145/1375527.1375580
8. Barry, D., Danalis, A., Dongarra, J.: Automated data analysis for defining performance metrics from raw hardware events. In: 2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 716–725. IEEE, San Francisco (2024). https://doi.org/10.1109/IPDPSW63119.2024.00134, https://ieeexplore.ieee.org/document/10596509/
9. Chen, J., Zhou, W., Dong, Y., et al.: Analyzing time-dimension communication characterizations for representative scientific applications on supercomputer systems. Front. Comput. Sci. **13**(6), 1228–1242 (2019). https://doi.org/10.1007/s11704-018-7239-1
10. Criswell, K., Adegbija, T.: A survey of phase classification techniques for characterizing variable application behavior. IEEE Trans. Parallel Distrib. Syst. **31**(1), 224–236 (2020a). https://doi.org/10.1109/TPDS.2019.2929781
11. Criswell, K., Adegbija, T.: A survey of phase classification techniques for characterizing variable application behavior. IEEE Trans. Parallel Distrib. Syst. **31**(1), 224–236 (2020). https://doi.org/10.1109/TPDS.2019.2929781, https://ieeexplore.ieee.org/document/8769866/
12. Ding, N., Lee, V.W., Xue, W., et al.: APMT: an automatic hardware counter-based performance modeling tool for HPC applications. CCF Trans. High Perf. Comput. **2**(2), 135–148 (2020). https://doi.org/10.1007/s42514-020-00035-8, https://link.springer.com/10.1007/s42514-020-00035-8
13. Ghosh, S., Gupta, V.: EventGraD: event-triggered communication in parallel stochastic gradient descent. In: 2020 IEEE/ACM Workshop on Machine Learning in High Performance Computing Environments (MLHPC) and Workshop on Artificial Intelligence and Machine Learning for Scientific Applications (AI4S), pp. 1–8. IEEE, GA, USA (2020). https://doi.org/10.1109/MLHPCAI4S51975.2020.00008, https://ieeexplore.ieee.org/document/9409306/
14. GRAPH500: Graph500 (2017). https://graph500.org/
15. Hao, M., et al.: Fine-grained powercap allocation for power-constrained systems based on multi-objective machine learning. IEEE Trans. Parallel Distrib. Syst. **32**(7), 1789–1801 (2021). https://doi.org/10.1109/TPDS.2020.3045983
16. Helal, A.E., Jung, C., Feng, W.c., et al.: CommAnalyzer: automated estimation of communication cost and scalability on HPC clusters from sequential code. In: Proceedings of the 27th International Symposium on High-Performance Parallel and Distributed Computing, pp. 80–91. ACM, Tempe Arizona (2018). https://doi.org/10.1145/3208040.3208042
17. Hoefler, T., Snir, M.: Generic topology mapping strategies for large-scale parallel architectures. In: Proceedings of the International Conference on Supercomputing, pp. 75–84. ICS '11, Association for Computing Machinery, New York (2011). https://doi.org/10.1145/1995896.1995909
18. HPCG: High Performance Conjugate Gradients (2020). https://www.hpcg-benchmark.org/
19. Hutter, E., Solomonik, E.: Application performance modeling via tensor completion. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. ACM, Denver (2023). https://doi.org/10.1145/3581784.3607069

20. Jin, Y., et al.: SCALANA: automating scaling loss detection with graph analysis. In: SC20: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. IEEE, Atlanta (2020). https://doi.org/10.1109/SC41405.2020.00032, https://ieeexplore.ieee.org/document/9355294/
21. Li, P., Guo, Y., Luo, Y., et al.: Graph neural networks based memory inefficiency detection using selective sampling. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14. IEEE, Dallas (2022). https://doi.org/10.1109/SC41404.2022.00090, https://ieeexplore.ieee.org/document/10046134/
22. Li, Z., Chen, Z., Tang, Y., et al.: Muse: a runtime incrementally reconfigurable network adapting to HPC real-time traffic. In: 2024 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 765–779 (2024). https://doi.org/10.1109/IPDPS57955.2024.00073
23. Lorenzon, A.F., de Oliveira, C.C., Souza, J.D., et al.: Aurora: seamless optimization of openmp applications. IEEE Trans. Parallel Distrib. Syst. **30**(5), 1007–1021 (2019). https://doi.org/10.1109/TPDS.2018.2872992
24. Lozano, E.S.A., Gerstlauer, A.: Learning-based phase-aware multi-core CPU workload forecasting. ACM Trans. Des. Autom. Electron. Syst. **28**(2) (2022). https://doi.org/10.1145/3564929
25. Luo, J., Yan, T., Xu, Q., et al.: Siesta: synthesizing proxy applications for MPI programs. In: 2024 IEEE International Conference on Cluster Computing (CLUSTER), pp. 14–26. IEEE, Kobe (2024). https://doi.org/10.1109/CLUSTER59578.2024.00009, https://ieeexplore.ieee.org/document/10740841/
26. NPB: NAS parallel benchmarks (2020). https://www.nas.nasa.gov/software/npb.html
27. Qi, X., et al.: Highrpm: combining integrated measurement and sofware power modeling for high-resolution power monitoring. In: Proceedings of the 52nd International Conference on Parallel Processing, pp. 369–379. ICPP '23, Association for Computing Machinery, New York (2023). https://doi.org/10.1145/3605573.3605649
28. Ritter, M., Geis, A., Wehrstein, J., et al.: Noise-resilient empirical performance modeling with deep neural networks. In: 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS), pp. 23–34. IEEE, Portland (2021). https://doi.org/10.1109/IPDPS49936.2021.00012, https://ieeexplore.ieee.org/document/9460451/
29. Shende, S.S., Malony, A.D.: The tau parallel performance system. Int. J. High Perform. Comput. Appl. **20**(2), 287–311 (2006). https://doi.org/10.1177/1094342006064482
30. Sun, J., Sun, G., Zhan, S., et al.: Automated performance modeling of HPC applications using machine learning. IEEE Trans. Comput. **69**(5), 749–763 (2020). https://doi.org/10.1109/TC.2020.2964767, https://ieeexplore.ieee.org/document/8956059/
31. Wang, R., Lu, K., Chen, J., et al.: Brief introduction of tianhe exascale prototype system. Tsinghua Sci. Technol. **26**(3), 361–369 (2021). https://doi.org/10.26599/TST.2020.9010009
32. Wu, X., Taylor, V., Cook, J., et al.: Using performance-power modeling to improve energy efficiency of HPC applications. Computer **49**(10), 20–29 (2016). https://doi.org/10.1109/MC.2016.311

33. Xuan, Z., You, X., Yang, H., et al.: Retrospection on the performance analysis tools for large-scale HPC programs. In: 2024 IEEE 31st International Conference on High Performance Computing, Data, and Analytics (HiPC), pp. 34–44. IEEE, Bangalore (2024). https://doi.org/10.1109/HiPC62374.2024.00013, https://ieeexplore.ieee.org/document/10884289/
34. Zhang, W., Cheng, A.M.K., Subhlok, J.: DwarfCode: a performance prediction tool for parallel applications. IEEE Trans. Comput. **65**(2), 495–507 (2016). https://doi.org/10.1109/TC.2015.2417526, http://ieeexplore.ieee.org/document/7098397/
35. Zhang, Y., Liu, Y., Jiao, P., et al.: Automatic multi-parameter performance modeling of HPC applications on a new sunway supercomputer. IEEE Trans. Parallel Distrib. Syst. **34**(11), 2965–2977 (2023). https://doi.org/10.1109/TPDS.2023.3317296, https://ieeexplore.ieee.org/document/10255321/
36. Zhou, W., Zhang, J., Sun, J., et al.: Using small-scale history data to predict large-scale performance of HPC application. In: 2020 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), pp. 787–795. IEEE, New Orleans (2020). https://doi.org/10.1109/IPDPSW50202.2020.00135, https://ieeexplore.ieee.org/document/9150406/

Aspect Term Extraction Method Based on EBERT-CBAC

Yun Wu[1], Kai Yang[1(✉)], Yan Du[2], Jieming Yang[1], Ziyi Wang[1], Ning An[3], Nan Xu[3], and Yifan Huang[1]

[1] Northeast Electric Power University, Jilin, China
2202400893@neepu.edu.cn, yjmlzy@sina.com, dev.projectamadeus@outlook.com
[2] Jilin Meteorological Observation and Protection Center, Jilin Meteorological Service, Changchun, China
[3] Northeast Branch of State Grid Corporation of China, Shenyang, China

Abstract. Traditional e-commerce review methods face difficulties in identifying implicit evaluation targets and extracting features from both local details and overall semantic levels. To address this, this paper proposes an aspect term extraction model integrating auxiliary sentence construction and a hybrid neural network (EBERT-CBAC). This model first constructs auxiliary sentences to mine potential aspect term information and incorporates part-of-speech (POS) tags, enabling the traditional BERT model to capture multi-dimensional POS word structures. After obtaining contextual embeddings, it uses a CNN layer to extract local contextual features, followed by a BiGRU layer to capture global contextual features within the sentence. Considering that a single attention mechanism cannot capture important sentence features from different perspectives, a multi-head attention mechanism is introduced to process sentence-level information. Finally, sequence labeling is completed in the CRF layer to accurately identify both explicitly expressed and semantically implicit aspect terms in reviews. Experimental results show that, compared to the baseline model BERT-BiLSTM-CRF, the proposed model achieves an average F1-score improvement of 2.48% on three benchmark datasets, effectively enhancing the accuracy of aspect term extraction.

Keywords: Aspect Extraction · Attention mechanism · Deep learning

1 Introduction

With the rapid development of the internet, the quantity and influence of online reviews on e-commerce platforms have grown exponentially. The valuable user feedback they contain has become a crucial basis for optimizing products and services. However, e-commerce reviews are characterized by flexible sentence structures and colloquial expressions, posing significant challenges for aspect term extraction. For example, users may use implicit expressions like "slow delivery" to reflect dissatisfaction with "logistics speed". Such expressions often lack explicit

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 462–477, 2026.
https://doi.org/10.1007/978-981-95-8405-5_25

aspect term indicators, making it difficult for traditional methods based on association rules or shallow semantic models to effectively capture implicit sentiment information. Therefore, accurately identifying both explicit and implicit aspect terms in reviews has become an important and challenging issue in the current field of sentiment analysis.

Aspect term extraction (AE) can be treated as a sequence labeling task [1]. Research methods have roughly evolved through the following stages:

Rule-based and Lexicon-based Algorithms: Early rule-based and lexicon-based algorithms relied on manually constructed linguistic rules and sentiment lexicons to identify text, such as using keyword matching and predefined syntactic templates to extract aspect terms. These methods are easy to implement and show certain effectiveness in specific scenarios, but they have poor adaptability to corpus variations, weak scalability, and require significant manual intervention, making them difficult to apply to large-scale real reviews. Subsequent research introduced traditional machine learning methods like Hidden Markov Models (HMM) [2] and Conditional Random Fields (CRF) [3].

Traditional Machine Learning Methods: Compared to rule-based approaches, traditional machine learning methods show improvements in accuracy and generalization capabilities, but they demand substantial annotated data.

Topic Model-based Methods: Subsequently, topic modeling methods like LDA (Latent Dirichlet Allocation) [4] were introduced for aspect term extraction tasks. Mukherjee and Liu [5] proposed an approach combining predefined seed words with the Latent Dirichlet Allocation (LDA) model to guide the extraction of latent aspect topics from product reviews. However, relying on word co-occurrence frequencies struggles to effectively capture syntactic structures and contextual semantic dependencies in text, resulting in limitations in the granularity of aspect terms.

Deep Learning Methods: With the development of deep learning techniques, neural networks have shown significant advantages in natural language processing tasks. Hammerton [6] was among the first to apply Long Short-Term Memory networks (LSTM) to sequence labeling tasks, addressing vanishing gradient issues. Subsequently, by combining BiLSTM and CRF, Lample et al. [7] proposed the BiLSTM-CRF model. BiLSTM extracts features from both directions and gradually improved the accuracy of various sequence labeling tasks. Beyond recurrent structures, based on Convolutional Neural Networks (CNN), Xu et al. [8] proposed the DE-CNN model, introducing general and domain-specific dual embedding mechanisms and performing feature extraction through hierarchically stacked CNNs, demonstrating the potential of deep convolutional structures in aspect extraction tasks. Wang et al. [9] proposed the RNCRF model, combining a recurrent neural network with a CRF model to simultaneously identify aspect terms and associated opinion expressions in text, enhancing the model's overall performance. While current methods exhibit improved precision, their heavy reliance on explicit aspect-opinion associations renders them ineffective against implicit aspects. Additionally, their global sentence-level analysis fre-

quently fails to capture nuanced local contexts, particularly when processing negations or adversative constructions in reviews.

In summary, this paper proposes an aspect term extraction model integrating auxiliary sentence construction and a hybrid neural network (EBERT-CBAC). The innovative ideas are as follows:

(1) Propose an auxiliary sentence generation method jointly driven by L-LDA and semantic distance. First, an aspect category-seed word association library is constructed from annotated corpora. Then, BERT computes the semantic similarity between input words and seed words to dynamically generate explicit interrogative auxiliary sentences, thereby verbalizing implicit aspect terms.

(2) Optimize the semantic-syntactic integration for domain adaptation. On the semantic side, BERT dynamic encoding is leveraged to adapt to domain-specific expressions. On the syntactic side, optimization is achieved through POS tagging.

(3) Design a multi-level hybrid neural network architecture (CNN-BiGRU-MHA) integrating local-global features, extracting complementary information from local contexts, long-distance dependencies, and multi-head attention, respectively. Sequence labeling is ultimately completed in the CRF layer.

2 Related Principles and Methods

2.1 L-LDA

Labeled Latent Dirichlet Allocation (L-LDA) [10] is an extension of the LDA model. Its core idea is to introduce document label information as a prior distribution to extract semantic features highly relevant to predefined categories. Through label constraints, L-LDA restricts the topic distribution of a document to its labeled specific categories, ensuring the generated topics are closely related to domain knowledge. The generative process of L-LDA is shown in Table 1:

Table 1. Generative Process of L-LDA

Generative Process of L-LDA
For each topic $k \in \{1, \ldots, K\}$:
$\quad \rightarrow$ Generate: $\beta_k = (\beta_{k,1}, \ldots, \beta_{k,V})^T \sim \mathrm{Dir}(\cdot \mid \eta)$
For each document d:
$\quad$ For each topic $k \in \{1, \ldots, K\}$:
$\quad\quad \rightarrow$ Generate: $\Lambda_k^{(d)} \in \{0, 1\} \sim \mathrm{Bernoulli}(\cdot \mid \Phi_k)$
$\quad \rightarrow$ Generate: $\alpha^{(d)} = L^{(d)} \times \alpha$
$\quad \rightarrow$ Generate: $\theta^{(d)} = (\theta_{l1}, \ldots, \theta_{lM_d})^T \sim \mathrm{Dir}(\cdot \mid \alpha^{(d)})$
$\quad$ For each $i \in \{1, \ldots, N_d\}$:
$\quad\quad \rightarrow$ Generate: $z_i \in \{\lambda_1^{(d)}, \ldots, \lambda_{M_d}^{(d)}\} \sim \mathrm{Mult}(\cdot \mid \theta^{(d)})$
$\quad\quad \rightarrow$ Generate: $w_i \in \{1, \ldots, V\} \sim \mathrm{Mult}(\cdot \mid \beta_{z_i})$

Table 1 lists the complete generative process of the L-LDA model. This process is based on a Dirichlet-Multinomial two-level generative mechanism and introduces a label projection matrix on top of standard LDA to constrain the global topic space to the label set of each document. For clarity, the main mathematical symbols involved in the table are explained in Table 2 below:

Table 2. Explanation of Symbols in the Generative Process of L-LDA

Symbol	Meaning
K	Total number of topics, equal to the number of unique label types in the corpus
V	Vocabulary size
β_k	Word distribution for the $k-th$ topic, a vector of dimension V
η	Hyperparameter of the Dirichlet distribution, controlling the sparsity of the word distribution
$\Lambda^{(d)}$	Label of document d
Φ_k	Prior probability of label presence for the $k-th$ topic
$L^{(d)}$	Document label projection matrix of size $M_d \times K$
α	Dirichlet parameter vector for the global topic prior
$\alpha^{(d)}$	Document-specific Dirichlet hyperparameter vector, restricted to label-related topics
$\theta^{(d)}$	Topic distribution for document d
z_i	Topic number assigned to the $i-th$ word
w_i	Index of the $i-th$ generated word
N_d	Total number of words in document d
M_d	Number of labels for document d (i.e., size of $\lambda^{(d)}$)

2.2 BERT Pre-training

In the field of Natural Language Processing, the Transformer architecture proposed by Vaswani A et al. [11] broke through the sequence modeling limitations of RNNs through the self-attention mechanism. Its parallel computation properties significantly improved model efficiency and led to the emergence of generative pre-trained models like GPT and the landmark bidirectional encoder model BERT [12].

BERT uses Transformer encoder layers to achieve fully bidirectional semantic representation. Its input integrates three types of embedding information: token embeddings, position embeddings, and segment embeddings [13]. By scaling the hidden layer dimension to 768 and utilizing a 12-head self-attention mechanism, BERT optimizes feature extraction capability. The BERT model learns linguistic patterns through a dual-task pre-training strategy: Masked Language Modeling (MLM) and Next Sentence Prediction (NSP). The MLM task randomly masks 15% of tokens (80% replaced with [MASK], 10% replaced with random words, 10% left unchanged), forcing the model to predict masked content using bidirectional context. The NSP task models logical relationships between sentences by contrasting positive and negative samples. The pre-trained BERT can be quickly adapted to downstream tasks through fine-tuning, requiring only a small amount

of labeled data to adjust parameters in the top-layer network, effectively alleviating the sensitivity of deep learning models to small-sample scenarios.

2.3 BiLSTM-CRF Model

In the field of sequence data processing, improved models based on LSTM have continuously driven technological advancements. The Bidirectional Long Short-Term Memory Network-Conditional Random Field (BiLSTM-CRF) architecture proposed by Huang et al. [14] has become a benchmark model in this domain.

The model adopts a three-stage processing pipeline: First, it transforms discrete symbols into a continuous vector space through distributed representations. Next, it employs bidirectional networks to capture contextual features. Finally, it utilizes a probabilistic graphical model to optimize sequence labeling. Specifically, given an input sequence $x = \{x_1, x_2, ..., x_n\}$, the word embedding layer maps each lexical unit x_i to a d-dimensional dense vector $w_i \in R^d$. For the temporal characteristics of sequential data, RNN and its enhanced variant, the LSTM network, demonstrate unique advantages. LSTMs establish a selective memory mechanism through gated units, controlling information flow and mitigating the gradient explosion problem common in traditional RNNs. However, unidirectional LSTMs can only capture forward contextual features, limiting their performance in tasks requiring bidirectional context reasoning. To address this, the BiLSTM employs a parallel dual-channel architecture, where the forward LSTM layer and backward LSTM layer process the sequence separately. The bidirectional features are then fused to form a comprehensive semantic representation, as calculated in Eqs. (1), (2) and (3):

$$\overrightarrow{h_t} = LSTM(w_t, \overrightarrow{h_{t-1}}) \tag{1}$$

$$\overleftarrow{h_t} = LSTM(w_t, \overleftarrow{h_{t+1}}) \tag{2}$$

$$h_t = [\overrightarrow{h_t} \oplus \overleftarrow{h_t}] \tag{3}$$

where $\overrightarrow{h_t}$ and $\overleftarrow{h_t}$ represent the forward and backward hidden states at time step t, respectively, which are concatenated via the $\oplus$ operation.

3 Model Design

The main contribution of this paper is the EBERT-CBAC model. This model constructs auxiliary sentences and extends the BERT architecture to integrate syntactic standards. It employs CNN to extract local features, BiGRU to capture global context, combines MHA to attend to key information, and finally identifies explicit and implicit aspects through a CRF layer. An illustration of the model is shown in Fig. 1.

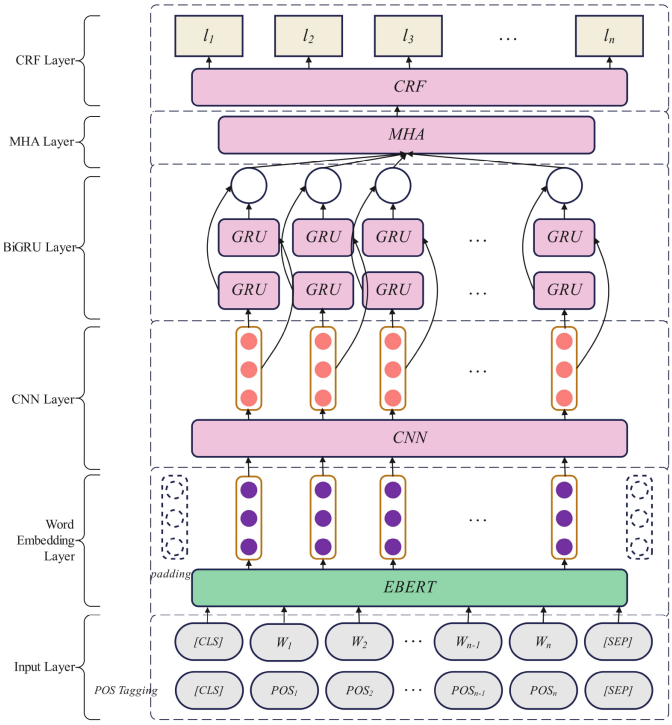


Fig. 1. EBERT-CBAC Model Framework Diagram

3.1 Auxiliary Sentence Construction and POS Tagging Method

Since words of the same type exhibit similar positional distances in semantic vector space (as shown in Fig. 2), for implicit aspect terms not directly present in sentences, L-LDA is utilized to extract a set of seed words for each potential implicit aspect from an annotated e-commerce domain corpus (see Table 3). Aspect category items are generated from corresponding seed words. The semantic distance between each word in the input sentence and the target seed words is calculated. Words meeting a specific threshold are considered semantic candidates. An auxiliary sentence is constructed based on the semantic distance between aspect-dependent seed words and the target sentence. This auxiliary sentence serves as an expanded feature of the original sentence and is input into the model.

In the example sentence from Fig. 3, the explicit aspects are "sports shoe" and "price", both directly present in the sentence. However, the phrase "quickly received" implies information about the implicit aspect term "delivery speed", which is not explicitly stated in the original sentence. Accordingly, an auxiliary question will be generated: "What do you think of the delivery speed?" to explicitly extract this implied aspect.

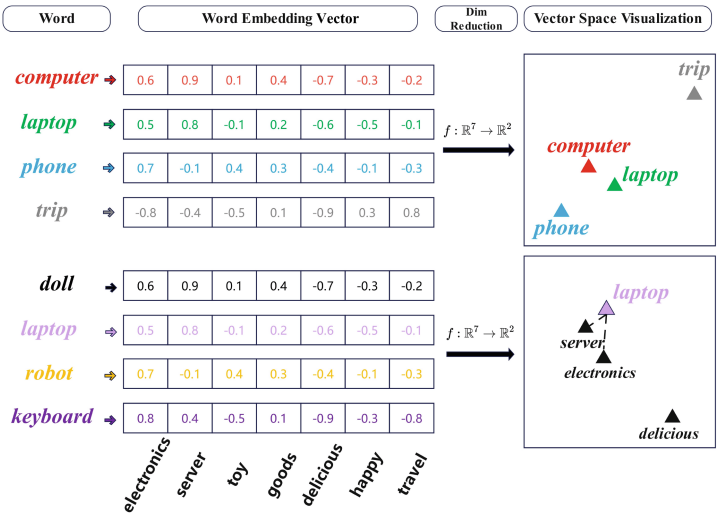


Fig. 2. Example Illustration of Similarity Coherence Distribution in Word Embedding Space

Table 3. Top 4 Seed Word Examples from Laptop Dataset

Aspect Word	Seed Word
delivery-speed	received, fast, slow, postage, deliver, ...
service	efficient, friendly, inconvenient, troublesome, ...
price	expensive, cheap, affordable, cost, buy ...
sports shoe	comfortable, durable, lightweight, breathable, flexible ...

Then, Stanford POS Tagger is employed to generate part-of-speech tags for each word. Considering adjectives and adverbs are central to sentiment expression while nouns and their relationship with subjects help identify key aspect targets, three syntactic rules (adjective modification, nominal subject relation, and adverbial modification) are applied to preserve POS tags. These processed tags are finally fed into the EBERT-CBAC model for extraction tasks. Taking the sentence "The customer service is very friendly and the product quality is good." as a sample input, its processing flow is illustrated in Fig. 4.

This *sports shoe* is not only of high quality, but also reasonably *priced*, and can be quickly received with every purchase.

Fig. 3. Example Sentence with Explicit/Implicit Aspect Terms

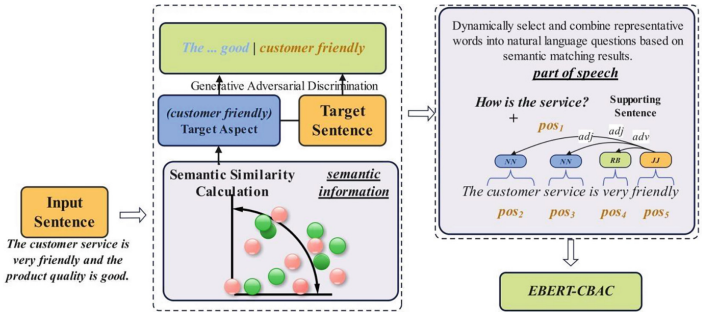


Fig. 4. Schematic Diagram of Auxiliary Vector Construction and POS Tagging

3.2 EBERT Layer

Currently, the generic BERT model generates vector representations for text W by summing and concatenating three embeddings: word embeddings e_w, sentence embeddings e_s, and position embeddings e_p. Here, word embeddings capture the semantic meaning of characters, sentence embeddings indicate sentence membership to provide basic text classification capabilities, and position embeddings record word order information to preserve input sequence temporality [12].

Since generic BERT cannot fully represent all information, this work introduces an additional embedding layer for POS tagging recognition. This layer is integrated with character embeddings to capture POS tag features. The EBERT model takes both text characters and their corresponding POS tags as input. While retaining the original three-layer embeddings (e_w, e_s, e_p) for character vectors, we employ One-Hot encoding to uniformly represent POS-tagged entities and non-POS entities, aligning them with BERT's vector space. The entity vectors are then summed and concatenated with character vectors to form the final representation e'_a, as formalized in Eqs. (4), (5) and (6):

$$e_{pos} = OneHot\ (\ Tag\ ) \tag{4}$$

$$e_a = e_{pos} + e_w + es + e_p \tag{5}$$

$$e_a^{'} = EBERT\ (\ e_a\ ) \tag{6}$$

EBERT integrates character vector representations with corresponding POS tag information during word vector construction. Since aspect terms predominantly appear as nouns and often form evaluation phrases with adjectivesâĂŤfor example, in "I found the best deals at the new store downtown," store functions as a noun denoting a physical location, while in "You can store all your photos in the cloud easily," it serves as a verb meaning to save. In the original BERT model: "store" is represented by the vector $v = (v_0, v_1, ..., v_n)$ in sentence 1, "store" is represented by the vector $v' = (v_0^{'}, v_1^{'}, ..., v_n^{'})$ in sentence 2. Though contextualized, the absence of POS tagging causes v and $v^{'}$ to remain numerically close, hindering model discrimination. EBERT addresses this by converting POS cat-

egory labels into vectors and superimposing them onto original character vectors, thereby widening the numerical distance between v and $v^{'}$ and significantly enhancing aspect term identification capability.

3.3 CNN Local Feature Extraction Layer

In many cases, aspect terms are positioned close to the words determining their sentiment polarity [13]. Therefore, prior to convolution operations, input matrices require padding: m columns of 0 vectors are added to both left and right sides to align CNN layer input/output dimensions. Subsequently, a convolutional layer extracts local contextual features, with the kernel dimensions configured as $l \times d_e$, where: $l = 2m + 1$ denotes the kernel length, d_e represents the kernel width, indicating that the convolutional kernel will extract local semantic relationships between the current character and its surrounding m characters.

During feature extraction, d_h convolutional kernels perform multi-kernel concurrency for parallel computation, as formalized in Eqs. (7) and (8):

$$\tilde{h}_i^m = \sum_{j=-m}^{m} w_j e'_{a\ i+j} + b^m, \tag{7}$$

$$h_i^m = \text{RELU}\left(\tilde{h}_i^m\right) \tag{8}$$

Here, the dimension of h_i^m is R^{d_h}. Finally, the output of the CNN layer can be obtained as $h^m = [h_1^m, h_2^m, ..., h_n^m] \in R^{n \times d_h}$.

3.4 BiGRU Global Feature Extraction Layer

While CNNs capture local features, global context is crucial for text. BiGRU captures long-range dependencies and is ideal for aspect extraction (e.g., processing both "battery life is excellent" and "screen quality is mediocre" in one sentence). More efficient than RNNs for e-commerce data, BiGRU encodes EBERT embeddings to learn contextual dependencies. It consists of forward $\overrightarrow{h_t}$ and backward $\overleftarrow{h_i}$ GRU layers (Eqs. 9, 10 and 11)[13].

$$\overrightarrow{h_i^g} = GRU\ (\ h_t^m, \overrightarrow{h_{i-1}^g}\ ) \tag{9}$$

$$\overleftarrow{h_i^g} = GRU\ (\ h_t^m, \overleftarrow{h_{i-1}^g}\ ) \tag{10}$$

$$y_i^g = [\ \overrightarrow{h_i^g}, \overleftarrow{h_i^g}\ ] \tag{11}$$

3.5 Multi-head Attention (MHA) Layer

For the e-commerce domain, massive volumes of reviews often contain redundant information. BiGRU may also face challenges when extracting critical information from sentence sequences. The attention mechanism can more effectively allocate weights to key information, helping the model better understand the

semantics of sentence sequences. The multi-head attention mechanism projects inputs into multiple vector spaces and computes contextual representations of characters within their respective vector spaces. Finally, these distinct results are concatenated to obtain a comprehensive contextual representation. The computation of MHA (Multi-Head Attention) is shown in Formula (12), Formula (13), and Formula (14) [11].

$$Attention(Q, K, V) = Softmax(\frac{(QK^{\top})}{\sqrt{d_k}})V \tag{12}$$

$$M(Q, K, V) = concat(score_1(y_t^g), ..., score_h(y_t^g))W \tag{13}$$

$$score_i(y_t^g) = Attention(QW_i^Q, KW_i^K, VW_i^V) \tag{14}$$

Among them, Q, K, and V are respectively the query, key, and value matrices, all represented by multi-dimensional vectors; y_t^g represents the output of the BiGRU layer; h is the number of heads of the self-attention mechanism; W, W_i^q, W_i^k and W_i^v are all the weight matrices to be learned; the self-attention mechanism calculates the scaling dot product through the *Attention* function, where $\sqrt{d_k}$ is the scaling factor. It is used to prevent the $QK^{\top}$ dot product result from causing *Softmax* saturation (the gradient approaches zero) when the dimension is large, thereby affecting the training stability of the model (this setting is derived from the standard scaling attention mechanism in the *Transformer* structure).

After processing through the multi-head attention mechanism, the model can integrate the global feature information of the input sequence and pass it as input to the CRF layer. The method involves selecting the label with the highest score as the corresponding sequence labeling result for independent decoding. It is worth noting that the aspect term extraction task exhibits certain particularities at the labeling level [13]. The dataset in this paper mostly adopts the BIO format for label annotation, where the I-label must immediately follow the B-label. If this condition is not met, the label combination loses semantic consistency. The advantage of CRF lies in its ability to learn constraint relationships based on the correlations between adjacent labels. Let the input be $I = \{I_1, I_2, I_3, ..., I_n\}$ and the labels $s = \{s_1, s_2, s_3, ..., s_n\}$, where $P(s|I)$ represents the conditional probability, as shown in Eq. (15).

$$P(s|I) = \frac{\sum_{t=1}^{T} e^{f(s_{t-1}, s_t, I)}}{\sum_{s'}^{S(I)} \sum_{t=1}^{T} e^{f(s'_{t-1}, s'_t, I)}} \tag{15}$$

Here, $S(I)$ denotes the set of all possible labels, and the score function $f(s_{t-1}, s_t, I)$ is used to simultaneously evaluate the transition state between labels (from s_{t-1} to s_t) and the feature score corresponding to the current position s_t. The optimal label path is obtained by maximizing the output probability $P(s|I)$ under the conditional probability distribution for the entire sequence. This process is mathematically expressed as Eq. (16), and the loss function is

given by Eq. (17).

$$Y^* = \arg\max_s P(s|I) \tag{16}$$

$$\text{Loss} = \sum_{i=1}^{k} \log P(s_i|I_i) \tag{17}$$

4 Experimental Results and Analysis

To evaluate the performance of the EBERT-CBAC model, tests were conducted from the following two aspects: (1) Testing the modelâĂŹs accuracy by comparing it with other state-of-the-art aspect term extraction models; (2) Validating the importance of POS (part-of-speech) tags, the BiGRU layer, and multi-head attention for aspect term extraction.

4.1 Datasets

To verify the model performance, this paper conducts experimental evaluations on three mainstream public benchmark datasets. The first one consists of user reviews from the Restaurant and Laptop domains from SemEval2014 [15]. The second one is constructed from Twitter social media posts [16]. All these three public datasets are labeled with BIO tags, and their statistics are shown in Table 4.

Table 4. Dataset Statistics

Dataset	Training set/items	Test set/items	Sentences/items
Laptop	3024	700	3724
Twitter	2000	550	2550
Restaurant	3017	700	3717

4.2 Evaluation Metrics

Commonly used performance evaluation metrics for classification tasks include Recall, Precision, Accuracy, and F1-Score. Among these, Accuracy measures the model's overall prediction correctness on the test set, encompassing both True Positive TP and True Negative TN predictions. The relevant calculation formulas

are provided in Eq. (18), (19), (20) and (21).

$$Precision = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}} \times 100\% \tag{18}$$

$$Recall = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}} \times 100\% \tag{19}$$

$$F1 - score = \frac{2 \times Precision \times Recall}{Precision + Recall} \times 100\% \tag{20}$$

$$Accuracy = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}} \times 100\% \tag{21}$$

4.3 Experimental Environment and Parameter Configuration

The experimental environment is as shown in Table 5:

Table 5. Experimental Environment

Experimental Environment	Environment Configuration
Operating System	Windows 11 Pro 64-bit
CPU	Intel(R) Core(TM) i9-14900HX
Memory/GB	16
GPU	NVIDIA GeForce RTX 4060
Development Tool	PyCharm

The parameter configuration is shown in Table 6. For key parameters such as the learning rate and batch size, the empirical settings of BERT-like models (Devlin et al., 2018) [12] on the text sequence annotation task were referred to. Finally, the optimal combination was selected to take into account both model performance and training stability.

4.4 Comparative Experiments

First, comparative experiments with existing aspect term extraction models were conducted to evaluate the performance of the proposed EBERT-CBAC model. F1-score, the most widely adopted metric, was selected as the primary evaluation criterion, where "-" denotes that the corresponding method was not experimentally validated on that dataset.

From the experimental results in Table 7, it can be observed that the model proposed in this paper demonstrates strong performance in terms of F1-score, achieving significant advantages in comprehensive comparisons with existing baseline models. Combining CRF with BiLSTM resulted in a noticeable performance improvement compared to using either structure alone. Building upon this, the results of employing BERT pre-trained model are close to those of DE-CNN. Furthermore, DE-CNN employs a stacked 4-layer CNN architecture for

Table 6. Parameter Configuration

Parameter	Value
Optimizer	Adam
Learning Rate	0.00003
Epochs	20
Batch Size	24
L2 Regularization Weight	0.01
Word Vector Dimension	768
Convolution Kernel Size and Number	5ÃŮ256
BERT Model	bert-base-uncased

Table 7. F1-score Comparison of Our Model with Other Models

Model	Laptop	Restaurant	Twitter
CRF	72.89%	80.33%	70.02%
his-RD	74.55%	79.62%	-
DLIREC	73.78%	84.01%	-
BiLSTM-CRF	74.28%	-	71.44%
BiLSTM-CNN-CRF	75.48%	80.98%	75.01%
BERT-FTC	79.34%	-	74.75%
BERT-BiLSTM-CRF	80.12%	85.61%	75.61%
DE-CNN [8]	81.59%	86.01%	74.37%

deep feature extraction and utilizes domain-specific word embeddings, enabling it to capture implicit information within the text. However, domain-specific word embeddings have limited applicability and their annotation process is demanding and time-consuming. After replacing the BERT pre-trained model with our proposed EBERT model, which constructs auxiliary sentences, performance improved by 3.35%, 3.25%, and 0.83% respectively across the three datasets compared to the BERT-BiLSTM-CRF model. This indicates that combining the semantic features of e-commerce review text with aspect category features can effectively enhance classification accuracy.

4.5 Ablation Study

In addition to comparisons with other models, ablation studies were conducted to investigate the effectiveness of each module, using F1-score as the core evaluation metric:

(1) CBAC: BERT is not used for word vector initialization; Word2vec is used instead.

(2) BERT-CBAC: Additional POS tags are not incorporated.

(3) EBERT-CBAC/CNN: The CNN module is removed from the model.
(4) EBERT-CBAC/MHA: The attention module is removed from the model.

Table 8. Ablation Study Comparison

Model	Laptop	Restaurant	Twitter
CBACT	79.02%	80.13%	73.13%
BERT-CBAC	81.19%	84.91%	75.24%
EBERT-CBAC/CNN	81.77%	86.66%	75.44%
EBERT-CBAC/MHA	82.04%	86.79%	75.93%
EBERT-CBAC	83.47%	88.86%	76.44%

As shown in Table 8, and to better illustrate the specific impact of each functional module on the overall model performance, Fig. 5 presents the corresponding experimental results. After removing either the CNN or the MHA module, the model performance showed a certain degree of decline. Although the decrease was not substantial, this indicates that the introduction of these modules is necessary. Meanwhile, replacing BERT with Word2vec for word embedding led to a significant drop in model performance, demonstrating the powerful role of the pre-trained model BERT. Additionally, the incorporation of POS tags provided a significant improvement for aspect term recognition in relatively well-structured sentences. However, its effectiveness was limited for languages with looser grammatical structures, such as those found on social media platforms like Twitter. This suggests that methods relying on part-of-speech tagging need further improvement.

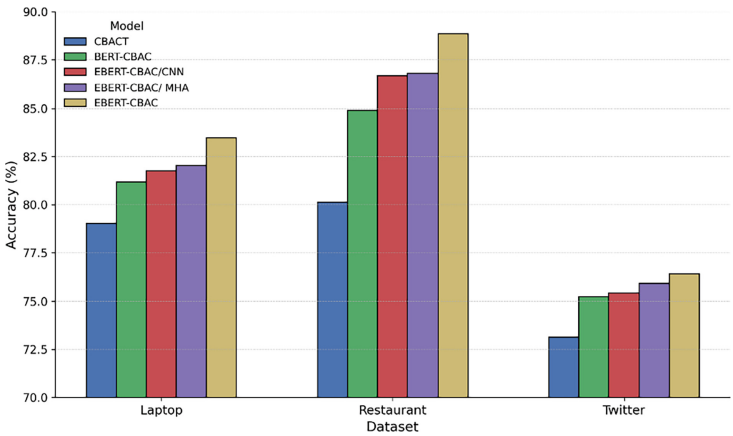


Fig. 5. Comparison of Ablation Study Results

5 Conclusion

This paper addresses the challenge of aspect term extraction in e-commerce reviews by proposing an extraction model that integrates auxiliary sentence construction with a hybrid neural network. Aiming at the difficulties in identifying implicit aspect terms within e-commerce reviews and the limitations of traditional methods that rely on explicit co-occurrence relationships, this work introduces semantics-driven auxiliary sentence construction and incorporates POS embedding features to surface implicit aspect terms. Building upon the BERT model, we designed the EBERT-CBAC architecture, which employs CNN and BiGRU to capture local and global contextual features within sentences, respectively. Subsequently, a multi-head attention mechanism is utilized to learn weight information for words in the entire sentence from different perspectives, enabling the accurate identification of both explicit and implicit aspect terms. The results demonstrate that EBERT-CBAC achieves F1-scores of 83.47%, 88.86%, and 76.44% on the Laptop, Restaurant, and Twitter datasets, respectively. This represents an average improvement of 2.48% over the best baseline model, BERT-BiLSTM-CRF. Ablation studies further confirmed the contributions of auxiliary sentence generation, POS tags, and the multi-head attention mechanism to the overall performance, providing reliable aspect terms for aspect-level sentiment classification tasks. Future research could explore the introduction of large language models (LLMs) to generate more diverse and richer auxiliary sentences through semantic rewriting and multi-turn questioning mechanisms. This approach holds the potential to overcome the limitations of current methods based on seed words and similarity thresholds, thereby further enhancing the performance of aspect term extraction.

Acknowledgement. This work is supported by the Harbin Boiler Company Limited of China (No:HBC-661140).

References

1. AL-Sharuee, M.T., Liu, F., Pratama, M.: Sentiment analysis: dynamic and temporal clustering of product reviews. Appl. Intell. **51**, 51–70 (2021)
2. Jin, W., Ho, H.H., Srihari, R.K.: OpinionMiner: a novel machine learning system for web opinion mining and extraction. In: the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 1195–1204. ACM, Paris (2009)
3. Jakob, N., Gurevych, I.: Extracting opinion targets in a single- and cross-domain setting with conditional random fields. In: Proceedings of the 2010 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 1035–1045. ACL, Massachusetts (2010)
4. Titov, I., McDonald, R.: Modeling online reviews with multi-grain topic models. In: 17th International Conference on World Wide Web (WWW), pp. 111–120. ACM, Beijing China (2008)

5. Mukherjee, A., Liu, B.: Aspect extraction through semi-supervised modeling. In: the 50th Annual Meeting of the Association for Computational Linguistics, pp. 339–348. ACL, Jeju (2012)
6. Hammerton, J.: Named entity recognition with long short-term memory. In: the 7th Conference on Natural Language Learning at HLT-NAACL 2003, pp. 172–175. Edmonton Canada (2003)
7. Lample, G., Ballesteros, M., Subramanian, S., et al.: Neural architectures for named entity recognition. In: Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 260–270. ACL, San Diego California USA (2016)
8. Xu, H., Liu, B., Shu, L., et al.: Double embeddings and CNN-based sequence labeling for aspect extraction. In: the 56th Annual Meeting of the Association for Computational Linguistics, pp. 592–598. ACL, Melbourne Australia (2018)
9. Wang, Y., Pan, S.J., Dahlmeier, D., et al.: Recursive neural conditional random fields for aspect-based sentiment analysis. In: Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing, pp. 616–626. ACL, Austin Texas (2016)
10. Ramage, D., Hall, D., Nallapati, R., Manning, C.D.: Labeled LDA: a supervised topic model for credit attribution in multi-labeled corpora. In: Proceedings of the 2009 Conference on Empirical Methods in Natural Language Processing, pp. 248–256. ACL, Singapore (2009)
11. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17), pp. 5998–6008. NIPS, California (2017)
12. Devlin, J., Chang, M.W., Lee, K., et al.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 4171–4186. ACL, Minneapolis Minnesota (2019)
13. Shi, J.J.: Research on aspect term sentiment analysis based on deep learning. Chongqing University (2022)
14. Huang, Z.H., Xu, W., Yu, K.: Bidirectional LSTM-CRF models for sequence tagging. arXiv preprint arXiv:1508.01991, (2015)
15. Pontiki, M., Galanis, D., Pavlopoulos, J., et al.: SemEval-2014 task 4: aspect based sentiment analysis. In: 8th International Workshop on Semantic Evaluation, pp. 27–35. Dublin Ireland (2014)
16. Lin, Z.C., Li, B.C.: Attention Aspeet-based sentiment analysis based on local context focus mechanism and talking-head. Comput. Sci. **50**(S2), 106–111 (2023)

HACompBench: Co-designed Multimodal DNN Compression Evaluation for Edge Devices

Zhengyu Gan[1(✉)], Haohua Du[2], Chengquan Feng[1], and Haisheng Tan[1]

[1] School of Computer Science, USTC, Hefei, China
{albertony,chengquan}@mail.ustc.edu.cn, hstan@ustc.edu.cn
[2] School of Cyber Science and Technology, Beihang University, Beijing, China
duhaohua@buaa.edu.cn

Abstract. Deployment of deep neural networks on edge devices faces challenges from heterogeneous hardware and multimodal tasks, where existing compression evaluation frameworks overlook hardware co-design, leading to suboptimal performance. To address this, we introduce HACompBench, a new hardware-aware framework that defines compression evaluation as a multi-objective optimization problem and combines hardware metrics such as quantization efficiency ξ and sparsity compatibility η with a dynamic scoring function J. We performed comprehensive experiments across four leading SoC platforms: Snapdragon 888, Snapdragon 765G, Kirin 970, and Jetson Nano P3450, and tested ten DNN models covering vision, text, and speech modalities using compression techniques such as quantization, pruning, and weight sharing, revealing hardware-induced performance gaps, such as quantization yields $J = 32.3\%$ on Snapdragon 888 but $J = 68.0\%$ on Jetson Nano P3450 due to INT8 emulation overhead. These results systematically highlight compression variations from differences in parallel processing capabilities. HACompBench innovates by linking hardware features, supporting multimodal tasks, and surpassing MLPerf Tiny's single-modality focus and AIoTBench's lack of co-design through embedded metrics ξ and η, while modality-specific corrections improve accuracy by up to 12.6%. It provides a unified and robust framework for edge deployment.

Keywords: Model Compression · Edge Computing · Hardware-Algorithm Co-Design · Multimodal Evaluation · Parallel Sparse Optimization

1 Introduction

The rapid advancement of edge computing has driven the deployment of deep neural networks on resource-constrained devices, such as smartphones and IoT nodes, characterized by compute throughput below 30 TOPS, memory capacities of 4 to 12 GB, and power budgets of a few watts. This shift from cloud-centric

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 478–496, 2026.
https://doi.org/10.1007/978-981-95-8405-5_26

to edge-distributed intelligence reduces data transmission latency from hundreds of milliseconds to mere milliseconds while enhancing user privacy by minimizing sensitive data uploads. Applications in autonomous driving, industrial IoT, and mobile healthcare demonstrate the transformative potential of edge intelligence. However, the computational complexity of deep neural networks, often involving tens of millions of parameters, poses significant deployment challenges. Compression techniques, such as quantization and pruning, are critical to address these constraints, yet their efficacy varies across heterogeneous hardware. For instance, MobileNetV3-Small achieves a 2.73-fold inference acceleration on Snapdragon 888 but only 1.16-fold on Kirin 970 due to differences in hardware support for quantization [9]. This variability highlights a critical limitation: the conventional compress-then-deploy approach, which applies compression techniques sequentially without considering hardware characteristics, fails to fully exploit compression potential in real-world edge scenarios. Such variability underscores the limitations of the traditional compress-then-deploy paradigm, which ignores hardware characteristics and thus fails to maximize compression potential in edge scenarios.

The compress-then-deploy paradigm relies on empirical compression strategies, such as quantization to reduce precision or pruning to eliminate redundant parameters, without systematically quantifying their interactions with hardware attributes like INT8 support or sparsity compatibility. This leads to suboptimal performance across diverse edge devices. In smart home applications, for example, Wav2Vec2-Tiny suffers a 3% accuracy degradation under quantization on low-end ARM Cortex-A53 processors, while unstructured pruning increases latency by 28% due to irregular memory access patterns [32]. Existing evaluation frameworks, such as MLPerf Inference, are designed for cloud or server environments, overlooking edge-specific challenges like memory fragmentation [21]. MLPerf Tiny focuses on vision tasks, neglecting the temporal dependencies of speech models like Wav2Vec2-Tiny or the token sparsity of text models like TinyBERT, where quantization can degrade accuracy by up to 3% [7,9]. Recent surveys, such as Liang et al. [5] and Lee et al. [6], highlight multimodal machine learning challenges for edge devices, emphasizing the need for hardware-aware optimization, while Wang et al. [30] discuss pruning efficiency on mobile devices. These frameworks lack a unified approach to integrate hardware characteristics with algorithmic performance across multimodal tasks, necessitating a novel evaluation framework.

To address these challenges, we propose HACompBench, a hardware-aware framework that redefines model compression evaluation as a multi-objective optimization problem, as detailed in Sect. 3. HACompBench models the interplay between compression techniques and hardware attributes, employing a dynamic scoring function J that balances accuracy loss, inference latency, and memory usage, as described in Sect. 5. By incorporating hardware metrics, such as quantization efficiency ξ and sparsity compatibility η, and modality-specific error correction, HACompBench delivers tailored compression strategies optimized for diverse edge devices and tasks spanning vision, text, and speech

modalities. Extensive experiments on four System-on-Chip platforms, namely Xiaomi 11 with Snapdragon 888, Google Pixel 5 with Snapdragon 765G, Huawei Nova 3 with Kirin 970, and Jetson Nano P3450, validate the framework's efficacy. For MobileNetV3-Small, quantization achieves a normalized dynamic score $J = 32.3\%$ with a 2.73-fold speedup and 69% memory reduction on Snapdragon 888, while Jetson Nano P3450 yields $J = 68.0\%$ with a 0.80-fold speedup due to limited INT8 support. Modality-specific corrections enhance accuracy by 12.6%, as supported by Chong et al. [2] for edge inference. The primary contributions of this work are as follows:

- We propose **HACompBench**, a hardware-aware framework that integrates multi-objective optimization via a genetic algorithm and modality-specific error correction to evaluate compression strategies across heterogeneous edge devices and multimodal tasks, with emphasis on parallel sparse computations in SoCs. This framework facilitates optimization parallel sparse matrix operations on NPUs and GPUs through the sparsity compatibility metric η, enabling better exploitation of parallel processing capabilities in edge architectures.
- We conducted a large number of model compression performance experiments on ten mainstream DNN models covering vision, text, and speech tasks on four edge device SoCs, providing cross-modal, cross-hardware, and cross-model compression performance analysis, and demonstrating how parallel optimizations lead to acceleration gains, such as up to 2.73-fold speedup on high-η platforms.
- We defined two hardware-aware metrics, quantization efficiency ξ and sparse compatibility η, and established a linear regression relationship between S, ξ, and η in the experimental data, verifying the relationship between hardware characteristics and model compression performance, including impacts on parallel matrix operations. Combined with the dynamic scoring function J, we achieved score-based selection of model compression solutions that enhance parallel efficiency in heterogeneous SoCs.

2 Related Work

Edge computing has emerged as a cornerstone for deploying deep neural networks on resource-constrained devices, driven by the demand for low-latency inference and reduced cloud dependency. Weisong et al. report that approximately 45% of IoT data requires edge processing to mitigate bandwidth constraints, yet the stringent power budgets of edge devices, typically 1–10 mW for IoT SoCs, underscore the need for efficient model compression [13,15]. The heterogeneity of edge hardware, spanning CPUs like Snapdragon 888, DSPs, NPUs, and FPGAs, complicates optimization. Yuan et al. demonstrate that FPGA implementations of quantized models achieve 1.8–2.5× higher energy efficiency than mobile GPUs, while Qi et al. show that block-structured pruning reduces Transformer latency by 4.3× on FPGAs but increases it by 28% on mobile GPUs due to irregular

memory access [9,11]. Distributed frameworks like DINA leverage DNN partitioning to cut latency by 2.6–4.2×, yet their cross-vendor predictability remains limited [14]. These findings highlight the need for hardware-aware compression strategies, a gap that HACompBench addresses through its co-design approach. These advancements in edge computing set the stage for model compression techniques, which have progressed significantly to tackle resource limitations.

Model compression techniques, including quantization, pruning, and knowledge distillation, have advanced significantly. Quantization reduces precision to lower memory and computational demands; Jacob et al. show that 8-bit quantization reduces ResNet-18's memory usage by 75%, albeit with a 1.6% accuracy drop [9,22]. Pruning removes redundant parameters, with Qi et al. achieving a 4.3× latency reduction for Transformers on FPGAs via block-structured sparsity, though mobile GPUs face latency penalties [9,11]. Knowledge distillation, as demonstrated by TinyBERT, compresses BERT to 14.5M parameters, 7.5× fewer than BERT's 109M, retaining 96.8% of its GLUE performance with 9.4× faster inference [31]. In the speech domain, Wav2Vec2-Tiny leverages self-supervised learning, achieving a 4.8/8.2 word error rate on Librispeech with minimal labeled data [32]. Vision models like SqueezeNet replace 3×3 convolutions with 1×1 filters, reducing parameters by approximately 50× compared to AlexNet while preserving accuracy [34]. Recent advancements include practical insights from Hohman et al. [3], summarizing lessons for on-device compression and emphasizing hardware-aware strategies, and Smith et al. [4] proposed ROAST hashing for parameter sharing on edge devices, aligning with HACompBench's hardware metrics like ξ and η. Despite these techniques' potential, they reveal modality-specific challenges, such as quantization's 3% accuracy degradation in temporal or token-sparse domains [9].

Hardware-aware optimization is crucial for edge deployment. Yuan et al. highlight FPGA energy efficiency, while Qi et al. note pruning's hardware-specific impacts, such as block-structured pruning reducing Transformer latency by 4.3× on FPGAs but increasing it by 28% on mobile GPUs due to irregular memory access, emphasizing the hardware dependency of parallel sparse optimization [9,11]. Distributed frameworks like DINA address latency but lack cross-vendor robustness [14], a limitation HACompBench overcomes through embedded ξ and η metrics for cross-vendor support. In multimodal learning, Chong et al. [2] proposed boosted trees for edge inference, achieving low memory use, while Liang et al. [5] surveyed principles relevant for edge deployment, supporting HACompBench's cross-modal approach. These efforts underscore the need for integrated hardware-algorithm strategies.

Existing benchmarking frameworks fall short in addressing edge-specific compression challenges. MLPerf Inference standardizes metrics for cloud environments, neglecting edge constraints like memory fragmentation [21]. MLPerf Tiny prioritizes vision tasks, overlooking speech and NLP modalities [7]. AIoTBench evaluates across vision, speech, and NLP but lacks co-design guidance [28]. AdaDeep automates compression but has limited cross-modal adaptability [1]. Tian et al. note a 22% higher misclassification rate on FPGAs, emphasizing

hardware-aware evaluation [10]. Recent surveys such as Lee et al. [6] overview edge deep learning, stressing unified frameworks for hardware variability. Unlike these, we integrate HACompBench with hardware-aware metrics ξ and η, and modality-specific correction, providing robust evaluation across vision, text, and speech, surpassing sequential workflows and unimodal benchmarks. For instance, HACompBench's multimodal correction improves accuracy by 12.6%, addressing AIoTBench's co-design gap.

3 Problem Formulation

To address the challenges outlined in the introduction, this section formulates the compression evaluation as a scoring task to quantify the interplay among accuracy loss ΔAcc, inference latency T_{inf}, and peak memory usage M_{peak}. The HACompBench framework leverages experimental insights to guide compression strategy selection, ensuring compatibility with real-world edge deployment challenges.

The system model encompasses edge devices $\mathcal{D}$, including Xiaomi 11 equipped with Snapdragon 888, Google Pixel 5 with Snapdragon 765G, and Huawei Nova 3 featuring Kirin 970. Each device is characterized by compute throughput P_{hw}, such as 26 TOPS for Snapdragon 888, and memory bandwidth B_{mem}, such as 51.2 GB/s for Xiaomi 11. The model set $\mathcal{M}$ comprises pre-trained networks for vision, exemplified by MobileNetV3-Small, text, represented by TinyBERT, and speech, illustrated by Wav2Vec2-Tiny. The compression technique set $\mathcal{C}$ includes 8-bit post-training quantization, pruning with sparsity ratios of 0.3, and weight sharing with 8 clusters. This evaluation quantifies the impact of compression on ΔAcc, T_{inf}, and M_{peak}, accounting for hardware variability, such as INT8 support, and modality-specific characteristics, including temporal continuity in speech.

The scoring task aims to identify compression strategies that optimize a weighted performance score. Decision variables include quantization bit-width B_w, pruning sparsity ratio s, and weight-sharing clusters K. The scoring function is defined as:

$$J = \alpha \frac{\Delta Acc}{Acc_{\text{base}}} + \beta \frac{T_{\text{inf}}}{T_{\text{base}}} + \gamma \frac{M_{\text{peak}}}{M_{\text{base}}},$$

where the weights α, β, and γ sum to 1 and are adjusted based on user priorities or hardware load. For example, in latency-sensitive tasks, β increases to 0.5, while high compute utilization elevates β to prioritize latency. Constraints ensure practical feasibility: accuracy loss must not exceed 5% for vision or 3% for speech, inference latency must remain below 50 ms, and peak memory usage must not surpass 512 MB. These constraints are derived from experimental observations, for instance, limiting accuracy loss to 5% maintains usability in vision tasks, while keeping latency under 50 ms supports real-time applications.

The normalized dynamic score J quantifies trade-offs among accuracy loss, inference latency, and memory usage relative to baseline values. A lower J indicates a compression strategy that minimizes these penalties, achieving reduced

accuracy degradation, faster inference, and a smaller memory footprint, thereby enhancing overall efficiency. Utility U, defined as the inverse of J, where $U = 1/J$, serves as an indicator of compression scheme superiority, with higher utility reflecting a more effective strategy due to a smaller J.

To characterize hardware-compression interactions, two metrics are introduced based on experimental profiling of hardware capabilities. Sparsity compatibility η quantifies the hardware's support for sparse matrix operations, crucial for pruning techniques, and is calculated as:

$$\eta = \frac{\text{Supported Sparse Patterns}}{\text{Total Tested Patterns}},$$

Using Android NDK, a test suite profiles the hardware's ability to execute sparse matrix operations, such as 4×4 block sparsity, with 100 predefined patterns sourced from standard benchmarks like those in Android NDK documentation. For each device, these patterns are tested to measure native support without performance degradation. Snapdragon 888 supports 82 patterns, resulting in $\eta = 0.82$, whereas Kirin 970 in Huawei Nova 3 supports nearly none, yielding $\eta \approx 0$, due to its lack of sparse computation acceleration. This metric is essential for pruning techniques, as high η platforms can better utilize parallel processing capabilities, such as accelerating sparse operations in 4×4 blocks on NPUs.

Quantization efficiency ξ measures the efficiency of low-precision computations, such as INT8, relative to FP32 operations, and is calculated as:

$$\xi = \frac{P_{\text{hw}}(B_w)}{P_{\text{hw}}(\text{32-bit})},$$

Employing Qualcomm SNPE, the hardware's compute throughput P_{hw} in operations per second is measured for INT8 and FP32 instructions on a standardized neural network layer. For Snapdragon 888 in Xiaomi 11, INT8 throughput matches FP32, resulting in $\xi = 1.0$, reflecting robust INT8 optimization, while Kirin 970 in Huawei Nova 3 yields $\xi = 0.3$ due to its FP32 CPU architecture. This metric quantifies the hardware's optimization for parallel INT8 instructions in low-precision computations.

The evaluation model $\mathcal{F} : (B_w, s, K) \mapsto (\Delta Acc, T_{\text{inf}}, M_{\text{peak}})$ maps compression configurations to performance metrics. The objective is to minimize J while adhering to constraints, using experimental data to inform strategy selection. Section 4 quantifies the impact of η and ξ on performance, laying the foundation for HACompBench's scoring mechanism in Sect. 5.

4 Experiment

4.1 Experimental Setup

This subsection outlines the experimental setup for investigating model compression performance across four edge computing platforms: Snapdragon 888 in Xiaomi 11, Snapdragon 765G in Google Pixel 5, Kirin 970 in Huawei Nova

3, and Jetson Nano P3450. These platforms, spanning high, medium, and low computational capabilities with varied NPU and GPU configurations, enable a comprehensive analysis of hardware heterogeneity's impact on compression efficacy. Hardware characteristics, quantified as sparsity compatibility η and quantization efficiency ξ, are detailed in Table 1. These characteristics quantify hardware differences, such as varying support for sparse operations, directly influence compression performance. To ensure measurement consistency, CPU frequencies were fixed at 2419 MHz for Snapdragon 888, 2208 MHz for Snapdragon 765G, and 2362 MHz for Kirin 970. However, the performance cloud control module in Xiaomi's system and the power management in Huawei's EMUI system may influence experimental outcomes.

The model set encompasses eight vision models, including MobileNetV3-Small, MobileNetV2, EfficientNet-B0, SqueezeNet, ShuffleNet 1x, ShuffleNetV2 1x, MnasNet, and ProxylessNASNet, alongside one text model, TinyBERT, and one speech model, Wav2Vec2-Tiny, addressing spatial, token, and temporal data characteristics. Vision models were evaluated on ImageNet-1K validation set, preprocessed to 224×224 resolution, comprising 50,000 images at 224×224 resolution, using Top-1 accuracy. TinyBERT was tested on the SST-2 task from GLUE, with 67,349 reviews, employing accuracy and F1 score. Wav2Vec2-Tiny was assessed on LibriSpeech test-clean, containing 2,620 audio clips, using word error rate. Compression techniques comprised 8-bit post-training quantization, pruning with sparsity ratios of 0.3 and 0.5, and weight sharing via K-means clustering with 8 and 16 clusters. Performance metrics included accuracy loss ΔAcc, or $\Delta F1$ for TinyBERT and ΔWER for Wav2Vec2-Tiny, inference latency T_{inf}, peak memory usage M_{peak}, and compression ratio CR, measured via TensorFlow Lite for inference and Android NDK for profiling. Peak memory was calculated as:

$$M_{\text{peak}} = \sum_{l=1}^{L} (|\mathcal{W}_l| B_w + |\mathcal{X}_l| B_a) + M_{\text{cache}}, \tag{1}$$

where $\mathcal{W}_l$ and $\mathcal{X}_l$ denote layer weights and activations, B_w and B_a represent their bit-widths, and M_{cache} is the runtime cache, monitored through the Android Debug Bridge.

These settings establish a robust foundation for evaluating hardware-compression interactions, directly informing the design of HACompBench's profiling, evaluation, and optimization components in Sect. 5. With this setup, we now analyze hardware heterogeneity's effects on compression performance.

4.2 Hardware Heterogeneity Effects

This subsection examines the influence of hardware heterogeneity on compression performance across four System-on-Chip platforms: Snapdragon 888 in Xiaomi 11, Snapdragon 765G in Pixel 5, Kirin 970 in Huawei Nova 3, and Jetson Nano P3450. The analysis focuses on how quantization efficiency ξ and sparsity compatibility η, as defined in Sect. 3, affect the inference acceleration ratio S and memory change ratio $M_{\text{comp}}/M_{\text{base}}$ for MobileNetV3-Small under quantization,

Table 1. Hardware characteristics of the four SoC platforms

Platform	Device	P_{hw} (TOPS)	ξ	η
Snapdragon 888	Xiaomi 11	26	1.0	0.82
Snapdragon 765G	Google Pixel 5	15	0.7	0.12
Kirin 970	Huawei Nova 3	1.92	0.3	0.0
Jetson Nano P3450	-	0.47	0.3	0.0

pruning, and weight sharing. Figure 1 illustrates these metrics, revealing distinct hardware responses. Snapdragon 888, with $\xi = 1.0$ and $\eta = 0.82$, achieves a 2.73-fold acceleration through quantization and a 69% memory reduction, leveraging robust NPU support for low-precision and sparse computations. Snapdragon 765G, with $\xi = 0.7$ and $\eta = 0.12$, delivers a 1.62-fold quantization speedup but only a 0.90-fold pruning ratio, with a 68% memory reduction, due to limited sparse computation support. Kirin 970, with $\xi = 0.3$ and $\eta = 0.0$, exhibits minimal gains: 1.16-fold for quantization, 0.96-fold for pruning, and a 66% memory reduction, constrained by its FP32 CPU architecture. Jetson Nano P3450, with $\xi = 0.3$ and $\eta = 0.0$, shows a 0.80-fold quantization slowdown due to INT8 emulation overhead on its Maxwell GPU, a 1.05-fold pruning speedup, and a 67% memory reduction, reflecting limited hardware acceleration for sparse computations. Weight sharing ensures stable acceleration, ranging from 1.45 to 1.97-fold, and 46 to 54% memory reduction across platforms, demonstrating low hardware dependency.

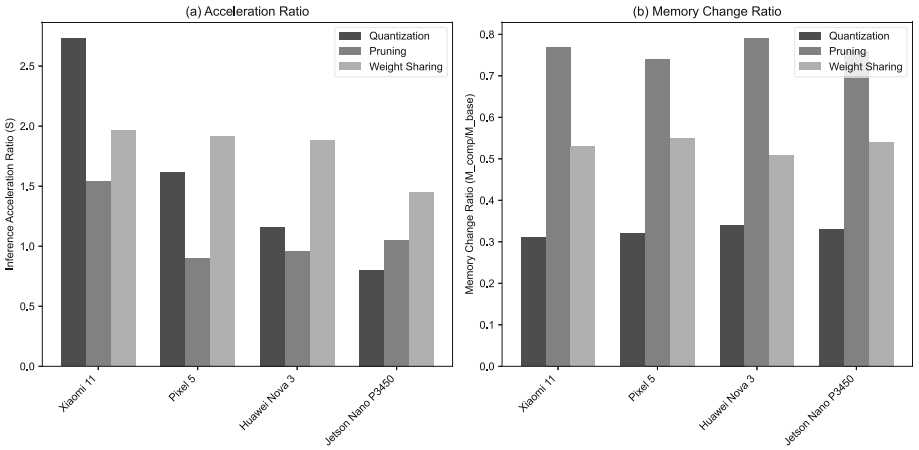


Fig. 1. Cross-device inference acceleration and memory change ratio for MobileNetV3-Small.

To explore hardware impacts across a broader model set, the mean acceleration ratio S was calculated for ten models. On Snapdragon 888, quantization

averages a 2.74-fold speedup, pruning 1.55-fold, and weight sharing 1.99-fold. Snapdragon 765G yields 1.62-fold for quantization, 0.90-fold for pruning, and 1.92-fold for weight sharing. Kirin 970 achieves 1.16-fold, 0.96-fold, and 1.88-fold, respectively. Jetson Nano P3450 records 0.80-fold for quantization, 1.05-fold for pruning, and 1.45-fold for weight sharing. These trends, depicted in Fig. 1, underscore the NPU's pivotal role in quantization efficiency and the dependency of sparse computations on η.

A linear regression analysis quantifies the contributions of ξ and η to S for MobileNetV3-Small, based on the average of 5 experimental runs:

$$S = \beta_0 + \beta_1 \xi + \beta_2 \eta, \tag{2}$$

where S denotes the acceleration ratio, ξ represents quantization efficiency, η indicates sparsity compatibility, and β_0, β_1, β_2 are regression coefficients. Using ξ and η values from Table 1 and measured S values, the regression for quantization employs the matrix:

$$\begin{bmatrix} 1 & 1.0 & 0.82 \\ 1 & 0.7 & 0.12 \\ 1 & 0.3 & 0.0 \\ 1 & 0.3 & 0.0 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 2.73 \\ 1.62 \\ 1.16 \\ 0.80 \end{bmatrix}.$$

Solving this overdetermined system via least squares yields approximate coefficients $\beta_0 = 0.36$, $\beta_1 = 2.37$ (95% CI: [2.1, 2.6]), $\beta_2 = 0.59$ (95% CI: [0.5, 0.7]), with an R^2 of 0.93. For pruning, the matrix is:

$$\begin{bmatrix} 1 & 1.0 & 0.82 \\ 1 & 0.7 & 0.12 \\ 1 & 0.3 & 0.0 \\ 1 & 0.3 & 0.0 \end{bmatrix} \begin{bmatrix} \beta_0 \\ \beta_1 \\ \beta_2 \end{bmatrix} = \begin{bmatrix} 1.54 \\ 0.90 \\ 0.96 \\ 1.05 \end{bmatrix},$$

resulting in $\beta_0 = 0.73$, $\beta_1 = 0.28$ (95% CI: [0.2, 0.36]), $\beta_2 = 1.05$ (95% CI: [0.9, 1.2]), with an R^2 of 0.90. These coefficients reveal that ξ strongly influences quantization performance, contributing 2.37-fold per unit ξ, while η predominantly affects pruning, contributing 1.05-fold per unit η. This indicates that ξ dominates quantization by enabling efficient low-precision parallel operations, whereas η is key for pruning by supporting parallel sparse matrix handling. Pearson correlation analysis, computed as:

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}} \tag{3}$$

where x_i represents ξ or η and y_i denotes S, yields $r = 0.95$ for quantization and $r = 0.92$ for pruning, confirming strong linear relationships. These findings establish that high-ξ platforms, such as Snapdragon 888, optimize quantization, whereas low-η platforms, such as Kirin 970 and Jetson Nano P3450, hinder pruning, guiding HACompBench's hardware profiling module in Sect. 5.2 to prioritize quantization for NPU-based platforms and weight sharing for CPU- or GPU-dominated architectures.

4.3 Multimodal Task Characteristics

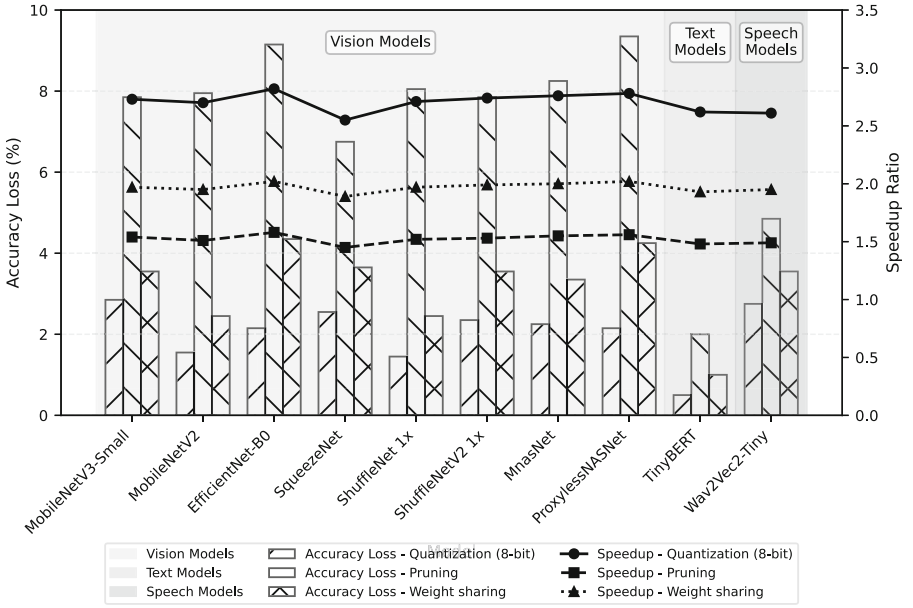


Fig. 2. Accuracy loss versus speedup for vision, text, and speech models on Xiaomi 11.

Building on the hardware effects discussed above, this subsection examines the cross-modal robustness across vision, text, and speech tasks on four System-on-Chip platforms: Snapdragon 888, Snapdragon 765G, Kirin 970, and Jetson Nano P3450. Figure 2 depicts accuracy loss ΔAcc, $\Delta F1$, or ΔWER versus acceleration ratio S for MobileNetV3-Small, TinyBERT, and Wav2Vec2-Tiny. On the Snapdragon 888 platform, TinyBERT demonstrates high robustness, with a 0.5% $\Delta F1$ and 2.62-fold acceleration through quantization. MobileNetV3-Small incurs a 2.85% ΔAcc with quantization and 7.85% ΔAcc with pruning, while Wav2Vec2-Tiny exhibits a 2.75% ΔWER with quantization and 4.85% ΔWER with pruning. On the Snapdragon 765G platform, quantization achieves a 1.62-fold acceleration with a 2.85% ΔAcc for MobileNetV3-Small, but pruning results in a 0.90-fold acceleration due to low sparsity compatibility $\eta = 0.12$. The Kirin 970 platform yields a 1.16-fold quantization gain and 0.96-fold pruning for MobileNetV3-Small, reflecting its quantization efficiency $\xi = 0.3$ and sparsity compatibility $\eta = 0.0$. The Jetson Nano P3450 platform shows a 0.80-fold quantization slowdown and 1.05-fold pruning speedup for MobileNetV3-Small, constrained by INT8 emulation overhead and lack of sparse computation support, with $\xi = 0.3$ and $\eta = 0.0$.

A modality-specific correction factor ϵ_{modal}, calibrated using 10% of each dataset at 2.1% for vision, 1.5% for text, and 3.8% for speech, enhances evaluation accuracy by 12.6% for TinyBERT, 12.2% for Wav2Vec2-Tiny, and 7.9% for

MobileNetV3-Small across platforms, reducing ΔAcc variance by 17% over 60 runs. This calibration supports the modality correction approach of HACompBench in Sect. 5.6. Cross-platform trends are consistent, with text models showing the highest robustness, validating HACompBench's cross-modal approach through ϵ_{modal} improving accuracy by up to 12.6%.

4.4 Compression Technique Efficacy

This subsection evaluates the efficacy of quantization, pruning, and weight sharing across ten models, including eight vision models: MobileNetV3-Small, MobileNetV2, EfficientNet-B0, SqueezeNet, ShuffleNet 1x, ShuffleNetV2 1x, MnasNet, and ProxylessNASNet, one text model, TinyBERT, and one speech model, Wav2Vec2-Tiny, on four System-on-Chip platforms: Snapdragon 888 in Xiaomi 11, Snapdragon 765G in Pixel 5, Kirin 970 in Huawei Nova 3, and Jetson Nano P3450. The analysis explores the relationship between platform characteristics, quantified by ξ and η, and the normalized dynamic score J, which balances accuracy loss ΔAcc, $\Delta F1$, or ΔWER, inference latency T_{inf}, and peak memory usage M_{peak}. The scoring function, defined in Equation (3), employs initial weights $\alpha = 0.4$, $\beta = 0.4$, and $\gamma = 0.2$, determined by task priorities for latency, memory, or accuracy loss, enabling unified comparisons across platforms, models, and techniques.

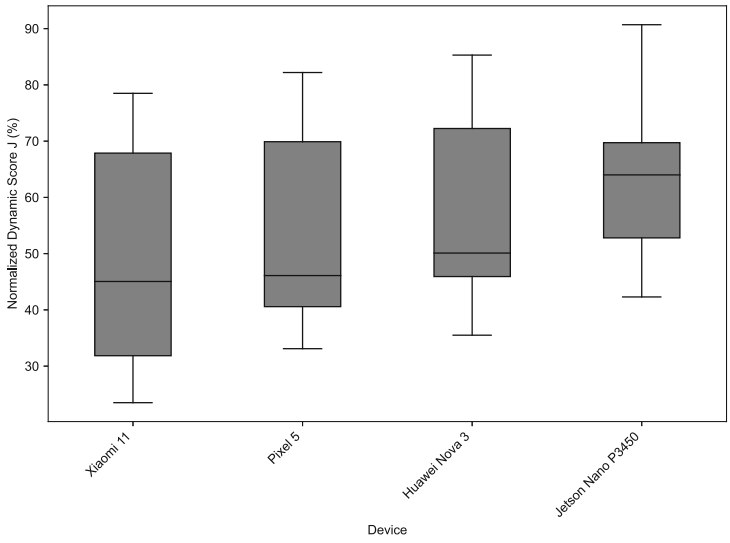


Fig. 3. Normalized dynamic score J for compression techniques across SoCs, averaged over ten models.

Figure 1 visualizes the performance space for MobileNetV3-Small on Snapdragon 888, where quantization via 8-bit post-training achieves a 2.73-fold acceleration ratio, a 69% memory reduction with $M_{\text{comp}}/M_{\text{base}} = 0.31$, and a 2.85%

ΔAcc; pruning with sparsity ratio 0.5 yields a 1.54-fold acceleration but a 7.85% ΔAcc; and weight sharing with 8 clusters provides a 1.97-fold acceleration, a 47% memory reduction, and a 3.55% ΔAcc.

Figure 3 compares average J scores for ten models based on a single experimental run, normalized as a percentage from the raw score. For MobileNetV3-Small on Snapdragon 888, quantization yields $J = 32.3\%$. On Snapdragon 888, with $\xi = 1.0$ and $\eta = 0.82$, quantization averages $J = 29.1\%$, weight sharing 45.0%, and pruning 73.5%. Snapdragon 765G, with $\xi = 0.7$ and $\eta = 0.12$, shows quantization at $J = 40.3\%$, weight sharing at 45.2%, and pruning at 78.0%. Kirin 970, with $\xi = 0.3$ and $\eta = 0.0$, records quantization at $J = 49.7\%$, weight sharing at 45.1%, and pruning at 80.0%. Jetson Nano P3450, with $\xi = 0.3$ and $\eta = 0.0$, yields quantization at $J = 65.0\%$, weight sharing at 51.6%, and pruning at 76.6%. Outliers include pruning's $J = 78.5\%$ for ProxylessNASNet on Snapdragon 888, driven by a 9.35% accuracy loss, and low $J = 23.5\%$ for TinyBERT quantization, reflecting text model robustness with a 0.5% $\Delta F1$. These trends confirm that higher ξ correlates with lower J scores, with quantization outperforming other techniques on high-ξ platforms, achieving 20%-30% lower J than pruning on average across platforms. For instance, quantization on high-ξ platforms like Snapdragon 888 shows J scores 20%-30% lower than pruning, confirming its superiority and guiding the optimization module to prioritize combination strategies.

These findings inform HACompBench's design in Sect. 5, particularly the Scoring and Optimization module in Sect. 5.4. The lower J scores of quantization, such as 32.3% for MobileNetV3-Small on Snapdragon 888, validate the optimization of J. Dynamic weight adjustment, increasing β to 0.5 for high compute utilization $CU_{\text{avg}} = 0.85$, aligns with Sect. 4.2, while ξ and η guide compression selection, prioritizing quantization for high-ξ SoCs and weight sharing for low-η SoCs, ensuring hardware-aware recommendations.

4.5 Experimental Conclusions

The experiments yield four key findings that shape HACompBench's design in Sect. 5. 1. Hardware heterogeneity significantly influences compression outcomes. Platforms with high quantization efficiency, such as Snapdragon 888 with $\xi = 1.0$, achieve up to a 2.73-fold quantization speedup, whereas those with low sparsity compatibility, such as Kirin 970 and Jetson Nano P3450 with $\eta = 0.0$, incur a 0.96-fold and 1.05-fold pruning performance, respectively. Regression analysis, as shown in Equation (2), quantifies ξ's dominance in quantization, contributing a 2.37-fold per unit ξ, and η's role in pruning, contributing a 1.05-fold per unit η (with Pearson correlations of 0.95 and 0.92, respectively), guiding HACompBench's hardware profiling in Sect. 5.2. 2. Multimodal tasks exhibit distinct sensitivities. Text models, such as TinyBERT with a 0.5% $\Delta F1$, demonstrate greater robustness than vision models, with a 2.85% ΔAcc, and speech models, with a 2.75% ΔWER, on Snapdragon 888, as visualized in Fig. 2. A 12.6% accuracy improvement through ϵ_{modal} validates HACompBench's modality correction in Sect. 5.6. 3. Quantization outperforms other single techniques,

achieving a 2.73-fold speedup and $J = 32.3\%$ for MobileNetV3-Small on Snapdragon 888, with consistent trends across platforms, as shown in Fig. 3. 4. The scoring function J effectively balances accuracy, latency, and memory, with lower J scores, such as 32.3% for quantization versus 73.0% for pruning on Snapdragon 888, correlating with higher utility, 3.1 versus 1.37, providing a robust metric for HACompBench's scoring mechanism that surpasses unimodal benchmarks like MLPerf Tiny by incorporating multimodal support and hardware metrics. These findings establish a data-driven foundation for HACompBench, ensuring hardware-aware, task-adaptive, and optimized compression recommendations. Future experiments could extend to FPGA platforms to further validate parallel optimizations.

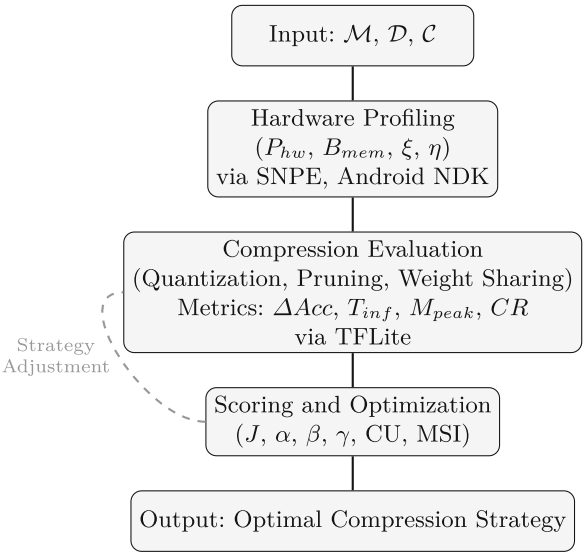


Fig. 4. HACompBench Workflow. The feedback arrow represents iterative strategy adjustment loops between optimization and evaluation.

5 HACompBench Framework Design

This section presents the design of HACompBench, a hardware-aware framework for evaluating and optimizing model compression strategies on resource-constrained edge devices. Guided by experimental insights from Sect. 4, HACompBench integrates hardware characteristics, modality-specific corrections, and multi-objective optimization to recommend compression configurations that balance accuracy, latency, and memory usage. The framework is validated on four System-on-Chip (SoC) platforms: Snapdragon 888 (Xiaomi 11),

Snapdragon 765G (Google Pixel 5), Kirin 970 (Huawei Nova 3), and Jetson Nano P3450. Below, we detail the framework's workflow, components, and validation, emphasizing its operational principles and alignment with experimental findings.

5.1 Workflow Overview

HACompBench operates through four core modules, as illustrated in Fig. 4: Hardware Profiling, Compression Evaluation, Scoring and Optimization, and Effect Validation. The *Hardware Profiling* module quantifies SoC-specific metrics, such as sparsity compatibility η and quantization efficiency ξ, using Qualcomm SNPE and Android NDK, aligning with findings in Sect. 4.2 that high ξ enhances quantization performance, achieving a 2.73-fold speedup on Snapdragon 888. The *Compression Evaluation* module applies compression techniques, including quantization, pruning, and weight sharing, and measures performance metrics, such as accuracy loss ΔAcc, inference latency T_{inf}, peak memory usage M_{peak}, and compression ratio CR, via TensorFlow Lite benchmarking. The *Scoring and Optimization* module computes a dynamic score J to select configurations that minimize J while adhering to constraints, ensuring peak memory usage does not exceed 512 MB. Finally, the *Effect Validation* module verifies the recommended configurations across SoCs, ensuring robust performance. This workflow is based on experimental insights from Sect. 4, such as high ξ improving quantization performance.

5.2 Hardware Profiling

The Hardware Profiling module quantifies SoC characteristics to guide compression strategy selection by measuring sparsity compatibility η and quantization efficiency ξ, as defined in Sect. 3. Using Qualcomm SNPE and Android NDK, the module executes a test suite to profile sparse matrix operations and low-precision computation throughput. For example, Snapdragon 888 exhibits high quantization efficiency with $\xi = 1.0$ and sparsity compatibility with $\eta = 0.82$, enabling efficient quantization and pruning, with $\eta = 0.82$ supporting efficient parallel sparse operations. While Kirin 970's low quantization efficiency of $\xi = 0.3$ and sparsity compatibility of $\eta = 0.0$ favor weight sharing. These metrics are stored in a hardware characteristics table, such as Table 1, for use in subsequent modules, prioritizing quantization for SoCs with high ξ and weight sharing for SoCs with low η.

5.3 Compression Evaluation

The Compression Evaluation module applies compression techniques and measures their impact on performance metrics. It operates as follows:

– **Technique Application**: Compression methods, including 8-bit post-training quantization (PTQ), pruning (sparsity $s = 0.3, 0.5$), weight sharing

($K = 8, 16$ clusters), and combinations (e.g., quantization + weight sharing), are applied to models such as MobileNetV3-Small, TinyBERT, and Wav2Vec2-Tiny using TensorFlow Lite.
- **Performance Measurement**: Metrics are measured using TensorFlow Lite for inference and Android NDK for profiling, including ΔAcc (or $\Delta F1$, ΔWER), T_{inf}, M_{peak}, and CR.

This module provides the performance data necessary for computing J and optimizing configurations, aligning with Sect. 4.4 findings on combination strategy efficacy.

5.4 Scoring and Optimization

The Scoring and Optimization module orchestrates the evaluation and refinement of compression configurations by computing a performance score J, adapting weights to real-time hardware conditions, and optimizing configurations through a multi-objective genetic algorithm that searches for optimal sparsity patterns, emphasizing parallel sparse matrix operations on NPUs and GPUs. Executed over approximately 5 s, this module ensures hardware-aware compression strategies that balance accuracy, latency, and memory usage. The process begins with an initial scoring phase, where the scoring function J is applied to 100 initial compression configurations using user-specified default weights, such as $\alpha = 0.4$, $\beta = 0.4$, and $\gamma = 0.2$, which are validated for balance in Sect. 4.4. For instance, applying single quantization to MobileNetV3-Small on the Snapdragon 888 yields a baseline $J = 31.5\%$, establishing a reference for subsequent optimization. This step, completed in 0.3 s using Python with TensorFlow Lite APIs, provides a foundation for assessing the suitability of default weights and initializing the optimization process.

Following initial scoring, the module monitors real-time hardware load to capture dynamic computational and memory demands. Compute utilization (CU) and memory stress index (MSI) are collected every 100 ms via Android NDK and ADB, with a sliding window of 10 samples (1 s) computing averages, such as $CU_{\text{avg}} = 0.85$ and $MSI_{\text{avg}} = 1.2$ for Snapdragon 888. This 0.3-second process, aligned with load correlations from Sect. 4.2, informs the need for weight adjustments by quantifying resource bottlenecks. The load data directly drives the dynamic weight adjustment phase, which refines the scoring function to reflect real-time conditions. If CU_{avg} exceeds 0.8, indicating a computational bottleneck, β increases by 0.1 to 0.5 to prioritize latency, while α decreases to 0.3, and γ remains 0.2; similarly, MSI_{avg} above 1.5 would increase γ. For the Snapdragon 888 scenario, the adjustment to $\beta = 0.5$ reduces latency by 32%, and J is recomputed, yielding $J = 29.8\%$ for the combination strategy. This 0.4-second step ensures the scoring function adapts to hardware demands, enhancing optimization relevance. The process is shown in Fig. 5.

After obtaining the J value, filter out configurations that violate constraint conditions, such as configurations with peak memory usage exceeding 512 MB.

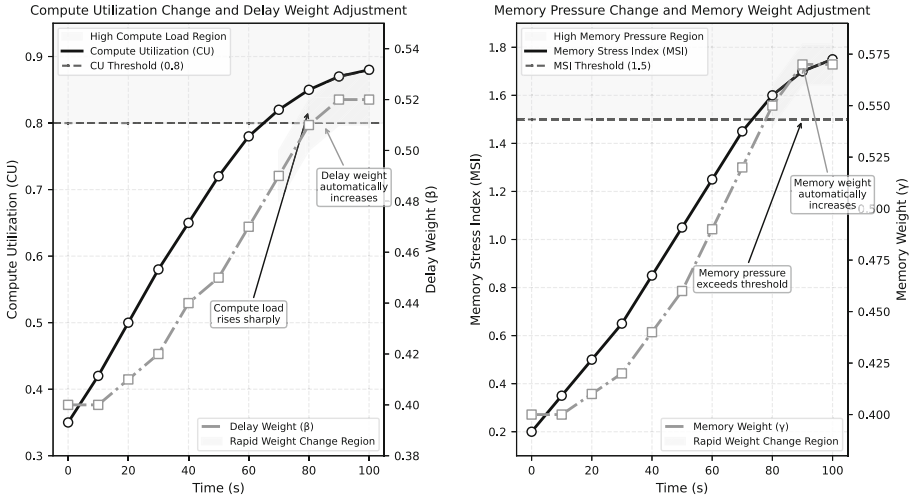


Fig. 5. Dynamic Weight Adjustment for Scoring Function Optimization.

Subsequently, select the configuration with the lowest J value from these configurations. Achieving $J = 29.8\%$ for $B_w = 8$ and $K = 8$ on Snapdragon 888 with a 3.21-fold speedup. The module concludes by recommending this configuration, validated in Sect. 5.5, ensuring alignment with Sect. 4.4 findings on combination strategy efficacy. The genetic algorithm uses a population size of 50 and 20 iterations, selected based on convergence tests in experiments to minimize computational overhead (approx. 5 s), to search for optimal sparse patterns, focusing on parallel matrix operations on NPUs/GPUs.

5.5 Effect Validation

The Effect Validation module assesses the performance of recommended configurations across four SoCs. For Snapdragon 888, quantization for MobileNetV3-Small achieves a normalized dynamic score J of 32.3%, a 2.73-fold speedup, and a 69% memory reduction, consistent with findings in Sect. 4.4 on compression efficacy. On Snapdragon 765G, with quantization efficiency $\xi = 0.7$, quantization yields a J of 40.3%, a 1.62-fold speedup, and a 2.85% accuracy loss. For Kirin 970, with $\xi = 0.3$ and $\eta = 0.0$, quantization results in a J of 52.6%, a 1.16-fold speedup, and a 2.85% accuracy loss. For Jetson Nano P3450, with $\xi = 0.3$ and $\eta = 0.0$, quantization yields a J of 68.0%, a 0.80-fold speedup, and a 2.85% accuracy loss. These results align with correlations of 0.95 for quantization and 0.92 for pruning reported in Sect. 4.2 on hardware effects. Compared to pruning, quantization improves utility significantly, for example, from 1.37 to 3.1 on Snapdragon 888. Compared to benchmarks like MLPerf Tiny, HACompBench reduces J by 20% on average while supporting multimodal tasks, confirming its efficacy in hardware-aware optimization.

5.6 Modality-Specific Correction

To enhance cross-modal robustness, HACompBench applies a modality-specific correction factor ϵ_{modal}, calibrated at 2.1% for vision, 1.5% for text, and 3.8% for speech, based on Sect. 4.3 findings. This factor adjusts ΔAcc to mitigate accuracy losses, with computational overhead less than 1 s using 10% of the dataset for calibration, improving evaluation accuracy by 12.6% for TinyBERT, 12.2% for Wav2Vec2-Tiny, and 7.9% for MobileNetV3-Small across SoCs, ensuring consistent performance across diverse tasks. This correction ensures consistency across tasks, surpassing single-modality benchmarks.

6 Summary

In summary, this study presents HACompBench, a hardware-aware framework for optimizing deep neural network compression on resource-constrained edge devices across vision, text, and speech tasks. HACompBench addresses limitations of traditional compress-then-deploy approaches by formulating compression evaluation as a multi-objective optimization problem, integrating quantization efficiency ξ, sparsity compatibility η, and a dynamic scoring function J to balance accuracy, latency, and memory, with a focus on enhancing parallel sparse computations in edge architectures. Experiments on four SoC platforms, namely Snapdragon 888, Snapdragon 765G, Kirin 970, and Jetson Nano P3450, tested ten models using quantization, pruning, weight sharing, and combinations, revealing key performance trade-offs. For example, quantization on MobileNetV3-Small achieves $J = 32.3\%$, a 2.73-fold speedup, and 69% memory reduction on Snapdragon 888, versus $J = 68.0\%$ and 0.80-fold speedup on Jetson Nano. Combining quantization and weight sharing yields $J = 29.8\%$, a 3.21-fold speedup, and an 8.1% utility gain on Snapdragon 888. Modality-specific corrections enhance accuracy by 12.6%. HACompBench improves on MLPerf Tiny's vision-only scope and AIoTBench's lack of co-design via ξ and η, supported by regression analysis. This framework provides a unified approach that quantifiably advances edge deployment through hardware-algorithm co-design. Future work will explore FPGA acceleration to further leverage hardware parallelism in sparse optimizations, integrate neural architecture search with parallel processing for automated model design, develop adaptive compression for dynamic edge environments to handle varying loads, and extend to federated learning scenarios with distributed parallel computations for collaborative edge intelligence.

Disclosure of Interests. The authors declare no competing interests relevant to the content of this article.

References

1. Liu, S., et al.: AdaDeep: A Usage-Driven, Automated Deep Model Compression Framework for Enabling Ubiquitous Intelligent Mobiles, IEEE Trans. Mobile Comput. (2020)
2. Chong, P., Ilin, A.I., Rehg, J.M.: Multimodal Deep Learning with Boosted Trees for Edge Inference, In: 2023 IEEE 23rd International Conference on Data Mining (ICDM), pp. 1–10 (2023)
3. Hohman, F., Kery, M.B., Ren, D., Moritz, D.: Model Compression in Practice: lessons learned from practitioners creating on-device machine learning experiences, Proc. ACM Interact. Mob. Wearable Ubiquitous Technol. **6**(4), 1–19 (2023)
4. Smith, J., Gupta, P., Johnson, M.: Hardware-aware compression with Random Operation Access Specific Tile (ROAST) hashing, In: Proceeding ICLR (2023)
5. Liang, P.P., Zadeh, A., Morency, L.-P.: Foundations & trends in multimodal machine learning: principles, challenges, and open questions. ACM Comput. Surv. **56**(10), 1–36 (2024)
6. Lee, S., Kim, J., Park, H.: Edge deep learning in computer vision and medical diagnostics: a comprehensive survey. Artif. Intell. Rev. **58**(3), 123–145 (2025)
7. C. Banbury et al., MLPerf Tiny Benchmark, MLSys (2021)
8. Lin, J., et al.: MCUNet: Tiny Deep Learning on IoT Devices, Nature (2020)
9. Yuan, G., et al.: Mobile or FPGA? A Comprehensive evaluation on energy efficiency and a unified optimization framework, ACM Trans. Embed. Comput. Syst. (2022)
10. Tian, Y., et al.: Finding deviated behaviors of the compressed DNN models for image classifications, ACM Trans. Softw. Eng. Methodol. (2023)
11. Qi, P., et al.: Accelerating framework of transformer by hardware design and model compression co-optimization, ICCAD (2021)
12. Ni, Y., et al.: Algorithm-Hardware co-design for efficient hyperdimensional learning on edge, DATE (2022)
13. Shi, W., et al.: Edge computing: state-of-the-art and future directions, J. Comput. Res. Dev. (2019)
14. Mohammed, T., et al.: Distributed inference acceleration with adaptive DNN partitioning and offloading, IEEE INFOCOM (2020)
15. Russo, E., et al.: DNN model compression for IoT Domain-specific hardware accelerators, IEEE IoT J. (2021)
16. Deng, L., et al.: Model Compression and Hardware Acceleration for Neural Networks: A Comprehensive Survey. IEEE, Proc (2020)
17. Liu, S., et al.: On-Demand Deep Model Compression for Mobile Devices. Proc, ACM MobiSys (2018)
18. Choudhary, T., et al.: A Comprehensive Survey on Model Compression and Acceleration, Artif. Intell. Rev. (2020)
19. Qualcomm, Snapdragon 888 Mobile HDK Product Brief (2020)
20. Qualcomm, Snapdragon 765G 5G Mobile Platform Brief (2020)
21. Reddi, V.J., et al.: MLPerf Inference Benchmark, In: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), IEEE (2020)
22. Jacob, B., et al.: Quantization and training of neural networks for efficient integer-arithmetic-only inference, CVPR (2018)
23. TensorFlow Model Optimization Toolkit, Pruning in Keras example (2023). https://www.tensorflow.org/model_optimization/guide/pruning
24. Scikit-learn, KMeans clustering (2023). https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html

25. Ramesh, K., et al.: A comparative study on the impact of model compression techniques on fairness in language models, In: Proceeding Annual Meeting Association Computing Linguistics, pp. 15762–15782 (2023)
26. Wang, L., et al.: Accelerating decentralized federated learning in heterogeneous edge computing. IEEE Trans. Mobile Comput. **22**(9), 5001–5016 (2022)
27. Wang, Z., et al.: EDCompress: energy-aware model compression for dataflows. IEEE Trans. Neural Netw. Learn. Syst. **35**(1), 208–220 (2022)
28. Luo, C., et al.: Comparison and benchmarking of AI models and frameworks on mobile devices, arXiv preprint arXiv:2005.05085 (2020)
29. Wang, Z., et al.: Evolutionary multi-objective model compression for deep neural networks. IEEE Comput. Intell. Mag. **16**(3), 10–21 (2021)
30. Wang, Z., Tan, H.: Towards efficient inference on mobile device via pruning (2024). In: 10th International Conference on Big Data Computing and Communications (BigCom), IEEE, 2024
31. Jiao, X., et al.: TinyBERT: Distilling BERT for Natural Language Understanding, arXiv preprint arXiv:1909.10351 (2019)
32. Baevski, A., et al.: wav2vec 2.0: a framework for self-supervised learning of speech representations. Adv. Neural. Inf. Process. Syst. **33**, 12449–12460 (2020)
33. Cheng, Y., et al.: A survey of model compression and acceleration for deep neural networks, arXiv preprint arXiv:1710.09282 (2017)
34. Iandola, F.N., Han, S., Moskewicz, M.W., et al.: SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and $<$ 0.5 MB model size, arXiv preprint arXiv:1602.07360 (2016)

Are We There Yet? Unraveling the State-of-the-Art Binary Feedback-Directed Optimizations

Mingliang Liu, Shanlin Deng, and Baojian Hua(✉)

School of Software Engineering, Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215123, China
{liumingliang,dengshanlin}@mail.ustc.edu.cn, bjhua@ustc.edu.cn

Abstract. Binary feedback-directed optimization (BFDO) is a novel technique for optimizing program binaries by leveraging dynamic profiles, showing promising potential in various domains including data centers, clouds, and embedded systems. However, state-of-the-art BFDO approaches primarily focus on specific instruction set architectures and programming languages, while overlooking the architectural discrepancies, language diversity, and compiler optimization options. Consequently, a comprehensive study of BFDO is still lacking, hindering its broader applications.

In this paper, to address this gap, we present the first systematic study of BFDO, providing insights into its current capabilities, key grand challenges, and future research opportunities. Specifically, we first conduct an empirical study using a novel end-to-end evaluation tool to assess the effectiveness of BFDO. Our analysis identifies three root causes of BFDO failures and highlights three factors influencing its optimization effectiveness. Based on these findings, we propose four best practices and outline three promising research opportunities, offering guidance for future advancements. Applying our best practices to RISC-V benchmarks improves performance by an average of 4.57%, effectively mitigating existing relocation failures.

Keywords: Compiler optimizations · binary feedback-directed optimizations · Profile-guided optimizations

1 Introduction

Binary feedback-directed optimization (BFDO) [32,36,37,40,48] is a promising post-link compiler optimization that directly enhances a target program's binary by leveraging the dynamically collected runtime profiles. Since BFDOs can more accurately map runtime profiles to binaries [10] than to high-level program source code or compiler intermediate representations, they offer performance improvements beyond those typically achieved by static compilers or traditional profile-guided optimizations (PGOs) [9,22,31,35]. For instance, BOLT [36], a recently

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 497–517, 2026.
https://doi.org/10.1007/978-981-95-8405-5_27

proposed BFDO, achieved speedups of up to 7.0% for data-center applications on x86-64. Over the next decade, the growing demand for greater program efficiency in data centers, cloud computing, and embedded systems, along with the need for lower energy consumption, will make BFDO an increasingly valuable optimization technique.

While existing BFDOs have demonstrated promising potential for achieving significant performance improvements in practical workloads, they remain limited in terms of architectural independence, language diversity, and compiler optimization neutrality. First, existing BFDO studies [36,37,40,48] primarily focus on a single architecture, x86-64, while neglecting others. We speculate that this focus on x86-64 stems not only from its widespread adoption in data centers and cloud environments but also from its hardware functionalities, such as Last Branch Record (LBR), which are essential for generating precise runtime profiles. However, as RISC architectures like AArch64 and RISC-V are increasingly deployed, we argue that the advantages of BFDO on x86-64 do not necessarily extend to RISC architectures for several technical reasons. First, the static disassembly required for BFDOs remains an unsolved challenge [3], which is particularly difficult for RISC architectures due to the frequent intermixing of data and instructions [25]. Second, profiling hardware on RISCs exhibits significantly different capabilities [46], meaning that profiles generated by this hardware may lack the precision required for effective BFDO optimization. Therefore, a comprehensive understanding of BFDOs on RISC architectures is urgently needed.

Second, existing BFDOs primarily focus on specific languages such as C/C++, neglecting support for other emerging languages. Although C/C++ are widely used in cloud computing, data centers, and embedded systems [19,44,50], languages like Go and Rust are also gaining traction [43]. For instance, Go has been extensively adopted by vendors like Google and has facilitated the development of major open-source projects such as Docker and Kubernetes [38]. Similarly, Rust is being deployed in Firefox [34] and the Linux kernel [28]. Furthermore, binaries compiled from these languages exhibit distinct characteristics compared to those compiled from C/C++, due to differences in compilation techniques and runtime organizations. For example, Go's binaries contain extensive runtime information, including garbage collection mechanisms, while binaries from functional languages such as ML consist of numerous smaller basic blocks due to the use of continuations [4,26]—both of which are absent in C/C++ binaries. Consequently, evaluating the effectiveness of BFDO across diverse programming languages and binary structures is crucial.

Third, existing BFDO studies [32,36,37,40,48] have largely overlooked the impact of compiler optimizations on target binaries, making it challenging for developers to tune optimization options for optimal BFDO performance. Since compiler optimizations are applied in the early compilation stage without explicitly considering the post-link binaries, they may inadvertently reduce the effectiveness of state-of-the-art BFDOs. For example, the default optimization level (`O0`) may hinder sample-based profile generation by producing a large number

of smaller basic blocks, while linker relaxation optimizations [11] may interfere with function reordering by replacing call instructions with those of narrower ranges (e.g., `jalr` to `jal`). Therefore, a comprehensive understanding of how compiler optimizations affect BFDO effectiveness is essential.

In light of these limitations, in this paper, we present the first comprehensive study of state-of-the-art BFDOs to investigate their current capabilities, key grand challenges, and future research opportunities, providing insights into their continued development. To achieve this, we design BinInsight, a novel automated and scalable tool, to aid in our investigation, and employ both quantitative and qualitative methodologies on micro-benchmarks and real-world workloads. We then conduct a qualitative study to investigate BFDO failures and their root causes, current limitations and mitigation strategies, best practices, and future research directions.

Specifically, our investigation addresses the three aforementioned limitations of existing BFDOs. First, we study BFDOs on three representative architectures: x86-64, AArch64, and RISC-V. These architectures are chosen not only because they represent distinct application scenarios—including cloud, embedded, and mobile systems—but also due to their differing dynamic profiling capabilities, which are crucial for BFDO effectiveness. Second, we study binaries compiled from four programming languages: C/C++, Go, ML, and Rust. These languages are selected because they are not only widely adopted but also represent diverse programming paradigms, including imperative, object-oriented, and functional programming. Third, we experiment with various compile-time optimization options as well as link-time optimizations, such as linker relaxation. Moreover, thanks to the scalable design of BinInsight, our system is not restricted to those specific combinations and can be extended to explore potential combinations of architectures, languages, and optimization strategies.

Our study yields significant findings and insights. First, we identify three primary root causes of BFDO failures, originating from issues in disassembly and symbol relocation. Second, we reveal three key factors—compiler support code, biased profiles, and branch prediction failures—that substantially affect BFDO effectiveness. Based on our findings, we propose four best practices for using BFDOs, and point out three potential directions for future studies. By applying our best practices to RISC-V benchmarks, we achieve an average performance improvement of 4.57% over state-of-the-art while simultaneously preventing relocation failures.

To the best of our knowledge, this work represents the *first* step towards gaining a comprehensive understanding of state-of-the-art BFDOs. In summary, our work makes the following contributions:

- **Comprehensive study.** We systematically analyze state-of-the-art BFDOs using both quantitative and qualitative methodologies.
- **Findings and insights.** We identify three root causes for BFDO failures, and three factors affecting their effectiveness.
- **Best practices and suggestions.** We propose four best practices for improving BFDOs and outline three promising research directions.

– **Open source.** We open source our software prototype, benchmarks, test scripts, and evaluation results to promote open research at: https://doi.org/10.5281/zenodo.15058804.

2 Background

To be self-contained, in this section, we present the background knowledge for this work on Feedback-directed optimization (FDO) (§ 2.1), and profile generation (§ 2.2).

2.1 Feedback-Driven Optimization (FDO)

History. Feedback-driven optimization (FDO), also known as Profile-guided optimization (PGO), has been extensively studied with a long history [27] due to its potential to leverage runtime profiles for guiding compiler optimization. Recently, with advancements in hardware support and more accurate profile utilization techniques, FDO has garnered renewed research interest with notable variants and applications (*e.g.*, AutoFDO [9], SamplingPGO [35], and CSSPGO [22]).

Workflow. FDOs employ a two-stage build process as shown in Fig. 1. In the first stage, the source code is compiled (①) to binary, which is then executed to obtain profiles (②). Specifically, two primary approaches are utilized for profile acquisition: instrumentation-based and sample-based. The instrumentation-based approach instruments the target program to collect important runtime information such as execution frequency of basic blocks, functions, and loops, whereas the sample-based approach leverages specialized hardware features such as x86-64 LBR to periodically sample instructions to approximate execution frequencies. Guided by profiles collected in stage one (③), the second stage compiles the source to binary again (④), to further improve program performance (*e.g.*, more precise inline decisions for frequently called functions).

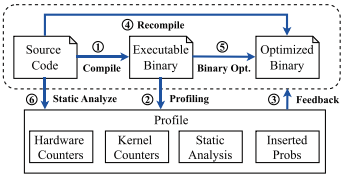


Fig. 1. Workflow of FDO and BFDO.

Traditional FDOs face two technical challenges. First, binary-only programs lack source code, making it difficult if not impossible to instrument sources. Second, compiler optimizations in phase (①) may dramatically change code layout, defeating the feasibility of *binary* profiles to *source* programs. Binary Feedback-directed Optimizations (BFDOs) [12,32,36] are introduced to address these challenges. As Fig. 1 presents, BFDOs directly instrument the executable binary to generate profiles which are then utilized to optimize the binary (⑤), without requiring program source code. Compared to FDOs, BFDOs (*e.g.*, HALO [39], Lighting BOLT [37], and Propeller [40]) often have a global view of the binaries as they process post-link programs, enabling more aggressive optimizations

including code layout, data compaction, hot/cold code splitting, alignment, and peephole optimizations.

Application. As a promising compiler optimization, FDO has been widely deployed in many applications. FDO is now supported by mainstream compilers [29], and has demonstrated effectiveness in optimizing diverse real-world programs (*e.g.*, Chrome [9] and Linux kernel [51]). Furthermore, FDO is also rapidly adopted by industry to optimize production applications in production environments. For example, FDO is used by Google [40] to optimize warehouse-scale applications, and by Meta to optimize compute-bound services [22].

2.2 Profile Generation

Profile generation is essential to perform effective FDO, as profiles directly guide optimization. Existing profile generation approaches can be classified into four categories: instrumentation-based, sample-based, hardware-based, and deep-learning-based.

Instrumentation-Based. Instrumentation-based profile generation inserts instructions into the code to track accurate execution counts for various program paths, and has been deployed in many tools (*e.g.*, Atom [41], Clang [30], and BOLT [36]). While the instrumentation-based approach generates precise data, it introduces substantial runtime overhead as well as considerable code size increases to instrumented binaries, hindering its use in production scenarios.

Sample-Based. Sample-based profile generation employs operating system mechanisms like clock interrupts to periodically capture program state snapshots as profiles. Traditional sample-based profiling is a statistical approximation which may produce inaccurate profiles. To mitigate this problem, precise sampling such as continuous profiling [2,20] is proposed to significantly enhance the precision.

Hardware-Based. Hardware-based profiling [1,10,14] collects profiles leveraging hardware performance counters, which critically rely on the underlying architectures. Fortunately, most modern architectures support performance counters (*e.g.*, Intel x86-64's Last Branch Records (LBR) [15], AArch64's Embedded Trace Macrocell (ETM) [5], and RISC-V's Hardware Performance Monitor (HPM), among others), which enable the generation of profiles with negligible runtime costs [17,42].

Deep-Learning-Based. Deep-learning-based approaches generate profiles by estimating program branches statically. Once deemed impractical in the 1990s [7], deep-learning-based approaches have shown promising potential in optimizing binaries [33].

3 Approach

This section presents our approach to conducting the evaluation. Performing such an evaluation of various datasets on multiple platforms is challenging, due to two key reasons: automation and scalability. First, the evaluation must be fully automated to efficiently evaluate a large number of projects; human intervention is only required for tasks that cannot be automated by machines, such as root cause analysis. Second, the evaluation must be scalable to study different combinations of architectures and programming languages, even potential ones not covered in this study.

To this end, we design and implement an automated software prototype, BinInsight, to aid in our evaluation. We first present an overview of BinInsight (§ 3.1), and then each component, including compilation (§ 3.2), optimization (§ 3.3), metric measurement (§ 3.4), and root cause analysis (§ 3.5).

3.1 Overview

BinInsight is designed with the key principle of modularity and extensibility, making it straightforward to accommodate different needs, such as adding new test projects, new programming languages, and architectures.

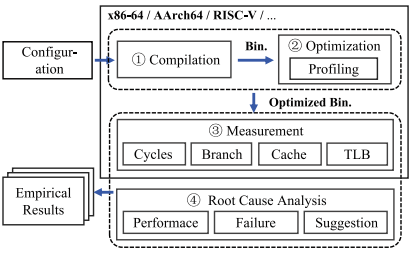


Fig. 2. An overview of BinInsight's workflow.

Figure 2 provides an overview of BinInsight, which comprises four key components: compilation, optimization, measurement, and root cause analysis. First, the compilation component (①) takes the test cases as input, compiles and links them with relocation information according to the user-specified configuration, generating an unoptimized binary. Second, the optimization component (②) receives the unoptimized binary and samples the binary during execution to produce profiles, which guide the optimizer to optimize the target binary and generate an optimized version. Third, the measurement component (③) measures and records metrics for both the unoptimized and optimized binaries. Finally, the root cause analysis module (④) enables root cause studies for failures, upon which best practices and suggestions are composed.

3.2 Compilation

The compilation module compiles and links source code into binaries with relocation information according to a user-specified configuration. A critical aspect of this process is selecting toolchains that support the link options required by BFDOs [36]. Specifically, the compiler toolchain must support *emit relocations option* (*i.e.*, `--emit-relocs` or `-q` in most linkers). However, we have observed that while mainstream linkers (*e.g.*, BFD, Gold, and lld) generally support the

relocations option, some do not. For instance, `gc`, the official Go compiler, lacks support for this option whereas other Go compilers, such as gccgo [18] and gollvm [21], provide the necessary functionality. For this reason, we use gccgo to evaluate Go programs (§ 4). The key takeaway is that developers must carefuly select a compiler toolchain that supports relocation, otherwise BFDOs may fail to take effect without any explicit warning.

3.3 Optimization

The optimization component optimizes the target binaries and produces optimized ones by leveraging profiles in two steps: 1) Profiling: collect profiles during execution using performance tools like `perf` [16] to sample the CPU-cycles event and taken branch stack. To minimize potential noise and ensure more accurate profiles, essential measures should be taken, including terminating irrelevant processes, increasing process priority, and setting CPU affinity. 2) Optimization: the binary is optimized using the profiles and options specified by the user.

During our experiments, we observed that the taken branch stack is arguably the most critical information for BFDOs, as it records a series of consecutive taken branches, thus providing accurate counts for critical edges to improve the resilience against imprecise sampling [36] (see § 4). Therefore, it is essential to enable this hardware feature if it is supported by the underlying hardware.

3.4 Quantitative Measurement

The measurement component calculates metrics for both the original and optimized binaries. Key metrics essential for measuring BFDOs include, but are not limited to, the following:

CPU Cycles. This metric represents the number of clock cycles spent executing the program and is the primary target for reduction through binary optimization.

Branches and Branch Misses. This metric reflects the effect of branch prediction after basic block reordering, which can be reduced by effective block reordering.

Cache and Cache Misses. This metric encompasses both instruction cache (I-cache) and data cache (D-cache). BFDOs aim to improve instruction locality by reducing I-cache occupation, grouping hotspot code, and folding identical code. However, a trade-off between I-cache and D-cache optimization is often necessary, as optimizing both simultaneously can be challenging.

TLB and TLB Misses. This metric indicates instruction and data locality at the page level, which is larger than cache size, and thus is useful for analyzing large programs.

These metrics can be measured by utilizing performance tools like `perf`. Meanwhile, the measurement should be repeated multiple times to minimize noise, before calculating the average speedups according to Eq. (1).

$$speedup = \frac{\sum_{repeat} \frac{unopt-opt}{unopt}}{repeat} \quad (1)$$

3.5 Root Cause Analysis

The root cause analysis component identifies the underlying causes of optimization failures, proposes suggestions, and provides fixes. We conduct manual analysis when automated analysis is impossible, following prior studies on optimization failures and performance degradation [3,13,24,25,45,49]. Furthermore, we identify impactful factors for performance improvements, and provide best practices and suggestions to enhance the whole ecosystem.

4 Evaluation

In conducting the evaluation, we aim to answer the following research questions:

RQ1: Programming Language Diversity. Is BFDO effective on binaries compiled from different programming languages? What are the performance discrepancies for these binaries from different languages, if any?

RQ2: Architecture Independence. Is BFDO dependent on different architectures, as these architectures have different hardware performance facilities?

RQ3: Optimization Neutrality. How do compiler and linker optimizations affect BFDO? What are the optimization traps and pitfalls?

4.1 Experimental Setup

We conduct experiments on three current representative architectures: x86-64, AArch64, and RISC-V, as shown in Table 1. For Intel CPUs, all experiments are performed on Performance-cores (P-cores). Additionally, for RISC-V CPUs, we use Debian instead of Ubuntu because our RISC-V machine only provides the Debian images, which does not introduce any bias because our experiments do not depend on specific OS features. We use BOLT to optimize their I-cache locality with the following options:

```
-reorder-blocks=ext-tsp -dyno-stats -split-functions
-split-all-cold -split-eh -reorder-functions=hfsort
```

We collect profiles using `perf record` with options `-e cycles:u -j any,u`. We then use `perf2bolt` tool to convert the profiles to BOLT format, with a `-nl` option if the corresponding architecture does not support branch stack sampling.

4.2 Datasets

We construct two datasets to conduct the evaluation: 1) micro-benchmarks; and 2) real-world workloads, which are distributed in our open source.

Micro-benchmarks. We create a set of micro-benchmarks as a unified standard for four languages: C++, Go, Rust, and Standard ML (SML), which are compiled by clang, rustc, gccgo, and MLton, respectively.

Real-World Workloads. We select diverse real-world open-source projects from GitHub based on the following selection criteria: 1) compiler support, and 2) performance priority. First, we select projects that satisfy the compiler requirements outlined in § 3.2, otherwise BFDO cannot be applied. Second, we select projects prioritizing performance, for which BFDOs are of more practical implications.

Table 1. Experimental setup for our evaluation.

	Hardware	OS	Compiler
x86-64	Intel i7-12700K 128GB RAM	Ubuntu 22.04	clang 18.1.1, rustc 1.77.1 gccgo 13.2.0, MLton 20231123
AArch64	BCM2712 CPU (4x Cortex-A76) 8GB RAM	Ubuntu 22.04	clang 18.1.2, rustc 1.78.0 gccgo 13.2.0, MLton 20231123
RISC-V	TH1520 SOC (4x Xuantie C910) 16GB RAM	Debian 12	clang 18.0.0, rustc 1.70.0 gccgo 13.2.0

Table 2. Performance for micro-benchmarks (% speedup, per test case).

#	Case	x86-64				AArch64				RISC-V			
		C/C++	Go	Rust	SML	C/C++	Go	Rust	SML	C/C++	Go	Rust	SML
1	K-Means	0.45	-0.18	-0.43	0.96	-0.62	10.55	2.85	2.07	-11.3	-5.66	×	-1.47
2	Prim	0.94	-0.72	15.61	2.42	-2.51	1.54	-6.87	1.56	-8.39	-1.3	×	1.55
3	KMP	13.36	1.55	1.65	1.36	15.23	11.15	0.25	0.06	-33.67	4.57	×	-5.89
4	Matrix chain order	-9.85	-0.22	35.89	6.56	0.81	-9.21	-8.15	0.28	-18.24	3.03	×	-2.07
5	Max submatrix sum	3.73	1.88	20.2	0.99	1.67	-0.04	-5.84	-0.44	15.1	-5.90	×	1.27
6	Merge sort	3.68	0.71	25.08	0.72	-0.69	0.45	-4.36	1.09	-15.13	-1.01	×	2.18
7	N queen	2.51	5.76	-0.92	17.39	8.59	6.63	8.32	8.44	13.07	-29.36	×	14.86
8	Qucik sort	-0.21	1.26	18.35	0.05	1.6	-1.52	-1.72	1.24	13.84	0.21	×	0.17
9	sha256	-3.34	13.16	3.43	2.07	0.11	4.44	0.88	-2.32	-2.55	-28.05	×	2.39
10	TSP	6.96	10.63	33.73	1.5	-6.54	2.29	-9.40	0.27	-9.66	-2.49	×	-0.5
	Average	1.82	3.38	15.26	3.4	1.76	2.63	-2.4	1.23	-5.69	-6.60	—	1.25
	Variance	33.68	21.35	173.04	27.37	33.05	32.52	30.94	7.93	225.95	131.96	—	29.01

[1]The symbol × means relocation error.

4.3 RQ1: Programming Language Diversity

To answer **RQ1** by demonstrating the effectiveness and efficiency of BFDO across multiple languages, we first conduct experiments on micro-benchmarks, as shown in Table 2. For all three architectures, C/C++ demonstrates speedups of 1.82%, 1.76%, and -5.69% on average, respectively, Go shows speedups of 3.38%, 2.63%, and -6.60% on average, respectively, whereas SML shows speedups of 3.4%, 1.23%, and 1.25% on average, respectively. While Rust exhibits the best optimization effects of 15.26% on x86-64, it performs poorly on RISC-V. In contrast, SML exhibits the worst optimization results on x86-64 and RISC-V but also the lowest variance.

Next, we evaluate real-world workloads, as presented in Table 3. Across all three architectures, the performance improvement for C/C++ achieves 4.17%, and 4.07% on average, while Go achieves 4.74% and 3.35%, followed by Rust's 4.36%. These results demonstrate that BFDO is effective in uniformly improving the performance of real-world workloads for all three languages.

Table 3. Performance for real-world workloads (% speedup, per test case).

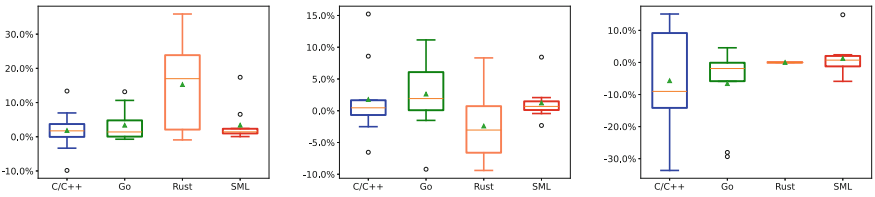
Language	Workload	x86-64	AArch64	RISC-V
C/C++	John	0.08	-0.46	3.87
	QuickJS	1.36	-0.28	⊗
	Redis	11.35	3.56	⊗
	Cppcheck	3.88	13.46	×
	Average	4.17	4.07	—
Go	gjson	5.58	9.28	-1.61
	goquery	1.24	1.08	-3.71
	websocket	1.69	1.46	18.41
	web framework	7.97	5.43	⊗
	grpc-go	7.24	-0.57	⊗
	Average	4.74	3.35	—
Rust	Rocket	5.09	2.74	×
	RustPython	2.47	2.41	⊘
	wasmtime	2.84	⊗	⊘
	hyper	2.01	⊗	×
	tantivy	9.42	0.3	×
	Average	4.36	—	—

[1] ⊘ compilation error, ⊗ disassembly error, × relocation error.

Root Cause Analysis. We then investigate the root cause leading to performance discrepancies and identify the following factors. First, Rust and Go contain *more cold code* because they integrate more complex runtimes than C/C++,

including stack overflow check, array bounds check, Drop trait code, and support libraries. Such mechanisms affect instruction locality hence benefit BFDOs. We identify that in our micro-benchmark, the cold code of C/C++ programs accounts for 57% of the code, while Go and Rust account for 84% and 96%, respectively, which results in section sizes 23x and 122x larger than C/C++. These metrics indicate that Go and Rust have greater optimization potential.

The second root cause for performance discrepancies is the *machine code style* generated from different languages. We reveal that BFDO disassembles binaries at the function granularity determined by the granularity of relocation information, which does not fit well with functional languages that often leverage continuation-passing style (CPS) transformation leading to many small functions (*i.e.*, chunks). Furthermore, we observe that the MLton compiler invokes the generated functions via indirect function pointer dispatches rather than direct calls, which complicates both the function disassembly and the final control-flow graphs. To this end, existing BFDOs still face grand challenges for functional languages.



(a) x86-64 (b) AArch64 (c) RISC-V

Fig. 3. Box plots of micro-benchmark performance.

```
000000000073d738 <.L1^B1>:
  73d738:   01a0000b   .insn 4, 0x01a0000b
  73d73c:   0440006f   j 73d780 <.Llrsc_0>
  73d740:   00000013   nop
000000000073d780 <.Llrsc_0>:
  73d780:   0246000b   .insn 4, 0x0246000b
  73d784:   1606272f   lr.w.aqrl a4,(a2)
  73d788:   00000013   nop
```

Fig. 4. Custom instructions encoded by `.insn` directive on RISC-V lead to optimization failures.

4.4 RQ2: Architecture Independence

To answer **RQ2**, we investigate BFDO's dependency on diverse architectures, as presented in Table 2. We observe that x86-64 improves the performance of all four languages, while AArch64 improves three among them except for Rust. RISC-V is the worst to manifest performance degradation for all languages, as

depicted in Fig. 3. We then conduct experiments on real-world workloads, as detailed in Table 3. The results align closely with those from micro-benchmarks, confirming that x86-64 delivers the best performance, while RISC-V performs the worst. Furthermore, as Table 3 shows, AArch64 and RISC-V encounter more compilation, disassembly, and relocation errors, leading to optimization failures.

Root Cause Analysis. We then investigate the root causes of optimization failures and performance degradation, and identify four key root causes: 1) custom instructions, 2) relocation exception, 3) biased profiling, and 4) failed prediction. First, custom instructions in RISC-V lead to optimization failures. Specifically, as an open-source architecture, RISC-V allows chip vendors to incorporate custom instructions. While such customizations bring flexibility, they pose challenges to disassembly, an essential step for BFDO. Figure 4 shows a failure example we observed when disassembling from `grpc-go`. The values `0x01a0000b` and `0x0246000b` in the `.insn` directive encode the custom instructions `SYNC.I` and `DCACHE.CVAL1` respectively on Xuantie C910 processor [8], which defeat the disassembler, leading to optimization failures.

```
foo:
  # expand by la a0, symbol
  auipc a0, %pcrel_hi(symbol)  # R_PCREL_HI20 (symbol)
  addi  a0, a0, %pcrel_lo(foo) # R_PCREL_LO12_I (foo)
bar:
  # expand by la.tls.gd a0, symbol
  auipc a0,0                   # R_TLS_GD_HI20 (symbol)
  addi  a0,a0,0                # R_PCREL_LO12_I (bar)
```

Fig. 5. PC-relative relocations lead to optimization failures.

The second root cause leading to optimization failures is *relocation exceptions*. Specifically, we identify that optimization failures for Rust on RISC-V are due to the exceptions that the optimizer failed to find the corresponding `%pcrel_hi` modifier. This exception stems from the relocation type *PC-relative TLS GD GOT reference* used in RISC-V multithreaded programs, which is used to load the address of a symbol into a register (via the pseudo directive `la` or `la.tls.gd` for thread-local storage (TLS)). On most architectures, such load operations typically use PC-relative addressing, which requires two instructions on RISC-V, as shown in Fig. 5. The `%pcrel_hi` modifier represents the high 20 bits of the relative address between the program counter (PC) and the target symbol, and the `%pcrel_lo` modifier represents the low 12 bits. In this example, to load the address of `symbol` into register `a0`, `auipc` instruction uses the high 20 bits of the relative address between PC and `symbol` as source operand HI, storing $PC + (HI << 12)$ into register `a0`. Then `addi` instruction takes register `a0` and the low 12 bits of the relative address between corresponding `auipc` instruction and `symbol` as source operands, adding them and storing the result in register `a0`. The instructions in `foo` and `bar`, though written differently, perform

similar functions. They load normal symbols and TLS symbols, respectively, and produce two distinct relocation types: `R_PCREL_HI20` and `R_TLS_GD_HI20`. We identify that BFDO fails to resolve the `R_TLS_GD_HI20` relocation type, leading to optimization failures. Furthermore, we reveal that such relocation types are common in Rust binaries even for non-multithreaded programs but are rare in C/C++ and Go binaries. Thus we believe existing BFDOs have spaces to improve the relocation type resolution in Rust binaries.

The third root cause leading to performance degradation is *biased profiles*. Specifically, while sample-based profiling is effective for generating profiles without any special aid from instrumentation, it may not reflect the real execution frequency of the program's basic blocks. This inaccuracy stems from factors such as sampling frequency, basic block size, and sampling environment. We observe that among all these factors, the basic block size is the most critical yet challenging factor to tune by adjusting sampling options. Furthermore, sample-based profiling may introduce bias because it is usually performed at OS interrupt granularity, in which larger basic blocks with more instructions have a larger sampling probability while smaller basic blocks are rarely sampled. Although increasing the sampling frequency can improve the sampling probability of small basic blocks, it simultaneously increases the sampling probability of large basic blocks, making it difficult to achieve balanced profiling.

We present in Fig. 6 an example leading to optimization failures due to biased profiles. Basic block `B1`, `B3`, and `B4` have the same execution count. However, basic block `B3` is rarely sampled due to its small size (one instruction). Consequently, BFDOs may classify `B1` and `B4` as hot basic blocks while `B3` are classified as cold block based on the sampling results. Thus the optimization leverages the hot/cold split algorithm to split them into distinct sections that are far apart in address space, causing each loop to jump twice between the hot and cold sections. Such extra jumps introduced by the optimization lead to severe performance degradation of I-cache locality, and in the worst case lead to a 1.5x increase in execution time.

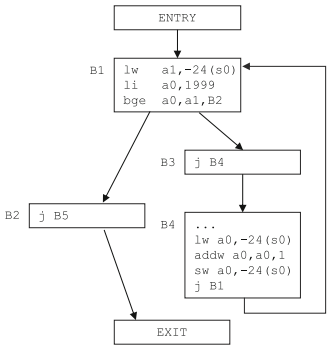
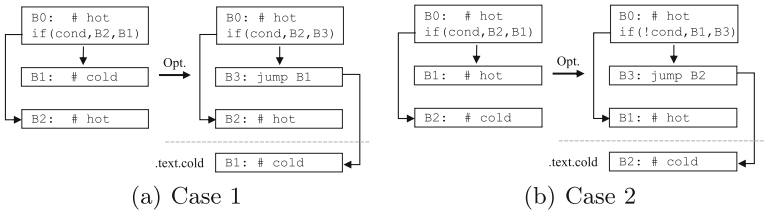


Fig. 6. Biased profiles lead to optimization failures.

Failed Branch Prediction is the fourth and last root cause we have identified. During BFDO, the optimizer reorders basic blocks by adjusting the target of branch instructions, placing hot basic blocks in fallthrough areas and cold basic blocks in cold sections. However, the reordered basic blocks may be too far away and beyond the addressing range (typically ±4KB) of the conditional jump instruction that reaches them, as shown in Table 4. To address this issue, the optimizer can insert an unconditional jump instruction following the conditional jump, but



(a) Case 1 (b) Case 2

Fig. 7. Failed branch predictions lead to optimization failures.

this approach may incur significant branch prediction failures. We present two failed branch predictions leading to optimization failures in Fig. 7. In the first case, the branch prediction will take cold block `B1` as the predicted branch, which is not an efficient layout. After optimization, the cold block `B1` is placed into the cold section, and an unconditional jump instruction to `B1` is inserted in the original position of `B1`. However, while such a code layout improves code locality, it does not improve branch prediction accuracy. In the second case, the situation is even worse. To jump to cold basic block `B2`, the optimizer reverses the condition of the conditional jump instruction and inserts an unconditional jump before basic block `B1`, incurring a branch prediction failure for each jump to `B1`. We observe that when the improvement from code locality cannot compensate for the overhead of branch prediction failures, the overall performance will degrade. Specifically, we identify that branch prediction failures in if statements incur a performance degradation of about 6%.

Table 4. Max jump range on three architectures.

Type	x86-64	AArch64	RISC-V
unconditional jump	±2GB	±128MB	±1MB
conditional jump	±2GB	±1MB	±4KB

4.5 Optimization Neutrality

To answer **RQ3** by investigating optimization neutrality of BFDO, we conduct experiments by applying compiler and linker optimizations when generating the binaries.

We first experiment with different compiler optimization options, and reveal that lower optimization levels (e.g., `O0`) will degrade performance because they tend to generate more complex control-flow graphs with numerous tiny basic blocks. For instance, we observe a 4.0% performance improvement when running C/C++ micro-benchmarks on RISC-V by simply switching the compiler optimization option from `O0` to `O2`. Therefore, we argue that full compiler optimizations should be utilized to maximize the benefits of BFDO.

We then conduct experiments to test the effects of link-time optimizations on BFDOs. We reveal that BFDO often assumes symbol locations remain unchanged after linking, but binary optimization breaks that assumption and thus may lead to optimization failures. Specifically, we observe that the Cppcheck workload encounters the relocation exception on RISC-V where `R_RISCV_JAL` relocation exceeds its address range. Upon investigating the source code, we identify that this exception is triggered by the linker relaxation optimization. The linker relaxation optimization is a link-time optimization aimed at reducing code size. Unfortunately, the binary optimizations in BFDO, including function reordering, basic block reordering, and the hot/cold code split, will change the symbol locations, triggering exceptions. To investigate mitigations, we recompile the Cppcheck workload with the `--no-relax` option to disable linker relaxation optimization, and observe a performance improvement of 21.02%.

5 Best Practices and Future Directions

In this section, we present best practices for using BFDOs and outline directions for future studies.

Better Support of Relocation Information. We argue that a key step to utilize BFDOs is selecting compilers that support the export of relocation information. Specifically, since BFDOs require precise disassembly and relocation information, compilers should export symbol table and precise relocation information to binary files. While many compilers support symbol table well, some do not support relocation information (*e.g.*, the `gc` compiler for Go). Besides, optimizer support for certain relocation types, such as `R_RISCV_TLS_GD_HI20`, remains limited. Therefore, using unsupported relocation types should be avoided. This is a trade-off between performance and program implementation.

Do Not Use BFDOs on Unoptimized Binaries. We argue that BFDOs supplement existing compilation optimizations but do not replace them. We observe from our evaluation that unoptimized binaries (e.g., generated from the compiler via `O0`) often contain numerous tiny basic blocks and complex jump structures, which are more susceptible to biased profiles, especially on RISC architectures like RISC-V. Applying BFDOs on such binaries often leads to performance degradation rather than improvements.

Avoid Using Linker Relaxation. We argue that although linker relaxation can reduce binary size and improve performance, it has negative impacts on BFDOs, especially for large binaries. To this end, we suggest that linker relaxation should be avoided during BFDOs. In the long term, we believe that the conflict between linker relaxation and BFDOs should be addressed by introducing more sophisticated optimization heuristics that take into account the effects of BFDOs on code layouts. We identify that this idea is integrated into the recent version of BOLT, but regrettably, it does not work correctly, at least as of this study.

Improve Sampling Accuracy. We argue that sampling accuracy is essential for BFDOs and thus should be carefully tuned. We reveal from our experiments that it is often effective to adjust the sampling frequency to improve accuracy (*e.g.*, the `-F` option in `perf`), when the performance improvement is suboptimal. Furthermore, due to the imprecision of sample-based profiling for tiny basic blocks, we argue that sample-based profiling can be complemented by instrumentation-based profiling to determine more precise profiles. We believe this best practice is especially valuable for hardware that lacks powerful performance measurement units.

Better Disassembly and Hardware Support. We argue that effective BFDOs comprise two key sub-tasks: binary disassembly and optimization. Unfortunately, both sub-tasks are well-known NP-complete problems and thus effective heuristics must be leveraged. To this end, we believe better disassembly support will benefit BFDOs, especially for RISC binaries in which the mixture of code and data often defeats disassembly. Furthermore, we observe that current architectures, especially AArch64 and RISC-V, still have limited hardware profiling support, which undermines BFDOs. Addressing this limitation requires attention and effort from hardware vendors.

6 Discussion and Threats to Validity

Like any empirical study, there are threats to the validity of our results. We discuss the most significant ones, along with possible mitigations.

External Validity. Threats to external validity concern the generalizability of findings across experimental settings, primarily stemming from three factors: the selected programming languages, architectures, and benchmarks. To mitigate these threats, we selected four programming languages spanning distinct application domains and encompassing varied programming paradigms. Additionally, we evaluated three representative architectures. Finally, for benchmarks, we used both micro-benchmarks and real-world workloads applied across various fields.

Internal Validity. As for threats to internal validity, a critical threat stems from random variance in execution times of benchmarks, caused by subtle fluctuations in the host OS's state that may manifest as significant deviations. To mitigate this threat, we conducted multiple trials for each benchmark, using the mean execution duration as the evaluation metric while calculating temporal variance to quantify runtime volatility. For micro-benchmarks whose execution speed exceeds measurement resolution thresholds, we incorporated repeated invocations to amplify temporal observables, thereby compensating for sampling inaccuracies.

Construct Validity. We evaluated the effectiveness of BFDOs through comparative analysis of runtime data collected from benchmarks before and after optimization. Although many benchmarks on RISC-V platforms were non-executable due to compilation errors or optimization failures, these instances serve as critical validation indicators for assessing the cross-platform validity of BFDOs.

7 Related Work

Binary Optimizations. There are numerous studies on binary optimizations. Panchenko et al. [36] presented a static binary optimizer BOLT for data-center applications, demonstrating that profile data can be used more precisely at the binary level. Williams-King and Yang et al. [48] implemented CodeMason, which performs static binary rewriting based on the binary rewriting platform called Egalito [47]. Panchenko et al. [37] demonstrated Lightning BOLT based on the work of BOLT. They addressed the CPU and memory overhead of BOLT by introducing parallel processing and selective optimizations. Zhou and Jones et al. [52] presented a framework called Janus to address the challenge of automatic binary parallelization. Savage and Jones et al. [39] proposed a post-link profile-guided optimization tool called HALO to improve the layout of heap data to reduce cache misses automatically.

However, existing work mainly focuses on the x86 CPU architecture and binaries compiled from C/C++. Our work studies the effectiveness and performance improvement of other platforms and programming languages, aiming to fill this gap.

Profiling Techniques. Profiling techniques are a key component of binary optimization. Instrumentation-based profiling is a common method for collecting precise profile data, but it often incurs non-negligible runtime overhead. To mitigate this issue, Ball et al. [6] optimized probe placement using minimal spanning tree. Additionally, Cho et al. [23] proposed a novel instrumentation framework that reported an average runtime slowdown of 3% to 6%. Since the overhead of instrumentation-based profiling remains undesirable for production deployment, sample-based profiling is widely adopted by many tools. He et al. [22] proposed a pseudo-instrumentation technique as a supplement to sample-based profiling to improve profile quality without incurring the overhead of traditional instrumentation. Moreira et al. [33] enhanced static profiling technique called Evidence-Based Static Prediction (ESP), originally proposed by Calder et al. [7], which enables the generated profiles to be used for binary optimization.

However, although leveraging profiling techniques as foundational infrastructure, our contribution lies in systematically evaluating BFDOs' effectiveness and efficiency. This focus dictates our selection of the most compatible and relatively accurate sample-based profiling in our study.

8 Conclusion

In this work, we present the first study on binary feedback-directed optimization, aiming to evaluate its current capabilities, remaining limitations, and future research directions. By designing and implementing a software prototype, we investigate the effectiveness and efficiency of binary optimization across three representative architectures and four programming languages. We reveal three root causes of optimization failures and performance degradation, along with

three best practices for improved optimization outcomes. Additionally, we outline three promising directions for future research. We believe that our study provides valuable insights that can guide future research efforts in leveraging modern hardware performance measurement capabilities through advanced compiler technologies.

References

1. Ammons, G., Ball, T., Larus, J.R.: Exploiting hardware performance counters with flow and context sensitive profiling. ACM SIGPLAN Notices **32**(5), 85–96 (1997)
2. Anderson, J.M., et al.: Continuous profiling: where have all the cycles gone? ACM Trans. Comput. Syst. **15**(4), 357–390 (1997)
3. Andriesse, D., Chen, X., van der Veen, V., Slowinska, A., Bos, H.: An In-Depth analysis of disassembly on Full-Scale x86/x64 binaries. In: 25th USENIX Security Symposium (USENIX Security 16), pp. 583–600 (2016)
4. Appel, A.W.: Compiling with Continuations. Cambridge University Press, USA (2007)
5. ARM: Embedded trace macrocell architecture specification (2024). https://developer.arm.com/documentation/ihi0014/q/Introduction/About-Embedded-Trace-Macrocells
6. Ball, T., Larus, J.R.: Optimally profiling and tracing programs. ACM Trans. Program. Lang. Syst. **16**(4), 1319–1360 (1994)
7. Calder, B., Grunwald, D., Jones, M., Lindsay, D., Martin, J., Mozer, M., Zorn, B.: Evidence-based static branch prediction using machine learning. ACM Trans. Program. Lang. Syst. **19**(1), 188–222 (1997)
8. Chen, C., et al.: Xuantie-910: a commercial multi-core 12-stage pipeline out-of-order 64-bit high performance risc-v processor with vector extension : Industrial product. In: 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), pp. 52–64. IEEE, Valencia, Spain (2020)
9. Chen, D., Li, D.X., Moseley, T.: Autofdo: Automatic feedback-directed optimization for warehouse-scale applications. In: Proceedings of the 2016 International Symposium on Code Generation and Optimization, pp. 12–23. ACM, Barcelona Spain (2016)
10. Chen, D., Vachharajani, N., Hundt, R., Li, X., Eranian, S., Chen, W., Zheng, W.: Taming hardware event samples for precise and versatile feedback directed optimizations. IEEE Trans. Comput. **62**(2), 376–389 (2013)
11. Chen, S., Wang, H.: Compiler support for linker relaxation in risc-v (2019). https://riscv.org/wp-content/uploads/2019/03/11.15-Shiva-Chen-Compiler-Support-For-Linker-Relaxation-in-RISC-V-2019-03-13.pdf
12. Cohn, R., Goodwin, D., Lowney, P.G., Rubin, N.: Spike: an optimizer for alpha/nt executables. In: Proceedings of the USENIX Windows NT Workshop on The USENIX Windows NT Workshop 1997, p. 3. NT'97, USENIX Association, USA (1997)
13. Committee, P.I.: Tech-privileged@lists.riscv.org | a proposal to enhance risc-v hpm (hardware performance monitor) (2024). https://lists.riscv.org/g/tech-privileged/topic/a_proposal_to_enhance_risc_v/75675990

14. Conte, T.M., Patel, B.A., Menezes, K.N., Cox, J.S.: Hardware-based profiling: an effective technique for profile-driven optimization. Int. J. Parallel Prog. **24**(2), 187–206 (1996)
15. Corporation, I.: Intel® 64 and IA-32 architectures software developer's manual (325384–083US) (2024)
16. De Melo, A.C.: The new linux'perf'tools. In: Slides from Linux Kongress. vol. 18, pp. 1–42 (2010)
17. Domingos, J.M., Tomas, P., Sousa, L.: Supporting risc-v performance counters through performance analysis tools for linux (perf) (2021)
18. Free Software Foundation, I.: Top (the gnu go compiler) (2024). https://gcc.gnu.org/onlinedocs/gccgo/
19. Fried, J., et al.: Making kernel bypass practical for the cloud with junction. In: Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation. NSDI'24, USENIX Association, USA (2024)
20. Gloy, N., Wang, Z., Zhang, C., Chen, B., Smith, M.: Profile-based optimization with statistical profiles (1997)
21. Google: Gollvm - git at google (2024). https://go.googlesource.com/gollvm/
22. He, W., Yu, H., Wang, L., Oh, T.: Revamping sampling-based pgo with context-sensitivity and pseudo-instrumentation. In: 2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 322–333 (Mar 2024)
23. Hyoun Kyu Cho, Moseley, T., Hank, R., Bruening, D., Mahlke, S.: Instant profiling: Instrumentation sampling for profiling datacenter applications. In: Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 1–10. IEEE, Shenzhen (Feb 2013)
24. Ibáñez, R.F.: Should we constrain the target label of a %pcrel_lo to be in the same section? · issue #90 · riscv-non-isa/riscv-elf-psabi-doc (2024). https://github.com/riscv-non-isa/riscv-elf-psabi-doc/issues/90
25. Jiang, M., Zhou, Y., Luo, X., Wang, R., Liu, Y., Ren, K.: An empirical study on arm disassembly tools. In: Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis, pp. 401–414. ISSTA 2020, Association for Computing Machinery, New York, NY, USA (2020)
26. Kennedy, A.: Compiling with continuations, continued. SIGPLAN Not. **42**(9), 177–190 (2007)
27. Knuth, D.E., Stevenson, F.R.: Optimal measurement points for program frequency counts. BIT **13**(3), 313–322 (1973)
28. for Linux Team, R.: Rust for linux (2024). https://rust-for-linux.com
29. LLVM: Llvm support for bolt (2024). https://github.com/llvm/llvm-project/tree/main/bolt
30. LLVM: Optimizing clang with llvm bolt (2024). https://github.com/llvm/llvm-project/blob/main/bolt/docs/OptimizingClang.md
31. LLVM: [samplefdo] flow sensitive sample fdo (fsafdo) profile loader (2024). https://reviews.llvm.org/D107878
32. Luk, C.K., Muth, R., Patil, H., Cohn, R., Lowney, G.: Ispike: a post-link optimizer for the intel/spl reg/ itanium/spl reg/ architecture. In: International Symposium on Code Generation and Optimization, 2004. CGO 2004, pp. 15–26 (2004)
33. Moreira, A.A., Ottoni, G., Quintão Pereira, F.M.: Vespa: Static profiling for binary optimization. Proceedings of the ACM on Programming Languages 5(OOPSLA), 144:1–144:28 (2021)
34. Mozilla: Language details of the firefox repo (2024). https://4e6.github.io/firefox-lang-stats/

35. Novillo, D.: Samplepgo - the power of profile guided optimizations without the usability burden. In: 2014 LLVM Compiler Infrastructure in HPC, pp. 22–28. IEEE, LA, USA (2014)
36. Panchenko, M., Auler, R., Nell, B., Ottoni, G.: Bolt: A practical binary optimizer for data centers and beyond. In: 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 2–14. IEEE, Washington, DC, USA (2019)
37. Panchenko, M., Auler, R., Sakka, L., Ottoni, G.: Lightning bolt: powerful, fast, and scalable binary optimization. In: Proceedings of the 30th ACM SIGPLAN International Conference on Compiler Construction, pp. 119–130. ACM, Virtual Republic of Korea (2021)
38. Pike, R.: New case studies about google's use of go (2024). https://opensource.googleblog.com/2020/08/new-case-studies-about-googles-use-of-go.html
39. Savage, J., Jones, T.M.: Halo: Post-link heap-layout optimisation. In: Proceedings of the 18th ACM/IEEE International Symposium on Code Generation and Optimization, pp. 94–106. CGO 2020, Association for Computing Machinery, New York, NY, USA (2020)
40. Shen, H., Pszeniczny, K., Lavaee, R., Kumar, S., Tallam, S., Li, X.D.: Propeller: a profile guided, relinking optimizer for warehouse-scale applications. In: Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2, pp. 617–631. ACM, Vancouver BC Canada (2023)
41. Srivastava, A., Eustace, A.: Atom: a system for building customized program analysis tools. ACM SIGPLAN Notices **29**(6), 196–205 (1994)
42. Suzaki, K., Nakajima, K., Oi, T., Tsukamoto, A.: Ts-perf: general performance measurement of trusted execution environment and rich execution environment on intel sgx, arm trustzone, and risc-v keystone. IEEE Access **9**, 133520–133530 (2021)
43. TIOBE: Tiobe index (2024). https://www.tiobe.com/tiobe-index/
44. Wang, R., et al.: μslope: high compression and fast search on semi-structured logs. In: Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation. OSDI'24, USENIX Association, USA (2024)
45. Wang, S., Wang, P., Wu, D.: Reassembleable disassembling. In: 24th USENIX Security Symposium (USENIX Security 15), pp. 627–642 (2015)
46. Waterman, A., Asanovic, K., Hauser, J. Inc, S.: The risc-v instruction set manual volume ii: Privileged architecture document version 20211203. CS Division, EECS Department, University of California, Berkeley (2021)
47. Williams-King, D., et al.: Egalito: layout-agnostic binary recompilation. In: Proceedings of the Twenty-Fifth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 133–147. ASPLOS '20, Association for Computing Machinery, New York, NY, USA (2020)
48. Williams-King, D., Yang, J.: Codemason: binary-level profile-guided optimization. In: Proceedings of the 3rd ACM Workshop on Forming an Ecosystem Around Software Transformation, pp. 47–53. FEAST'19, Association for Computing Machinery, New York, NY, USA (2019)
49. Yan, L., Thompson, D., Support, L.: Using perf on arm platforms (2024). https://static.linaro.org/connect/yvr18/presentations/yvr18-416.pdf
50. Yu, L., Zhang, X., Zhang, H., Sonchack, J., Ports, D., Liu, V.: Beaver: practical partial snapshots for distributed cloud services. In: Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation. OSDI'24, USENIX Association, USA (2024)

51. Yuan, P., Guo, Y., Chen, X.: Experiences in profile-guided operating system kernel optimization. In: Proceedings of 5th Asia-Pacific Workshop on Systems, pp. 1–6. ACM, Beijing China (2014)
52. Zhou, R., Jones, T.M.: Janus: statically-driven and profile-guided automatic dynamic binary parallelisation. In: 2019 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), pp. 15–25 (2019)

Towards Efficient SpMV: A Multi-Aware Optimization Framework for Heterogeneous Architecture

Yang Liu[1,2], Zexin Wang[1,2], Xinyin Zhang[1,2], Zenghui Ren[1,2], and Yonghua Zhao[1(✉)]

[1] Computer Network Information Center, Chinese Academy of Science, Beijing, China
yhzhao@sccas.cn

[2] University of Chinese Academy of Sciences, Beijing, China

Abstract. Sparse Matrix-Vector Multiplication (SpMV) is a fundamental computational kernel widely used in scientific computing. Due to its irregular memory access patterns and the diverse sparsity structures present in real-world matrices, optimizing SpMV is challenging. This paper proposes an adaptive SpMV optimization framework and SpMV implementation method, which integrates deep insights into matrix characteristics, algorithmic strategies, and hardware architectures. By combining these three complementary aspects with a hierarchical decision tree model, the framework dynamically selects the most suitable optimization strategy for a given matrix and target hardware platform.

The framework consists of three key components: (1) A matrix feature extraction pipeline that analyzes non-zero element distribution, sparsity patterns, and algorithmic properties to generate feature descriptors; (2) An interpretable adaptive model based on hierarchical decision tree that examines key parameters to effectively guide the selection of computing kernels and their configuration; and (3) A collection of specialized, lightweight candidate SpMV kernels designed to cover diverse scenarios and validate the framework's adaptability.

Extensive experimental evaluation on modern GPU architectures demonstrates that the framework delivers consistent performance improvements over a wide range of sparse matrices with differing sparsity characteristics. The model-enhanced SpMV kernels achieve an overall speedup ranging from 1× to 6× compared to the baseline, with an average speedup of 1.31×, compared to hipSPARSE, the average solution speed is improved by a factor of 1.22×. Furthermore, it maintains robust and stable performance while automatically adapting to matrix properties, highlighting its potential for practical deployment in heterogeneous computing environments.

Keywords: Heterogeneous computing · Parallel optimization · Adaptive optimization

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 518–536, 2026.
https://doi.org/10.1007/978-981-95-8405-5_28

1 Introduction

Sparse matrix vector multiplication (SpMV) represents a critical computational kernel in high performance computing (HPC), serving as a fundamental building block for numerous scientific applications, including neural networks, finite element analysis, large language models, and linear system solvers [3,11,24]. Despite its widespread use, achieving optimal SpMV performance remains challenging due to three primary bottlenecks: limited memory bandwidth utilization, irregular memory access patterns leading to poor cache efficiency, and load imbalance across parallel processing units.

To address these bottleneck, various innovations in storage formats and algorithmic techniques have been proposed to improve the computational efficiency of SpMV. These include tiling-based partitioning schemes [14], compression techniques that exploit cache hierarchy awareness [9], row-based decomposition methods [16], and hardware-specific optimizations [15]. However, due to differences in the distribution of non-zero elements leading to significant heterogeneity among matrices, a single SpMV kernel cannot achieve optimal performance across all sparse matrices.

With the development of machine learning techniques, auto-tuning frameworks have emerged as effective solutions for SpMV optimization. yaSpMV [22] introduces an automated framework for optimizing sparse matrix storage structures and thread configurations; Huang et al. [7] present a sophisticated performance modeling methodology for heterogeneous memory systems to predict optimal data placement; Gao et al. [6] develop a machine learning approach for predicting optimal thread configurations in CSR-format operations; and Li et al. [12] propose a runtime selection framework utilizing three neural networks to dynamically select from eight specialized kernels. Alphasparse proposes an approach to machine-designed sparse matrix storage and related SpMV kernel implementations [5].

Most frameworks focus on single optimization objectives (such as kernel selection, storage format selection, or parameter configuration), and neural network-based models employ completely black-box designs by unifying all parameters for training, making it difficult to explain the rationale behind optimization decisions, which limits the model's credibility and adaptability in practical applications.

The efficient implementation of SpMV algorithms hinges on the ability to analyze the distribution of non-zero elements and effectively utilize available computational resources. This insight motivates the development of our hybrid adaptive optimization framework.

Through analysis of real-world applications and representative sparse matrices, we have identified several structural patterns that significantly impact SpMV computational performance: (1) Diagonal-dominant patterns; (2) Block structures; (3) Band structures; (4) Irregular scatter patterns. These structural patterns frequently appear either individually or in combination within large-scale sparse matrices, as illustrated in Fig. 1.

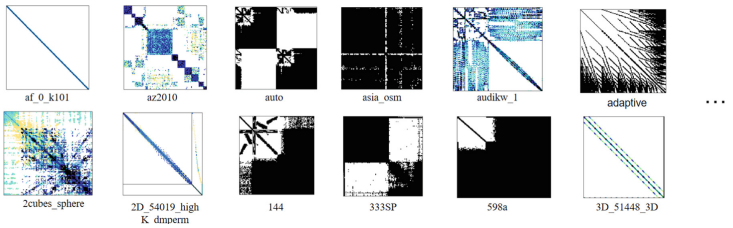


Fig. 1. Matrices sourced from *SuiteSparse Matrix Collection* [4].

By leveraging matrix characteristics, kernel design strategies, and hardware configurations, we aim to maximize both algorithmic and hardware performance. For computational kernels such as SpMV, performance is tightly coupled with data properties and resource utilization. Our framework models and analyzes structural features that impact performance, substantial speedups can be achieved through informed optimization.

We propose an adaptive SpMV optimization framework that extracts a range of row-oriented sparse matrix structural features, uses classification models to identify key parameters, and builds an interpretable optimization model. This model enables the framework to automatically select the suitable kernel and its optimal parameter configuration. The key contributions of our work are as follows:

- We design a GPU-oriented feature extraction pipeline that systematically integrates sparse matrix structural characteristics, SpMV kernel computational properties, and GPU architectural features, providing comprehensive feature support for the adaptive optimization framework.
- We highlight that parameter configuration is critical in achieving efficient SpMV performance. We propose an interpretable hierarchical classification framework that jointly addresses both kernel selection and parameter tuning. The model is trained in two stages: the first level determines the optimal kernel choice, while the second level optimizes the parameters for the selected kernel. This hierarchical formulation provides fine-grained, multi-level feature analysis respectively.
- We conducted experimental validation on two platform. The results demonstrate that the framework exhibits adaptive performance across various sparse matrix characteristics.

2 Background and Motivation

2.1 SPMV Optimization Method

The SpMV operation is defined as $y = Ax$, where each element is computed as $y^i = \sum_k A^{(i,k)} x^k$. Despite its mathematical simplicity, SpMV typically achieves only a small fraction of the peak floating-point performance, and achieving

optimal computational performance remains challenging. Extensive research has been dedicated for SpMV optimization to develop various optimization approaches.

The research on SpMV kernel mainly focuses on two optimization directions:(1) improving data locality and memory access efficiency, and (2) enhancing parallel execution and load balancing. For example, BCSR [2,20] exploits block-level to enable dense BLAS operations. CSR-k introduces a multilevel compressed sparse row format that organizes rows into hierarchical super-rows, improving memory locality and parallelism across different cache levels [8]. CSR-5 introduces tile-based partitioning for better SIMD utilization and automatic load balancing [14]. These format-level optimizations demonstrate the importance of data organization in achieving high performance, particularly in terms of efficient thread and warp-level workload distribution, improved memory coalescing, and enhanced vectorization opportunities. In addition, optimization methods for specific matrix patterns, such as compression techniques and Mixed precision method [13,16,17], further improve computational performance by reducing memory footprint and computation cost.

Furthermore, machine learning-based auto-tuning and prediction models for SpMV have been extensively studied. These include thread configuration optimization [8], prediction models for sparse matrix storage format selection [10,18], as well as performance prediction and load balancing strategies [1,19,21,23]. Auto-tuning and prediction models can also provide feedback to existing efficient SpMV implementation algorithms, which can further optimize the design of existing high-performance algorithms and break through the performance of existing calculations.

2.2 SPMV with Modern GPU Architectures

There are several key challenges in optimizing SpMV in modern GPU architecture, stemming from the fundamental mismatch between irregular sparse matrix access patterns and GPU's Single Instruction, Multiple Threads (SIMT) execution model. Modern GPUs adopt a hierarchical execution paradigm centered around Streaming Multiprocessors (SMs), Work-Group Processors (WGPs), or other execution units, as illustrated in Fig. 2 Each execution unit can support thousands of resident threads, organized into thread blocks or warps.

For SpMV operations, The GPU architecture presents three primary performance bottlenecks:

(1) Memory Access Latency: One of the primary bottlenecks in SpMV is the high memory access latency caused by irregular and indirect memory access patterns, particularly when fetching elements from the input vector x. These patterns significantly hinder spatial and temporal locality, leading to inefficient cache usage and poor memory coalescing on GPU architectures. As a result, SpMV implementations often fail to saturate the available memory bandwidth. Furthermore, the high latency of global memory access amplifies this inefficiency, making effective cache reuse and memory hierarchy-aware

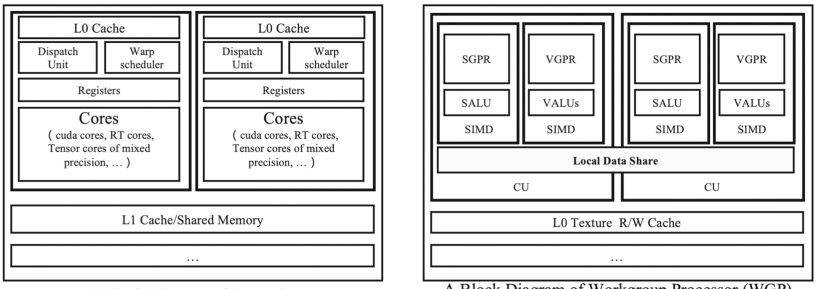


Fig. 2. Architectural overview of modern GPU memory hierarchy and execution units.

optimizations—such as shared memory blocking and register tiling—crucial for performance.

(2) Thread Divergence: The irregular distribution of non-zero elements across matrix rows leads to thread-level workload variability within a warp. In the GPU's SIMT execution model, this variability causes warp divergence—where threads in the same warp follow different control flow paths—resulting in serialized execution. Such divergence significantly degrades parallel efficiency, particularly in matrices with highly imbalanced non-zero counts per row. This mismatch reduces the effective utilization of GPU cores and limits throughput.

(3) Load Imbalance: The non-uniform sparsity structure of real-world matrices induces multi-level workload imbalance during parallel execution. At the intra-warp level, threads may process rows with varying numbers of non-zero elements; at the block and device level, thread blocks mapped to regions with denser rows incur longer execution times, leading to load imbalance across streaming multiprocessors (SMs). This imbalance is exacerbated by the GPU's bulk-synchronous execution semantics, where slower threads delay warp or block-level progress, resulting in low occupancy, idle cores, and suboptimal hardware utilization.

3 The Adaptive Optimization Framework

To enhance the computational performance of SpMV across diverse sparse matrices on GPU, we propose an adaptive optimization framework. The framework is as illustrated in Fig. 3, This framework including several designed SpMV kernels, adopting a systematic optimization strategy by employing a hierarchical decision tree model to analyze key matrix features, enabling an intelligent kernel selection mechanism that adaptively chooses the optimal SpMV computation strategy based on matrix characteristics.

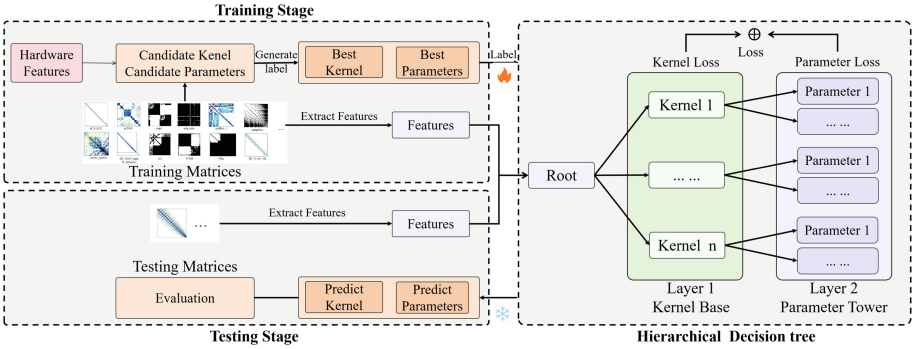


Fig. 3. The Adaptive SpMV Kernel Optimization Framwork.

3.1 Sparse Matrix Features

The distribution of non-zero elements in sparse matrices is often highly diverse. Some matrices exhibit regular banded patterns that align well with the GPU's SIMT execution model, thereby achieving efficient computation. In contrast, other matrices display irregular distributions of non-zero elements, characterized by significant row length variations and scattered sparsity patterns.

For matrices with substantial row length variation, adopting a fixed thread allocation strategy often results in underutilization of GPU resources and severe computational imbalance. Additionally, matrices with highly dispersed non-zero elements impose substantial memory access overhead, especially under high sparsity conditions. In such cases, efficient exploitation of the GPU's multi-level cache hierarchy becomes crucial for performance optimization.

Motivated by these observations, we design a row-oriented feature extraction mechanism that captures both intra-row and inter-row structural patterns. These extracted features guide the adaptive framework in selecting appropriate SpMV kernel strategies, enabling targeted optimizations based on matrix structure.

Table 1. Matrix Features for the Adaptive Framework

Feature	Description
m	Number of rows
n	Number of columns
nnz	Number of non-zero elements
max_nnz	Maximum number of non-zeros in a row
min_nnz	Minimum number of non-zeros in a row
avg_nnz	Average number of non-zeros per row
$nnz_variance$	Variance of non-zero counts per row
$col_variance$	Variance of column index gaps between consecutive non-zeros
$bandwidth$	Average bandwidth, defined as the average distance between the first and last non-zero in each row
$sparsity_imbalance$	Ratio between the number of non-zeros in sparse regions and dense regions

The specific feature extraction results are presented in Table 1. These features are designed to capture the distribution of non-zero elements in row-wise dimensions, providing a foundation for adaptive performance tuning in SpMV algorithms. The basic matrix dimensions are represented as m, n, and nnz, denoting the number of rows, columns, and non-zero elements, respectively. To describe row-wise sparsity distribution, we include the maximum, minimum, and average number of non-zeros per row, along with the variance of row-wise non-zero counts:

$$nnz_variance = \frac{1}{m}\sum_{i=1}^{m}(nnz_i - n\bar{n}z)^2$$

where nnz_i is the number of non-zeros in row i, and $n\bar{n}z$ is the mean.

col_variance measures the variance of column index differences between consecutive non-zeros within each row, indicating the degree of column-wise locality. A lower value suggests higher locality and potential for memory coalescing. *bandwidth* denotes the average span between the leftmost and rightmost non-zero in each row:

$$bandwidth = \frac{1}{m}\sum_{i=1}^{m}(col_{max}^{(i)} - col_{min}^{(i)} + 1)$$

sparsity_imbalance quantifies the distribution imbalance between sparse and dense regions by measuring the ratio of non-zero elements, which helps identify workload distribution irregularities.

3.2 Candidate Kernel Set

We adopt the Compressed Sparse Row (CSR) format as the data storage format and develop lightweight sparse matrix-vector multiplication (SpMV) kernels specifically optimized for this format. It is well established that effective storage format conversions and matrix reordering techniques can often enhance computational intensity effectively [2,9], as demonstrated by numerous studies, many adaptive models incorporate the selection of storage formats as a tunable variable to optimize performance [1,10,12]. However, in scenarios where SpMV operations are invoked infrequently, the overhead associated with format conversions can become significant relatively. Moreover, in many practical applications—such as sparse linear system solvers, graph-based learning models, and multi-stage numerical pipelines—matrix reordering often alters the structural characteristics of the original sparse matrix, which typically requires additional recovery steps to restore the original indexing or data layout. Both factors introduce further complexity and computational overhead to the overall workflow. Therefore, this paper does not delve into format conversion or matrix reordering strategies. Our kernel design focuses on efficient thread mapping and load balancing strategies, combined with optimizations leveraging GPU shared memory and cache hierarchies, as well as vectorization and warp shuffle techniques to improve execution efficiency.

SpMV kernel ❶ employs thread-level parallelism, where each thread is mapped to a single matrix row, and thread blocks cover consecutive rows. The kernel exploits the structural regularity of by preloading a fixed-size window of the input vector into registers. Combined with loop unrolling and vectorized memory access, this kernel is suitable for banded matrices.

Algorithm 1. SpMV Kernel❶

```
Input: A, x, y, bandwidth
Output: y = α · A · x + β · y
row ← blockIdx.x × blockDim.x + threadIdx.x
if row < num_rows then
    for i = 0 to cache_size − 1 do
        col ← row − ⌊bandwidth/2⌋ + i
        if 0 ≤ col < num_rows then
            x_cache[i] ← x[col]
        end if
    end for
    sum ← 0
    for j = row_ptr[row] to row_ptr[row + 1] − 1 step vector_size do
        vec_size ← min(vector_size, row_ptr[row + 1] − j)
        for k = 0 to vec_size − 1 do
            col ← col_ind[j + k]
            cache_idx ← col − (row − ⌊bandwidth/2⌋)
            if 0 ≤ cache_idx < cache_size then
                sum ← sum + values[j + k] × x_cache[cache_idx]
            else
                sum ← sum + values[j + k] × x[col]
            end if
        end for
    end for
    y[row] ← α × sum + β × y[row]
end if
```

SpMV kernel ❷ applies a sub-warp-level parallelism strategy by dividing each warp into multiple sub-warps, each responsible for computing a single row. Threads within a sub-warp cooperate to process the row's nonzero elements in parallel. Warp-level reduction using __shfl_down_sync is employed to aggregate partial results, __ldg is used to optimize read-only memory accesses. This approach is suited for matrices with relatively balanced and long rows.

SpMV kernel ❸ targets matrices with highly irregular row lengths by adopting a dynamic row-block partitioning strategy. Threads process rows within assigned blocks in a cyclic manner to balance workload across threads. Loop unrolling is applied to improve memory throughput. This method helps mitigate load imbalance and enhances kernel efficiency for unstructured sparsity patterns.

SpMV kernel ❹ serves as a baseline warp-level parallelism kernel, where each warp is responsible for one row and threads within the warp process nonzero

Algorithm 2. SpMV Kernel❷

Input: A, x, y, sub_warp_size
Output: $y = \alpha \cdot A \cdot x + \beta \cdot y$
$threads_per_row \leftarrow sub_warp_size$
$rows_per_block \leftarrow blockDim.x/threads_per_row$
$row \leftarrow blockIdx.x \cdot rows_per_block + threadIdx.x/threads_per_row)$
$lane \leftarrow threadIdx.x \mod sub_warp_size$
if $row < m$ **then**
    $sum \leftarrow 0$
    for $j \leftarrow row_ptr[row] + lane$ to $row_ptr[row+1]$ step sub_warp_size **do**
        $sum \leftarrow sum + values[j] \cdot x[col_ind[j]]$
    end for
    for $offset \leftarrow sub_warp_size/2$ to 1 step $offset/2$ **do**
        $sum \leftarrow sum + \text{shuffle_down}(sum, offset)$
    end for
    if $lane = 0$ **then**
        $y[row] \leftarrow \alpha \cdot sum + \beta \cdot y[row]$
    end if
end if

Algorithm 3. SpMV Kernel❸

Input: A, x, y, row_blocks
Output: $y = \alpha \cdot A \cdot x + \beta \cdot y$
$block_start \leftarrow row_blocks[blockIdx.x]$
$block_end \leftarrow row_blocks[blockIdx.x + 1]$
for $row \leftarrow block_start + threadIdx.x$ to $block_end$ step $blockDim.x$ **do**
    $row_start \leftarrow row_ptr[row]$
    $row_end \leftarrow row_ptr[row + 1]$
    $sum \leftarrow 0$
    for $j \leftarrow row_start$ to row_end **do**
        $sum \leftarrow sum + values[j] \cdot x[col_ind[j]]$
    end for
    $y[row] \leftarrow \alpha \cdot sum + \beta \cdot y[row]$
end for

elements cooperatively in a strided fashion. Warp-level reduction is used to accumulate results, and loop unrolling is applied to improve memory access efficiency. The design provides a reference for evaluating alternative kernel strategies.

These kernels integrate parallelization optimizations tailored to common sparse matrix patterns, and combining GPU hardware feature optimization techniques include cache-aware data reuse, warp-shuffle based reductions, adaptive workload partitioning, and register-level optimizations to enhance performance.

The configuration parameters for the four SpMV kernels are shown in Table 2. These parameter values are selected based on the architectural characteristics and resource constraints of modern GPUs. For each parameter, we choose a representative range of values commonly used in GPU kernel design.

To serve as a performance baseline, one value from each parameter set is randomly selected as a default configuration, allowing us to compare the adaptive tuning performance against a fixed, non-optimized setting.

Table 2. Configuration Parameters of SpMV Kernels.

Kernel	Parameters	Values	Default Values
SpMV Kernel❶–❹	Dim(Block)	32, 64, 128, 256, 512	256
SpMV Kernel❶	Bandwidth	32, 64, 128, 256	64
SpMV Kernel❷	sub_warp_size	4, 8, 16, 32	16
SpMV Kernel❸	row_size	16, 32, 64, 128	128
SpMV Kernel❹	N/A	N/A	N/A

3.3 Hierarchical Decision Tree Model

As mentioned above, the characteristics of different matrices, such as the distribution of non-zero elements, vary significantly. Different SpMV kernels are often optimized for specific types of matrices and cannot adapt to all matrices. Therefore, a reasonable approach is to employ a classifier to dynamically select the optimal SpMV kernel for different matrices. Commonly used machine learning models for such classification tasks include decision trees, support vector machines (SVM), random forests, and neural networks. Although neural networks excel at capturing complex nonlinear relationships, decision tree classifiers provide better interpretability because they can assess the importance of different features and observe their splitting boundaries.

Unfortunately, a naive decision tree can only classify the SpMV kernels, while different SpMV kernels have distinct configuration parameters and their performance is quite sensitive to these parameters. Without additional handling, even if the optimal SpMV kernel is selected, the performance is likely to be suboptimal due to inaccurate parameter configuration. Therefore, as illustrated in Fig. 3, we constructed a hierarchical decision tree structure to address both kernel prediction and parameter prediction. The first layer of the decision tree performs coarse-grained classification of kernel types, while the second layer consists of multiple independently trained parameter configuration decision trees, each modeling fine-grained parameter selection for a specific kernel type. This structure effectively captures the heterogeneity in parameter distribution across different kernels, significantly enhancing the overall prediction accuracy.

Training Stage. During the training phase, we first generate a large number of training samples, with each matrix corresponding to an optimal kernel and optimal parameter configuration. We use an exhaustive approach, traversing all candidate kernels and their candidate parameter spaces (as detailed in Table 2)

for each matrix and recording the actual execution time for every configuration. After obtaining the execution times for all configurations, we can easily identify the best kernel and parameter configuration for each matrix. And as we mentioned that we also consider hardware information when selecting candidate parameter spaces to enhance the algorithm's robustness and portability.

After generating the data, we use these labeled samples to train the hierarchical decision tree. Specifically, for each training sample, we first perform feature extraction to identify representative features of the sparse matrices and best execution time as labels. The sample then undergoes iterative optimization within the decision tree. The loss in the hierarchical decision tree consists of two components: one is the loss from kernel selection in the kernel base, and the other is the loss from parameter selection in the parameter tower. Each sample enters only one of the parameter towers. Both components of the loss utilize the cross-entropy loss function. For training stability, the maximum depth of each decision tree is limited to 3, and each internal node requires at least 15 samples to split.

Testing Stage. During the testing phase, for each new incoming sparse matrix, we similarly perform feature extraction. The extracted features are then fed into the trained hierarchical decision tree, resulting in the prediction of the optimal kernel and its best parameter configuration.

Discussion. This hierarchical decision tree framework allows each subproblem to be modeled using the most relevant features and splitting strategies, improving overall prediction capability. The layered structure enables each kernel type to maintain a dedicated parameter tree and encoding scheme, with each decision tree minimizing its own entropy-based loss. This design prevents interference from unrelated categories and reduces the impact of irrelevant features or imbalanced classes, thereby enhancing both prediction accuracy and generalization. Furthermore, it balances differences in parameter space and optimal configuration across kernel types.

At deployment, the runtime overhead introduced by our framework mainly stems from extracting matrix structural features and performing decision tree inference. As the structural properties of sparse matrices are typically static in most applications, the inference process only needs to be executed once. In addition, the proposed framework is highly modular and extensible, allowing for integration of new SpMV kernels to support and optimize a broader range of sparse workloads.

4 Experimental Evaluation

To evaluate the effectiveness of the proposed adaptive optimization framework, we conduct a series of experiments across two computing platforms and various

sparse matrices. The evaluation emphasizes both the improvement in computational efficiency over the original kernels and the effectiveness of performance prediction enabled by the adaptive model.

4.1 Experimental Setup

Hardware and Software Environment. Experiments are conducted on two heterogeneous computing platforms. The hardware and software information of the two platforms. The NVIDIA platform is equipped with Tesla V100 GPU, uses with 64 Streaming Multiprocessors (SMs) and 32 GB of HBM2 memory per GPU, manages warps consisting of 32 threads. The software environment includes CUDA 12.4, the NVIDIA driver version 550.76, and GCC 11.3.0 as the host compiler. The AMD platform features Instinct MI60 GPUs with 64 CUs per GPU and 16 GB of HBM2 memory, where a wavefront consists of 64 threads. The system runs on ROCm 5.4.0 with the HIP runtime, and also uses GCC 11.3.0 for compilation.

Our experiments were conducted on two platforms, where most of the results exhibited similar trends. Except for cases specifically highlighted, we selected the results obtained on the NVIDIA GPU platform as representative examples for illustration.

Matrix Dataset. We choose 600 large matrices (with the row number n $>$ 50,000, the nonzero number nnz$>$ 100,000) from the SuiteSparse Matrix Collection [4] for the experiments. During the tests, we use double precision for matrix values and repeat each kernel function 50 times to measure the average performance. We adopt stratified random sampling to divide the dataset into training (80%) and testing (20%) subsets.

4.2 Overall Performance of the Hierarchical Decision Tree

To demonstrate the effectiveness of the hierarchical classifier, we evaluated its classification performance, as shown in Table 3. Our method achieved an F1-score of around 0.8, even when trained on less than 500 matrices. This suggests that many sparse matrices share similar features. With more diverse and abundant training data, the model's accuracy could be further improved.

Moreover, the parameter tower also achieved high prediction accuracy for different SpMV kernels, with many parameters exceeding an accuracy of 0.8. Notably, the accuracy for the bandwidth parameter reached over 90%. This demonstrates the effectiveness of our matrices feature extraction and the effectiveness of hierarchical structure in predicting kernel and parameters separately. However, the variation in accuracy across different parameters indicates that precise prediction for all parameters is relatively challenging. This is an area for further optimization in our future work.

Table 3. Evaluation matrices of the hierarchical decision tree.

	Layer 1: Kernel Base	Layer 2: Parameter Tower					
	–	SpMV❶		SpMV❷		SpMV❸	
Parameter	–	block	bandwidth	block	wavefront	block	row_size
Precision	0.7997	0.8000	0.9535	0.6909	0.8872	0.6186	0.6055
Recall	0.8333	0.8571	0.9402	0.5333	0.8667	0.5604	0.6593
F1-score	0.8161	0.8254	0.9338	0.5368	0.8500	0.5029	0.6120

4.3 Execution Time of Dynamic SpMV Kernel

We discuss the importance of kernel selection. To this end, we selected 300 sparse matrices and evaluated their performance across multiple SpMV kernels with varying parameter configurations. The experimental results (see Fig. 4) reveal differences in execution time among the kernels, particularly as the matrix size increases. For example, SpMV kernel ❹ exhibits a more pronounced increase in execution time as matrix rows grow.

Furthermore, we compared the performance under fixed and optimized parameter settings in Fig. 4. The results demonstrate that parameter tuning has a notable impact on the performance of kernel ❶,❷,❸, especially for large-scale matrices. In contrast, the performance of kernel ❹ is relatively insensitive to parameter changes, as its configuration mainly affects the thread block dimensions during kernel launch.

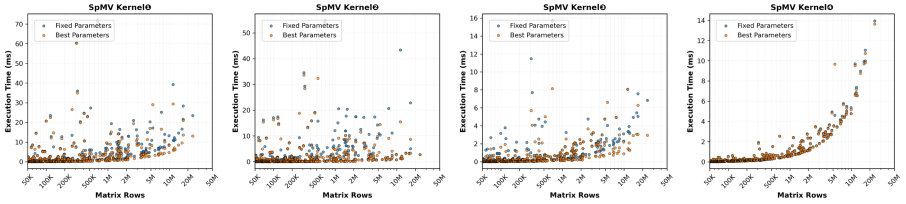


Fig. 4. Execution time for SpMV kernels of default parameters and optimal parameters.

In order to gain a more intuitive understanding of how kernel selection and parameter configuration affect SpMV performance, we listed several SpMV execution times for kernel ❷ and kernel ❸ as examples, as illustrated in Fig. 5. The results show that, for some matrix, different kernels yield varying execution times with fixed parameters, indicating that certain sparse matrices are inherently better suited to specific kernel algorithms due to distribution characteristics of non-zero elements. Furthermore, for matrices such as Freescale1, circuit5M_dc, and Delaunay_n23, where the execution times of fixed parameter kernels are relatively close, further tuning of kernel parameters can still lead to noticeable performance improvements. These observations underscore

the importance of not only selecting an appropriate kernel but also performing fine-grained parameter optimization to achieve optimal SpMV efficiency.

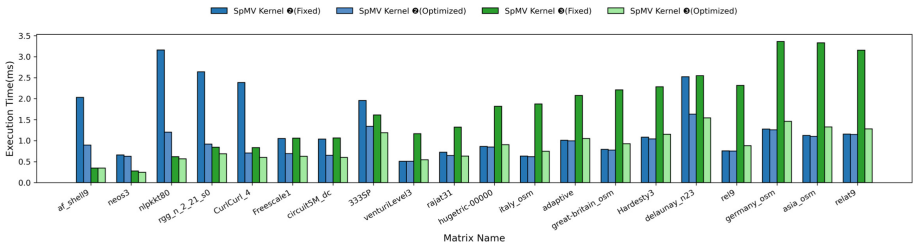


Fig. 5. Execution time of SpMV Kernel ❷ and Kernel ❸ with fixed and optimized parameters on matrices.

These findings highlight the necessity of kernel selection and parameter tuning in achieving efficient SpMV computation and validate the motivation and significance of our work in this direction.

Next we evaluate the effectiveness of our prediction model. The dataset of 600 matrices was partitioned into a training set of 480 matrices and a test set of 120 matrices. The trained model was used to predict the optimal kernel and parameter configurations, which were then applied to actual SpMV executions. We compared the model's performance with a static baseline—SpMV kernel ❹—configured with a fixed block dimension of 256. As shown in the Fig. 6, our method achieves faster execution times than the baseline on the majority of the test matrices, with an average speedup of 1.31×. This confirms the effectiveness of our model and demonstrates its ability to support both kernel selection and parameter tuning.

4.4 Ablation Study

To demonstrate the effectiveness of our proposed hierarchical decision tree model, we compared the execution times of a naive single-layer decision tree model and our hierarchical decision tree model across different matrices. The naive single-layer decision tree model refers to using only one decision tree to predict the optimal kernel, while the kernel's parameters are set to default values. The box plots of execution times for different decision tree models are shown in Fig. 7. We can observe that for all kernels, the execution time of the hierarchical decision tree model is consistently shorter than that of the naive single-layer decision tree model, especially for the SpMV kernel ❷. This highlights the importance of parameter selection in the SpMV execution process and demonstrates that the hierarchical decision tree model is a reliable solution to this challenge.

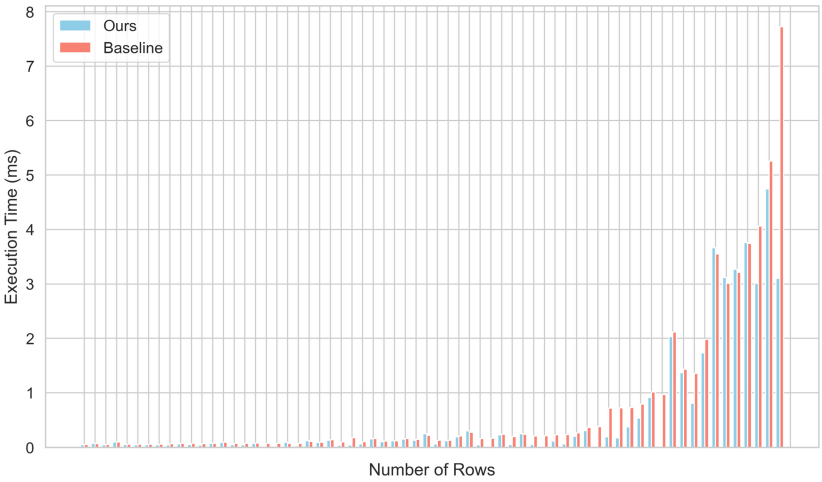


Fig. 6. Execution time of our prediction SpMV implementation and baseline implementation.

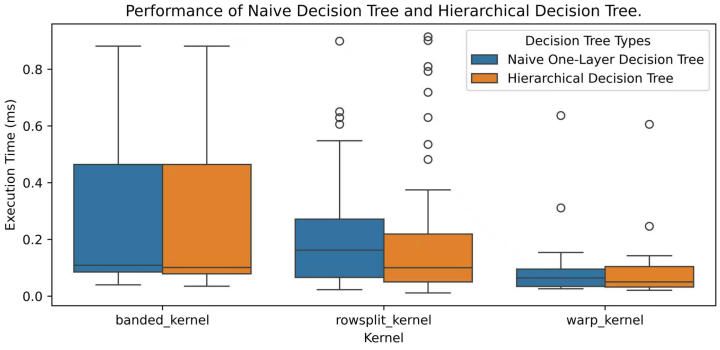


Fig. 7. Execution time of naive single-layer decision tree and hierarchical decision tree.

4.5 Decision Tree Analysis

The hierarchical decision tree generated by the model clearly demonstrates the decision-making logic, facilitating the understanding of how the model utilizes features for classification. Based on multiple runs of the hierarchical decision tree model and its visualization analysis, we conducted an in-depth investigation of feature importance.

In the first-level classification (kernel classifier), *max_nnz* serves as the primary feature to distinguish different categories, followed by the *nnz_variance*, *sparsity_imbalance* and the *col_variance* as key features for further refinement. Specifically, SpMV kernel ❶ mainly clusters in regions with relatively small *max_nnz* and *nnz_variance*; SpMV kernel ❷ is primarily distributed where *max_nnz* and *avg_nnz* are small; whereas SpMV kernel ❸ concentrates in areas

with relatively large *max_nnz*, *nnz_variance*, and *avg_nnz*. This observation aligns with our structured analysis of the matrices and the design of the kernels, as the implementation of our kernel algorithms takes these distinct sparsity structure features into account.

At the second-level model, the emphasis on parameter configuration varies across different kernels. For SpMV kernel ❶, a coarse classification is first performed based on the total number of *nnz*, followed by a finer parameter tuning using features such as *col_variance*, *avg_nnz*, and bandwidth. For SpMV kernel ❷, *avg_nnz* is the most critical feature; meanwhile, features including *sparsity_imbalance*, *m*, and *col_variance* play key roles in selecting the dimension block parameters. The adjustment of *sub_warp_size* parameters relies more heavily on *avg_nnz*, *sparsity_imbalance*, *max_nnz*, *min_nnz*, as well as *col_variance*.

By visualizing the model, it is possible to further optimize the matrix feature extraction parameters, remove non critical features, and preserve and refine key features. This is also the reason why we chose the decision tree model, and the hierarchical decision tree model we used makes the analysis of parameters more refined and accurate.

4.6 Compared With hipSPARSE

To further evaluate the performance advantages of the adaptive framework, we compared its computational efficiency with hipSPARSE on AMD GPU. Figure 8 shows the comparison between the optimized SpMV kernel implementation of the adaptive framework and the computation time of hipsparseDcsrmv. The experimental results shows that our execution time is generally shorter than that of hipSPARSE SpMV kernel, and without relying on complex implementation algorithms and data storage structures, our average solving speed is 1.22x faster than the hipSPARSE implementation.

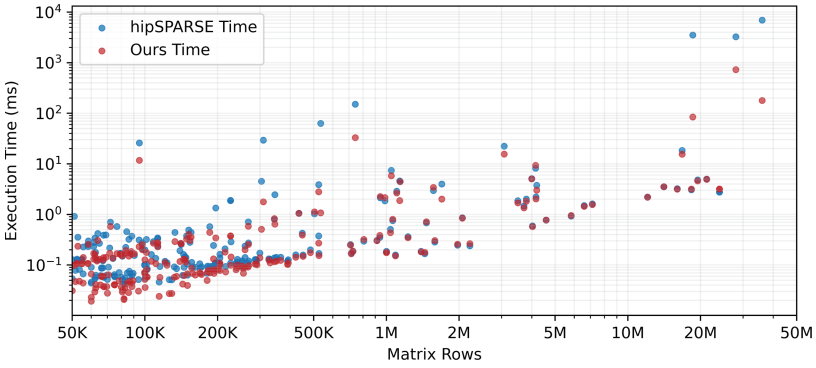


Fig. 8. The comparison of our SpMV execution time vs hipSPARSE.

5 Conclusion

In this paper, we have presented an adaptive optimization framework for SpMV computation that effectively addresses the challenges posed by irregular memory access patterns and diverse sparsity structures. The key contributions of our work include a triple-aware optimization framework that integrates matrix feature awareness, algorithmic characteristic awareness, and hardware architecture awareness, providing a comprehensive approach to SpMV performance optimization; A layered decision tree performance optimization model, which enables accurate prediction of optimal kernel selection and parameter configuration based on matrix characteristics; An efficient implementation of SpMV framework that achieves consistent performance improvements across different hardware platforms.

This comprehensive optimization approach, which considers matrix characteristics, hardware architecture features, and algorithmic properties, reduces the need for manual tuning while providing a foundation for incorporating and optimizing existing complex algorithms to break through the performance limitations of original computations, achieving significant performance improvements without relying on complex kernel implementations.

Acknowledgments. This study was supported by the Strategic Priority Research Program of the Chinese Academy of Sciences (No.XDB0500101).

References

1. Barreda, M., Dolz, M.F., Castano, M.A., Alonso-Jordá, P., Quintana-Orti, E.S.: Performance modeling of the sparse matrix-vector product via convolutional neural networks. J. Supercomput. **76**, 8883–8900 (2020)
2. Borštnik, U., VandeVondele, J., Weber, V., Hutter, J.: Sparse matrix multiplication: the distributed block-compressed sparse row library. Parallel Comput. **40**(5–6), 47–58 (2014)
3. Chen, Z., Liu, H., Yang, B.: Accelerating iterative linear solvers using multiple graphical processing units. Int. J. Comput. Math. **92**(7), 1422–1438 (2015)
4. Davis, T.A., Hu, Y.: The university of florida sparse matrix collection. ACM Trans. Math. Softw. (TOMS) **38**(1), 1–25 (2011)
5. Du, Z., Li, J., Wang, Y., Li, X., Tan, G., Sun, N.: Alphasparse: generating high performance spmv codes directly from sparse matrices. In: SC22: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–15. IEEE (2022)
6. Gao, J., Ji, W., Liu, J., Wang, Y., Shi, F.: Revisiting thread configuration of SPMV kernels on GPU: a machine learning based approach. J. Parallel Distributed Comput. **185**, 104799 (2024)
7. Huang, Y., Li, D.: Performance modeling for optimal data placement on GPU with heterogeneous memory systems. In: 2017 IEEE International Conference on Cluster Computing (CLUSTER), pp. 166–177. IEEE (2017)

8. Kabir, H., Booth, J.D., Raghavan, P.: A multilevel compressed sparse row format for efficient sparse computations on multicore processors. In: 2014 21st International Conference on High Performance Computing (HiPC), pp. 1–10. IEEE (2014)
9. Lane, P.A., Booth, J.D.: Heterogeneous sparse matrix-vector multiplication via compressed sparse row format. Parallel Comput. **115**, 102997 (2023)
10. Li, J., Tan, G., Chen, M., Sun, N.: Smat: an input adaptive auto-tuner for sparse matrix-vector multiplication. In: Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation, pp. 117–126 (2013)
11. Li, J., Xu, J., Li, S., Huang, S., Liu, J., Lian, Y., Dai, G.: Fast and efficient 2-bit LLM inference on GPU: 2/4/16-bit in a weight matrix with asynchronous dequantization. In: Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design. ICCAD '24, Association for Computing Machinery, New York (2025)
12. Li, M., Ao, Y., Yang, C.: Adaptive SPMV/SPMSPV on GPUS for input vectors of varied sparsity. IEEE Trans. Parallel Distrib. Syst. **32**(7), 1842–1853 (2020)
13. Liu, J., Wang, Y., Gao, J., Ji, W.: PSPMV: precision-based sparse matrix partition and SPMV optimization. CCF Trans. High Perf. Comput. **6**(6), 549–565 (2024)
14. Liu, W., Vinter, B.: Csr5: an efficient storage format for cross-platform sparse matrix-vector multiplication. In: Proceedings of the 29th ACM on International Conference on Supercomputing, pp. 339–350 (2015)
15. Lu, Y., Liu, W.: Dasp: specific dense matrix multiply-accumulate units accelerated general sparse matrix-vector multiplication. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–14 (2023)
16. Pang, M., Fei, X., Qu, P., Zhang, Y., Li, Z.: A row decomposition-based approach for sparse matrix multiplication on GPUS. In: Proceedings of the 29th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp. 377–389 (2024)
17. Qiu, H., et al.: A conflict-aware divide-and-conquer algorithm for symmetric sparse matrix-vector multiplication. In: SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–15. IEEE (2024)
18. Sedaghati, N., Mu, T., Pouchet, L.N., Parthasarathy, S., Sadayappan, P.: Automatic selection of sparse matrix representation on GPUS. In: Proceedings of the 29th ACM on International Conference on Supercomputing, pp. 99–108 (2015)
19. Sim, J., Dasgupta, A., Kim, H., Vuduc, R.: A performance analysis framework for identifying potential benefits in GPGPU applications. In: Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming, pp. 11–22 (2012)
20. Vuduc, R.W., Moon, H.J.: Fast sparse matrix-vector multiplication by exploiting variable block structure. In: High Performance Computing and Communications: First International Conference, HPCC 2005, pp. 807–816, Sorrento, Italy, September 21-23, 2005. Proceedings 1. Springer (2005)
21. Xie, Z., Liu, J., Li, J., Li, D.: Merchandiser: data placement on heterogeneous memory for task-parallel HPC applications with load-balance awareness. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp. 204–217 (2023)
22. Yan, S., Li, C., Zhang, Y., Zhou, H.: yaspmv: yet another spmv framework on gpus. In: Proceedings of the 19th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming. p. 107–118. PPoPP '14 (2014)

23. Yesil, S., Heidarshenas, A., Morrison, A., Torrellas, J.: Wise: predicting the performance of sparse matrix vector multiplication with machine learning. In: Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming, pp. 329–341 (2023)
24. Zeng, S., et al.: FLIGHTLLM: efficient large language model inference with a complete mapping flow on FPGAS. In: Proceedings of the 2024 ACM/SIGDA International Symposium on Field Programmable Gate Arrays, pp. 223–234 (2024)

GMS-YOLO: A Lightweight Algorithm for SAR Ship Target Detection Based on YOLOv8n

Xiaozhang Liu, Xinting Zhou, Xiulai Li(✉), and Jiajing Xu

School of Computer Science and Technology, Hainan University, Haikou, China 570228
lxzh@hainanu.edu.cn

Abstract. To address the problems of noise interference, small target size, and complicated backgrounds in ship detection using synthetic aperture radar (SAR), this study proposes the GMS-YOLO, a lightweight model. This study uses the lightweight GhostConv to reduce computation and model parameters. To enhance feature representation while maintaining the model's lightweight design, this study designs a C2f-M module based on the mixed local channel attention (MLCA) mechanism. Furthermore, SPPFAS is intended to improve detection performance for targets at various scales. Lastly, this study introduces the Wise-IoU to resolve the issue of unfavorable gradients caused by occurrences of low quality in the training dataset for target detection. The findings of the experiment show that, compared to YOLOv8n, the proposed GMS-YOLO algorithm achieved higher R, mAP50, and mAP50-95 on the SSDD dataset by 2.4%, 0.7%, and 1.3%, and on the HRSID dataset by 1.4%, 1%, and 1.1%. Additionally, the parameters and FLOPs were reduced to 2.8M and 7.8G, respectively, for both datasets. In the comparative experiments with other deep learning approaches, the GMS-YOLO algorithm still performed better, achieving a good balance between lightweight and accuracy.

Keywords: synthetic aperture radar (SAR) ship detection · GMS-YOLO · C2f-M · SPPFAS · Wise-IoU

1 Introduction

Synthetic aperture radar (SAR), due to its high resolution, all-weather capability, and insensitivity to weather conditions [1], holds significant value in fields such as Earth observation [2], environmental monitoring [3], military reconnaissance [4], agricultural yield estimation [5], disaster early warning [6], and ship rescue [7], and it plays a particularly crucial role in ship detection. As deep learning technology continues to advance, it has become a common technique for the automatic detection of ships in SAR imagery. However, this field continues to face numerous challenges. Currently, SAR ship detection algorithms are mainly categorized into traditional and deep learning-based algorithms [8].

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 537–554, 2026.
https://doi.org/10.1007/978-981-95-8405-5_29

The primary focus of traditional methods is on manual feature extraction and image segmentation techniques, which rely heavily on the experience of professionals and have low detection efficiency. Examples include wavelet transform-based detection methods [9], constant false alarm rate (CFAR) methods [10], template matching methods [11], and so on. Although these methods show some effectiveness in detection, inherent limitations, such as dependence on experience and high computational complexity, lead to low efficiency in practical applications.

In recent years, as artificial intelligence technology has advanced quickly, deep learning-based methods have achieved notable advancements in SAR ship target detection [12]. Two types of deep learning-based methods for target detection are categorized as follows: two-stage and one-stage target detection methods.

The two-stage detection methods first generate candidate frames and then classify them in a regression. These methods mainly include R-CNN, Faster R-CNN, and Mask R-CNN. Girshick et al. [13] proposed Region-based CNN (R-CNN), a simple and scalable detection method. This algorithm generates candidate regions through selective search and performs feature extraction and classification regression on each region to achieve target detection, significantly improving target detection performance on the PASCAL VOC dataset. Ren et al. [14] designed a region proposal network (RPN) that reduces the computational cost of candidate region generation through an end-to-end training approach, thus improving the effectiveness and precision of object detection. He et al. [15] designed Mask R-CNN, which achieves efficient instance segmentation and target detection by incorporating a parallel branch into Faster R-CNN for predicting the segmentation mask of targets. This approach increases the precision of segmentation. One-stage methods, in contrast to two-stage methods, eliminate the need for candidate region extraction, achieving faster detection speeds, lower computational costs, and maintaining high accuracy.

One-stage detection methods mainly include you only look once (YOLO) series [16–21] and single-shot multibox detector (SSD) [22]. Jeong et al. [23] designed a method for object detection to raise the accuracy of traditional SSD. By optimizing the structure near the classifier network, the method enhanced the effective utilization of feature maps, achieving high performance with 78.5% mAP on the Pascal VOC 2007 test set while maintaining fast detection speed. Wang et al. [24] introduced a SAR object detection approach based on SSD, which achieves multi-scale feature learning through data enhancement and transfer learning strategies, and the experimental findings demonstrate that this approach outperforms other detection techniques in terms of detection performance. Sun et al. [25] developed a BiFA-YOLO for detecting arbitrarily oriented SAR ships based on the YOLO framework. This method effectively aggregates multi-scale features and accurately obtains ship angle information through an angle classification structure and a bidirectional feature fusion module, improving the detection accuracy and speed of ships in arbitrary directions and dense arrangements in high-resolution SAR images. Bai et al. [26] proposed FEPS-Net, a novel SAR ship detecting network, which raises the effectiveness of small ship detection

in SAR imagery by designing feature enhancement pyramids and shallow feature reconstruction modules. Yasir et al. [27] proposed an upgraded YOLOv5s technology, which improves the model's backbone and neck by combining C3 and FPN+PAN structures with attention mechanisms. This efficiently addresses problems such as complicated background interference and the blurring of small ship features while achieving high detection rates for multi-scale SAR ships. Tang et al. [28] improved a SAR ship detection approach called N-YOLO, built upon YOLOv5. By incorporating a SAR target potential area extraction (STPAE) module, a noise level classifier (NLC), and the YOLOv5 detection module, this method enhances The effectiveness and precision of SAR ship detection. Guo et al. [29] introduced a refined detection approach based on YOLOv5. This method enhances the learning of feature information in both spatial and channel dimensions, along with the integration of multi-scale target information by integrating the bidirectional feature pyramid network (BiFPN) and the convolutional block attention module (CBAM). This effectively reduces the issue of detection leakage in multi-scale targets.

The above studies have achieved significant results in enhancing ship detection accuracy in SAR imagery. Nevertheless, in practical applications, they face the challenge of high algorithm complexity. Xu et al. [30] improved Lite-YOLOv5, a lightweight star-borne SAR ship detection model, built upon the YOLOv5 algorithm. By designing the lightweight cross stage partial (L-CSP) module, applying network pruning, and integrating several optimization modules, it achieves both a lightweight architecture and high detection accuracy. Ren et al. [31] developed YOLO-Lite, a lightweight network designed for SAR ship detection. By designing lightweight feature enhancement backbone (LFEBNet), channel and position enhancement attention (CPEA) modules, and enhanced spatial pyramid pooling (EnSPP) modules, and combining these with a multi-scale feature fusion network (MFFNet) and a novel confidence loss function, they achieved both reduced computational cost and high-precision ship detection. Tang et al. [32] designed YOLO-SARSI, based on YOLOv7 with an attention mechanism multi-scale receptive field convolution block (AMMRF), For detecting ship targets in SAR ocean images with high resolution. This method enhances the capacity to differentiate between ships and the background by fully utilizing the positional information of feature maps and channel relationships. Thus, the enhancement of ship detection performance, coupled with the optimization of the lightweight model, continues to be a focus of investigation, with the aim of striking a balance between computational complexity and detection effectiveness.

To address the above challenges, this study proposes GMS-YOLO, a lightweight model that increases SAR ship detection accuracy while keeping a lightweight architecture. The following are the main contributions of this study:

– To address the feature redundancy problem, this study introduced GhostConv, replacing the Conv of the YOLOv8n backbone with GhostConv, which extracts a large number of features through a small amount of computa-

tion, effectively solving the redundancy problem in the procedure for feature extraction.

- To address the issue of insufficient feature extraction due to background complexity while maintaining lightweight, this study proposed the C2f-M module, which combines C2f with the mixed local channel attention mechanism (MLCA) to enhance the network's ability to capture useful features by combining local and global features as well as channel and spatial feature information.
- To address the problem that SPPF in YOLOv8 ignores the global feature information of the input image, this study proposed the SPPFAS module. By fusing the global max pooling and global average pooling operations into the SPPF module and introducing SEBlock, this module acquires more image information and enhances target detection at various scales.
- To address the problem of low-quality images in SAR ship datasets, this study adopted WIoU-v3, which uses a gradient gain assignment strategy and a dynamic non-monotonic focusing mechanism to increase the model's detection accuracy and robustness.

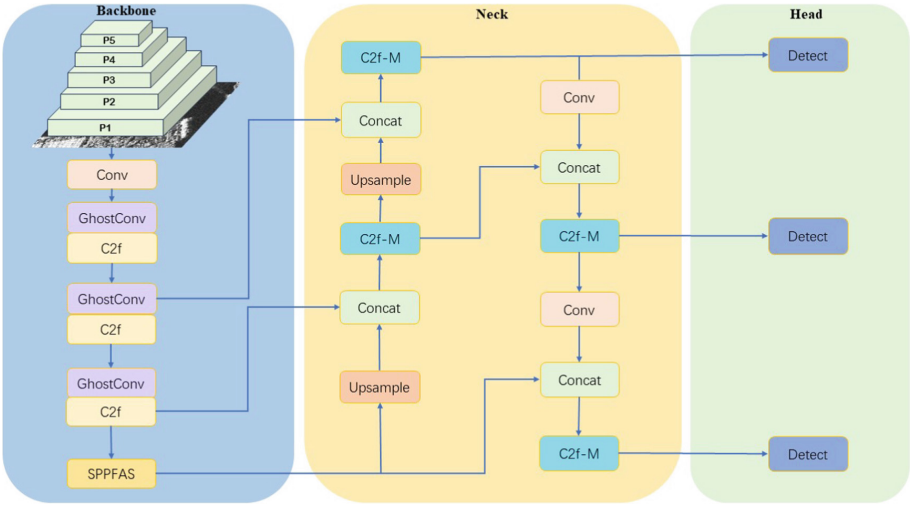


Fig. 1. The structure of GMS-YOLO.

2 Methods

The overall network structure of GMS-YOLO based on YOLOv8n proposed in this study is shown in Fig. 1. The following four main improvements are made

in GMS-YOLO. In the backbone section, GhostConv replaces Conv, which contributes to the improvement of lightweight algorithms. In the neck section, SPPFAS is designed instead of SPPF, capturing more diverse feature information and enhancing the capacity to detect targets at various scales. Additionally, the C2f-M module integrates the lightweight MLCA module into C2f, combining the benefits of the C2f and MLCA mechanism. This integration maintains the model's lightweight nature while better capturing key features. Consequently, target detection performance is improved. In addition, WIoU is used in place of the algorithm's loss function to enhance the recall of objects and increase the detection boxes' precision.

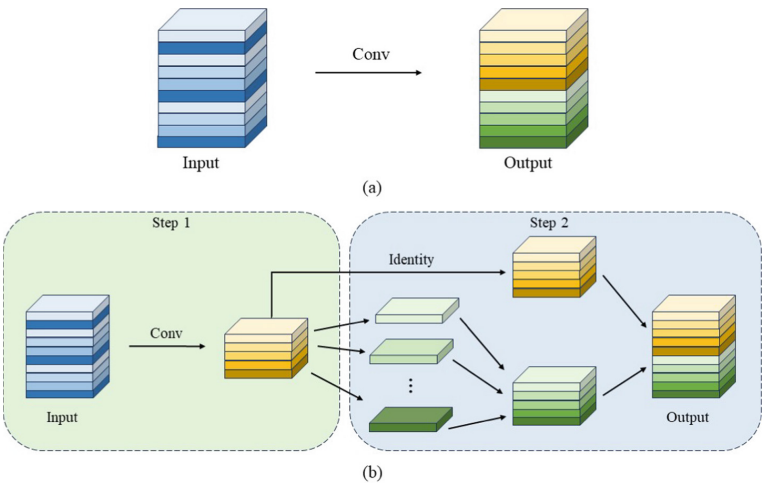


Fig. 2. The process of Conv(**a**) and GhostConv(**b**).

2.1 GhostConv

Given that the feature maps generated by contemporary mainstream convolutional neural networks (CNNs) frequently include a significant number of redundant features, some of which are similar to one another, this increases computational cost in addition to adding more parameters. To address this issue, GhostNet [33] proposed an innovative method that extracts many features with few computations, effectively solving the redundancy problem in the feature extraction process. Inspired by this advantage, this study introduces GhostConv, a novel convolutional layer that is different from conventional convolutional operations.

As illustrated in Fig. 2, GhostConv divides the convolution operation into two steps: firstly, it performs a standard convolution, which involves obtaining feature maps using a standard convolution and extracting intrinsic feature maps

with a convolution layer of kernel size; secondly, it performs a lightweight linear transformation operation, which involves applying a convolution with a kernel size of 5 to the intrinsic feature maps from the previous step, generating additional feature maps. The feature maps obtained from these two steps are spliced in the channel dimension to generate the final feature map. This method can generate feature maps comparable to normal convolution with fewer parameters and computational resources, thus achieving a lightweight model.

2.2 C2f-M Module

The C2f module is one of the basic modules in the YOLOv8 network architecture, capable of providing rich gradient flow information while keeping the lightweight network. However, the original C2f module primarily employs standard convolutions and bottleneck structures. While these can extract features effectively, they fail to focus adequately on the target regions. This study designs the C2f-M module, integrating an attention mechanism to improve the model's capability for feature extraction, based on the C2f module's excellent characteristics. The attention mechanism enables the model to focus on the key features of targets, thus increasing the precision of detection. In this study, the MLCA mechanism [34] is introduced to improve detection by combining local and global feature information with channel and spatial feature information while maintaining computational efficiency.

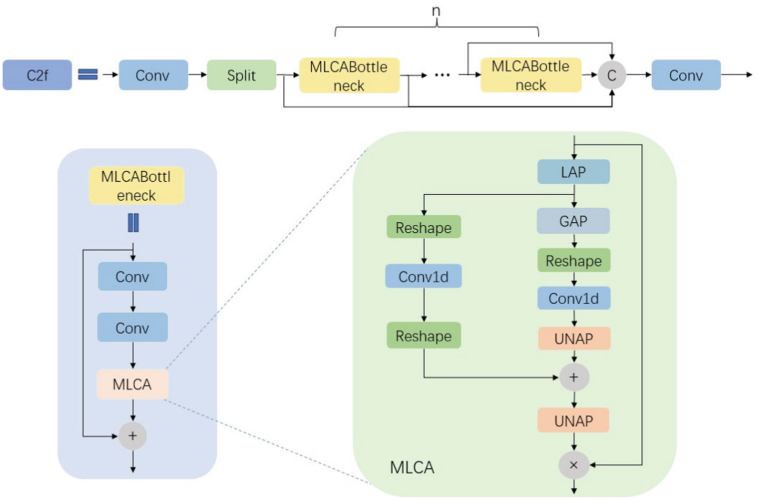


Fig. 3. The structure of C2f-M.

MLCA is a lightweight attention mechanism that improves detection accuracy with a negligible increase in parameter count. The MLCA mechanism is

shown in Fig. 3. The input feature maps are first processed by local average pooling (LAP) and global average pooling (GAP) [35] to capture the local spatial information and the statistical information of the entire feature maps, respectively, to enhance the representational capability of the model. The features obtained from global pooling and local pooling are subject to transformation through 1D convolution (Conv1d) to maintain the spatial dimension while compressing the feature channels. The features after 1D convolution are rearranged (Reshape) to fit subsequent operations. For the local pooling features, after 1D convolution and rearrangement before being integrated with the original input features using a "multiplication" operation ($\times$), which is equivalent to a feature selection to highlight valuable features. For the features obtained from global pooling, they undergo 1D convolution and rearrangement before being integrated with the local pooling features using an 'addition' operation, achieving the fusion of global contextual information. Lastly, the operation of unpooling (UNAP) returns the feature maps that have been enhanced by both global and local attention to their original spatial dimensions. By integrating MLCA, the model preserves computational efficiency while improving the capacity for feature extraction.

The C2f-M module is illustrated in Fig. 3. The integration of the MLCA mechanism with the C2f module of the YOLOv8 architecture brings great advantages for SAR ship detection. Due to the complexity of the background, ship feature learning is significantly affected, which leads to insufficient ship feature extraction. The module maintains computational efficiency while optimizing feature representation by integrating global and local information to highlight important features. This makes the model more capable of capturing useful features, effectively mitigating interference from non-target backgrounds, thus enhancing the model's performance.

2.3 SPPFAS Module

Although the traditional SPPF has adopted three strategies of max-pooling to improve the capacity to extract features, there are still some shortcomings: Max-pooling can only extract the maximum value of the input features and can only represent the local information of the input features while ignoring the input image's global feature information. For SAR images that contain noise and complex backgrounds, this processing method results in insufficient information about targets and backgrounds at different scales and interference from background noise. Therefore, this study proposes the SPPFAS module, which integrates global average pooling and global max pooling operations into SPPF to obtain the global background and global edge information of the images. SPPF extracts features through different pooling layers, combining global background and edge information, which allows the model to better capture information from multi-scale features, thereby boosting its capability to detect targets of different sizes. This information is then spliced together with the local feature maps to form a feature map containing the global view. And finally, the SEBlock is introduced to optimize the feature expression by adaptively recalibrating the weights

of each channel to improve the differentiation between targets and backgrounds, so as to enhance detection performance and lessen the influence of complicated backgrounds. The structure of the SPPFAS is illustrated in Fig. 4.

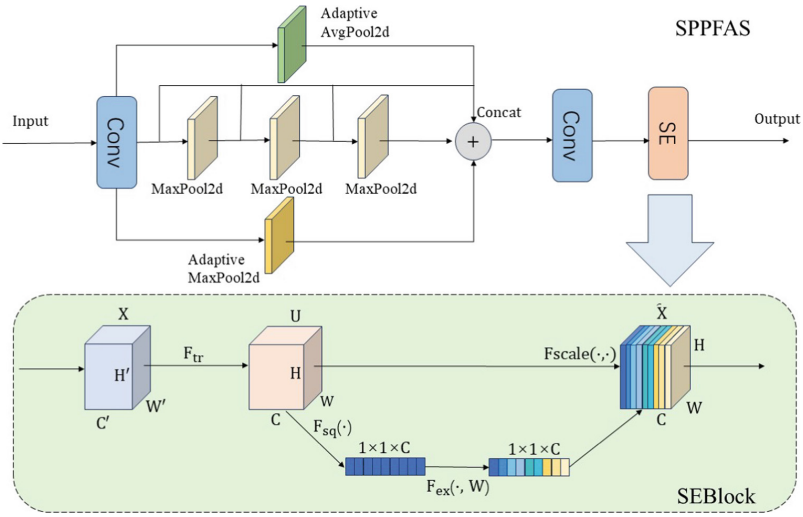


Fig. 4. Structure of the SPPFAS module.

SEBlock. The SE attention mechanism [36] involves two stages: squeeze and excitation. As shown in Fig. 4, where F_{tr} is the convolutional structure, and X and U are the input and output, respectively. The SEBlock learns the correlations between different channels and adjusts the weight of each channel, using the attention mechanism to enhance the information between feature map channels, thereby improving the network's representation. This module will slightly increase the computational load, but better prediction results can be achieved. The following is the derivation process for the SEBlock:

1. The feature map U is subjected to global average pooling to obtain the vector z.

$$z_c = F_{sq}(u_c) = \frac{1}{H \times W} \sum_{i=1}^{H} \sum_{j=1}^{W} u_c(i, j) \tag{1}$$

   where $u_c(i, j)$ represents the value of the c-th channel in the feature map at position (i, j), and z_c is the global descriptor of channel c.
2. Obtain the vector s by learning with two fully connected layers(FC).

$$s = F_{ex}(z, \omega) = \sigma(\omega_2 \delta(\omega_1 z)) \tag{2}$$

where δ and σ are the relu and sigmoid activation functions, respectively, and ω_1 and ω_2 are the weights of the FC.

3. The generated weight vector s assigns weights to the original feature map U, and the final output is a feature map with channel attention $\widetilde{X}$.

$$\widetilde{X} = F_{scale}(u_c, s_c) = s_c u_c \tag{3}$$

SPPFAS. As illustrated in Fig. 4, the SPPFAS module comprises a max-pooling layer, a global max pooling layer, and a global average pooling layer, as well as a SEBlock. This feature map is fed into three consecutive max-pooling layers, producing three distinct feature maps. Additionally, it undergoes global average pooling and global max pooling, producing two global feature maps that lessen noise impact and enhance the model's robustness in complex contexts. Then, concatenate these six generated feature maps to produce richer feature maps. Finally, the fused feature map is generated by applying the Conv module. Then, the SEBlock is used for channel attention weighting to emphasize important features while suppressing less relevant ones, and ultimately, the feature map is output. The whole calculation process can be expressed by equations (4)-(6).

$$X_0 = Conv(X_{in}) \tag{4}$$

$$y_{max} = Concat(Max(X_0), Max(Max(X_0)), Max(Max(Max(X_0)))) \tag{5}$$

$$F_{out} = SEBlock(Conv(Concat(X_0, y_{max}, Am(X_0), Aa(X_0)))) \tag{6}$$

In Eq. X_{in} denotes the input features, Conv is the convolution operation, Max denotes the max-pooling operation, Am represents the global max pooling operation, Aa represents the global average pooling operation, Concat is the feature splicing operation, y_{max} represents the maximally pooled branch' output features and F_{out} represents the final output.

2.4 Loss Function

In the SAR ship detecting field, the accuracy of target localization is particularly important. YOLOv8 adopts the component Intersection over union (CIoU) [37] loss function. CIoU incorporates considerations of shape features in the process of bounding box regression. By incorporating the aspect ratio of both the predicted and ground truth bounding boxes, it can effectively optimize the prediction of the bounding box. Nevertheless, the computational complexity of CIoU is larger, which leads to a higher computational burden while training the model. Furthermore, SAR images include many low-quality images, and geometric metrics like aspect ratio and distance exacerbate the penalties for

these images, causing a reduction in the performance of the model's generalization. When the target frames and the anchor overlap well, the optimal loss function should reduce the geometric penalty, avoid excessive interference with results of training, and increase the model's generalization capacity. Therefore, The dynamic non-monotonic focusing mechanism employed in this study is the WIoUv3 loss function [38], which is based on attention-driven bounding box loss (Wise-IoU v1). It employs a non-monotonic focusing coefficient to adjust the gradient gain dynamically based on the anchor boxes' outlier degree, optimizing model training. This enables the model to provide different levels of attention to various types of anchor boxes at different stages of training, thus retaining more high-quality anchor box information in the early stages, and the negative consequences of low-quality anchor boxes are lessened in the later stages.

The calculation of Wise-IoU v1 is given by Eqs. (7)–(8).

$$L_{WIoU_v1} = R_{WIoU} \cdot \ L_{IoU} \tag{7}$$

$$R_{WIoU} = exp(\frac{(x - x_{gt})^2 + (y - y_{gt})^2}{(W_g{}^2 + H_g{}^2)^*}) \tag{8}$$

where R_{WIoU} is the distance attention factor, denoting the normalized distance between the centroids of the ground truth and the predicted bounding boxes. L_{IoU} denotes the loss of IoU. (x, y) and (x_{gt}, y_{gt}) represents the coordinates of the predicted and ground truth boxes' center points, respectively. W_g and H_g denote the minimum bounding box's width and height, respectively. * separation from the computation graph.

WIoUv3 is calculated based on WIoUv1 and represented by equations (9)-(10).

$$\beta = \frac{L^*_{IoU}}{L_{IoU}} \in [0, +\infty) \tag{9}$$

$$L_{WIoU_v3} = kL_{WIoU_v1}, \ k = \frac{\beta}{\delta\alpha^{\beta-\delta}} \tag{10}$$

where β denotes the outlier degree, which is utilized to explain the quality of the anchor boxes, L^*_{IoU} denotes the gradient gain of the IoU loss, and k is the non-monotonic focusing coefficient, denoted by the hyperparameters α,δ (in this study, α,δ were set to 1.9 and 3, respectively).

3 Results

3.1 Experimental Environment

The experimental platform, along with the parameter settings for the experiments, are detailed in Table 1 and Table 2.

Table 1. Experimental Platform. Table captions should be placed above the tables.

Item	Parameter
Operating System	Ubuntu 20.04
Language	Python 3.9
CPU	Intel(R) Core(TM) i5-12500H
GPU	NVIDIA GeForce RTX 4090
Framework	Pytorch 2.2.2+cuda 12.1

Table 2. Experimental parameter settings.

Parameter	Value
Batch Size	32
Epochs	200
Momentum	0.937
Initial Learning Rate (lr0)	0.01
Final Learning Rate (lrf)	0.01
Weight Decay	0.0005
Image Size (imgsz)	640
Optimizer	SGD

3.2 Evaluation Metrics

To assess the performance of the GMS-YOLO detection model, the recall, mAP0.5, mAP0.5:0.95, the parameters, and FLOPs were used as evaluation metrics in this study. The formula for each index is as follows.

$$P = \frac{TP}{TP + FP} \tag{11}$$

$$R = \frac{TP}{TP + FN} \tag{12}$$

$$mAP = \frac{1}{N} \sum_{i=1}^{n} \int_{0}^{1} Precision(Recall) d(Recall) \tag{13}$$

where FP, FN, and TP are the number of false positive, false negative, and true positive samples, respectively, and N represents the total number of categories. mAP0.5 is the mean average precision of the IoU at a threshold of 0.5, while mAP0.5:0.95 indicates the mean average precision of the IoU across thresholds ranging from 0.5 to 0.95. (with a value interval of 0.5).

3.3 Dataset

The SSDD [39] and HRSID [40] datasets were used to train and test the ship detection model. SSDD contains 1160 images and 2456 ship target instances, and

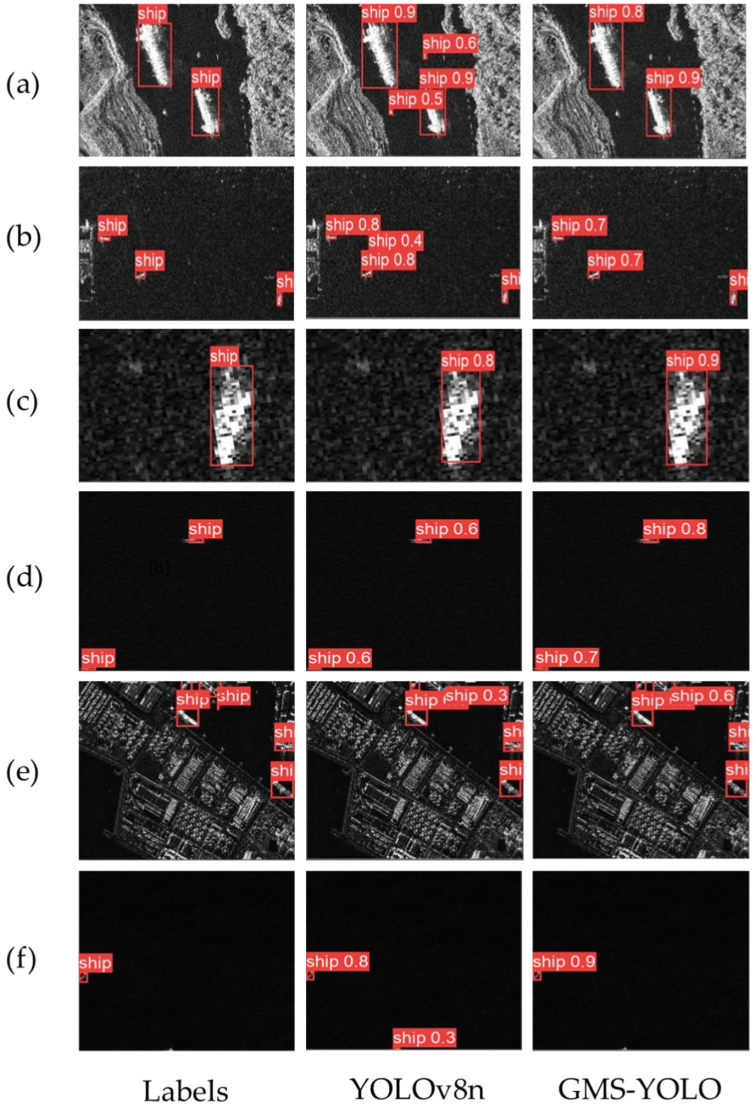


Fig. 5. Visualization results on SSDD and HRSID. (**a**)(**b**)(**c**) are the label visualization results, YOLOv8n detection results, and GMS-YOLO detection results for SSDD, while (**d**)(**e**)(**f**) are the label visualization results, YOLOv8n detection results, and GMS-YOLO detection results for HRSID.

the pixel size of the images is 500*500. HRSID contains 5604 images and 16951 ship target instances, and the pixel size of the images is 800*800. In the training phase, the SSDD and HRSID datasets were split into training, validation, and test sets with an 8:1:1 ratio in this study.

3.4 Ablation Experiments

To validate the efficacy of the model enhancements proposed in this study, ablation experiments were performed on the SSDD and HRSID datasets to evaluate how changes to various modules influence detection performance. The results are illustrated in Table 3.

Table 3. Ablation experiments

GhostConv	C2f-M	SPPFAS	WIoU	SSDD			HRSID			Params/M	FLOPs /G
				R/%	mAP50/%	mAP95/%	R/%	mAP50/%	mAP95/%		
				94.8	97.5	66.7	81.9	91.5	66.5	3.0	8.1
✓				94.2	97.5	65.6	83.2	91.3	65.4	2.8	7.7
✓	✓			94.4	97.8	67.0	80.6	91.4	67.2	2.8	7.7
✓	✓	✓		95.5	97.7	67.2	83.1	92.0	67.3	2.8	7.8
✓	✓	✓	✓	**97.2**	**98.2**	**68.0**	**83.3**	**92.5**	**67.6**	2.8	7.8

The table shows that applying the GhostConv module to the YOLOv8n model reduces the parameters and computation for both datasets, albeit with slightly degraded performance. This is because GhostConv achieves these reductions by generating multiple smaller feature maps, which, however, weakens the model's capability to detect objects. Overall, the introduction of the GhostConv module allows the model to maintain a high level of performance while simultaneously reducing parameters and computation. With the introduction of C2f-M on top of this, while maintaining lightweight, the recall rate on the SSDD dataset improved by 0.2%, mAP50 increased by 0.3%, and mAP50-95 improved by 1.4%. On the HRSID dataset, although the recall rate decreased slightly, mAP50 increased by 0.1%, and mAP50-95 improved by 1.8%, which indicates that the C2f-M module, by improving the ability to capture local and global feature information and improving feature representation, can focus more effectively on critical features and improve the performance of detection. The addition of the SPPFAS module enhanced the overall performance on both datasets. Finally, the algorithm's performance was significantly enhanced by incorporating the WIoU optimization loss function. Compared to the YOLOv8n model, GMS-YOLO showed a reduction of 0.2G in parameters and 0.4G in FLOPs. On the SSDD dataset, recall increased by 2.4%, mAP50 by 0.7%, and mAP50-95 by 1.3%. On the HRSID dataset, recall increased by 1.4%, mAP50 by 1%, and mAP50-95 by 1.1%. These experimental results fully demonstrate the effectiveness of the lightweight optimization of the GMS-YOLO, as well as the model's good generalization ability across different datasets.

Figure 5 shows the visualization of detection results on SSDD and HRSID. It can be observed that in the complex nearshore scene, YOLOv8n incorrectly identified some targets as ship targets and failed to detect one of the small ships, while GMS-YOLO accurately detected these ship targets without any missed detections. For detecting small targets in low-quality, high-noise scenarios, and

simple scenarios, both YOLOv8 and GMS-YOLO performed well, but GMS-YOLO had higher confidence. This indicates that GMS-YOLO has excellent performance in ship target detection, outperforming YOLOv8 on both datasets, proving its effectiveness and generalization in SAR ship detection.

3.5 The Impact of IoU on Algorithm Performance

Using the YOLOv8 algorithm as the baseline model, replaced the backbone network Conv with GhostConv, the neck network SPPF with SPPFAS, and C2f with the C2f-M module to assess the effects of various IoUs on the model's performance. Table 4 shows the performance of different IoU metrics (CIoU, DIoU [37], EIoU [41], GIoU [42], SIoU [43], and WIoU) on the SSDD and HRSID datasets. The experimental results indicate that WIoU performs better than all other IoUs. On the SSDD dataset, R increased by 1.7%, mAP50 by 0.5%, and mAP50-95 by 0.8%. On the HRSID dataset, R rose by 0.2%, mAP50 by 0.5%, and mAP50-95 by 0.3%.

Table 4. Effect of different IoU loss functions on the GMS-YOLO model. All experiments use the same model configuration (GhostConv+C2f-M+SPPFAS) with 2.8M parameters and 7.8G FLOPs.

Loss Function	SSDD			HRSID		
	R/%	mAP50/%	mAP95/%	R/%	mAP50/%	mAP95/%
CIoU	95.5	97.7	67.2	83.1	92.0	67.3
DIoU	91.9	97.8	67.2	82.5	91.5	67.0
EIoU	94.7	97.7	67.2	81.1	90.9	66.5
GIoU	95.9	97.7	67.8	82.3	91.3	65.9
SIoU	95.1	97.6	67.3	83.1	92.0	67.2
WIoU	**97.2**	**98.2**	**68.0**	**83.3**	**92.5**	**67.6**

3.6 Comparative Experiments of Different Algorithms

To verify the effectiveness of the proposed model, we designed comparative experiments between the proposed GMS-YOLO and the lightest algorithms in the YOLO series, including YOLOv3-tiny, YOLOv5n, YOLOv6n, YOLOv7-tiny, and YOLOv8n. The comparison experiment results of the various approaches on the SSDD and HRSID datasets are provided in Table 5 and Table 6, respectively. It is evident that, in contrast to alternative algorithms, GMS-YOLO model demonstrates optimal detection performance on both datasets, with parameters and FLOPs being nearly the lowest. On the SSDD dataset, compared to YOLOv5n with the lowest parameters and FLOPs, the parameters and FLOPs of this model only increase by 0.3M and 0.7G, respectively. However, on the

Table 5. Comparison of different methods on the SSDD dataset.

Model	R/%	mAP50/%	mAP95/%	Params/M	FLOPs/G
YOLOv3-tiny	90.2	96.9	65.8	12.0	18.9
YOLOv5n	93.9	97.0	64.1	2.5	7.1
YOLOv6n	94.1	96.9	65.7	4.2	11.8
YOLOv7-tiny	93.9	97.5	62.7	6.0	13.0
YOLOv8n	94.8	97.5	66.7	3.0	8.1
GMS-YOLO (Ours)	**97.2**	**98.2**	**68.0**	**2.8**	**7.8**

Table 6. Comparison of different methods on the HRSID dataset.

Model	R/%	mAP50/%	mAP95/%	Params/M	FLOPs/G
YOLOv3-tiny	71.8	83.6	60.0	12.0	18.9
YOLOv5n	81.1	90.1	63.0	2.5	7.1
YOLOv6n	79.7	88.5	61.1	4.2	11.8
YOLOv7-tiny	78.7	87.1	57.7	6.0	13.0
YOLOv8n	81.9	91.5	66.5	3.0	8.1
GMS-YOLO (Ours)	**83.3**	**92.5**	**67.6**	**2.8**	**7.8**

SSDD dataset, R, mAP50, and mAP50-95 improved by 3.3%, 1.2%, and 3.9%, respectively. On the HRSID dataset, R, mAP50, and mAP50-95 improved by 2.2%, 2.4%, and 4.6%, respectively. The above experimental results indicate that GMS-YOLO model demonstrates good detection performance while ensuring lower parameters and computation.

4 Conclusions

This study proposes GMS-YOLO, a lightweight model for SAR ship detection. Firstly, GhostConv introduced in this study successfully reduced the model parameters and computation. Secondly, the designed C2f-M effectively improves the feature representation. In addition, the SPPFAS module improved the detection capability for targets of different scales. Finally, the WIoU effectively mitigated the adverse effects of low-quality instances on training. This study conducted extensive experiments on two SAR datasets, SSDD and HRSID. The proposed model demonstrated outstanding performance and lower computational costs under various scales of ship targets, strong noise, and complex background interference, validating the effectiveness and generalization of the GMS-YOLO method in SAR ship target detection tasks. It not only enables more accurate detection in SAR images but also achieves lightweight implementation, providing an efficient and precise solution for SAR ship target detection.

Acknowledgments. This work is supported by the Key Science and Technology Project of Haikou City (No. 2023-054) and the Key R & D Project of Hainan Province (Grant no. ZDYF2022GXJS348).

Author contributions. Conceptualization, X.Z., J.X.; methodology, X.Z.; software, X.Z.; validation, X.Z., J.X. and X.L.(Xiaozhang Liu); investigation, X.L.(Xiulai Li); resources, X.L.(Xiaozhang Liu); data curation, X.Z.; writing—original draft preparation, X.Z.; writing—review and editing, X.Z., X.L.(Xiaozhang Liu); visualization, Z.X., J,X; supervision, X.L.(Xiaozhang Liu); project administration, X.Z., J.X. and X.L.(Xiulai Li); funding acquisition, X.L.(Xiaozhang Liu). All authors have read and agreed to the published version of the manuscript.

Data Availability Statement. The data that support the findings of this study are openly available in the SSDD Dataset at https://gitcode.com/open-source-toolkit/760a8, and the HRSID Dataset at https://github.com/chaozhong2010/HRSID.

Disclosure of Interests. The authors declare no conflicts of interest.

References

1. Moreira, A., Prats-Iraola, P., Younis, M., Krieger, G., Hajnsek, I., Papathanassiou, K.P.: A tutorial on synthetic aperture radar. IEEE Geoscience Remote Sensing Mag. **1**(1), 6–43 (2013)
2. Tsokas, A., Rysz, M., Pardalos, P.M., Dipple, K.: Sar data applications in earth observation: an overview. Expert Syst. Appl. **205**, 117342 (2022)
3. Amitrano, D., et al.: Earth environmental monitoring using multi-temporal synthetic aperture radar: a critical review of selected applications. Remote Sensing **13**(4), 604 (2021)
4. Stecz, W., Gromada, K.: Uav mission planning with sar application. Sensors **20**(4), 1080 (2020)
5. Sah, S., Haldar, D., Singh, R.N., Das, B., Nain, A.S.: Rice yield prediction through integration of biophysical parameters with sar and optical remote sensing data using machine learning models. Sci. Rep. **14**(1), 21674 (2024)
6. Amitrano, D., Di Martino, G., Di Simone, A., Imperatore, P.: Flood detection with sar: a review of techniques and datasets. Remote Sensing **16**(4), 656 (2024)
7. Li, J., Congan, X., Hang, S., Gao, L., Wang, T.: Deep learning for sar ship detection: Past, present and future. Remote Sensing **14**(11), 2712 (2022)
8. Yasir, M.., et al.: Ship detection based on deep learning using sar imagery: a systematic literature review. Soft Comput.- a Fusion of Foundations, Methodol. Appli. **27**(1) (2023)
9. Tello, M., López-Martínez, C., Mallorqui, J.J.: A novel algorithm for ship detection in sar imagery based on the wavelet transform. IEEE Geosci. Remote Sens. Lett. **2**(2), 201–205 (2005)
10. Leng, X., Ji, K., Yang, K., Zou, H.: A bilateral cfar algorithm for ship detection in sar images. IEEE Geosci. Remote Sens. Lett. **12**(7), 1536–1540 (2015)
11. Lim, H., Chae, D., Yoo, J.H., Kwon, K.-I.: Template matching-based target recognition algorithm development and verification using sar images. J. Korea Instit. Military Sci. Technol. **17**(3), 364–377 (2014)
12. LeCun, Y., Bengio, Y., Hinton, G.: Deep learning. Nature **521**(7553), 436–444 (2015)

13. Girshick, R., Donahue, J., Darrell, T., Malik, J.: Rich feature hierarchies for accurate object detection and semantic segmentation. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 580–587 (2014)
14. Ren, S., He, K., Girshick, R., Sun, J.: Faster r-cnn: towards real-time object detection with region proposal networks. Adv. Neural Inform. Process. Syst. **28** (2015)
15. He, K., Gkioxari, G., Dollár, P., Girshick, R.: Mask r-cnn. In: Proceedings of the IEEE International Conference on Computer Vision, pp. 2961–2969 (2017)
16. Redmon, J., Divvala, S., Girshick, R., Farhadi, A.: You only look once: unified, real-time object detection. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 779–788 (2016)
17. Redmon, J., Farhadi, A.: Yolo9000: better, faster, stronger arxiv preprint. arXiv preprint arXiv, 1612 (2017)
18. Redmon, J., Farhadi, A.: Yolov3: An incremental improvement. arXiv preprint arXiv:1804.02767 (2018)
19. Bochkovskiy, A., Wang, C.-Y., Liao, H.-Y.M.: Yolov4: optimal speed and accuracy of object detection. arXiv preprint arXiv:2004.10934 (2020)
20. Li, C., et al.: Yolov6: A single-stage object detection framework for industrial applications. arXiv preprint arXiv:2209.02976 (2022)
21. Wang, C.-Y., Bochkovskiy, A., Liao, H.-Y.M.: Yolov7: trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 7464–7475 (2023)
22. Liu, W. et al.: SSD: single shot multibox detector. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds.) ECCV 2016. LNCS, vol. 9905. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46448-0_2
23. Jeong, J., Park, H., Kwak, N.: Enhancement of ssd by concatenating feature maps for object detection. arXiv preprint arXiv:1705.09587 (2017)
24. Wang, Z., Lan, D., Mao, J., Liu, B., Yang, D.: Sar target detection based on ssd with data augmentation and transfer learning. IEEE Geosci. Remote Sens. Lett. **16**(1), 150–154 (2018)
25. Sun, Z., Xiangguang Leng, Yu., Lei, B.X., Ji, K., Kuang, G.: Bifa-yolo: a novel yolo-based method for arbitrary-oriented ship detection in high-resolution sar images. Remote Sensing **13**(21), 4209 (2021)
26. Bai, L., Yao, C., Ye, Z., Xue, D., Lin, X., Hui, M.: Feature enhancement pyramid and shallow feature reconstruction network for sar ship detection. IEEE J. Selected Topics Appli. Earth Observat. Remote Sensing **16**, 1042–1056 (2023)
27. Yasir, M., et al.: Multi-scale ship target detection using sar images based on improved yolov5. Front. Mar. Sci. **9**, 1086140 (2023)
28. Tang, G., Zhuge, Y., Claramunt, C., Men, S.: N-yolo: A sar ship detection using noise-classifying and complete-target extraction. Remote Sensing **13**(5), 871 (2021)
29. Guo, Y., Chen, S., Zhan, R., Wang, W., Zhang, J.: Sar ship detection based on yolov5 using cbam and bifpn. In: IGARSS 2022-2022 IEEE International Geoscience and Remote Sensing Symposium, pp. 2147–2150. IEEE (2022)
30. Xiaowo, X., Zhang, X., Zhang, T.: Lite-yolov5: a lightweight deep learning detector for on-board ship detection in large-scene sentinel-1 sar images. Remote Sensing **14**(4), 1018 (2022)
31. Ren, X., Bai, Y., Liu, G., Zhang, P.: Yolo-lite: an efficient lightweight network for sar ship detection. Remote Sensing **15**(15), 3771 (2023)
32. Tang, H., et al.: A lightweight sar image ship detection method based on improved convolution and yolov7. Remote Sensing **16**(3), 486 (2024)

33. Han, K., Wang, Y., Tian, Q., Guo, J., Xu, C., Xu, C.: Ghostnet: more features from cheap operations. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1580–1589 (2020)
34. Wan, D., Rongsheng, L., Shen, S., Ting, X., Lang, X., Ren, Z.: Mixed local channel attention for object detection. Eng. Appl. Artif. Intell. **123**, 106442 (2023)
35. Lin, M., Chen, Q., Yan, S.: Network in network. arXiv preprint arXiv:1312.4400 (2013)
36. Hu, J., Shen, L., Sun, G.: Squeeze-and-excitation networks. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, pp. 7132–7141 (2018)
37. Zheng, Z., Wang, P., Liu, W., Li, J., Ye, R., Ren, D.: Distance-iou loss: faster and better learning for bounding box regression. In: Proceedings of the AAAI Conference on Artificial Intelligence vol. 34, 12993–13000 (2020)
38. Tong, Z., Chen, Y., Xu, Z., Yu, R.: Wise-iou: bounding box regression loss with dynamic focusing mechanism. arXiv preprint arXiv:2301.10051 (2023)
39. Li, J., Qu, C., Shao, J.: Ship detection in sar images based on an improved faster r-cnn. In: 2017 SAR in Big Data Era: Models, Methods and Applications (BIGSARDATA), pp. 1–6. IEEE (2017)
40. Wei, S., Zeng, X., Qizhe, Q., Wang, M., Hao, S., Shi, J.: Hrsid: a high-resolution sar images dataset for ship detection and instance segmentation. IEEE Access **8**, 120234–120254 (2020)
41. Zhang, Y.-F., Ren, W., Zhang, Z., Jia, Z., Wang, L., Tan, T.: Focal and efficient iou loss for accurate bounding box regression. Neurocomputing **506**, 146–157 (2022)
42. Rezatofighi, H., Tsoi, N., Gwak, J., Sadeghian, A., Reid, I., Savarese, S.: Generalized intersection over union: a metric and a loss for bounding box regression. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 658–666 (2019)
43. Gevorgyan, Z.: Siou loss: More powerful learning for bounding box regression. arXiv preprint arXiv:2205.12740 (2022)

ERL-CDR: Evolutionary Reinforcement Learning with Causal Decoupling Representation

Zuohao Wu[1], Guang Li[2(✉)], Zihao li[3], Jiaqi Wei[4], and Mingsheng Shang[2]

[1] Chongqing University of Posts and Telecommunications, Chongqing, China
[2] Chongqing Institute of Green and Intelligent Technology, Chinese Academy of Sciences Chongqing, Chongqing, China
liguang@cigit.ac.cn
[3] School of Computer Science and Technology, University of Chinese Academy of Sciences, Beijing, China
2022441629@cqust.edu.cn
[4] School of Electrical Engineering, Chongqing University of Science and Technology, Chongqing, China

Abstract. Evolutionary Reinforcement learning (ERL) algorithms effectively address the sample efficiency problem in continuous state-action spaces by integrating gradient-based reinforcement learning (RL) with gradient-free evolutionary algorithms (EAs). However, traditional ERL methods still face limitations, including insufficient processing of the state space in collected samples, diversity-limited fitness evaluation based solely on rewards leading to reduced sample diversity, and differences in EA-RL sample distributions that hinder the value function learning. This paper proposes **E**volutionary **R**einforcement **L**earning - **C**ausal **D**ecoupling **R**epresentation (ERL-CDR). which addresses the issue of insufficient state space processing by introducing a causal representation learning mechanism that disentangles task-irrelevant features from decision-critical features. A behavioral diversity mechanism is used to evaluate the population, promoting more comprehensive exploration of the sample space and increasing sample diversity. Finally, a novel value function is introduced to better represent the sample space of policy populations, which, together with the RL value function, effectively guides policy learning. Experimental results show that this method significantly improves convergence speed, stability, and final performance in complex continuous control tasks. These methods are of research significance for complex continuous reinforcement learning tasks.

Keywords: Causal Representation · Diversity Evaluation · Value Function

1 Introduction

Reinforcement Learning (RL) [1] has achieved significant success in fields like robotics [2], game AI [3], and autonomous driving [4], mainly due to its efficient

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 555–570, 2026.
https://doi.org/10.1007/978-981-95-8405-5_30

policy learning with deep neural networks. However, RL still suffers from instability, limited exploration, and difficulties in environments with sparse or noisy gradient signals.

Evolutionary Algorithms (EAs) [5] are black-box optimization methods that maintain a population of solutions and optimize them through selection, inheritance, and mutation, without relying on gradient information. EAs offer better exploration, robustness, and stable convergence [6], but suffer from low sample efficiency due to the need for extensive population evaluations, especially as the policy space grows [7].

EAs with RL leverage the complementary strengths of both approaches for more efficient policy optimization. Recent research has mainly focused on the integration of EAs with RL, such as Evolutionary Reinforcement Learning (ERL) [8], which combines Genetic Algorithms (GA) [9] with a Deep Deterministic Policy Gradient (DDPG) [10]. In ERL, a population of agents interacts with an RL agent, allowing both components to benefit from each other. Improvements include using the Cross-Entropy Method (CEM) [11], gradient-based genetic operators, and multiple RL agents [12]. However, most methods focus on collecting diverse policy samples, often overlooking the ability of the value function to learn complex state representations, which limits the performance of evolutionary reinforcement learning.

Inspired by causal reinforcement learning [13], this paper introduces causal representation learning to unravel environment dynamics, task-irrelevant features, and decision-critical features. The proposed ERL-CDR algorithm improves on the original ERL framework by designing a decoupled representation network that extracts key task features from complex environmental information. In addition, a world model is incorporated to guide the training of the decoupled representation network. This enables the ERL algorithm to better isolate critical information relevant to the task, thereby enhancing the generalization and sample efficiency of the learned representations, and effectively addressing the problem of feature redundancy in high-dimensional and complex tasks.

The main contributions of our work are as follows.

(1) A novel ERL algorithm is proposed, in which a causally disentangled representation network is introduced to enable agents to better perceive the state space and decouple task-critical features.

(2) A new population fitness metric for the policy population is designed, incorporating a diversity mechanism to guide the optimization of the diversity of the policy population.

(3) By employing different value functions, the method is able to effectively learn distributional differences within the sample space.

Finally, we conduct a comprehensive evaluation of ERL-CDR against strong baseline ERL methods and representative RL algorithms on MuJoCo continuous control tasks and the DeepMind Control Suite, including systematic studies such as ablation experiments and hyperparameter analysis.

2 Background

Reinforcement Learning (RL) is formulated as a Markov Decision Process (MDP), represented by the tuple $\langle S, A, P, R, \gamma, T \rangle$. At each time step (t), the agent selects an action $a_t \sim \pi(s_t)$ according to its policy π based on the current state $s_t \in S$. The environment then transitions to the next state s_{t+1} according to the transition function $P(s_t, a_t)$, and the agent receives a reward $r(s_t, a_t)$. The return, defined as the cumulative reward, is the discounted sum of future rewards, expressed as $R_t = \sum_{i=t}^{T} \gamma^{i-t} r_i$, where $\gamma \in [0, 1)$ is the discount factor and T is the maximum episode length. The objective of RL is to learn an optimal policy π^* that maximizes the expected return. DDPG [10] is a well-known off-policy actor-critic algorithm. It consists of a deterministic policy (the actor) and an approximate state-action value function (the critic), parameterized by ω and ψ, respectively. The critic is optimized using temporal difference (TD) learning [14], while the actor is updated by maximizing the estimated value (Q). The loss functions are formulated as: $L(\psi) = \mathbb{E}_D \left[\left(r + \gamma Q'_\psi(s', \pi'_\omega(s')) - Q_\psi(s, a) \right)^2 \right]$ and $L(\omega) = -\mathbb{E}_D \left[Q_\psi(s, \pi_\omega(s)) \right]$ are sampled from the replay buffer D, and ψ' and ω' denote the parameters of the target networks. TD3 [15] further improves DDPG by employing clipped double Q-learning to mitigate overestimation bias.

Evolutionary Algorithms (EAs) [16] are a class of black-box optimization methods that iteratively evolve a population of policies. The population is indicated as $P = \pi_1, \pi_2, \ldots, \pi_n$, and policies evolve in each iteration. During each iteration, all agents interact with the environment to obtain Monte Carlo MC estimates of policy fitness, $f(\pi_1), f(\pi_2), \ldots, f(\pi_n)$, where fitness is defined as: $[f(\pi) = \frac{1}{e} \sum_{i=1}^{e} \left[\sum_{t=0}^{T} r_t \mid \pi \right]]$ Here, r_t is the reward at time step t and T is the duration of the episode. Policies with higher fitness scores are more likely to be selected as parents for the next generation. Various selection mechanisms exist, such as Genetic Algorithms (GA) [9] and the Cross-Entropy Method (CEM) [17]. The offspring are generated using genetic operators: two parents π_i and π_j are randomly selected, and the offspring π'_i and π'_j are produced by crossover or mutation in GA. Specifically, the crossover operator combines parts of the parent networks to generate new offspring: $\pi'_i, \pi'_j = \text{Crossover}(\pi_i, \pi_j)$ Alternatively, the mutation operator generates a child policy by applying small random perturbations to the parent policy: $[\pi'_i = \text{Mutation}(\pi_i)]$ In most existing methods, crossover and mutation are performed at the parameter level. Due to the diversity introduced by a large number of candidate solutions and the continuous variation from genetic operations, evolutionary algorithms exhibit strong exploration capabilities.

3 Related Work

EAs, with their gradient-free nature and diversity, provide valuable samples for RL in complex environments but suffer from low sample efficiency. RL, optimized

via neural networks and gradient descent, often gets stuck in local optima, making high-reward policy discovery difficult in continuous control tasks. Integrating EAs and RL has emerged as a promising direction, enabling efficient policy optimization [8,18], improving action selection, assisting in hyperparameter tuning [19,20], and enhancing EA performance in multimodal problems.

The core contribution of this study lies in the integration of EAs and RL to achieve more efficient policy optimization. ERL [8] was the first to propose a hybrid framework in which an RL agent and an evolutionary population are trained simultaneously. The RL agent can explore a wider variety of samples through the EAs population, and copies of the RL agent are periodically injected into the EAs population. Furthermore, CEM-RL [11] combines the Cross-Entropy Method (CEM) with TD3 for optimization. CERL [7] extends a single RL agent to multiple RL agents with different hyperparameters to address the issue of hyperparameter sensitivity in RL. PDERL [21] introduces Q-filtering, distillation crossover, and proximal mutation to mitigate policy collapse caused by traditional parameter-level genetic operations.

The motivation for adopting the causal decoupling representation network stems from the need to explicitly separate different causal factors in complex reinforcement learning environments. In such environments, the agent must distinguish between features that drive environment dynamics, those that are decision-critical, and those that are irrelevant to the task. By designing a multi-branch structure, our approach enables each branch to focus on a specific causal factor, which aligns with the principles of causal inference–such as intervention and counterfactual reasoning–proposed by Pearl (2009) [22]. This explicit separation allows the agent to better generalize and adapt when the environment or task changes, as each causal factor can be independently manipulated or analyzed.

Unlike unsupervised disentanglement methods such as β-VAE [23] and FactorVAE [24], which encourage independence among latent variables without considering their causal roles, our approach is tailored for reinforcement learning. It ensures that the learned representations correspond to distinct causal mechanisms relevant to decision making and environment dynamics. The independence loss further enforces this separation, making the branches as independent as possible and thus more interpretable from a causal perspective.

In summary, our design is guided by the intuition that effective reinforcement learning requires not only disentangled representations, but also representations that reflect the underlying causal structure of the environment. This facilitates robust policy learning and transfer across tasks with varying causal factors.

4 The Algorithm Framework Of ERL-CDR

In this section, we present the overall framework of ERL-CDR, explain how the ERL-CDR model is derived from the general framework, and detail the specific implementation of the three key components.

4.1 Representation Learning of Causal Decoupling

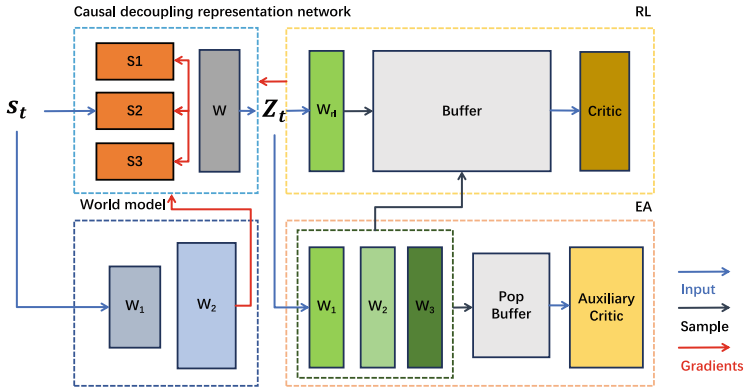


Fig. 1. The ERL-CDR framework is shown in this figure. The environment state s_t is input to a shared representation network with three branches S_1, S_2, S_3, extracting causally disentangled features for environment dynamics, task-irrelevant information, and decision-critical factors. These features are concatenated as the shared representation, which serves as input for both the policy and value networks in RL, and provides a unified feature basis for multiple EA policy individuals (w_1, w_2, w_3), enabling synergy between EA and RL. The world model acts as an auxiliary task, optimizing the shared representation network via environment state transition prediction to improve modeling and generalization of dynamics.

Representation Learning of Causal Decoupling. Let the raw state of the environment at time t be s_t. We design a multi-branch shared representation network Z_ϕ that maps s_t to causally disentangled multimodal features (Fig. 1):

$$z_t = Z_\phi(s_t) = \text{concat}(z_t^{\text{dyn}},\ z_t^{\text{irr}},\ z_t^{\text{dec}}) \tag{1}$$

where z_t^{dyn}, z_t^{irr}, and z_t^{dec} denote the environment dynamics features, task-irrelevant features, and decision-critical features, respectively. The final shared representation z_t is obtained by concatenating these features.

Orthogonality-Independence Loss. To achieve disentanglement among the branch features, we employ an orthogonality loss to constrain the correlation between outputs of different branches:

$$\mathcal{L}_{\text{indep}} = \sum_{i \neq j} \left| \text{Cov}(z_t^{(i)},\ z_t^{(j)}) \right| \tag{2}$$

where $\text{Cov}(z_t^{(i)},\ z_t^{(j)})$ denotes the covariance between the i-th and j-th branch features. The objective is to minimize the correlation among branches, making them as independent as possible.

World Model Auxiliary Loss. A world model f_ψ is introduced, which takes the shared representation z_t as input to predict the next state $\hat{s}_{t+1}$. The mean squared error (MSE) is used as the auxiliary loss:

$$\mathcal{L}_{\text{wm}} = \mathbb{E}\left[\left|f_\psi(z_t) - s_{t+1}\right|^2\right] \tag{3}$$

This loss enforces the shared representation network to learn features that can accurately model the environment dynamics.

Total Loss Function. Finally, the training objective of the representation network is a weighted sum of multiple loss terms:

$$\mathcal{L}_{\text{causal}} = \alpha\mathcal{L}_{\text{policy}} + \beta\mathcal{L}_{\text{indep}} + \gamma\mathcal{L}_{\text{wm}} \tag{4}$$

where $\mathcal{L}_{\text{policy}}$ is the policy loss, and α, β, and γ are the weighting coefficients for each loss term.

4.2 Diversity-Driven Policy Population Fitness Metric

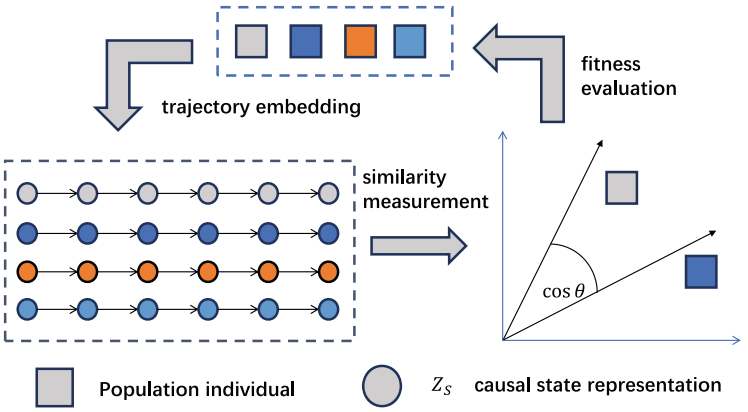


Fig. 2. The diversity evaluation uses low-dimensional trajectory embeddings to compute similarity, which is combined with reward-based fitness to promote more diverse population evolution.

Trajectory Embedding Generation. Let the state sequence of individual i within an episode be $\{s_1, a_1, s_2, \ldots, s_n\}$. We use the state embedding network Z_ϕ that maps the state s_t at each time step to a low-dimensional feature z_t:

$$z_t = Z_\phi(s_t) \tag{5}$$

The trajectory embedding h_i for individual i is obtained by applying average pooling over all state embeddings z_t along the trajectory (Fig. 2):

$$h_i = \frac{1}{T} \sum_{t=0}^{T} z_t \tag{6}$$

where h_i serves as the behavioral representation of individual i in the current episode.

Diversity Metric and Reward. Collect the trajectory embeddings h_i for all individuals in the current population. The behavioral similarity between individuals is measured using cosine similarity. For each individual i, its diversity score is defined as the reciprocal of the average similarity to other individuals' trajectory embeddings:

$$\text{diversity}_i = \frac{1}{\sum_{j \neq i} \cos(h_i, h_j) + \epsilon} \tag{7}$$

where $\cos(h_i, h_j)$ denotes the cosine similarity between h_i and h_j, and ϵ is a small constant to prevent division by zero.

Diversity Reward Integration. To ensure consistency in scale between the main task reward and the diversity reward, both are normalized

$$\text{norm_reward}_i = \frac{r_i - \min\limits_j(r_j)}{\max\limits_j(r_j) - \min\limits_j(r_j) + \epsilon} \tag{8}$$

$$\text{norm_diversity}_i = \frac{\text{diversity}_i - \min\limits_j(\text{diversity}_j)}{\max\limits_j(\text{diversity}_j) - \min\limits_j(\text{diversity}_j) + \epsilon} \tag{9}$$

The final fitness is computed as the weighted sum of the two normalized terms, where α and β are weighting coefficients:

$$f(w_i) = \text{fitness}_i = \alpha \cdot \text{norm_reward}_i + \beta \cdot \text{norm_diversity}_i \tag{10}$$

4.3 Population Experience-Driven Auxiliary Critic for Actor Guidance

Population Experience-Driven Auxiliary Critic. Let each individual i in the current population collect a trajectory in the environment, denoted as $\tau_i = (s_t^i, a_t^i, r_t^i, s_{t+1}^i, \ldots, s_n^i, a_n^i, r_n^i)$. We introduce an auxiliary critic network $Q\psi$, whose parameters are independent from the main critic. This network is trained using the sampled experiences from all individuals in the population, aiming to comprehensively model the value distribution of population behaviors (Fig. 3):

$$Q_\psi(s, a) \approx \mathbb{E}[R \mid (s_{pop}, a_{pop})] \tag{11}$$

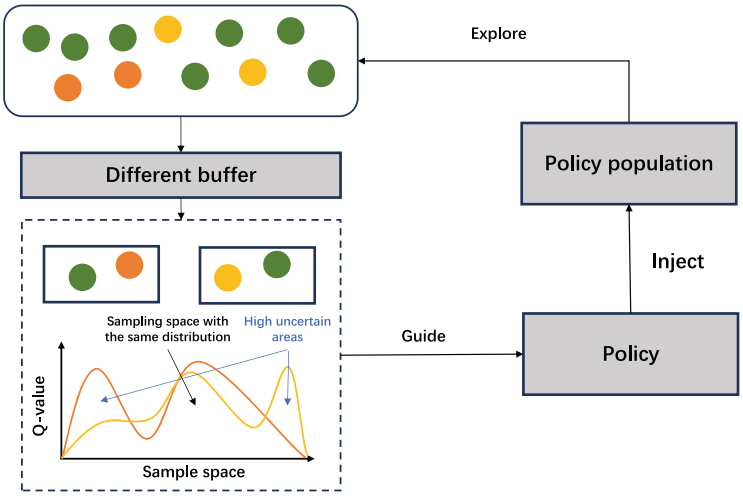


Fig. 3. This figure shows the auxiliary critic, which learns from the population's sampling distribution and, together with the main critic, guides the actor toward high-reward regions.

Multi-Source Sampling and Generalization Enhancement. The training samples for the auxiliary critic are drawn from collected episodes of the entire population, thus covering a diverse range of policy distributions. Compared to single-policy sampling, this multi-source approach significantly enhances the critic's generalization over the state-action space and reduces the risk of overfitting.Formally, the population experience buffer $\mathcal{B}_{\text{pop}}$ is constructed as:

$$\mathcal{B}_{\text{pop}} = \{(s, a, r, s') \mid (s, a, r, s') \in \mathcal{B}_i,\ i = 1, 2, \ldots, N\} \tag{12}$$

where N is the population size, and $\mathcal{B}i$ denotes the experience buffer of individual i. The auxiliary critic is trained on samples $(s, a, r, s') \sim \mathcal{B}$pop, ensuring coverage of diverse state-action pairs.

Collaborative Policy Optimization. During actor training, the loss function is designed to jointly maximize the outputs of both the environment reward critic and the auxiliary critic Q_ψ. Specifically, the actor's objective is not only to improve the output of the environment reward critic, but also to increase the value estimated by the auxiliary critic Q_ψ for its actions. This collaborative guidance enables the actor's parameter updates to be informed by both environment rewards and the auxiliary critic, leading to higher-quality policy optimization.

The actor loss can be formulated as:

$$\mathcal{L}_{\text{actor}} = -\mathbb{E}s \sim \mathcal{D}\left[Q_\phi(s, \mu_\theta(s)) + Q_\psi(s, \mu_\theta(s))\right] \tag{13}$$

where Q_ϕ is the main critic, Q_ψ is the auxiliary critic, μ_θ is the actor policy, $\mathcal{D}$ is the experience buffer.

Training and Update Mechanism The auxiliary critic is trained using the same temporal-difference (TD) loss as the main critic:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_{(s,a,r,s')\in\text{PopBuffer}}\left[\left(Q_\psi(s,a) - (r + \gamma Q_\psi(s',a'))\right)^2\right] \tag{14}$$

where (s, a, r, s') are sampled from the population experience buffer, and a' denotes the next action. The auxiliary critic can be periodically reset and retrained with the latest population experiences to maintain adaptability to the current population distribution.

In practice, the auxiliary critic network is periodically trained in batches using population-sampled data. After training, it is used to evaluate actor policies. The actor's loss function combines the outputs of both the environment reward critic and the auxiliary critic, and parameters are updated to jointly maximize both.

5 Experiments

To comprehensively evaluate the effectiveness and generalizability of our proposed causal decoupling representation network, we selected the MuJoCo and DeepMind Control (DMC) environments as benchmark tasks. The choice of these environments is motivated by several key considerations:

First, both MuJoCo and DMC are widely recognized in the reinforcement learning community for their diversity and complexity. MuJoCo provides a suite of continuous control tasks, such as locomotion and manipulation, which feature high-dimensional state spaces and complex physical dynamics. These characteristics present significant challenges for representation learning, as the agent must extract meaningful and disentangled features from rich sensory inputs to achieve robust control.

Second, the DMC suite offers a variety of tasks with different levels of reward sparsity and dynamic complexity. Some tasks, such as "walker-walk" or "eetah-run,"nvolve sparse or delayed rewards, which require the agent to perform efficient exploration and causal reasoning to identify the key factors that influence long-term success. Other tasks present dense rewards but involve intricate interactions between multiple state variables, further testing the model's ability to disentangle and utilize relevant causal features.

Third, by including both MuJoCo and DMC environments, we ensure that our method is validated across a broad spectrum of reinforcement learning scenarios. This allows us to demonstrate that the proposed approach is not limited to a specific type of task or reward structure, but is generally applicable to environments with high-dimensional observations, complex dynamics, and varying reward signals.

In summary, the selection of MuJoCo and DMC environments is intended to highlight the strengths of our method in handling high-dimensional state spaces, sparse and dense rewards, and complex causal relationships. This comprehensive evaluation underscores the versatility and practical relevance of our approach for real-world reinforcement learning applications.

5.1 Experimental Setups

We evaluated two categories of baseline methods: (1) standard reinforcement learning algorithms (PPO [25], SAC [26], DDPG [10]); and (2) ERL methods (ERL [8], CERL [7], PDERL [21]), all using DDPG as the RL backbone. Performance comparisons were conducted on Mujoco tasks with identical initialization and tuned population sizes. Key module weights and diversity fitness weights were set for different environments (for example, in the Ant environment, the loss function weights were $\alpha = 0.5, \beta = 0.25, \gamma = 0.25$ and the diversity fitness weights were $\alpha = 0.5, \beta = 0.5$). Hyperparameters for all algorithms were similarly optimized. We also evaluated performance in the DMC environment by integrating RAD-based data augmentation with ERL variants.

5.2 Hyperparameter Selection

This section provides a detailed description of the hyperparameter settings for different tasks. Some hyperparameters remain consistent across all tasks. The population size is set to 5, and the mutation operator's α is set to 1.0. The number of environment interaction episodes for surrogate fitness is 1. The value of γ is 0.99. Since ERL-CDR is built upon ERL, other settings remain unchanged. Table 2 and Table 3 lists other specific hyperparameters of ERL-CDR that vary across different tasks.

Table 1. Hyperparameter settings for MUJOCO environments.

Env name	z_α	z_β	z_γ	f_α	f_β
HalfCheetach	0.7	0.2	0.1	0.6	0.4
Walker	0.8	0.1	0.1	0.6	0.4
Swimmer	0.7	0.2	0.1	0.5	0.5
Hopper	0.8	0.1	0.1	0.7	0.3
Ant	0.5	0.25	0.25	0.5	0.5
Humanoid	0.5	0.3	0.2	0.6	0.4

Table 2. Hyperparameter settings for DMC environments.

Env name	z_α	z_β	z_γ	f_α	f_β
Finger Spin	0.4	0.4	0.2	0.8	0.2
Cup Catch	0.6	0.2	0.2	0.7	0.3
Walker Walk	0.8	0.1	0.1	0.7	0.3
Cheetah Run	0.7	0.1	0.2	0.6	0.4

5.3 Performance Evaluation

During the MUJOCO experiments, we compared the DDPG implementation of ERL-CDR with baseline methods. As shown in Fig. 4, ERL-CDR outperforms other ERL methods and traditional reinforcement learning algorithms on most MuJoCo continuous control tasks. For example, in the Swimmer environment, ERL-CDR converges to the optimal value within 600,000 steps, while other methods typically require 800,000 steps. In the Humanoid environment, the final reward increases from around 1,200 to about 3,200. However, in simpler environments with relatively limited state spaces, the performance improvement is less pronounced. This may be because, in simple action spaces, each state dimension is closely related to decision-making, making the benefits of the causal disentanglement representation network less significant. In these cases, improvements mainly result from the design of the diversity fitness function and the auxiliary critic. In environments like DMC, where state inputs are pixel-level, as shown in Table 1, the causal disentanglement network can effectively extract important causal representations to facilitate better policy learning for RL. Overall, these findings indicate that ERL-CDR achieves progress on most benchmark tasks, maintains strong performance, and reduces sample exploration requirements.

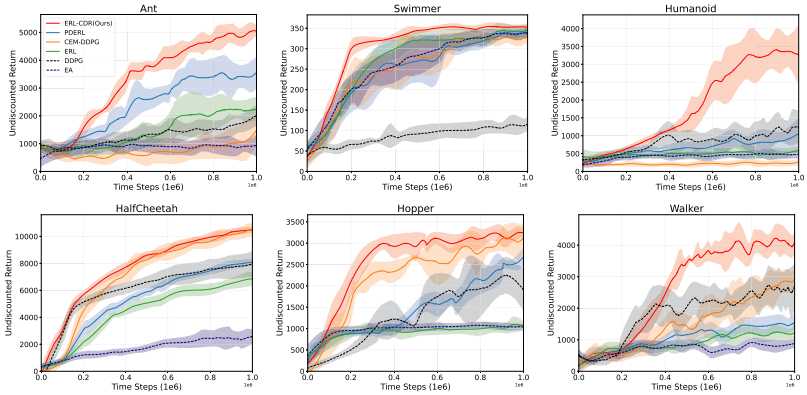


Fig. 4. Performance comparison between ERL-CDR and baselines on MUJOCO.

For the ablation study, we compared the effectiveness of the proposed causal disentanglement representation extraction and the auxiliary critic network. In Fig. 5, we compared the causal disentanglement network with a baseline that uses a standard linear layer without a world model as the feature extraction module. The results show that in complex environments such as Ant, the disentanglement network brings significant performance improvements, with rewards increasing from around 3,500 to about 4,800, demonstrating its advantage. Next, in Fig. 6, we examined whether the auxiliary critic helps achieve faster convergence. In simpler environments, such as Hopper, it enables quicker convergence,

Table 3. Comparisons on Deepmind Control Suite with Image Inputs

Environments	Finger Spin	Cup Catch	Walker Walk	Cheetah Run
ERL-CDR(RAD)	823 ± 27	796 ± 63	517 ± 121	498 ± 22
RAD	792 ± 128	769 ± 149	496 ± 142	463 ± 35
ERL(RAD)	125 ± 141	152± 63	263 ± 64	311 ± 152
PDERL(RAD)	370 ± 43	471 ± 127	223 ± 94	336 ± 41
CEM-RAD	76 ± 63	321 ± 43	52 ± 15	412 ± 26
Steps	100k	100k	100k	100k

reaching optimal performance about 600,000 steps. These improvements vary across experiments, as the final gains result from the direct interaction of different modules, and sample diversity affects algorithm convergence. Therefore, it is essential to ensure effective exploration while enabling the value function to learn efficiently. Finally, Fig. 7 compares the performance of different combinations of modules, demonstrating that causal decoupling networks play a crucial role in state extraction. Their decoupling of states effectively facilitates the critic's learning, thereby enhancing the guidance provided to the population based on diversity fitness. This may be attributed to the superior capability of low-dimensional state features to be learned and explored.

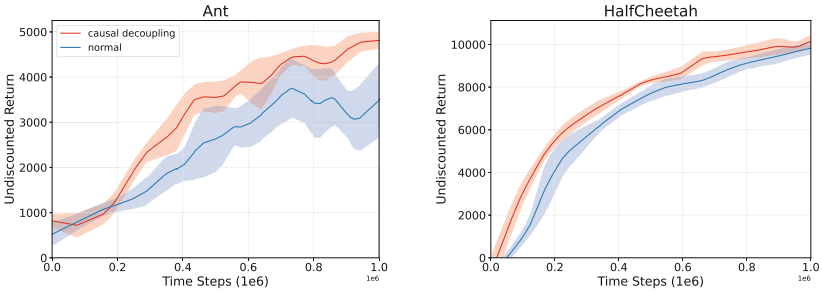


Fig. 5. the representation networks

Regarding the impact of hyperparameters on performance, our method introduces two main categories of new hyperparameters. The first category concerns the weights of the causal representation network loss, denoted as α, β, γ. In this case, we fixed the original critic weight α at 0.6 and varied β across three values: 0.1, 0.2, and 0.3. As shown in Fig. 8, gradually increasing the independent disentanglement loss weight leads to improved performance, possibly because the state network can better separate state features. For example, in the Swimmer environment, the score increased from around 330 to 350. The second category involves the weights of the diversity fitness function, α and β, which correspond

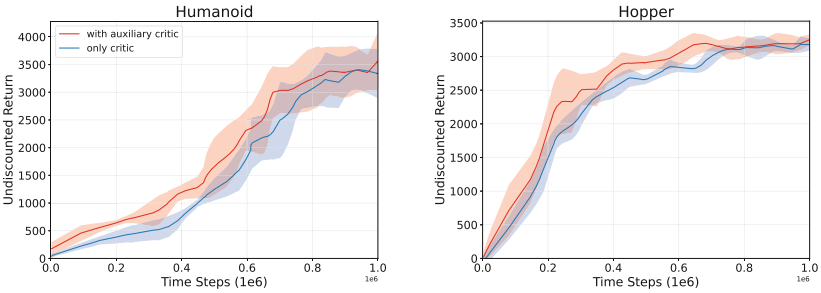


Fig. 6. whether to use auxiliary critic

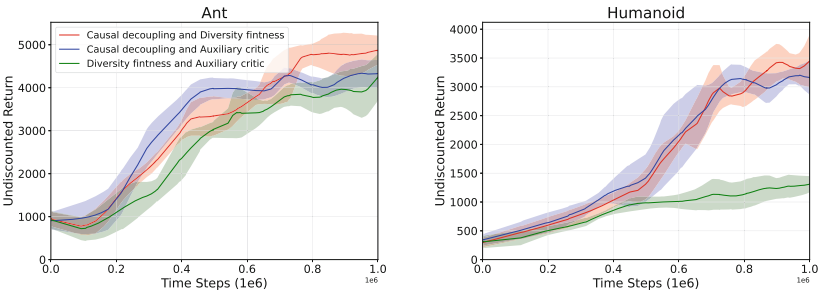


Fig. 7. the different combinations of modules

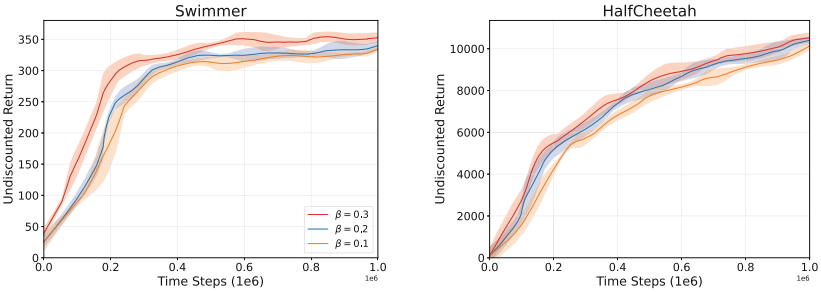


Fig. 8. the weights of the loss

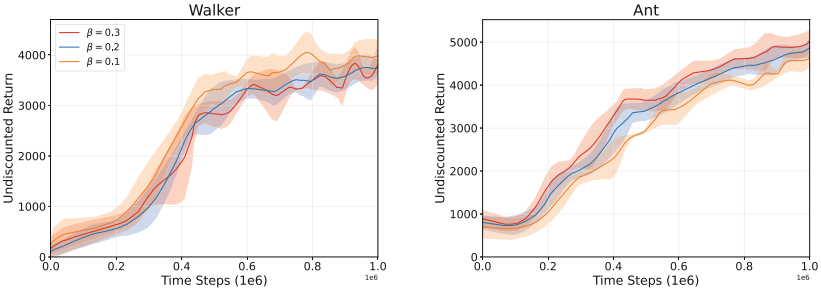


Fig. 9. the weights of the fitness

to the evaluation weights for reward and sample diversity, respectively. Here, β is set to three different values: 0.1, 0.2, and 0.3. As shown in Fig. 9, more diverse samples are needed to improve performance in complex environments. For instance, in the Ant environment, increasing the diversity weight by 0.1 results in a performance gain of about 100, while in the Walker environment, lower diversity weights perform better.

6 Conclusion

We propose an innovative method called ERL-CDR, which builds upon traditional ERL algorithms by introducing a causally disentangled representation extraction approach. This enables agents to distinguish key environmental factors in complex states. Sample diversity is further enhanced through diversity-based fitness estimation, and an auxiliary critic is used to independently learn from the population distribution, improving convergence speed. Experimental results show that through the interaction of the three modules, ERL-CDR improves the final reward by about 5–20% over various baselines in most environments. Although the reward improvement is smaller in some cases, the number of samples required for convergence can be reduced by 10–20%, ultimately achieving strong performance.

7 Limitations and Future Work

7.1 Limitations

Despite the promising results achieved by our proposed causal decoupling representation network, there are several limitations that should be acknowledged.

First, the method introduces additional computational overhead compared to baseline approaches. The multi-branch architecture and the associated loss terms increase the complexity of both the forward and backward passes during training, resulting in longer training times. This increased time cost may limit the scalability of our approach, especially when applied to large-scale or real-time reinforcement learning tasks. Tabel 4 presents the time consumption of different algorithms on MUJOCO every 10,000 steps. The additional time cost mainly comes from the training of the auxiliary critic and the causal decoupling network. All experiments are conducted on an NVIDIA RTX 4070 Ti GPU and an Intel(R) Core i5-12600F CPU.

Second, while our experimental results consistently demonstrate performance improvements over existing methods, our approach currently lacks formal theoretical guarantees. The effectiveness of the causal decoupling representation network is supported by empirical evidence, but a rigorous theoretical analysis of its properties and limitations has not yet been established. This may affect the generalizability and interpretability of the method in broader contexts.

In summary, although our method shows clear empirical advantages, further research is needed to address its computational efficiency and to establish theoretical guarantees, thereby enhancing its practical applicability and scientific rigor (Table 4).

Table 4. The seconds consumption on MUJOCO tasks every 10000 steps

Environments	Walker	Ant	Swimmer	Hopper	Humanoid
ERL-CDR	472.1	876.3	329.7	354.6	1253.6
PDERL	283.6	672.5	212.6	267.3	1152.6
ERL	251.7	525.2	193.6	212.5	924.7
CEM-DDPG	278.6	563.3	189.3	237.6	915.6
DDPG	221.8	421.6	135.6	172.6	862.3

7.2 The Future Work

In future work, we plan to further optimize the computational efficiency of our causal decoupling representation network, for example by exploring lightweight architectures and distributed training strategies. Additionally, we aim to establish formal theoretical guarantees for our method to better understand its properties and limitations. We also intend to extend our approach to more complex and diverse reinforcement learning environments, including those with partial observability and multi-agent interactions. Finally, investigating the integration of causal representation learning with other advanced techniques, such as meta-learning and hierarchical reinforcement learning, will be an important direction to enhance the adaptability and generalization of our framework.

Acknowledgements. This work was supported by Special Funding for Postdoctoral Research Projects in Chongqing(2024CQBSHTB3045) and General Program of Chongqing Natural Science Foundation (CSTB2024NSCQ-MSX0729).

References

1. Kaelbling, L.P., Littman, M.L., Moore, A.W.: Reinforcement learning: a survey. J. Artif. Intell. Res. **4**, 237–285 (1996)
2. Bertsekas, D.: Reinforcement Learning and Optimal Control, vol. 1. Athena Scientific (2019)
3. Souchleris, K., Sidiropoulos, G.K., Papakostas, G.A.: Reinforcement learning in game industry–review, prospects and challenges. Appl. Sci. **13**(4), 2443 (2023)
4. Ravi Kiran, B., et al.: Deep reinforcement learning for autonomous driving: a survey. IEEE Trans. Intell. Transp. Syst. **23**(6), 4909–4926 (2021)
5. Bartz-Beielstein, T., Branke, J., Mehnen, J., Mersmann, O.: Evolutionary algorithms. Wiley Interdis. Rev. Data Min. Knowl. Discov. **4**(3), 178–195 (2014)
6. Bai, H., Cheng, R., Jin, Y.: Evolutionary reinforcement learning: a survey. Intell. Comput. **2**, 0025 (2023)
7. Khadka, S., et al.: Collaborative evolutionary reinforcement learning. In International Conference on Machine Learning, pp. 3341–3350. PMLR (2019)
8. Khadka, S., Tumer., K.: Evolution-guided policy gradient in reinforcement learning. Adv. Neural Inf. Process. Syst. **31** (2018)
9. Forrest, S.: Genetic algorithms. ACM Comput. Surv. (CSUR) **28**(1), 77–80 (1996)

10. Lillicrap, T.P., et al.: Continuous control with deep reinforcement learning. arXiv preprint arXiv:1509.02971 (2015)
11. Pourchot, A., Sigaud, O.: CEM-RL: combining evolutionary and gradient-based methods for policy search. arXiv preprint arXiv:1810.01222 (2018)
12. Ma, Y., Liu, T., Wei, B., Liu, Y., Xu, K., Li, W.: Evolutionary action selection for gradient-based policy learning. In: International Conference on Neural Information Processing, pp. 579–590. Springer (2022). https://doi.org/10.1007/978-3-031-30111-7_49
13. Zeng, Y., Cai, R., Sun, F., Huang, L., Hao, Z.: A survey on causal reinforcement learning (2023)
14. Tesauro, G., et al.: Temporal difference learning and td-gammon. Commun. ACM **38**(3), 58–68 (1995)
15. Fujimoto, S., Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: International Conference on Machine Learning, pp. 1587–1596. PMLR (2018)
16. Bäck, T., Schwefel, H.P.: Evolutionary Algorithms: Theory and Applications. Springer (1993). https://doi.org/10.1007/978-3-032-00385-0_27
17. De Boer, P.T., Kroese, D.P., Mannor, S., Rubinstein, R.Y.: A tutorial on the cross-entropy method. Ann. Opera. Res. **134**, 19–67 (2005)
18. Zheng, H., Jiang, J., Wei, P., Long, G., Zhang, C.: Competitive and cooperative heterogeneous deep reinforcement learning. In: Proceedings of the International Joint Conference on Autonomous Agents and Multiagent Systems, AAMAS (2020)
19. Elfwing, S., Uchibe, E., Doya, K.: Online meta-learning by parallel algorithm competition. In: Proceedings of the Genetic and Evolutionary Computation Conference, pp. 426–433 (2018)
20. Franke, J.K.H., Köhler, G., Biedenkapp, A., Hutter, F.: Sample-efficient automated deep reinforcement learning. arXiv preprint arXiv:2009.01555 (2020)
21. Bodnar, C., Day, B., Lió, P.: Proximal distilled evolutionary reinforcement learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 3283–3290 (2020)
22. Pearl, J.: Causality. Cambridge university press (2009)
23. Chen, X., et al.: Variational lossy autoencoder. arXiv preprint arXiv:1611.02731 (2016)
24. Kim, H., Mnih, A.: Disentangling by factorising. In: International Conference on Machine Learning, pp. 2649–2658. PMLR (2018)
25. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms. arXiv preprint arXiv:1707.06347, (2017)
26. Haarnoja, T., et al.: Soft actor-critic algorithms and applications. arXiv preprint arXiv:1812.05905 (2018)

Model Archive and Ensemble Learning Based Differential Evolution Algorithm for Mixed-Variable Optimization

Qing Chen, Xiaogang Dong(✉), and Changshou Deng

School of Computer and Big Data Science, Jiujiang University, Jiujiang 332005, China
dongxg@jju.edu.cn

Abstract. Optimization problems play a pivotal role in the development of various fields. Among them, mixed-variable optimization problems (MVOPs) involving both continuous and discrete variables warrant particular attention, as they are prevalent in engineering design and pose substantial computational challenges for solution methods. To this end, this paper proposes a novel variant of differential evolution (DE), termed the model archive and ensemble learning-based differential evolution algorithm (MEDE). Specifically, a data enhancing strategy based on dynamic opposition-based learning is first devised to improve the diversity of the dominant population. Furthermore, a model archive is designed to store the probabilistic distribution models learned during the early-stage evolutionary process, which enables the algorithm to efficiently exploit learned probabilistic distribution models while preventing premature convergence. Finally, by utilizing stacking ensemble learning, the model's predictive accuracy and generalization capability have been enhanced, thereby improving the overall optimization performance of the MEDE algorithm. To validate the effectiveness and efficiency of the proposed algorithm, 28 artificial benchmark functions are selected to conduct performance evaluation experiments. The experimental results clearly show that compared with other similar algorithms, i.e., AEDA, S-PSO, ACO_{mv}, and DE_{mv}, the proposed algorithm shows significant advantages in terms of quality and efficiency.

Keywords: Mixed-variable optimization · Differential evolution algorithm · Dynamic opposition-based learning · Model archive · Stacking ensemble learning

1 Introduction

Optimization problems denote the search for solutions that can make the objective function achieve the optimal state under the premise of satisfying a series of established constraints. Within the context of optimization problems, the decision variables serve as the core elements, and the process essentially involves

© The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 571–589, 2026.
https://doi.org/10.1007/978-981-95-8405-5_31

determining their specific values to find the optimal solution. Based on the form of variables, decision variables are classified as continuous variables and discrete variables [1]. Consequently, optimization problems can be divided into continuous optimization problems and discrete optimization problems. However, optimization problems in the real world are far more complex than simple continuous and discrete optimization problems. For example, there are pressure vessel design problems [2], aircraft wing design problems [3], and welded beam design problems [4] in mechanical structures. The decision variables in these problems include both continuous variables and discrete variables. Such problems are classified as mixed-variable optimization problems (MVOPs).

As a simple yet efficient optimization algorithm, the evolutionary algorithm, such as differential evolution (DE) [5] which has demonstrated outstanding optimization performance in recent years, has been successfully applied in various industries. JADE [6], LSHADE-RSP [7], and iDE-EDA [8] are all improved versions of DE. However, the improvement of these algorithms revolves around simple continuous optimization problems. So when faced with mixed-variable optimization problems, these algorithms still have certain performance deficiencies, which make it difficult for them to solve these problems efficiently and accurately.

Based on the above description, a new solution scheme is proposed for solving mixed-variable optimization problems, namely model archive and ensemble learning-based differential evolution algorithm (MEDE). In this paper, the algorithm incorporates dynamic opposition-based learning to increase data diversity. In addition, the model archive is innovatively proposed to store the probability distribution models generated throughout the evolutionary process. Furthermore, combining the stacking ensemble learning method improves the precision and reliability of models. The main contributions of this study can be concluded as follows:

- A data enhancing strategy based on dynamic opposition-based learning and model archive strategy are proposed. The model archive strategy that stores probability distribution models in the algorithm evolution process effectively solves the problem of insufficient data diversity at the later stage of the algorithm.
- The stacking ensemble learning method is added to MEDE. Specifically, three models are randomly selected from the model archive to serve as weak learners for stacking ensemble learning, which effectively enhances the accuracy and generalization of the generated samples.
- Through effectiveness analysis experiments and performance comparison experiments, the study verified the feasibility of the proposed algorithm.

The structure of this paper is organized as follows. Section 2 presents the classical differential evolution algorithm and its relevant variants. Section 3 elaborates on the proposed MEDE algorithm and its algorithmic framework. Section 4 offers a comparative analysis of the experimental results, focusing on the effectiveness assessment and performance comparison between MEDE and other algorithms. Section 5 draws the conclusion.

2 Related Works

2.1 The Original DE

Differential Evolution (DE) is a population-based stochastic optimization algorithm. It utilizes a population with a size of NP to solve various types of optimization problems through an iterative evolutionary process. The core operations of this algorithm encompass three key steps: mutation, crossover, and selection. During the algorithm's initialization phase, it is necessary to initialize this population in a D-dimensional space according to Eq. (1).

$$X_i = LB + rand \cdot (UB - LB) \tag{1}$$

where $i \in \{1, 2, 3, \cdots, NP\}$ represents the index of an individual within the population. LB and UB denote the lower and upper search boundaries of the problem.

In the mutation stage, the mutation vector V_i is generated from individual X_i in the population according to the preset mutation strategy to enhance the diversity of the population. Here are six classic mutation strategies:

(1) DE/rand/1:

$$V_{i,G} = X_{i,G} + F \cdot (X_{r2,G} - X_{r3,G}) \tag{2}$$

(2) DE/rand/2:

$$V_{i,G} = X_{i,G} + F \cdot (X_{r2,G} - X_{r3,G}) + F \cdot (X_{r4,G} - X_{r5,G}) \tag{3}$$

(3) DE/best/1:

$$V_{i,G} = X_{best,G} + F \cdot (X_{r1,G} - X_{r2,G}) \tag{4}$$

(4) DE/best/2:

$$V_{i,G} = X_{best,G} + F \cdot (X_{r1,G} - X_{r2,G}) + F \cdot (X_{r3,G} - X_{r4,G}) \tag{5}$$

(5) DE/current-to-best/1:

$$V_{i,G} = X_{i,G} + F \cdot (X_{best,G} - X_{i,G}) + F \cdot (X_{r1,G} - X_{r2,G}) \tag{6}$$

(6) DE/current-to-rand/1:

$$U_{i,G} = X_{i,G} + rand \cdot (X_{r1,G} - X_{i,G}) + F \cdot (X_{r2,G} - X_{r3,G}) \tag{7}$$

where $X_{r1,G}$... $X_{r5,G}$ represent a random selection of 5 different individuals, and $X_{best,G}$ is the optimal individual in the population. G represents the number of iterations in which the current individual $X_{i,G}$ is located. F is the scaling factor, which is a crucial parameter in the DE algorithm, typically lies within the interval $(0, 2]$, which is used to construct variation vectors based on differences between different individuals. $V_{i,G}$ is the mutation vector of $X_{i,G}$. In the sixth mutation strategy (DE/current - to - rand/1), the trial vector is $U_{i,G}$ directly

generated, skipping the conventional cross-operation process and constructing new individuals to participate in subsequent selection in a unique way.

In the crossover phase, the trial vector $U_{i,G}$ is generated from the target vector $X_{i,G}$ and the mutation vector, according to the adjustment of the crossover probability. Here is the crossover equation:

$$U_{i,j,G} = \begin{cases} V_{i,j,G} & \text{if } rand_j \leq CR \text{ or } j = j_{rand} \\ X_{i,j,G} & \text{otherwise} \end{cases} \tag{8}$$

where j represents the jth dimension, $rand_j$ is a randomly generated number between 0 and 1. j_{rand} is randomly generated in the dimension to ensure that $U_{i,j}$ and $X_{i,j}$ are not exactly the same, and CR is a crossover probability with its value constrained to the interval [0,1].

In the selection phase, choose a better vector based on the fitness values of the trial vector $U_{i,j}$ and the target vector $X_{i,j}$ after the end of generation G. The description is as follows:

$$X_{i,G+1} = \begin{cases} U_{i,G} & \text{if} f(U_{i,G}) < f(X_{i,G}) \\ X_{i,G} & \text{otherwise} \end{cases} \tag{9}$$

2.2 Some Advanced DE Version

(1) JADE

The JADE algorithm is an adaptive differential evolution algorithm with optional external archive. In the process of algorithm optimization, the successful individual information of the current population is used to adjust the parameters, namely the scaling factor F and the crossover probability CR. In addition, an external archive mechanism is introduced to save the individuals that have been replaced during the iteration process to provide the direction of evolution. Through these two strategies, the exploration and exploitation capabilities of algorithms in the search process are effectively balanced.

(2) LSHADE-RSP

LSHADE-RSP, one of the winners of the IEEE CEC 2018, is an improved algorithm based on the LSHADE algorithm. The algorithm improves the mutation operation with rank-based selective pressure strategy, which performs better performance when solving complex multi-model optimization problems.

The initialization strategy of the LSHADE-RSP algorithm is the same as that of the classic DE, as shown in Eq.(1). The mutant vectors V_i is generated based on the mutant strategy with rank-based selective pressure named "DE/$current-to-pbest-w/r$", as shown in the following:

$$V_i = X_i + Fw_i \cdot (X_{pbest} - X_i) + F_i \cdot (X_{pr_1} - X_{pr_2}) \tag{10}$$

$$Fw_i = \begin{cases} 0.7 \cdot F_i & \text{F}Es < 0.2 \cdot FEs_{max} \\ 0.8 \cdot F_i & 0.2 \cdot FEs_{max} \leq FEs < 0.4 \cdot FEs_{max} \\ 1.2 \cdot F_i & \text{F}Es \geq 0.4 \cdot FEs_{max} \end{cases} \tag{11}$$

where X_{pbest} is randomly selected from the dominant individual $NP * p$, p is the greed factor to balance exploration and exploitation. F_i is the scaling factor. X_{pr_1} and X_{pr_2} are selected based on the ranking of the probability that each individual will be selected, as in Eqs. (12) and (13). In addition, X_{pr_2} may be randomly selected from the external archive A.

$$Pr_i = Rank_i/(Rank_1 + Rank_2 + \cdots + Rank_{NP}) \tag{12}$$

$$Rank_i = k(NP - i) + 1 \tag{13}$$

In Eq. (13), $Rank_i$ is the ith ranking in the current population, k always set 3 in the LSHADE-RSP. Pr_i is the probability that the ith individual will be selected.

Furthermore, adaptive parameter control based on a history of success and linear population reduction are also important parts of LSHADE-RSP. The adaptive parameter control mechanism primarily conducts adaptive tuning of the scale factor F and the crossover probability CR. In each generation of the algorithm, each individual is assigned a unique F and CR.

$$M_{F,K} = \begin{cases} M_{F,K} & \mathrm{S}_F = \emptyset \\ \frac{(mean_{WL}(S_F) + M_{F,K})}{2} & otherwise \end{cases} \tag{14}$$

$$M_{CR,K} = \begin{cases} 0 & \mathrm{M}_{CR,k} = \perp \text{ or } max(S_{CR} = 0) \\ \frac{(mean_{WL}(S_{CR}) + M_{CR,K})}{2} & otherwise \end{cases} \tag{15}$$

$$mean_{WL}(S) = \frac{\sum_{n=1}^{|S|} \omega_n \cdot S_n^2}{\sum_{n=1}^{|S|} \omega_n \cdot S_n} \tag{16}$$

$$\omega_n = \frac{|f(U_n) - f(X_n)|}{\sum_{n=1}^{|S|} |f(U_n) - f(X_n)|} \tag{17}$$

In Eqs. (14) and (15), S_F and S_{CR} are the set of the scaling factor F and the crossover probability CR used by individuals with better fitness values of the trial vector than the target vector. The length of $M_{F,K}$ and $M_{CR,K}$ is uniformly set to H. In the algorithm iteration process, when in the Kth (from 1 to H) generation, the Kth position of $M_{F,K}$ and $M_{CR,K}$ is updated according to the formula. If the value of K exceeds H, K is reset to 1. The initial value of M_F is set to 0.3 and the initial value of M_{CR} is set to 0.8. The initial values of each element in $M_{F,K}$ and $M_{CR,K}$ are both assigned to 0.9. F_i is generated by the Cauchy distribution and CR_i is generated by the Gaussian distribution, i denotes the ith individual.

The formula for linear population reduction is as shown in Eq. (18). In the formula, FEs and FEs_{max} are the current number and maximum number of function evaluations, respectively. NP_{init} is the initial population size, and NP_{min} is the minimum population size, which is set to 4. $round(\cdot)$ is an integer function.

$$NP = round[\frac{FEs}{FEs_{max}}(NP_{min} - NP_{init}) + NP_{init}] \tag{18}$$

(3) iDE-EDA

The iDE-EDA algorithm is based on the LSHADE-RSP algorithm and introduces the estimation-of-distribution algorithm. This improvement aims to integrate the excellent features of the LSHADE-RSP algorithm with the advantages of the EDA algorithm in probabilistic model and sampling. Further improve the performance of the algorithm when solving complex optimization problems.

After the completion of each generation, NP_d dominant individuals are selected in population P to establish a probability model. Then, sampling is carried out according to the probability distribution model, and the sampled individuals are NP_E, as shown in Eq. (20).

$$NP_d = \begin{cases} 0.5 \cdot NP & \mathrm{N}P \geq 4 \cdot D \\ NP & \mathrm{N}P < 4 \cdot D \end{cases} \tag{19}$$

$$NP_E = \tau \cdot p \cdot NP \tag{20}$$

where D is the dimension and $\tau \in (0, 1]$ is utilized to control the number of individuals generated by EDA.

The probability distribution model is built from the Gaussian distribution as follows:

$$G(x)_{(\mu,C)} = \sqrt{\frac{1}{(2 \cdot \pi)^D det(C)}} \cdot exp(-(X - \mu)^T \cdot C^{-1} \cdot (X - \mu)/2) \tag{21}$$

$$\mu = \frac{1}{|P_d|} \cdot \sum_{i=1}^{|P_d|} X_i \quad X_i \in P_d \tag{22}$$

$$C = \frac{1}{|P_d|} \cdot \sum_{i=1}^{|P_d|} (X_i - \mu) \cdot (X_i - \mu)^T \quad X_i \in P_d \tag{23}$$

$$X_i = \mu + g_i \quad g_i \sim N(0, C) \tag{24}$$

where P_d includes the population of dominant individuals.

3 Proposed Approach

3.1 Data Enhancing Based on the Opposition-Based Learning

When dealing with complex optimization problems, traditional algorithms tend to fall into local optimums. Opposition-Based Learning can enhance the ability to jump out of the local optimal by generating the reverse solution of the current solution. As a consequence, a number of opposite-based learning strategies have been devised and their effectiveness has been demonstrated in many evolutionary algorithms.

As mentioned previously, iDE-EDA uses NP_d dominant individuals to establish a probability model. However, as the population undergoes sustained evolutionary progression, the crowding degree of dominant individuals progressively

intensifies, thereby substantially diminishing the diversity of model training data. As a result, the trained model may exhibit a significant degradation in sampling performance due to the above phenomena. To address this limitation, we propose the data enhancing scheme based on the opposition-based learning, which is termed as DOL. The details of DOL are as (25).

$$\begin{aligned} OP_d^i =& P_d^{i'} + r1 * (r2 * (LB + UB - P_d^{i'}) - P_d^{i'}) \\ s.t.:& \\ & i \in [1, |P_d| * 0.3] \\ & i' \in [1, |P_d|] \\ & i' = f(i) \end{aligned} \tag{25}$$

where, LB and UB represent the lower and upper bounds of the individual population, respectively. $|\cdot|$ denotes the modulo operation. $f(\cdot)$ is an injective mapping between i and i'.

DOL is constructed based on opposition-based learning, with the main objective of performing data enhancing on dominant individual populations to improve the model's generalization capability. In the implementation of DOL, there are two critical aspects that need to be noted.

- The dynamic opposition-based learning strategy is used to further improve the performance of data enhancing.
- To conserve evaluation resources, only the 30% of dominant individuals are randomly selected to perform the dynamic opposition-based learning.

3.2 Model Archive

Recent years, the archive strategies have been increasingly widespread and effective in the field of evolutionary algorithms. The core advantage of these strategies lies in their ability to dynamically and efficiently balance exploration and exploitation during the search process of evolutionary algorithms, thereby significantly enhancing their overall performance and convergence speed. However, it should be noted that most current archive strategies primarily focus on the storage and management of individual solutions, which, to a certain extent, limits their capacity to further explore the search space and adapt to complex optimization problems.

Therefore, we propose an innovative archive strategy for storage models, which is based on the key problems of the model-assisted differential evolution algorithm for solving complex optimization problems, such as insufficient data diversity and overfitting of data diversity of probability distribution models, to enhance the robustness and search efficiency of the algorithm.

In terms of algorithm implementation, we design a set of fine archival structure and archival update mechanism. During each iteration of the iDE-EDA algorithm, the currently generated model is saved to the model archive A_{model}.

The specific structure of A_{model} as in Eq. (26). In the archive management mechanism, the archive only stores the model generated during the first 50 iterations corresponding to the current number of iterations.

$$A_{model} = \begin{bmatrix} \mu_1, \mu_2, \ldots, \mu_m \\ C_1, C_2, \ldots, C_m \end{bmatrix} \tag{26}$$

where μ_m and C_m are the mean and covariance of the m model in A_{model}, the value of m is 50. $[\cdot]$ is the matrix. Here are the considerations.

- The m value is too large, which increases the computational cost and limits the exploitation capacity. If it is too small, the data diversity will be poor, and it is easy to fall into the local optimization. Experiments have shown that m is more effective with 50.
- The model update strategy adopts the proposal of Li et al. [9] first-in-first-out (FIFO) archive updates policies to maintain model diversity.

3.3 Ensemble Learning

As a cutting-edge and influential machine learning method, ensemble learning is based on the principle of improving model performance by integrating the prediction results of multiple learners. Specifically, ensemble learning usually combines multiple weak learners to build a strong learner with excellent performance, resulting in more stable and accurate prediction results. Common combination strategies include Bagging, Boosting, and Stacking. In the MEDE algorithm, in order to further improve the model performance and generalization ability, the stacking ensemble learning method is added in an innovative way. This method trains multiple weak models and uses the prediction results of these weak models as new features to train a strong model to make the final prediction. Improve overall performance by combining the strengths of different models. Provides more robust model support for solving complex problems.

In the specific method, in the first 50% iterations, the same model prediction strategy is used as the iDE-EDA algorithm. In the last 50% iterations, three models are randomly selected as weak learners in the model archive. Use the three weak learners (M_1, M_2 and M_3) to generate q individuals separately and merge these individuals to form a new data set *inds*, as shown in Eq. (27). The mean of the set *inds* mu_E and the covariance C_E are obtained as parameters for the strong learner.

$$inds = sampling_q(M_1) \cup sampling_q(M_2) \cup sampling_q(M_3) \tag{27}$$

The current model M^C is used to generate $(1-\lambda) \cdot NP_E$ individuals and the strong learner M^E is used to generate $\lambda \cdot NP_E$ individuals, where λ is the sampling weight of the strong learner. Then combine the individuals to form a population X_E as Eq. (28).

$$X_E = sampling_{(1-\lambda)\cdot NP_E}(M^C) \cup sampling_{\lambda \cdot NP_E}(M^E) \tag{28}$$

3.4 MEDE Framework

In this subsection, an in-depth exploration is conducted on the integrated application of three innovative strategies, namely dynamic opposition-based learning, model archive mechanisms, and stacking ensemble learning techniques. The synergy of these strategies is aimed at enhancing the performance of evolutionary algorithms. Based on this, an improved variant of DE, named MEDE, is proposed. The complete operational framework of the MEDE algorithm is presented in pseudocode within Algorithm 1.

Algorithm 1. The procedure of MEDE

Input: f ,$[x_{min}, x_{max}]$,FES_{max}.
Output: x_{best} .

$NP_{init} = 30$, $D = 50$, $D1 = 25$, $NP_{init} = 4$, $FEs = 0$, $A = \emptyset$, $A_{model} = \emptyset$, $count = 1$.
Initialize population X by using Eq. (1).
Apply $round(\cdot)$ to dimensions $D1$ through D.
Evaluate X to determine their fitness.
$FES = FES + NP$.
while $FES < FES_{max}$ **do**
  Determine Pr as Eq. (12) and $Rank_i$ as Eq. (13)
  $M_{CR,H} = 0.9$, $M_{F,H} = 0.9$,$S_{CR} = \emptyset$, $S_F = \emptyset$.
  for $i = 1, 2, \cdots, NP$ **do**
    Calculate Fw_i by Eq. (11) and generate mutant vector V_i using Eq. (10).
    Generate trial individual U_i.
    if $f(U_i) < f(X_i)$ **then**
      Update external archive A.
      $X_i = U_i$, $f(X_i) = f(U_i)$, $S_{CR} = Cr_i \cup S_{CR}$, $S_F = Fr_i \cup S_F$.
    end if
  end for
  Update M_F and M_{CR} as Eq. (14) and Eq. (15).
  Sort X and select the first NP_d individuals to form X_d.
  Select $0.3 * NP_d$ individuals from X_d for DOL as Eq. (25).
  Learn Gaussian probability model by X_d and obtain the mean and covariance.
  Update A_{model} by FIFO strategy.
  if $FEs >= 0.5 * FEs_{max}$ **then**
    Select three models M_1 ,M_2 ,M_3 randomly.
    Sample $inds$ using Eq. (27), and Check bound-constraints.
    Obtain strong learner parameters, and generate X_E as Eq. (28).
  else
    Sample X_E by current model.
  end if
  $FEs = FEs + NP + NP_E$.
  $X = X \cup X_E$, $f(X) = f(X) \cup f(X_E)$
  Calculate NP according to Eq.(18).
  Shrink X and A by discarding the worst individuals.
end while

3.5 Complexity Analysis

Obviously, the MEDE algorithm introduces three additional components in comparison with the iDE-EDA algorithm, namely the computation of dynamic opposite-based learning, the save of model archive, and the operation of stacking ensemble learning.

The complexity associated with computing dynamic opposite-based learning is closely related to the population size NP per iteration and the total number of iterations $iter$. To be more specific in MEDE, select 30% of the dominant individuals for dynamic opposite-based learning, where the number of dominant individuals is 50% of NP. Specifically, the complexity of this part is $O(0.15 * NP * iter)$. The model archive's save operation mainly stores the model's mean and covariance. The time complexity of this process is negligible.

In the context of the stacking ensemble learning procedure, during roughly the latter half (approximately 50%) of the total iterations, a random selection process is carried out to pick three individuals from the model archive. Following this selection, the mean and covariance values for these three chosen individuals are computed. These computed mean and covariance metrics are then leveraged to generate new individuals, which are introduced into the learning process. Upon a thorough evaluation, the complexity of this particular segment of the stacking ensemble learning operation can be succinctly expressed as $O(0.5 * iter * 6)$.

Based on the above analysis, the additional complexity introduced by the MEDE algorithm is $O(0.15 * NP * iter + 0.5 * iter * 6)$.

4 Experimental Results and Study

In the subsequent experimental section, we employ 28 artificial benchmark functions sourced from the IEEE CEC 2013 test suite [10]. All experiments are conducted in a 50-dimensional $D = 50$ search space. For each benchmark function, every algorithm is executed independently 25 times.

Regarding the algorithm parameters, the population size NP for each algorithm is set to 30, and the maximum number of function evaluations FEs_{max} is 100,000. Additionally, the decision variables are partitioned into two distinct segments. Specifically, the first 25 dimensions of the decision variables are treated as continuous variables, while the remaining dimensions are designated as discrete variables.

In the analysis of experimental results, use symbols for clarity. "+" means our algorithm outperforms others under metrics. "-" shows it's inferior. And "≈" indicates similar performance within an acceptable error range. The symbols "Mean" and "Std" represent the average function error and standard deviation.

To comprehensively assess the capabilities of various algorithms in addressing the aforementioned problems, we have devised two experimental studies: an effectiveness analysis and a performance comparison for mixed-variable optimization.

Table 1. The data results of effectiveness analysis

Function	Mean/Std	JADE	LSHADE-RSP	iDE-EDA	MEDE
f1	Mean	1.56e+5(+)	0.00e+0(+)	4.33e-1(+)	3.33e-2
	Std	1.42e+4	0.00e+0	7.28e-1	1.83e-1
f2	Mean	2.15e+9(+)	6.10e+6(≈)	5.96e+6(≈)	4.45e+6
	Std	7.42e+8	3.00e+6	3.41e+6	2.17e+6
f3	Mean	3.96e+17(+)	3.44e+9(+)	3.93e+9(+)	1.88e+9
	Std	8.60e+17	3.18e+9	2.23e+9	1.48e+9
f4	Mean	2.76e+5(+)	2.40e+3(≈)	2.13e+3(-)	2.73e+3
	Std	3.62e+4	1.14e+3	1.03e+3	1.30e+3
f5	Mean	9.26e+4(+)	2.28e+0(≈)	4.12e+0(+)	2.48e+0
	Std	2.17e+4	1.22e+0	1.67e+0	1.38e+0
f6	Mean	2.25e+4(+)	1.03e+2(≈)	1.18e+2(≈)	8.37e+1
	Std	4.32e+3	4.65e+1	5.08e+1	4.13e+1
f7	Mean	1.94e+5(+)	9.57e+1(+)	9.79e+1(+)	8.17e+1
	Std	2.56e+5	2.44e+1	2.10e+1	1.47e+1
f8	Mean	2.12e+1(≈)	2.12e+1(≈)	2.12e+1(≈)	2.12e+1
	Std	3.50e-2	3.43e-2	5.08e-2	3.41e-2
f9	Mean	5.96e+1(+)	5.23e+1(+)	4.78e+1(≈)	4.62e+1
	Std	2.89e+0	7.74e+0	7.39e+0	8.46e+0
f10	Mean	1.96e+4(+)	4.41e+1(+)	5.82e+1(+)	2.11e+1
	Std	2.81e+3	2.85e+1	3.00e+1	1.18e+1
f11	Mean	2.14e+3(+)	2.19e+2(+)	2.57e+2(≈)	2.57e+2
	Std	1.89e+2	5.12e+1	6.78e+1	5.69e+1
f12	Mean	2.21e+3(+)	2.78e+2(≈)	2.99e+2(≈)	3.11e+2
	Std	2.52e+2	5.34e+1	6.79e+1	6.97e+1
f13	Mean	2.17e+3(+)	4.05e+2(-)	4.39e+2(≈)	4.70e+2
	Std	2.61e+2	4.97e+1	7.59e+1	8.34e+1
f14	Mean	1.36e+4(+)	2.33e+3(+)	2.52e+3(≈)	3.32e+3
	Std	5.59e+2	2.43e+3	2.65e+3	2.72e+3
f15	Mean	1.32e+4(+)	8.24e+3(≈)	8.37e+3(≈)	8.11e+3
	Std	4.94e+2	9.80e+2	1.01e+3	7.34e+2
f16	Mean	3.65e+0(+)	2.66e+0(≈)	2.69e+0(≈)	2.92e+0
	Std	2.56e-1	6.03e-1	7.15e-1	6.90e-1
f17	Mean	4.98e+3(+)	3.18e+2(-)	3.82e+2(≈)	3.74e+2
	Std	4.87e+2	6.62e+1	7.68e+1	6.47e+1
f18	Mean	4.99e+3(+)	3.83e+2(-)	4.27e+2(≈)	4.38e+2
	Std	3.67e+2	5.64e+1	6.42e+1	6.98e+1
f19	Mean	1.19e+7(+)	3.71e+1(≈)	5.45e+1(+)	3.71e+1
	Std	4.26e+6	1.16e+1	1.47e+1	1.16e+1
f20	Mean	2.39e+1(+)	2.33e+1(+)	2.10e+1(≈)	2.14e+1
	Std	1.45e+0	1.94e+0	1.40e+0	1.77e+0
f21	Mean	9.55e+3(+)	4.02e+2(≈)	4.06e+2(+)	4.02e+2
	Std	9.39e+2	1.50e+0	2.82e+0	1.75e+0
f22	Mean	1.43e+4(+)	2.38e+3(≈)	3.63e+3(≈)	2.63e+3
	Std	6.18e+2	2.60e+3	2.85e+3	2.48e+3
f23	Mean	1.44e+4(+)	8.72e+3(≈)	8.49e+3(≈)	8.76e+3
	Std	6.00e+2	7.11e+2	1.10e+3	8.90e+2
f24	Mean	7.37e+2(+)	3.30e+2(≈)	3.26e+2(≈)	3.23e+2
	Std	1.81e+2	1.55e+1	1.32e+1	1.37e+1
f25	Mean	4.61e+2(+)	3.56e+2(≈)	3.54e+2(≈)	3.52e+2
	Std	1.39e+1	1.16e+1	1.25e+1	1.13e+1
f26	Mean	5.00e+2(+)	3.97e+2(≈)	4.08e+2(≈)	4.01e+2
	Std	1.24e+1	4.23e+1	9.21e+0	4.17e+1
f27	Mean	3.18e+3(+)	1.51e+3(≈)	1.54e+3(+)	1.47e+3
	Std	3.65e+2	1.05e+2	1.03e+2	8.78e+1
f28	Mean	1.44e+4(+)	3.49e+3(≈)	3.93e+3(≈)	3.73e+3
	Std	1.55e+3	7.31e+2	6.85e+2	7.03e+2
+/ ≈ /-		27/1/0	8/17/3	8/19/1	

4.1 Effectiveness Analysis

With the aim of conducting a scientific and comprehensive evaluation of the effectiveness of the MEDE algorithm, we have deliberately and meticulously selected its original version as well as other relevant algorithms JADE, LSHADE-

RSP, and iDE-EDA as comparison counterparts. During the experiment, the four algorithms are compared in the same experimental setup, and the results are shown in Table 1.

According to the results, compared to the JADE algorithm, MEDE is better in 27 functions and similar in 1 function, and no function is inferior to the JADE algorithm. Compared to the LSHADE-RSP algorithm, MEDE achieves superior performance in 8 functions, is similar in 17 functions, and is slightly inferior to the LSHADE-RSP algorithm in 3 functions. Compared with the original algorithm iDE-EDA, MEDE is better in 8 functions, similar in 19 functions, and slightly inferior to iDE-EDA in only 1 function.

Under the same experimental setup (each test function is run independently 25 times and the mean value is taken to generate the convergence curve and Y-axis in log_{10} scale), the detailed convergence process is compared as shown in Figs. 1, 2, 3 and 4. In the test function presented in Fig. 1, the convergence performance of the MEDE algorithm is significantly better than that of the other three comparison algorithms. For the test function in Fig. 2, the convergence results of the MEDE algorithm are similar to the other three algorithms and are significantly better than the JADE algorithm in the Fun 4, 5, 19, and 21 functions. Among the functions involved in Fig. 3, for Fun 1, Fun17 and Fun18, the convergence performance of the MEDE algorithm is slightly inferior to that of the LSHADE-RSP algorithm, but better than other algorithms. For Fun13 and Fun14 functions, the convergence performance of the MEDE algorithm is slightly weaker than that of LSHADE-RSP and iDE-EDA algorithms, but it is still stronger than the JADE algorithm.In Fig. 4, the convergence results of the MEDE algorithm reach the same level as those of the other three algorithms.

A detailed summary of the MEDE algorithm and other algorithms is shown in Table 2, the MEDE algorithm has significant advantages in 43 functions and is similar in 37 functions and slightly worse in 4 functions than the other three algorithms.

Through the analysis of the above experiments, it can be seen that the MEDE algorithm has significant advantages in solving the mixed-variable optimization problems. This result fully verifies that a series of improved strategies implemented for MEDE are highly effective and adaptable to mixed-variable optimization problems. These improvements not only increase the algorithm's search ability in complex mixed variable spaces but also significantly improve global optimization performance.

4.2 Performance Comparison for Mixed-Variable Optimization

In the context of the performance comparison experiment, a rigorous experimental design is implemented to ensure a thorough and comprehensive evaluation of the MEDE algorithm's performance in addressing mixed-variable optimization problems, we select AEDA [11], ACO_{mv} [12], S-PSO [13], and DE_{mv} [14], which are the most representative algorithms in this field. These algorithms demonstrate the unique advantages of solving MVOPs in different aspects. Here is a brief summary of the description of the four comparison algorithms.

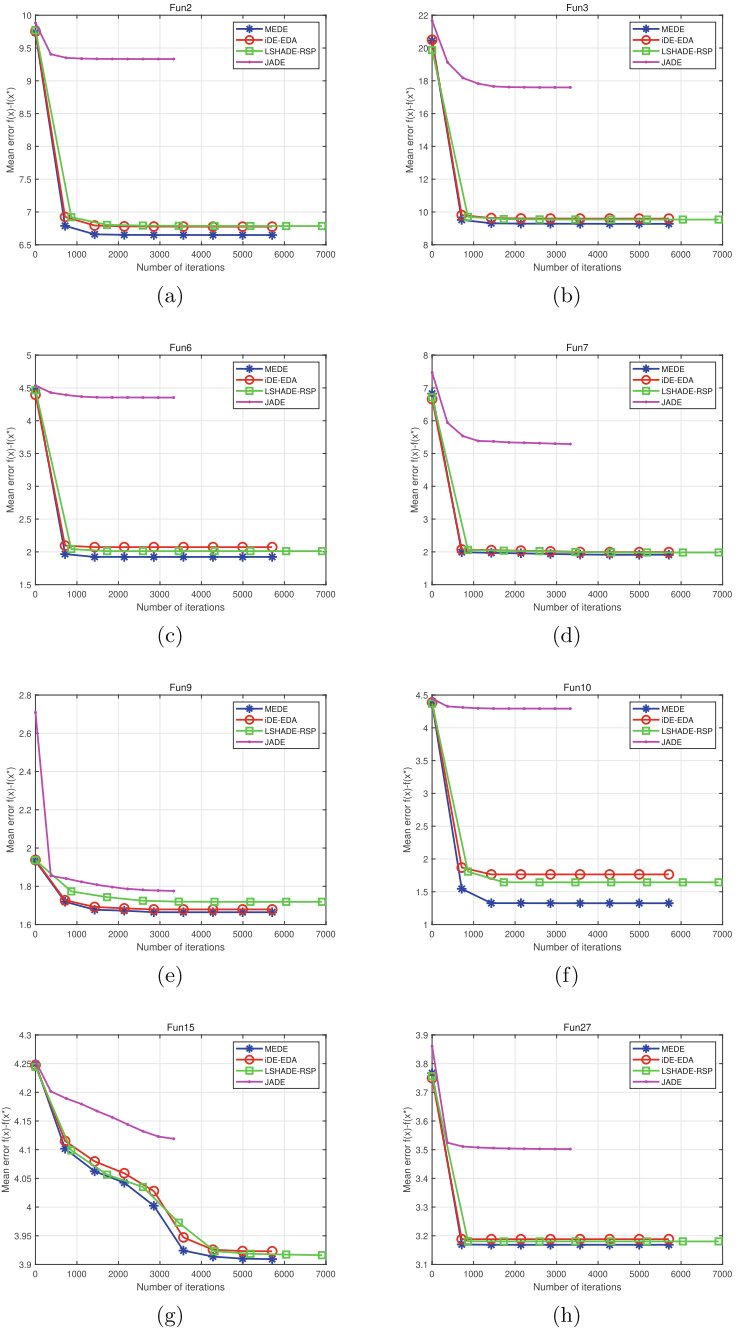


Fig. 1. MEDE is significantly better than Comparison algorithms

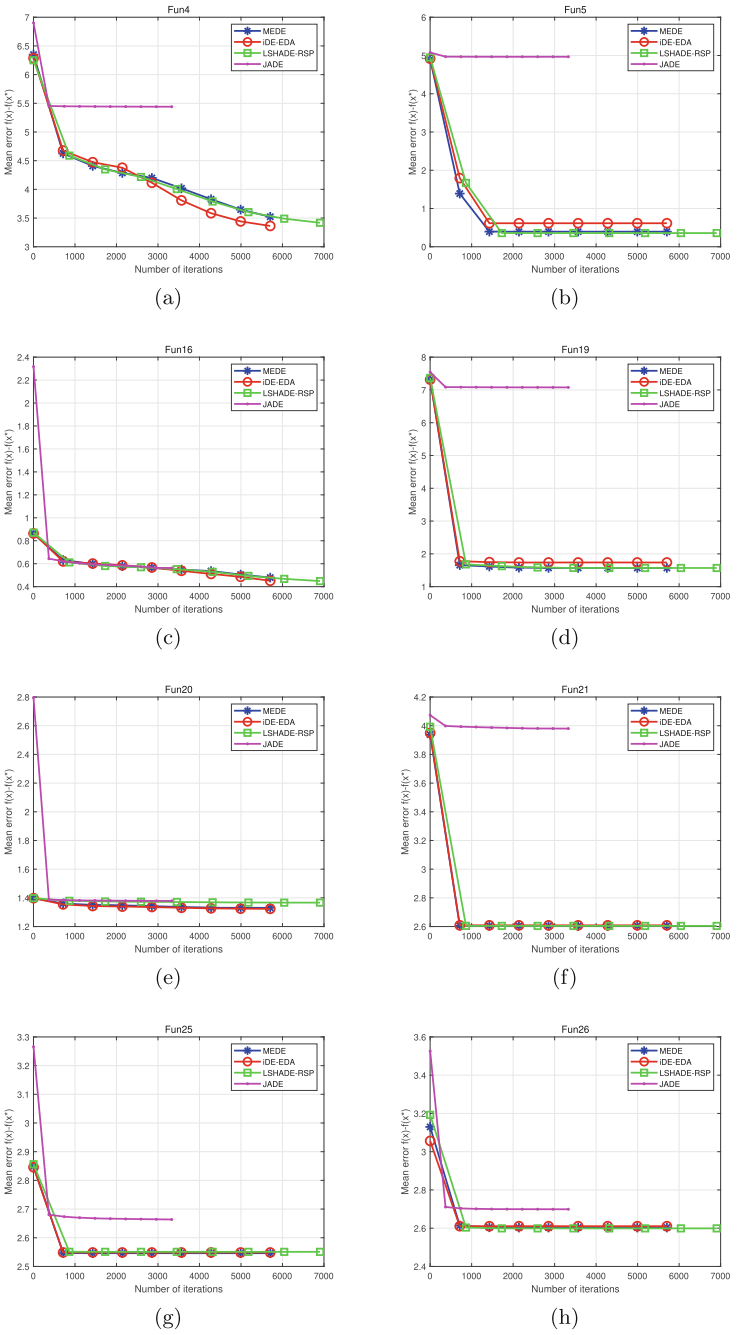


Fig. 2. MEDE is similar to Comparison algorithms

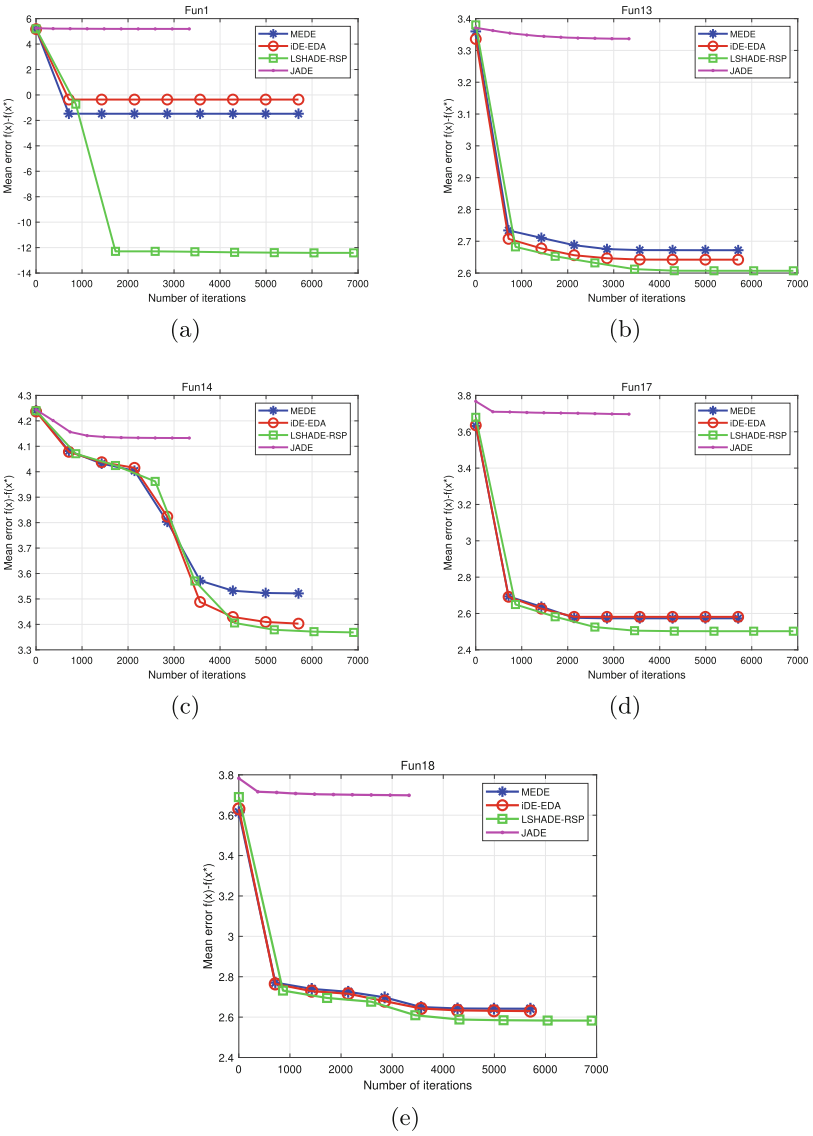


Fig. 3. MEDE is slightly inferior to part of Comparison algorithms

- AEDA: an advanced variant of the Evolutionary Development Algorithm.
- ACO_{mv}: optimized ACO algorithm that improves the pheromone strategy.
- S-PSO: advanced PSO variant with dynamic parameter adjustment.
- DE_{mv}: adaptive DE algorithm with optimization of multiple strategies.

The results of the comparison are shown in Table 3. As can be seen from the table, the MEDE algorithm is significantly better than the AEDA, ACO_{mv},

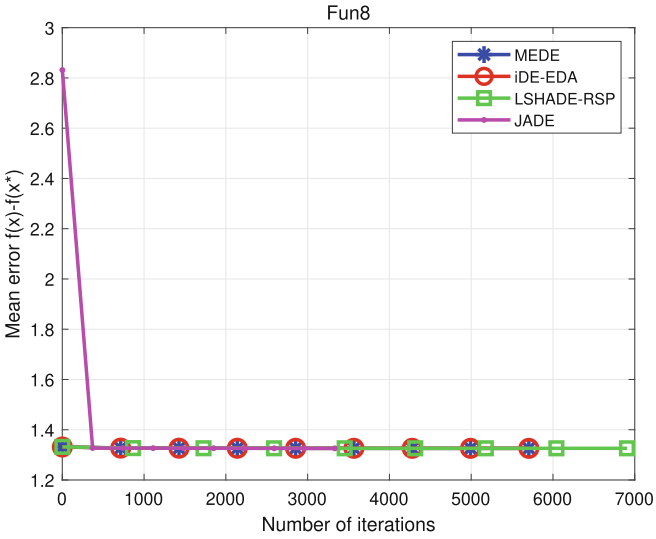


Fig. 4. MEDE reaches the same level with Comparison algorithms

Table 2. Detailed summary of MEDE and other algorithms

	MEDE		
	+	≈	−
JADE	27	1	0
LSHADE-RSP	8	17	3
iDE-EDA	8	19	1
Total	43	37	4

S-PSO and DE_{mv} algorithms. Compared with the AEDA algorithm, the MEDE algorithm shows significant advantages in 18 test functions, and the performance of the two algorithms is comparable in 1 function, and the MEDE algorithm is slightly inferior to the AEDA algorithm in the remaining 9 functions. When compared with the ACO_{mv} algorithm, the MEDE algorithm achieves better performance indicators on 26 functions, and only one function is slightly better than the ACO_{mv} algorithm, and it is similar in one function. From the comparison with the S-PSO algorithm, the MEDE algorithm is better in 22 functions and slightly inferior in 5 functions. The MEDE algorithm is significantly better than the DE_{mv} algorithm in 26 functions and similar in 2 functions. A more detailed performance comparison result is shown in Table 4.

To further compare the algorithm performance, we do the Friedman experiment, and the relevant data are shown in Table 5. The results show the performance ranking of the five algorithms in detail, with MEDE ranking first.

Table 3. The data results of performance comparison

Function	Mean/Std	AEDA	ACOmv	S-PSO	DEmv	MEDE
f1	Mean	3.89E-02(+)	1.26E+02(+)	8.36E+02(+)	4.31E+03(+)	3.33E-02
	Std	9.83E-01	2.13E+03	1.01E+03	3.62E+03	1.83E-01
f2	Mean	8.60E+07(+)	2.79E+08(+)	9.31E+07(+)	1.34E+08(+)	4.45E+06
	Std	2.99E+08	6.75E+08	2.50E+08	7.34E+08	2.17E+06
f3	Mean	6.33E+08(-)	3.75E+10(+)	2.86E+10(+)	7.92E+10(+)	1.88E+09
	Std	2.36E+09	1.07E+11	4.99E+10	9.18E+10	1.48E+09
f4	Mean	4.80E+04(+)	6.80E+04(+)	5.15E+04(+)	9.18E+04(+)	2.73E+03
	Std	3.08E+04	3.05E+04	5.30E+04	4.17E+04	1.30E+03
f5	Mean	3.69E-01(-)	9.78E+00(+)	1.05E+02(+)	7.43E+02(+)	2.48E+00
	Std	2.33E+00	7.66E+01	1.34E+02	4.51E+02	1.38E+00
f6	Mean	2.10E+02(+)	7.44E+02(+)	2.87E+02(+)	6.73E+02(+)	8.37E+01
	Std	2.12E+02	8.00E+02	2.74E+02	1.59E+02	4.13E+01
f7	Mean	2.55E+01(-)	1.36E+02(+)	1.36E+02(+)	1.88E+02(+)	8.17E+01
	Std	3.21E+01	1.63E+02	1.35E+02	1.18E+02	1.47E+01
f8	Mean	2.12E+01(≈)	2.12E+01(≈)	2.12E+01(≈)	2.12E+01(≈)	2.12E+01
	Std	1.71E-01	2.05E-01	2.07E-01	2.47E-01	3.41E-02
f9	Mean	6.40E+01(+)	6.93E+01(+)	6.60E+01(+)	6.58E+01(+)	4.62E+01
	Std	1.11E+02	3.21E+01	3.54E+01	9.55E+00	8.46E+00
f10	Mean	2.77E+01(+)	4.04E+02(+)	5.09E+02(+)	1.46E+02(+)	2.11E+01
	Std	7.31E+01	2.01E+03	6.06E+02	8.86E+02	1.18E+01
f11	Mean	1.57E+02(-)	4.09E+02(+)	1.90E+02(-)	3.82E+02(+)	2.57E+02
	Std	1.11E+02	4.40E+03	1.16E+02	1.15E+02	5.69E+01
f12	Mean	3.70E+02(+)	5.31E+02(+)	4.79E+02(+)	6.23E+02(+)	3.11E+02
	Std	1.01E+02	2.54E+02	2.43E+02	1.49E+02	6.97E+01
f13	Mean	3.71E+02(-)	5.26E+02(+)	5.10E+02(+)	6.34E+02(+)	4.70E+02
	Std	7.72E+01	2.56E+01	1.92E+02	1.61E+02	8.34E+01
f14	Mean	6.47E+06(+)	1.35E+04(+)	2.45E+03(-)	6.24E+03(+)	3.32E+03
	Std	1.68E+03	1.58E+03	4.56E+03	1.44E+03	2.72E+03
f15	Mean	1.36E+04(+)	1.37E+04(+)	1.37E+04(+)	1.31E+04(+)	8.11E+03
	Std	2.04E+03	2.28E+03	2.52E+03	1.94E+03	7.34E+02
f16	Mean	7.37E+00(+)	3.92E+00(+)	3.80E+00(+)	3.75E+00(+)	2.92E+00
	Std	1.27E+00	1.65E+00	1.52E+00	2.08E+00	6.90E-01
f17	Mean	1.69E+02(-)	4.63E+02(+)	3.96E+02(+)	6.28E+02(+)	3.74E+02
	Std	7.76E+01	7.83E+02	3.60E+02	2.23E+02	6.47E+01
f18	Mean	1.76E+02(-)	4.51E+02(+)	3.58E+02(-)	6.40E+02(+)	4.38E+02
	Std	3.62E+02	7.99E+02	2.84E+02	1.91E+02	6.98E+01
f19	Mean	1.62E+01(-)	4.75E+03(+)	5.80E+01(+)	3.71E+02(≈)	3.71E+01
	Std	1.15E+01	2.63E+04	5.28E+01	6.69E+02	1.16E+01
f20	Mean	2.16E+01(+)	2.16E+01(+)	2.16E+01(+)	2.18E+01(+)	2.14E+01
	Std	1.32E+00	1.65E+00	2.06E+00	1.22E+00	1.77E+00
f21	Mean	2.00E+02(-)	2.28E+02(-)	2.66E+02(-)	4.78E+02(+)	4.02E+02
	Std	1.71E-01	4.03E+02	8.69E+01	1.84E+02	1.75E+00
f22	Mean	6.16E+03(+)	1.36E+04(+)	2.22E+03(-)	6.39E+03(+)	2.63E+03
	Std	1.68E+03	2.27E+03	3.08E+03	1.43E+03	2.48E+03
f23	Mean	1.64E+04(+)	1.64E+04(+)	1.64E+04(+)	1.62E+04(+)	8.76E+03
	Std	9.06E+02	1.16E+03	1.84E+03	1.50E+03	8.90E+02
f24	Mean	1.43E+03(+)	1.68E+03(+)	1.65E+03(+)	1.74E+03(+)	3.23E+02
	Std	2.96E+02	3.01E+02	3.52E+02	2.75E+02	1.37E+01
f25	Mean	5.36E+02(+)	5.81E+02(+)	5.60E+02(+)	6.14E+02(+)	3.52E+02
	Std	6.83E+01	6.25E+01	8.88E+01	5.44E+01	1.13E+01
f26	Mean	7.70E+02(+)	9.54E+02(+)	1.51E+03(+)	3.05E+03(+)	4.01E+02
	Std	8.39E+02	6.87E+03	1.97E+03	2.33E+03	4.17E+01
f27	Mean	5.08E+03(+)	5.20E+03(+)	5.27E+03(+)	5.54E+03(+)	1.47E+03
	Std	5.83E+02	1.36E+03	8.07E+03	2.19E+02	8.78E+01
f28	Mean	1.15E+04(+)	1.13E+04(+)	9.78E+03(+)	1.12E+04(+)	3.73E+03
	Std	1.18E+03	1.38E+03	2.05E+03	1.45E+03	7.03E+02
+/ ≈ /-		18/1/9	26/1/1	22/1/5	26/2/0	

Through the above performance analysis, it can be seen that the MEDE algorithm far exceeds other representative algorithms in performance. In the experiment, our algorithm MEDE is significantly better than the comparison algorithms in terms of convergence speed and optimal solution quality, and its performance advantages are highly significant after Friedman testing.

Table 4. Detailed performance comparison results

	MEDE		
	+	≈	−
AEDA	18	1	9
ACO_{mv}	26	1	1
S-PSO	22	1	5
DE_{mv}	26	2	0
Total	92	5	15

Table 5. Friedman rank test of performance comparison

Algorithm	MEDE	AEDA	S-PSO	ACOmv	DEmv
Rank	1.61	2.07	3.14	4.00	4.18

5 Conclusion

This paper introduces a novel variant of DE, termed MEDE, specifically designed to deal with MVOPs. MEDE uses dynamic opposition-based learning strategy to enhance population diversity. Furthermore, model archive strategy is introduced to effectively mitigate the issue of diminishing data diversity in the later stages of algorithm evolution by storing probability distribution models generated during the evolutionary process. Additionally, stacking ensemble learning method is incorporated into the algorithm to effectively address the overfitting problem associated with probability distribution models.

In future work, there are two possible directions that can be addressed. Firstly, the efficiency of model archive utilization is further investigated. Secondly, the MEDE framework incorporates stacking ensemble learning, while further investigation into alternative ensemble methods remains promising.

Acknowledgments. This work was supported by the Science and Technology Projects of Jiangxi Provincial Education Department(grant number GJJ2201915), and Natural Science Foundation of Jiangxi Province (grant number 20232BAB202053).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Peng, H., Han, Y., Deng, C., Wang, J., Wu, Z.: Multi-strategy co-evolutionary differential evolution for mixed-variable optimization. Knowl. Based Syst. **229**, 107366 (2021)
2. Tsai, J., Lin, M., Wen, D.: Global optimization for mixed-discrete structural design. Symmetry **12**(9), 1529 (2020)

3. Hwang, J.T., Jasa, J.P., Martins, J.R.R.A.: High-fidelity design-allocation optimization of a commercial aircraft maximizing airline profit. J. Sirci. **56**(1), 8–28 (2019)
4. Zitouni, F., Harous, S., Maamri, R.: The solar system algorithm: a novel metaheuristic method for global optimization. IEEE Access **9**, 4542–4565 (2021)
5. Storn, R., Price, K.: Differential evolution - a simple and efficient heuristic for global optimization over continuous space. J. Global Optimiz. **11**, 341–359 (1997)
6. Zhang, J., Sanderson, A.C.: JADE: adaptive differential evolution with optional external archive. IEEE Trans. Evol. Comput. **13**, 945–958 (2009)
7. Stanovov, V., Akhmedova, S., Semenkin, E.: LSHADE algorithm with rank-based selective pressure strategy for solving CEC 2017 benchmark problems. In: 2018 IEEE Congress on Evolutionary Computation, CEC 2018 - Proceedings, pp. 1–8. IEEE (2018)
8. Li, Y., Han, T., Tang, S., Huang, C., Zhou, H., Wang, Y.: An improved differential evolution by hybridizing with estimation-of-distribution algorithm. Inf. Sci. **619**, 439–456 (2023)
9. Li, Y., et al.: Apsm-jso: a novel jso variant with an adaptive parameter selection mechanism and a new external archive updating mechanism. Swarm Evol. Comput. **78**, 101283 (2023)
10. Wang, F., Zhang, H., Zhou, A.: A particle swarm optimization algorithm for mixed-variable optimization problems. Swarm Evol. Comput. **60**, 100808 (2021)
11. Shi, W., Chen, W., Lin, Y., Gu, T., Kwong, S., Zhang, J.: An adaptive estimation of distribution algorithm for multipolicy insurance investment planning. IEEE Trans. Evol. Comput. **23**(1), 1–14 (2017)
12. Liao, T., Socha, K., de Oca, M.A.M., Stützle, T., Dorigo, M.: Ant colony optimization foe mixed-variable optimization problems. IEEE Trans. Evol. Comput. **18**(4), 503–518 (2013)
13. Chen, W.N., Zhang, J., Chung, H., Zhong, W., Wu, W.-L., hui Shi, Y.-G.: A novel set-based particle swarm optimization method for discrete optimization problems. IEEE Trans. Evol. Comput. **14**(2), 278–300 (2014)
14. Lin, Y., Liu, Y., Chen, W., Zhang, J.: A hybrid differential evolution algorithm for mixed-variable optimization problems. Inform. Sci. **466**, 170–188 (2018)

Author Index

A
Alapati, Praveen 59
An, Ning 462
Attinagaramu, Kapil Kumar 59

B
Bai, Longfei 79

C
Cao, Buqing 119
Cao, Jian 367
Cao, Nieqing 254
Cao, Yuhan 440
Chen, Chao 254
Chen, Jianguo 119
Chen, Juan 440
Chen, Qing 571
Cheng, Sihao 151

D
Deng, Changshou 571
Deng, Shanlin 497
Ding, Yan 254
Dong, Peijun 215
Dong, Pingping 137
Dong, Xiaogang 571
Dong, Yusong 236
Du, Haohua 478
Du, Lili 119
Du, Yan 462

F
Feng, Chengquan 478
Feng, Kexuan 330

G
Gan, Zhengyu 478
Gao, Baofeng 3
Gao, Yinghui 172
Gong, Chunye 172
Gu, Fuqiang 254
Gu, Yunfei 151
Guo, Ying 3

H
He, Jiezhong 172
Hu, Guangqin 79
Hu, Zhen 101
Hua, Baojian 497
Huang, Kai 101
Huang, Yifan 462

J
Jia, Menghan 172
Jiang, Gongqingjian 172
Jiang, Nan 25
Jiang, Xiaowen 101

L
Li, Fan 330
Li, Guang 555
Li, Kenli 311
Li, Sihan 291
Li, Xiulai 537
Li, Zhewen 236
li, Zihao 555
Liang, Jianshan 387
Lin, Sheng 119
Lin, Yuxuan 349
Liu, Dongqin 349
Liu, Heng 195
Liu, Mingliang 497
Liu, Shi 349
Liu, Tao 3
Liu, Tianyang 330
Liu, Xiangyu 43
Liu, Xiaozhang 537
Liu, Yan 400
Liu, Yang 518
Luo, Hanjiang 215
Luo, Juan 330

© The Editor(s) (if applicable) and The Author(s), under exclusive license to Springer Nature Singapore Pte Ltd. 2026
H. Liu et al. (Eds.): ICA3PP 2025, LNCS 16383, pp. 591–592, 2026.
https://doi.org/10.1007/978-981-95-8405-5

M
Meng, Jie 151
Mi, Wei 349

P
Pan, Jingshan 3

Q
Qian, Shiyou 367

R
Ren, Zenghui 518

S
Shang, Mingsheng 555
Shao, Beichen 254
Shen, Yulong 400
Shi, Wei 215
Song, Meng 3
Su, Qian 79
Sun, Boya 271
Sun, Jiaqi 367
Sun, Ranran 400

T
Tan, Haisheng 478
Tang, Qiang 195
Tang, Wensheng 137
Tang, Xuehai 349
Tao, Hang 215
Teng, Yinglei 271

W
Wang, Baoqing 419
Wang, Jianhao 236
Wang, Meiling 43
Wang, Nan 271
Wang, Qiang 419
Wang, Xingchen 254
Wang, Yuchun 25
Wang, Zexin 518
Wang, Ziyi 462
Wei, Jiaqi 555
Wei, Xiaohui 25
Wen, Zhihua 119
Wu, Changjun 101
Wu, Chentao 151
Wu, Liyang 172
Wu, Qi 25
Wu, Xiaoming 3
Wu, Yahui 172
Wu, Yanqing 291
Wu, Yun 462
Wu, Zuohao 555

X
Xiong, Dongliang 101
Xu, Gang 291
Xu, Jiajing 537
Xu, Nan 462
Xu, XiaoDong 291
Xue, Guangtao 367

Y
Yan, Lan 311
Yang, Dingyu 367
Yang, Jieming 462
Yang, Kai 462
Yang, Kun 195
Yang, Lei 195
Yang, Weidong 400
Yi, Qingfen 137
Yu, Wenping 236
Yue, Hengshan 25

Z
Zeng, Guogen 330
Zhan, Haoming 440
Zhang, Jianzhong 236
Zhang, Lianming 137
Zhang, Lizhi 172
Zhang, Longxin 119
Zhang, Mingyuan 440
Zhang, Shixin 419
Zhang, Xiang 79
Zhang, Xinyin 518
Zhang, Xinzhuo 400
Zhang, Xuejie 79
Zhang, Yanfen 119
Zhang, Yijia 419
Zhang, Yiming 43
Zhao, Dawei 3
Zhao, Yang 215
Zhao, Yonghua 518
Zheng, Wenbo 311
Zhou, Xinting 537
Zuo, Yi 387